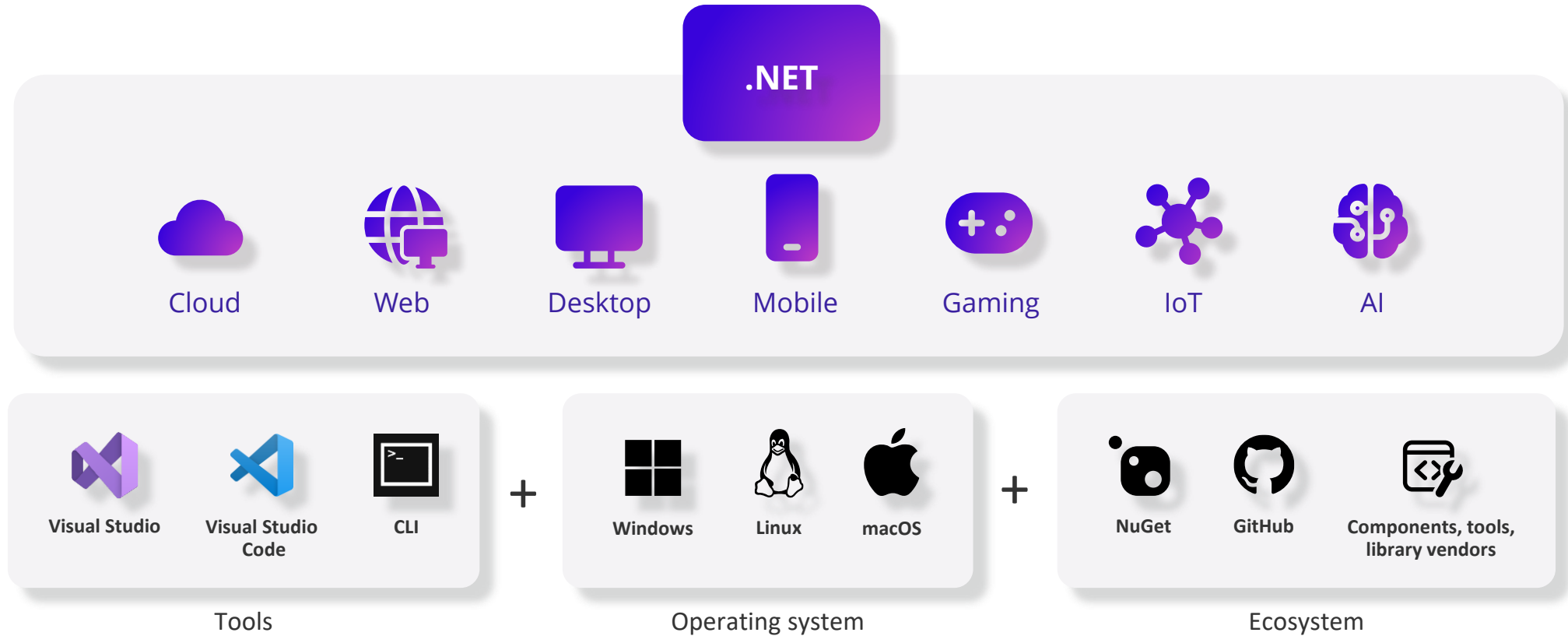


Build Anything, Anywhere with .NET 8



.NET Momentum

6.1+ Million

Monthly active users

53,000+

Community members
have contributed to .NET

#1 2023  stackoverflow

Most Admired Developer Framework

Top 5  CLOUD NATIVE
COMPUTING FOUNDATION

Highest velocity OSS project

Top 5 

Top Programming Language on GitHub

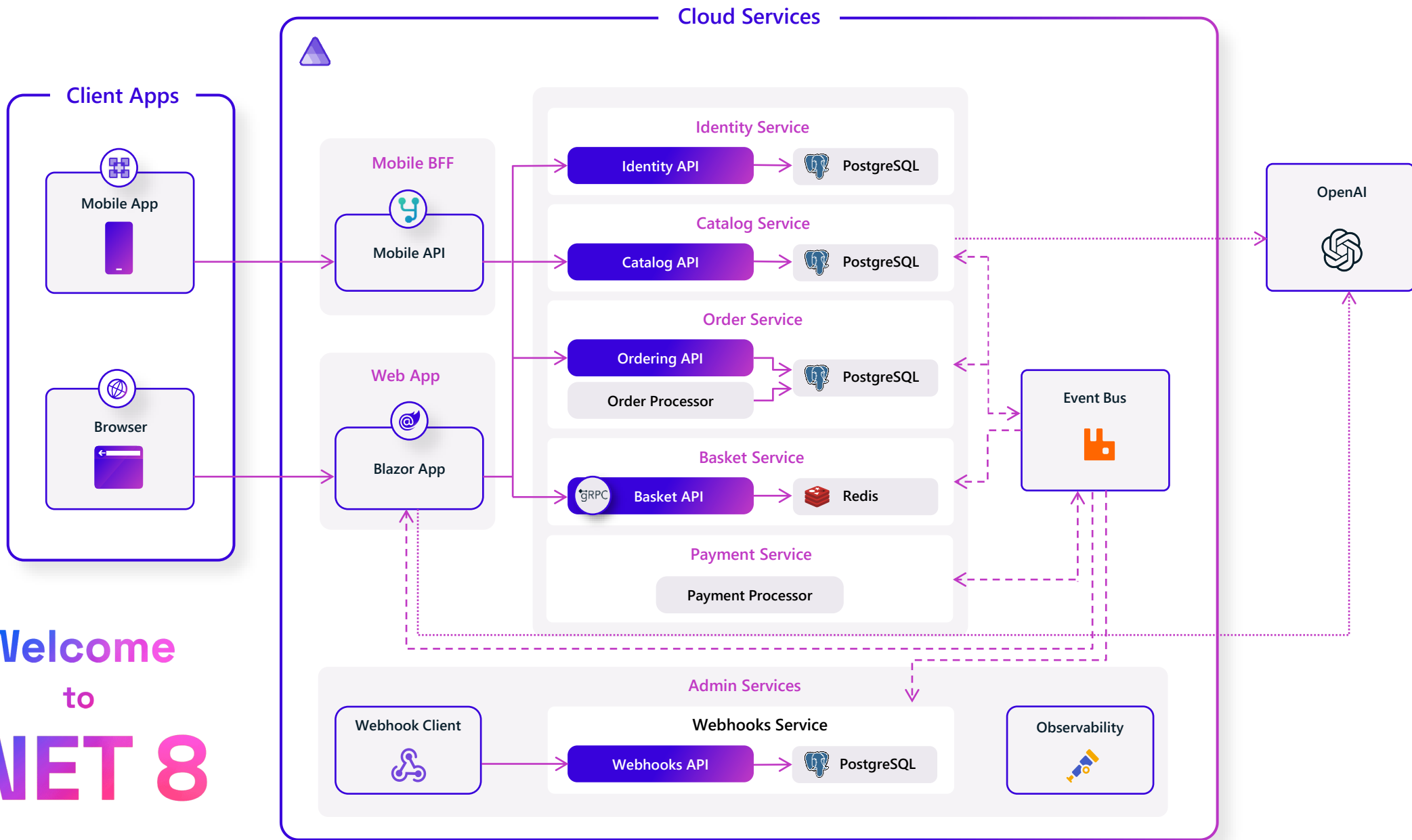
>21k

Contributions

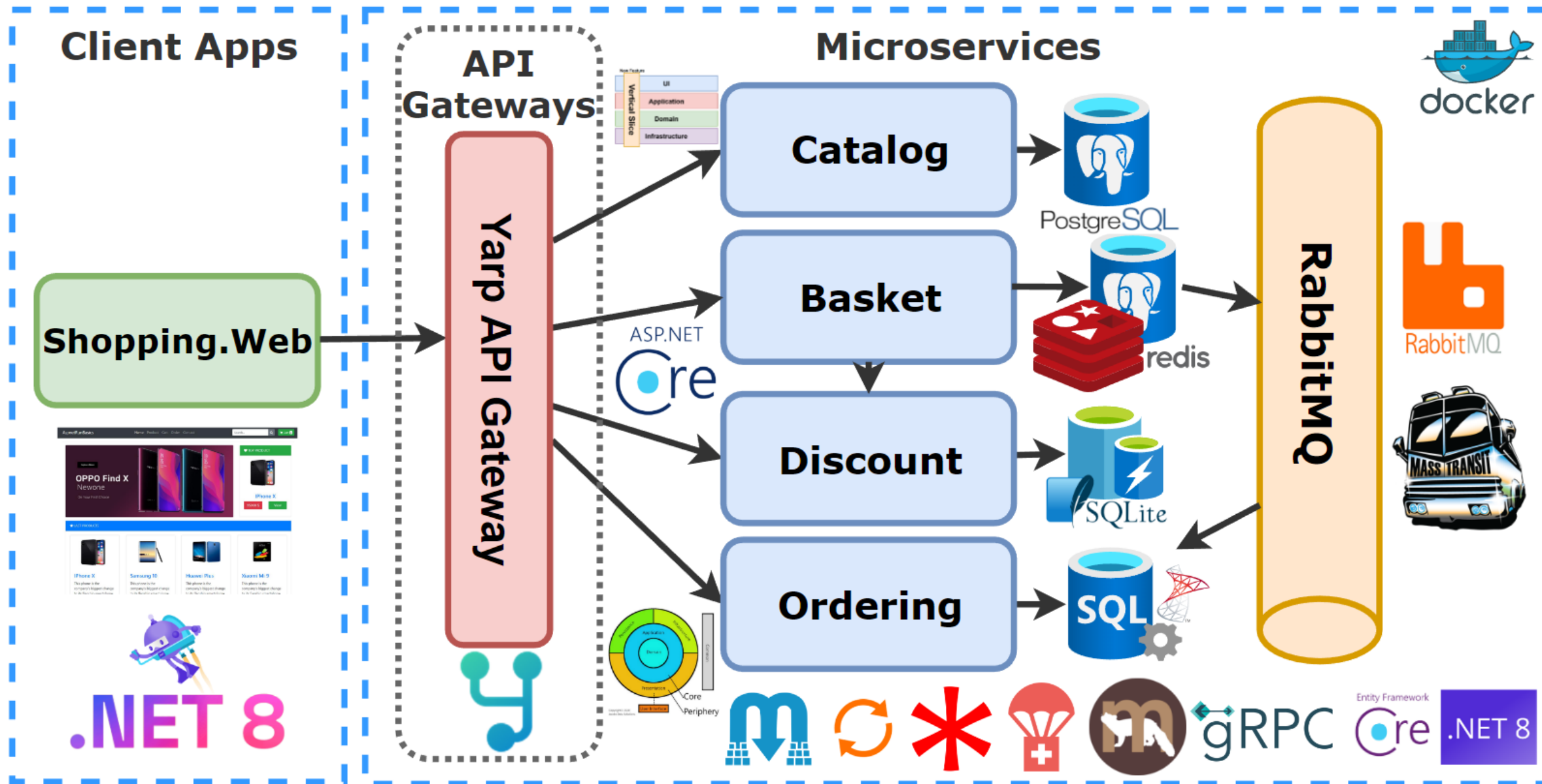
>9k

Community members

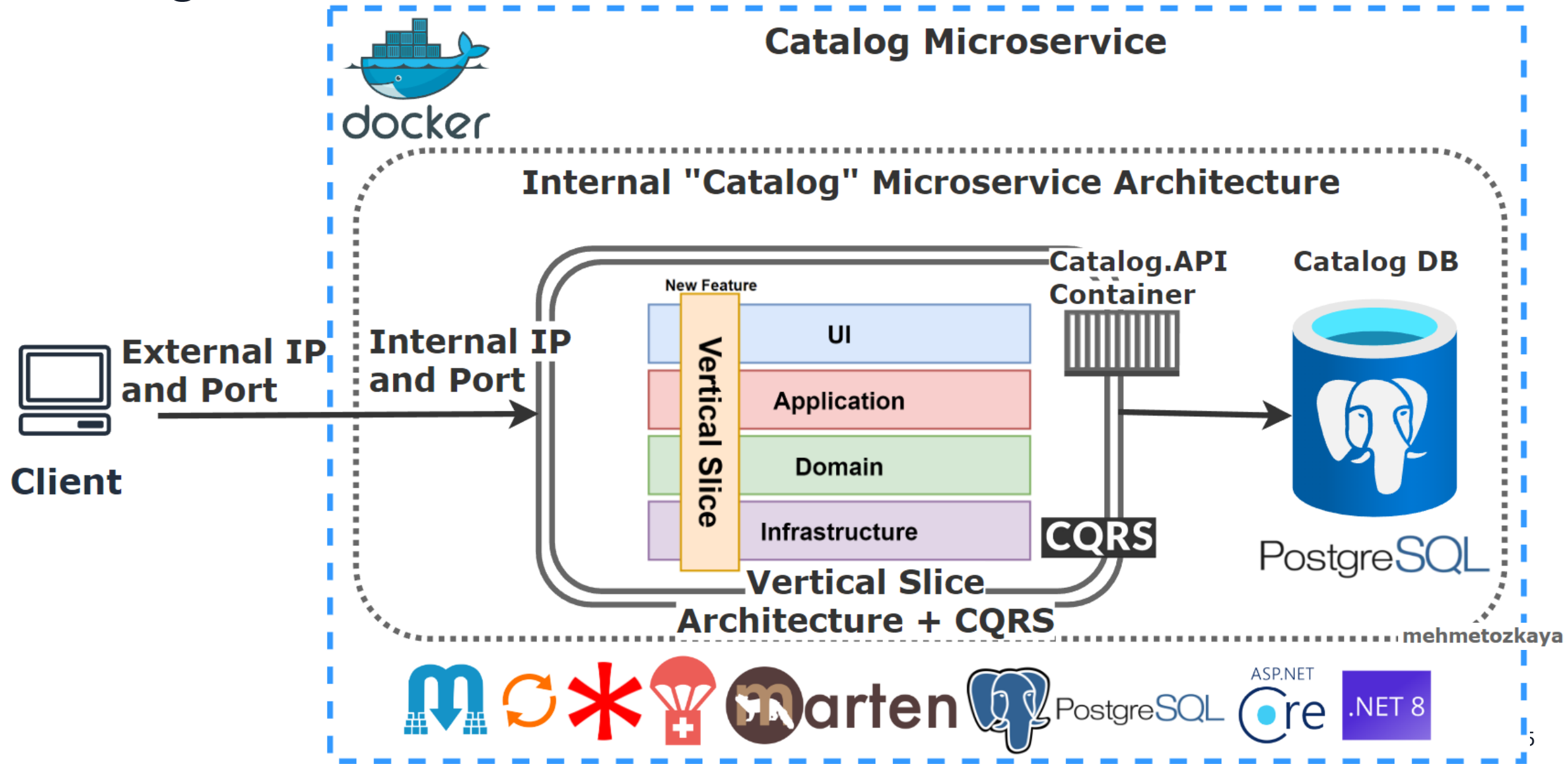
Welcome to .NET 8



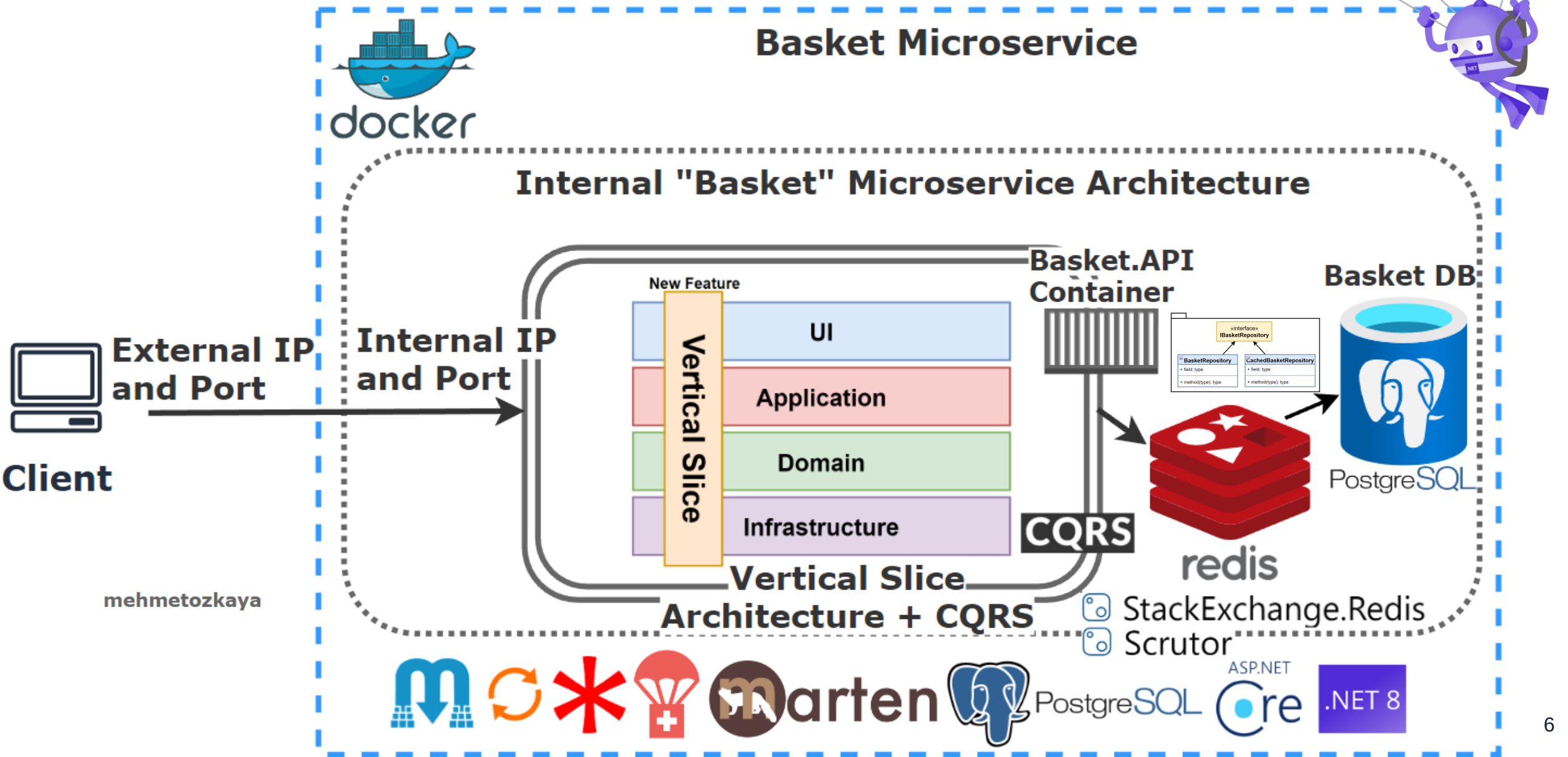
Big Picture – Course Project



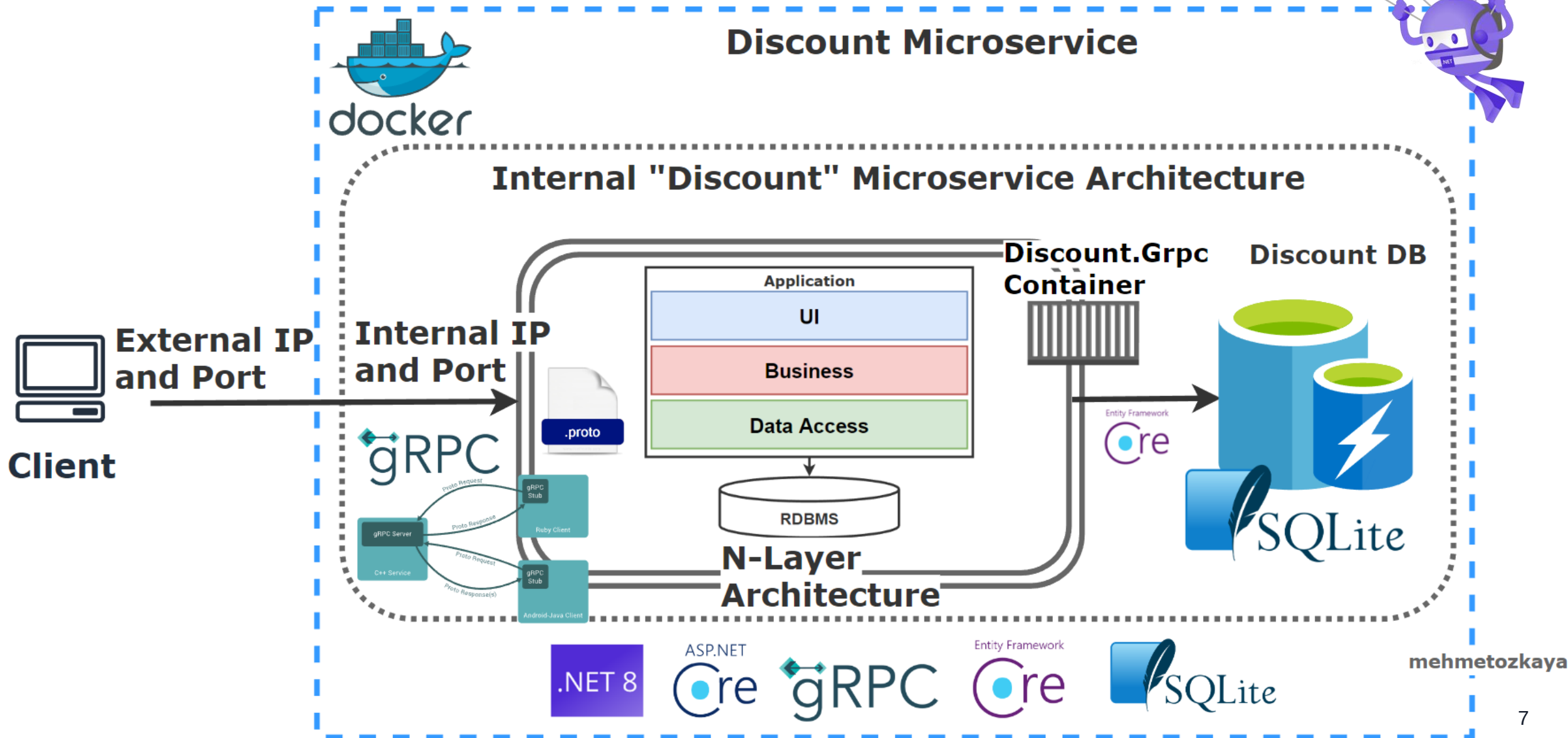
Catalog Microservice Internal Architecture



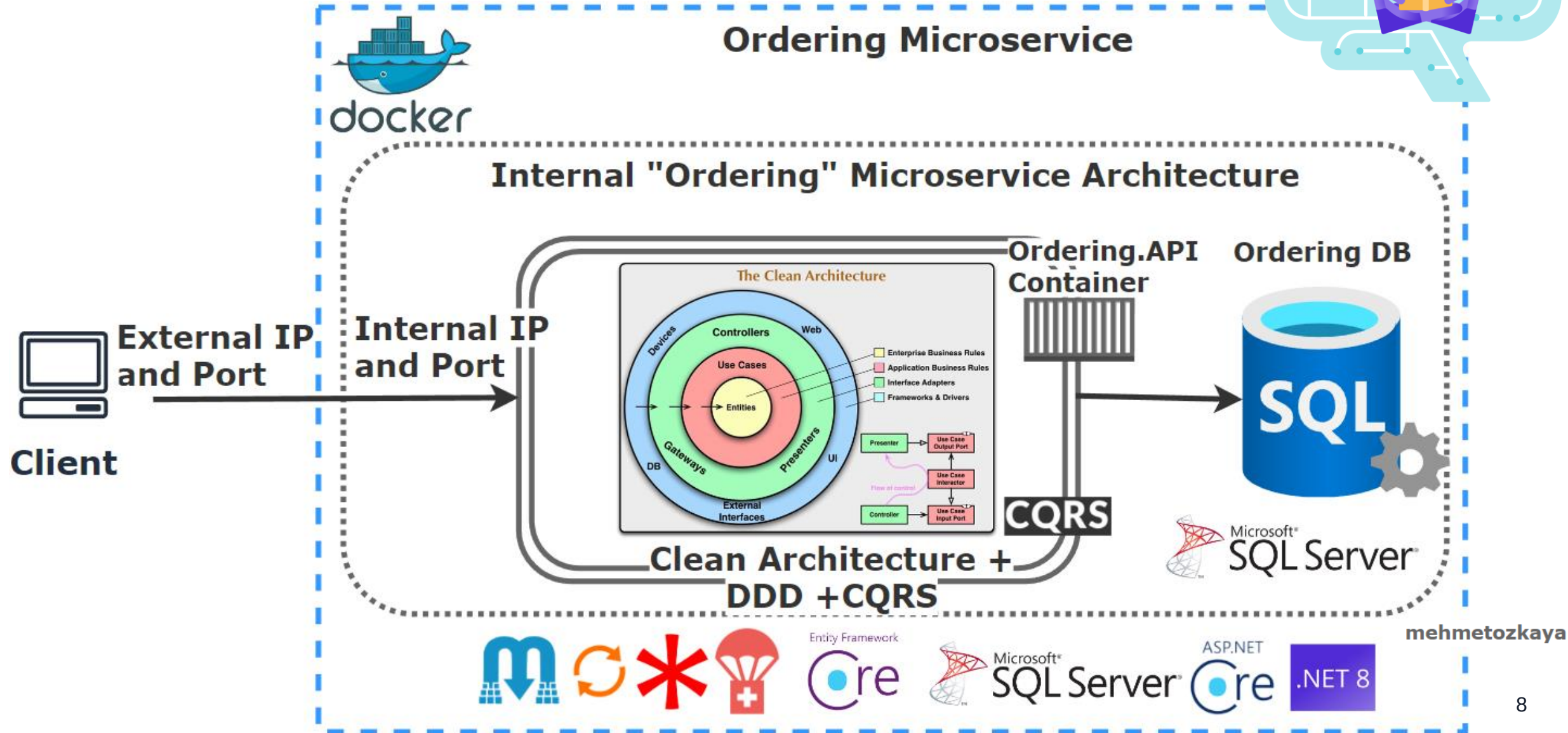
Developing Basket Microservices w/ Redis



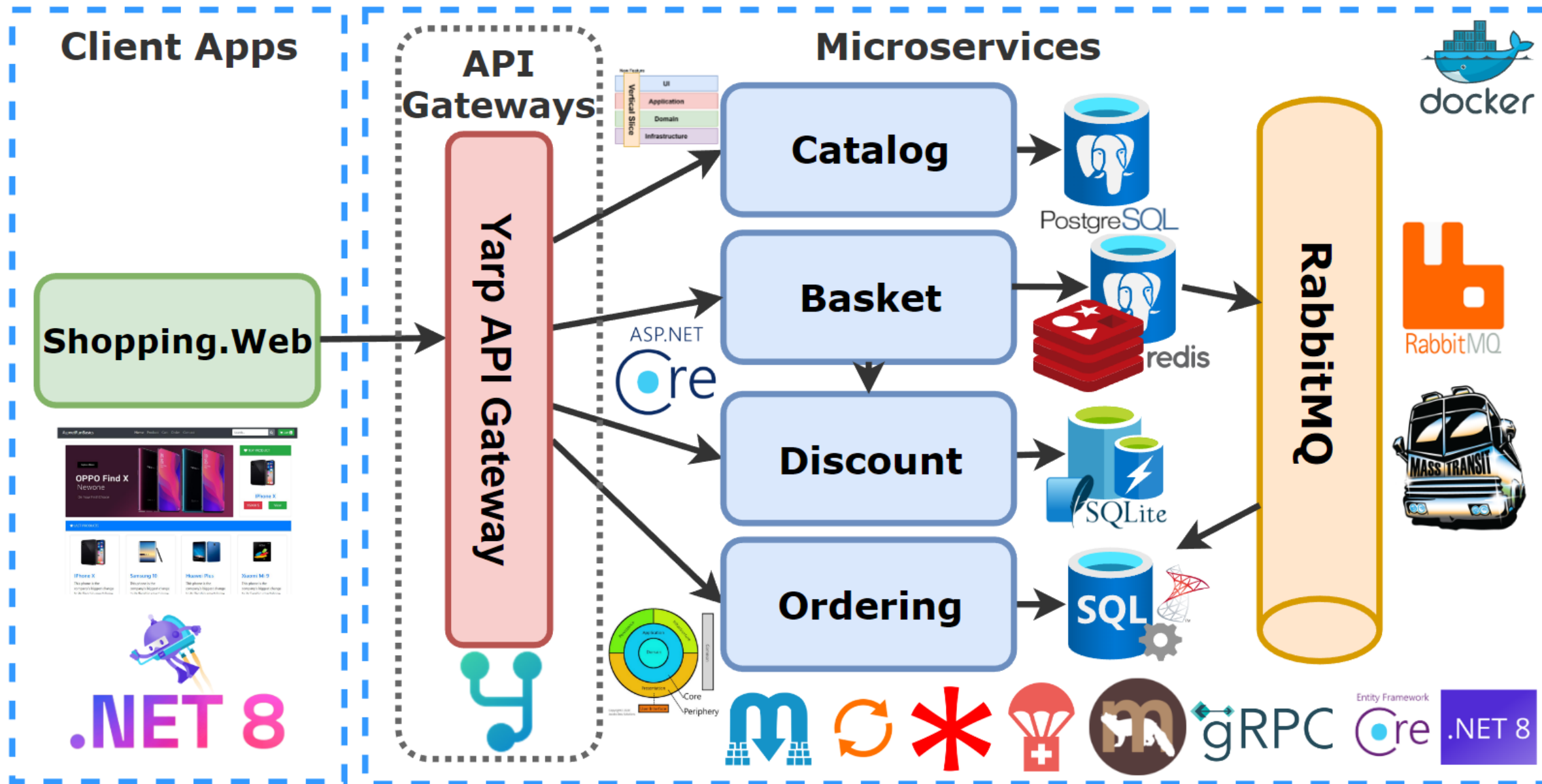
Developing Discount.Grpc Microservices



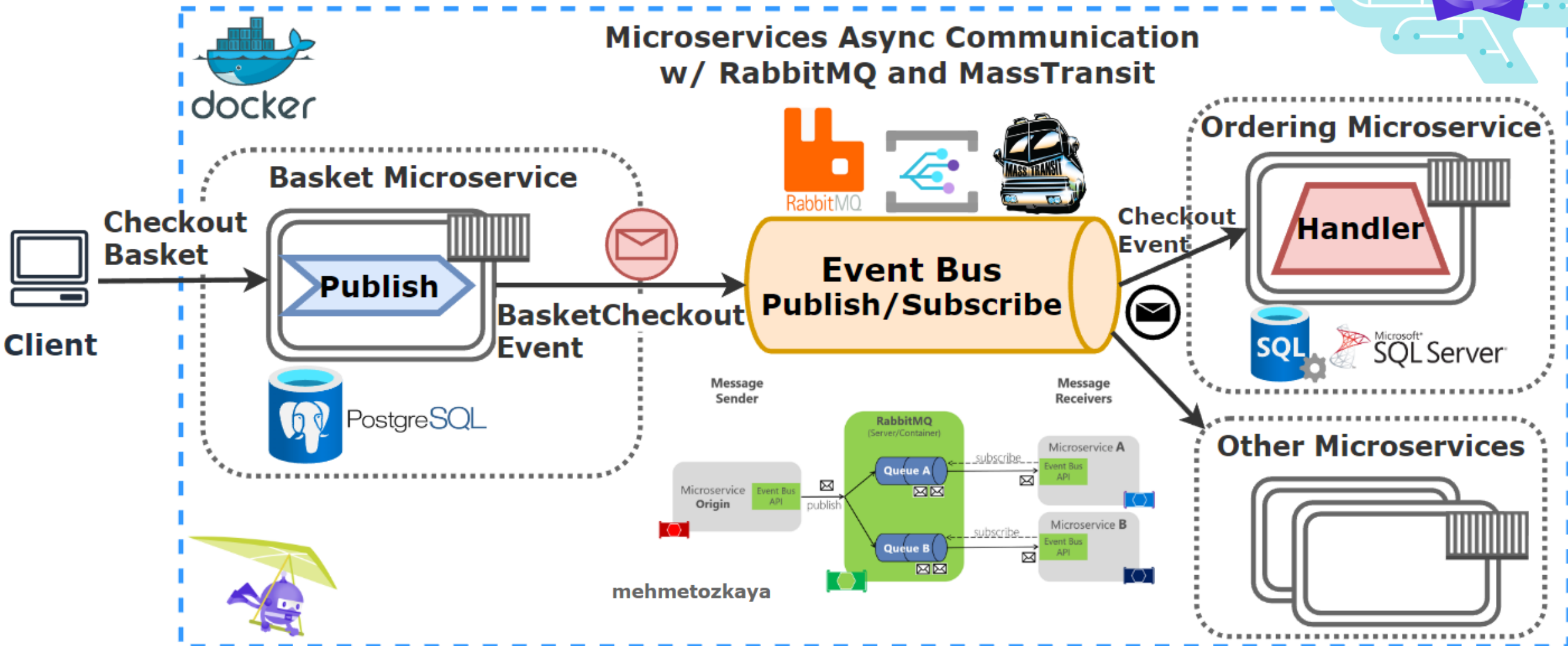
Ordering Microservice w/ DDD, CQRS and Clean Architecture



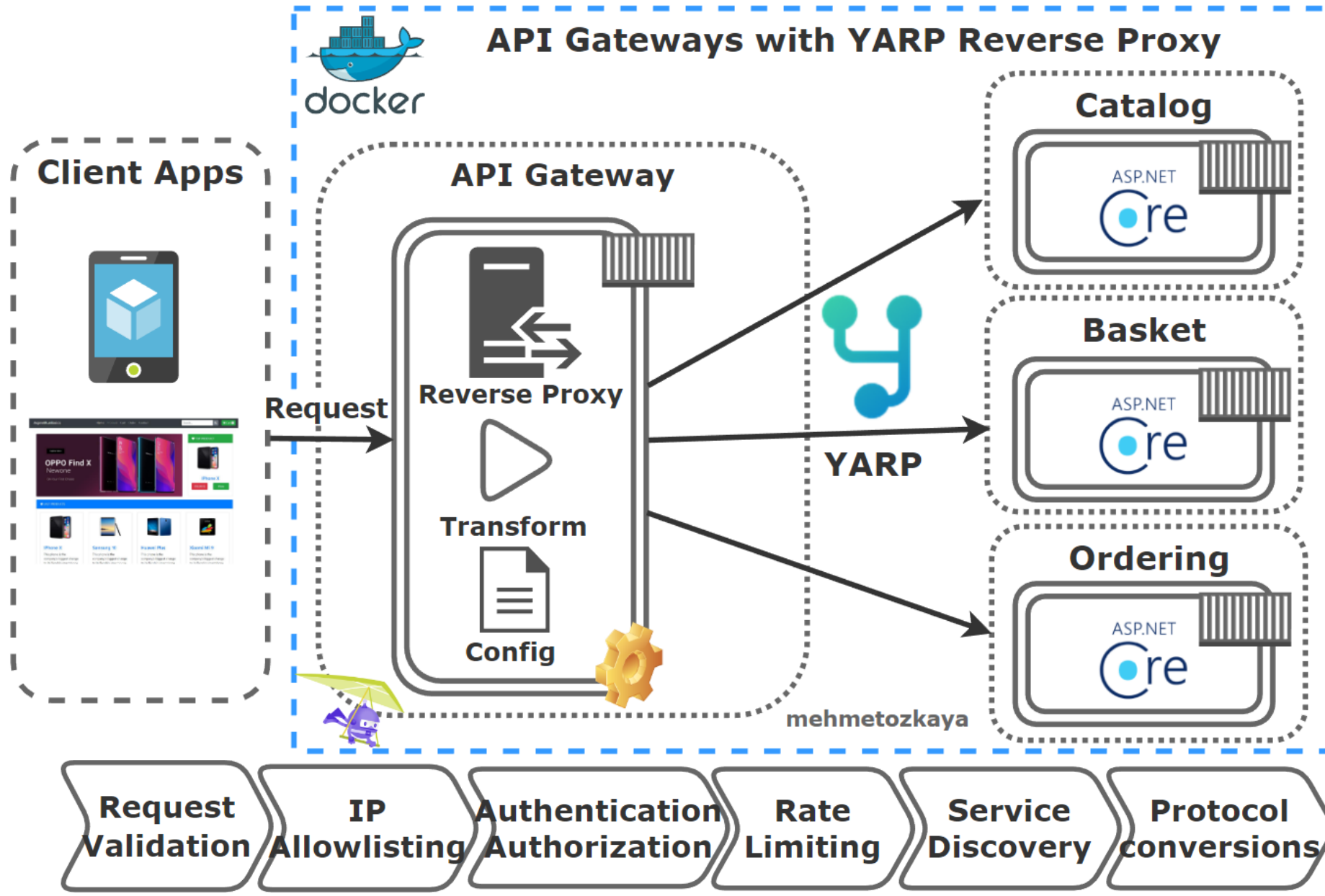
Big Picture – Course Project



Microservices Async Communication w/ RabbitMQ & MassTransit



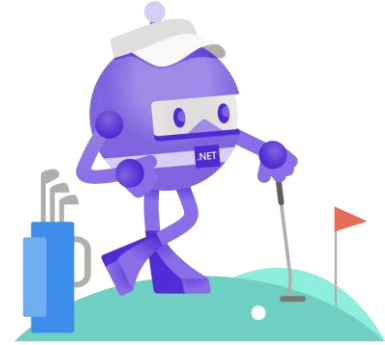
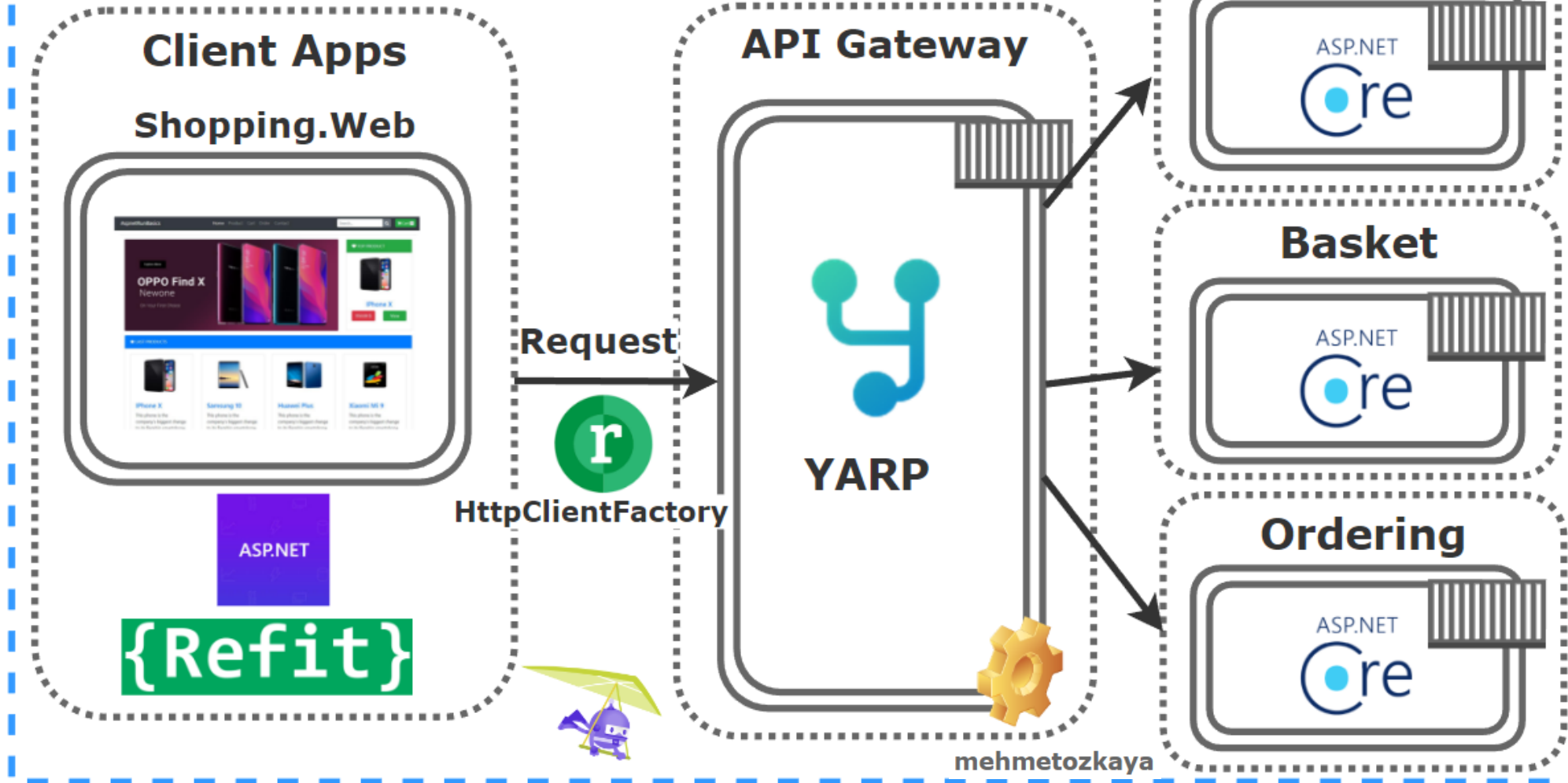
API Gateways with YARP Reverse Proxy



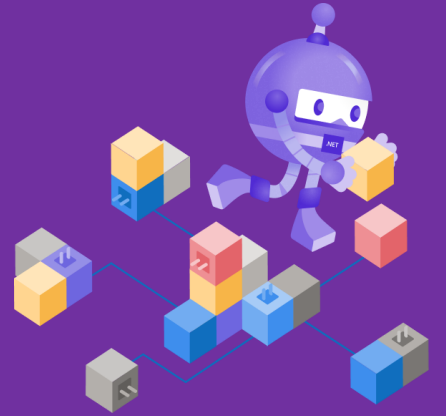
Shopping Web Client Application with Refit



Shopping.Web Client Application w/ Refit HttpClientFactory

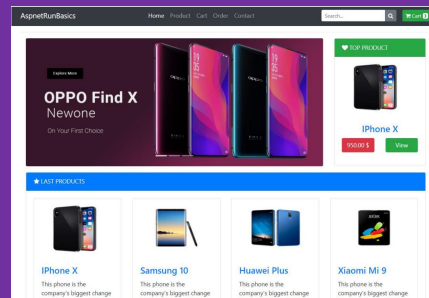


.Net 8 Microservices: DDD, CQRS, Vertical/Clean Architecture

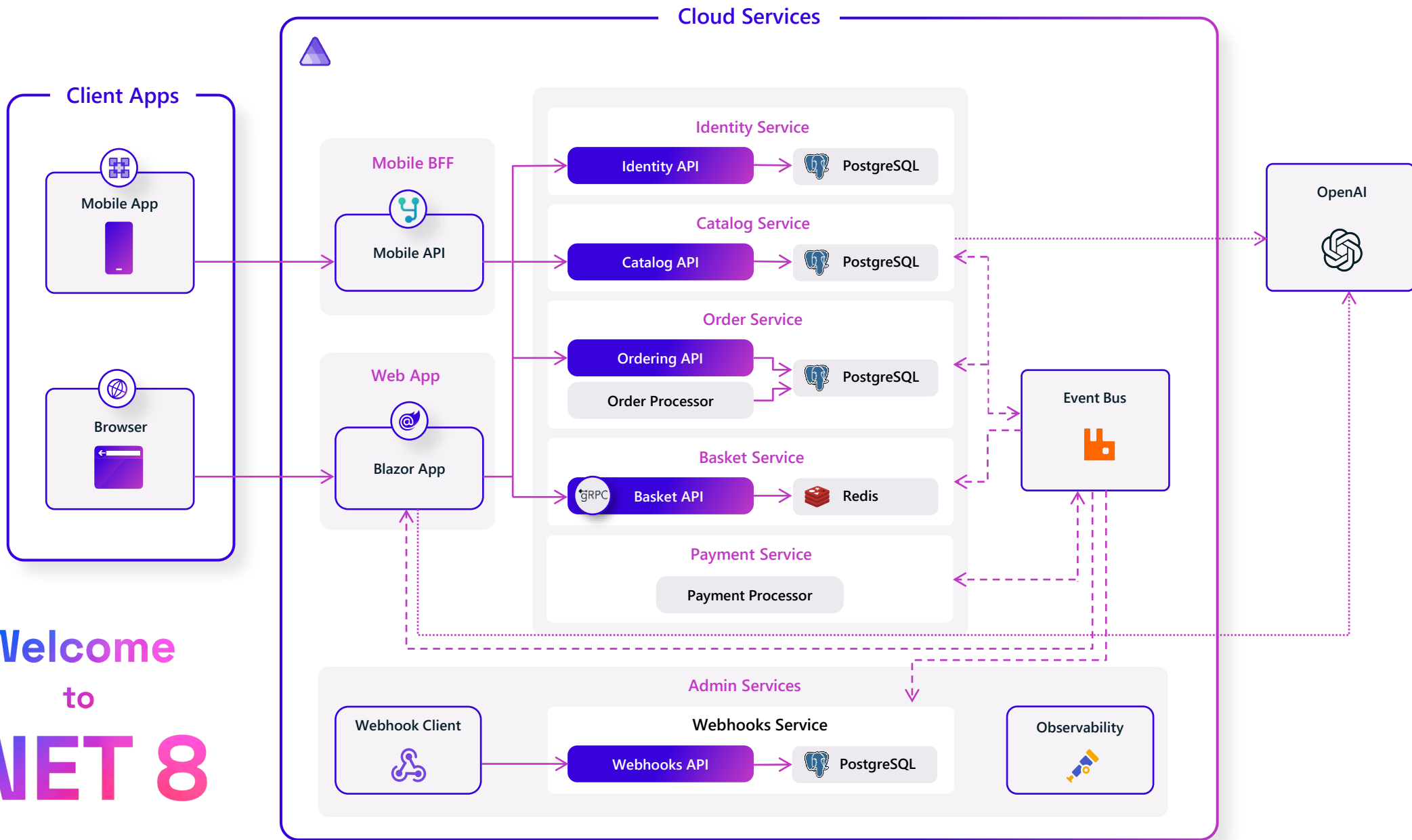


Microservices on .Net platforms which used Asp.Net Web API, Docker, RabbitMQ, MassTransit, Grpc, Yarp API Gateway, PostgreSQL, Redis, SQLite, SqlServer, Marten, Entity Framework Core, CQRS, MediatR, DDD, Vertical and Clean Architecture implementation.

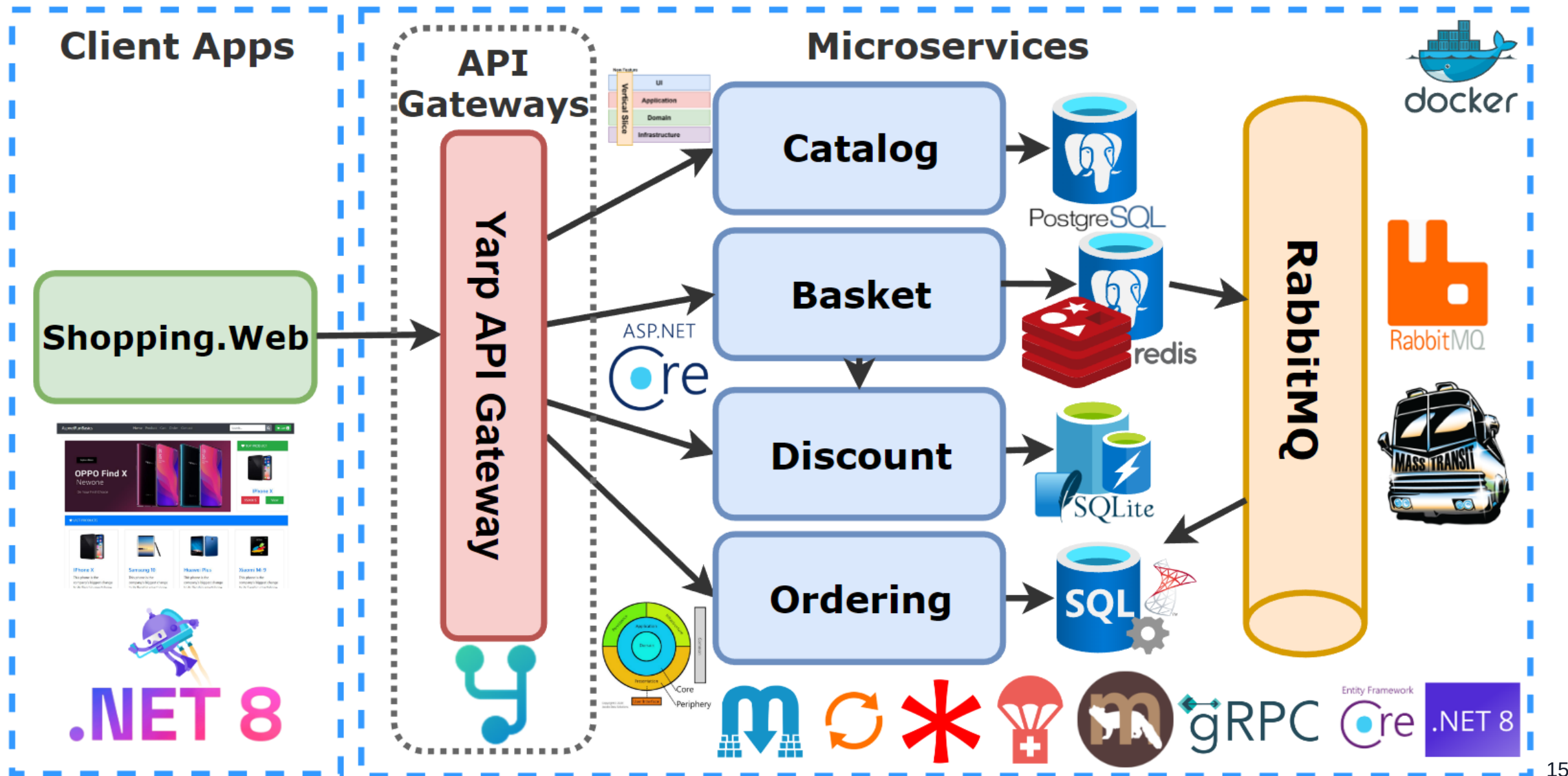
Mehmet Ozkaya



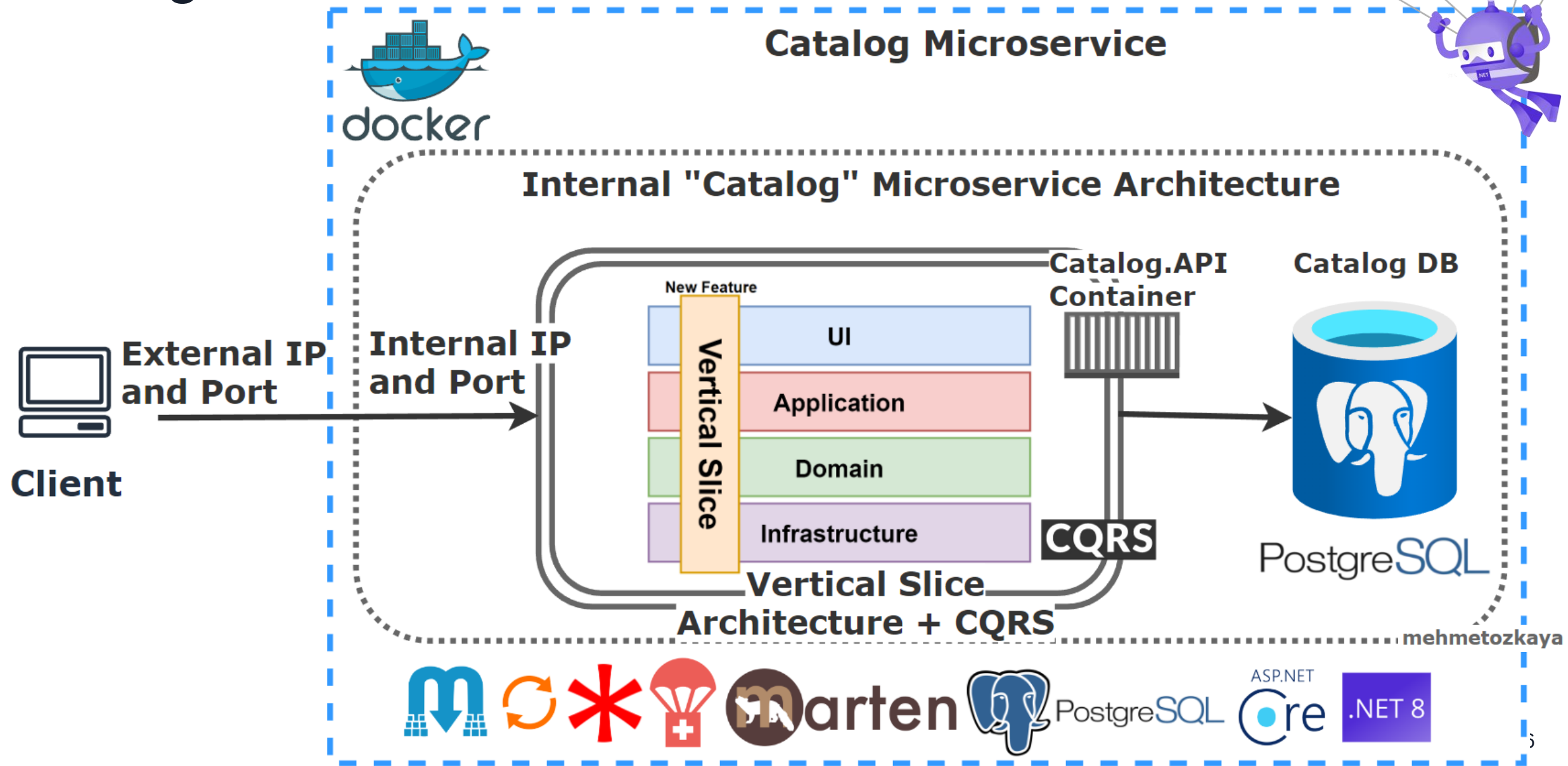
Welcome to .NET 8



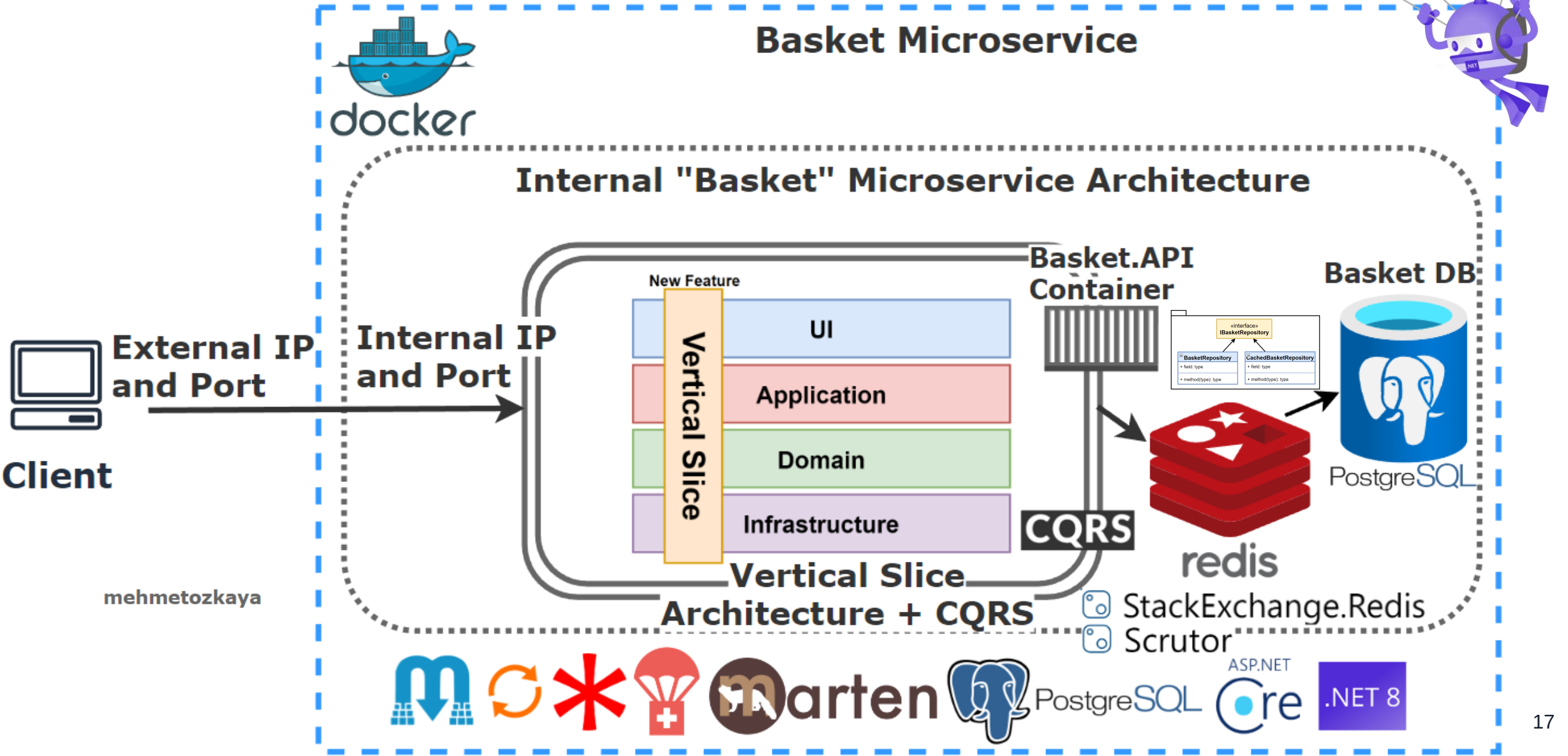
Big Picture – Course Project



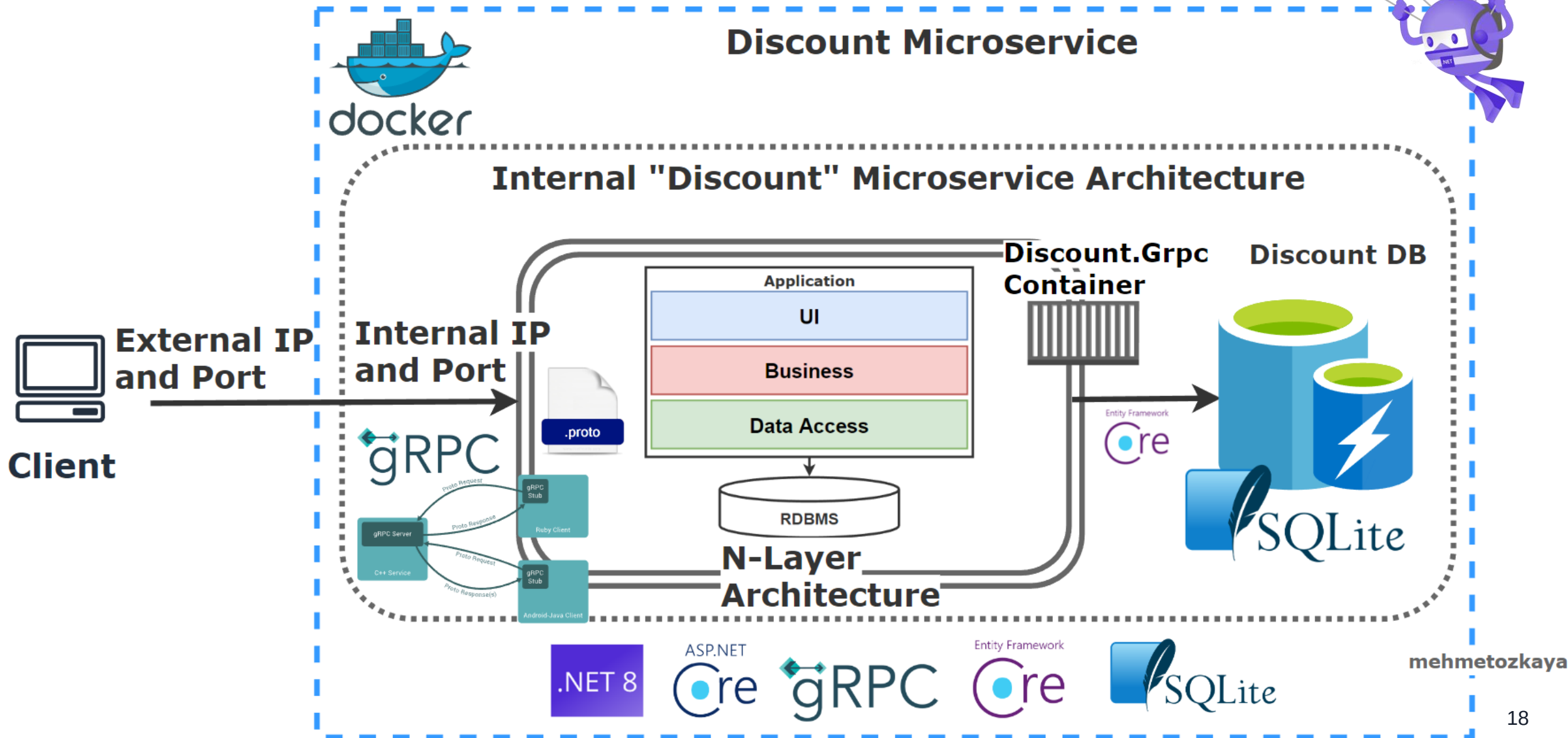
Catalog Microservice Internal Architecture



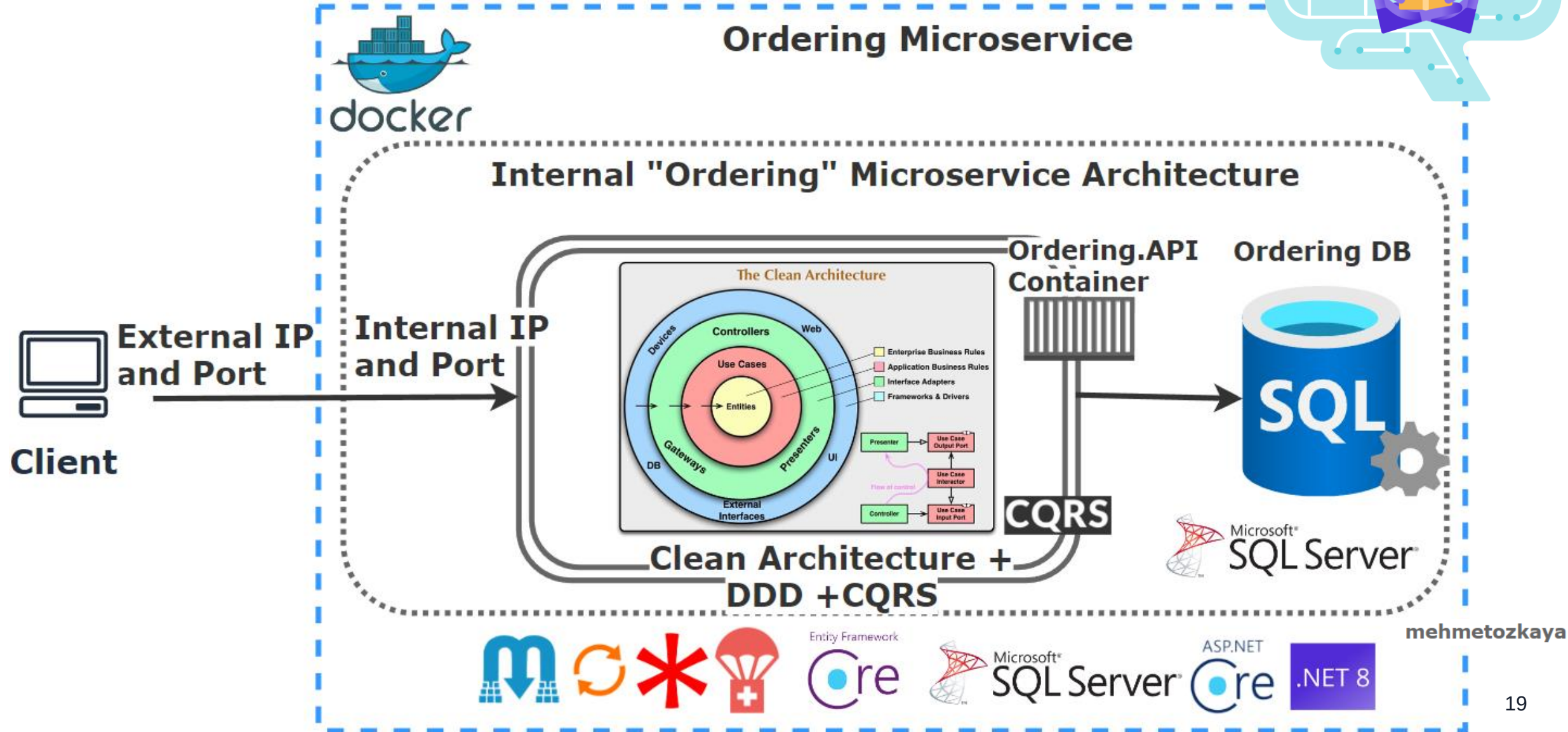
Developing Basket Microservices w/ Redis



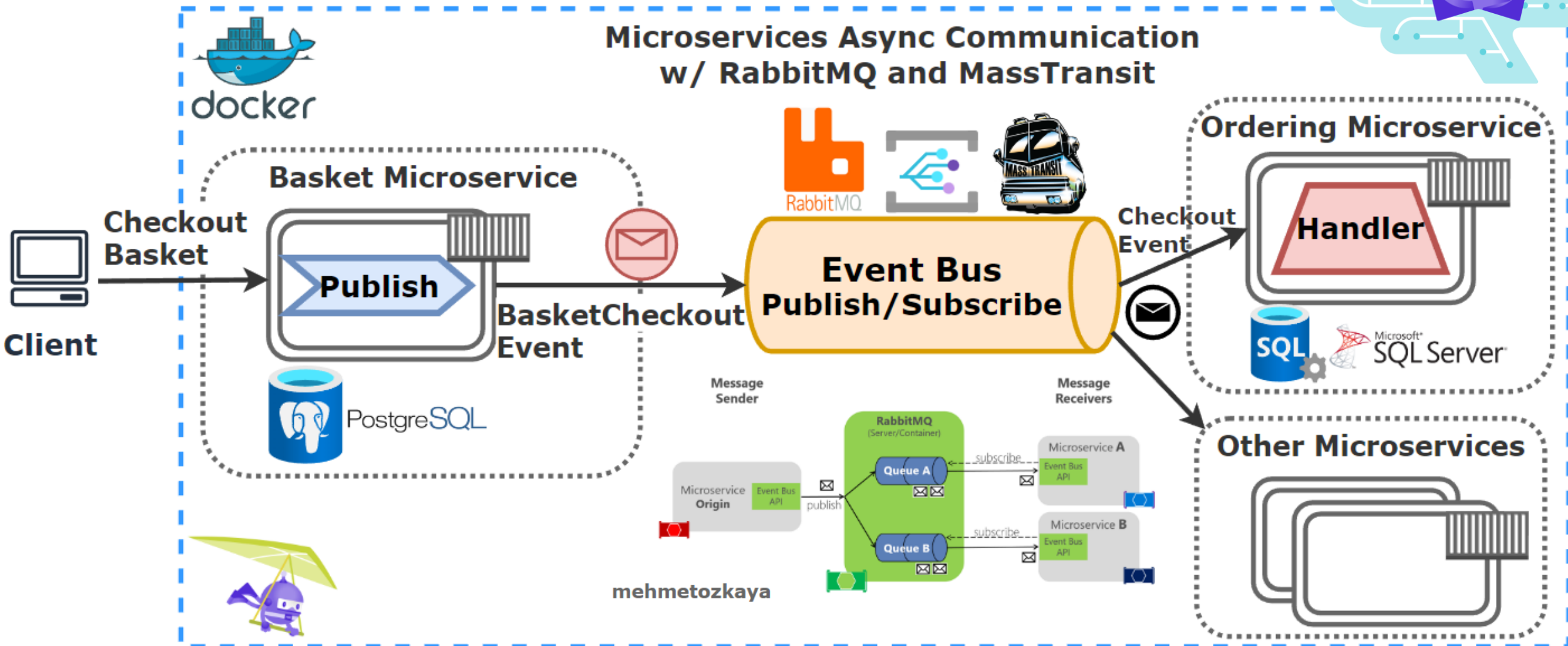
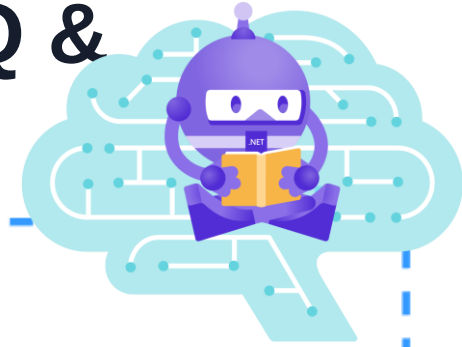
Developing Discount.Grpc Microservices



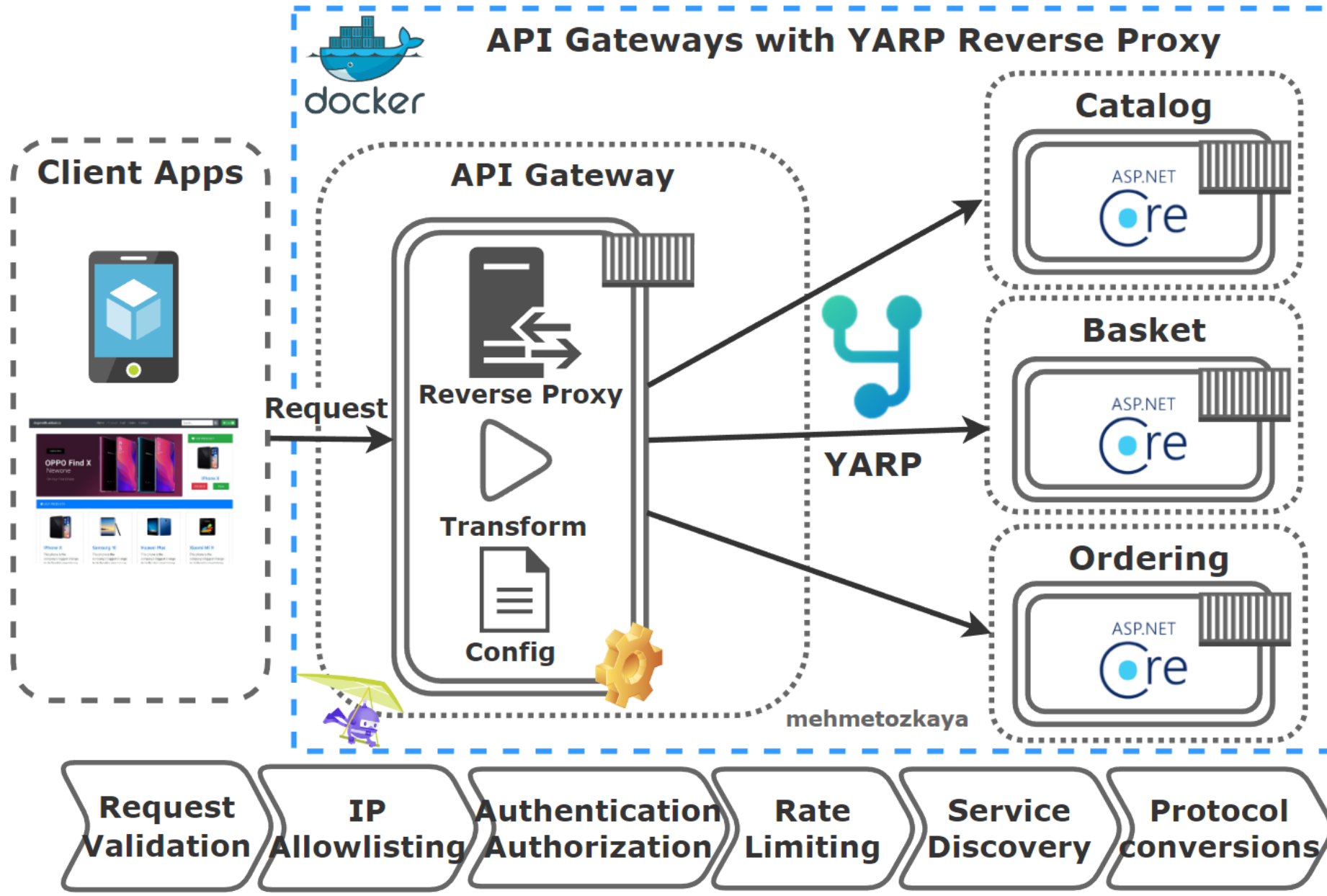
Ordering Microservice w/ DDD, CQRS and Clean Architecture



Microservices Async Communication w/ RabbitMQ & MassTransit



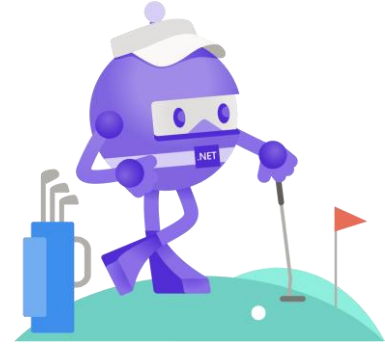
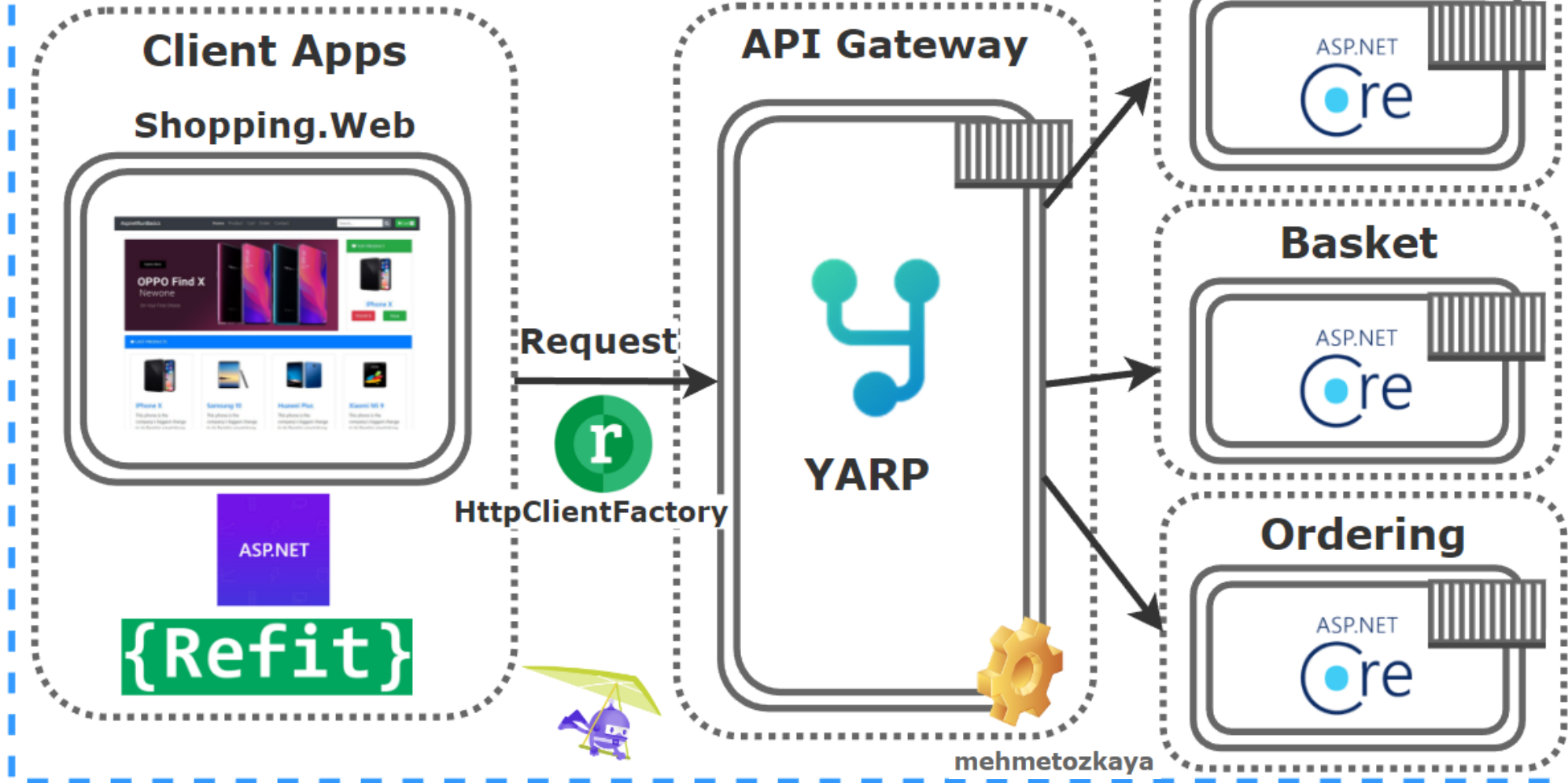
API Gateways with YARP Reverse Proxy



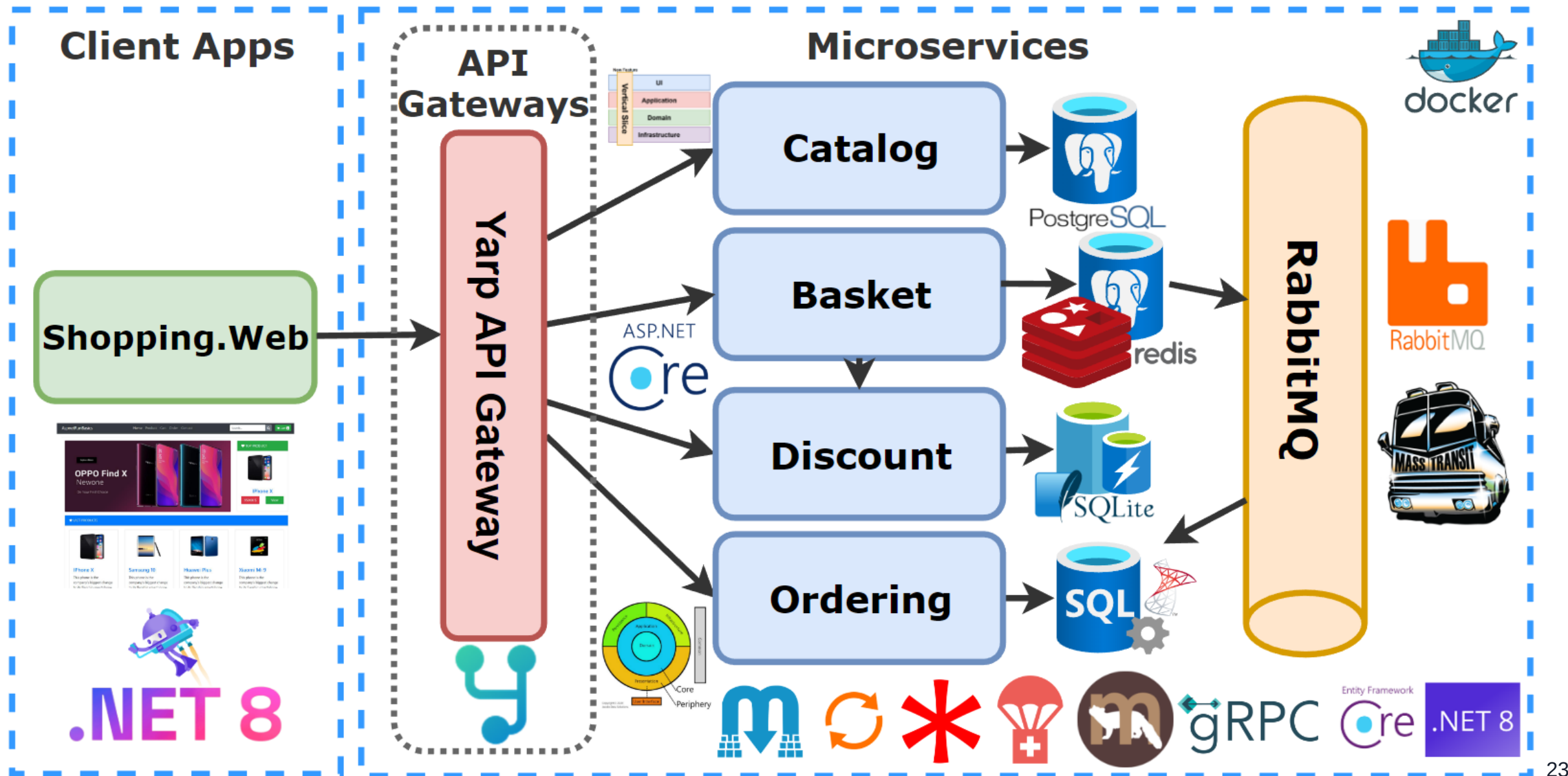
Shopping Web Client Application with Refit



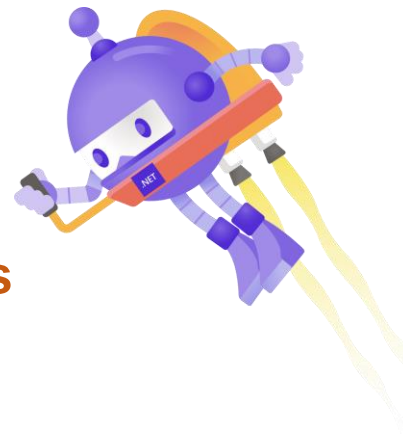
Shopping.Web Client Application w/ Refit HttpClientFactory



Big Picture – Course Project



Eshop Microservices Project



Architectures

- Layered Architecture
- DDD
- Vertical Slide Architecture
- Clean Architecture

Patterns&Principles

- SOLID
- CQRS
- Mediator, Decorator, Options
- Pub/Sub, Caching
- Api Gateway

Databases

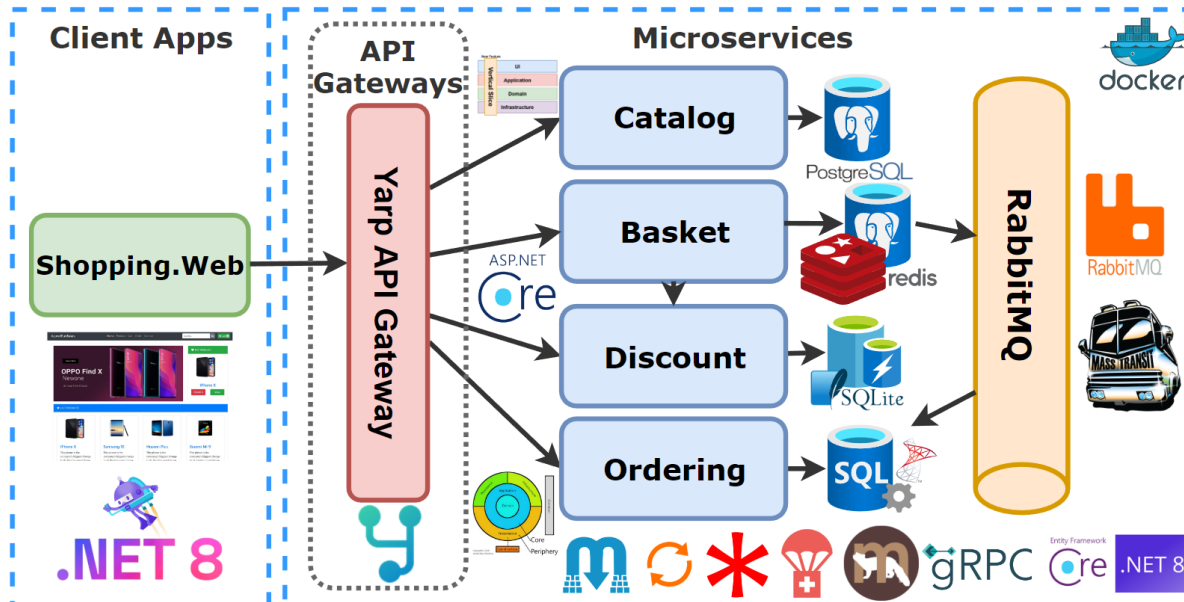
- Transactional Document DB
- Postgres
- Redis
- SQLite
- SQL Server

Libraries

- Carter
- Marten
- MediatR
- Mapster
- MassTransit
- FluentValidation
- EF Core, Refit

Communications

- Sync: gRPC
- Async: Publish/Subscribe pattern w/ MassTransit and RabbitMQ
- Yarp Api Gateway





aspnetrun

aspnetrun

The best path to leverage your aspnet skills. Onboarding to .net software architect jobs. Building production-ready enterprise

👤 421 followers

📍 Istanbul

🔗 <https://medium.com/aspnetrun>

🐦 @ezozkme

✉️ ezozkme@gmail.com

🏠 Overview

📁 Repositories 13

📁 Projects

📦 Packages

👤 People 1

Pinned

📁 learn Public

The best path to .Net Microservices Udemy Learning Path. .Net world evolving to the microservices and Cloud-native systems to provide rapid change, large scale, and resilience cutting-edge systems....

★ 41 🍴 14

📁 run-aspnetcore-microservices Public

Microservices on .Net platforms which used Asp.Net Web API, Docker, RabbitMQ, MassTransit, Grpc, Ocelot API Gateway, MongoDB, Redis, PostgreSQL, SqlServer, Dapper, Entity Framework Core, CQRS and C...

● C# ★ 2.2k 🍴 1.2k

📁 run-aspnetcore Public template

A starter kit for your next ASP.NET Core web application. Boilerplate for ASP.NET Core reference application, demonstrating a layered application architecture with applying Clean Architecture and D...

● C# ★ 503 🍴 158

📁 run-aspnet-identityserver4 Public

Secure microservices with using standalone Identity Server 4 and backing with Ocelot API Gateway. Protect our ASP.NET Web MVC and API applications with using OAuth 2 and OpenID Connect in IdentityS...

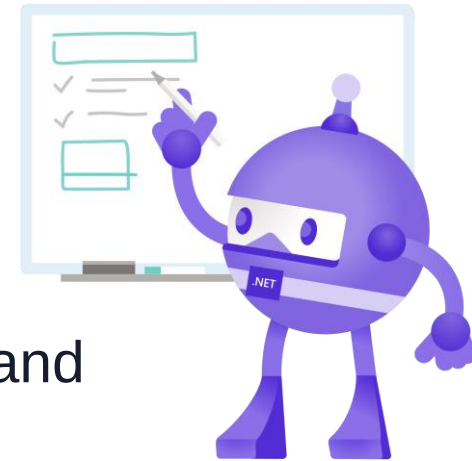
● SCSS ★ 214 🍴 139

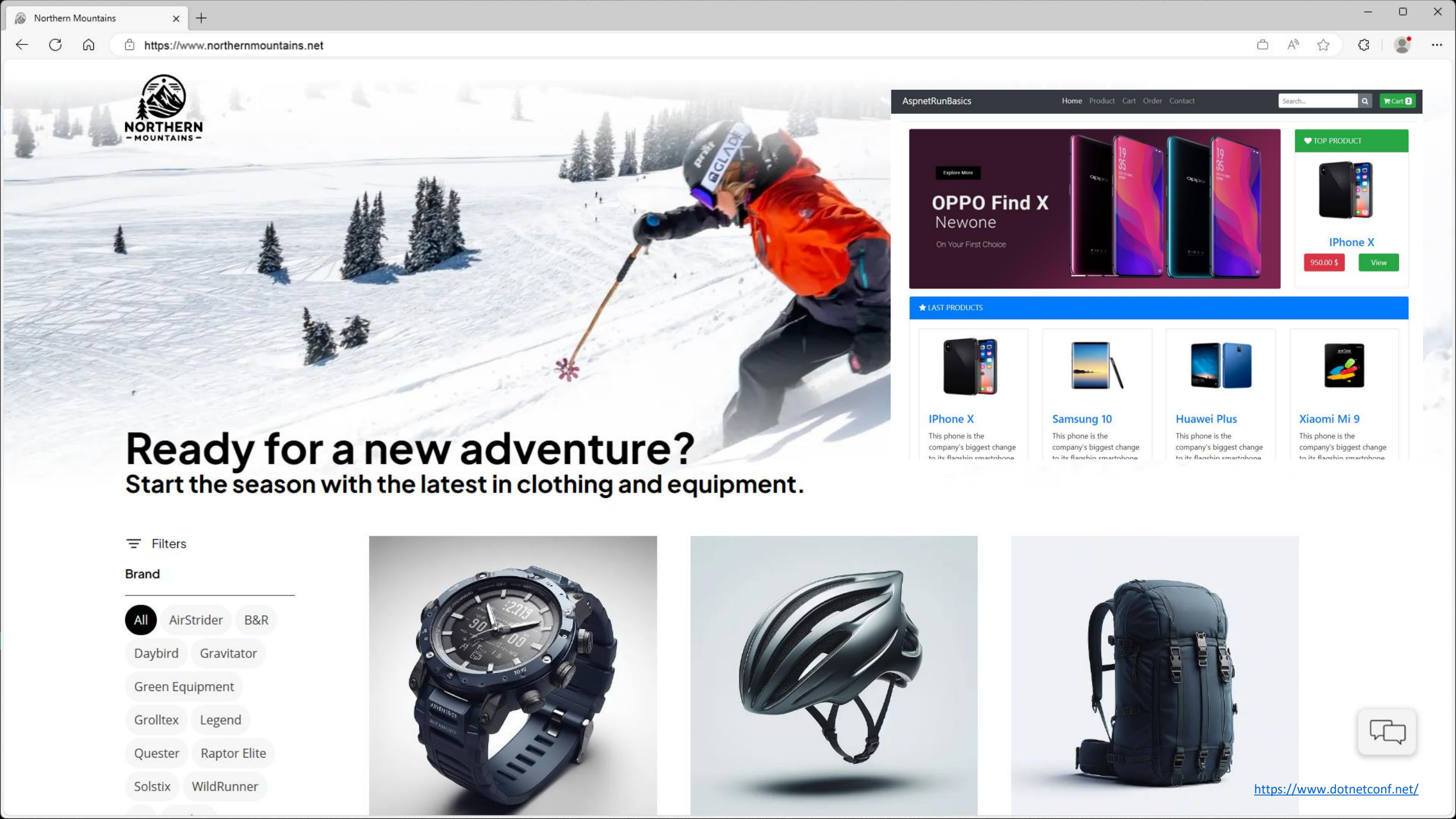
<https://github.com/aspnetrun>

<https://github.com/aspnetrun/run-aspnetcore-microservices>

Course Target

- To be a **master of developing .Net 8 microservices** with **patterns, principles** and best **practices**.
- Designed for **software developers** and **architects**
- **%95** of Hands-on Design and Development Activities
- Apply **best practices** with **cloud-native microservices design patterns** and **principles**
- **Latest features** of **C# 12**, **.Net 8** and **Asp.Net 8** like Minimal Apis, primary constructors and so on.
- **Solid understanding** of the **.Net 8 Microservices** and will be ready to **design, develop, and deploy** your own **microservices applications**.





Ready for a new adventure?
Start the season with the latest in clothing and equipment.

Filters

Brand

- All
- AirStrider
- B&R
- Daybird
- Gravitator
- Green Equipment
- Grolltex
- Legend
- Quester
- Raptor Elite
- Solstix
- WildRunner



<https://www.dotnetconf.net/>

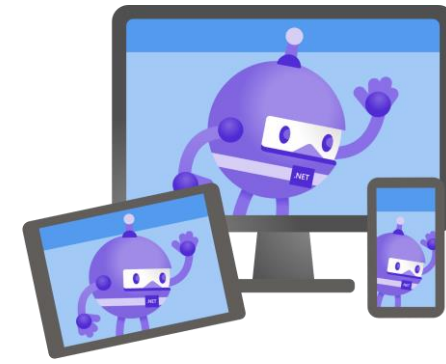
Prerequisites

- [.Net 8](#)
- [Visual Studio](#)
- [Docker Desktop](#)
 - At least 7 GB of free space
- [Postman](#)

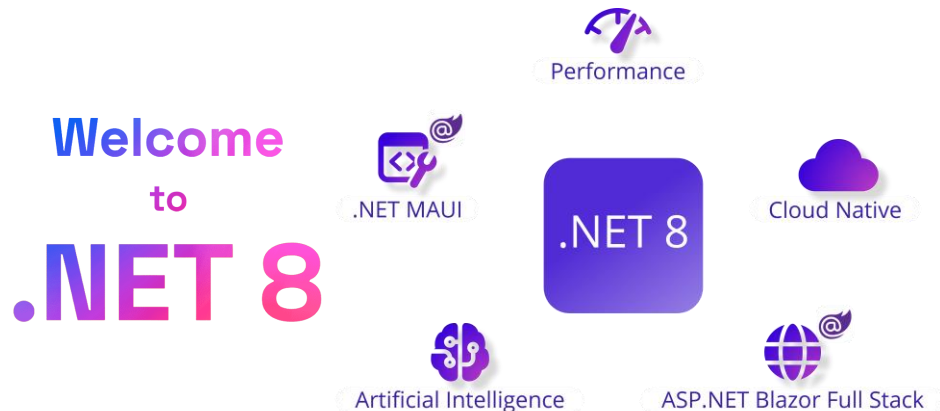
.NET 8



Visual Studio

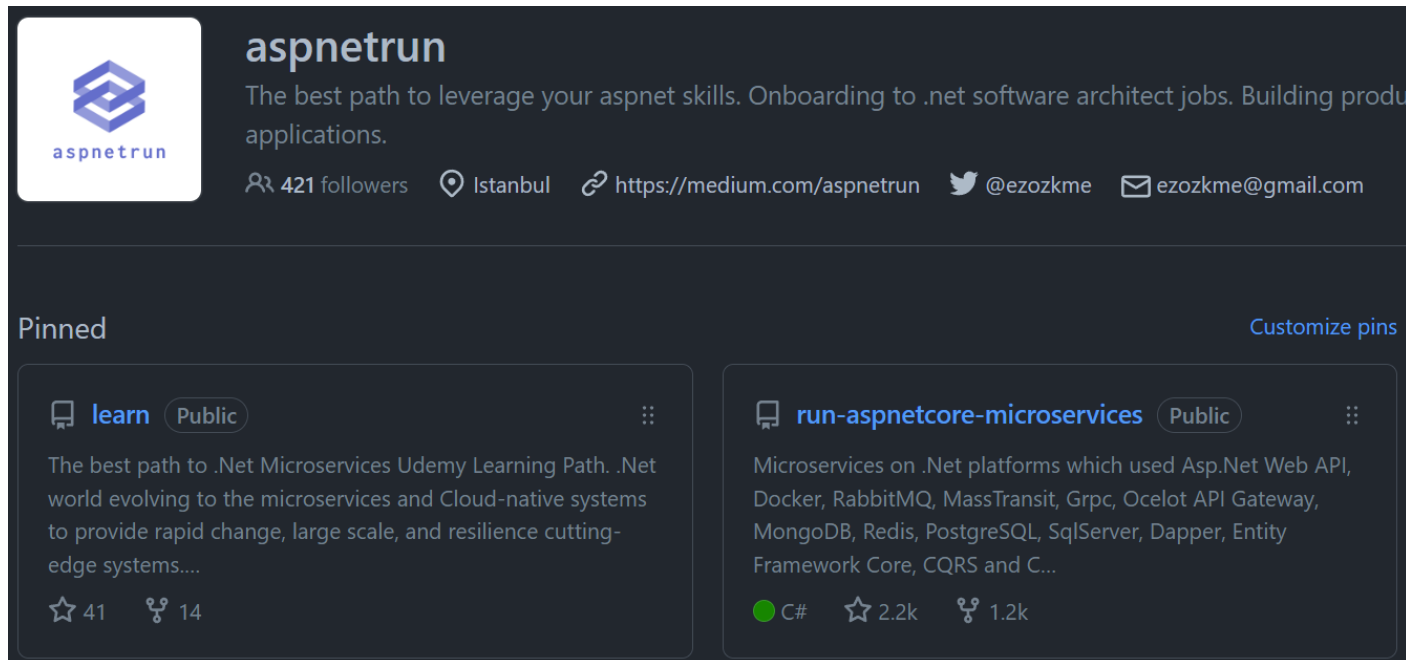


- Install & Download before starting course



Source Code on GitHub

- **Aspnetrun GitHub EShop Microservices Repository**
 - <https://github.com/aspnetrun/run-aspnetcore-microservices>
 - Alternative: <https://github.com/mehmetozkaya/EShopMicroservices>
- **Section by section GitHub EShop Microservices Repository**
 - <https://github.com/mehmetozkaya/EShopMicroservices-Udemy-Sections>



Course Slides

- Download Course Slides from **This Lecture Resources**
- Section by section GitHub Microservices Repository
 - <https://github.com/aspnetrun/run-aspnetcore-microservices>

Course content ×

☐ 23. Variables
9min

Resources ▾

☐ 24. Starting out with Expressions
7min

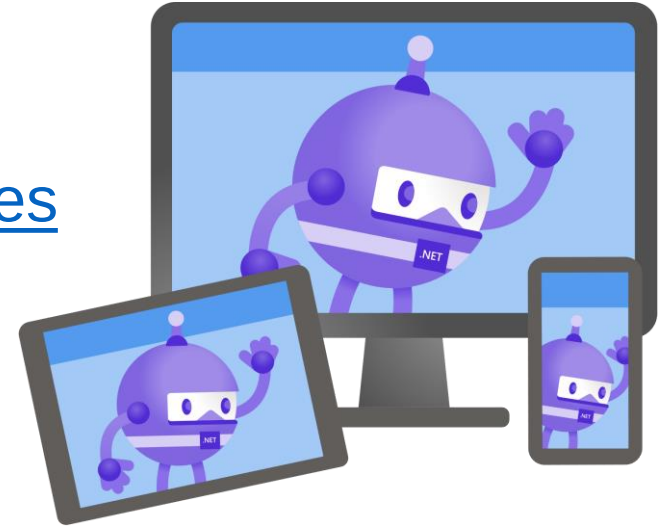
Resources ▾

☐ 25. Primitive Types
12min

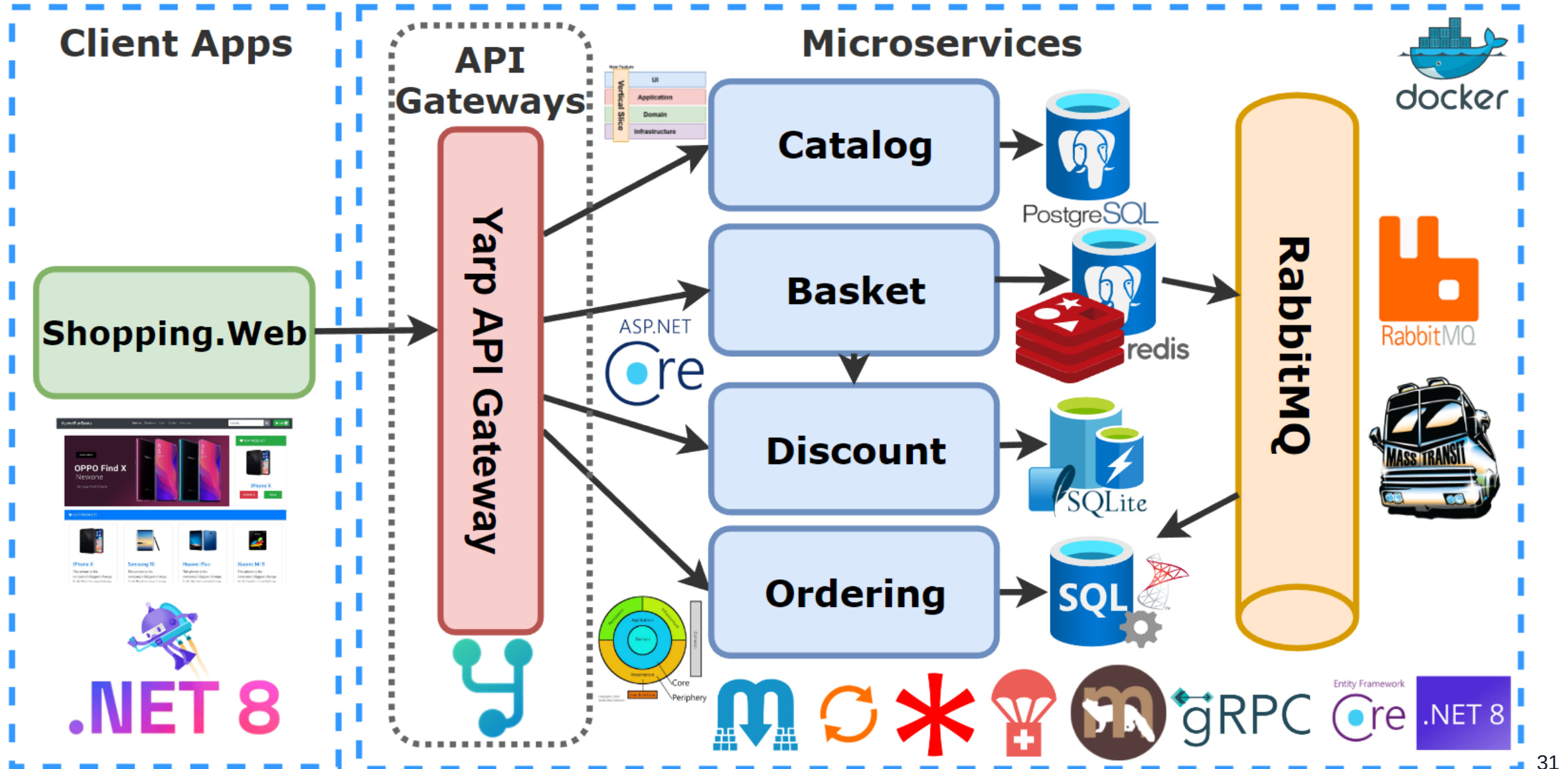
Resources ▾

☐ 26. byte, short, long and width
10min

Resources ▾

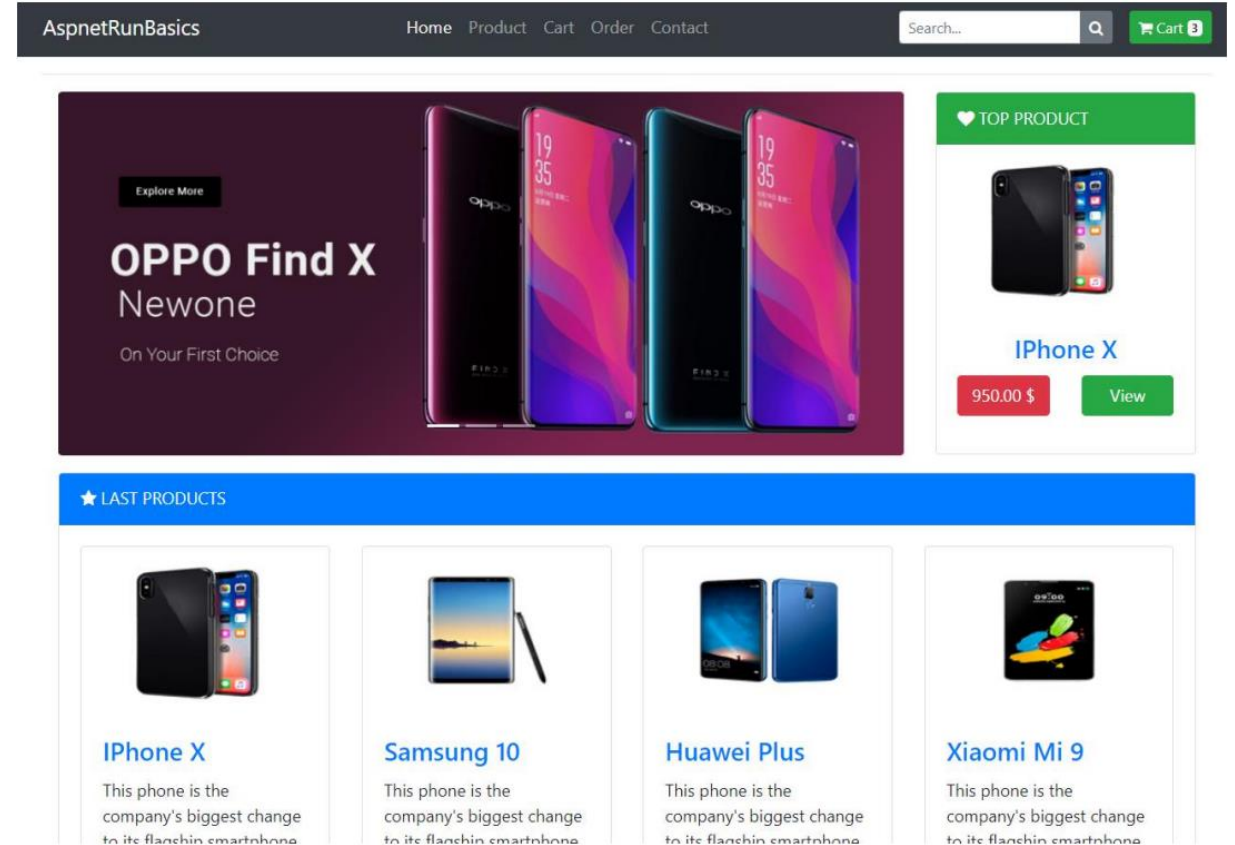
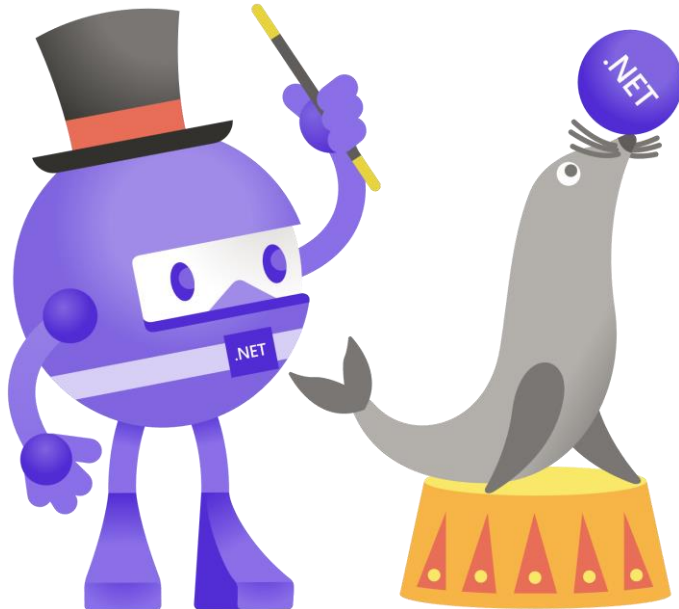


Big Picture – Course Project

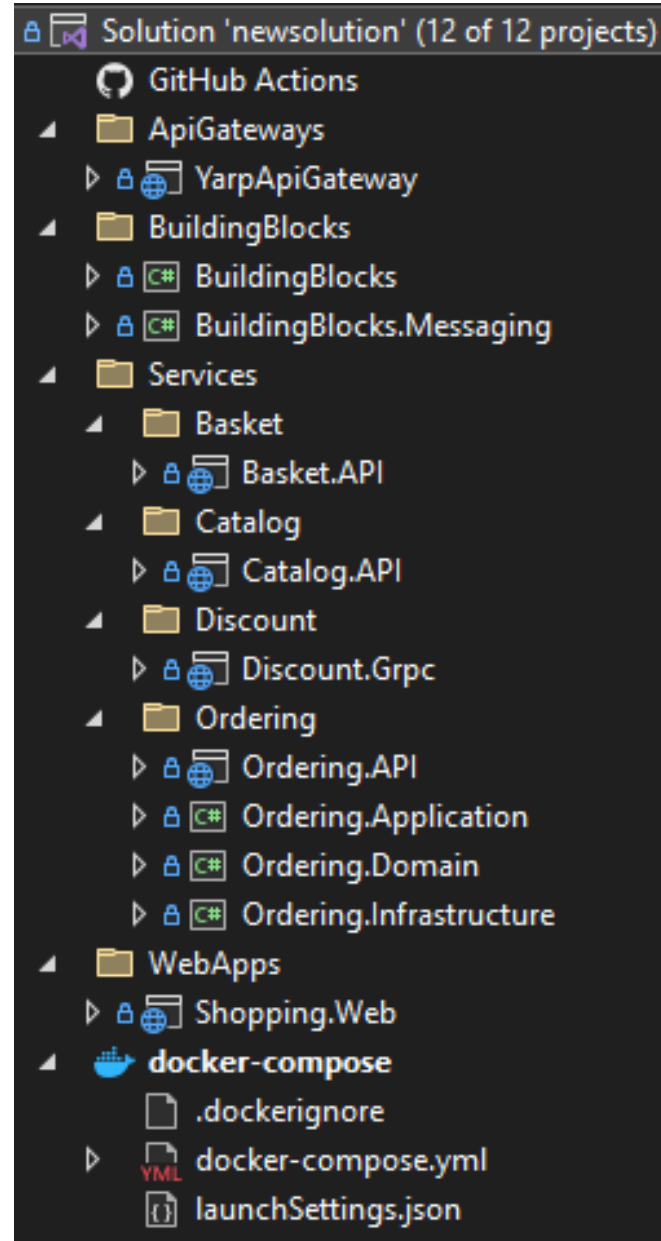


DEMO: Run Final Application

- Clone Microservices Repository: <https://github.com/aspnetrun/run-aspnetcore-microservices>
- Docker-compose up
- Follow steps on GitHub documentation
- Run the Project Section
- DEMO



Project Code Structure



Architectures, Patterns, Libraries and Best Practices



Architectures

- Layered Architecture
- Vertical Slide Architecture
- DDD
- Clean Architecture

Patterns&Principles

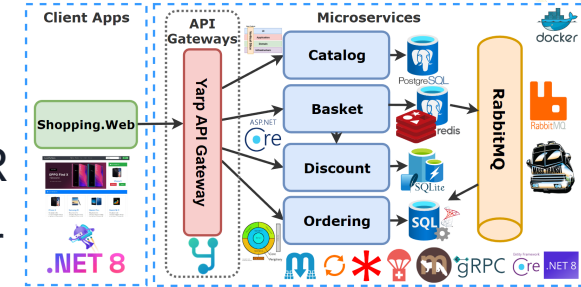
- SOLID, DI
- CQRS
- Mediator, Proxy Decorator, Options
- Pub/Sub, Caching
- Api Gateway

Databases

- Transactional Document DB
- Postgres
- Redis
- SQLite
- SQL Server

Libraries

- Carter
- Marten
- MediatR
- Mapster
- MassTransit
- FluentValidation
- EF Core, Refit



Containerization and Orchestration

- Dockerfiles and Docker Compose to containerize and orchestrate

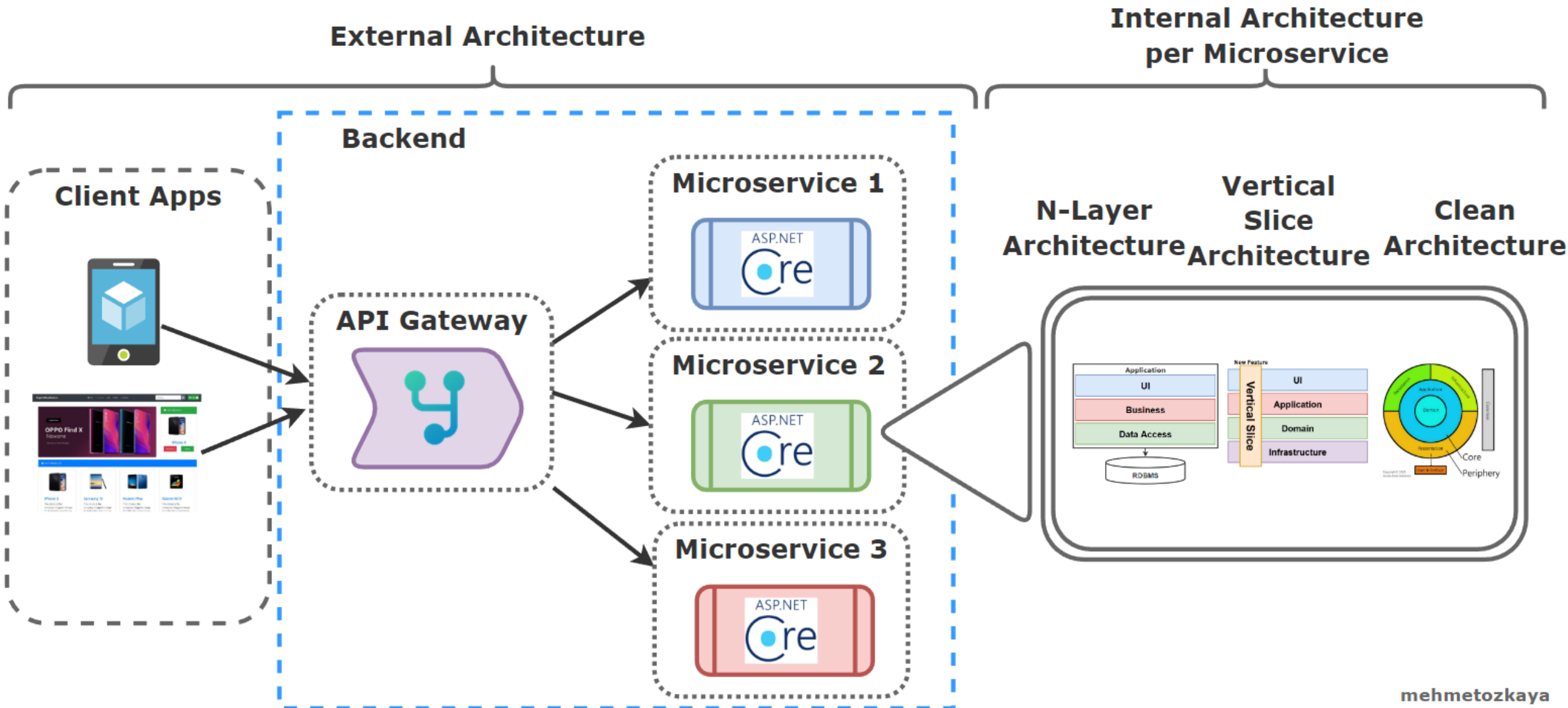
Api Gateway and Client

- Yarp Api Gateway reverse proxy – Gateway Routing
- Shopping Web consume APIs using the Refit library

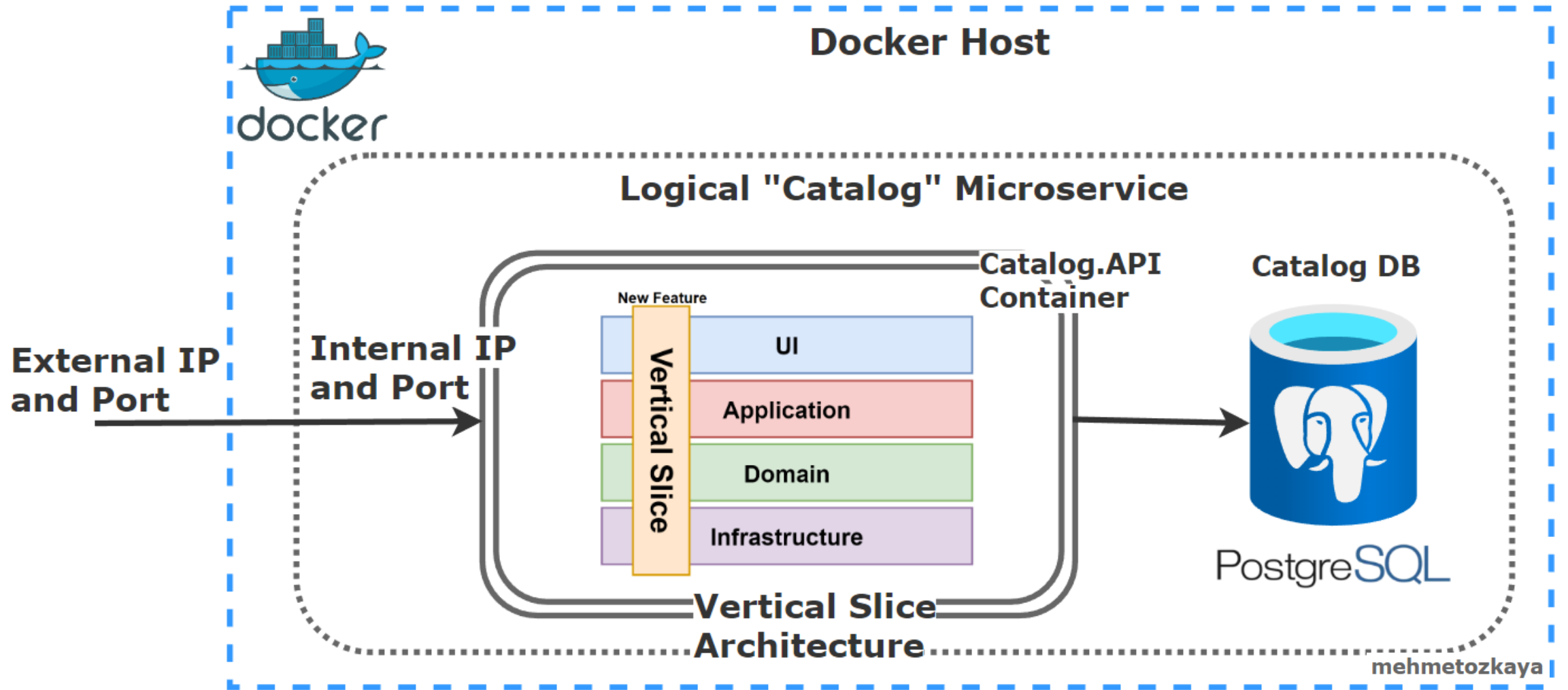
Communications

- Sync: gRPC
- Async: Publish/Subscribe pattern w/ MassTransit and RabbitMQ

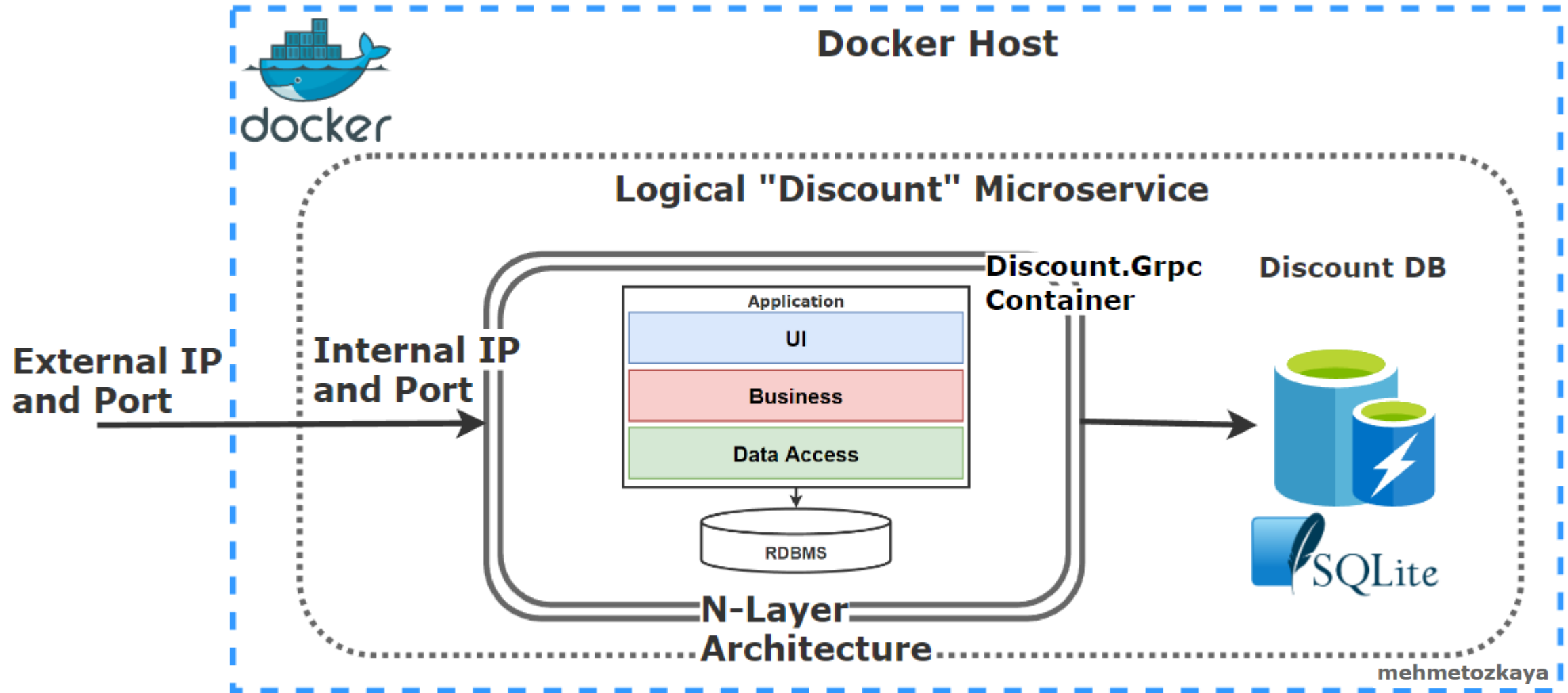
External vs. Internal Architecture



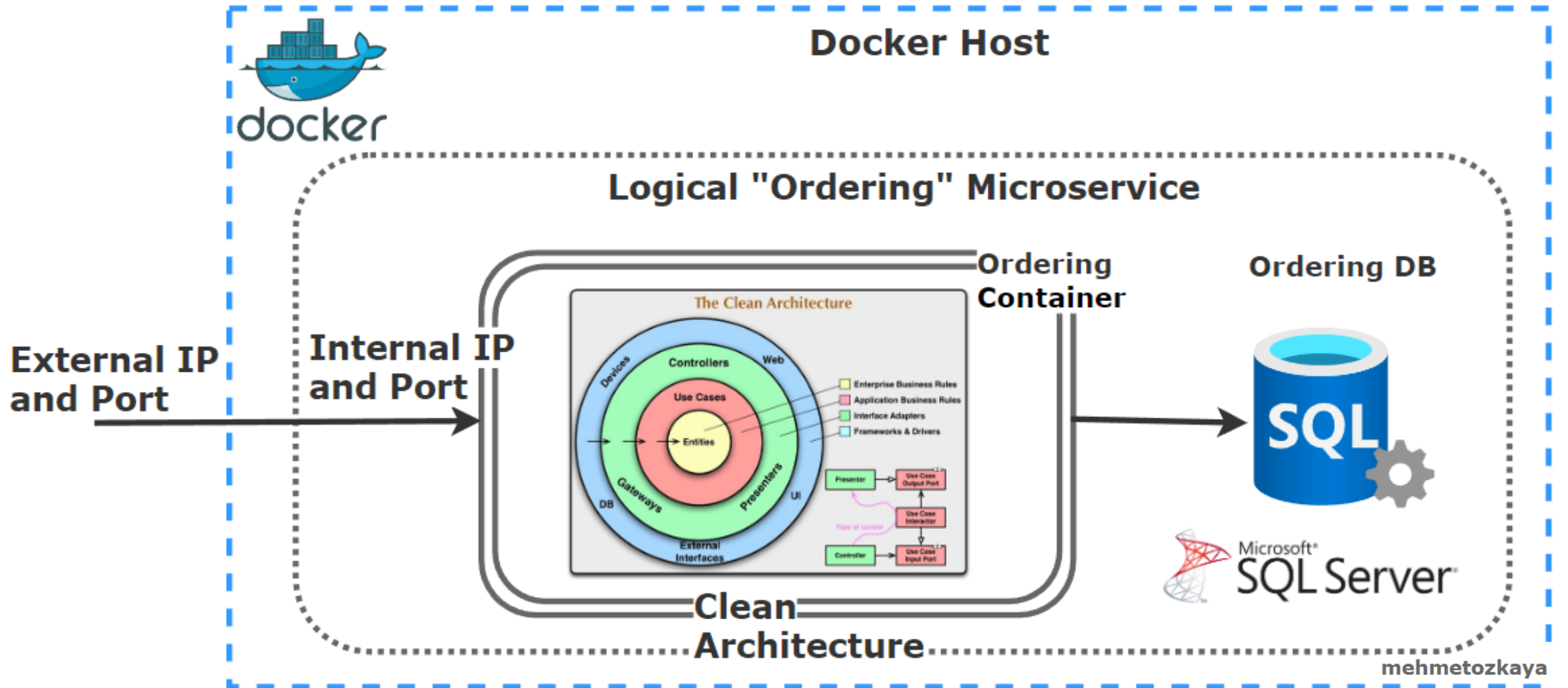
Catalog and Basket Microservices



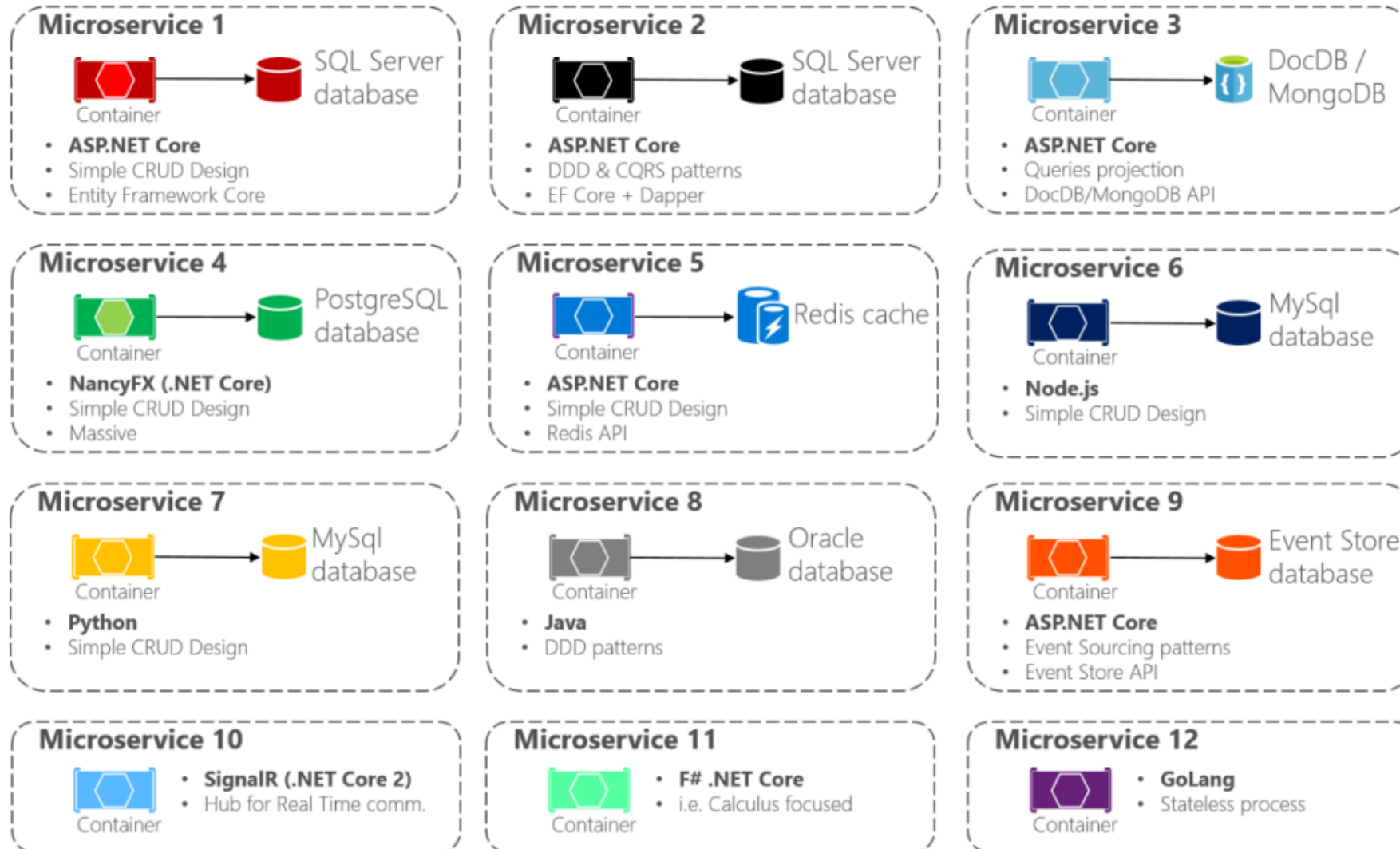
Discount Microservice



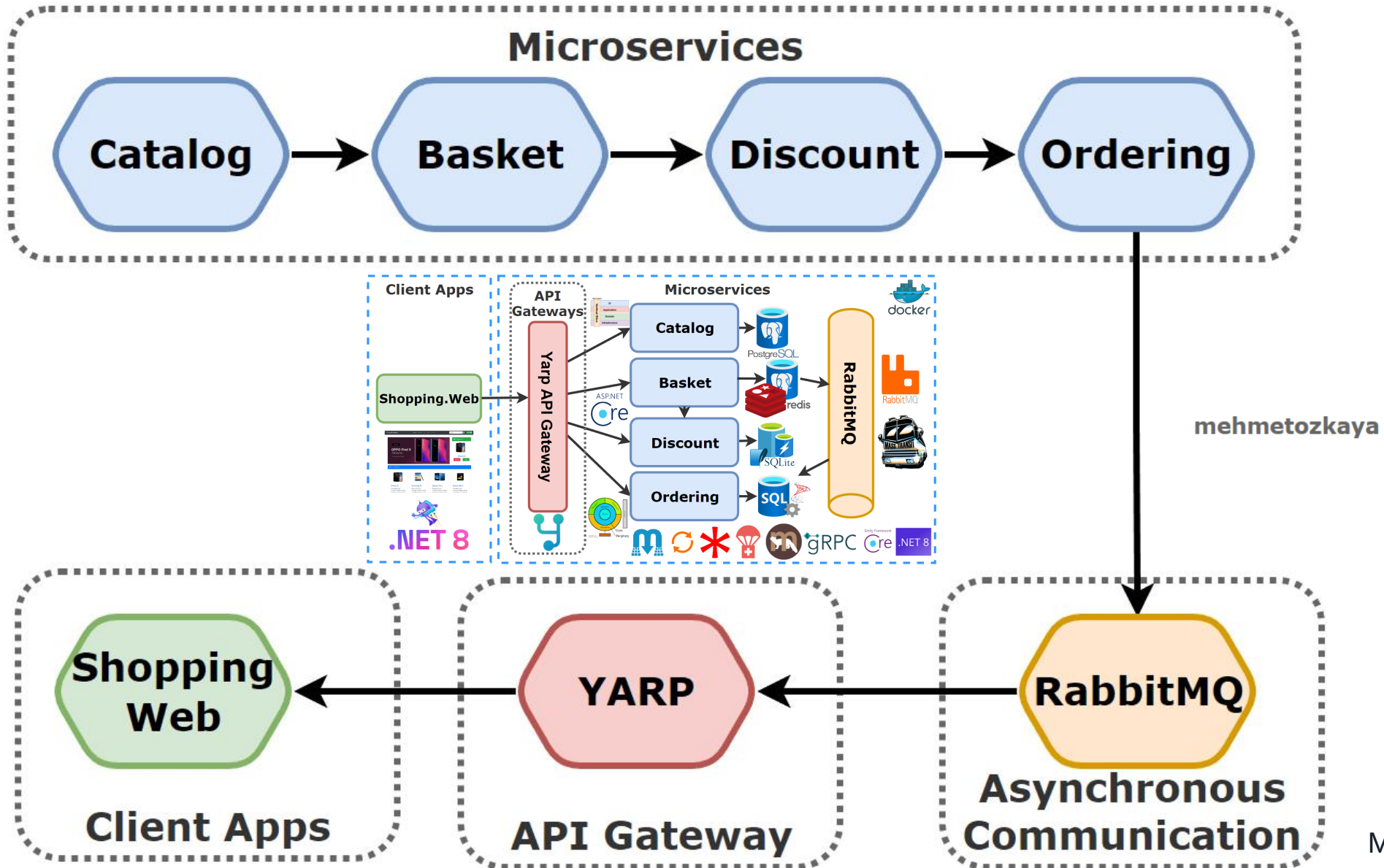
Ordering Microservice



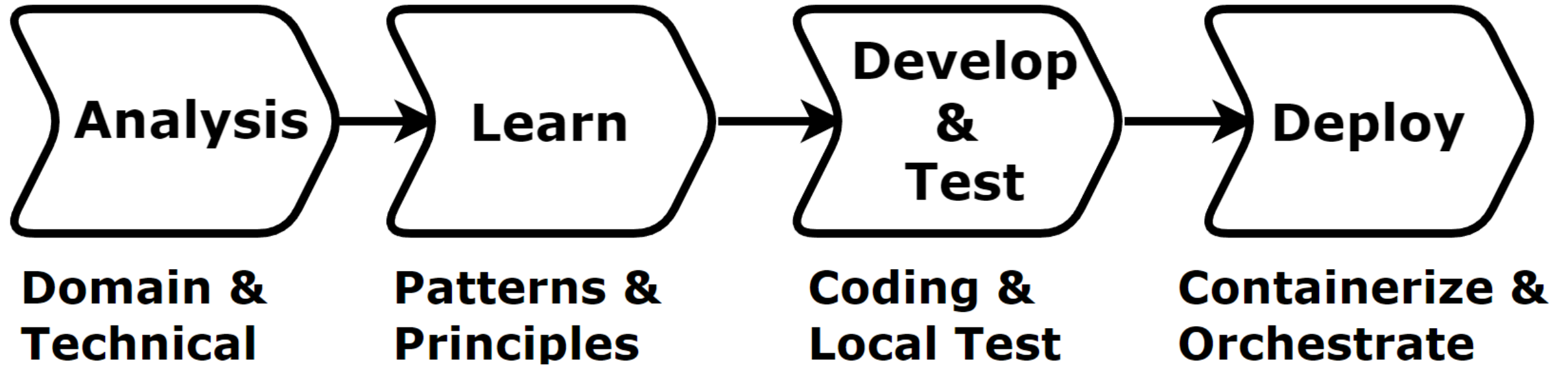
Multi-Architectural Polyglot Microservices

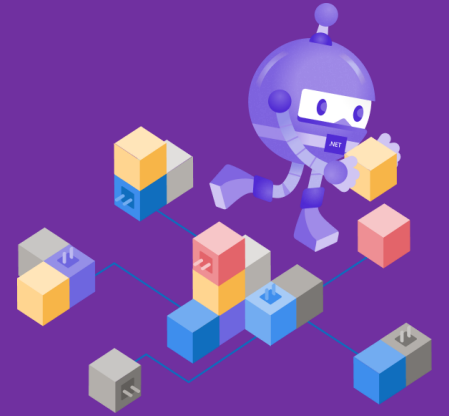


How to Follow the Course (The Course Flow)



Way of Learning (For Each Microservices)





What are Microservices ?

What is Microservices Architecture ?

Microservices Characteristics

Benefits & Challenges of Microservices Architectures

When and When Not to Use Microservices Architecture

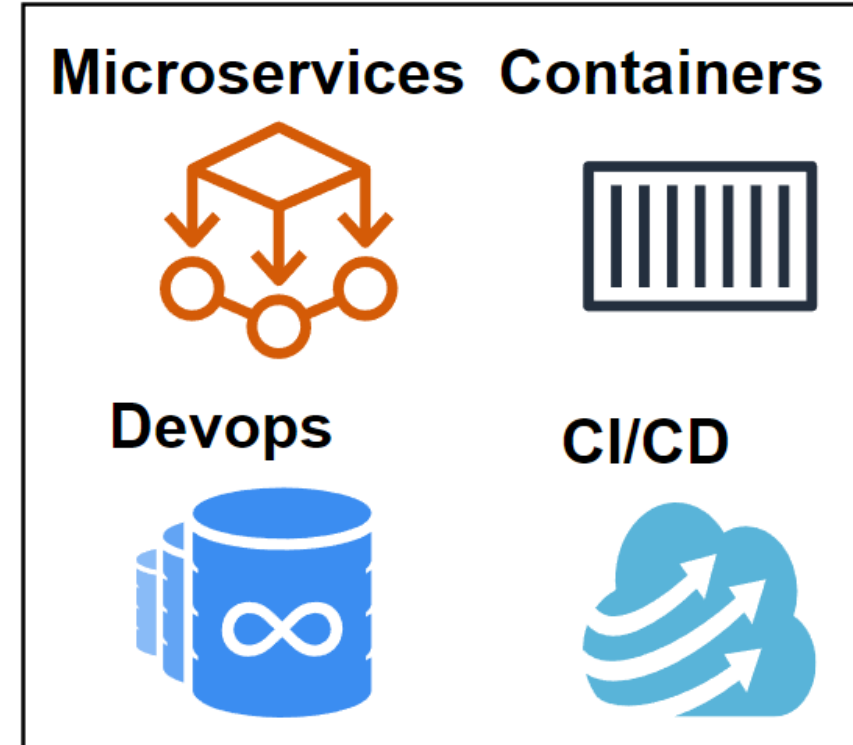
Monolithic vs Microservices Architecture Comparison

Mehmet Ozkaya

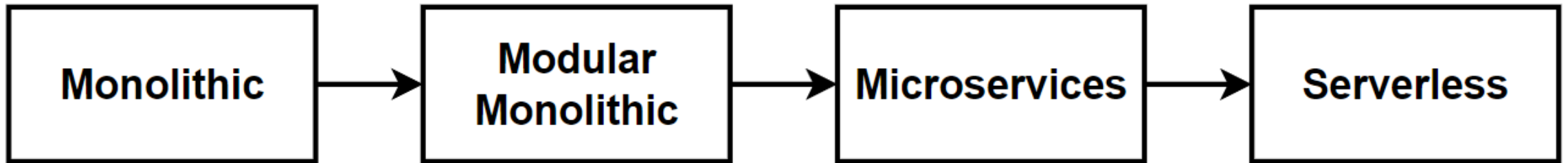


Learn: Microservices

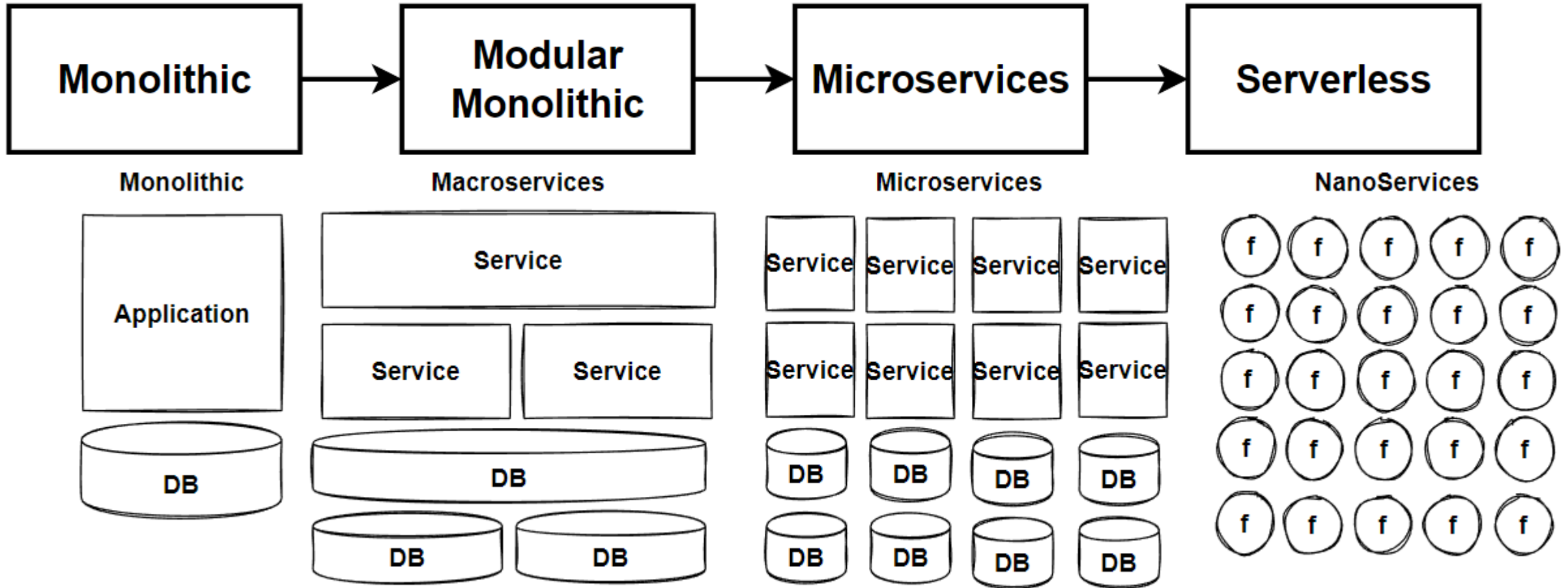
- What are Microservices ?
- What is Microservices Architecture ?
- Microservices Characteristics
- Benefits of Microservices Architecture
- Challenges of Microservices Architecture
- When to Use Microservices Architecture - Best Practices
- When Not to Use Microservices - Anti-Patterns of Microservices
- Monolithic vs Microservices Architecture Comparison
- The Database-per-Service Pattern - Polygot Persistence
- Design the Architecture - Microservices Architecture - First Iteration



Architecture Design Journey

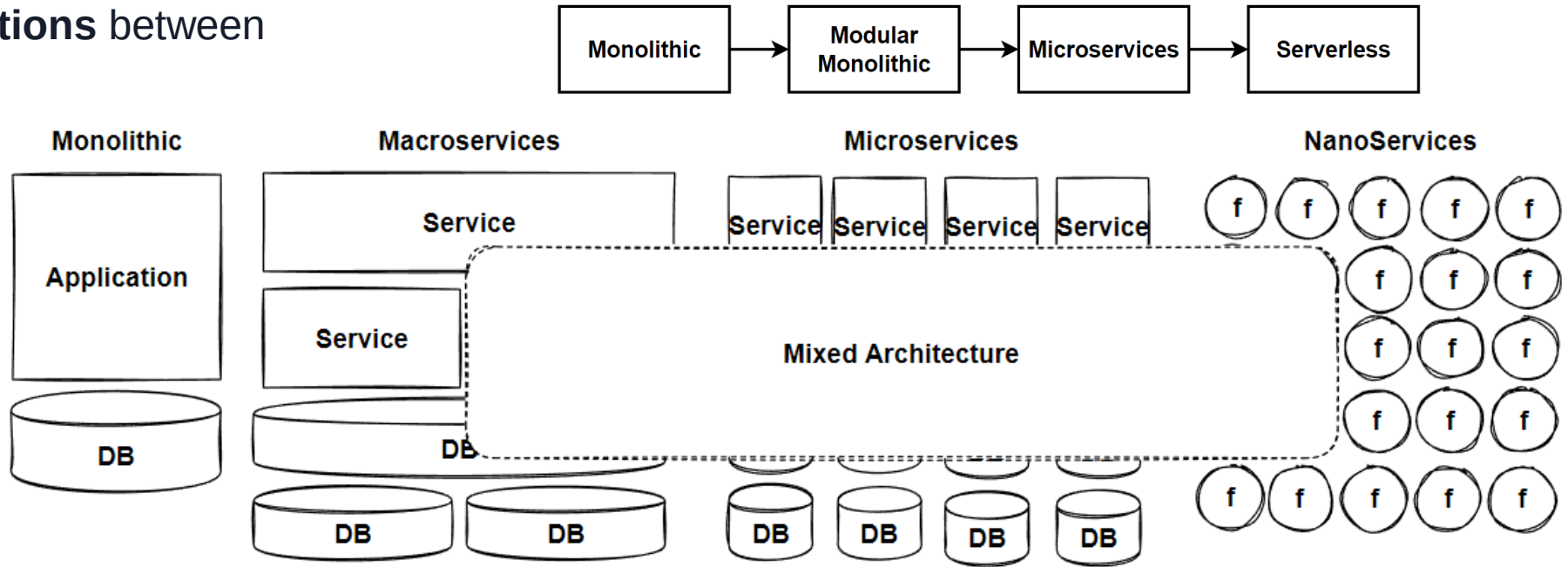


Macroservices to Nanoservices



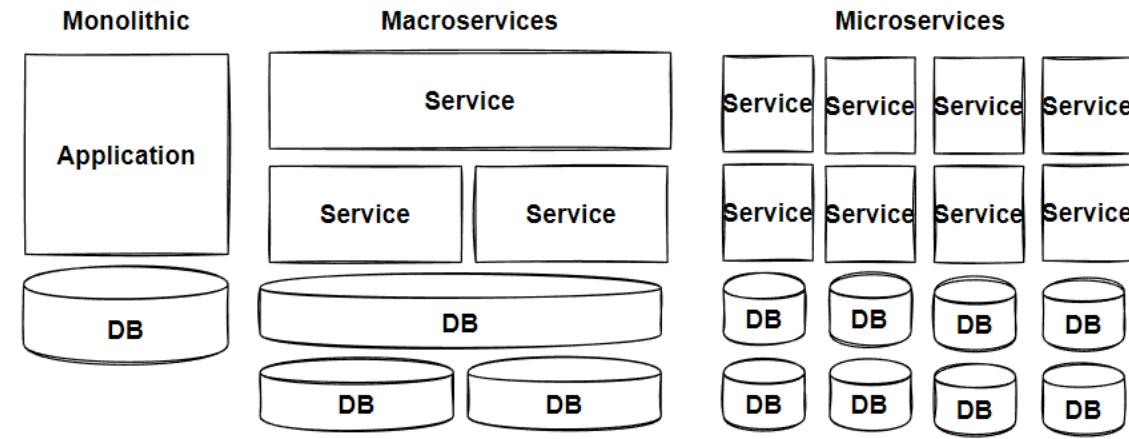
Which architecture approach we should choose ?

- It depends on your project
- Use **Mixed Architecture**
- Provide **transitions** between architectures



What are Microservices ?

- **Microservices** are **small, independent, and loosely coupled** services that can work together.
- Each service is a **separate codebase**, which can be managed by a small development team.
- Microservices **communicate** with each other by using **well-defined APIs**.
- Microservices can be **deployed independently** and **autonomously**.
- Microservices can work with many different technology stacks which is **technology agnostic**.
- Microservices has its **own database** that is not shared with other services.



What is Microservices Architecture ?

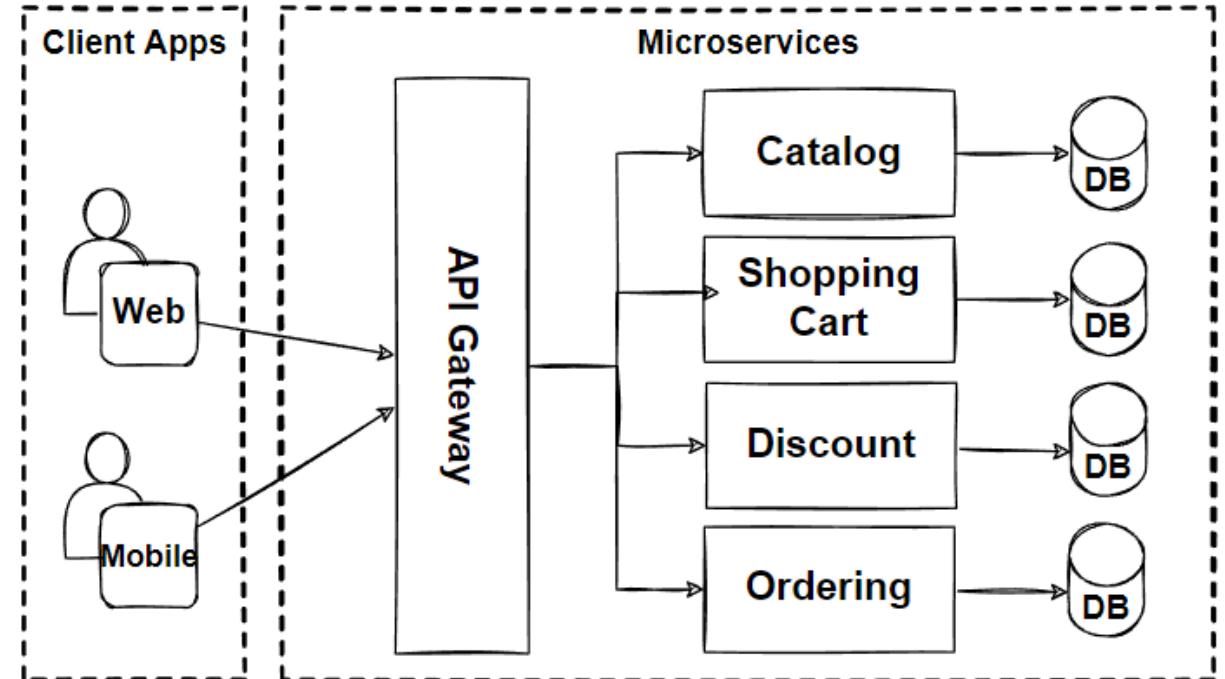
- [From Martin Fowlers Microservices article;](#)

*The microservice architectural style is an approach to developing a single application as a **suite of small services**, each running in **its own process** and **communicating** with lightweight mechanisms, often an **HTTP** or **gRPC API**.*

- Microservices are **built around business capabilities** and **independently deployable** by fully automated deployment process.
- Microservices architecture **decomposes** an application into **small independent services** that communicate over **well-defined APIs**. Services are owned by **small, self-contained teams**.
- Microservices architecture is a **cloud native architectural approach** in which services composed of many **loosely coupled** and **independently deployable** smaller components.
- Microservices have their **own technology stack**, communicate to each other over a combination of **REST APIs**, are organized by **business capability**, with the **bounded contexts**.
- Following **Single Responsibility Principle** that referring separating responsibilities as per services.

Microservices Characteristics

- [From Martin Fowlers Microservices article;](#)
- Componentization via Services
- Organized around Business Capabilities
- Products not Projects
- Smart endpoints and dumb pipes
- Decentralized Governance
- Decentralized Data Management
- Infrastructure Automation
- Design for failure



Benefits of Microservices Architecture

- **Agility, Innovation and Time-to-market**

Microservices architectures make applications easier to scale and faster to develop, enabling innovation and accelerating time-to-market for new features.

- **Flexible Scalability**

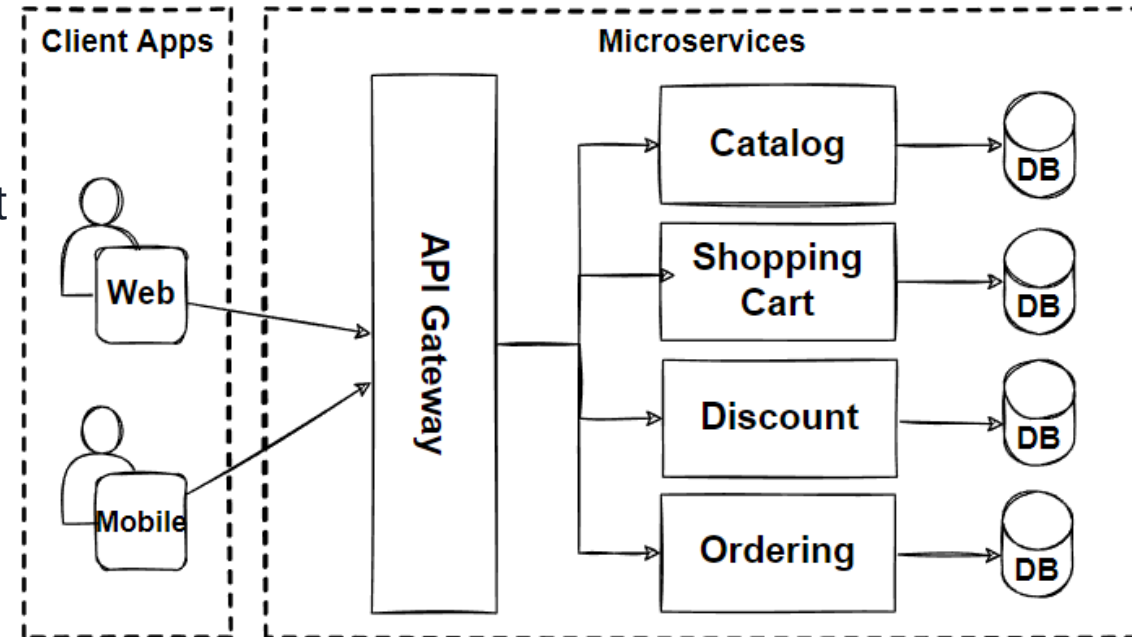
Microservices can be scaled independently, so you scale out sub-services that require less resources, without scaling out the entire application.

- **Small, focused teams**

Microservices should be small enough that a single feature team can build, test, and deploy it.

- **Small and separated code base**

Microservices are not sharing code or data stores with other services, it minimizes dependencies, and that makes easier to adding new features.



Benefits of Microservices Architecture -2

- **Easy Deployment**

Microservices enable continuous integration and continuous delivery, making it easy to try out new ideas and to roll back if something doesn't work.

- **Technology agnostic, Right tool for the job**

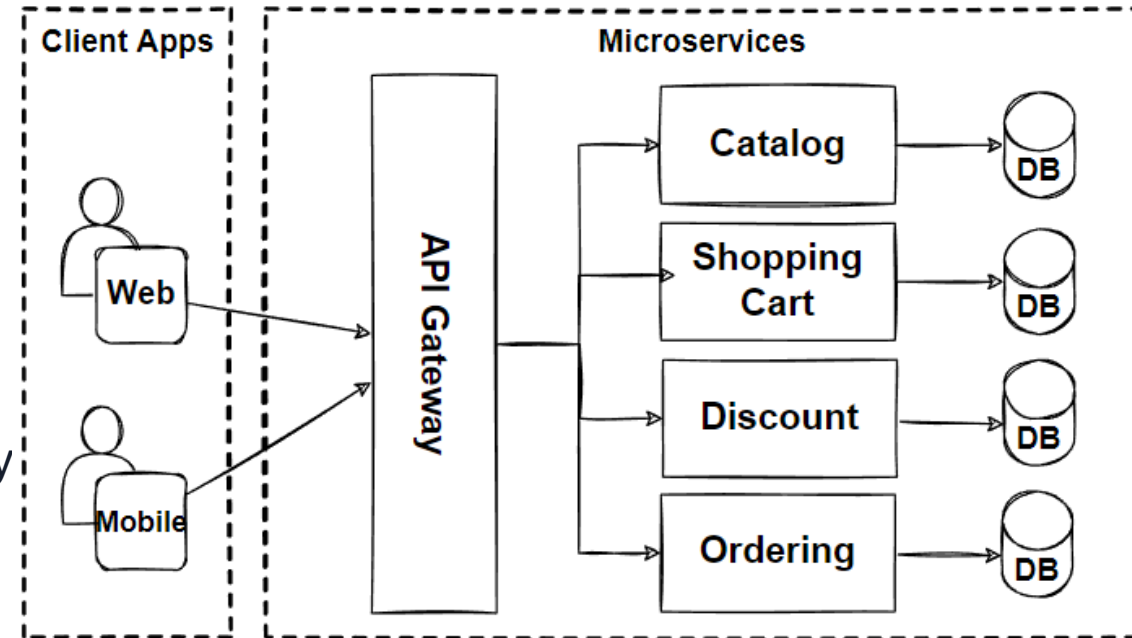
Small teams can pick the technology that best fits their microservice and using a mix of technology stacks on their services.

- **Resilience and Fault isolation**

Microservices are fault tolerated and handle faults correctly for example by implementing retry and circuit breaking patterns.

- **Data isolation**

Databases are separated with each other according to microservices design. Easier to perform schema updates, because only a single database is affected.



Challenges of Microservices Architecture

- **Complexity**

Each service is simpler, but the entire system is more complex. Deployments and Communications can be complicated for hundreds of microservices.

- **Network problems and latency**

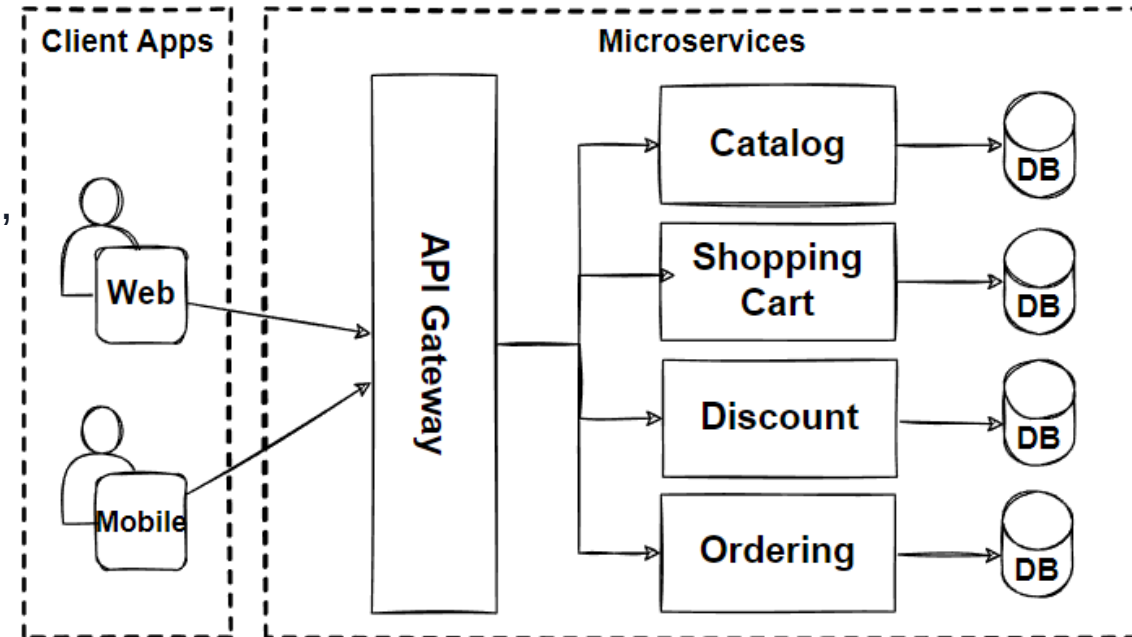
Microservice communicate with inter-service communication, we should manage network problems. Chain of services increase latency problems and become chatty API calls.

- **Development and testing**

Hard to develop and testing these E2E processes in microservices architectures if we compare to monolithic ones.

- **Data integrity**

Microservice has its own data persistence. Data consistency can be a challenge. Follow eventual consistency where possible.



Challenges of Microservices Architecture -2

- **Deployment**

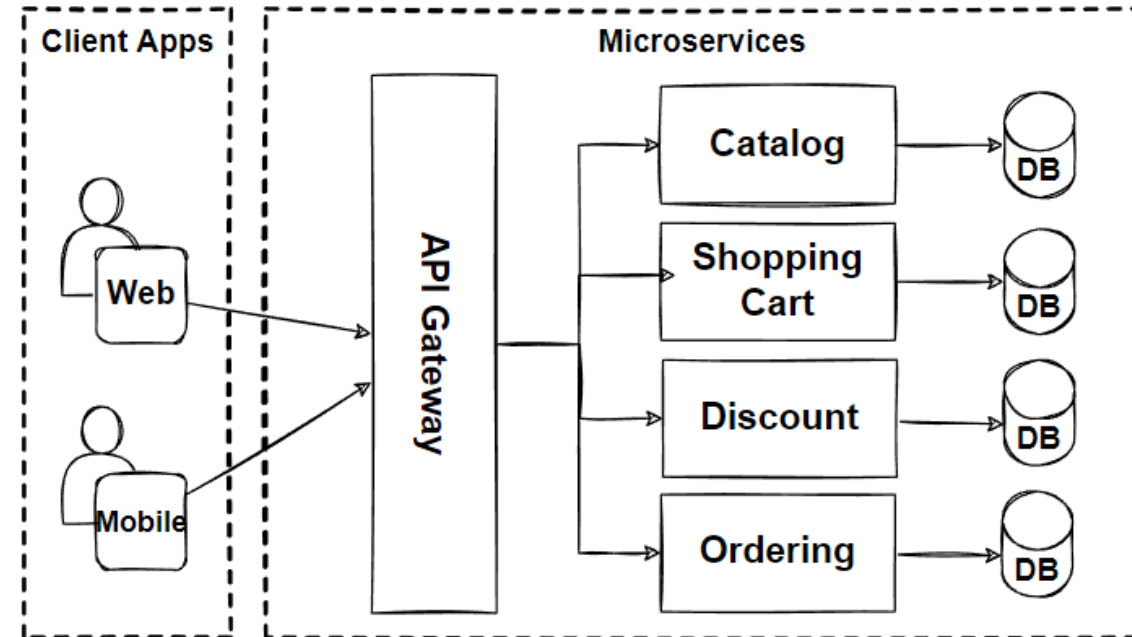
Deployments are challenging. Require to invest in quite a lot of devops automation processes and tools. The complexity of microservices becomes overwhelming for human deployment.

- **Logging & Monitoring**

Distributed systems are required to centralized logs to bring everything together. Centralized view of the system to monitor sources of problems.

- **Debugging**

Debugging through local IDE isn't an option anymore. It won't work across dozens or hundreds of services.



When to Use Microservices Architecture

- **Make Sure You Have a “Really Good Reason” for Implementing Microservices**

Check if your application can do without microservices. When your application requires agility to time-to-market with zero-down time deployments and updated independently that needs more flexibility.

- **Iterate With Small Changes and Keep the Single-Process Monolith as Your “Default”**

Sam Newman and Martin Fowler offers Monolithic-First approach. Single-process monolithic application comes with simple deployment topology. Iterate and refactor with turning a single module from the monolith into a microservices one by one.

- **Required to Independently Deploy New Functionality with Zero Downtime**

When an organization needs to make a change to functionality and deploy that functionality without affecting rest of the system.

- **Required to Independently Scale a Portion of Application**

Microservice has its own data persistence. Data consistency can be a challenge. Follow eventual consistency where possible.

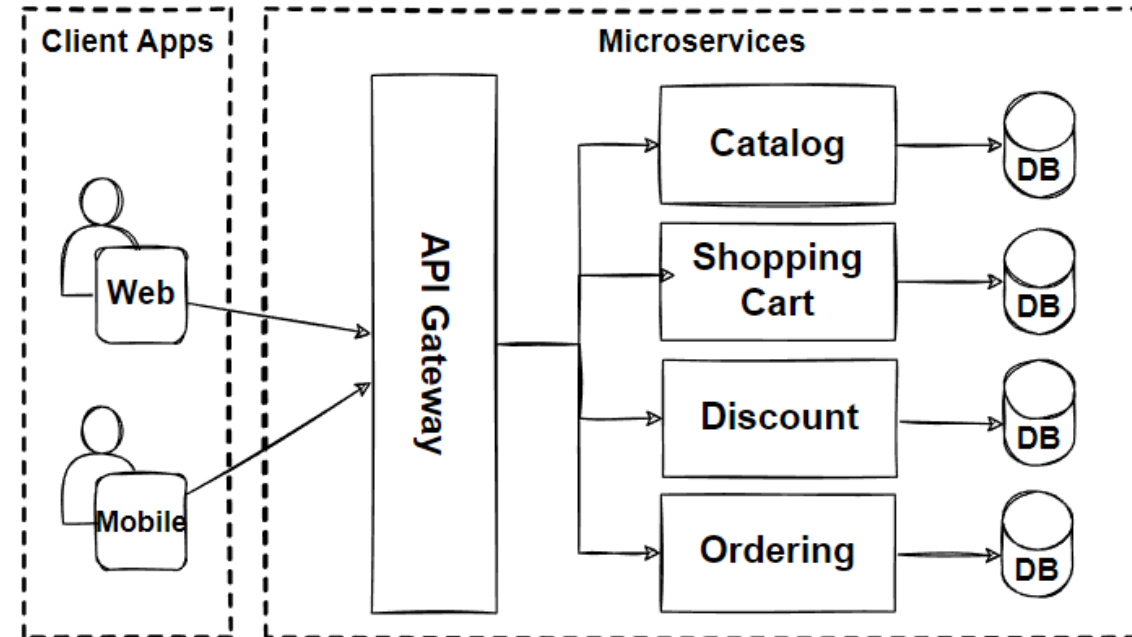
When to Use Microservices Architecture -2

- **Data Partitioning with different Database Technologies**

Microservices are extremely useful when an organization needs to store and scale data with different use cases. Teams can choose the appropriate technology for the services they will develop over time.

- **Autonomous Teams with Organizational Upgrade**

Microservices will help to evolve and upgrade your teams and organizations. Organizations need to distribute responsibility into teams, where each team makes decisions and develops software autonomously.



When **Not** to Use Microservices

Anti-Patterns of Microservices

- **Don't do Distributed Monolith**

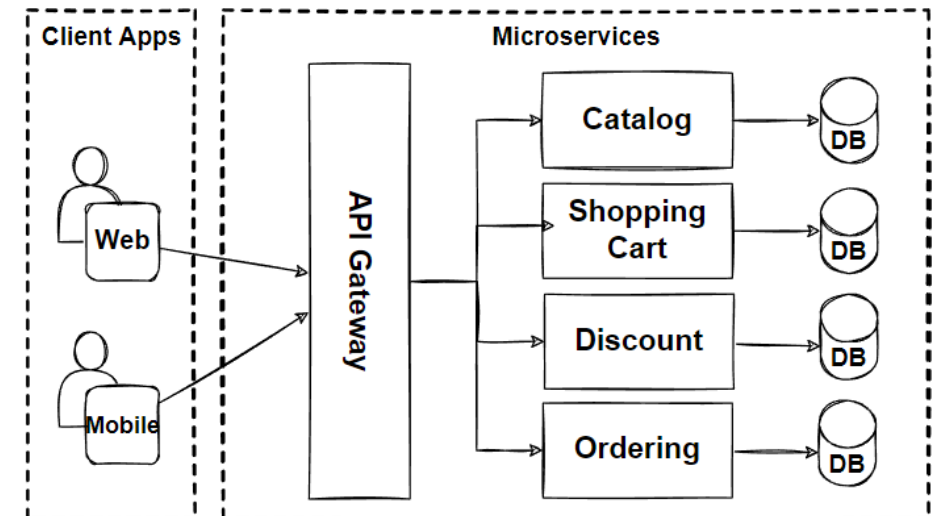
Make sure that you decompose your services properly and respecting the decoupling rule like applying bounded context and business capabilities principles.

- Distributed Monolith is the worst case because you increase complexity of your architecture without getting any benefit of microservices.

- **Don't do microservices without DevOps or cloud services**

Microservices are embrace the distributed cloud-native approaches. And you can only maximize benefits of microservices with following these cloud-native principles.

- CI/CD pipeline with devops automations
- Proper deployment and monitoring tools
- Managed cloud services to support your infrastructure
- Key enabling technologies and tools like Containers, Docker, and Kubernetes
- Following async communications using Messaging and event streaming services



When **Not** to Use Microservices

- **Limited Team sizes, Small Teams**

If you don't have a team size that cannot handle the microservice workloads, This will only result in the delay of delivery.

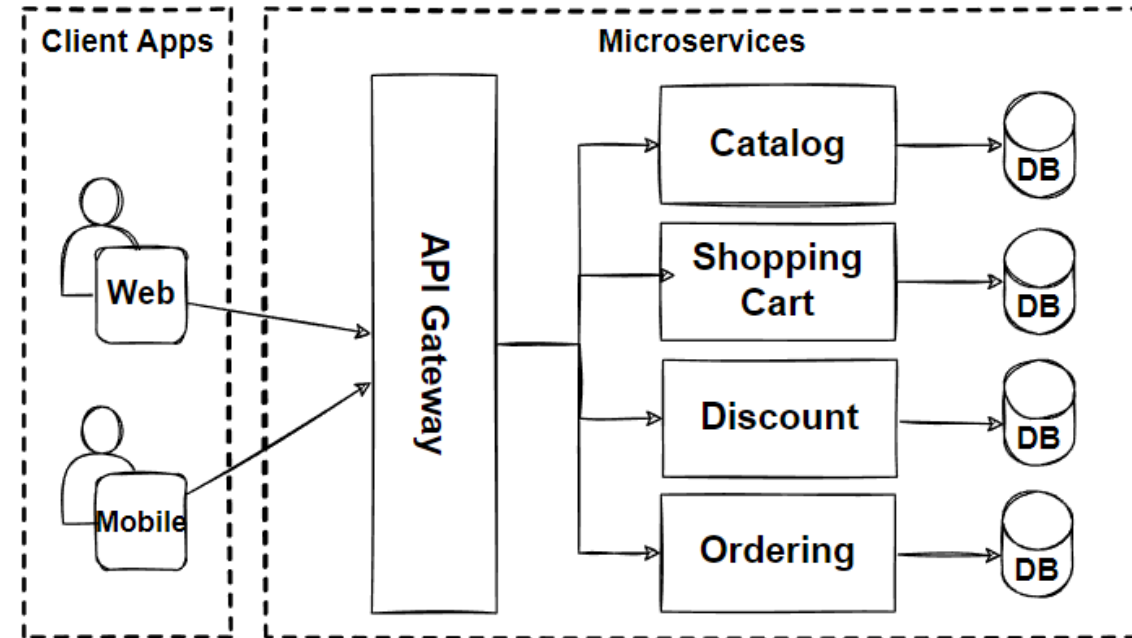
- For a small team, a microservice architecture can be hard to justify, because team is required just to handle the deployment and management of the microservices themselves.

- **Brand new products or startups**

If you are working on a new startup or brand new product which require significant change when developing and iterating your product, then you should not start with microservices.

- Microservices are so expensive when you re-design your business domains. Even if you do become successful enough to require a highly scalable architecture.

- **The Shared Database anti-pattern**



Monolithic vs Microservices Architecture Comparison

- **Application Architecture**

Monolith has a simple straightforward structure of one undivided unit. Microservices have a complex structure that consists of various heterogeneous services and databases.

- **Scalability**

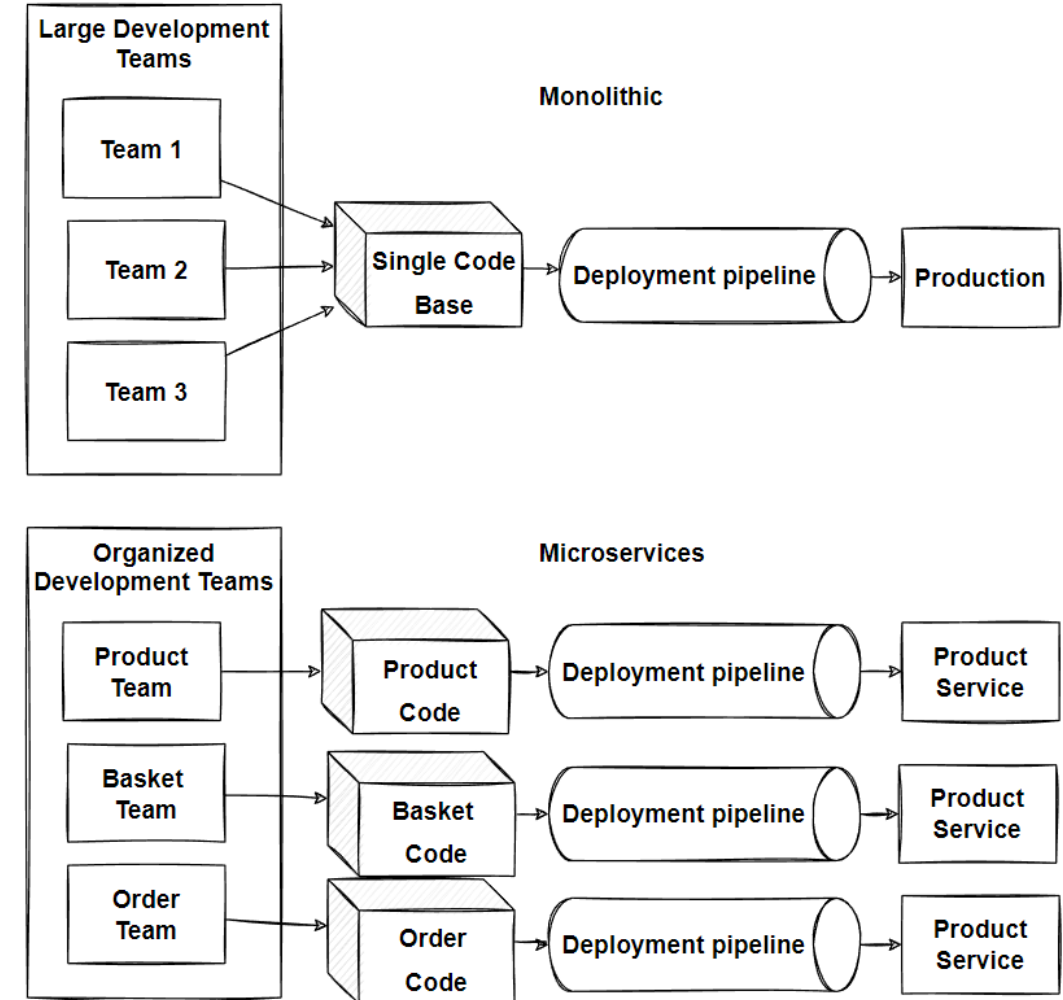
Monolithic application is scaled as a whole single unit, but microservices can be scaled unevenly. encourages companies to migrate their applications to microservices.

- **Deployment**

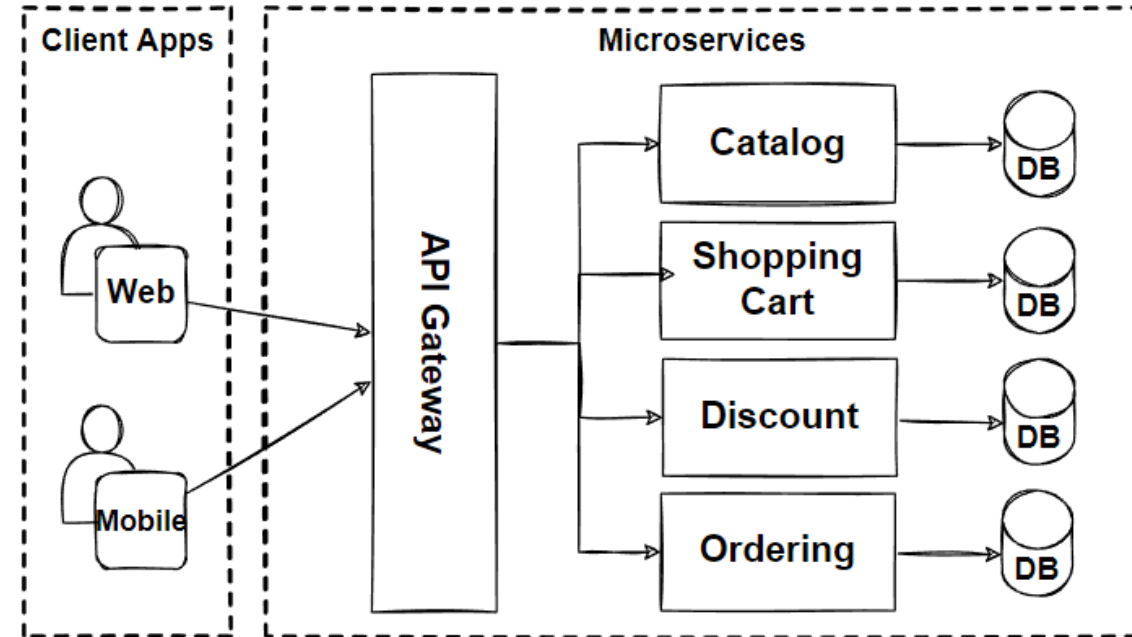
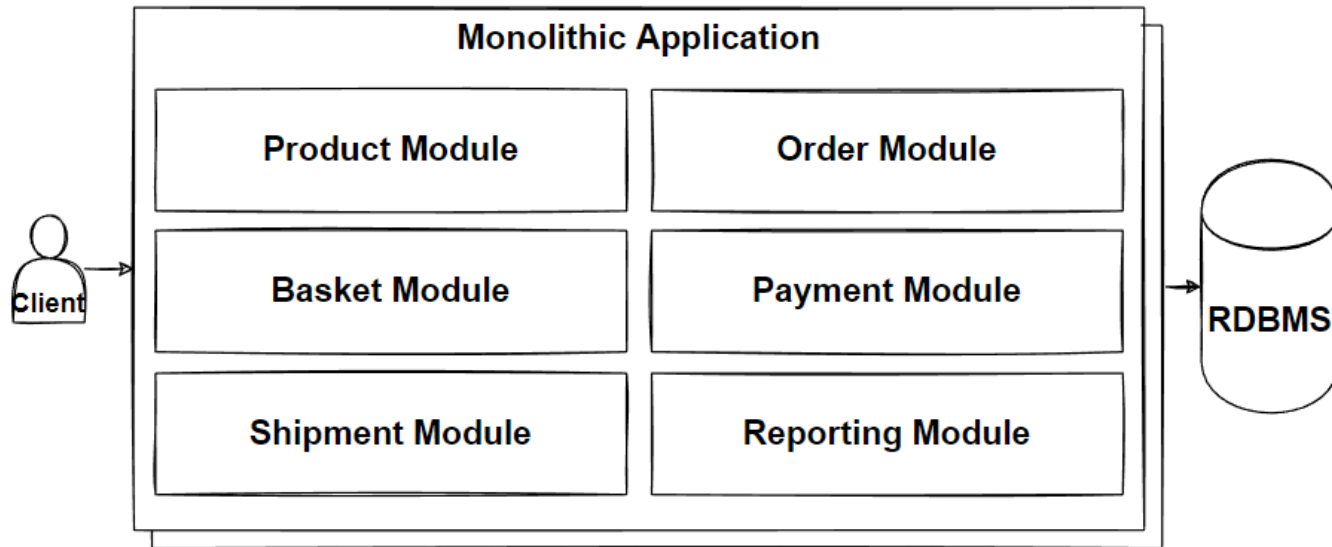
Monolithic application provides fast and easy deployment of the whole system. Microservices provides zero-downtime deployment and CI/CD automation.

- **Development team**

If your team doesn't have experience with microservices and container systems, building a microservices-based application will be difficult.

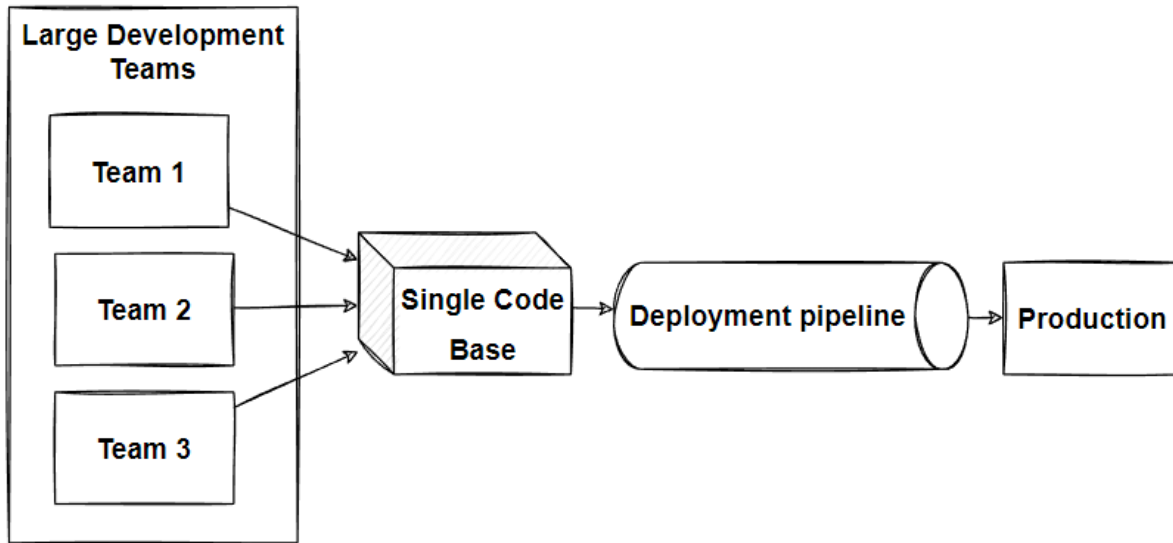


Architecture Comparison

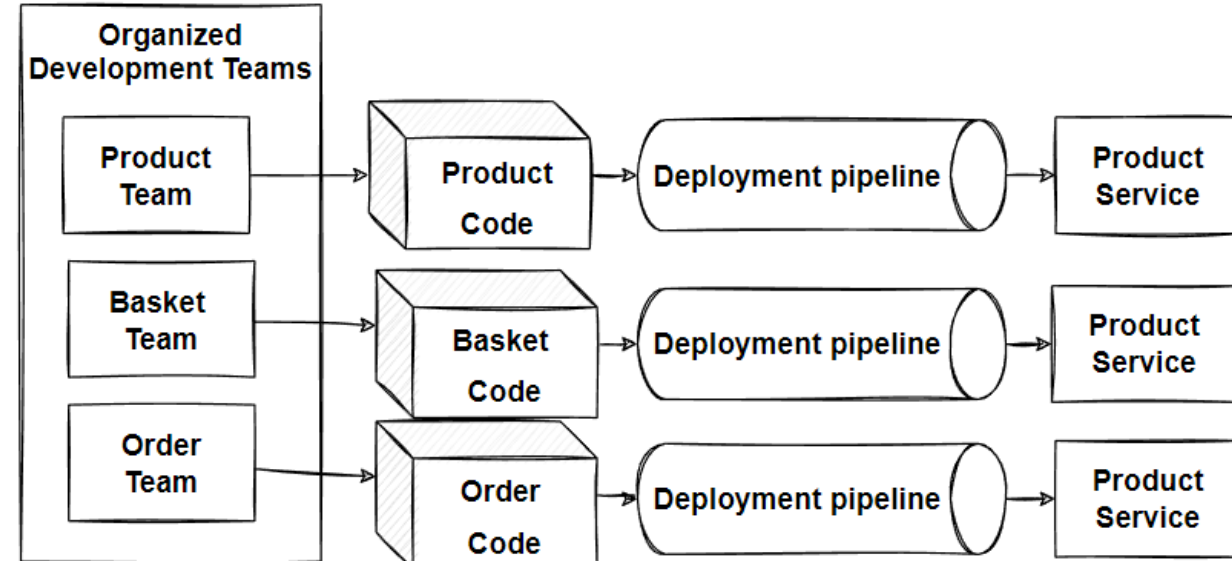


Deployment Comparison

Monolithic

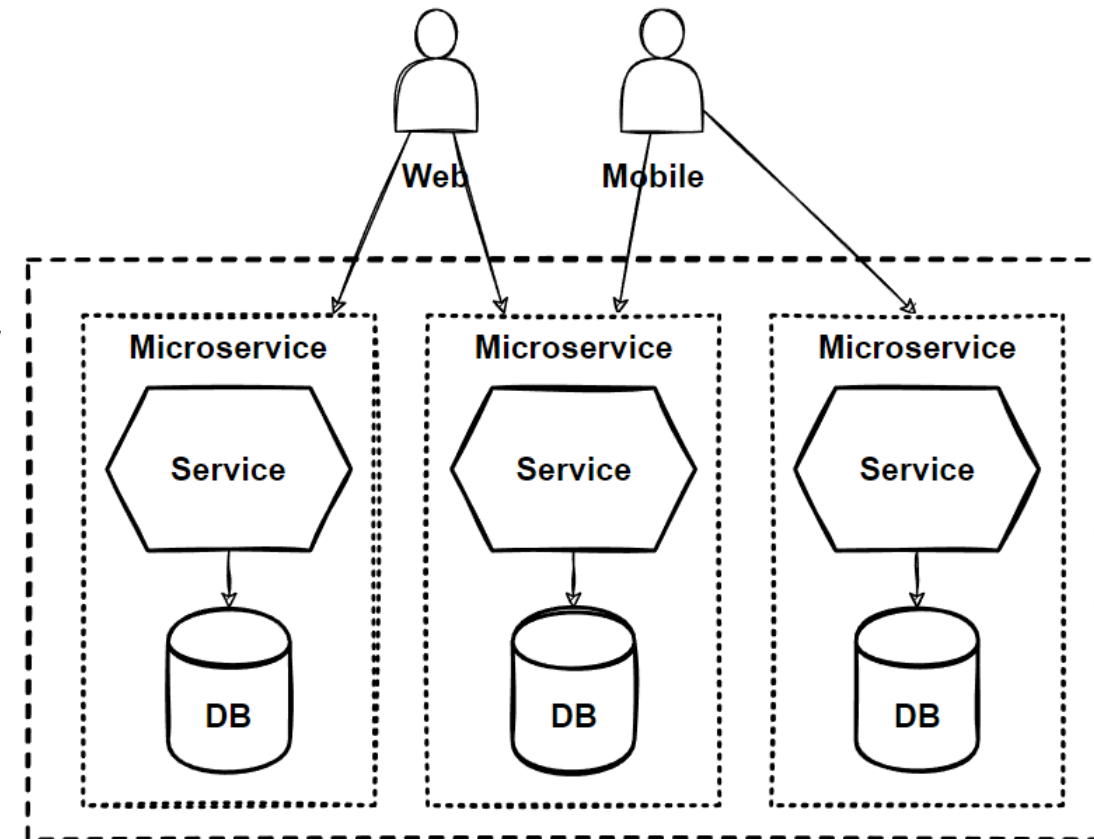


Microservices



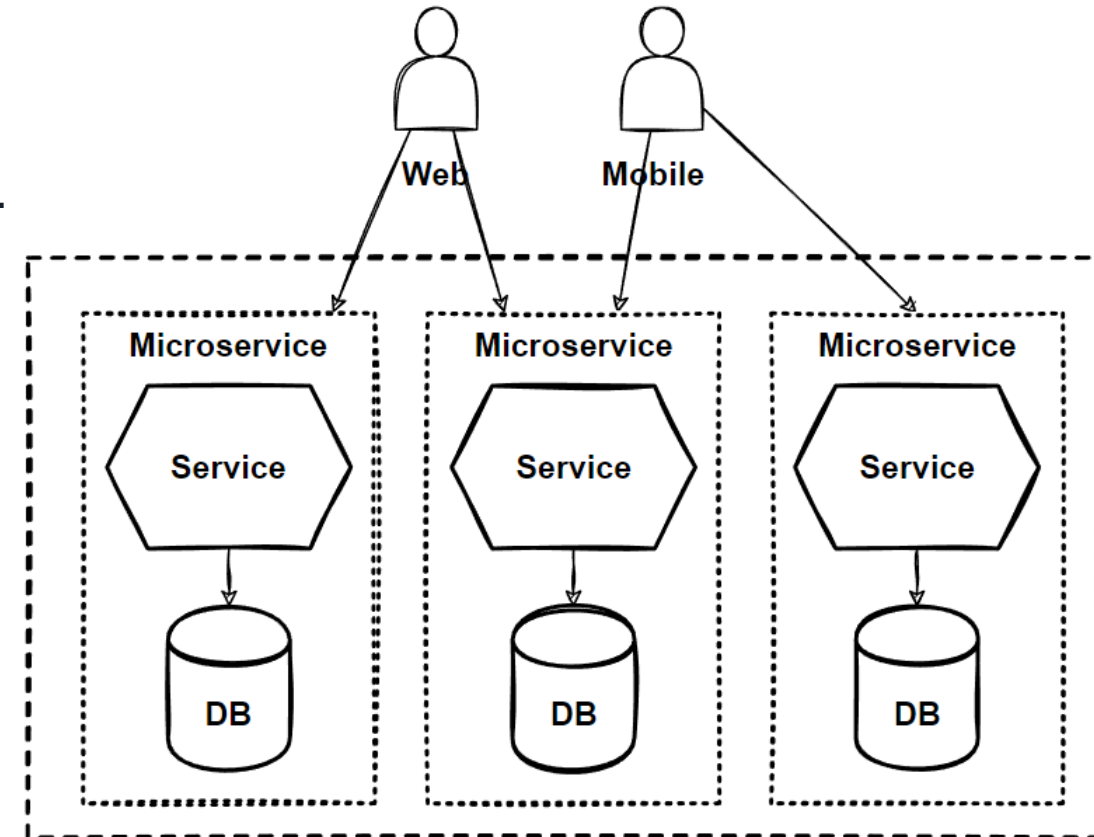
The Database-per-Service Pattern

- Core characteristic of the microservices architecture is the **loose coupling of services**. every service should have its **own databases**, it can be **polyglot persistence** among to microservices.
- E-commerce application. We will have **Product - Ordering** and **SC** microservices that **each services data in their own databases**. Any changes to one database **don't impact other microservices**.
- The **service's database can't be accessed** directly by other microservices. Each service's persistent data can only be accessed via **Rest APIs**.



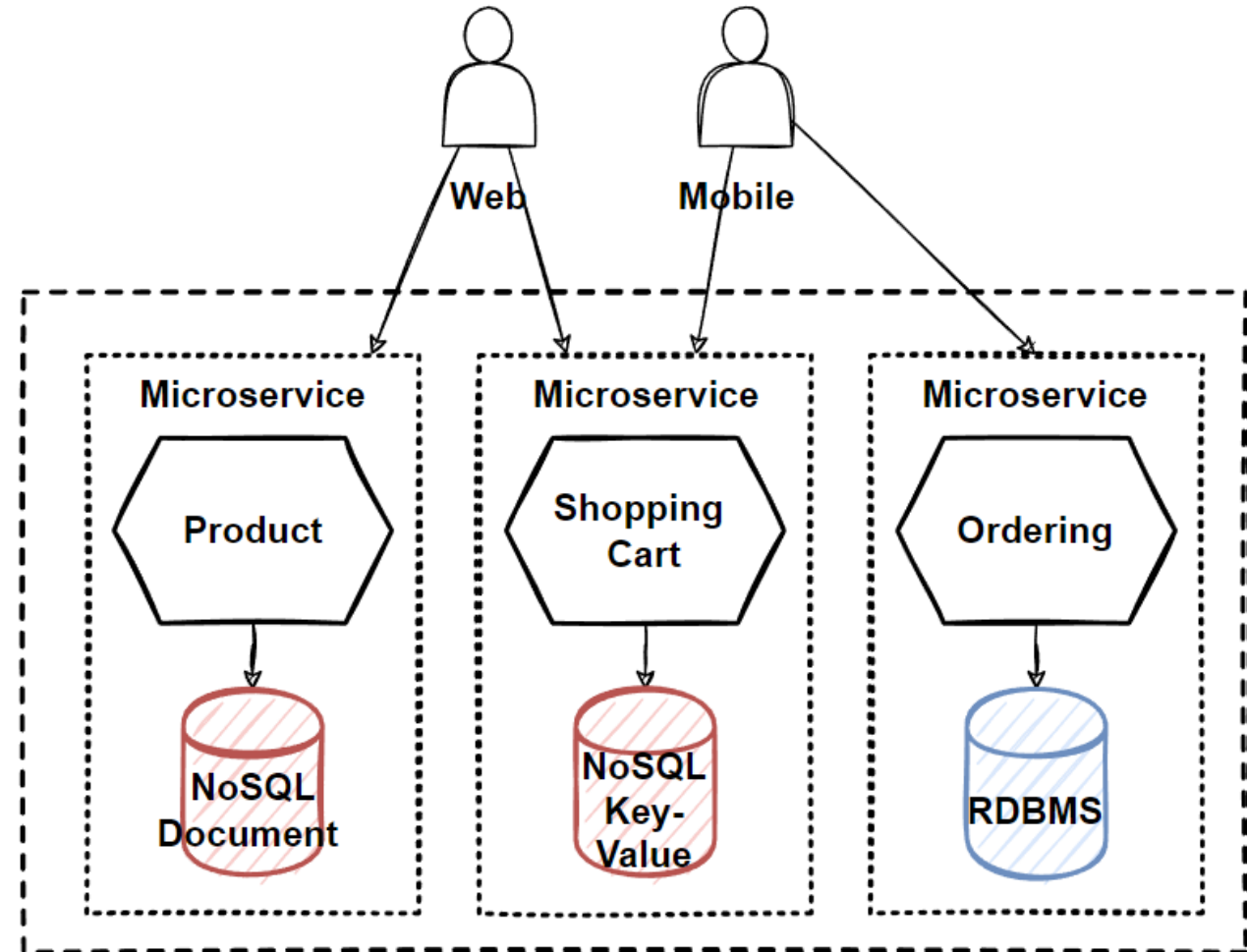
Benefits of the Database-per-Service Pattern with Polygot Persistence

- **Data schema changes** made easy without impacting other microservices.
- Each **database** can **scale independently**.
- Microservices domain data is **encapsulated** within the service.
- If one of the database server is down, this will **not affect to other services**.
- **Polyglot data persistence** gives ability to select the **best optimized storage** needs per microservices.



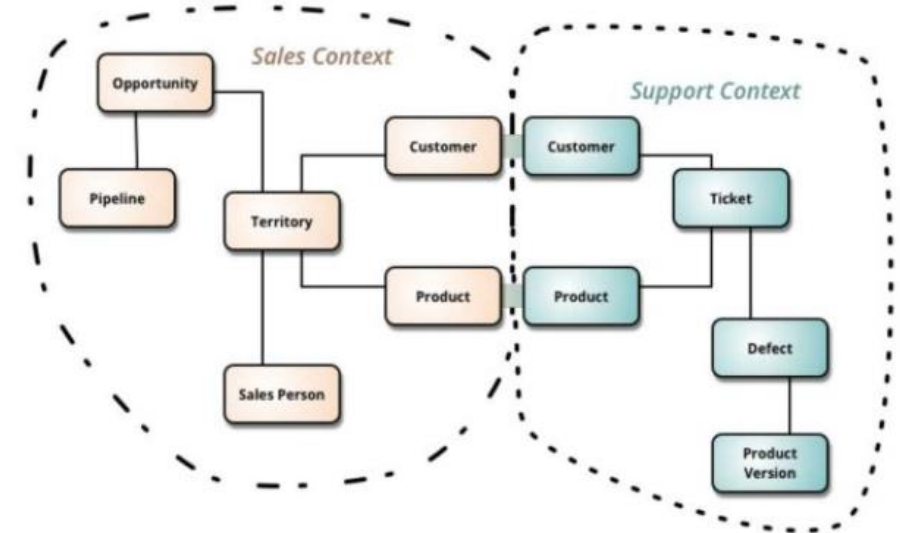
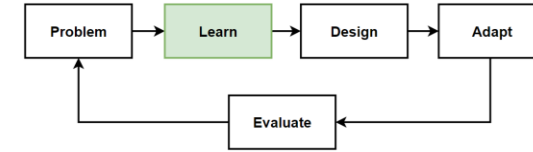
E-Commerce with Database-per-Service Pattern and Polygot Persistence

- Product service using **NoSQL document database** for storing catalog related data.
- Shopping cart service using a **distributed cache** that supports its simple, **key-value data store**.
- Ordering service using a **relational database** to handle the rich relational structure.
- **NoSQL databases** able to massive scale and high availability, and also schemaless structure give flexibility.



Identify Bounded Context Boundaries for Each Microservices

- How we can identify **Bounded Context** ?
- To identify bounded contexts use **DDD**.
- In **DDD**, use the **Context Mapping pattern** for identification of bounded contexts.
- **With Context Mapping Pattern**, we can identify the whole bounded contexts in the application and with their **logical boundaries**.
- The solution is using **Context Mapping**.
- **The Context Map** is a way to define **logical boundaries** between domains.

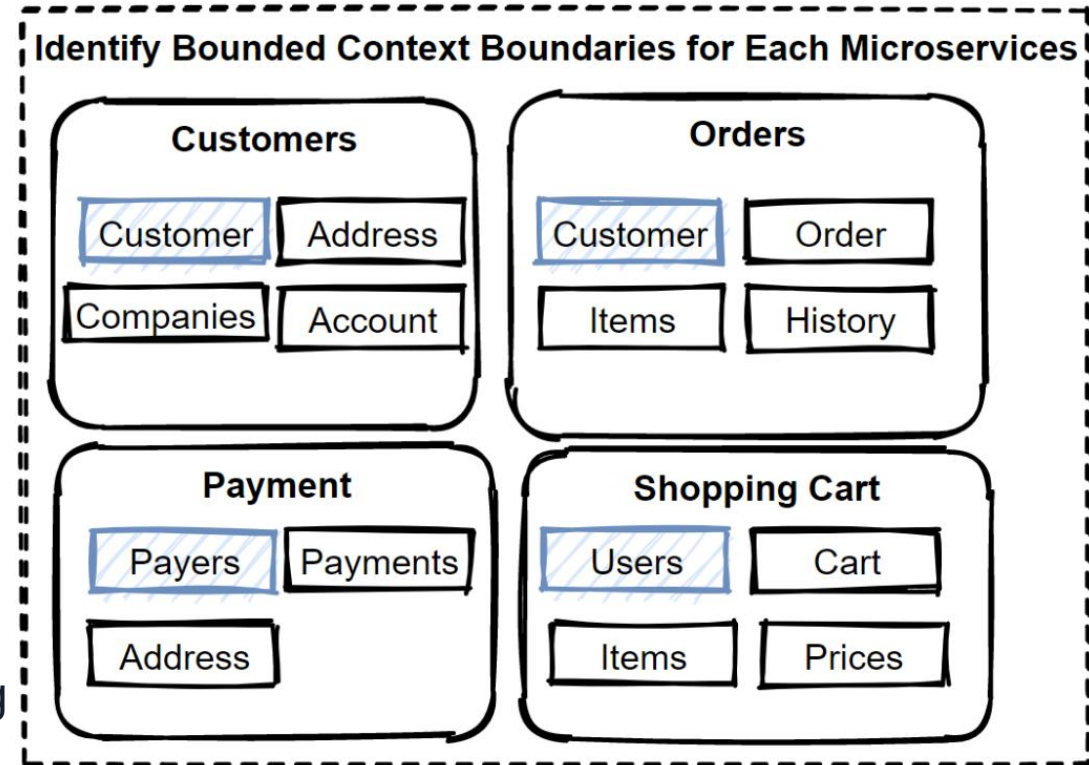
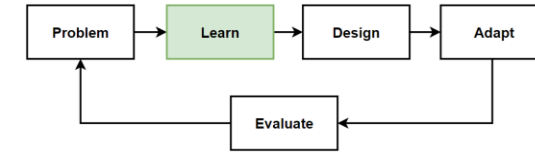


DDD deals with large models by dividing them into different Bounded Contexts and being explicit about their interrelationships.

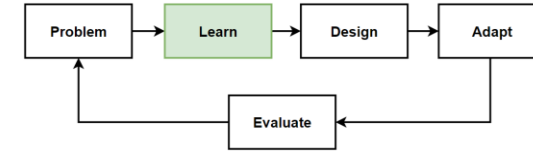
<http://martinfowler.com/bliki/BoundedContext.html>

Identify Bounded Context Boundaries for Each Microservices-2

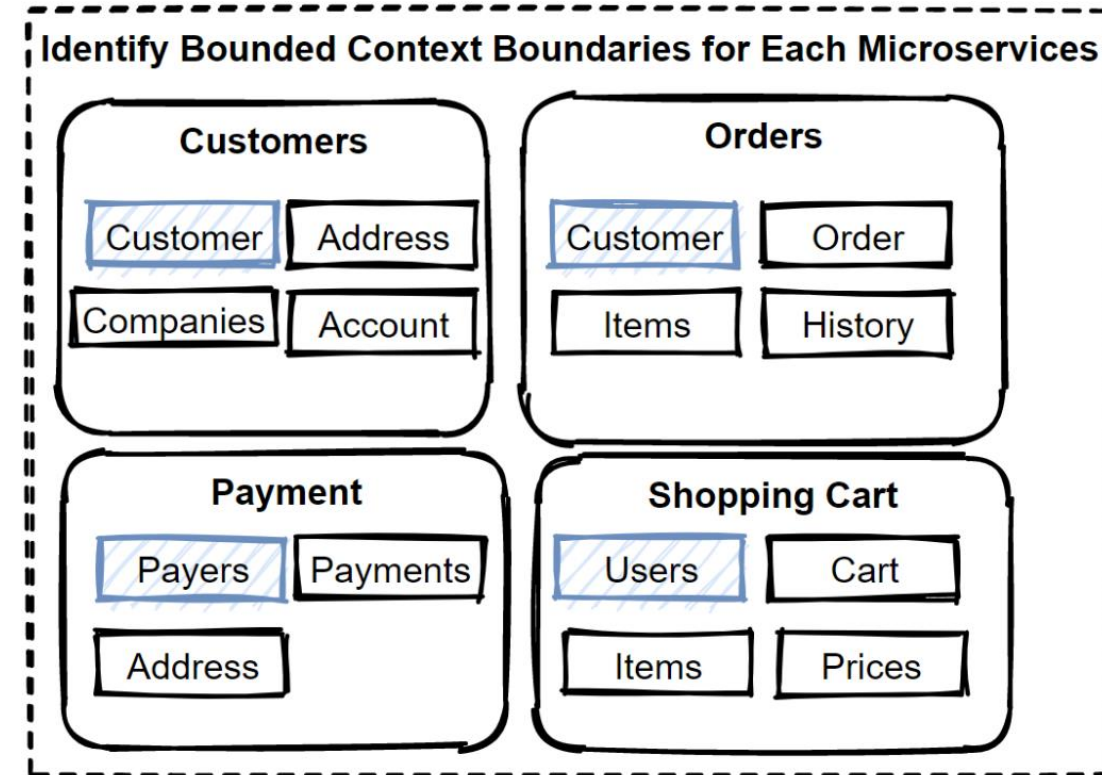
- **Identify** the **Bounded Contexts** by talking to the **domain experts** and using some clues.
- Once defined the Bounded Contexts, iterate design, those are **not immutable**.
- **Reshape** your **Bounded Contexts** by talking to the domain experts and consider **refactoring's** with the changing conditions.
- Its crucial to discuss with **domain experts** to defining **domains** and **sub domains**.
- **Evaluate Bounded Context** with the **domain experts** will help you identify to microservices.
- **Sub domains** inside of the **Bounded Context** are representing same data but naming differently due to domain experts areas.
- Should **discuss several domain experts** for their expertise areas.



A Bounded Context == A Microservice ?



- **No right answer** to this question under all circumstances.
- **Bounded Context** can create **more than one Microservice**.
- Decision to be made based on the **microservice's** need for **scalability** and **independence**.
- Since Bounded Context defines the boundaries of the domain, a **Microservice** determines the **technical** and **organizational boundaries**.
- Similar to Microservices, **Bounded Contexts** are **autonomous** and **responsible** by certain **domain capability**.
- **Context Mapping** and the **Bounded Context pattern** are good approaches for **identifying microservices**.

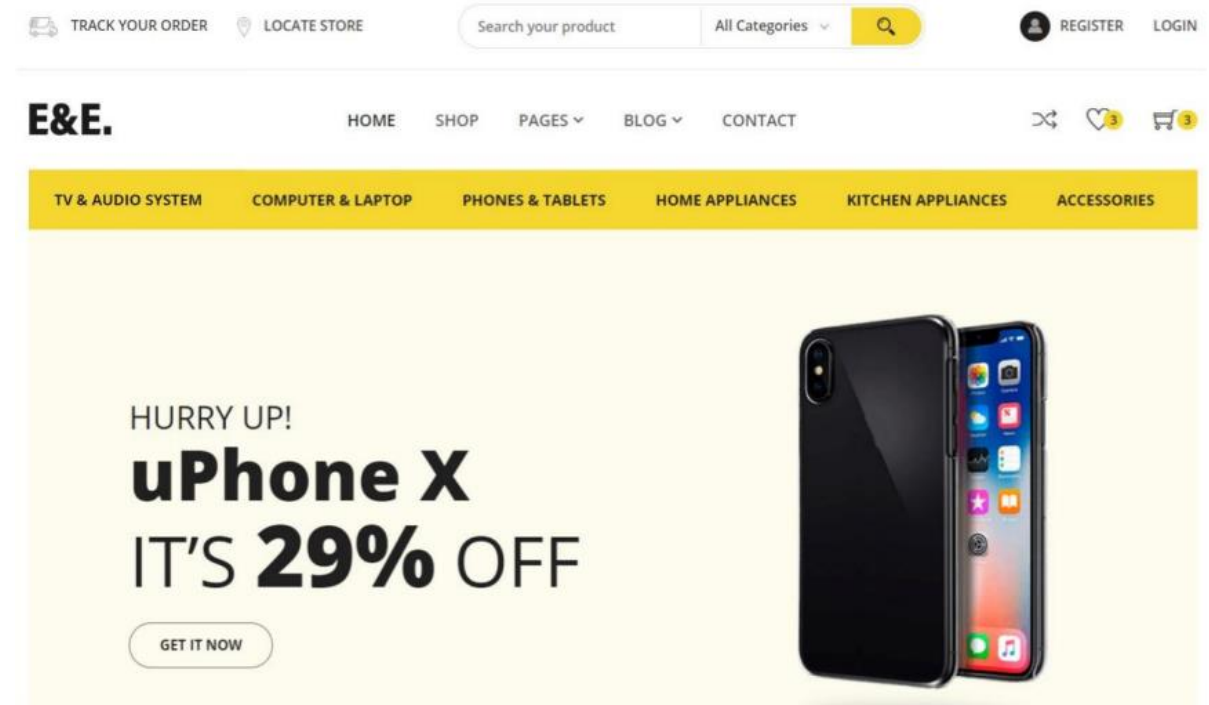


Understand E-Commerce Domain

- Our Domain: E-Commerce
- Understand Domain and Decompose small pieces
 - Use Cases
 - Functional Requirements

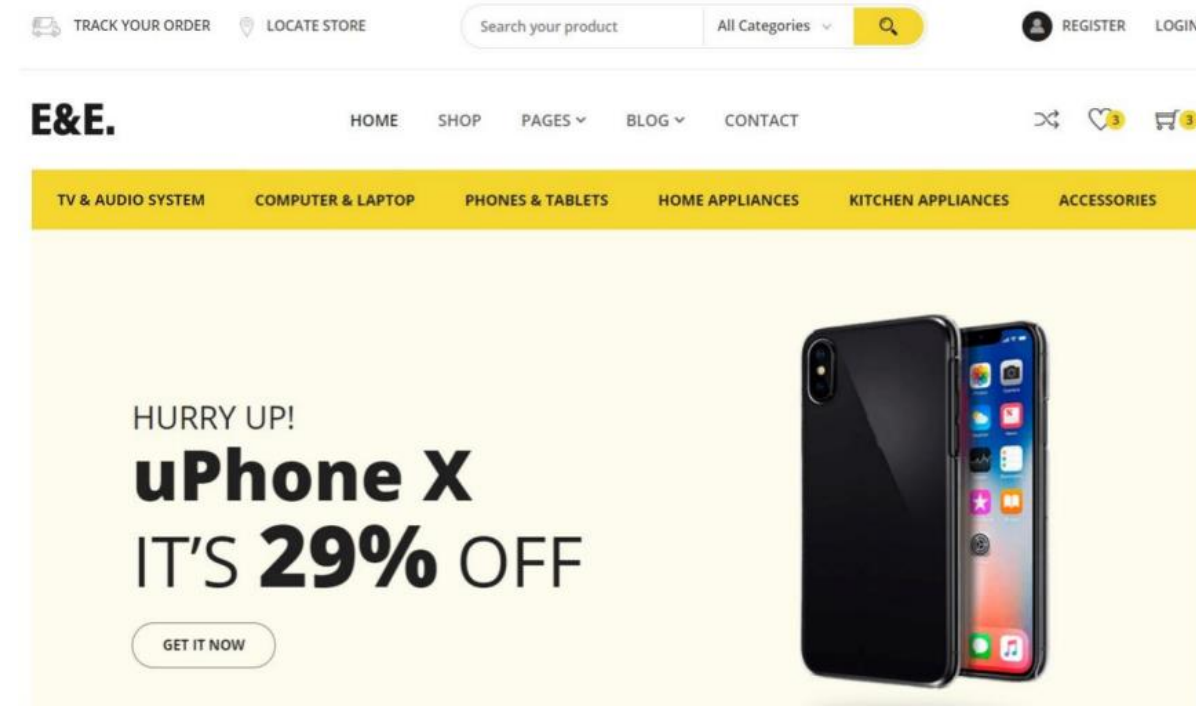
Identify steps;

- Requirements and Modelling
- Identify User Stories
- Identify the Nouns in the user stories
- Identify the Verbs in the user stories



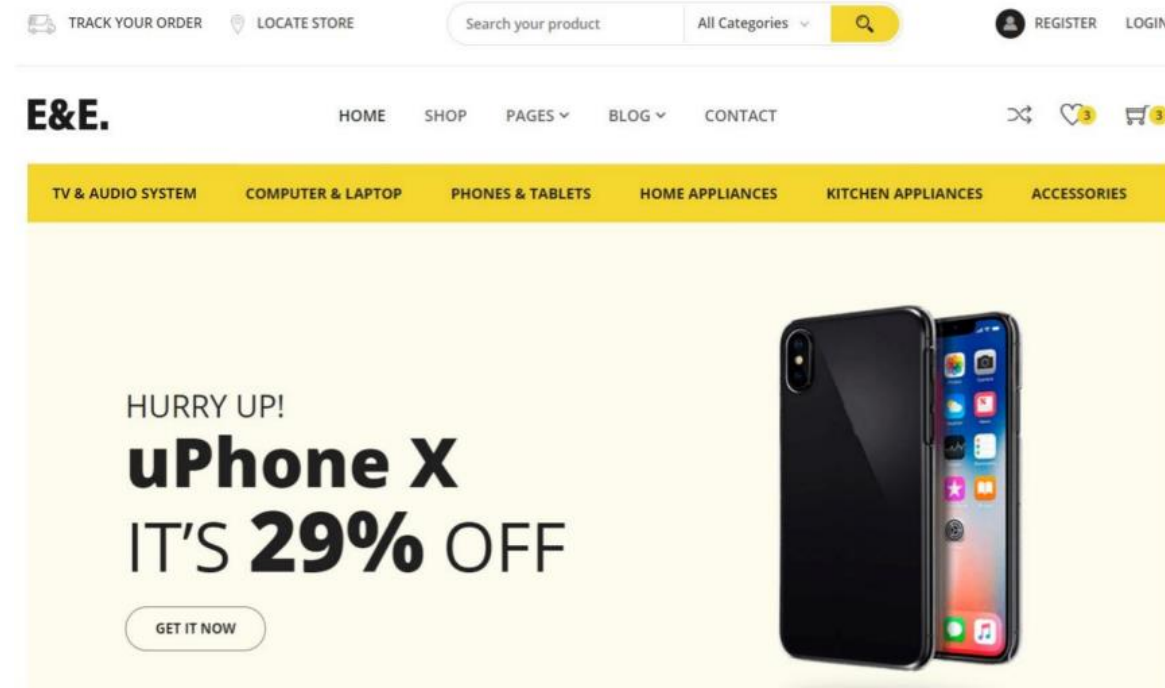
Understand E-Commerce Domain: Functional Requirements

- List products
- Filter products as per brand and categories
- Put products into the shopping cart
- Apply coupon for discounts and see the total cost all for all of the items in shopping cart
- Checkout the shopping cart and create an order
- List my old orders and order items history



Understand E-Commerce Domain: User Stories (Use Cases)

- As a user I want to list products
- As a user I want to filter products as per brand and categories
- As a user I want to put products into the shopping cart so that I can check out quickly later
- As a user I want to apply coupon for discounts and see the total cost all for all of the items that are in my cart
- As a user I want to checkout the shopping cart and create an order
- As a user I want to list my old orders and order items history
- As a user I want to login the system as a user and the system should remember my shopping cart items

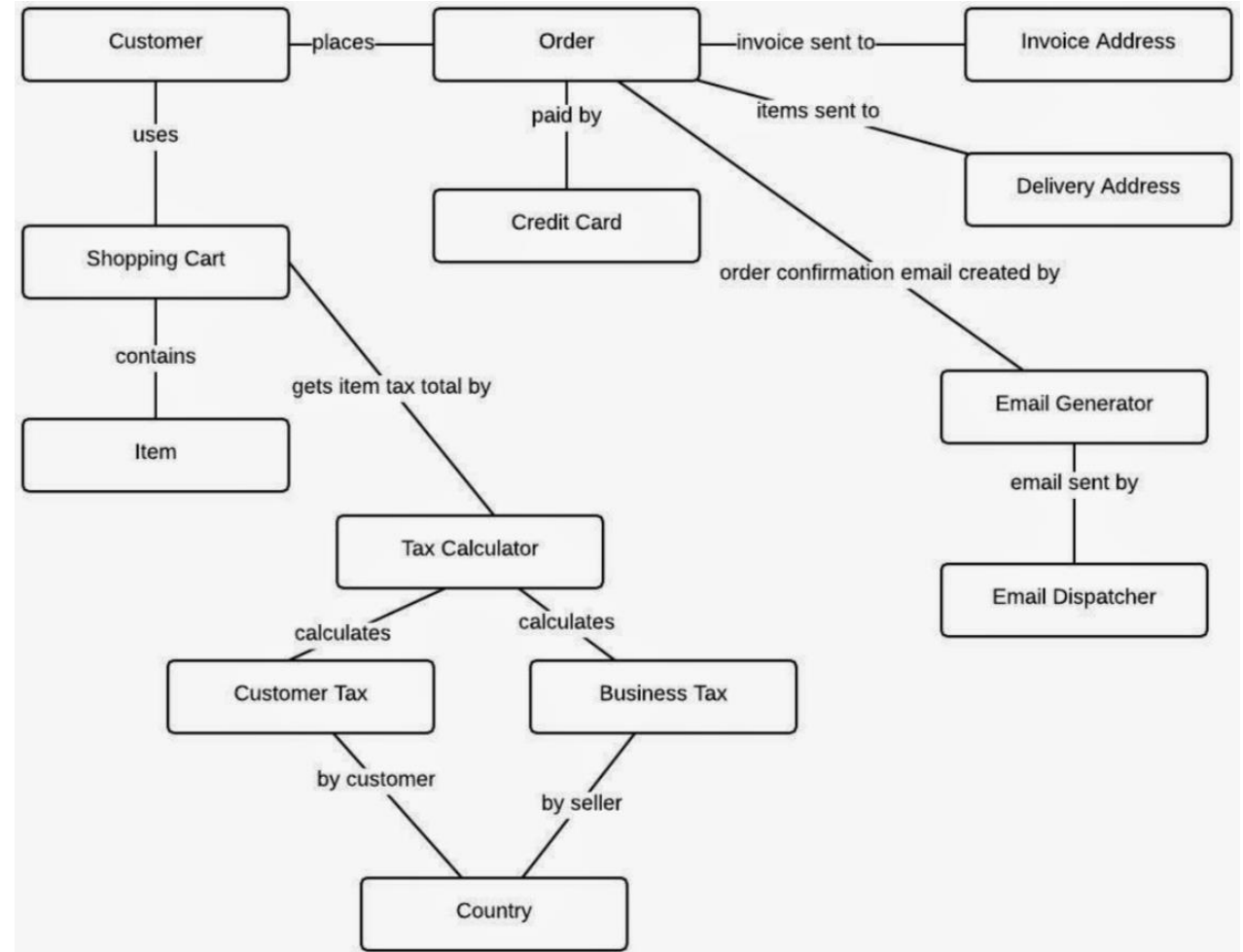


Analysis E-Commerce Domain - Nouns and Verbs

- As a user I want to **list** **products**
- As a user I want to be able to **filter** **products** as per **brand** and **categories**
- As a user I want to **see** the supplier of **product** in the product detail screen with all characteristics of product
- As a user I want to be able to **put** **products** that I want to **purchase** in to the **shopping cart** so I can check out
- As a user I want to **see** the total cost all for all of the **items** that are in my **cart** so that I see if I can afford to buy
- As a user I want to **see** the total cost of each **item** in the **shopping cart** so that I can re-check the price for items
- As a user I want to be able to **specify** the **address** of where all of the products are going to be sent to
- As a user I want to be able to **add** a note to the **delivery address** so that I can provide special instructions
- As a user I want to be able to **specify** my **credit card** information during **check out** so I can pay for the items
- As a user I want system to **tell** me how many items are in **stock** so that I know how many items I can purchase
- As a user I want to **receive** **order** confirmation email with **order** number so that I have proof of purchase
- As a user I want to **list** my old **orders** and **order items history**
- As a user I want to **login** the system as a **user** and the system should remember my shopping cart items

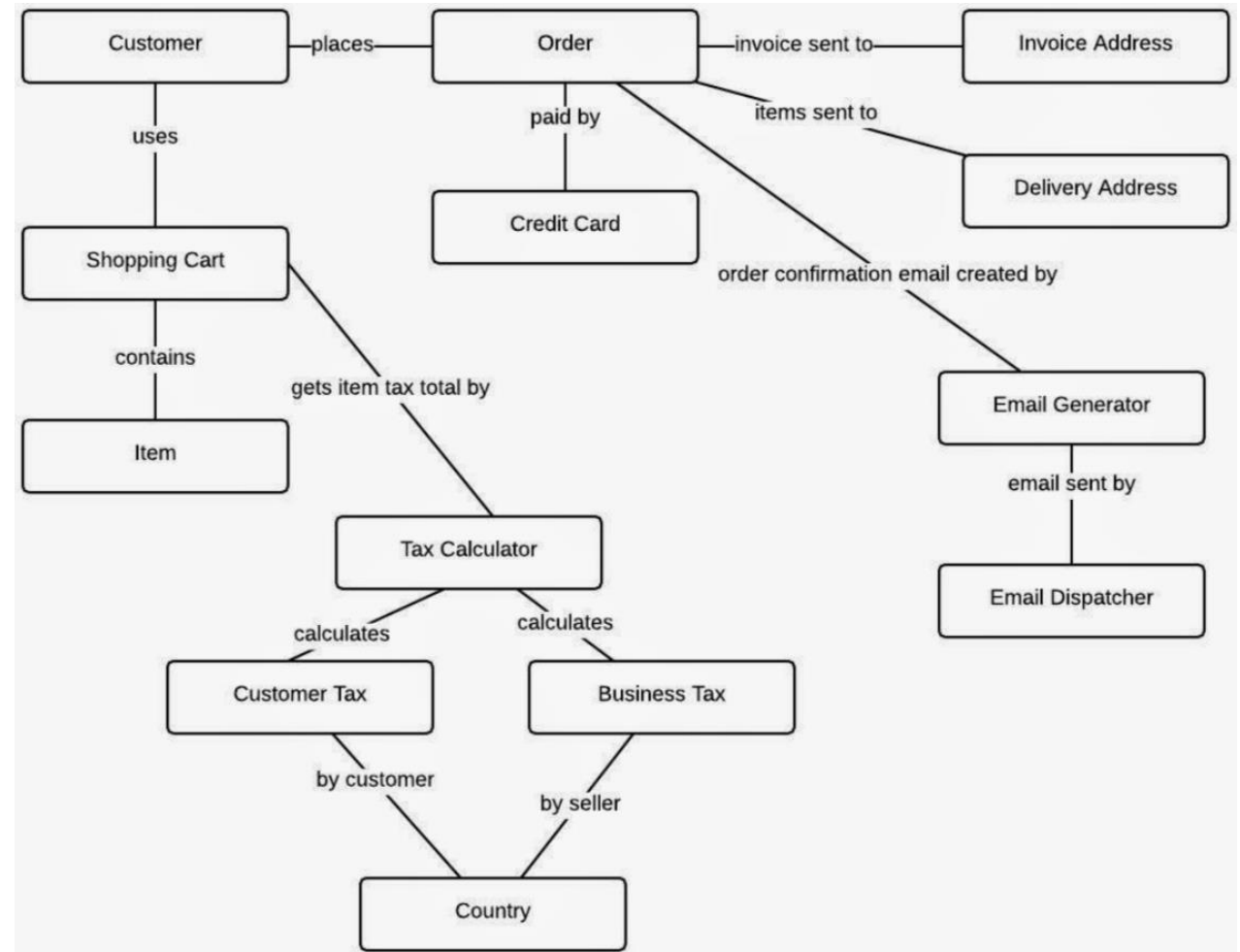
Analysis E-Commerce Domain - Nouns

- Customer
- Order
- Order Details
- Product
- Shopping Cart
- Shopping Cart Items
- Supplier
- User
- Address
- Brand
- Category

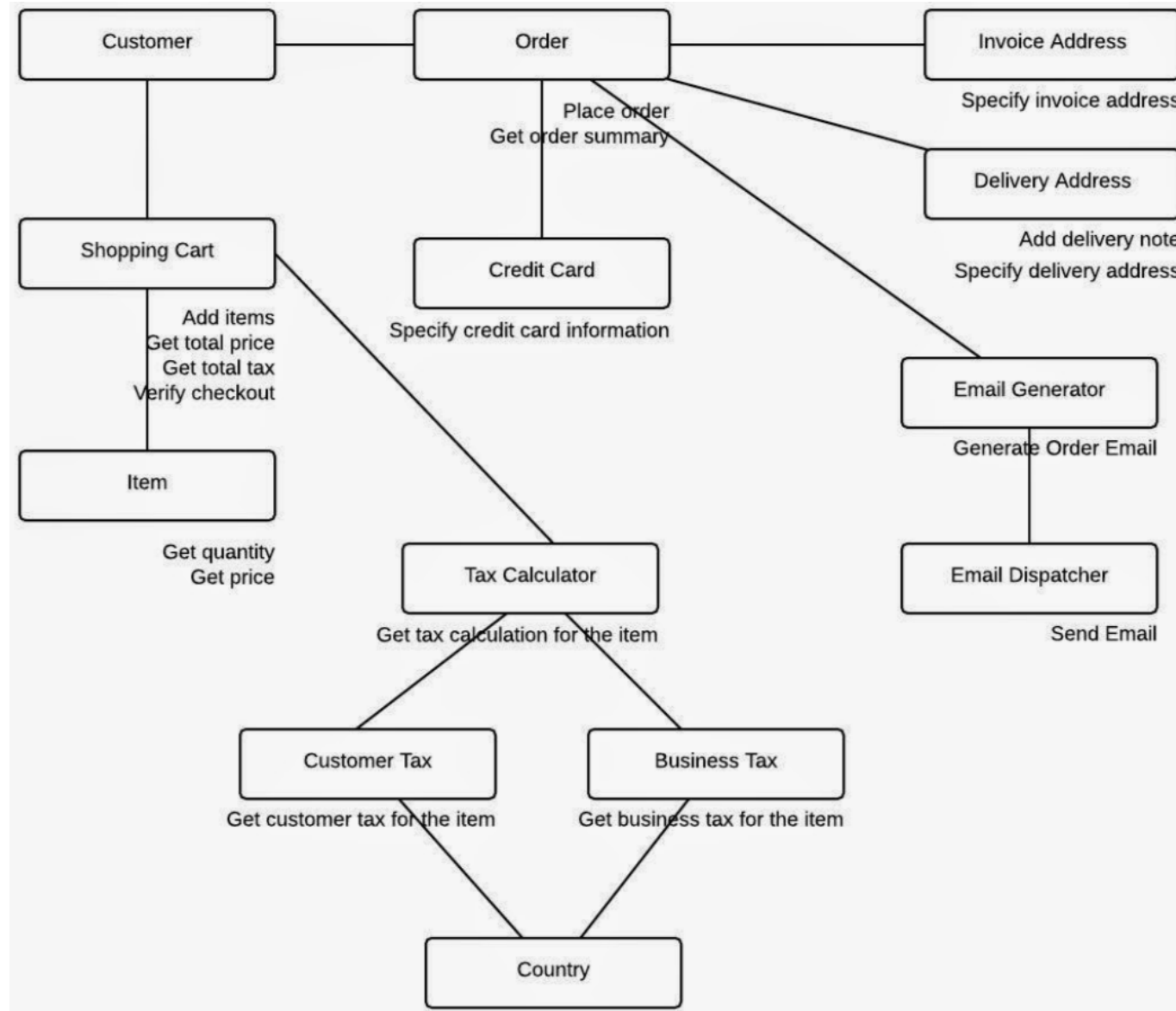


Analysis E-Commerce Domain - Verbs

- List products applying to paging
- Filter products by brand, category and supplier
- See product all information in the details screen
- Put products in to the shopping cart
- See total cost for all of the items
- See total cost for each item
- Checkout order with purchase steps
- Specify delivery address
- Specify delivery note for delivery address
- Specify credit card information
- Pay for the items
- Tell me how many items are in stock
- Receive order confirmation email
- List the order and details history
- Login the system and remember the shopping cart items



Object Responsibility Diagram



Identifying and Decomposing Microservices for E-Commerce Domain

Main Microservices

Users

Product

Customers

Shopping Cart

Discount

Orders

Order Transactional Microservices

Orders

Payment

Inventory

Shipping

Billing

Notification

Intelligence Microservices

Identity

Marketing

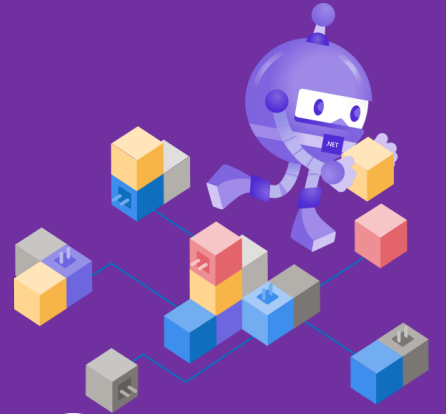
Location

Rating

Recommendation

Why .NET ?

What is new .NET 8 and C# 12 ?



Build anything with a unified platform with .Net

.NET Momentum: Monthly active users 6.1+ Million

.Net 8 Most performant release

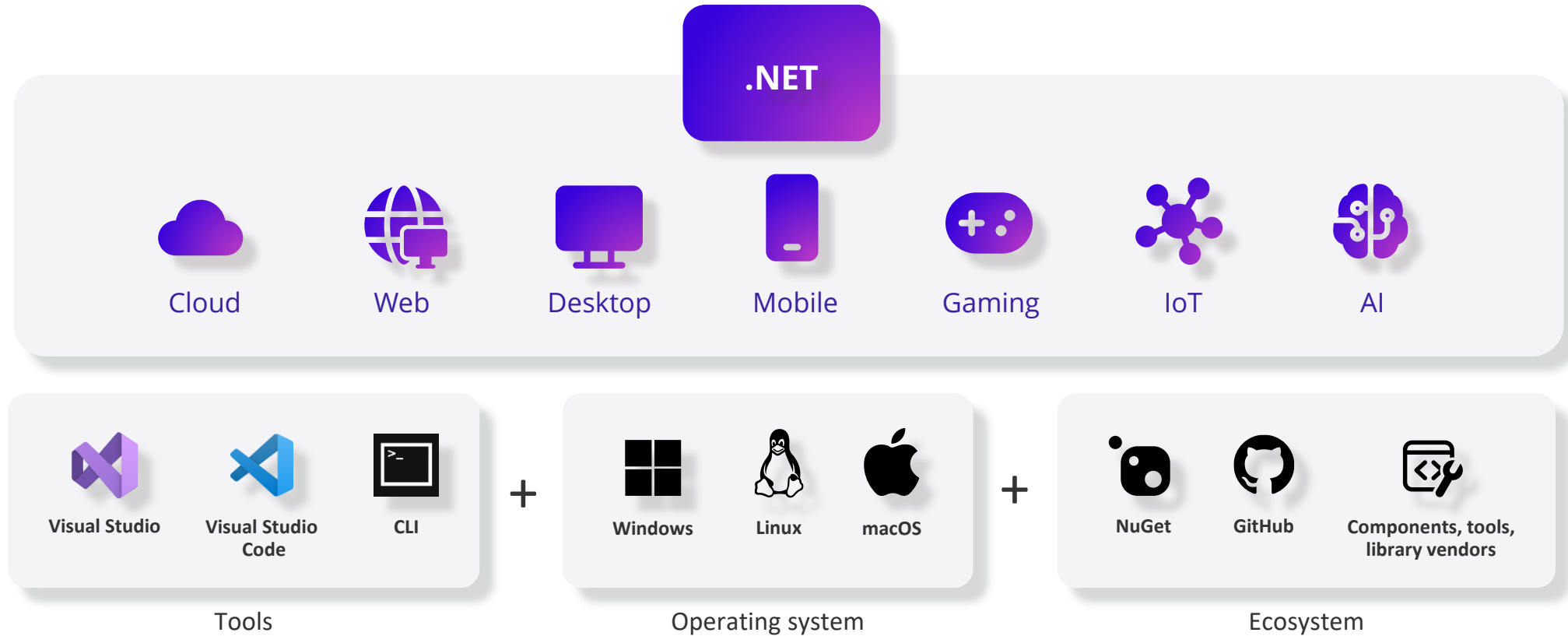
Cloud-Native Development with .Net

C# 12 Primary constructors, Collection expressions, Default lambda

Mehmet Ozkaya



Build Anything, Anywhere



.NET Momentum

6.1+ Million

Monthly active users

53,000+

Community members
have contributed to .NET

>21k

Contributions

>9k

Community members

#1 2023  **stackoverflow**

Most Admired Developer Framework

Top 5



CLOUD NATIVE
COMPUTING FOUNDATION

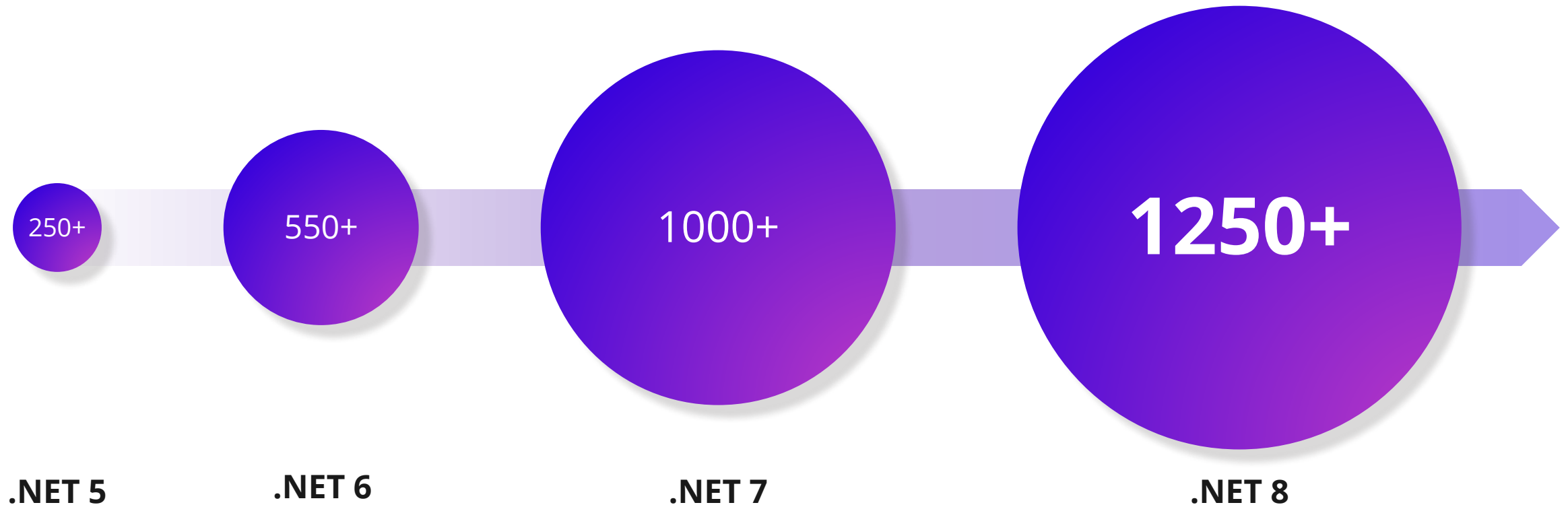
Highest velocity OSS project

Top 5

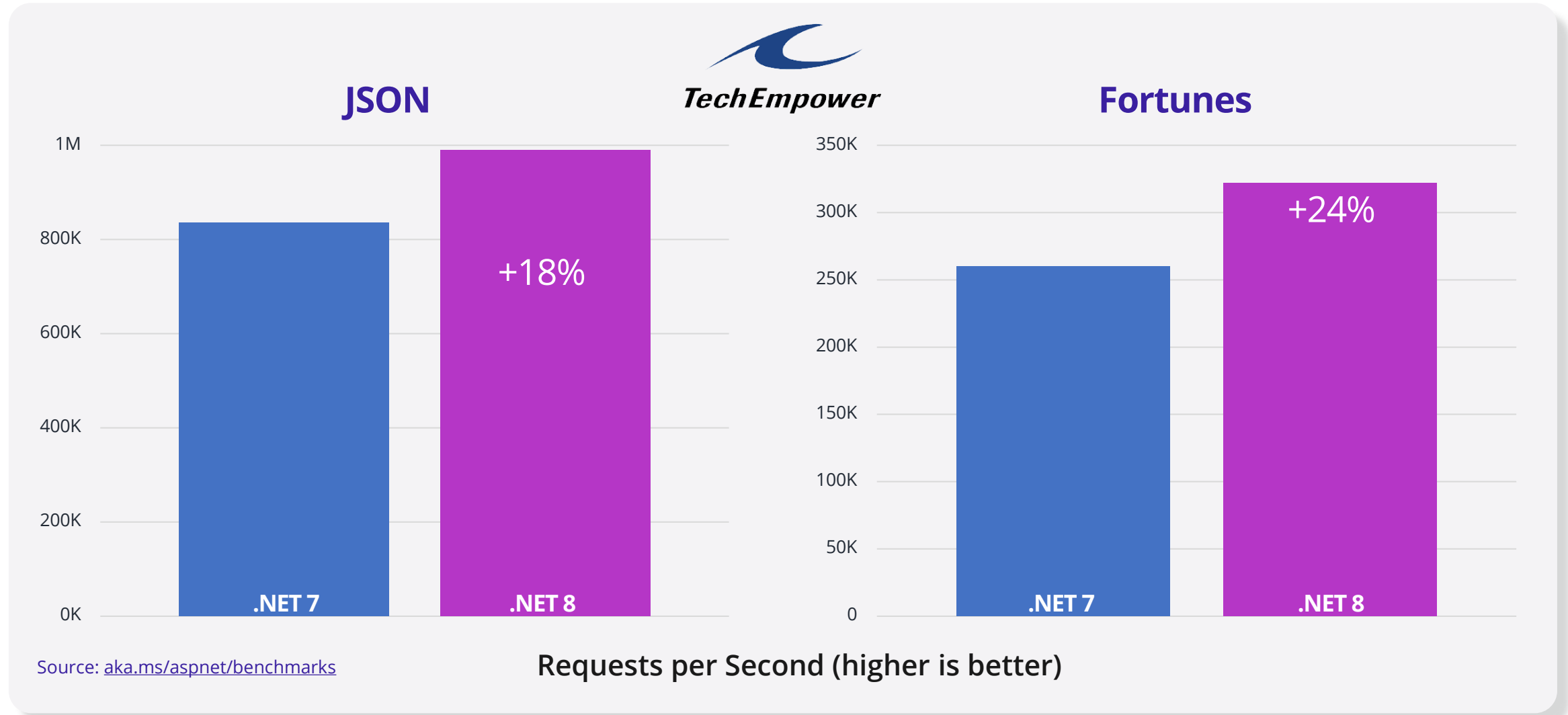


Top Programming Language on GitHub

Most performant release yet



.NET 8 API Performance



.NET is the best for modern apps

Modern Workloads



Web



Client



Artificial
Intelligence



Cloud Native

Developer Productivity



Language



Productivity



Modernization

Fundamentals



Performance



Supply Chain

Cloud-Native Development with .Net 8

Cloud Native



Observability



Resiliency



Scalability



Manageability

High-scale and high-availability services in .NET 8

Cloud Native



Resilience



Health Checks



Testing/Fakes



Observability

Extensions.Resilience

Extensions.Http.Resilience

Extensions.Diagnostics.HealthChecks.Commo

Extensions.Diagnostics.Probes

Extensions.Telemetry

Extensions.Compliance.Redaction

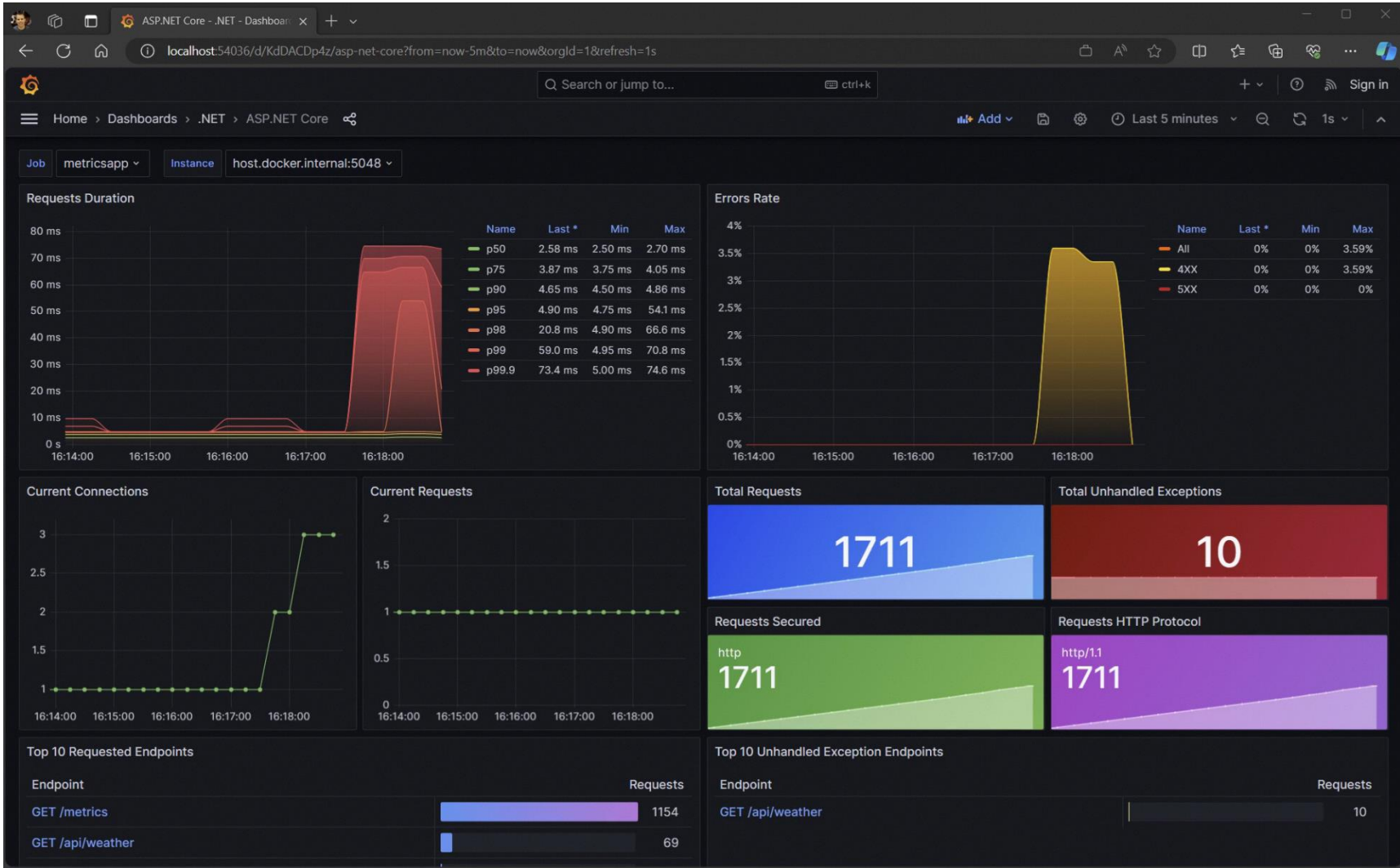
Extensions.Http.Telemetry

AspNetCore.Testing

Hosting.Testing

Extensions.TimeProvider.Testing

Enhanced Open Telemetry Support



Cloud-Native in .NET 8



Observability

Built in metrics with dimensions

DI integration for metrics

Better Logging support
(faster, can object serialization)

Enrichment

Redaction

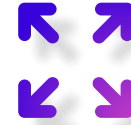
Testing fakes for Logging & Metrics



Resiliency

New Polly based
resiliency packages

SignalR Stateful Reconnect



Scalability

AOT
(increased density)

Performance

Chiseled Ubuntu



Manageability

Certificate auto-rotation
support in Kestrel

.NET ❤ Containers



Hardened

Non-Root Base Images

USER "app"

Default port - 8080

Mariner distroless



Smaller

AOT base images

"Composite" base images

"extra" base images

Distroless / Chiseled



More Productive

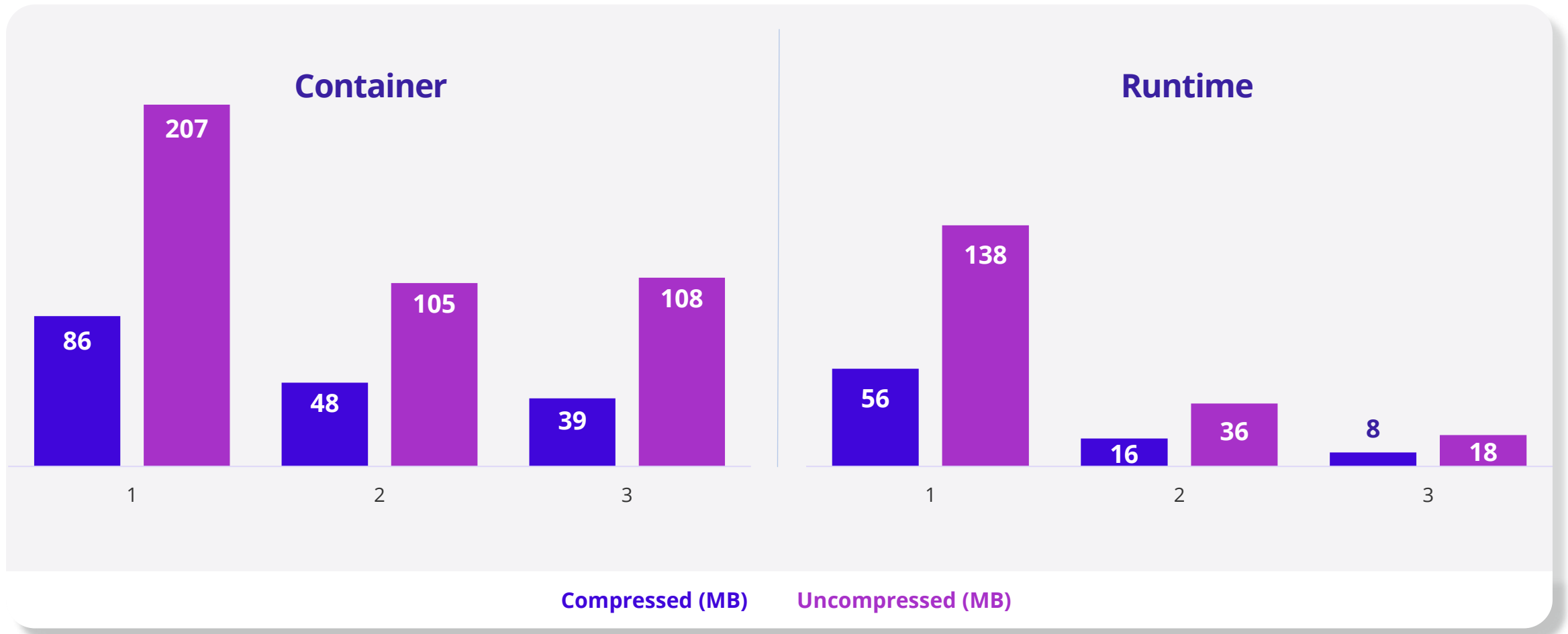
Publish with .NET SDK

Cross compilation

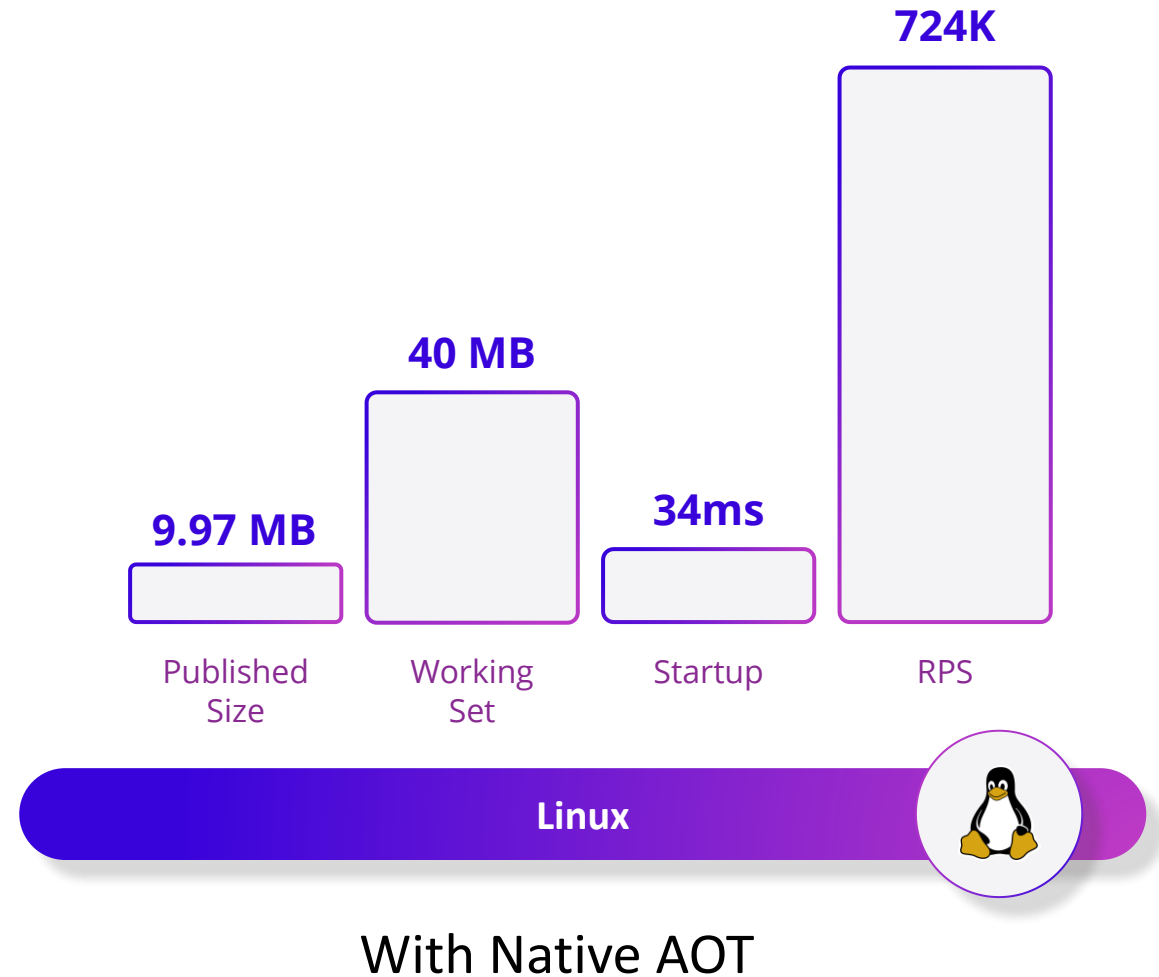
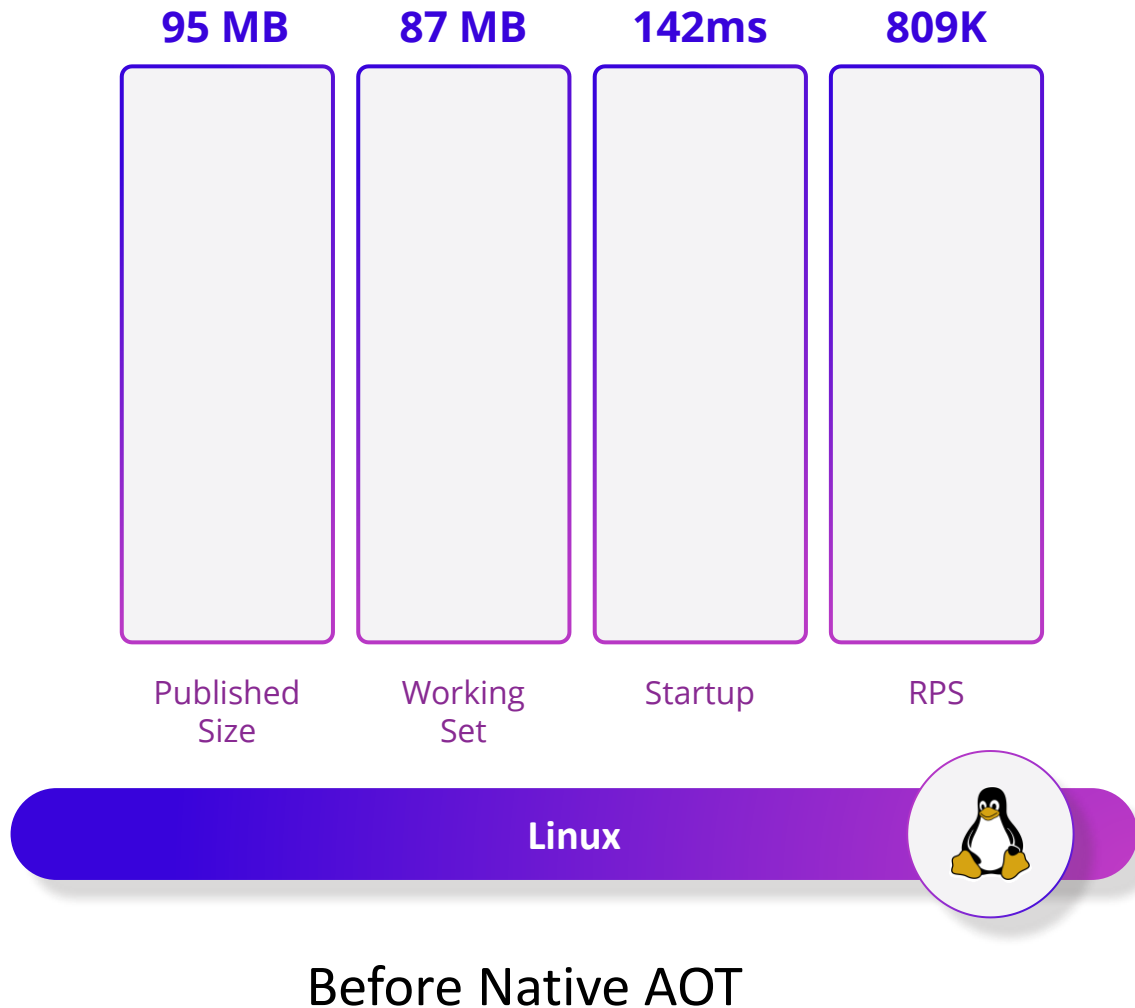
Non-root by default

Supports all Azure auth

Container Size Improvements



Native AOT - Container Size Improvements



What is new .NET 8 ?

.NET Aspire

- Cloud-ready stack designed for building observable, production-ready distributed apps.
- NuGet packages targeting specific cloud-native concerns, now available in preview.

Core .NET Libraries

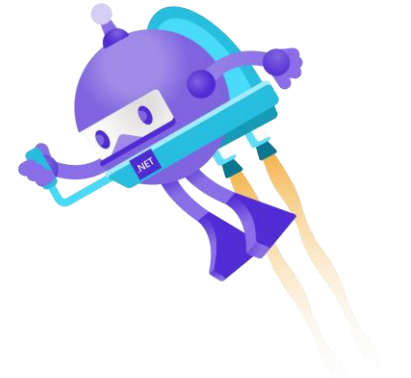
- Serialization enhancements, time abstraction, UTF8 improvements, methods for working with randomness, and performance-focused types like System.Numerics and System.Runtime.Intrinsics.

Metrics

- Attach key-value pair tags to Meter and Instrument objects, allowing for more nuanced differentiation in aggregated metric measurements.

Networking

- Support for HTTPS proxy, ensuring encrypted communication even in proxy scenarios, thus enhancing privacy and security.



What's new in .NET 8?



What is new .NET 8 ? – Part 2

Extension Libraries

- Options validation, LoggerMessageAttribute constructors, extended metrics, hosted lifecycle services, keyed DI services.

Garbage Collection

- On-the-fly adjustment of memory limits, a crucial feature for cloud-service scenarios where dynamic scaling is mandatory.

Reflection Improvements

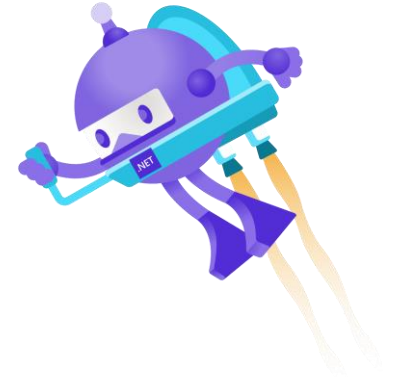
- Enhanced for better performance and more efficient memory usage. Function pointers also added reflection capabilities.

Native AOT Support

- Efficient compilation and execution, particularly beneficial for cloud-native and high-performance scenarios.

.NET SDK

- More robust and feature-rich, aligning with the evolving needs of modern .NET development. Enhanced dotnet publish and dotnet pack commands.



What's new in .NET 8?



What is new C# 12 ?

Primary Constructors

- Primary constructors have been extended beyond record types. Parameters are now in scope for the entire class body.
- Should assigned, explicitly declared constructors must call the primary constructor using this() syntax.
 - `public class Person(string name, int age)`
 - `{`
 - `// Name and Age are in scope for the entire class body`
 - `public string Name => name;`
 - `public int Age => age;`
 - `}`

Collection Expressions

- More concise syntax to create common collection values. Simplifies the way collections are initialized and manipulated.
 - `var numbers = new List<int> { 1, 2, 3, ..otherNumbers };`
 - `var numbers = [1, 2, 3, .. otherNumbers];`



What is new C# 12 ? – Part 2

Inline Arrays

- Enhance performance by enabling developers to create fixed-size arrays in struct types.
- Useful for optimizing memory layout and improving runtime performance.
 - `public struct Buffer`
 - `{`
 - `public Span<int> InlineArray => MemoryMarshal.CreateSpan(ref _array[0], 10);`
 - `private int[] _array;`
 - `}`

Optional Parameters in Lambda Expressions

- Default values for parameters in lambda expressions. This mirrors the syntax and rules for adding default values in methods, making lambda expressions more flexible.
 - `Func<int, int, int> add = (x, y = 1) => x + y;`
 - `Console.WriteLine(add(5)); // Outputs 6`



What is new C# 12 ? – Part 3

ref readonly parameters

- Enhances the way readonly references are passed in C#.
- Optimizing memory usage and performance in scenarios involving large data structures.
 - `public void ProcessLargeData(in LargeData data)`
 - `{`
 - `// Processing data without the risk of modifications`
 - `}`

Alias Any Type

- Using alias directive to include any type, not just named types.
- Creation of semantic aliases for complex types like tuples, arrays, and pointer types.
 - `using Coordinate = System.ValueTuple<int, int>;`
 - `Coordinate location = (10, 20);`



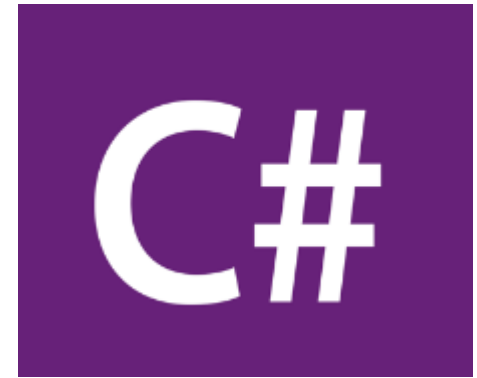
C# Top-level statements, Global usings, Pattern Matching

Top-Level Statements

- Simplify the entry point of your applications. Instead of wrapping your main logic in a Main method, you can directly write the code at the top level of your file.
 - `using System;`
 - `Console.WriteLine("Hello, World!");`

Global Usings

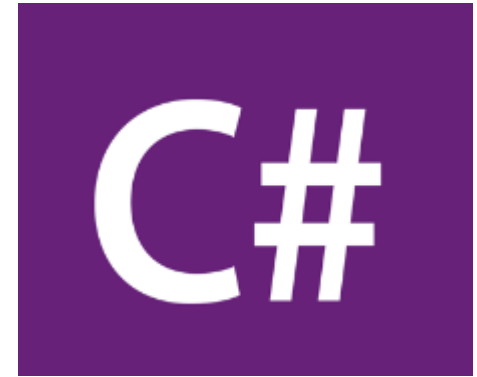
- Make namespaces available across your entire project. Instead of repeating using statements in every file, you declare them globally in one place.
 - `global using System;`
 - `global using System.Collections.Generic;`



C# Pattern Matching

Pattern Matching

- More expressive syntax for checking and deconstructing values in your code.
 - `public class Person`
 - `{`
 - `public string Name { get; set; }`
 - `public string Title { get; set; }`
 - `}`
 - `Person person = new Person { Name = "John", Title = "Manager" };`
 - `if (person is { Title: "Manager" })`
 - `{`
 - `Console.WriteLine($"{person.Name} is a manager.");`
 - `}`

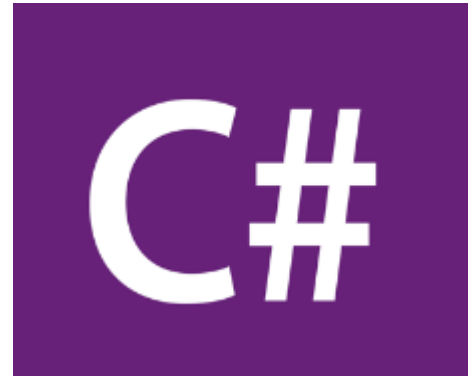


C# Pattern Matching with Switch Expressions

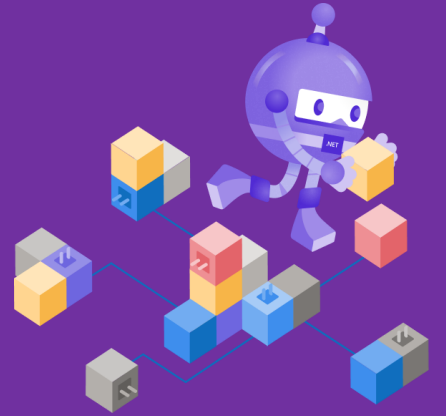
Pattern Matching

- Pattern Matching - using switch expressions to compare discrete values.

- `public State PerformOperation(Operation command) =>`
- `command switch`
- `{`
- `Operation.SystemTest => RunDiagnostics(),`
- `Operation.Start => StartSystem(),`
- `Operation.Stop => StopSystem(),`
- `Operation.Reset => ResetToReady(),`
- `_ => throw new ArgumentException("Invalid enum value for command",`
`nameof(command)),`
- `};`



ASP.NET 8 for Microservice Development



Build Web APIs and Web UI using ASP.NET Core

ASP.NET 8 Minimal APIs

Built-in dependency injection

Create ASP.NET Core Web API for Single Microservice

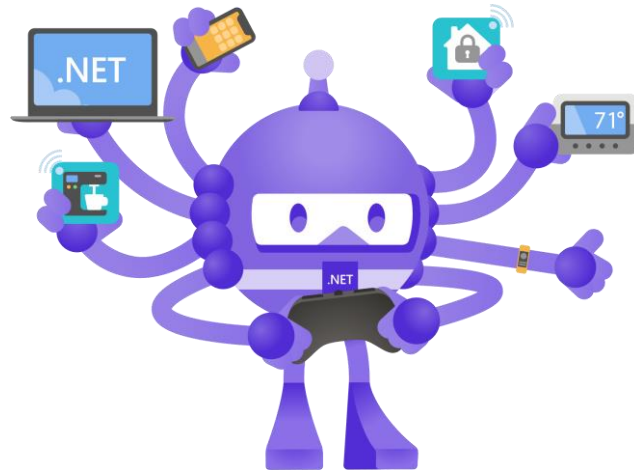
Containerize and Orchestrate services with Docker and Docker-compose

Mehmet Ozkaya



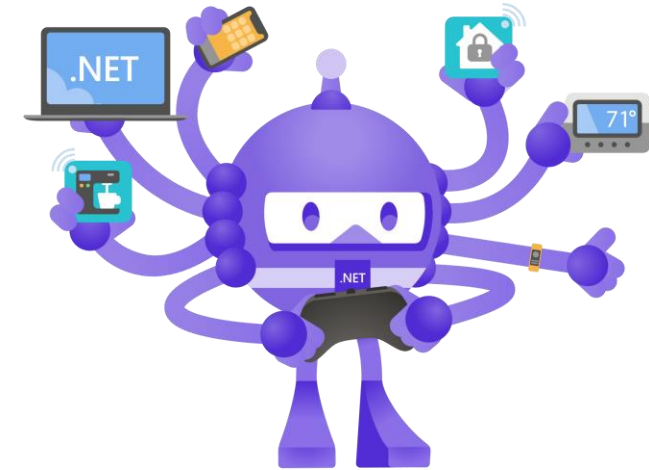
What is ASP.NET ?

- **ASP.NET Core** is a **cross-platform, high-performance, open-source** framework for building modern, cloud-enabled, Internet-connected apps.
- Build **web apps** and **services**, **Internet of Things (IoT)** apps, and **mobile backends**.
- **Deploy** to the **cloud** or **on-premises**.
- Run on **.NET Core**.



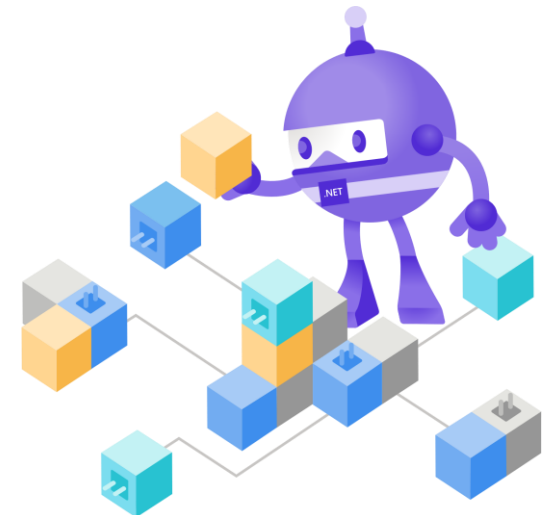
Why choose ASP.NET Core ?

- A **unified** story for building **Web UI** and **Web APIs**. Architected for testability.
- **Razor Pages** makes coding page-focused scenarios easier and more productive.
- **Blazor** lets you use C# in the browser alongside JavaScript.
- Ability to **develop** and **run** on **Windows**, **macOS**, and **Linux**.
- **Open-source** and **community-focused**.
- **Integration** of modern, **client-side frameworks** and development workflows.
- A **cloud-ready**, **environment-based configuration** system.
- **Built-in dependency injection**.
- A **lightweight**, **high-performance**, and **modular** HTTP request pipeline.



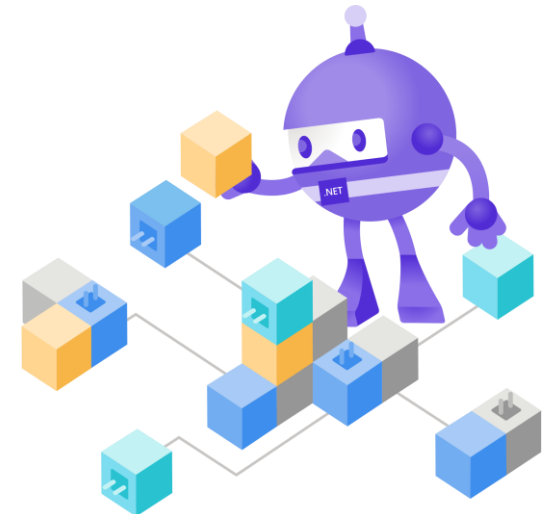
ASP.NET 8 for Microservices

- Supports building **high-performance** Web APIs and services.
- Creating **Web APIs** using ASP.NET Core, leveraging its power to build both service endpoints and user interfaces.
- Offers **best performance, scalability, and flexibility**.
- **Lightweight**, supports **cross-platform development**, and provides a vast ecosystem of libraries and tools.
- Optimized for **cloud deployment** and supports **modern architecture patterns** like CQRS and Event-driven design.



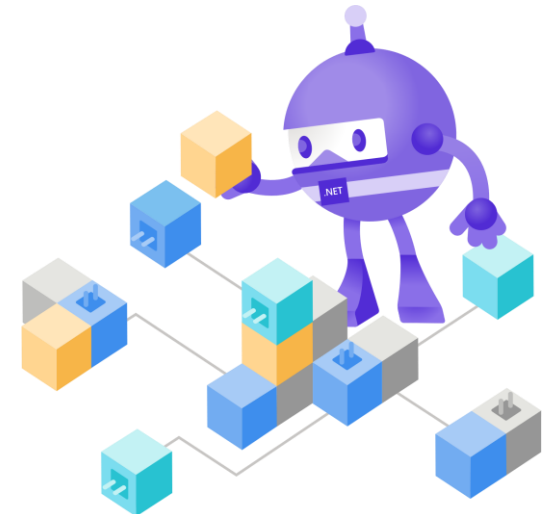
Minimal APIs in ASP.NET Core

- Simplified approach for **building fast and efficient HTTP APIs** in ASP.NET.
 - Create fully functional **REST endpoints** with **minimal coding** and **configuration**, **reducing** the need for **traditional scaffolding** and unnecessary controllers.
 - **Skip traditional scaffolding** and **avoid unnecessary controllers** by fluently declaring API routes and actions.
-
- `var app = WebApplication.Create(args);`
 - `app.MapGet("/", () => "Hello World!");`
 - `app.Run();`
-
- **GET request** to the root path. Ease of setting up a basic **API endpoint** with **Minimal APIs**.



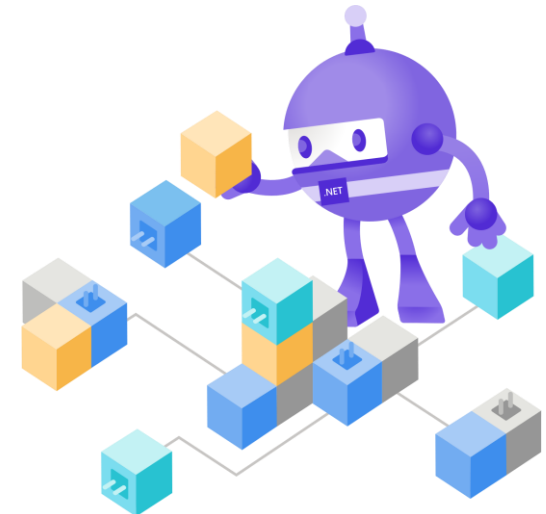
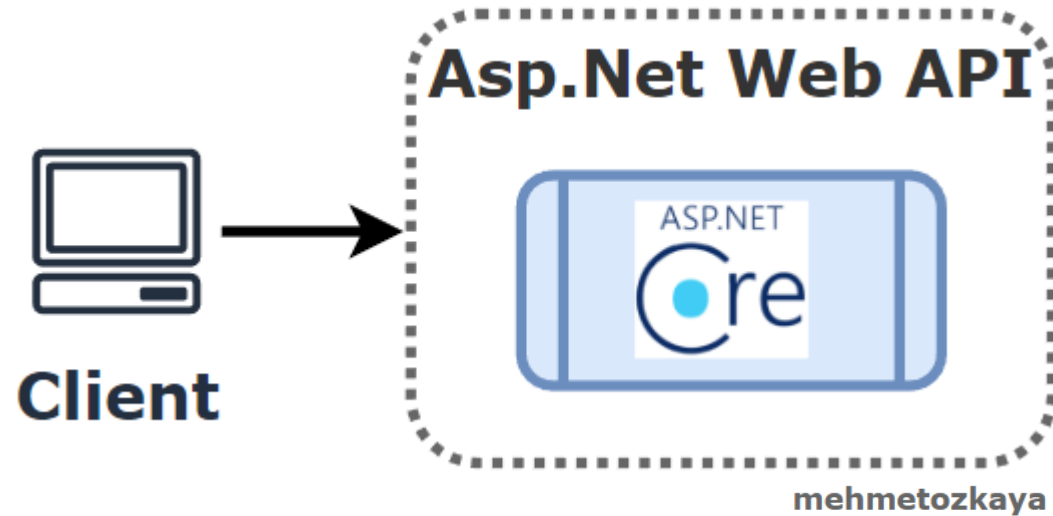
Routing with Parameters - Minimal APIs in ASP.NET Core

- Minimal APIs also support **dynamic routing**.
- *var builder = WebApplication.CreateBuilder(args);*
- *var app = builder.Build();*
- *app.MapGet("/users/{userId}/books/{bookId}",*
(int userId, int bookId) => \$"The user id is {userId} and book id is {bookId}");
- *app.Run();*
- **Route parameters** for a **userId** and a **bookId**. The API responds with these IDs.



Hands-on: Create ASP.NET Core Web API for Single Microservice

- Setting Up the Project
- Exploring Program.cs
- Running the Application

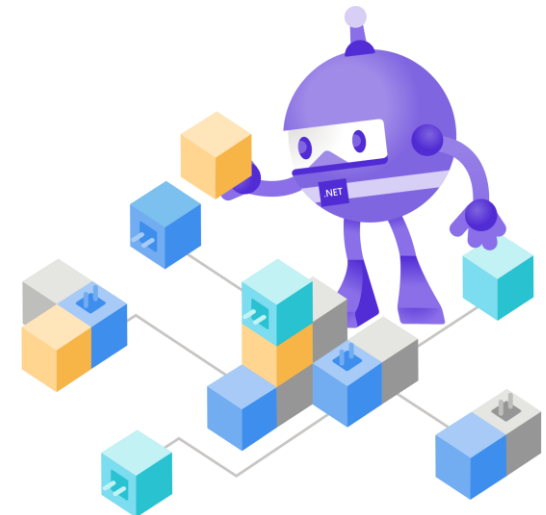
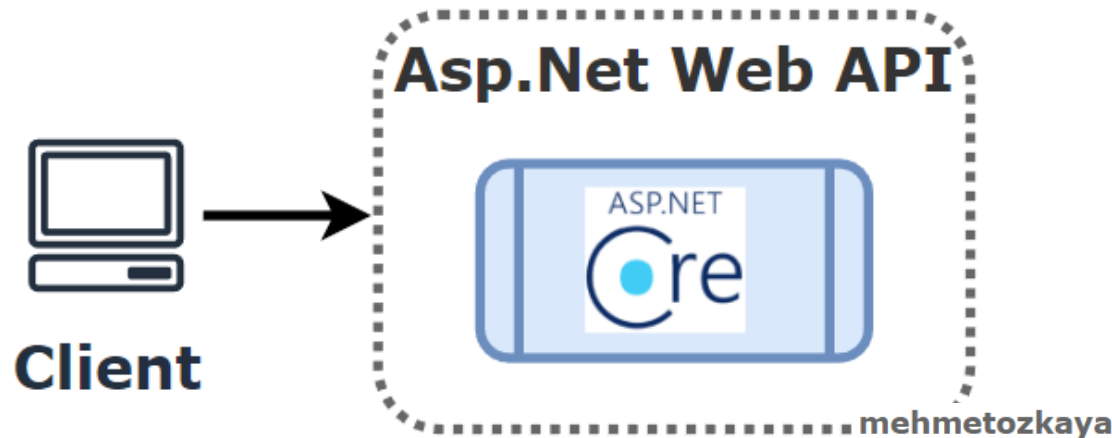


Hands-on: Create ASP.NET Core Web API for Single Microservice

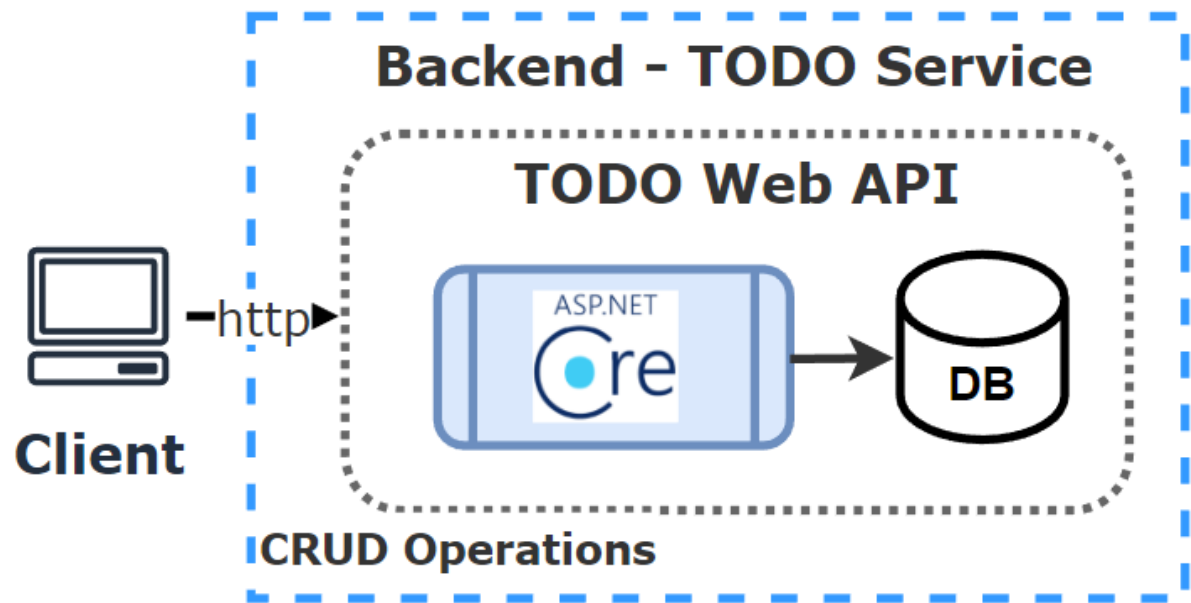
- 1- Create web application builder
- 2- Add services to the container. - AspNet built-in DI
- 3- Build the application
- 4- Configure the HTTP request pipeline.
- 5- Run the application

Before Building App

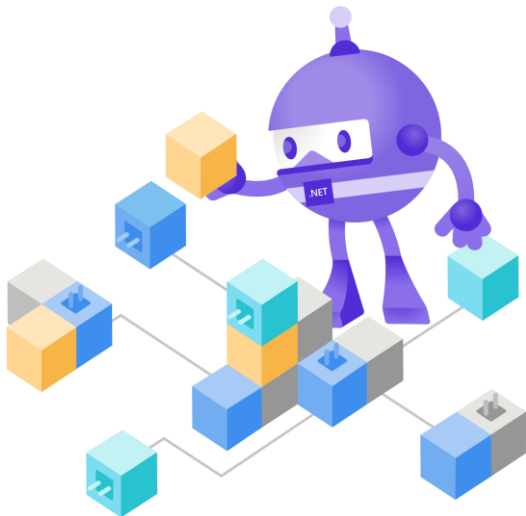
After Building App



Hands-on: Todo Service: Create Minimal API with ASP.NET Core for Single Microservice

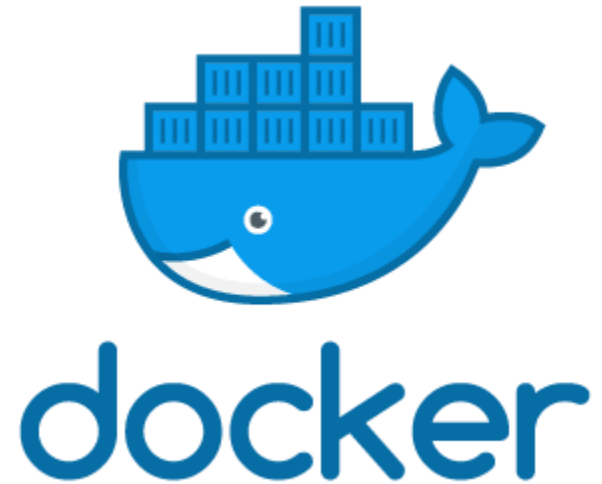


API	Description	Request body	Response body
GET /todoitems	Get all to-do items	None	Array of to-do items
GET /todoitems/complete	Get completed to-do items	None	Array of to-do items
GET /todoitems/{id}	Get an item by ID	None	To-do item
POST /todoitems	Add a new item	To-do item	To-do item
PUT /todoitems/{id}	Update an existing item	To-do item	None
DELETE /todoitems/{id}	Delete an item	None	None



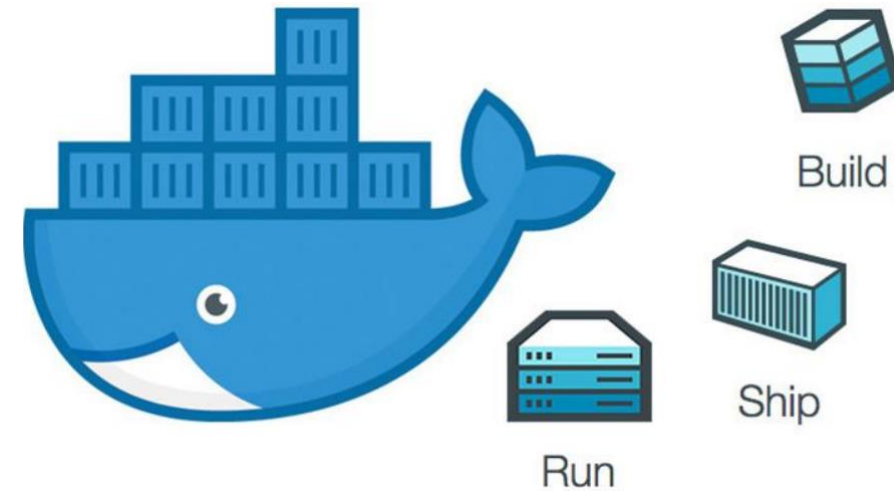
What is Docker ?

- **Docker** is an **open platform** for **developing, shipping, and running** applications.
- **Separate** your **applications** from your **infrastructure** so you can deliver software quickly.
- Advantages of Docker's methodologies for **shipping, testing, and deploying** code **quickly**.
- **Significantly reduce** the delay between **writing code** and **running** it in production.
- **Automating the deployment** of applications as **portable, self-sufficient containers** that can run on the **cloud** or **on-premises**.
- **Docker containers** can run anywhere, in your local computer to the cloud.
- **Docker image** containers can **run natively** on **Linux** and **Windows**.
- **Docker** is defacto standard for **containerization** of **microservices**.



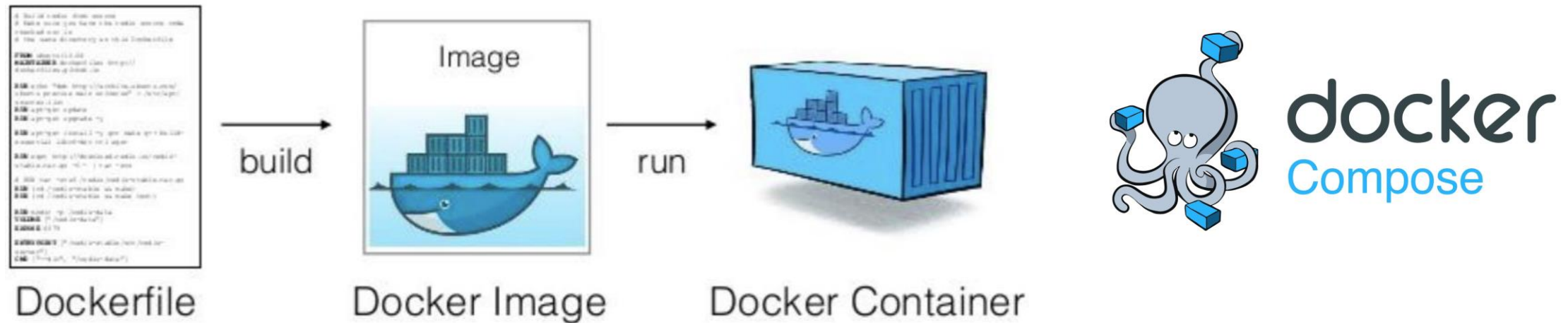
Docker Containers, Images, and Registries

- Developer **develops** and **packages** application **with** its **dependencies** into a **container image**, that is a **static representation** of the application **with** its **configuration** and **dependencies**.
- To run the application, the application's image is **instantiated to create a container**, which will be running on the Docker host.
- Store **images in a registry**, which is a **library of images** and is needed when deploying to production orchestrators.
- **Docker images are stores** a public **registry** like **Docker Hub**, **Azure Container Registry**.
- Developer **creates container** in local and **push the images** the **Docker Registry**.
- Developer **download existing image** from **registry** and create container from image in local environment.



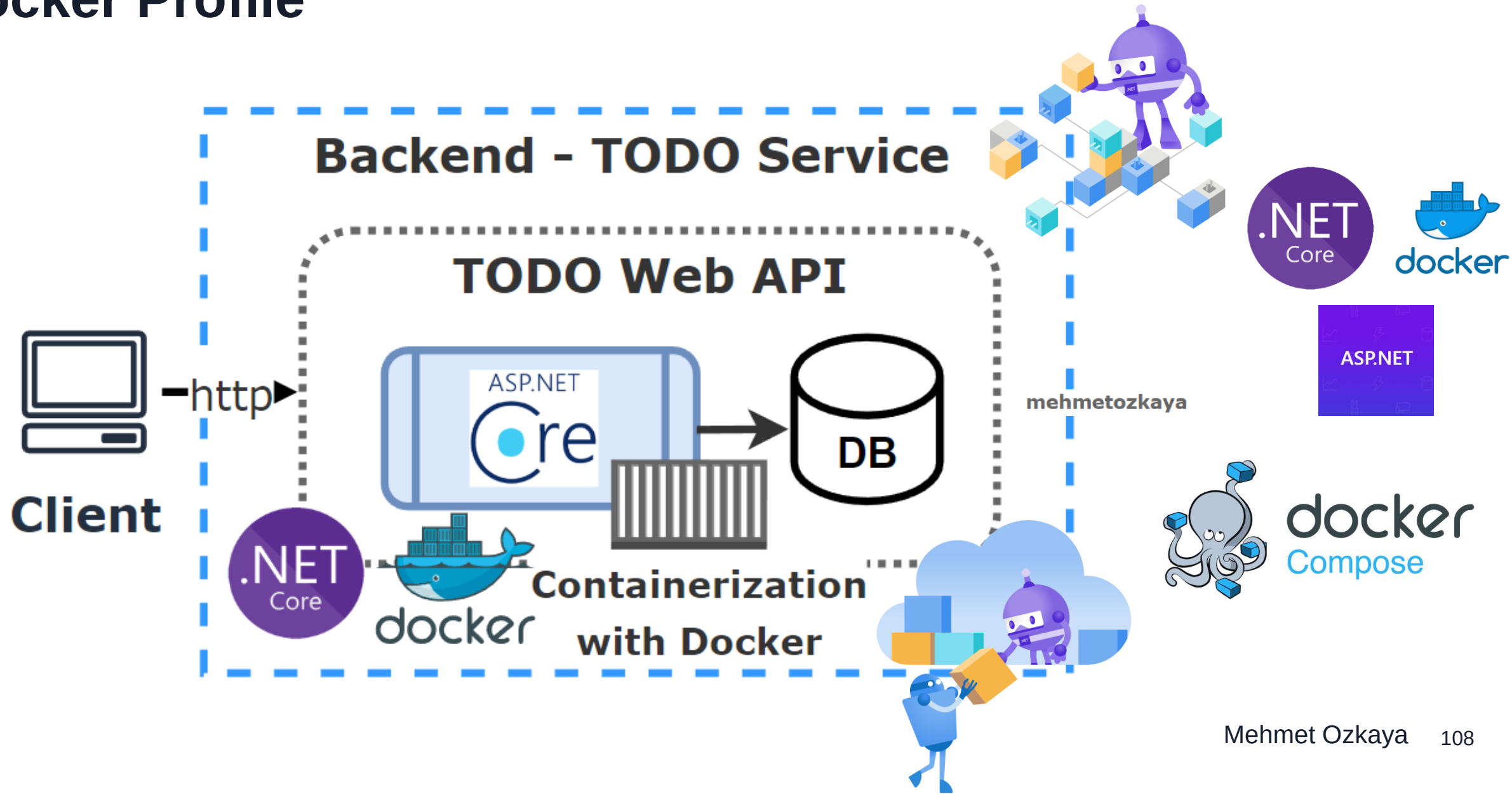
Application Containerization with Docker, Orchestration with Docker Compose

1. Write **Dockerfile** for our application.
2. **Build application** with this docker file and **creates the docker images**.
3. **Run this images** on any machine and creates **running docker container** from **docker image**.



- **Orchestrating** whole **microservices** application with **Docker-Compose** for running multi-container Docker images. single command, create and start all the services. Mehmet Ozkaya 107

Containerize Todo Asp.Net Web Api Microservice and Run Docker Profile



First Microservice: Catalog.API with Vertical Slice Architecture and CQRS

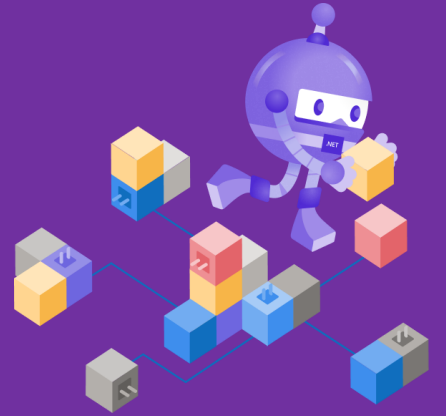
Minimal APIs and latest features of .Net8 and C# 12

Vertical Slice Architecture with Feature folders

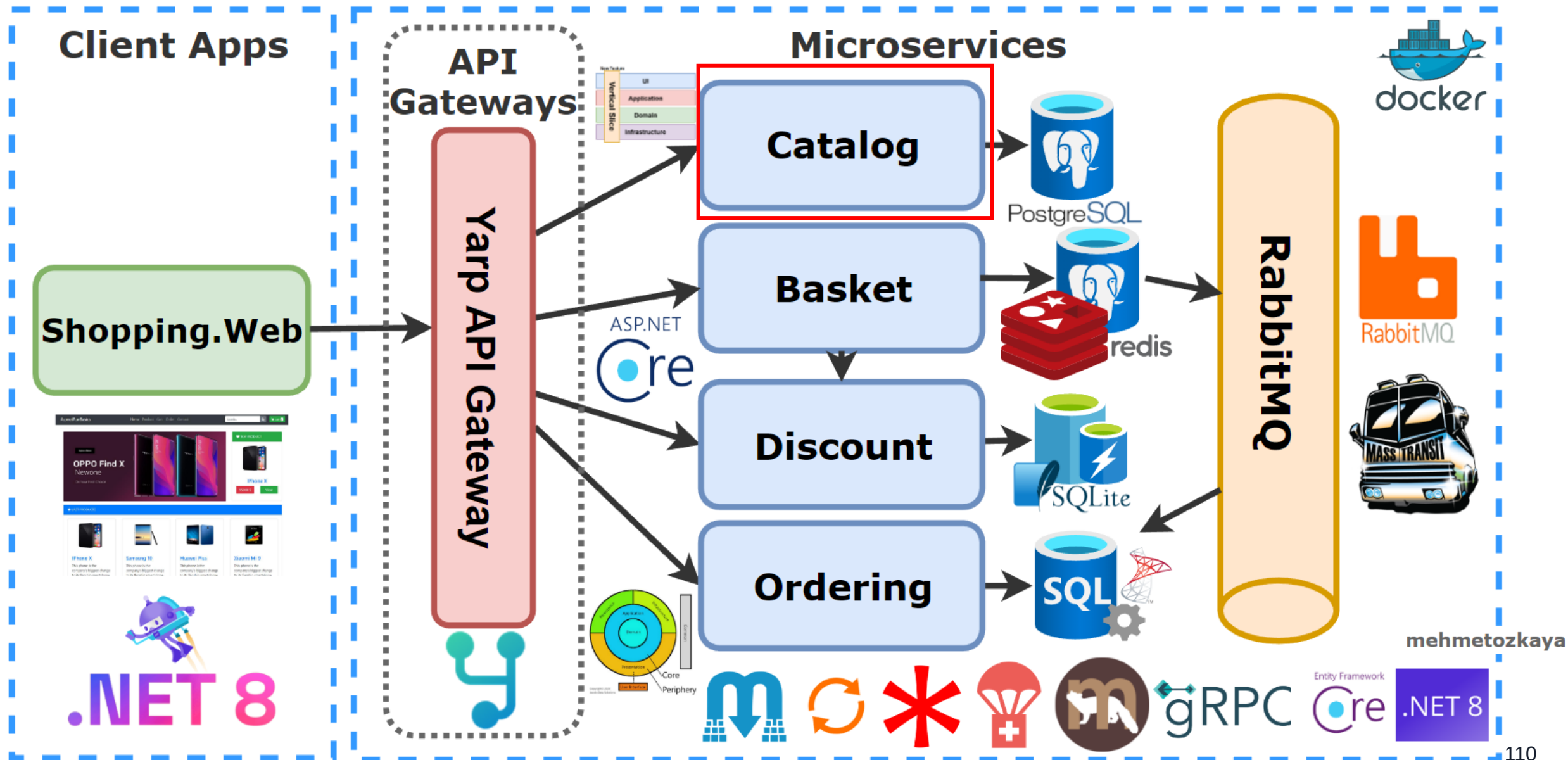
CQRS implementation using MediatR library

Marten library for .NET Transactional Document DB on PostgreSQL

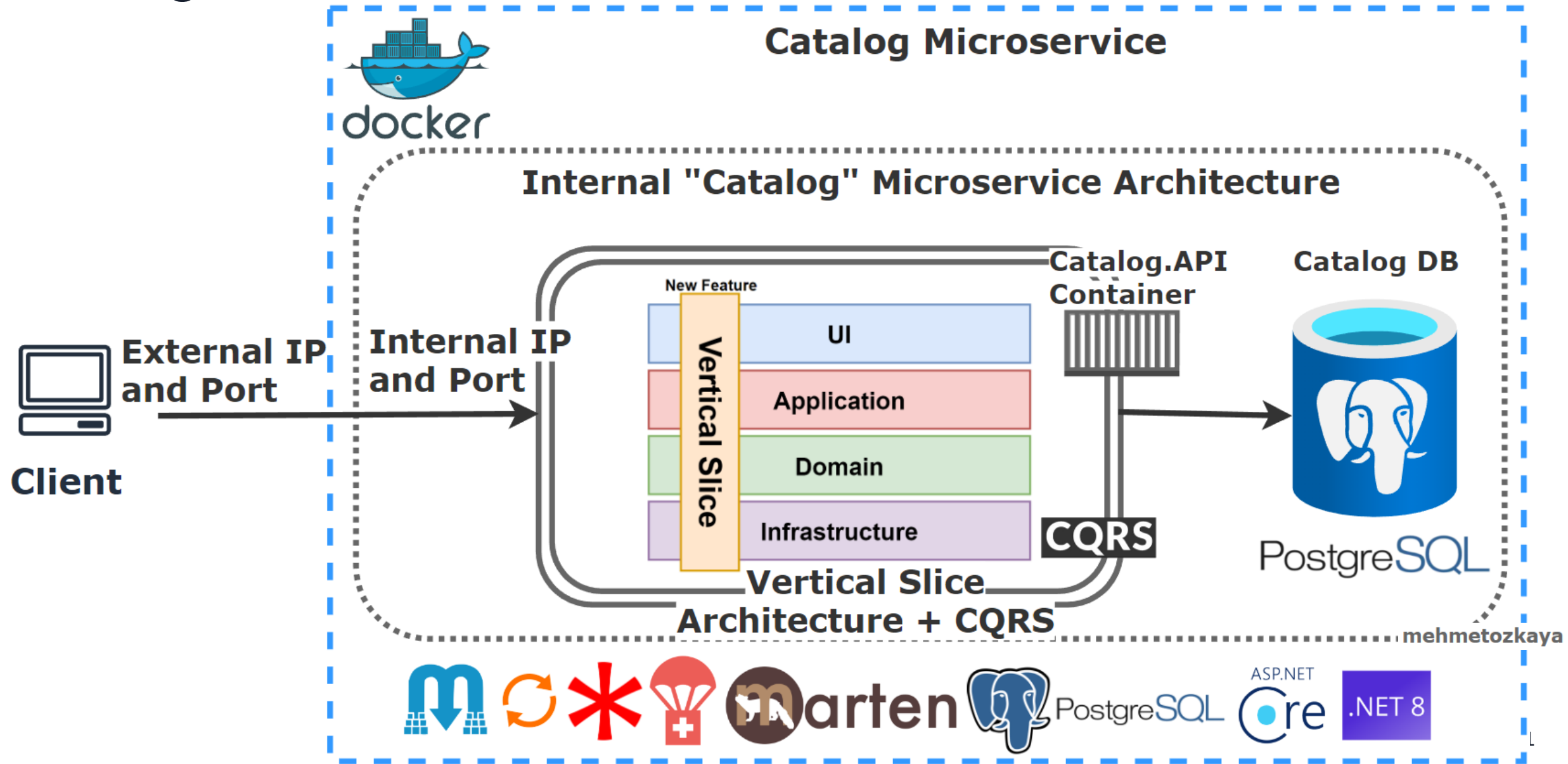
Mehmet Ozkaya



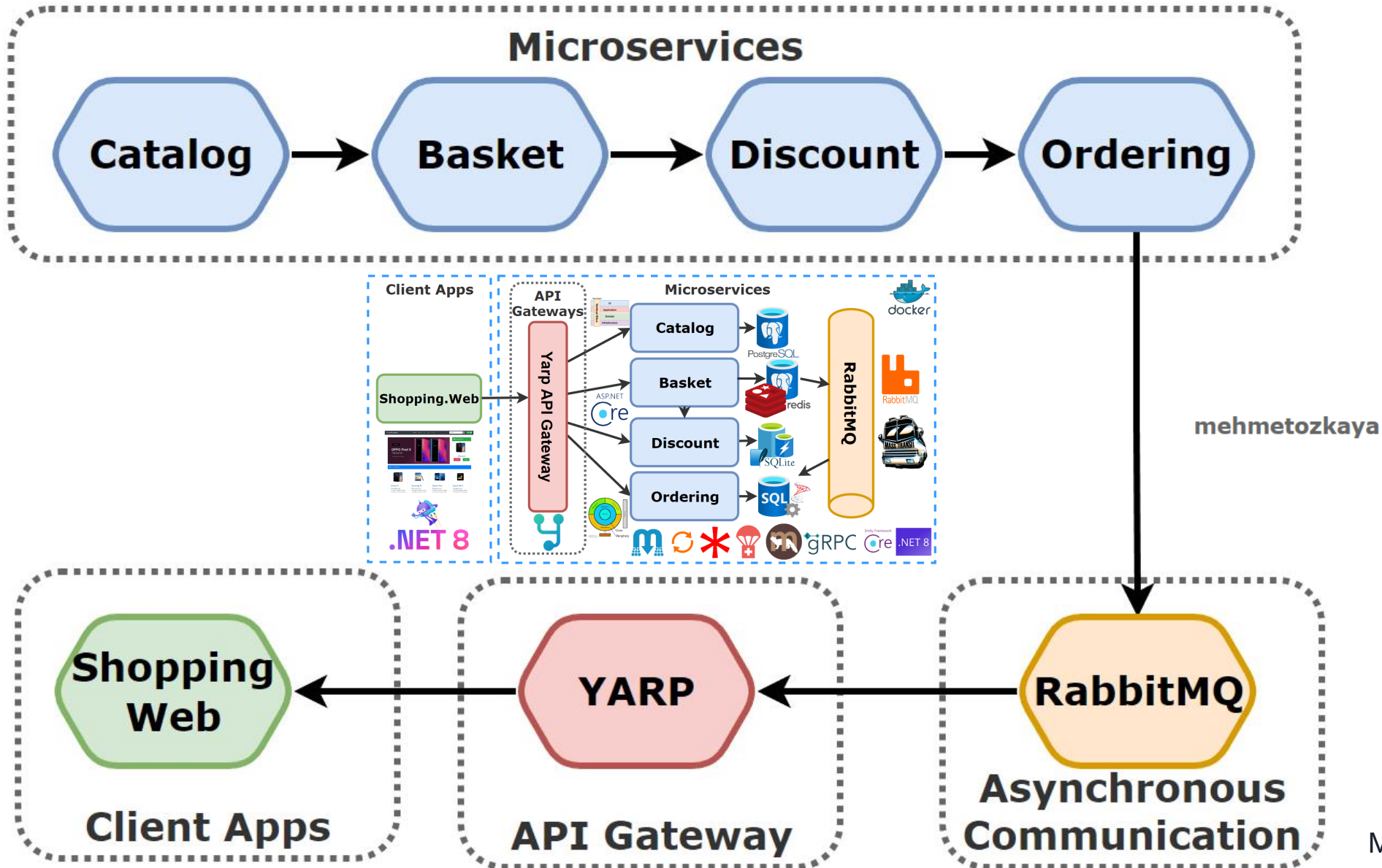
Big Picture – Course Project



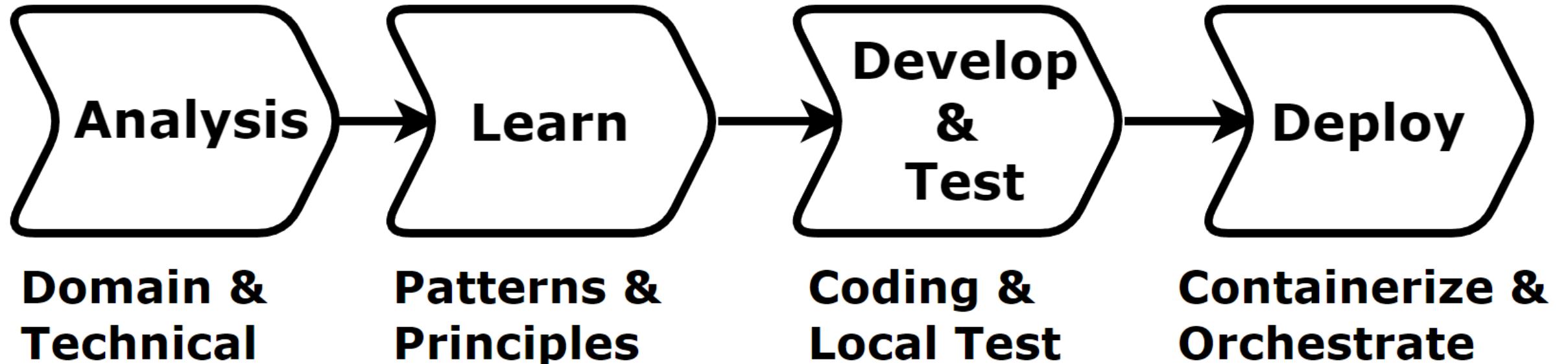
Catalog Microservice Internal Architecture



How to Follow the Course (The Course Flow)

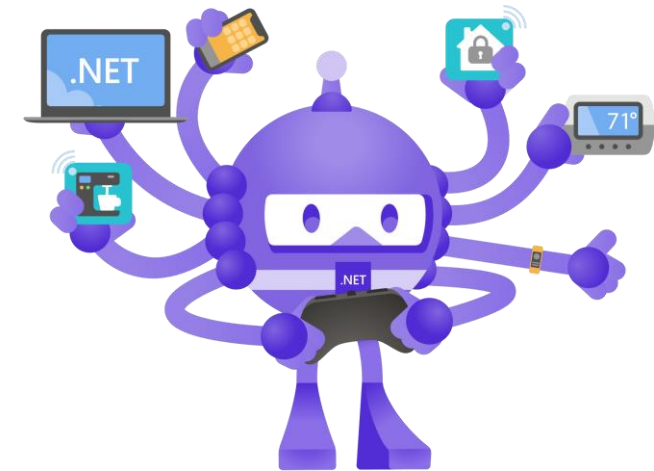


Way of Learning (For Each Microservices)

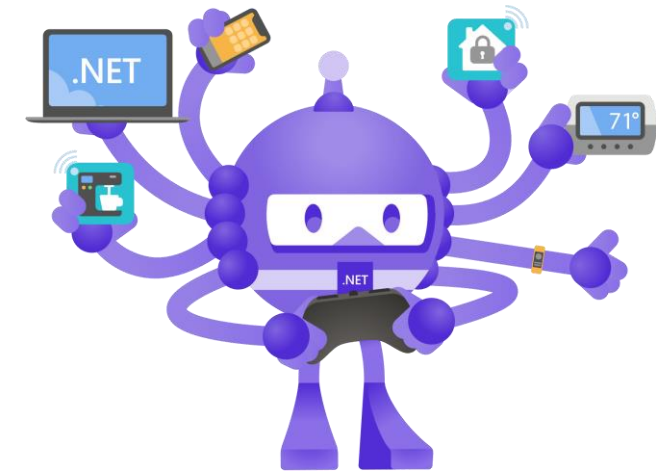
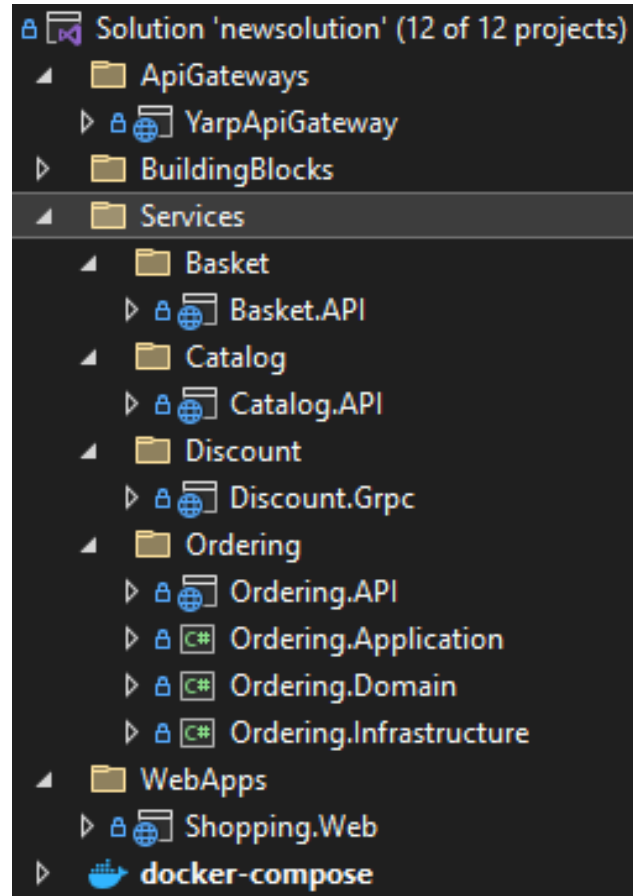


Catalog.API with Vertical Slice Architecture and CQRS

- **ASP.NET Core Minimal APIs** for building fast HTTP APIs-fully functioning REST endpoints and top-level Program.cs
- **Vertical Slice Architecture** implementation with Feature folders and single .cs file includes different classes in one file.
- **CQRS** implementation using **MediatR** library.
- **Marten** library for .NET Transactional Document DB on PostgreSQL
- **Carter** for Minimal Api endpoint definition
- **Mapster** when it possible for mapping dto classes
- **FluentValidation** to validate inputs and add MediatR validation pipeline
- **Dockerfile** and **docker-compose** file for running Catalog microservice and PostgreSQL database in docker environment



Create Asp.Net Web Api for Catalog.API Microservice

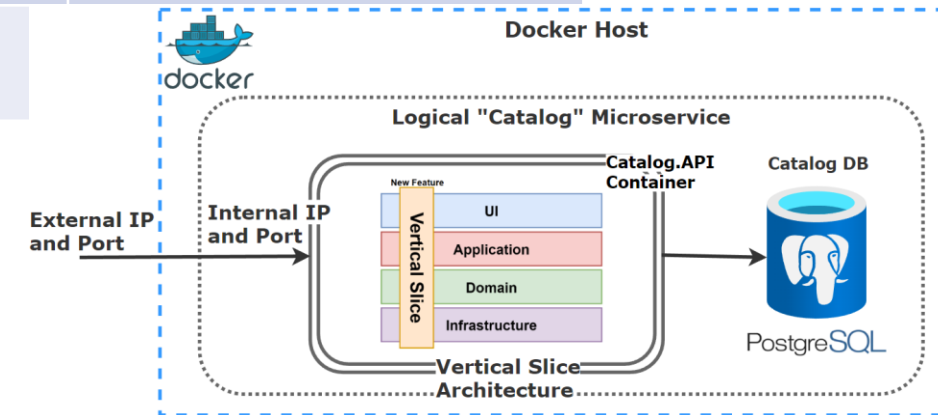


Port Numbers for Catalog.API Microservice

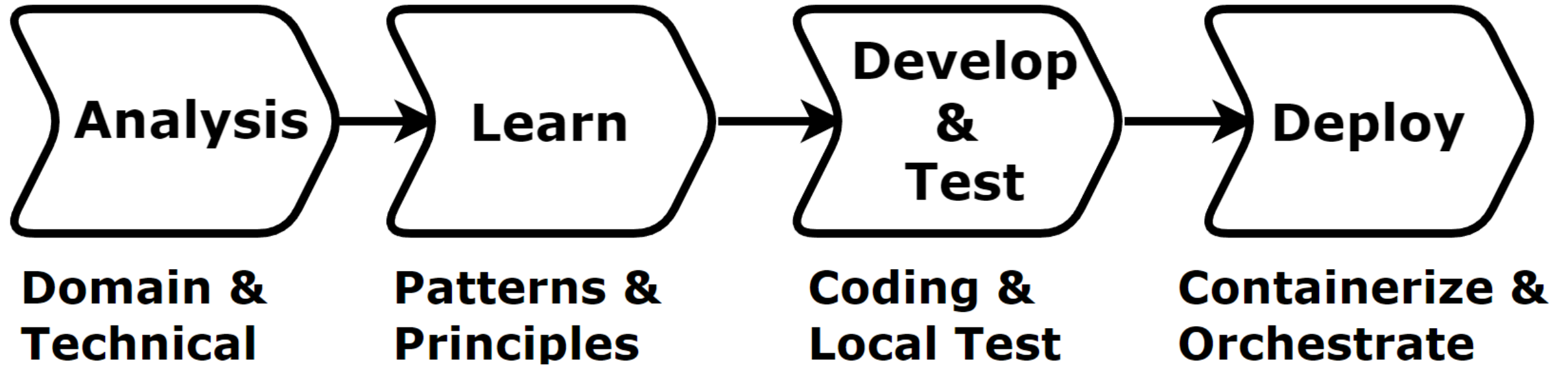


Microservices	Local Env	Docker Env	Docker Inside
Catalog	5000 – 5050	6000 – 6060	8080 – 8081
Basket	5001 – 5051	6001 – 6061	8080 – 8081
Discount	5002 – 5052	6002 – 6062	8080 – 8081
Ordering	5003 – 5053	6003 – 6063	

- ASP.NET Core **Ports** are **listed** as **http / https** for running application.

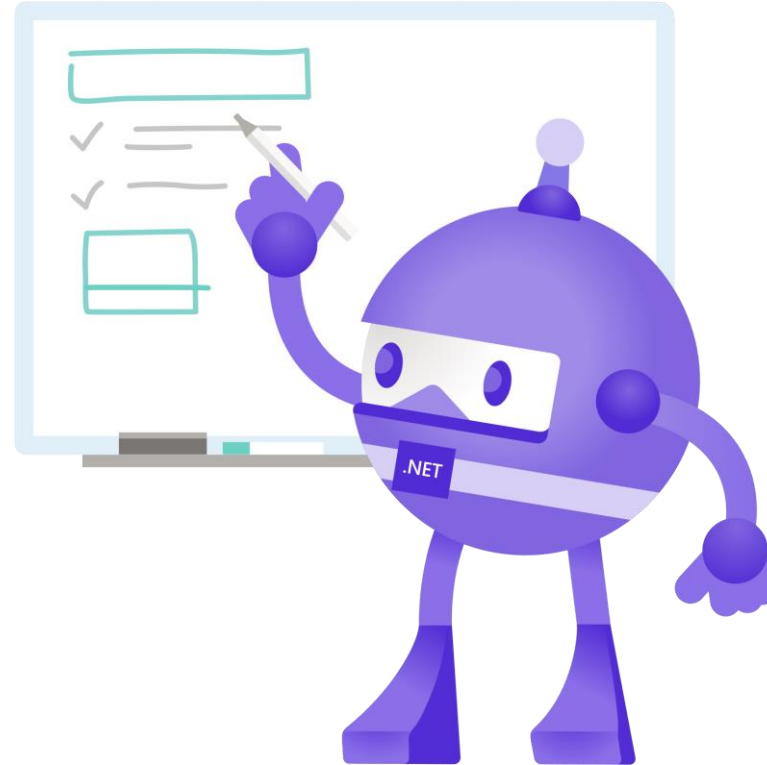


Way of Learning (For Each Microservices)



Domain Analysis of Catalog Microservices

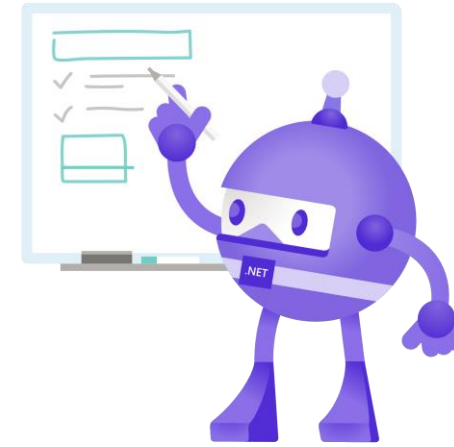
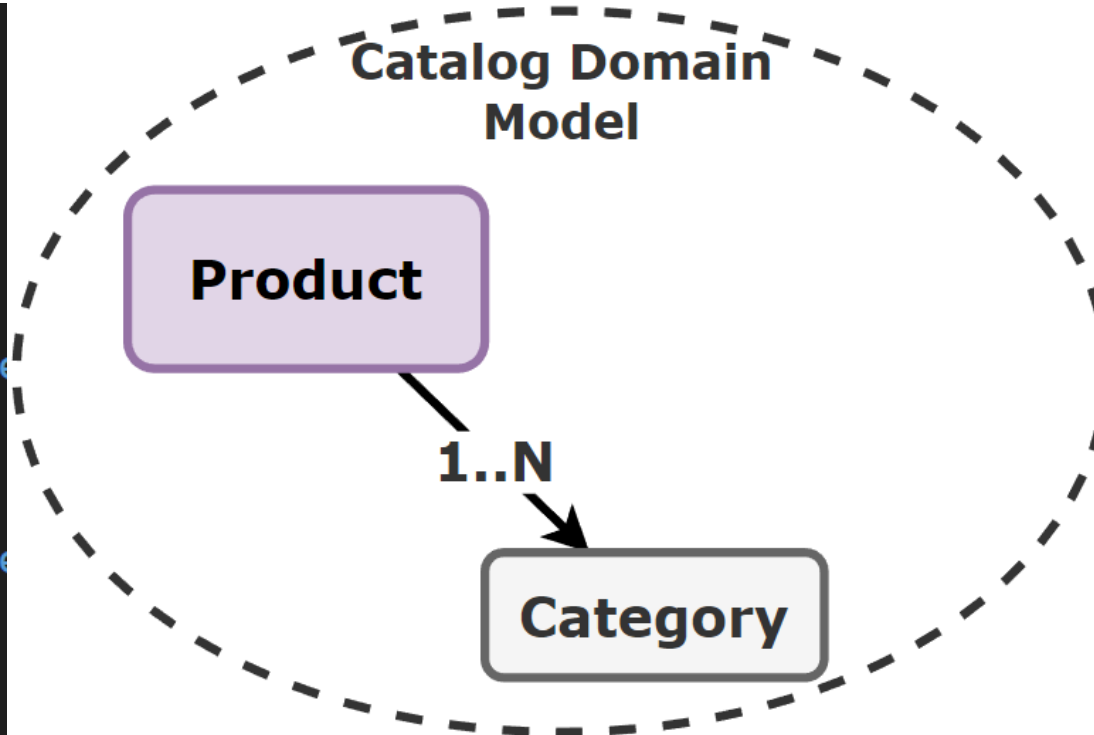
1. Domain Models
2. Application Use Cases
3. Rest API Endpoints
4. Underlying Data Structures



1. Domain Models of Catalog Microservices

- Primary domain model is 'Product', associated with 'Category'

```
public class Product
{
    8 references
    public Guid Id { get; set; }
    9 references
    public string Name { get; set; }
    10 references
    public List<string> Category { get; }
    9 references
    public string Description { get; }
    9 references
    public string ImageFile { get; set; }
    9 references
    public decimal Price { get; set; }
}
```



- Consider Domain event for price changes, leading to integration events.

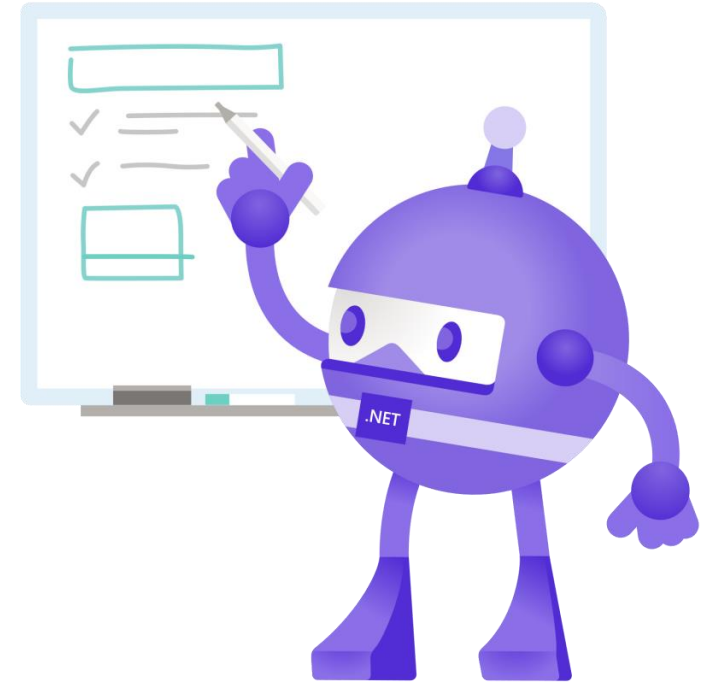
2. Application Use Cases of Catalog Microservices

- Main use case are **Listing Products** and **Categories**, able to **search products**.

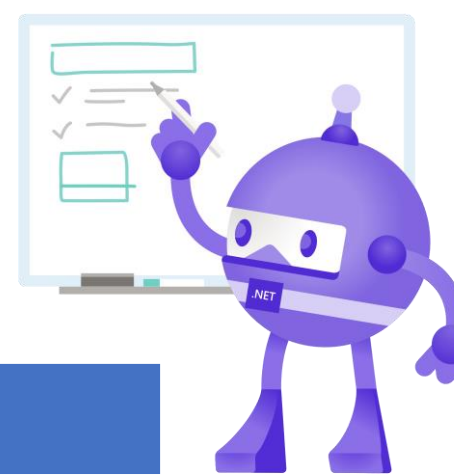
CRUD Operations:

- Listing Products and Categories
- Get Product with product Id
- Get Products with category
- Create new Product
- Update Product
- Delete Product

- *Along with this design our APIs according to REST perspective.



3. Rest API Endpoints of Catalog Microservices



Method	Request URI	Use Cases
GET	/products	List all products
GET	/products/{id}	Fetch a specific product
GET	/products/category	Get products by category
POST	/products	Create a new product
PUT	/products/{id}	Update a product
DELETE	/products/{id}	Remove a product

4. Underlying Data Structures of Catalog Microservices

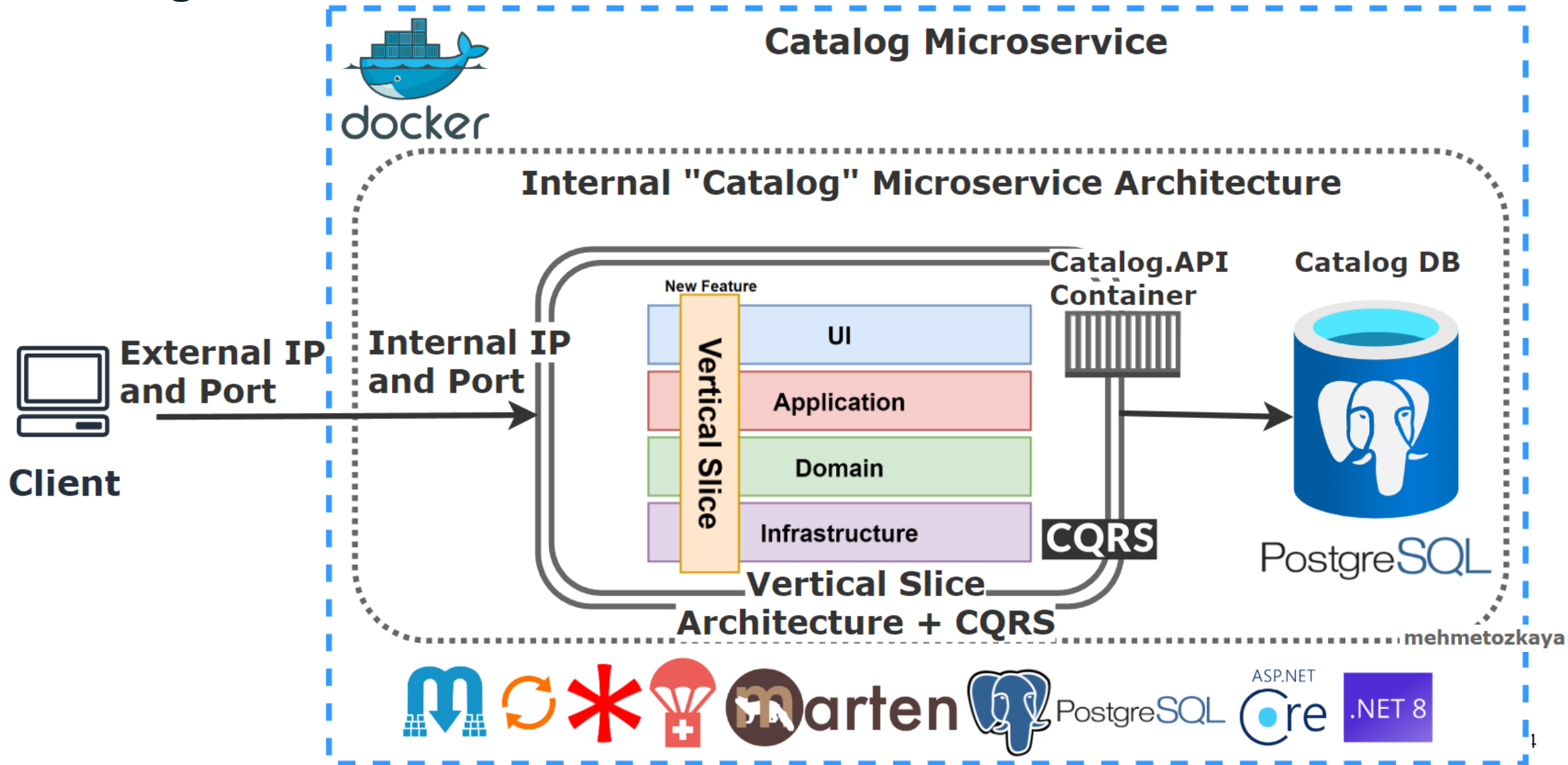
- **Document Database** with **Store** Catalog **JSON** data.
- **2 options:** MongoDB No-SQL database, PostgreSQL DB JSON Columns. **Chosen - PostgreSQL** with the **Marten** library.
- **Marten** is a powerful library that transforms **PostgreSQL** into a **.NET Transactional Document DB**.
- **PostgreSQL's JSON column features**, allowing us to store and query our data as JSON documents.
- Combines the **flexibility** of a **document database** with the reliability of **relational PostgreSQL** database.



The diagram illustrates a microservices architecture for an e-commerce application. It is divided into three main sections: Client Apps, API Gateways, and Microservices.

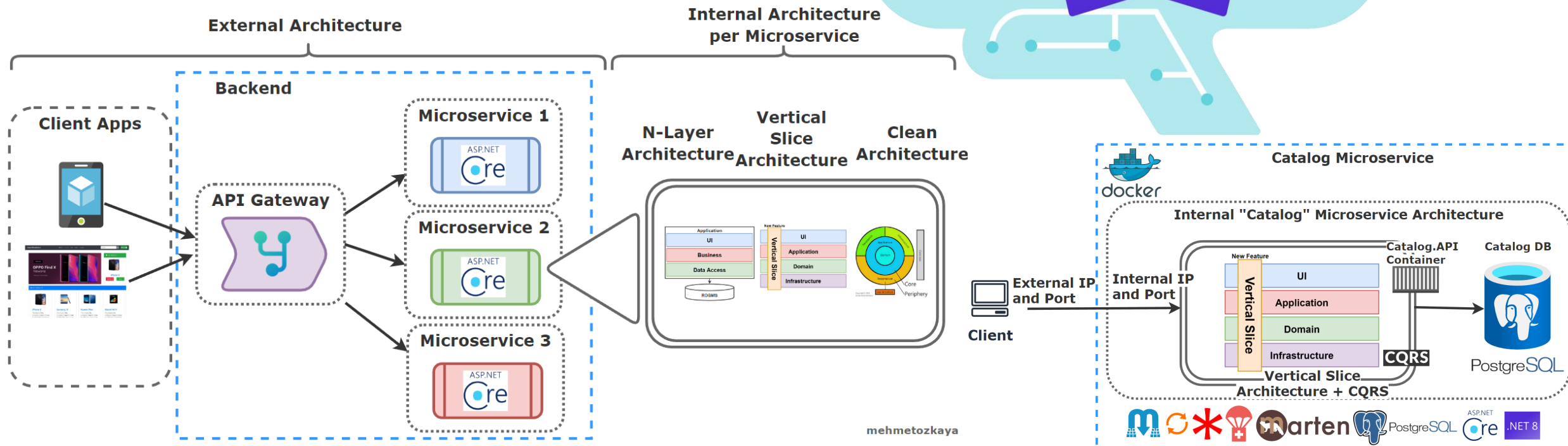
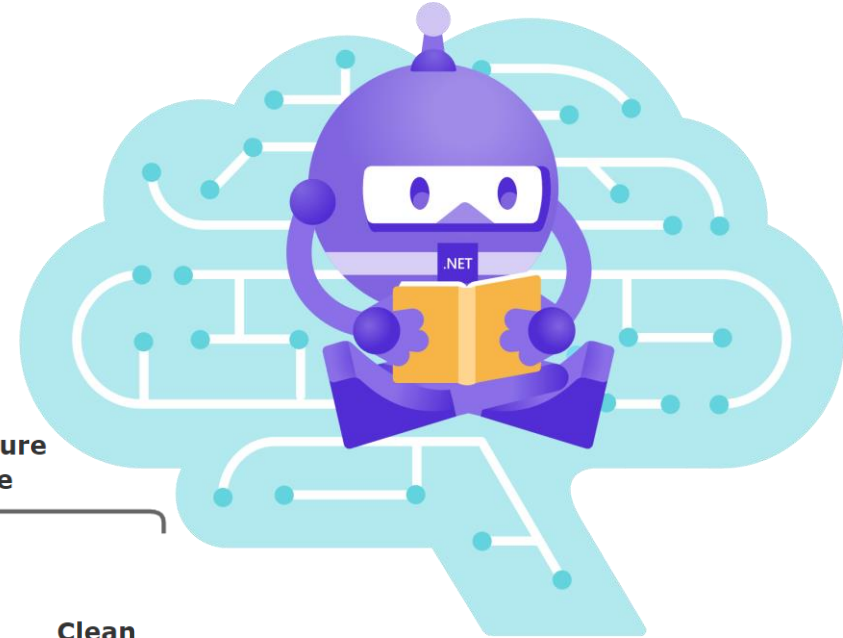
- Client Apps:** Includes "Shopping.Web" and a mobile application icon, both utilizing ".NET 8".
- API Gateways:** A "Yarp API Gateway" (indicated by a blue icon) acts as the central routing point, receiving requests from the client apps and directing them to the appropriate microservices.
- Microservices:** Four core services are shown:
 - Catalog:** Interacts with PostgreSQL and Docker.
 - Basket:** Utilizes ASP.NET Core and Redis.
 - Discount:** Uses SQLite.
 - Ordering:** Relies on SQL and RabbitMQ.
- Integration and Data Flow:**
 - The "Basket" service communicates with the "Discount" service.
 - The "Ordering" service interacts with the "Basket" service and the "RabbitMQ" message broker.
 - The "RabbitMQ" service (represented by a yellow cylinder) is connected to the "Discount" service.
- Additional Components:**
 - A "Vertical Slice" diagram shows the layered architecture: UI, Application, Domain, and Infrastructure.
 - Logos for various technologies are present: Docker, RabbitMQ, .NET 8, ASP.NET Core, PostgreSQL, Redis, SQLite, and Entity Framework.
 - A "Mass Transit" logo is also visible, likely referring to the message broker.

Catalog Microservice Internal Architecture



Technical Analysis of Catalog Microservices

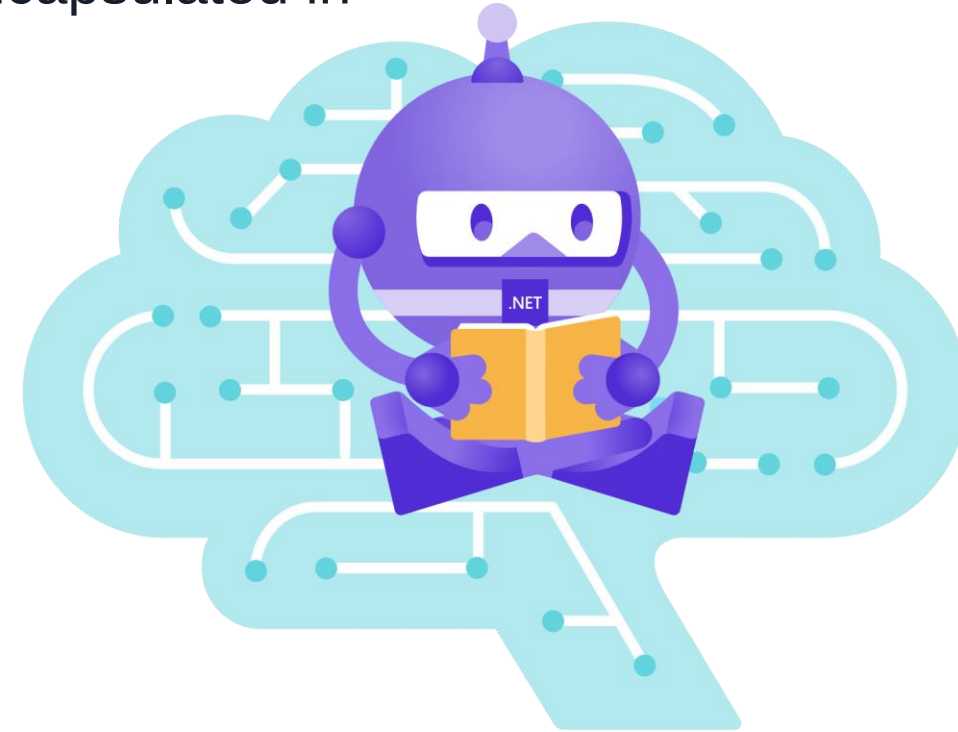
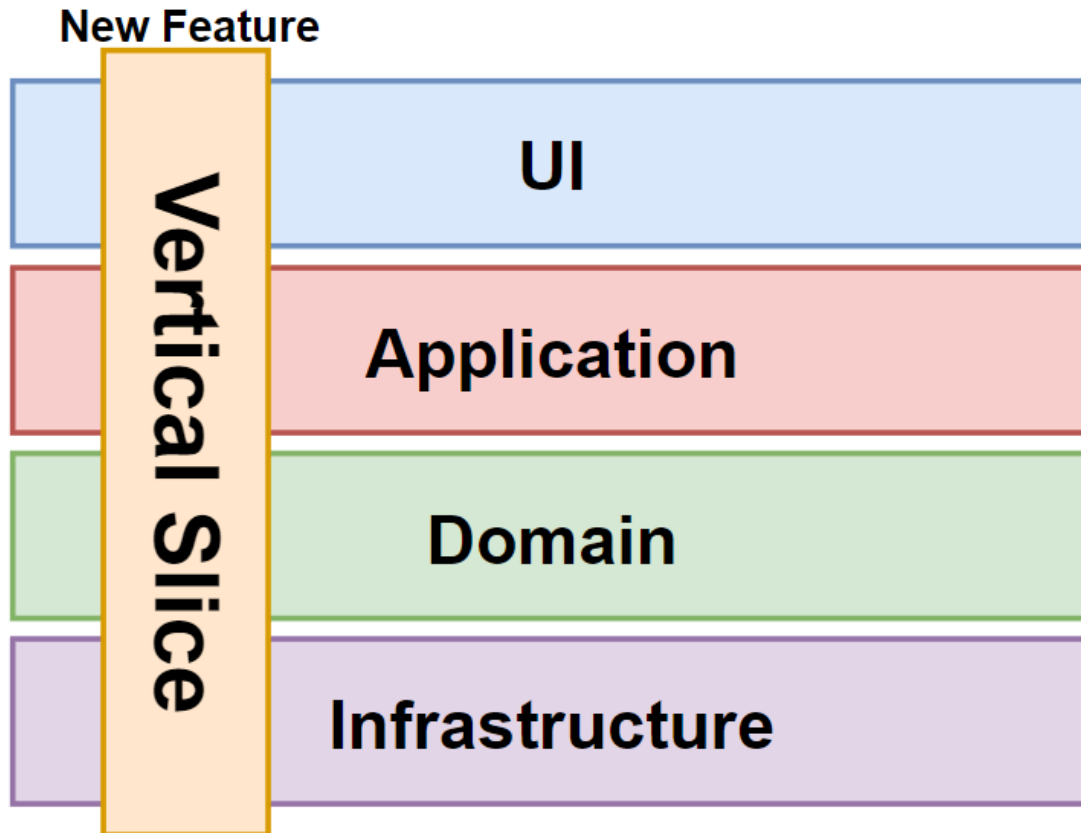
1. Application Architecture Style
2. Patterns & Principles
3. Libraries Nuget Packages
4. Project Folder Structure



1. Application Architecture Style of Catalog Microservices

Vertical Slice Architecture

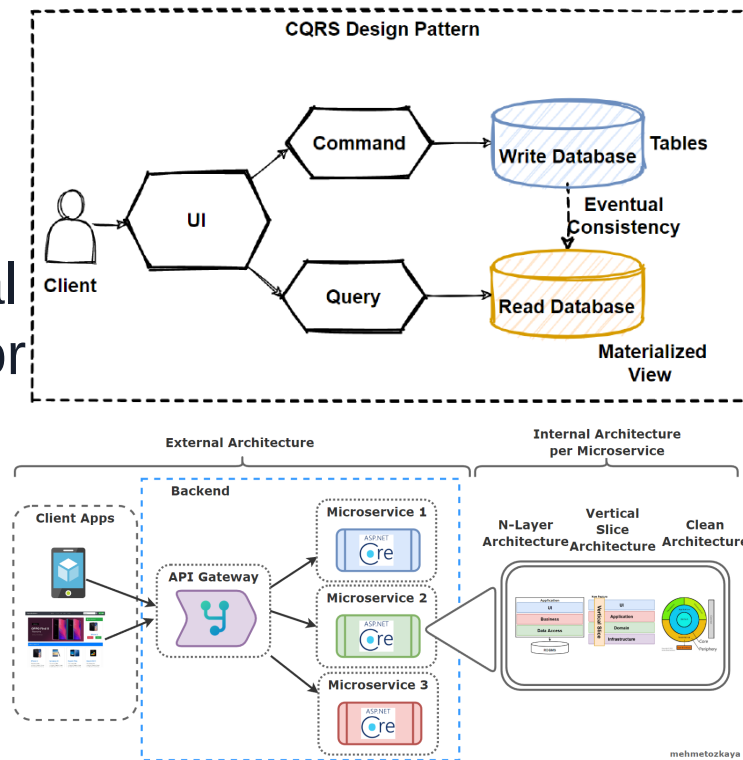
- Organizes our code into feature folders, each feature encapsulated in a single .cs file



2. Patterns & Principles of Catalog Microservices

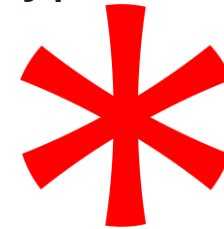


- **CQRS Pattern:** Command Query Responsibility Segregation divides operations into commands (write) and queries (read).
- **Mediator Pattern:** Facilitates object interaction through a 'mediator', reducing direct dependencies and simplifying communications.
- **DI in ASP.NET Core:** Dependency Injection is a core feature, allowing us to inject dependencies.
- **Minimal APIs and Routing in ASP.NET 8:** ASP.NET 8's Minimal APIs simplify endpoint definitions, providing lightweight syntax for routing and handling HTTP requests.
- **ORM Pattern:** Object-Relational Mapping abstracts database interactions, work with database objects using high-level codes.



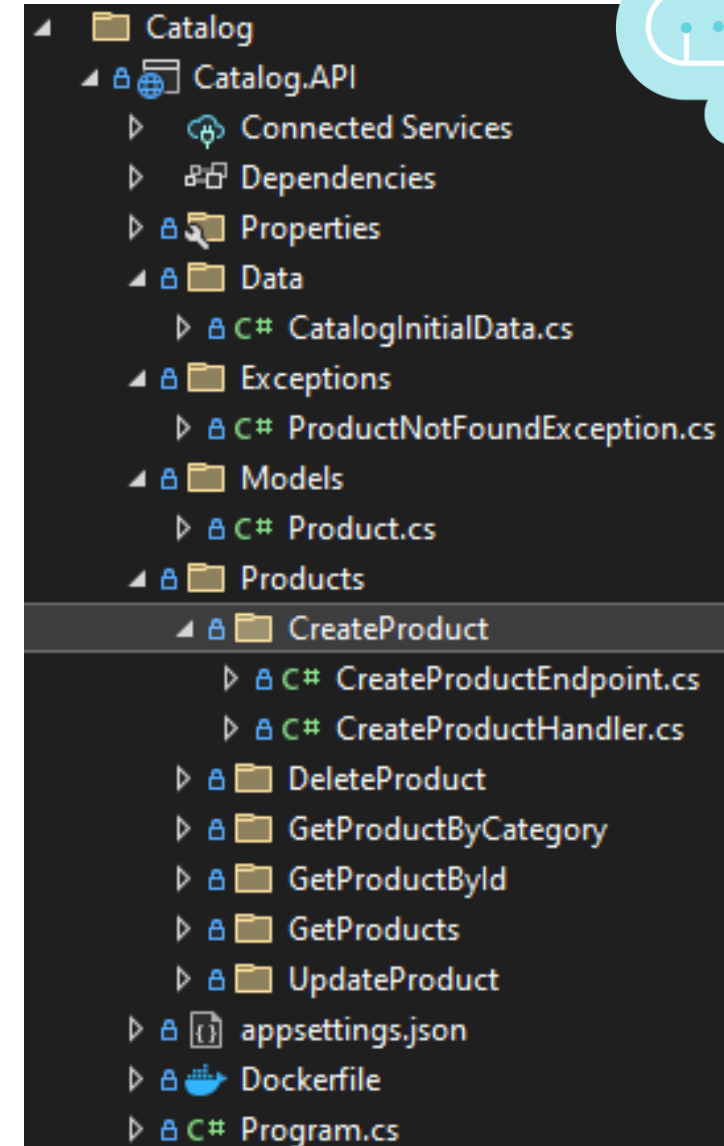
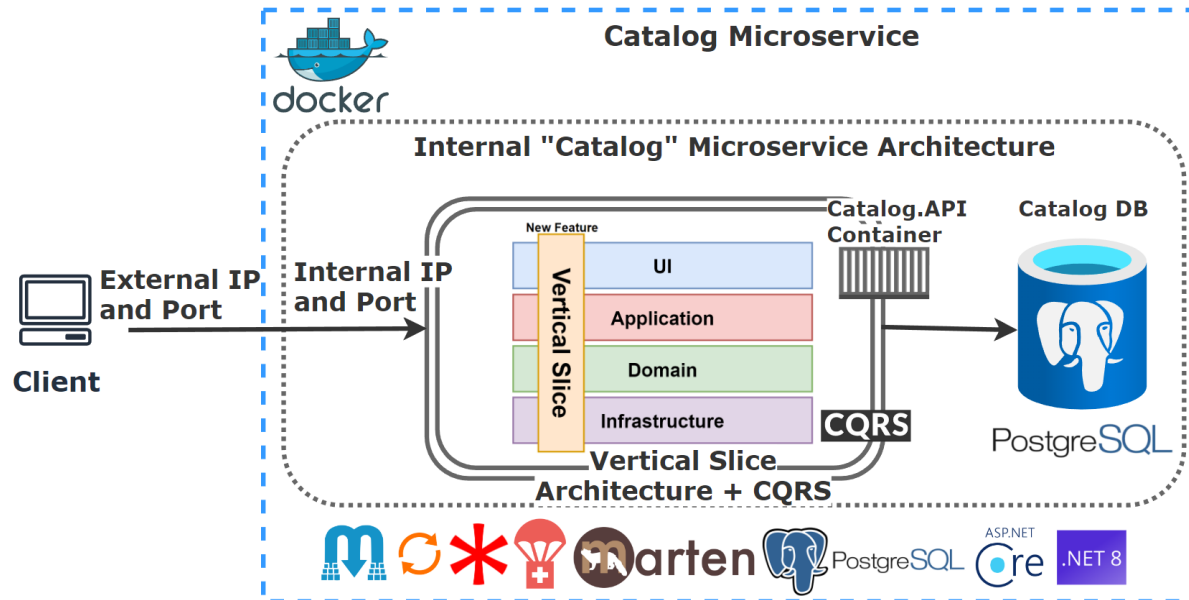
3. Libraries Nuget Packages of Catalog Microservice

- **MediatR for CQRS:** MediatR library simplifies the implementation of the CQRS pattern.
- **Carter for API Endpoints:** Routing and handling HTTP requests, easier to define API endpoints with clean and concise code.
- **Marten for PostgreSQL Interaction:** Use PostgreSQL as a Document DB. It leverages PostgreSQL's JSON capabilities for storing, querying, and managing documents.
- **Mapster for Object Mapping:** Mapster is a fast, configurable object mapper that simplifies the task of mapping objects.
- **FluentValidation for Input Validation:** For building strongly-typed validation rules, ensure inputs are correct before processed.



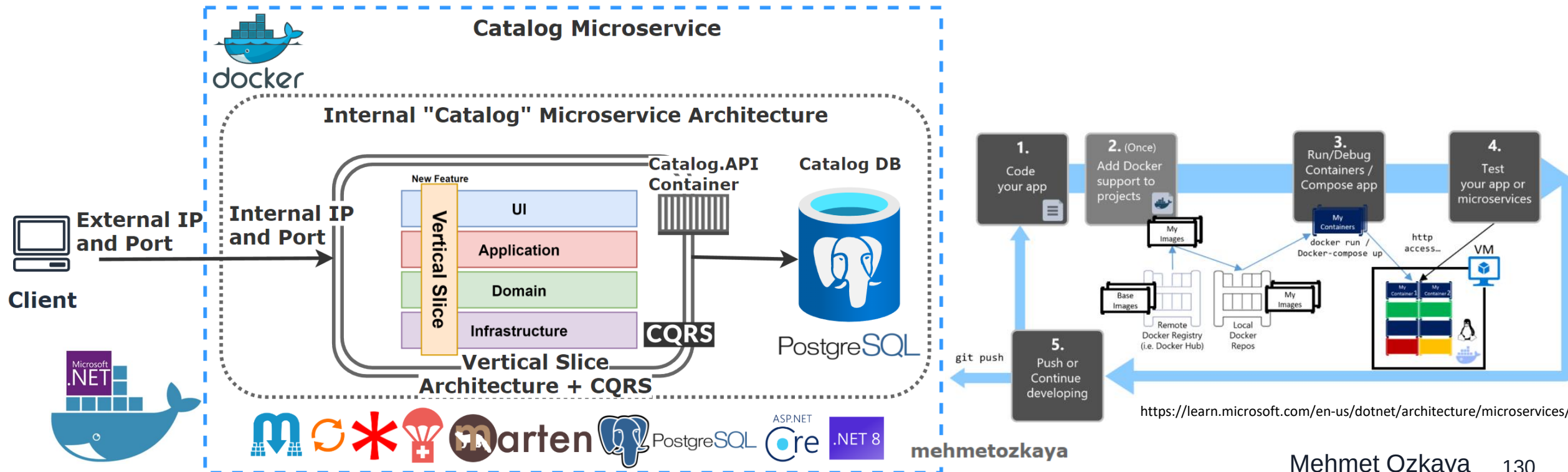
4. Project Folder Structure of Catalog Microservice

- The project is organized into **Model**, **Features**, **Data**, and **Abstractions**.
- **Features** like **CreateProduct** and **GetProduct** have dedicated handlers and endpoint definitions.
- **Feature** folder will be **Products**.
- **Data** folder and **Context** objects manages database interactions.



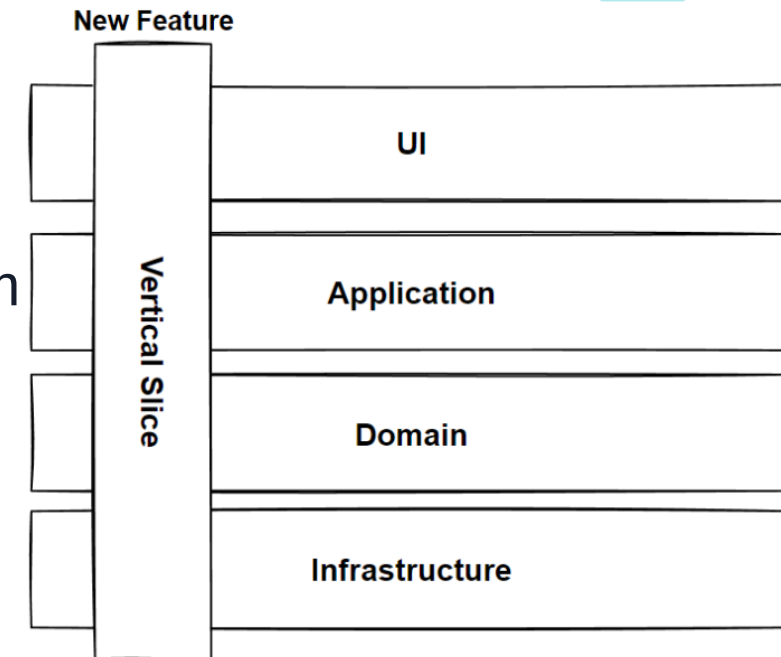
Deployment of Catalog Microservices

- **Containerize** our service with **Docker**, ensuring deployment and integration with PostgreSQL
- Implement **Dockerfile** and **docker-compose** file for running Catalog microservice and **PostgreSQL** database in Docker environment.



Vertical Slice Architecture

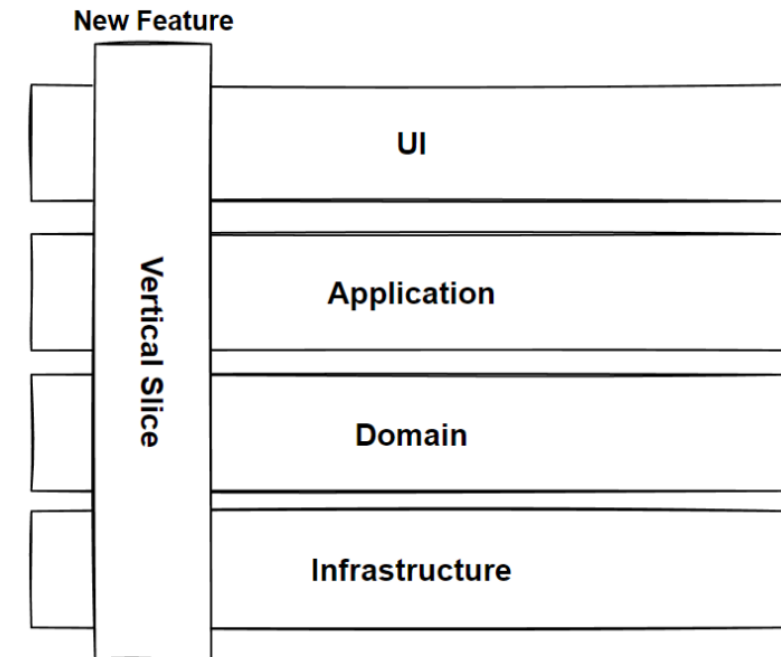
- [Introduces by Jimmy Bogard](#) offers this architecture **against to traditional layered/onion/clean architecture** approaches.
- Aims to **organize code around specific features or use cases**, rather than **technical concerns**.
- **Feature** is implemented **across all layers** of the software, from the user interface to the database.
- Often used in the development of **complex, feature-rich** apps
- **Divide** application into **distinct features or functionalities**, each of which cuts through all the layers of the application.
- **Contrast to traditional n-tier or layered architectures**, where the application is divided **horizontally** (e.g., presentation, business logic, data access layers).



Characteristics of Vertical Slice Architecture



- The application is divided **into feature-based slices**.
- **Each slice** is **self-contained** and **independent**.
- There are **reduced dependencies** between different parts of the application.
- It **promotes** the use of **cross-functional teams**.
- The architecture **supports scalability** and **maintainability**.
- It allows for **improved testing** and **deployment** processes.
- **Every microservice handles a specific piece of functionality** and communicates with other services through well-defined interfaces.



Benefits & Challenges of Vertical Slice Architecture

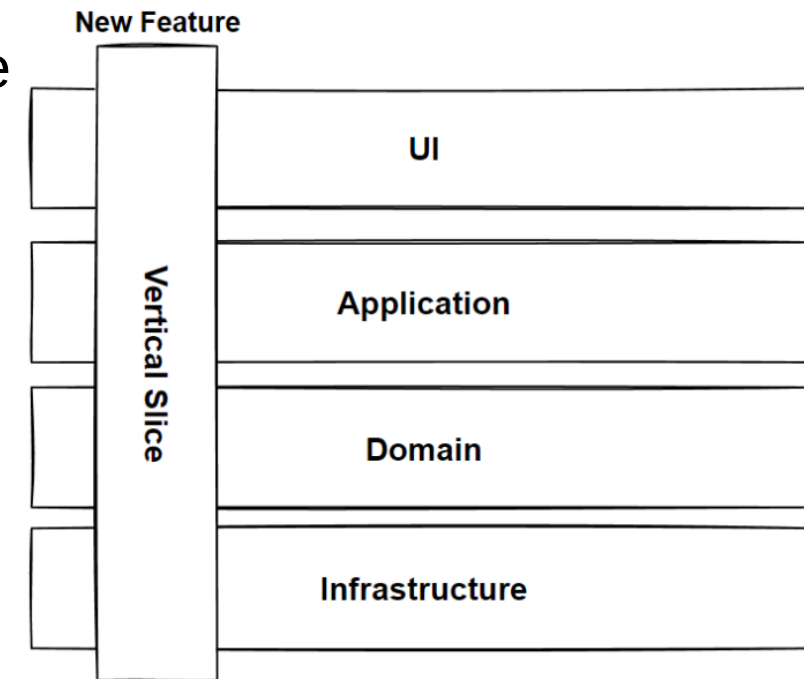


Benefits

- Focused development, teams can concentrate on one feature at a time.
- Simplifies refactoring and upgrades since changes in one slice usually don't affect others.
- Aligns well with Agile and DevOps practices, supporting incremental development and continuous delivery.

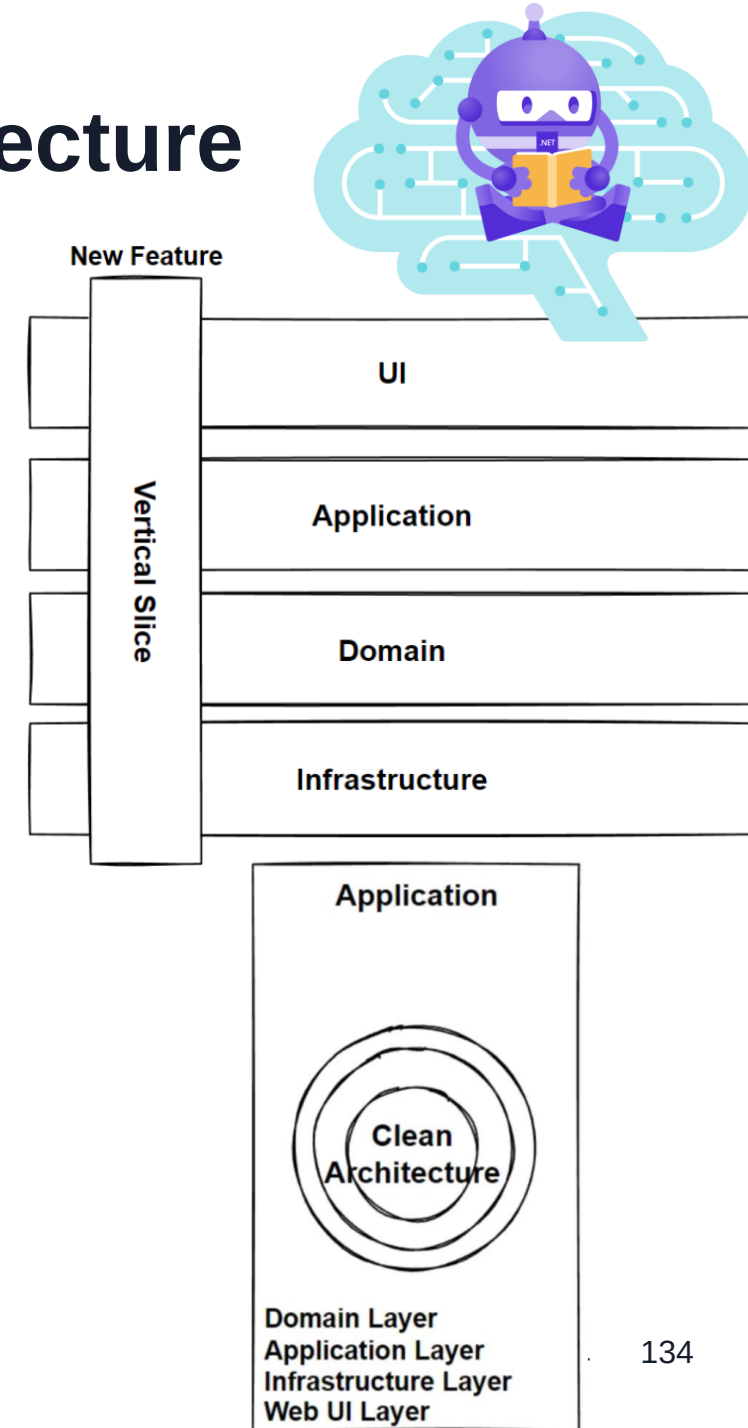
Challenges and Considerations

- Duplication of code across slices, particularly for common functionalities.
- Learning curve involved, especially for teams accustomed to traditional architectures.
- Design of each slice requires careful consideration to ensure independence and maintainability.



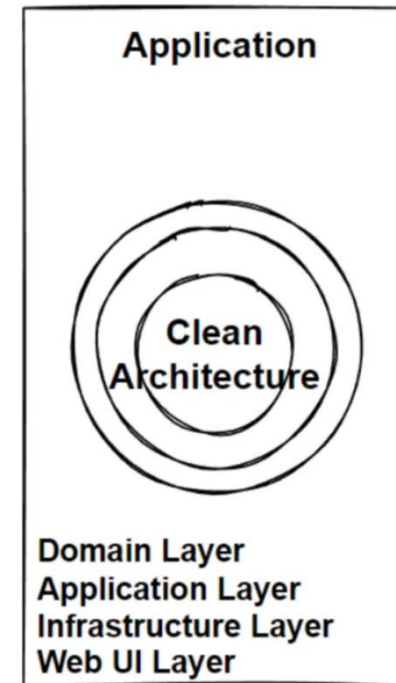
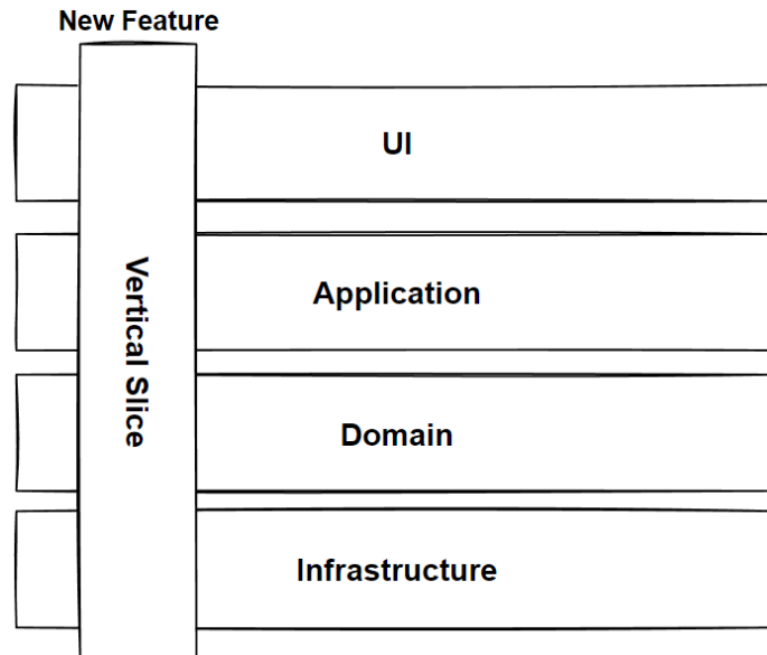
Vertical Slice Architecture vs. Clean Architecture

- **VSA** emphasizes **organizing** software development **around features**, cutting through all layers.
- **CA** focuses on **separation of concerns** and dependency rules. It **organizes** code into **layers**.
- **VSA**, development teams **concentrate** on **delivering complete features**, each potentially touching all layers of the stack.
- **CA** more **structured** approach, ensuring that **business logic** is **decoupled** from **external concerns**.
- **VSA** well-suited for **agile teams** working on **complex applications** with numerous features.
- **CA** ideal for **large-scale applications** where **long-term maintenance**, scalability, and the ability to **adapt to changing business requirements**.



When to Choose Vertical Slice over Clean Architecture

- **Rapid Development & Deployment:** When the priority is to develop and deploy features rapidly and independently.
- **Agile & Scrum Teams:** For teams practicing Agile or Scrum that need to deliver complete features in short cycles.
- **Microservices:** In a microservices architecture, where each service is often responsible for a distinct feature or business capability.



Eshop Microservices: Catalog vs Ordering

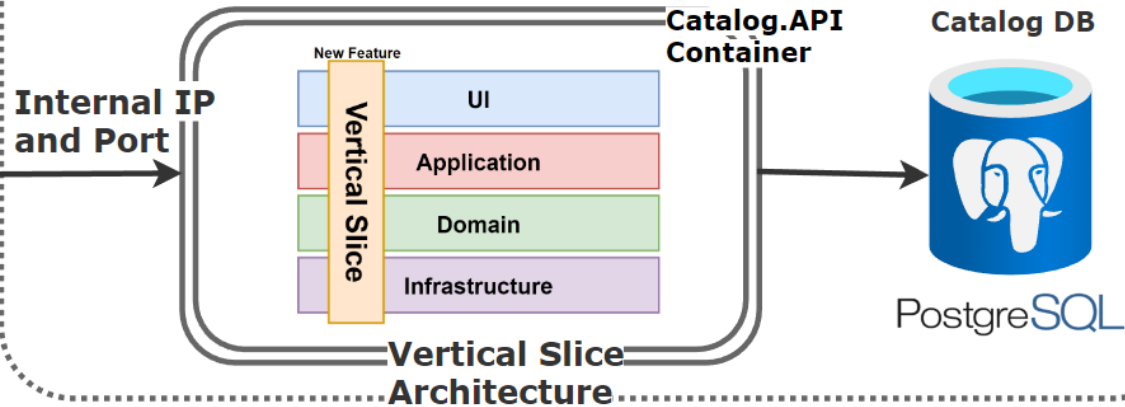


Catalog

Docker Host



Logical "Catalog" Microservice



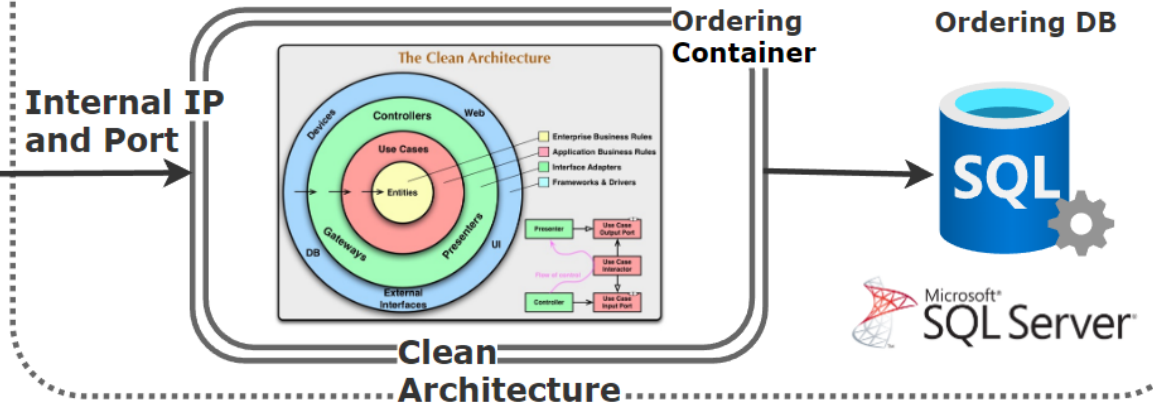
mehmetozkaya

Ordering

Docker Host

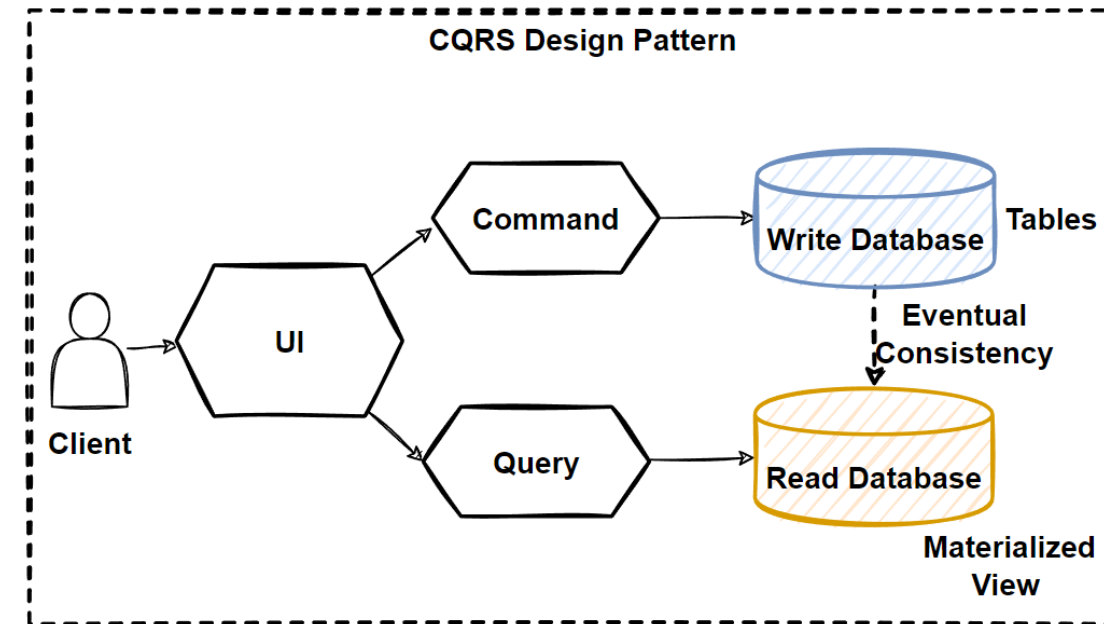


Logical "Ordering" Microservice



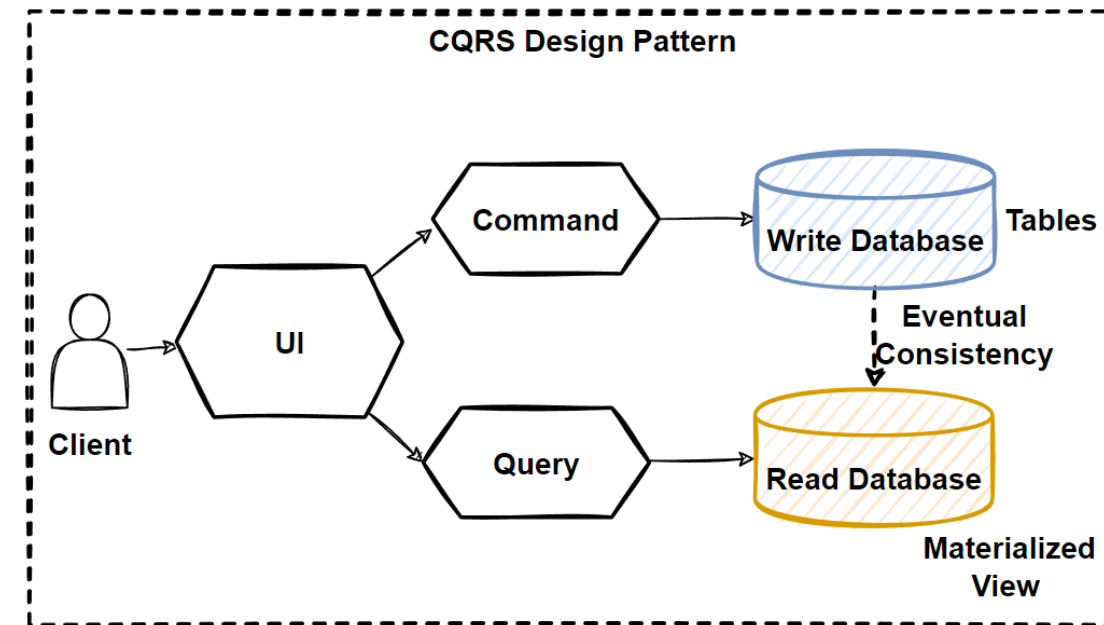
CQRS - Command Query Responsibility Segregation

- **CQRS design pattern** in order to avoid **complex queries** to get rid of **inefficient joins**.
- **Separates read and write operations** with separating databases.
- **Commands**: changing the state of data into application.
- **Queries**: handling complex join operations and returning a result and don't change the state of data into application.
- Large-scaled **microservices architectures** needs to manage **high-volume data requirements**.
- **Single database** for services can **cause bottlenecks**.
- Uses both **CQRS** and **Event Sourcing** patterns to improve application performance.
- **CQRS** offers to **separates read and write data** that provide to maximize query performance and scalability.



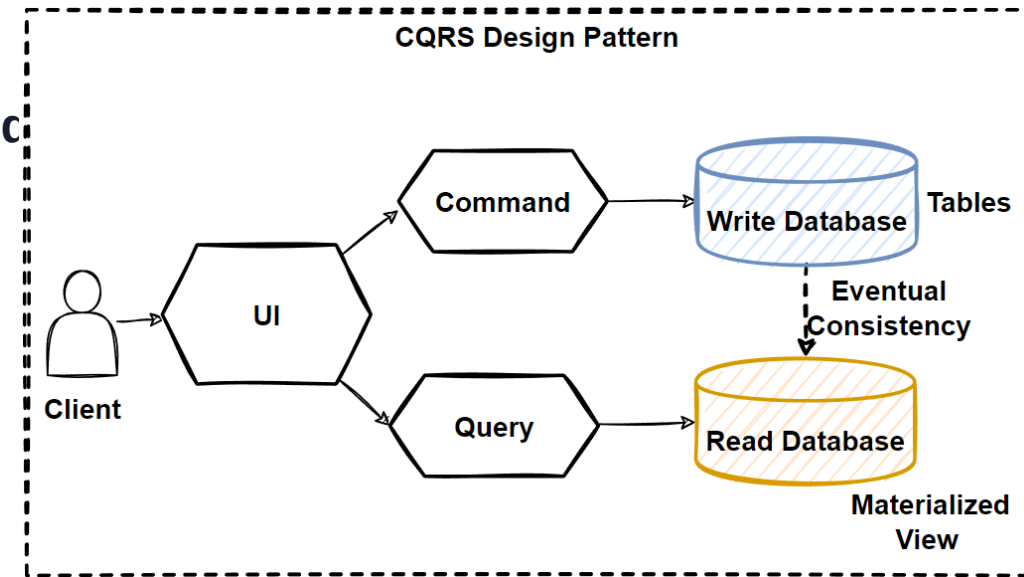
CQRS – Read and Write Operations

- **Monolithic** has **single database** is both working for **complex join queries**, and also perform **CRUD operations**.
- When **application** goes **more complex**, this **query** and **CRUD** operations will become **un-manageable situation**.
- Application required some **query** that **needs** to **join** more than **10 table**, will **lock** the **database** due to **latency** of **query computation**.
- Performing **CRUD operations** need to make **complex validations** and **process long business logics**, will cause to **lock database** operations.
- **Reading** and **writing database** has different approaches, define different strategy.
- «**Separation of concerns**» principles: separate reading database and the writing database with 2 database.
 - **Read database** uses No-SQL databases with denormalized data.
 - **Write database** uses Relational databases with fully normalized and supports strong data consistency.



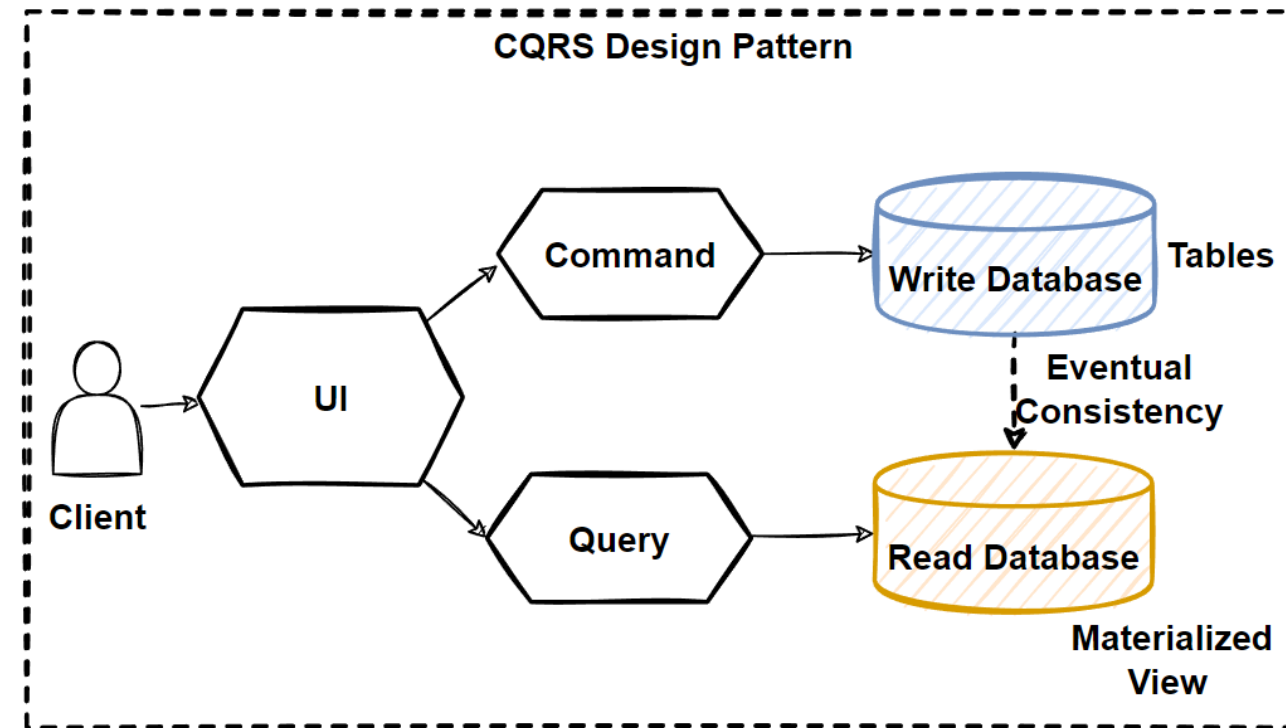
CQRS – Read and Write Operations - 2

- If our **application** is **mostly reading use cases** and not writing so much, it is **read-incentive application**.
- **Read** and **write operations** are **asymmetrical** and has very different performance and scale requirements.
- To **improve query performance**, the read operation perform **queries** from a **highly denormalized materialized views** to **avoid** expensive repetitive **table joins** and **table locks**.
- **Write operation** which is **command operation**, can perform commands into separate **fully normalized relational database**.
- Supporting **ACID transactions** and **strong data consistency**.
- Commands: **task-based operations** like "add item into shopping cart" or "checkout order".
- **Commands** can be handle with message broker systems that provide to process commands in async way.
- **Queries:** never modify the database, always return the JSON data with DTO objects.



CQRS – Synchronization with Read and Write DB

- Keep sync both read and write databases.
- Publishes an event that subscribe from **Read Database** and **update** the **read table** accordingly.
- **Synchronization** handles with async communication using message brokers.
- This creates **Eventual Consistency principle**.
- The **Read** database **eventually synchronizes** from the **Write** database.
- **Some lag** in the process due to **async communication** with message brokers that applies publish/subscribe pattern.
- Welcome to **Eventual consistency**.



Benefits of CQRS

- **Scalability**

When we separate Read and Write databases, we can also scale these separate databases independently. Read databases follows denormalized data to perform complex join queries.

- If application is read-incentive application, we can scale read database more then write database.

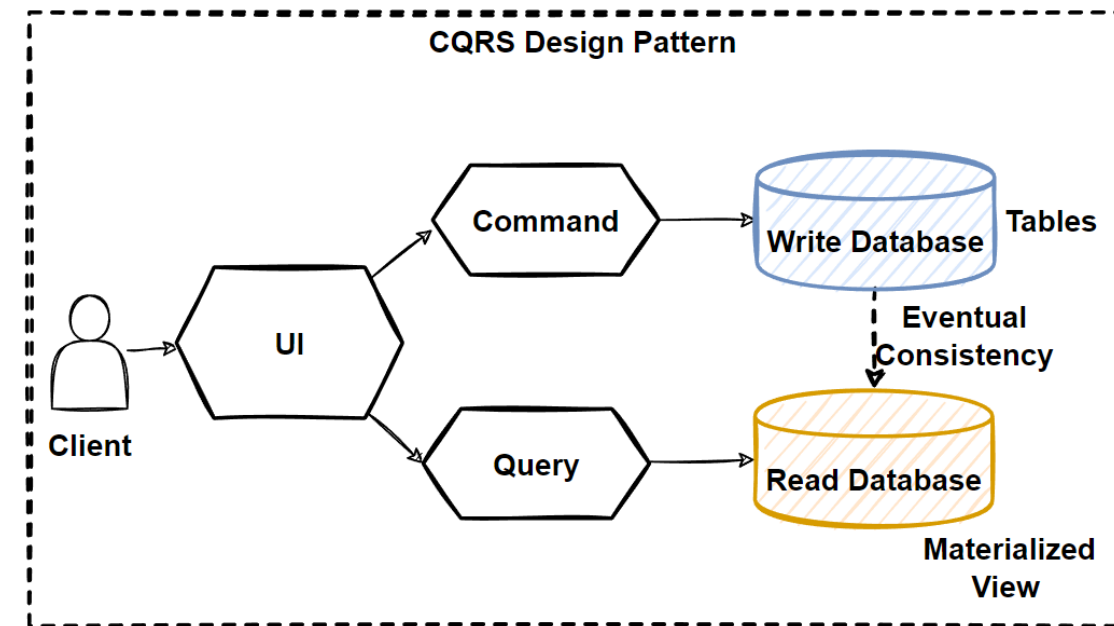
- **Query Performance**

The read database includes denormalized data that reduce to comlex and long-running join queries. Complex business logic goes into the write database. Improves application performance for all aspects.

- **Maintability and Flexibility**

Flexibility of system that is better evolve over time and not affected to update commands and schema changes by separating read and write concerns into different databases.

- Better implemented if we physically separate the read and write databases.



Drawbacks of CQRS

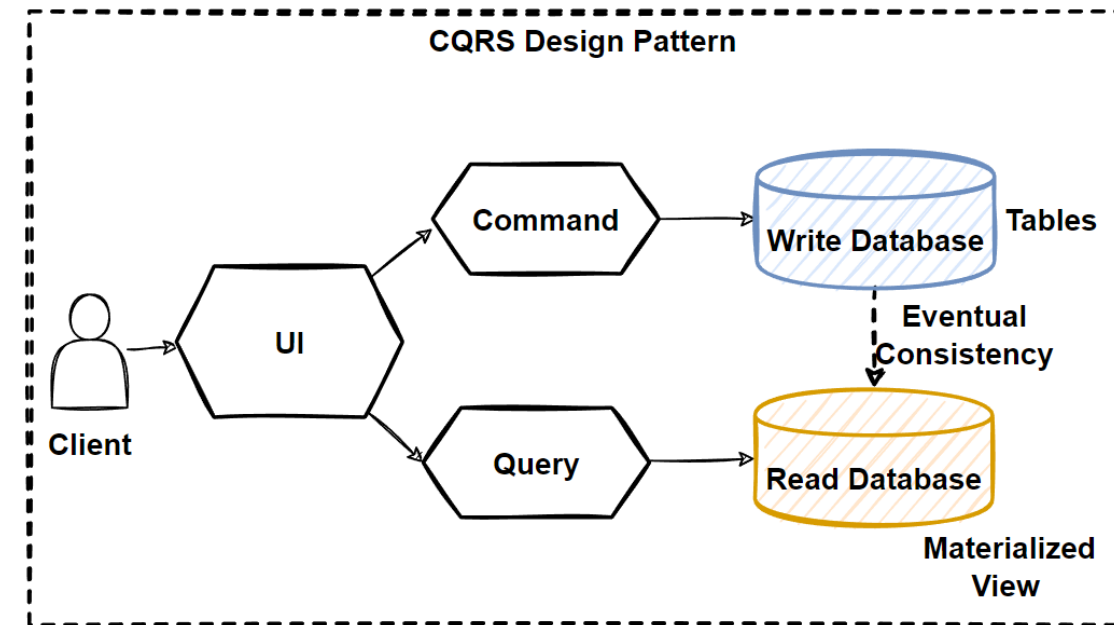
- **Complexity**

CQRS makes your system more complex design. Strategically choose where we use and how we can separate read and write database.

- **Eventual Consistency**

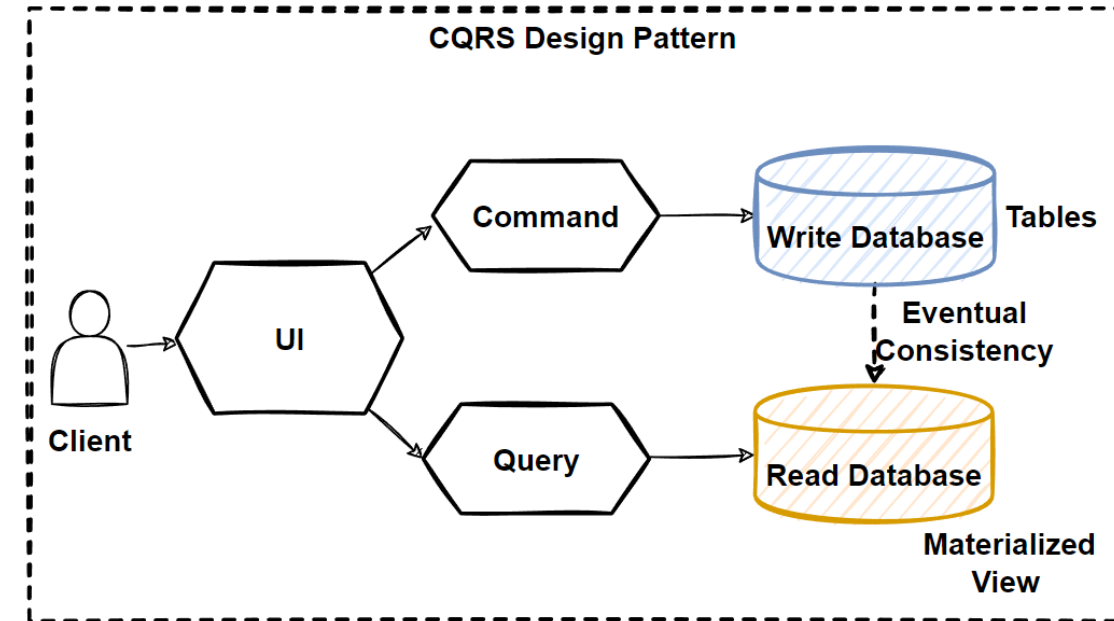
The read data may be stay old and not-updated for a particular time. So the client could see old data even write database updated, it will take some time to update read data due to publish/subscribe mechanism.

- We should **embrace** the **Eventual Consistency** when using **CQRS**, if your application **required strong consistency** than **CQRS is not good to apply**.



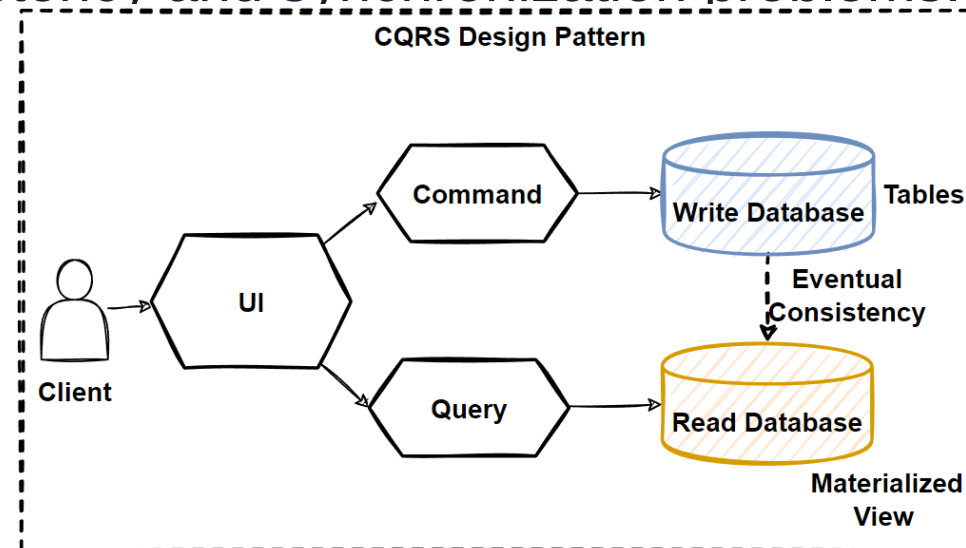
Best Practices for CQRS

- **Best practices to separate read and write database with 2 database physically.**
- **Read-intensive** that means **reading more than writing**, can define **custom data schema to optimized for queries.**
- **Materialized View Pattern** is good example to implement reading databases.
- **Avoid complex joins** and mappings with **pre-defined fine-grained data for query operations.**
- **Use different database** for reading and writing database types.
- Using **No-SQL** document database **for reading** and using **Relational** database **for CRUD** operations.



Logical and Physical Implementation of CQRS

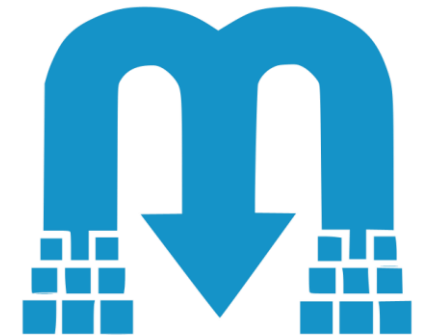
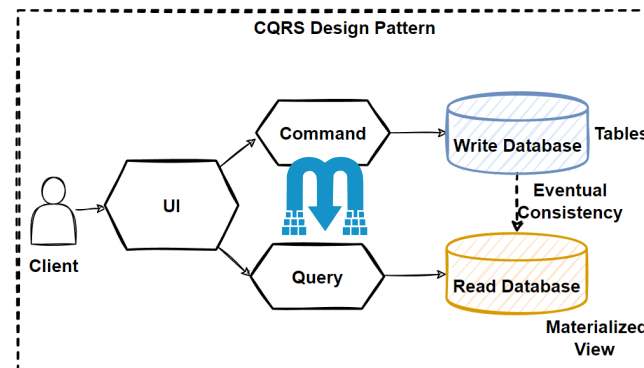
- **Logical Implementation of CQRS:** Splitting Operations, Not Databases. Separate the read (query) operations from the write (command) operations at the code level, but not necessarily at the database level.
- Even though the same database is used, the paths for reading and writing data are distinct.
- **Physical Implementation of CQRS:** Separate Databases. Splitting the read and write operations not just at the code level but also physically using separate databases.
- Introduces data consistency and synchronization problems.



CQRS Design Pattern With MediatR Library

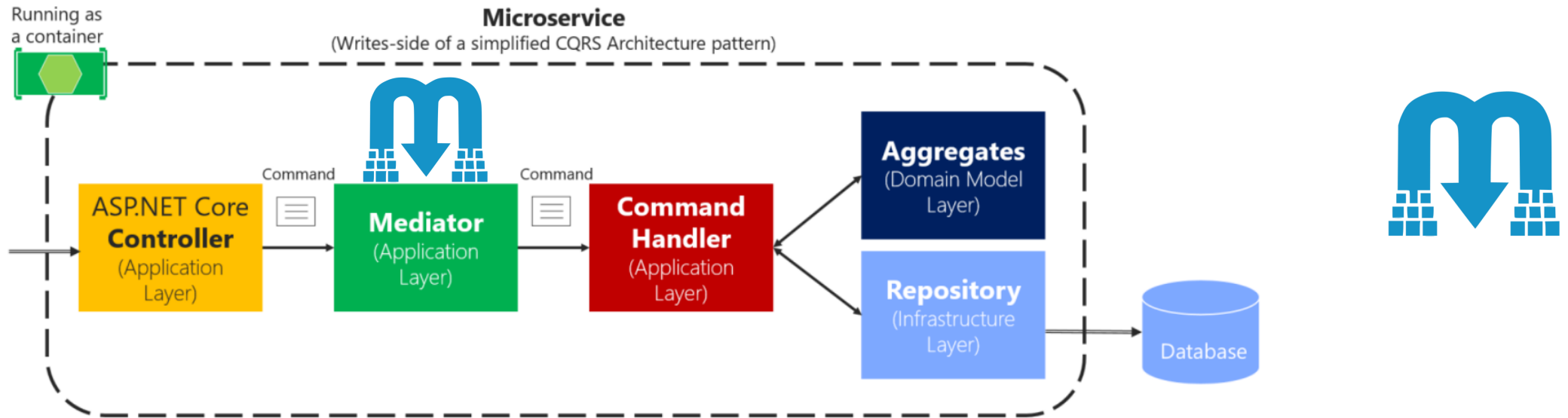


- **MediatR** is a popular .NET library used to implement the mediator pattern, which is a great fit for CQRS.
- **IRequest interface** is used to define a request, which can be either a command or a query. The return type of the request can be specified as a generic parameter.
- **Handler** inherits from **IRequestHandler<TRequest, TResponse>**, where TRequest is the type of the command or query, and TResponse is the return type.
- To make the distinction between commands and queries clearer, you can define two custom interfaces:
 ICommand and **IQuery**.
 - `public interface ICommand<TResult> : IRequest<TResult> { }`
 - `public interface IQuery<TResult> : IRequest<TResult> { }`



MediatR and ASP.NET Core Minimal Apis

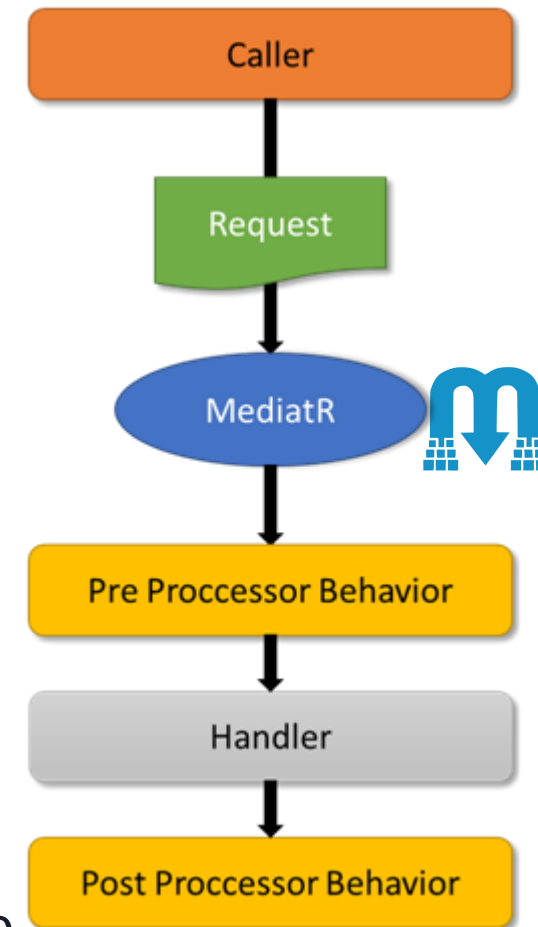
- ASP.NET Core using MediatR, minimal apis act as the entry point for handling HTTP requests. Instead of implementing the business logic directly, these actions delegate the responsibility to MediatR.



- Controller receives a request, it **creates** a **command** or **query** object and sends it to **MediatR**. MediatR then **dispatches** this object to the **appropriate handler**.

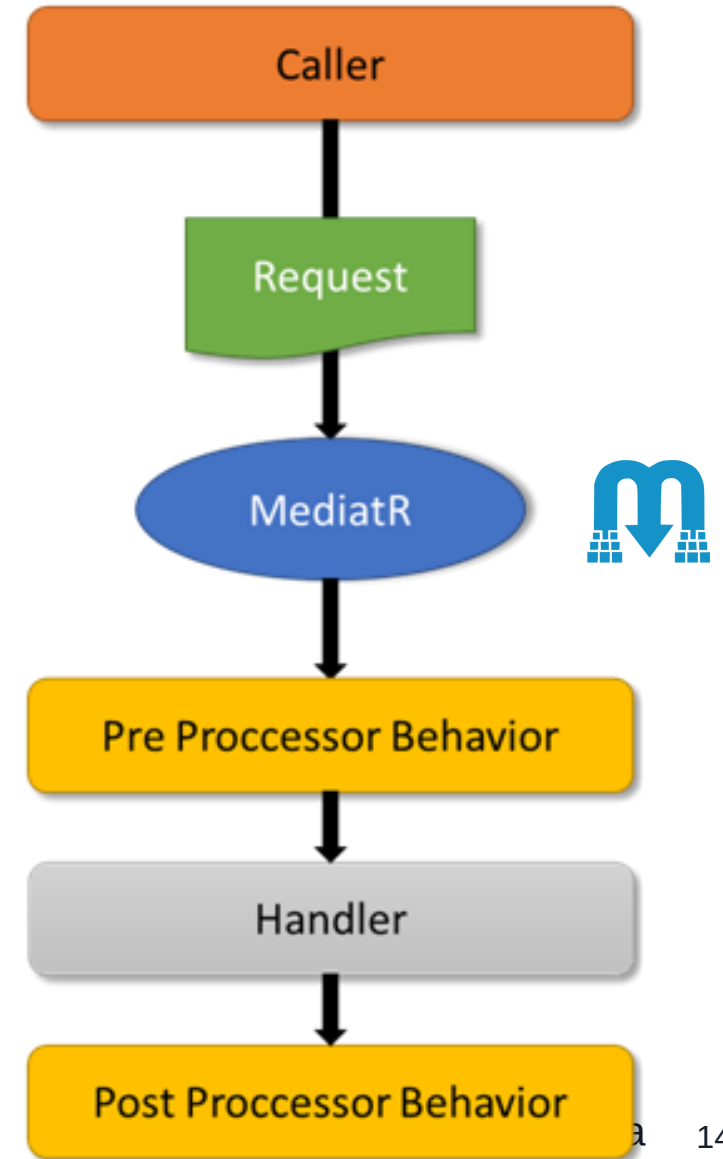
Mediator Pattern and Pipeline Behaviours

- **Mediator pattern** useful in **complex or enterprise-level** applications where request processing often involves more than just business logic.
- **Handling a request** require **additional steps** like logging, validation, auditing, and applying security checks. These are known as **cross-cutting concerns**.
- MediatR provides a **mediator pipeline** where these cross-cutting concerns can be inserted transparently.
- Pipeline **coordinates** the **request handling**, ensuring that all necessary steps are executed in the right order.
- In **MediatR**, **pipeline behaviors** are used to implement cross-cutting concerns.
- **Wrap** around the **request handling**, allowing you to execute logic before and after the actual handler is called.

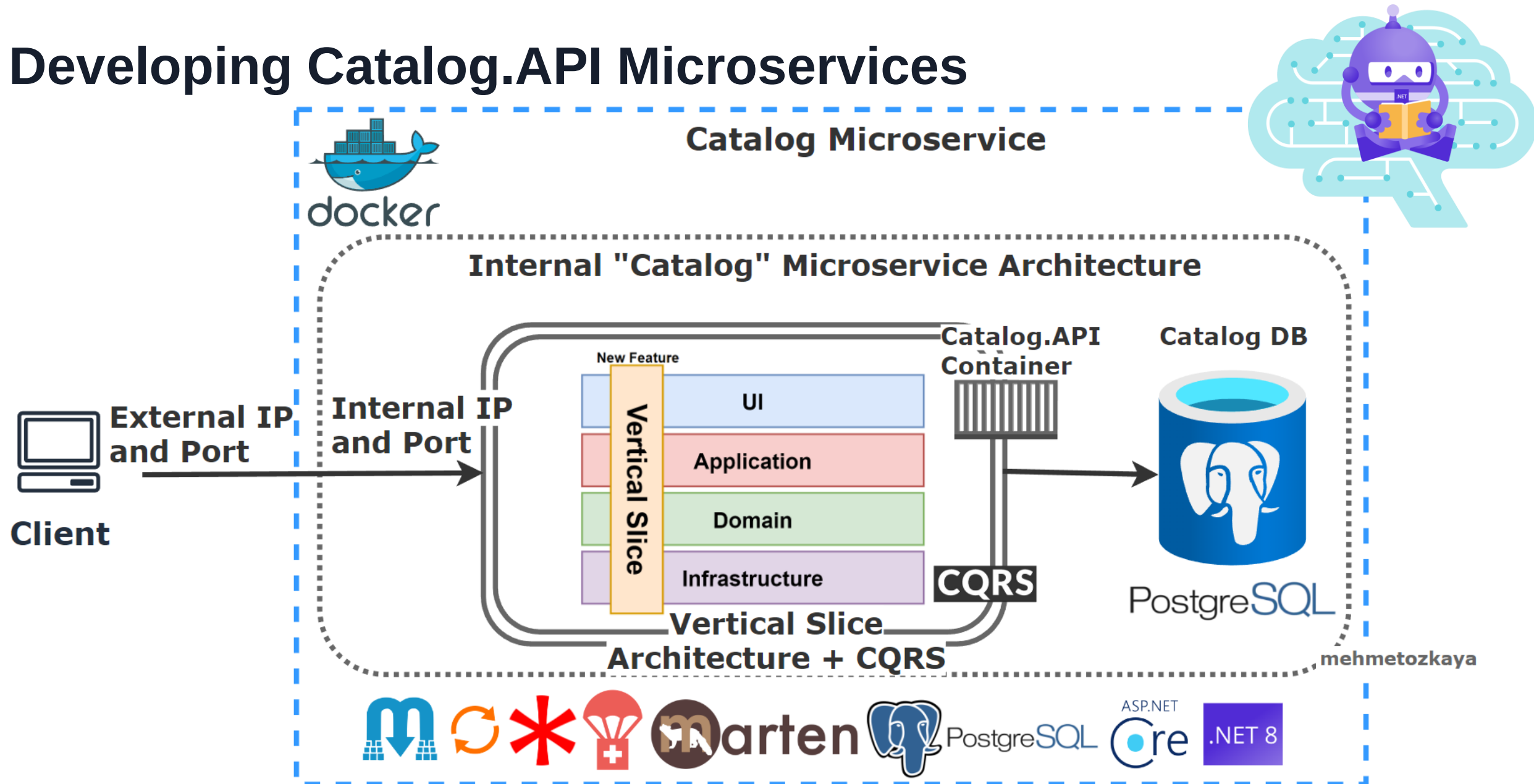


EShop Microservices Pipeline Behaviours

- **LogBehavior:** A behavior that logs details about the handling of a request.
- **ValidatorBehavior:** A behavior that validates incoming requests before they reach the handler.
- Every request processed by MediatR in the application is logged and validated consistently.
- Using MediatR with the mediator pattern in ASP.NET Core applications provides a structured and clean way to handle complex request processing.

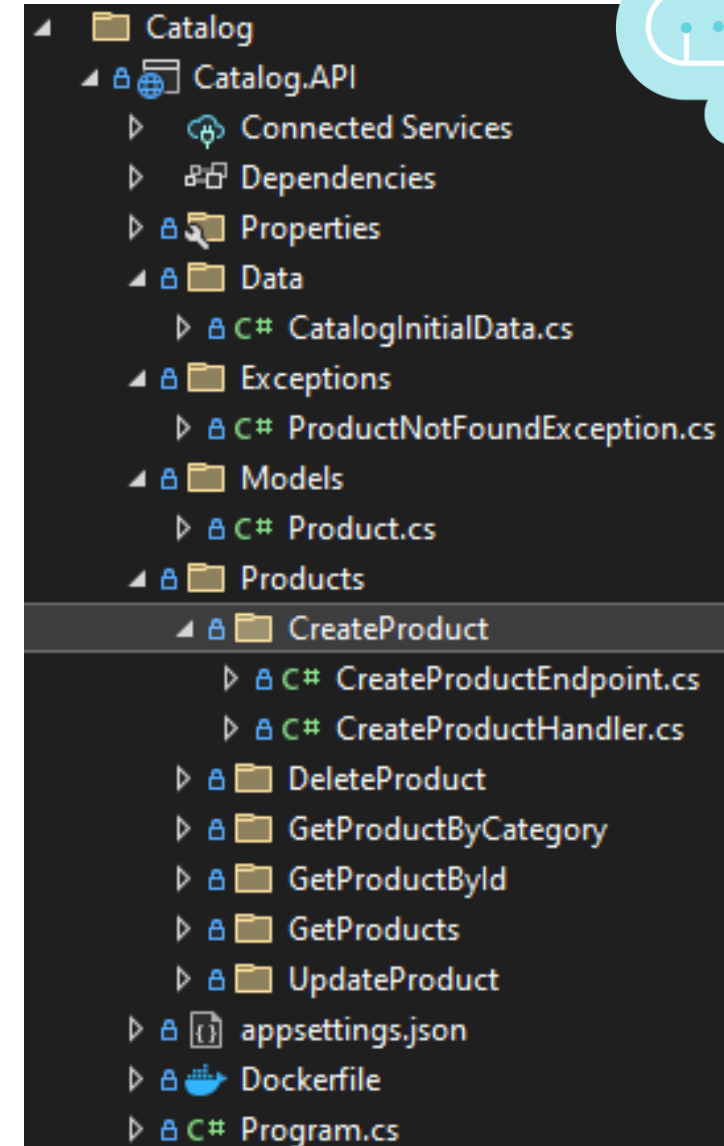
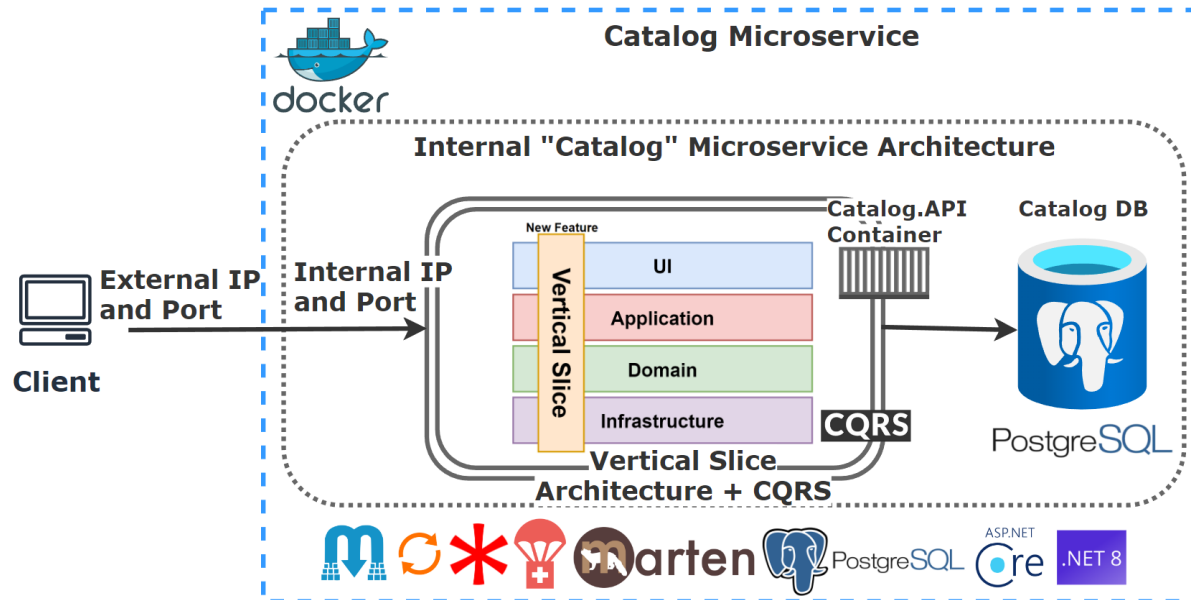


Developing Catalog.API Microservices

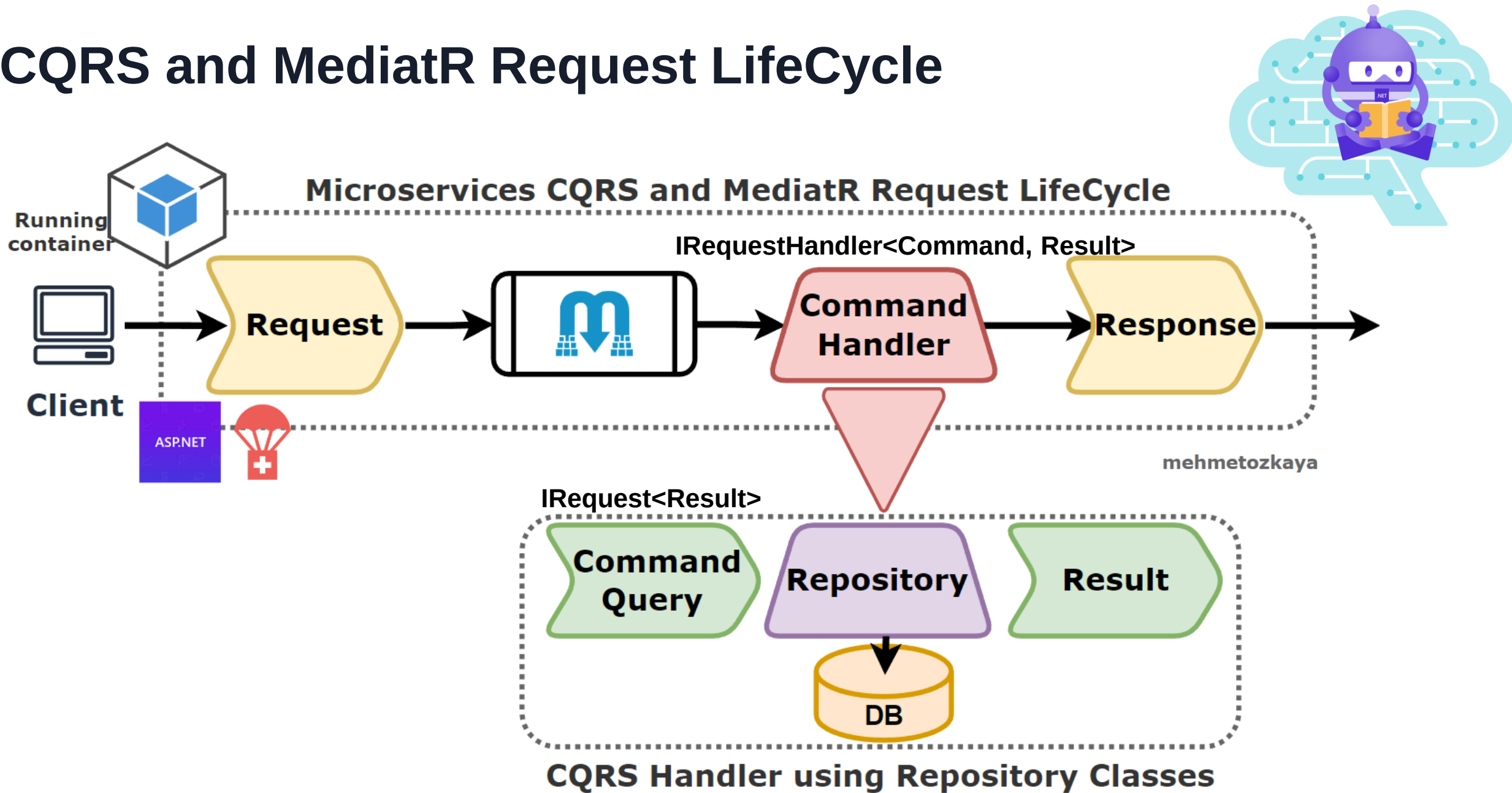


4. Project Folder Structure of Catalog Microservice

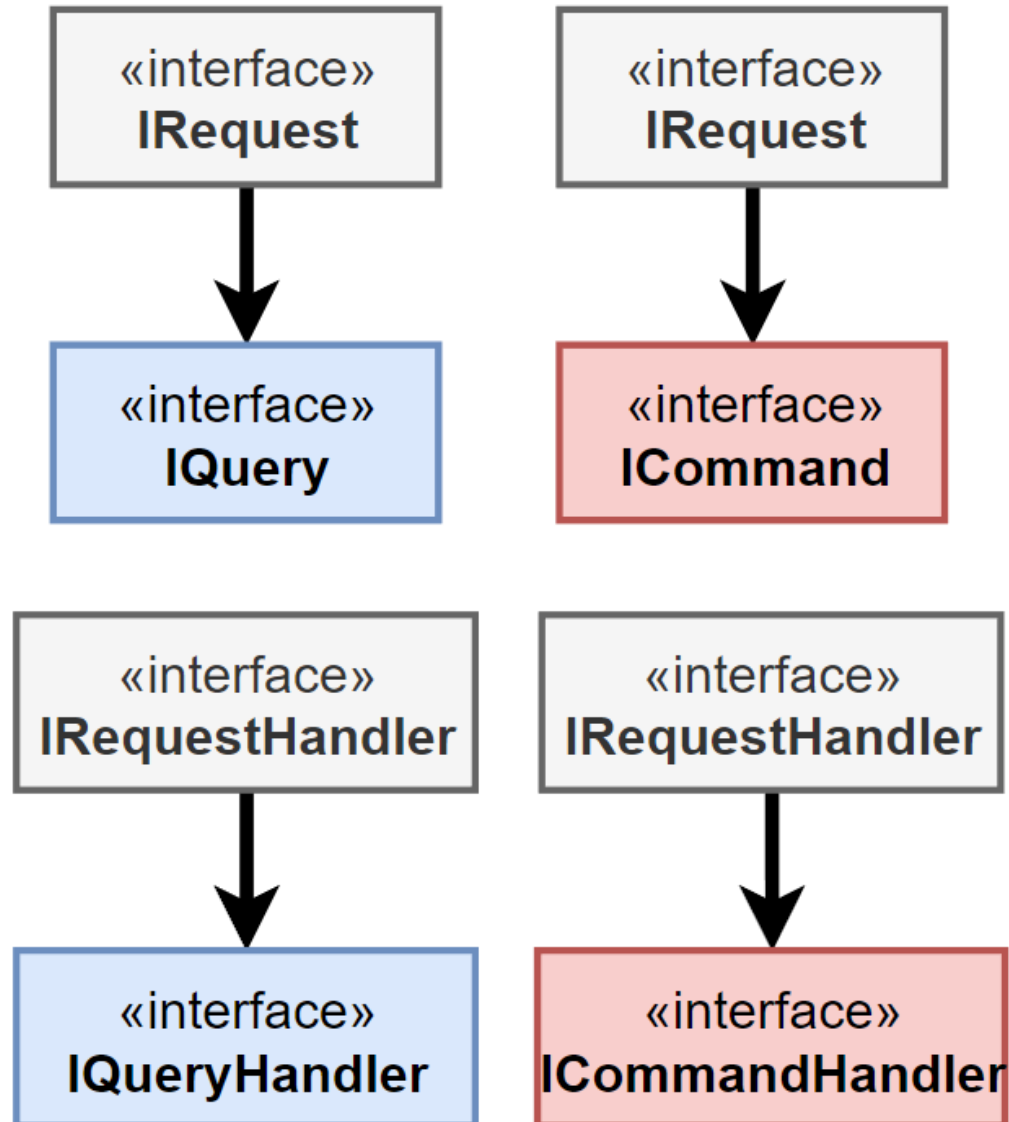
- The project is organized into **Model**, **Features**, **Data**, and **Abstractions**.
- **Features** like **CreateProduct** and **GetProduct** have dedicated handlers and endpoint definitions.
- **Feature** folder will be **Products**.
- **Data folder** and **Context objects** manages database interactions.



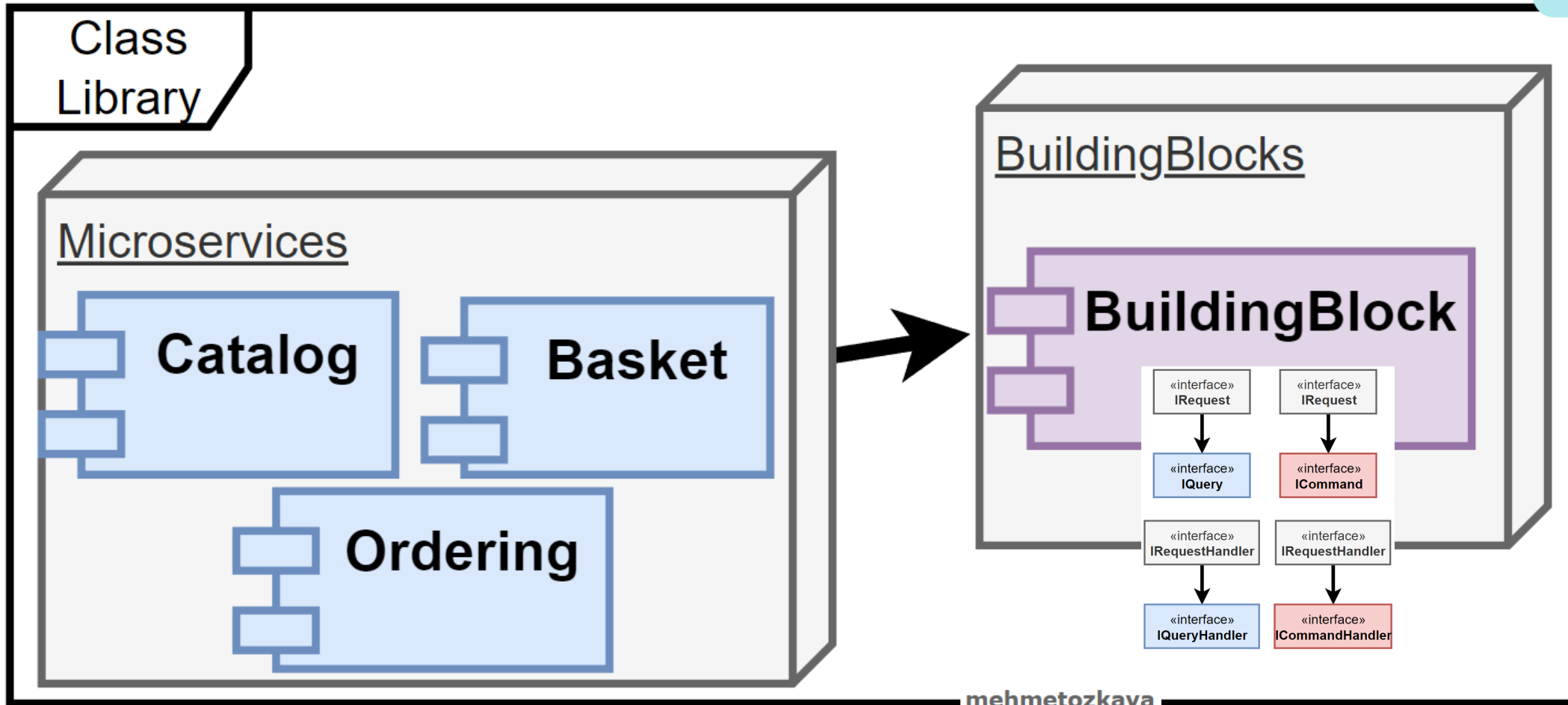
CQRS and MediatR Request LifeCycle



Create Abstraction on MediatR for CQRS



Eshop Microservices uses BuildingBlocks Library

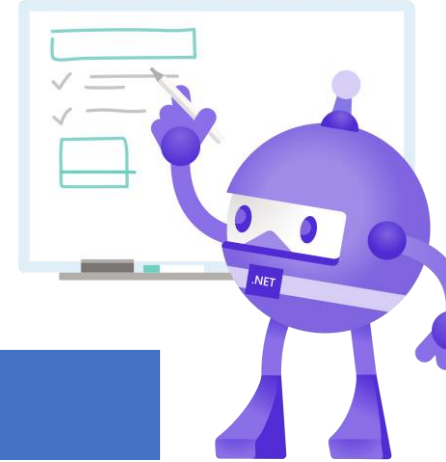


CQRS



mehmetozkaya

3. Rest API Endpoints of Catalog Microservices



Method	Request URI	Use Cases
GET	/products	List all products
GET	/products/{id}	Fetch a specific product
GET	/products/category	Get products by category
POST	/products	Create a new product
PUT	/products/{id}	Update a product
DELETE	/products/{id}	Remove a product

Carter Library for Minimal Api Endpoint Definition

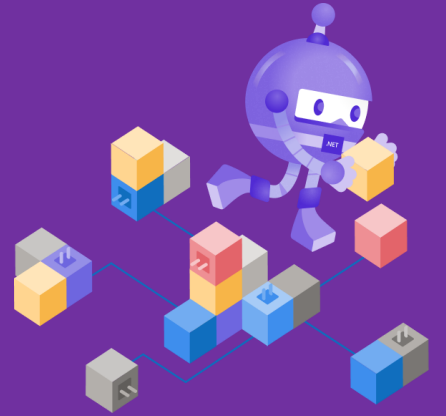
- **With ASP.NET Core 8, Minimal APIs** have become a popular way to build lightweight and performant microservices and web APIs.
- **Carter** is a library that **extends the capabilities** of ASP.NET Core's Minimal APIs.
- Provides a **more structured way** to **organize** our **endpoints** and simplifies the creation of HTTP request handlers.
- **Carter** is framework that is a **thin layer of extension methods** and functionality over ASP.NET Core.
- Carter, especially **beneficial in minimal APIs**, simplify the development of minimal API endpoints.
- **Goto Github page:**
- <https://github.com/CarterCommunity/Carter>



Develop Catalog.API Infrastructure, Handler and Endpoint Classes for CRUD

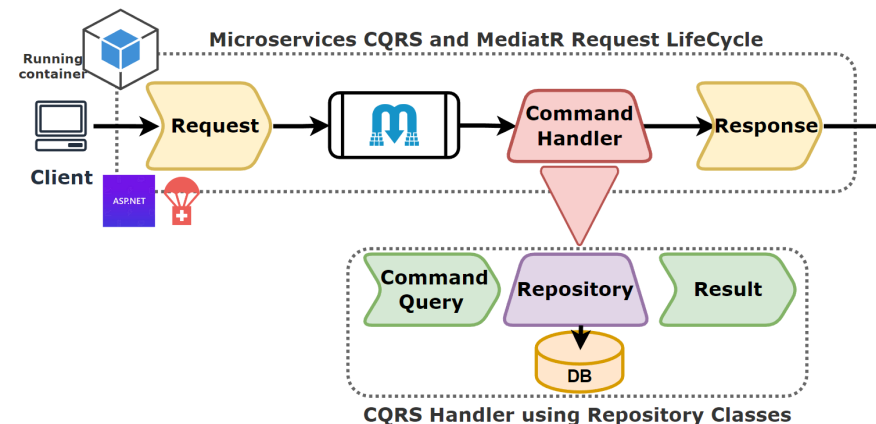
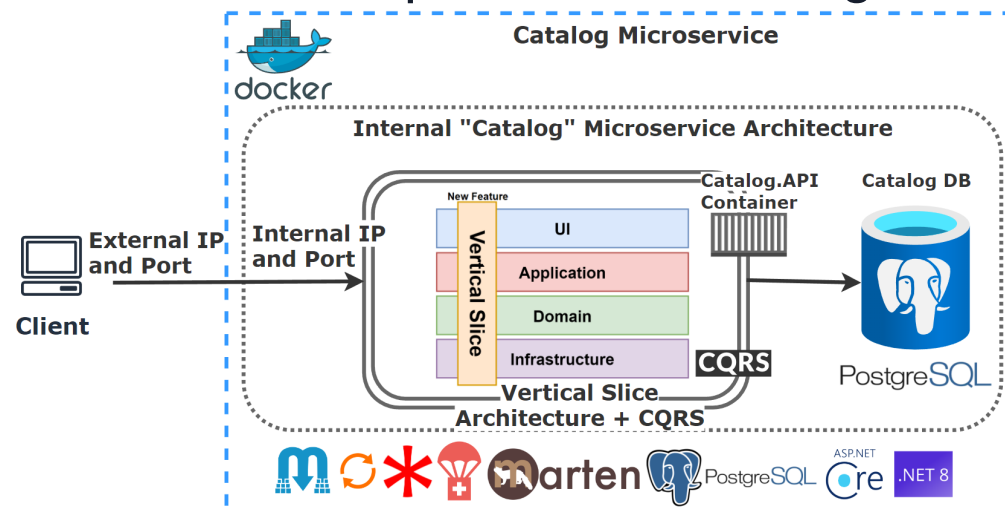
Infrastructure - Data Concerns for Catalog API - Marten .NET
Transactional Document DB for PostgreSQL Database

Mehmet Ozkaya

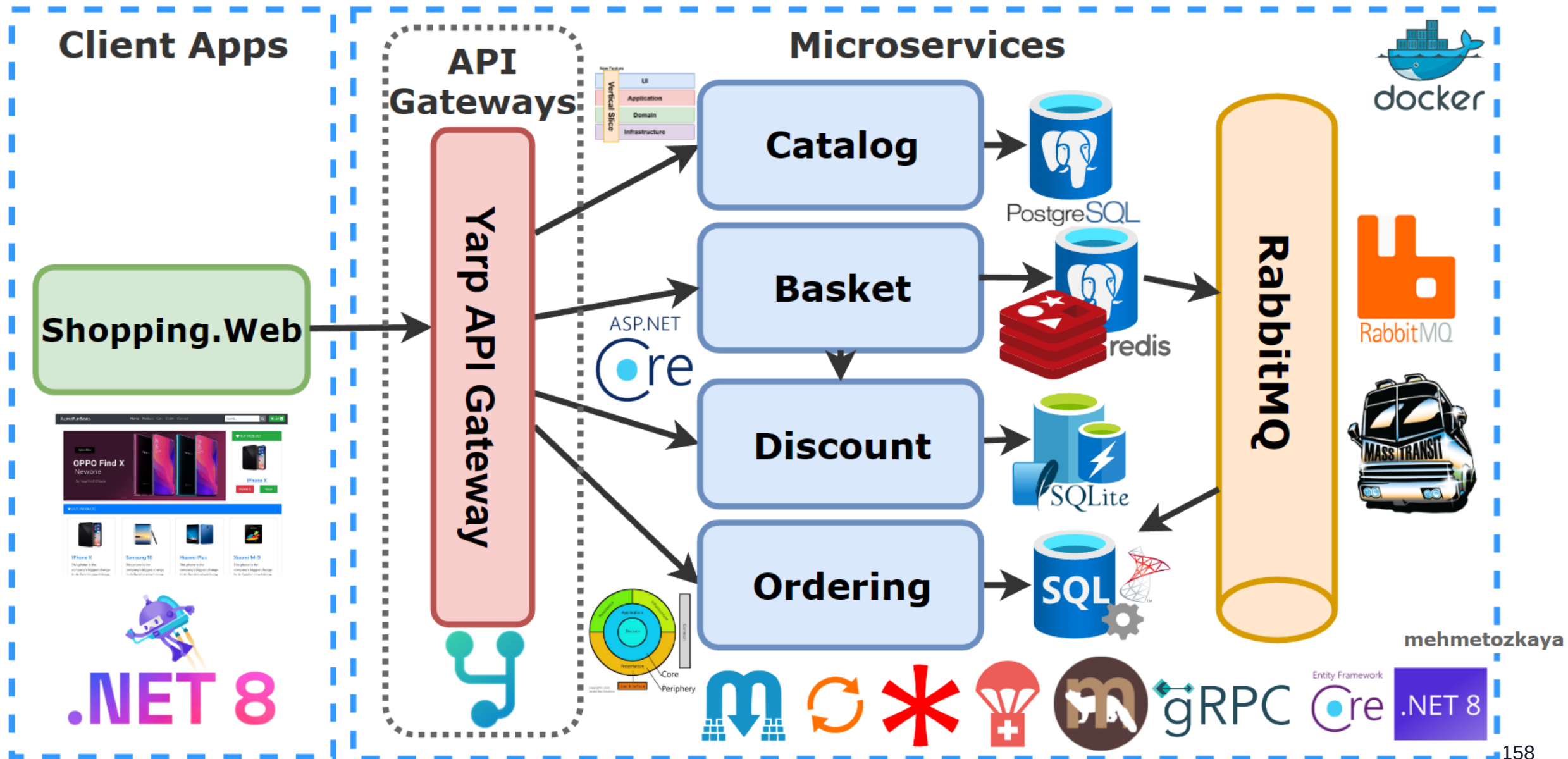


Develop Catalog.API Infrastructure, Handler and Endpoint

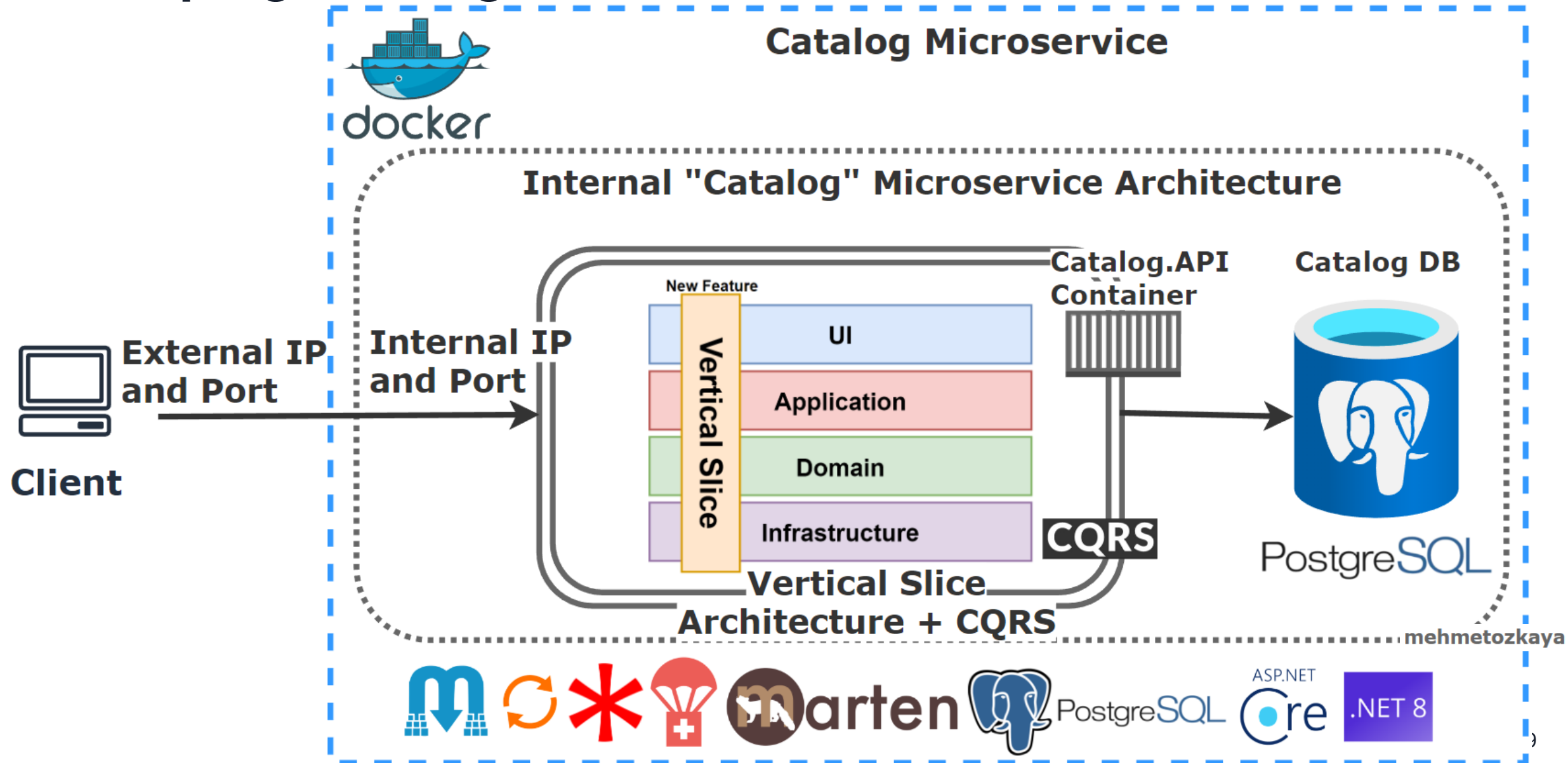
- Infrastructure - Data Concerns for Catalog API
- Marten .NET Transactional Document DB for PostgreSQL Database,
- Setup and Run PostgreSQL DB using Docker-compose file for Multi-container docker environment
- Develop Products Features in Vertical Slice with CQRS, MediatR in Handler class
- Develop Products Endpoints with Minimal Apis and Carter
- Test Product Endpoints Connecting to Docker Postgres Container



Big Picture – Course Project

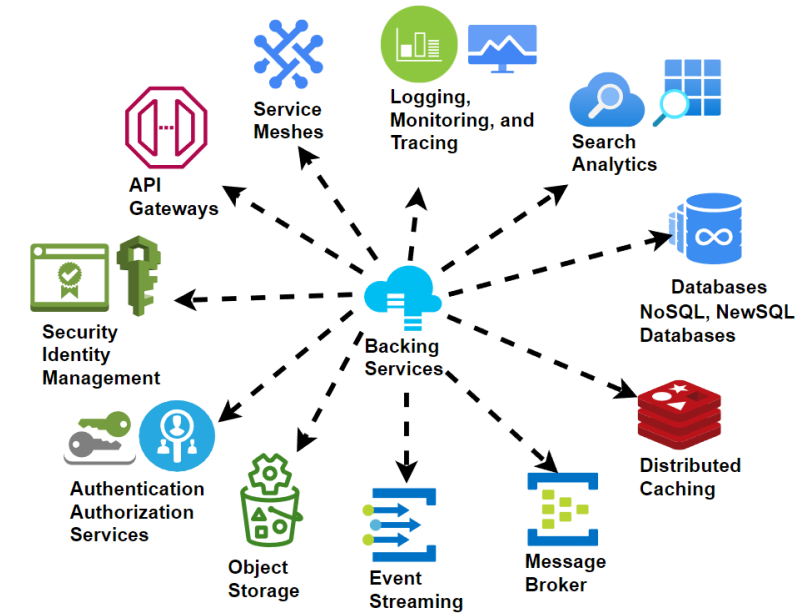


Developing Catalog.API Microservices

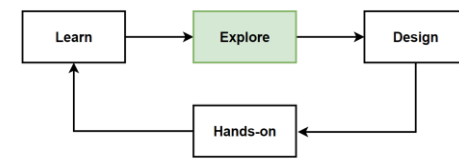
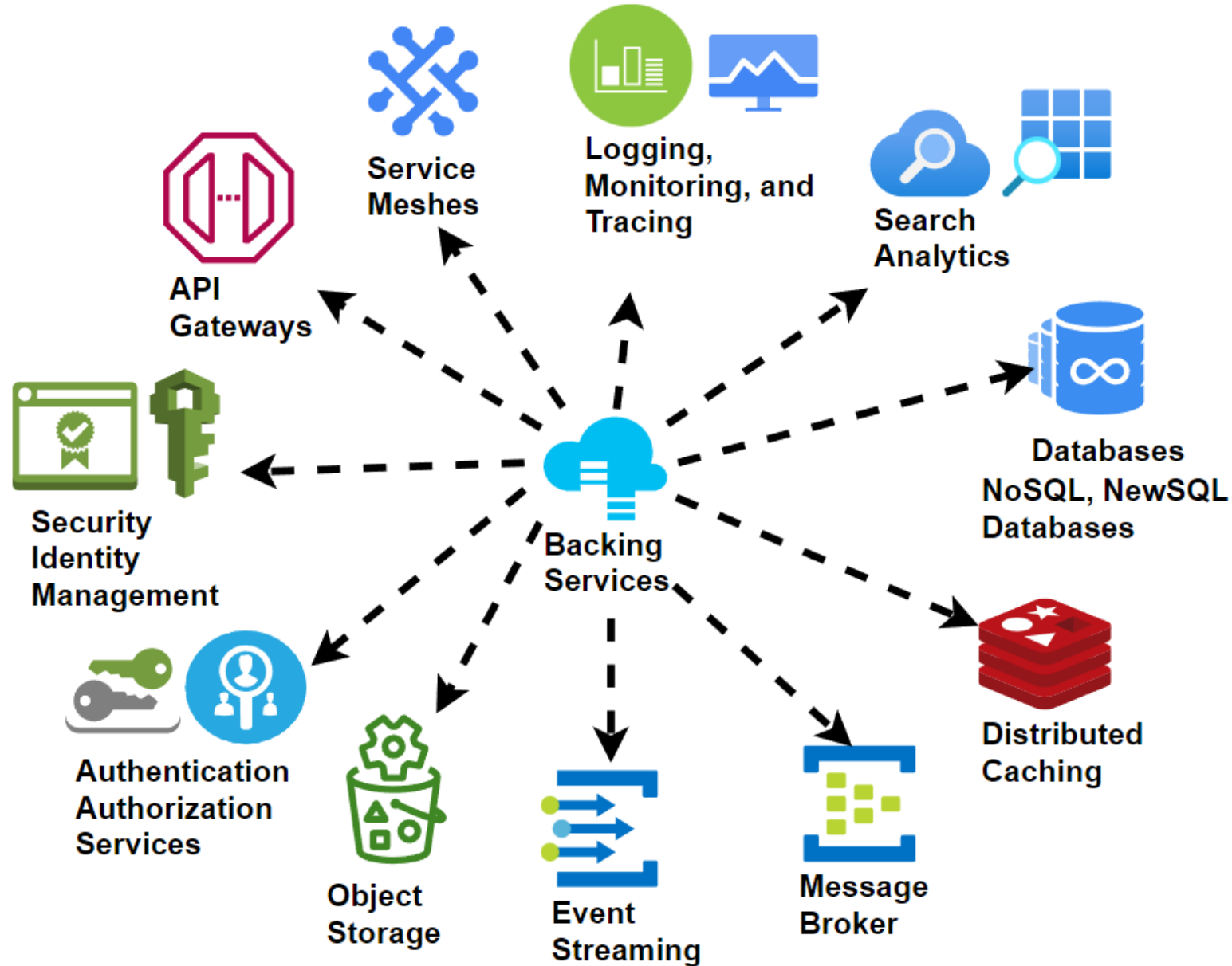


What is Backing Services for Cloud-Native Microservices

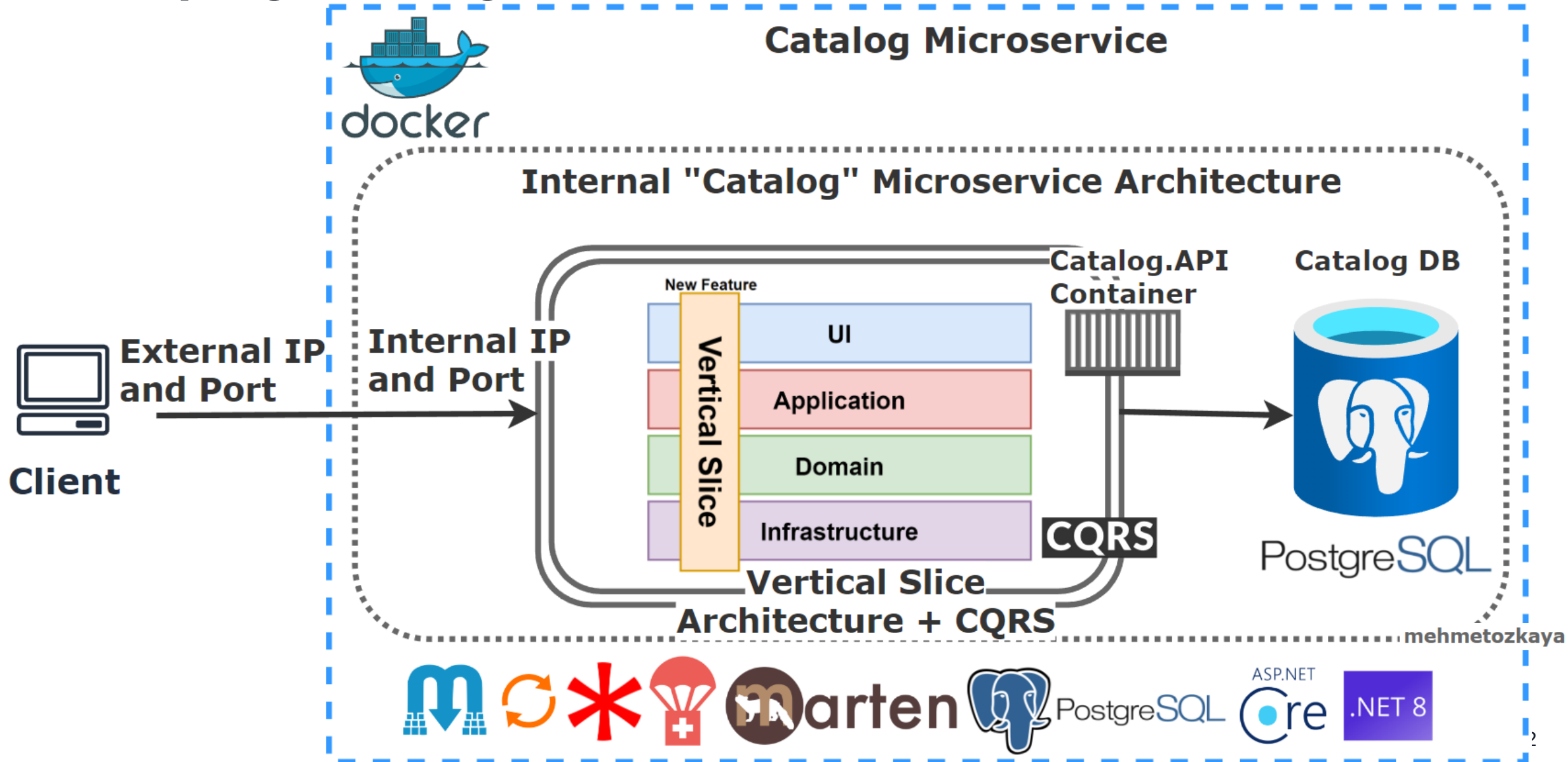
- **Backing services** for **cloud-native microservices** are the **external components** that microservices depend on for their operation.
- Provide support for **various functionalities**, such as **data storage, messaging, caching, and authentication**.
- **Backing services** like databases, messaging systems, and caching services are **treated as attached resources**.
- **Backing services** are external to the microservice and can be **swapped or replaced without changing** the microservice's core logic.
- **Decoupled from the microservices** themselves, promoting flexibility, scalability, and easier maintenance.



Backing Services for Cloud-Native Microservices



Developing Catalog.API Microservices

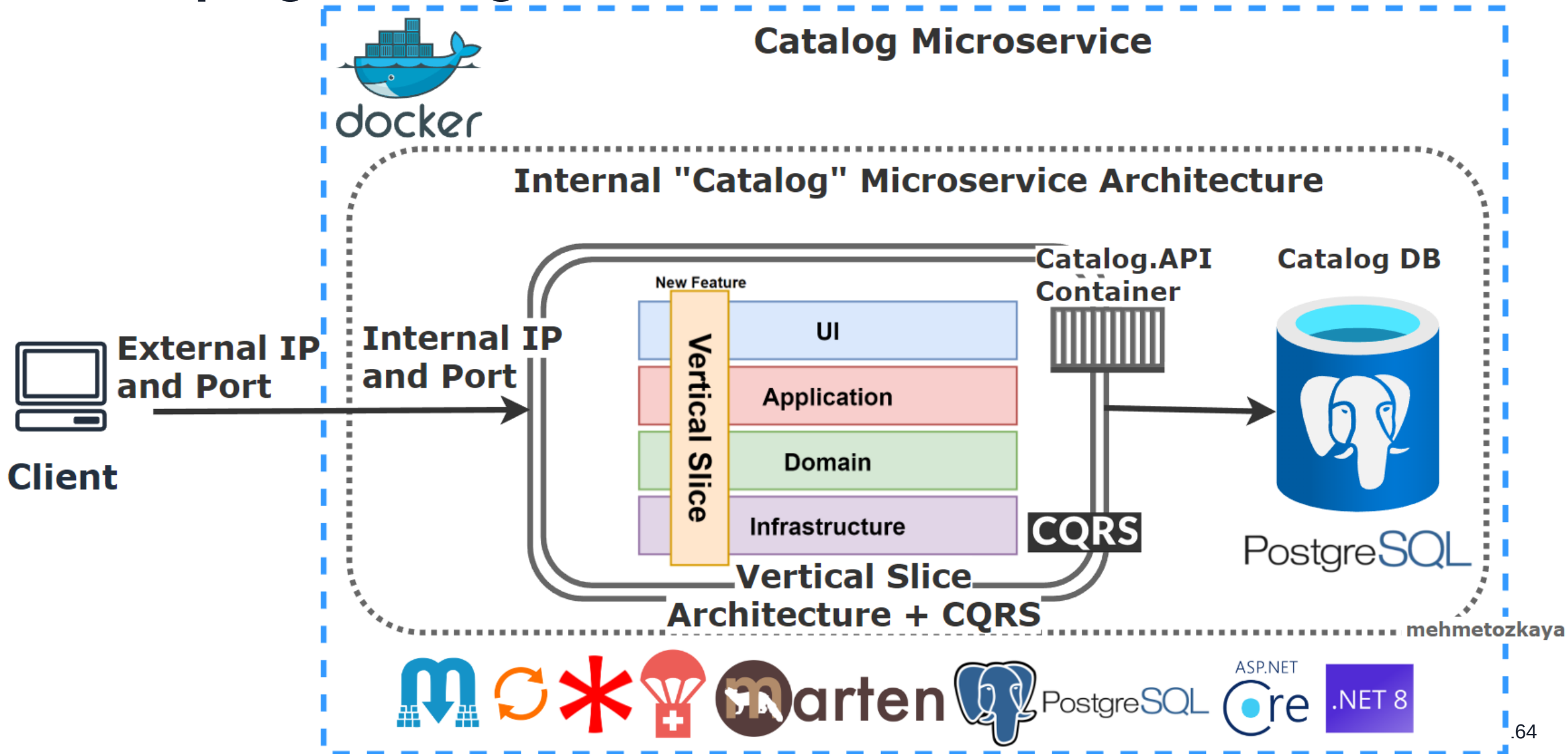


4. Underlying Data Structures of Catalog Microservices

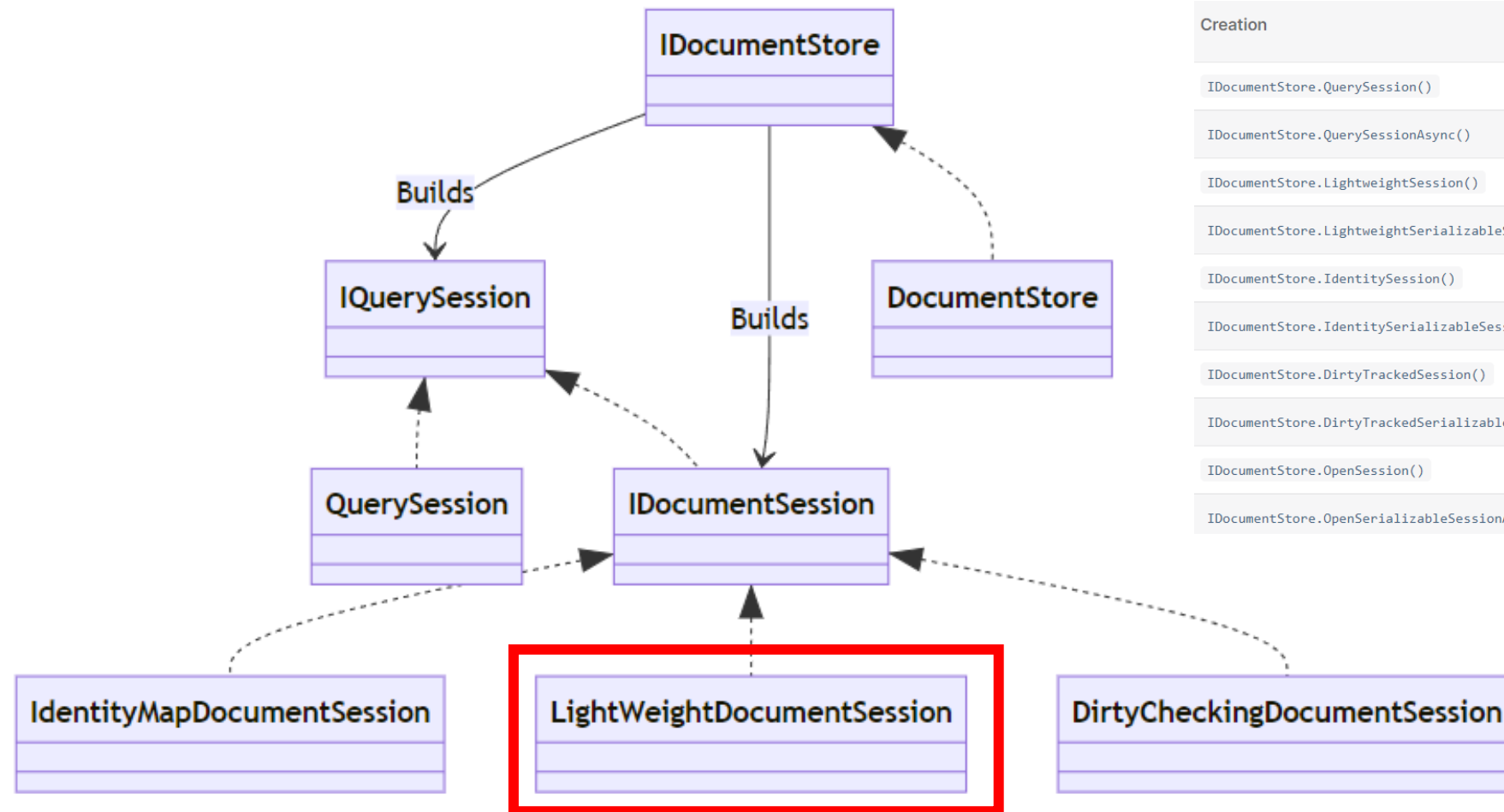
- **Marten** is an **ORM (Object-Relational Mapper)** that leverages **PostgreSQL's JSON** capabilities.
- **Marten** is a powerful library that transforms **PostgreSQL** into a **.NET Transactional Document DB**.
- **PostgreSQL's JSON column features**, allowing us to store and query our data as JSON documents.
- Combines the **flexibility** of a **document database** with the reliability of **relational PostgreSQL** database.
- **Catalog** service use **Marten** for **PostgreSQL** Interaction as a **Document DB**.



Developing Catalog.API Microservices



Opening Sessions in Marten as Document DB



Creation	Read/Write	Identity Map	Dirty Checking
<code>IDocumentStore.QuerySession()</code>	Read Only	No	No
<code>IDocumentStore.QuerySessionAsync()</code>	Read Only	No	No
<code>IDocumentStore.LightweightSession()</code>	Read/Write	No	No
<code>IDocumentStore.LightweightSerializableSessionAsync()</code>	Read/Write	No	No
<code>IDocumentStore.IdentitySession()</code>	Read/Write	Yes	Yes
<code>IDocumentStore.IdentitySerializableSessionAsync()</code>	Read/Write	Yes	Yes
<code>IDocumentStore.DirtyTrackedSession()</code>	Read/Write	Yes	Yes
<code>IDocumentStore.DirtyTrackedSerializableSessionAsync()</code>	Read/Write	Yes	Yes
<code>IDocumentStore.OpenSession()</code>	Read/Write	Yes	No
<code>IDocumentStore.OpenSerializableSessionAsync()</code>	Read/Write	Yes	No

***Best practice**

<https://martendb.io/documents/sessions.html>

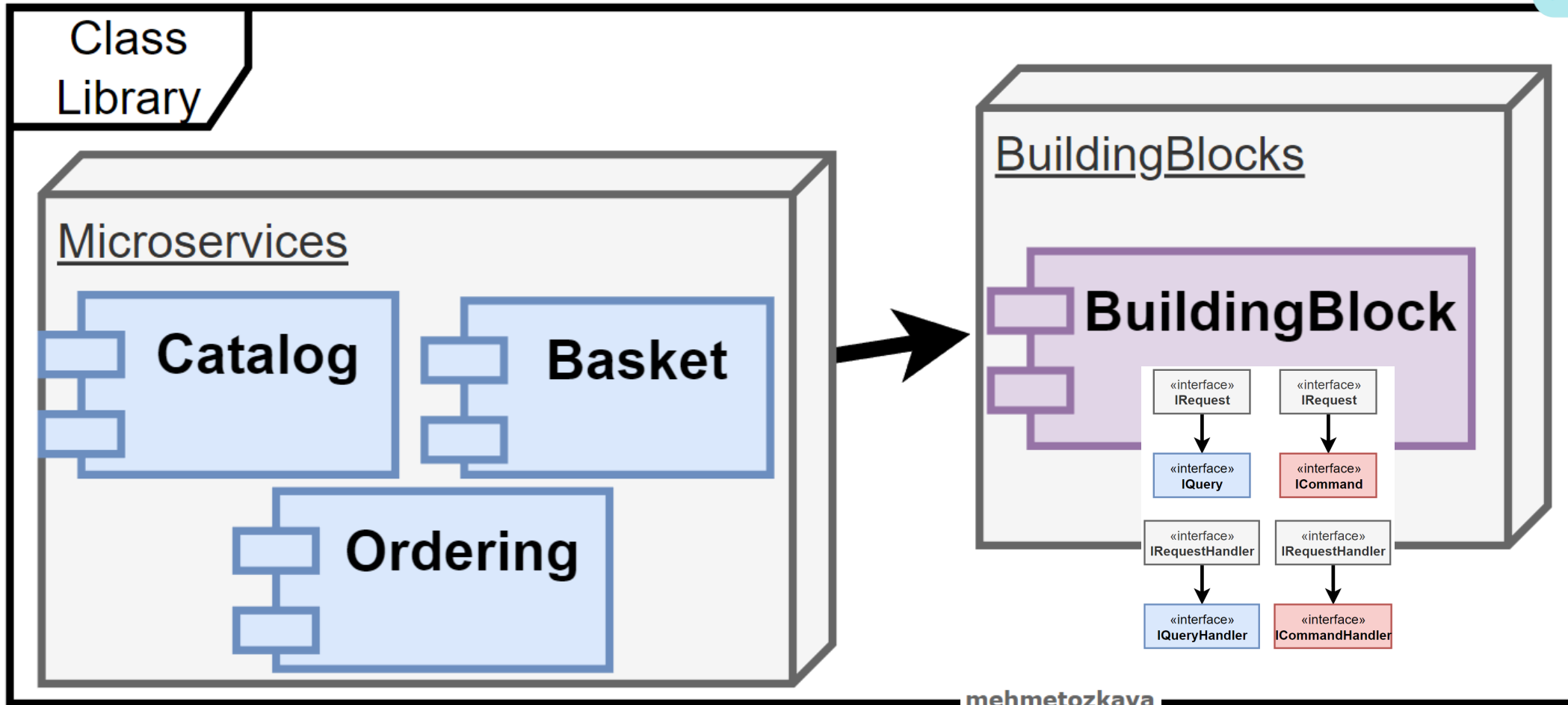


PostgreSQL



marten

Eshop Microservices uses BuildingBlocks Library



CQRS

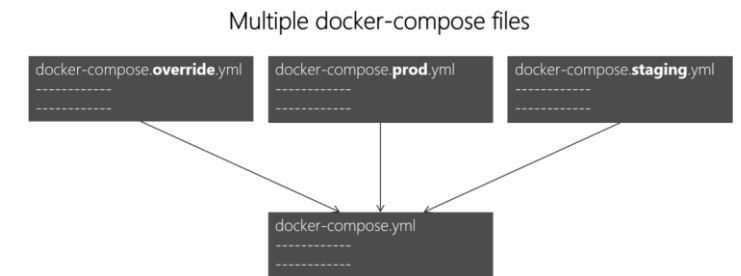
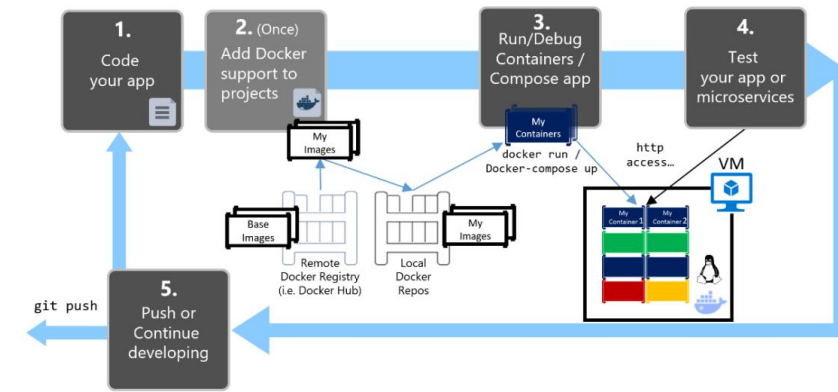
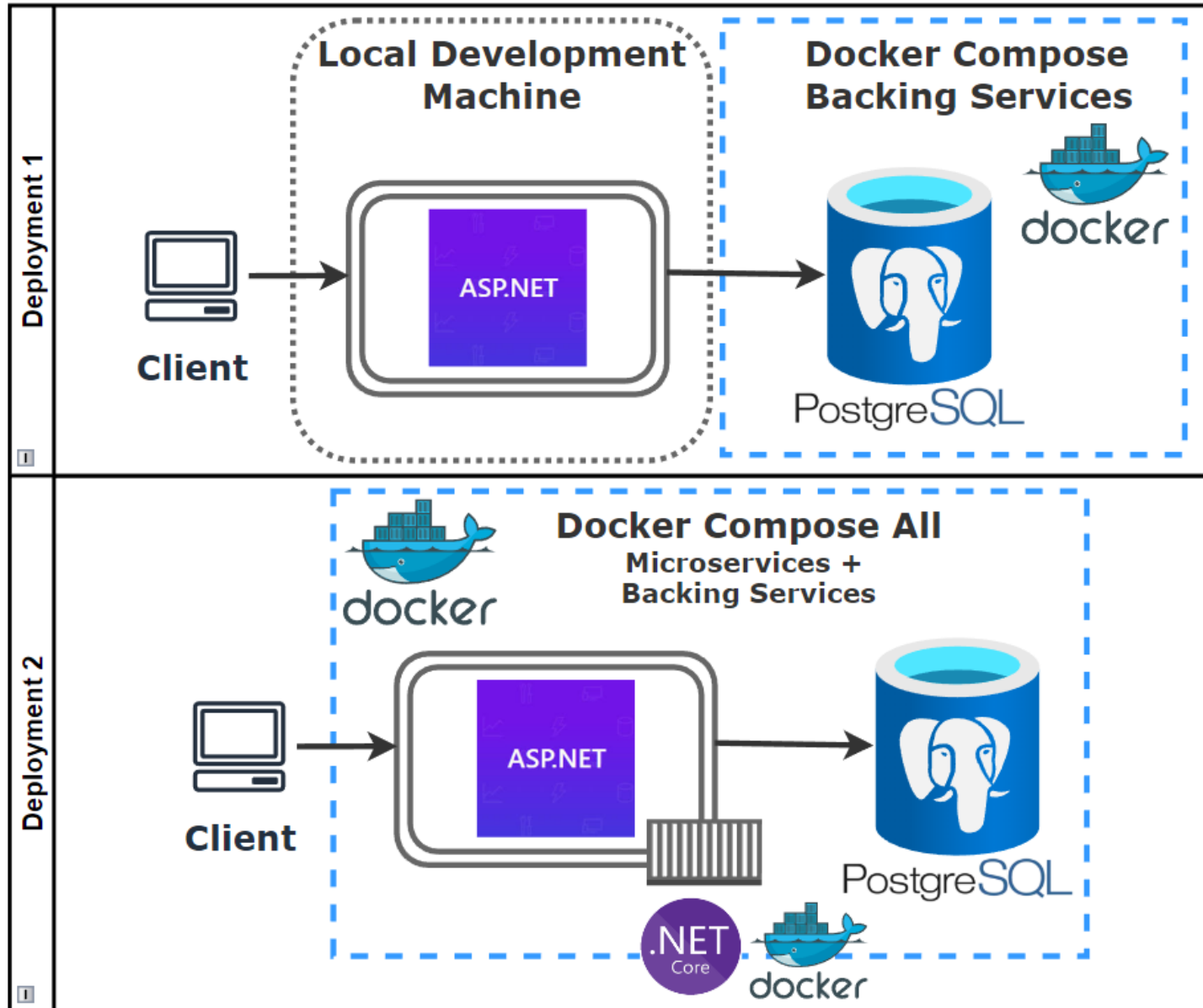


mehmetozkaya

The diagram illustrates a microservices architecture for an e-commerce system. It is divided into three main sections: Client Apps, API Gateways, and Microservices.

- Client Apps:** Includes "Shopping.Web" and a mobile application icon, both utilizing ".NET 8".
- API Gateways:** Features the "Yarp API Gateway", which also uses ".NET 8". It acts as the central point of contact for client requests.
- Microservices:** Consists of four core services:
 - Catalog:** Interacts with PostgreSQL.
 - Basket:** Interacts with PostgreSQL and Redis.
 - Discount:** Interacts with SQLite.
 - Ordering:** Interacts with SQL.
- Messaging:** RabbitMQ is used for asynchronous communication between the Basket and Ordering services.
- Supporting Technologies:** The architecture leverages Docker for containerization, ASP.NET Core for the framework, gRPC for service-to-service communication, and Entity Framework for database access.
- Architectural Patterns:** A "Vertical Slice" diagram shows the cross-cutting concerns (UI, Application, Domain, Infrastructure) across services. A "Core-Periphery" diagram shows the relationship between the core application and its periphery.

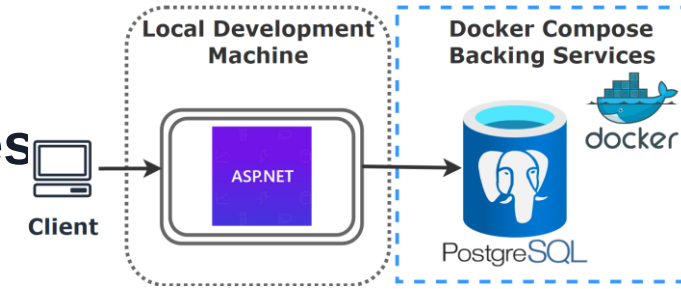
EShop Microservices Deployment Strategy



EShop Microservices Deployment Strategy

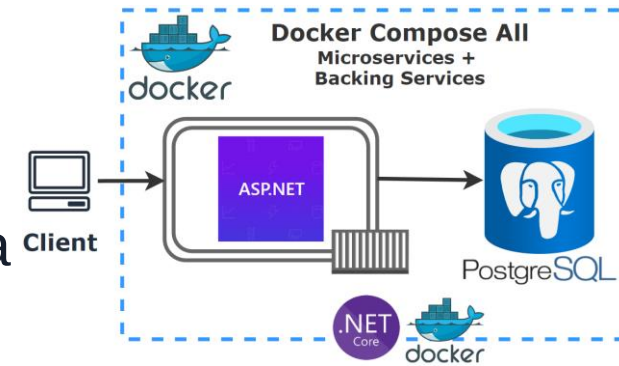
1. Local Development with Docker Compose for Backing Services

- Focus on local microservices development.
- Use Docker Compose to spin up backing services like databases, which are exposed to our local environment.
- Test microservices locally while connecting to containerized backing services.



2. Full Docker Environment Orchestration

- Orchestrating all microservices, including their dependencies, within a Docker Compose environment.
- Simulates a more production-like environment, allowing end-to-end testing.

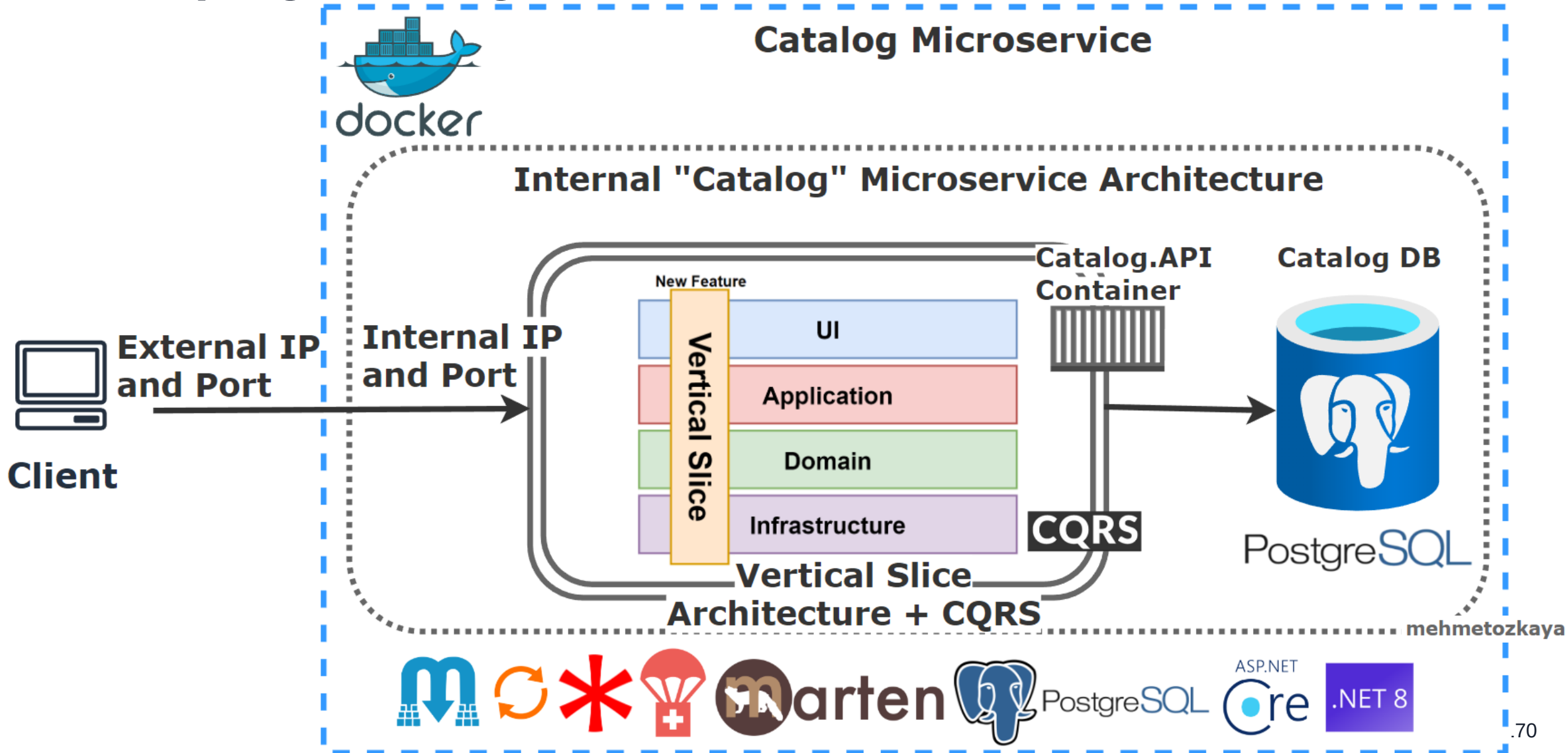


3. Future Deployment to Kubernetes and Cloud

- Orchestrating all Deploy microservices to Kubernetes, including local Minikube and cloud-based EKS (Elastic Kubernetes Service)



Developing Catalog.API Microservices

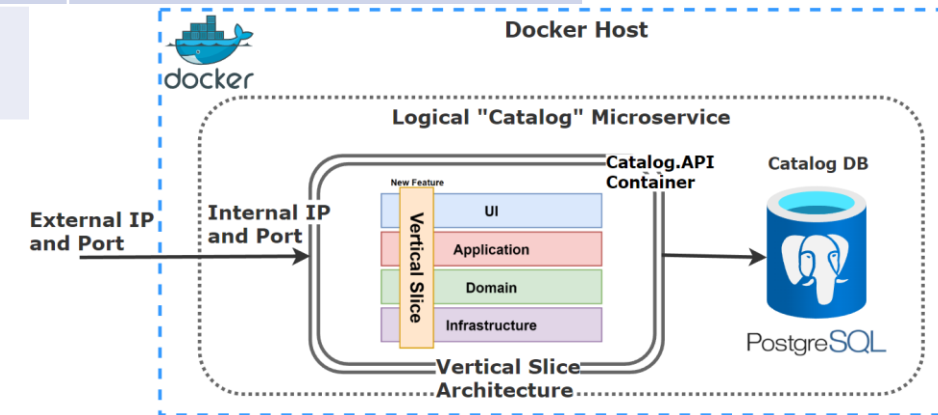


Port Numbers for Catalog.API Microservice

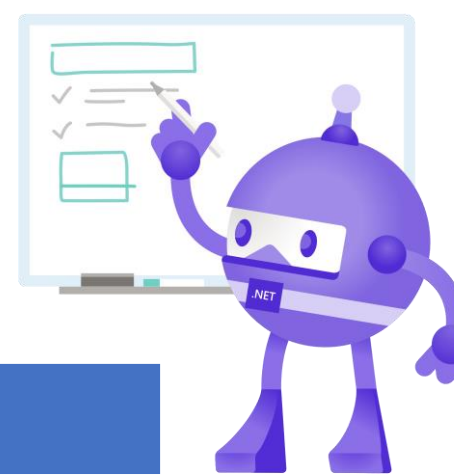


Microservices	Local Env	Docker Env	Docker Inside
Catalog	5000 – 5050	6000 – 6060	8080 – 8081
Basket	5001 – 5051	6001 – 6061	8080 – 8081
Discount	5002 – 5052	6002 – 6062	8080 – 8081
Ordering	5003 – 5053	6003 – 6063	

- ASP.NET Core **Ports** are listed as **http / https** for running application.

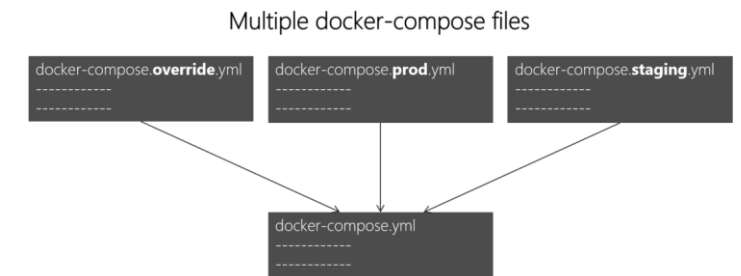
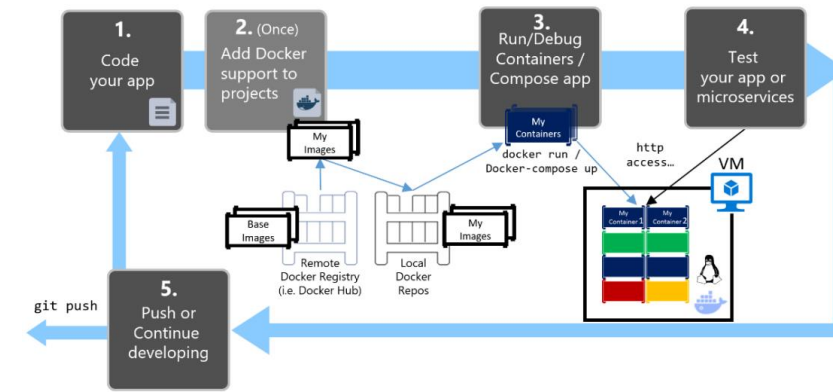
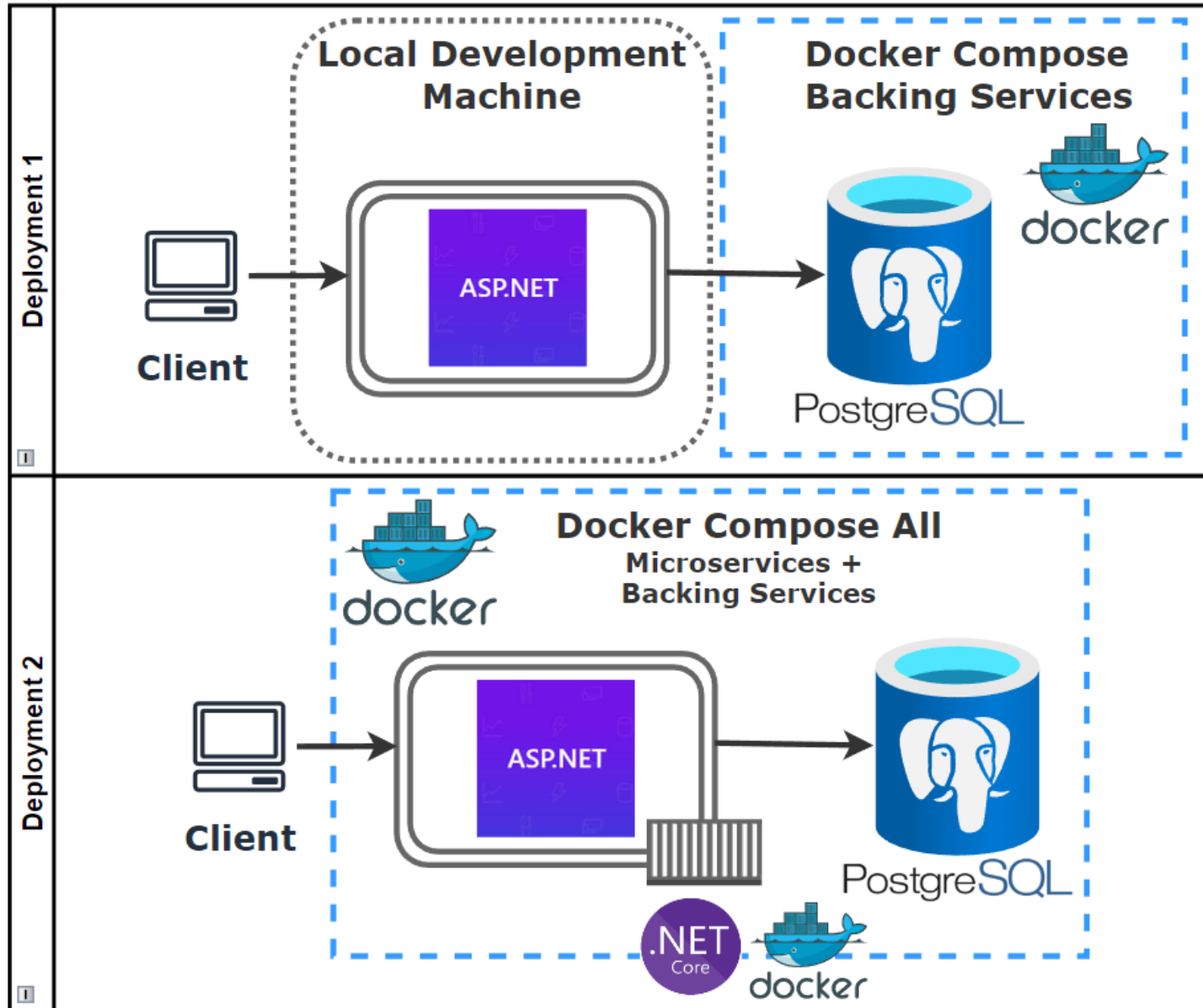


3. Rest API Endpoints of Catalog Microservices

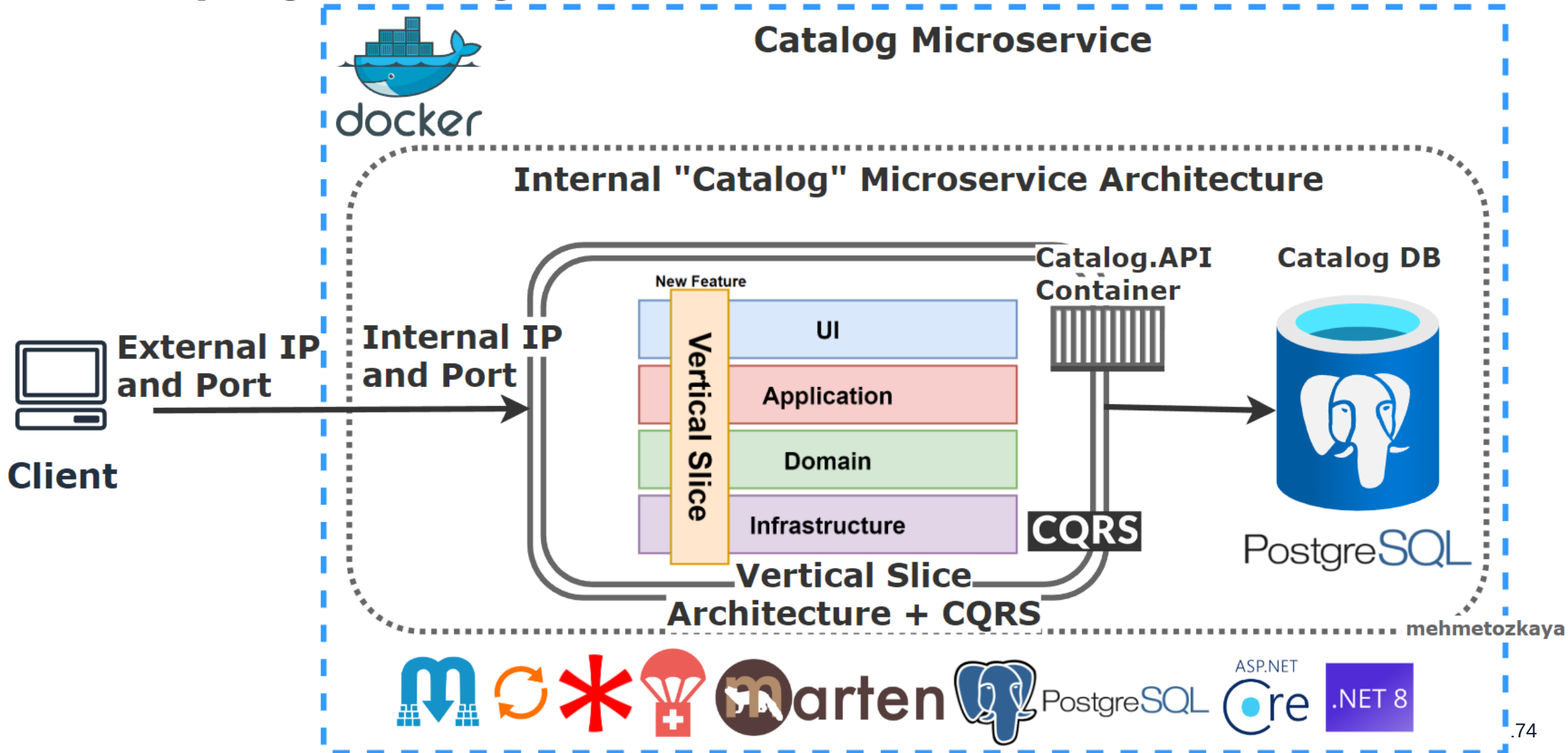


Method	Request URI	Use Cases
GET	/products	List all products
GET	/products/{id}	Fetch a specific product
GET	/products/category	Get products by category
POST	/products	Create a new product
PUT	/products/{id}	Update a product
DELETE	/products/{id}	Remove a product

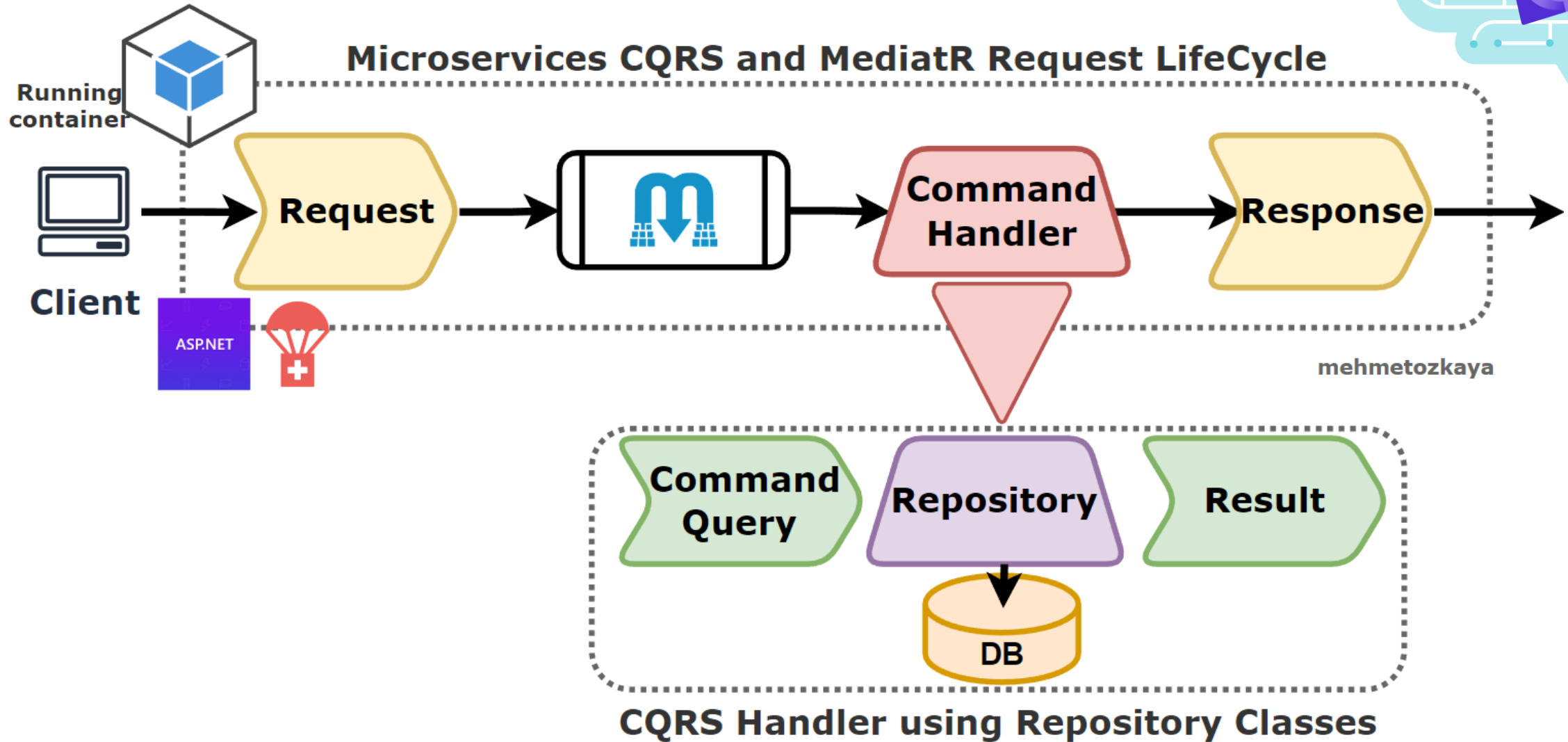
EShop Microservices Deployment Strategy



Developing Catalog.API Microservices



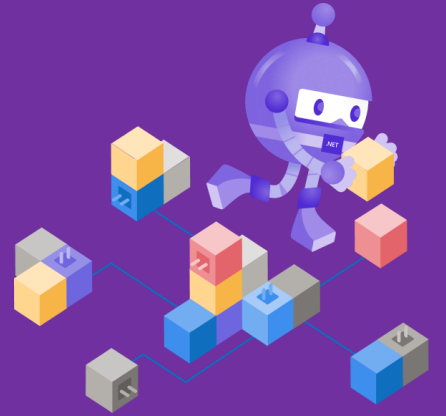
CQRS and MediatR Request LifeCycle



Create Postman Collection for EShop Microservices

- Microservices
- Endpoints
- Ports (local and docker)
- EShop Deployment Strategy
- Postman Collections
- Postman Environments and Global Variables

Develop Catalog.API Cross-cutting Concerns



MediatR Pipeline Behaviours and Fluent Validation Library

Logging and Validation Pipeline Behaviours

Global Handling Exceptions in ASP.NET Core

Seeding Catalog Database with Marten

Health Check for Catalog Microservices with PostgresDB

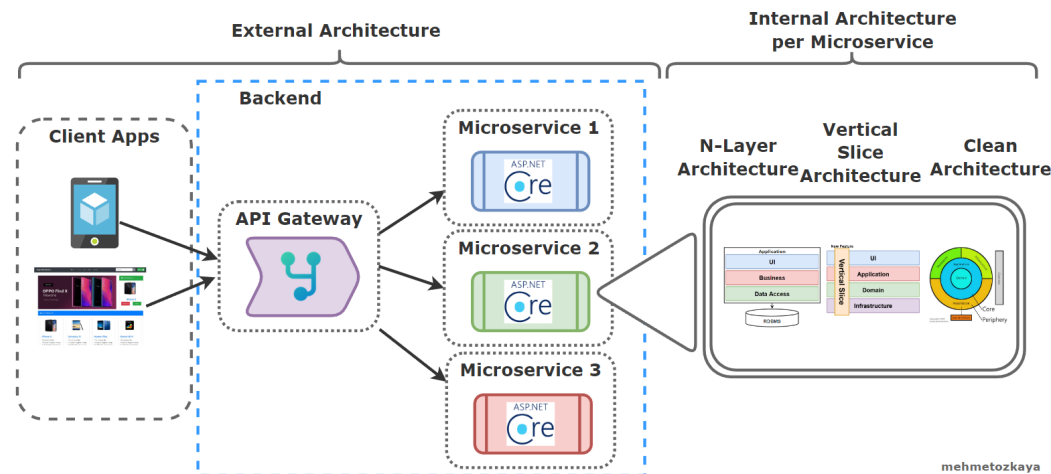
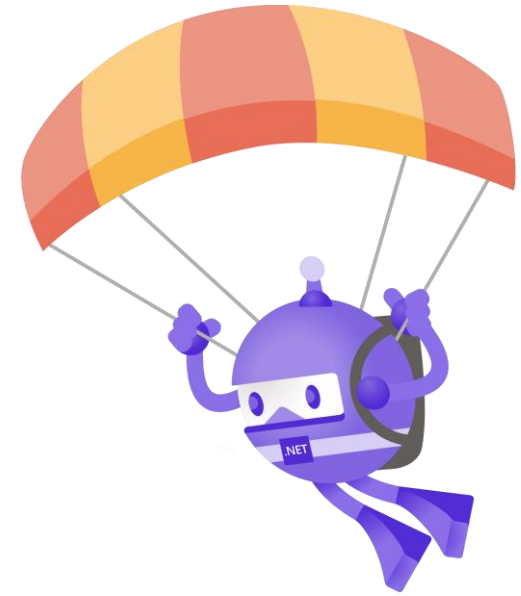
Containerize and Orchestrate Catalog.API microservices with Docker and Docker-compose

Mehmet Ozkaya

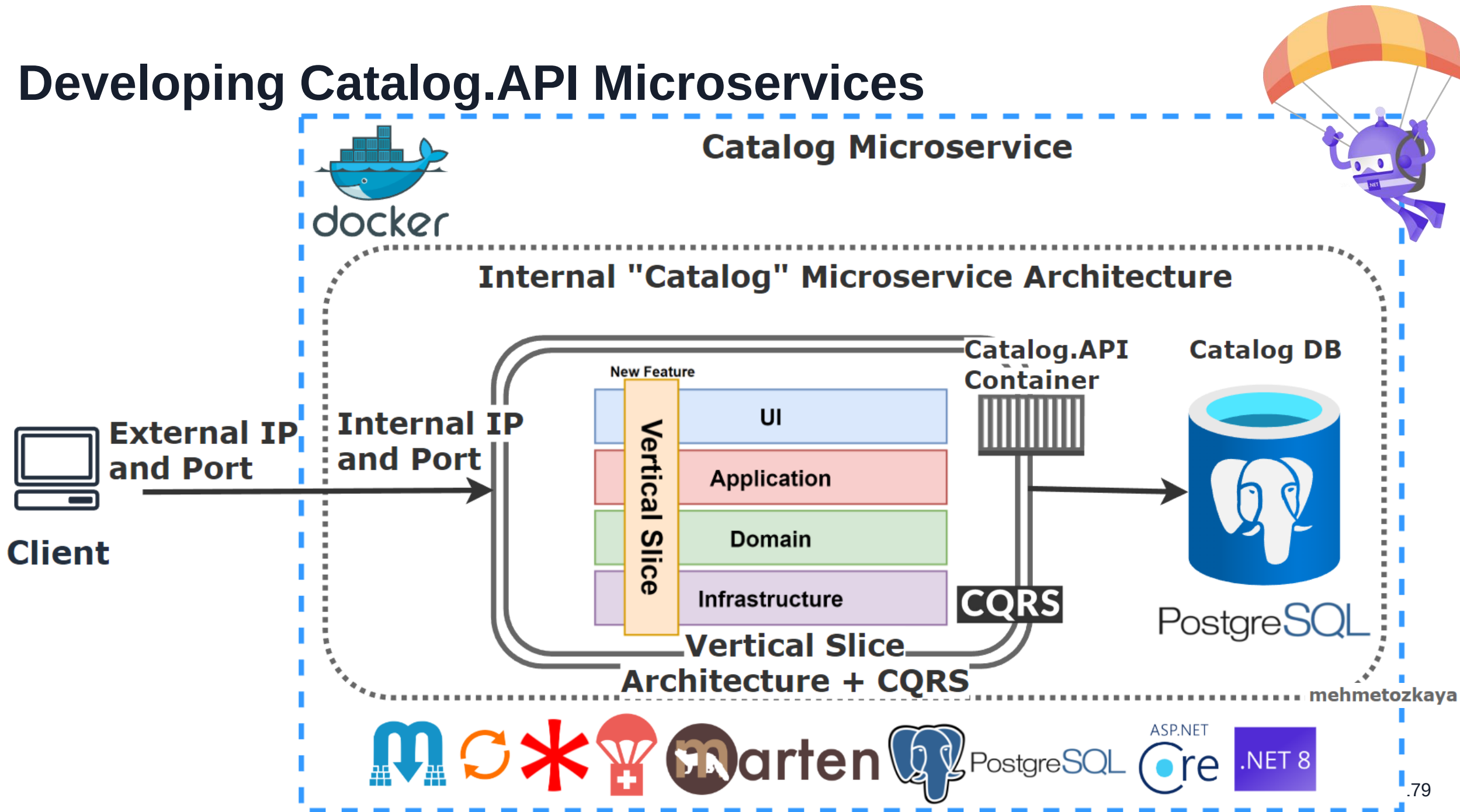


Develop Catalog.API Cross-cutting Concerns

- MediatR Pipeline Behaviours and Fluent Validation Library
- Logging and Validation Pipeline Behaviours
- Global Handling Exceptions in ASP.NET Core
- Seeding Catalog Database with Marten
- Health Check for Catalog Microservices with PostgresDB
- Develop Pagination for GetProducts Query
- Containerize and Orchestrate Catalog.API microservices with Docker and Docker-compose



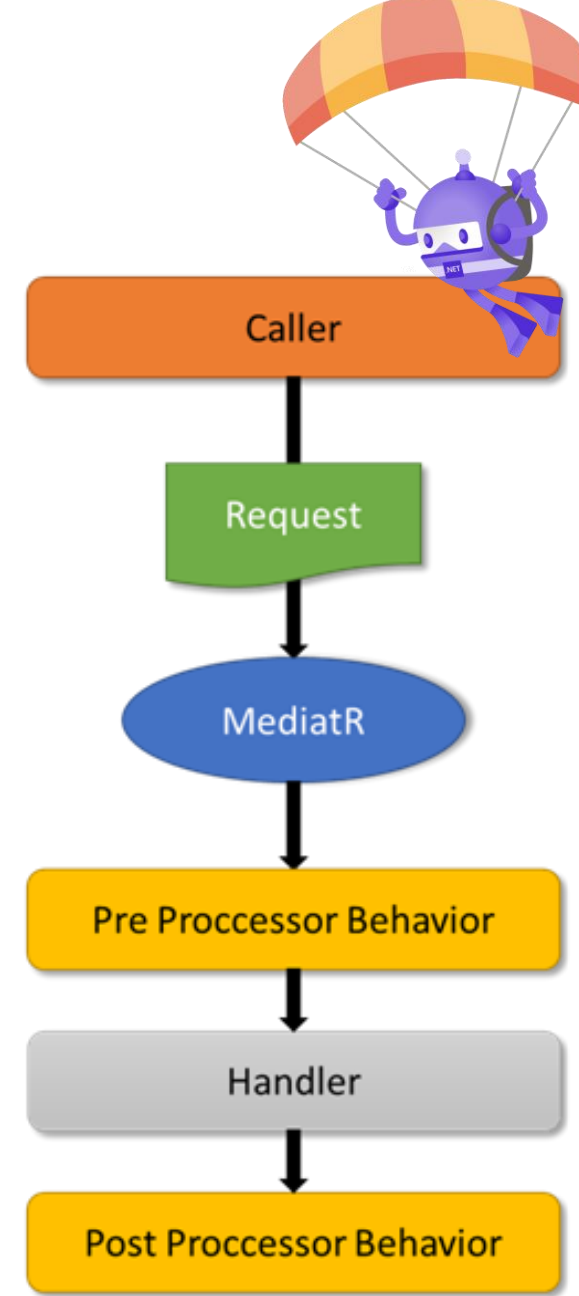
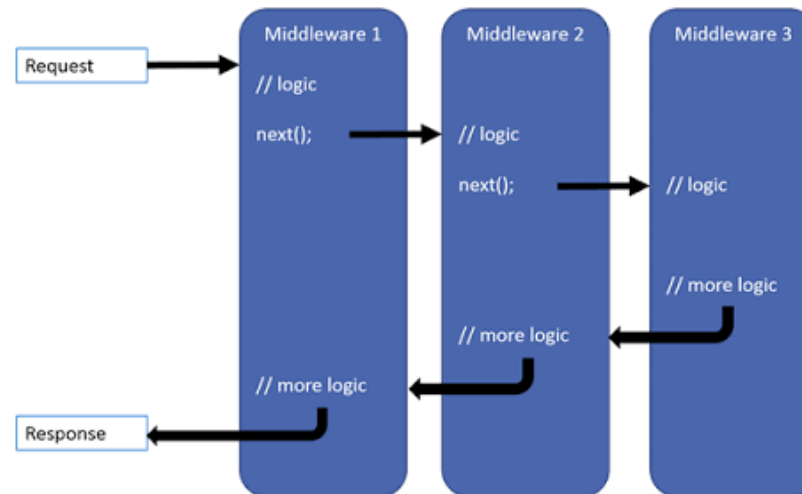
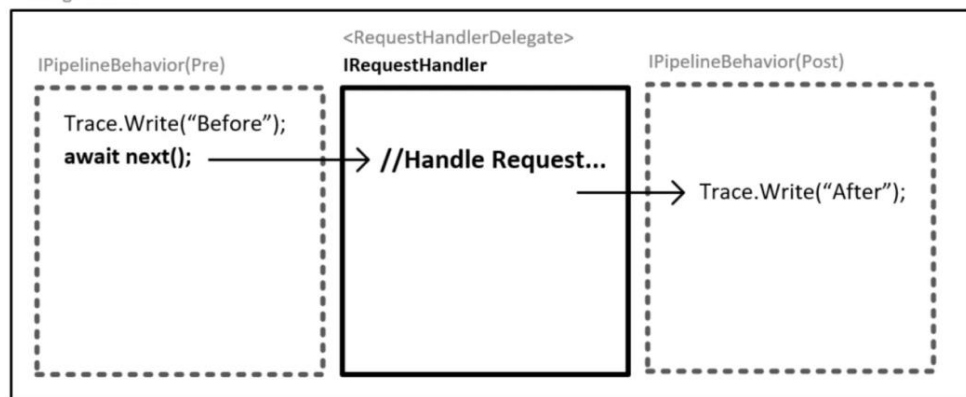
Developing Catalog.API Microservices



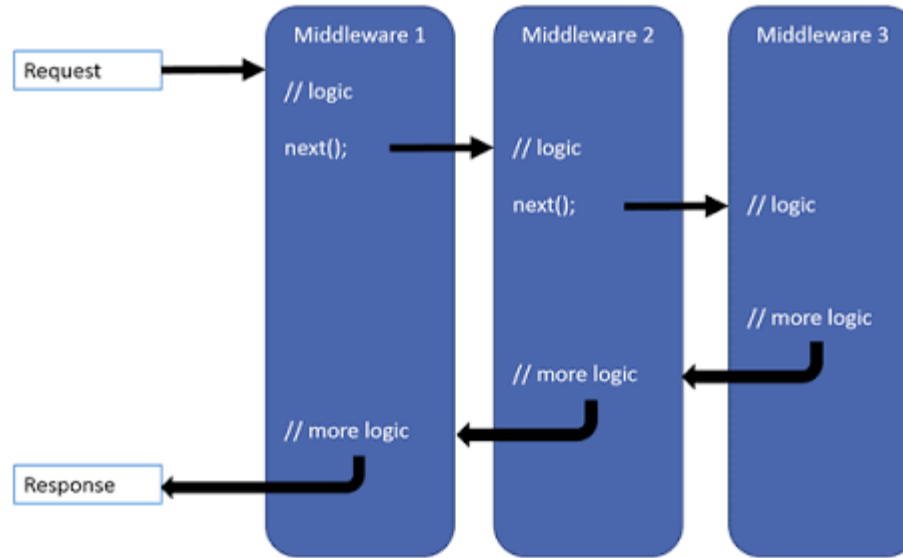
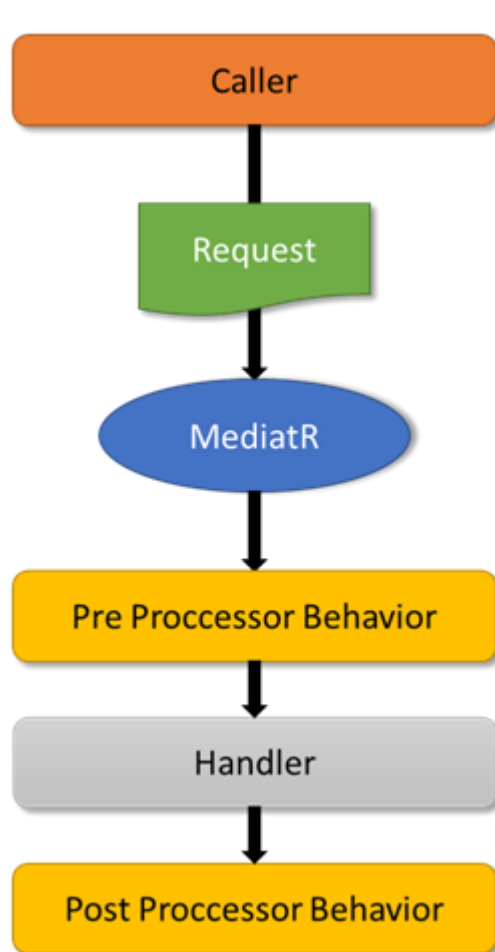
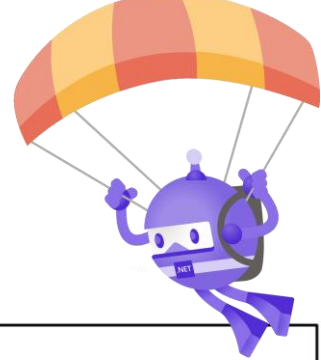
MediatR Pipeline Behaviours

- MediatR powerful features is 'Pipeline Behaviors' allows to additional logic into the **request handling process**: validation, logging, exception handling and performance tracking.
- **Pipeline Behaviors act as middleware** in the MediatR library, wrap around the **request handling process**, enabling to implement **cross-cutting concerns**.
- A class that implements `IPipelineBehavior<TRequest, TResponse>`.
- **Handle** method where you can execute code **before** and **after** the **next delegate** is invoked.

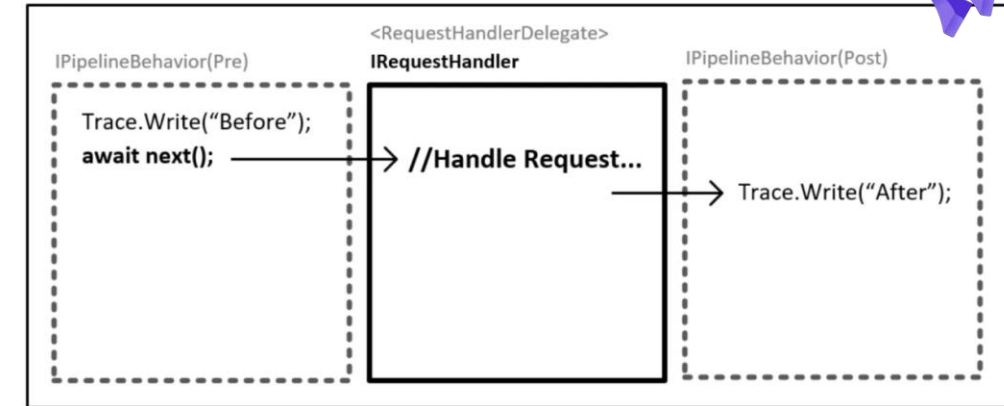
TracingBehavior



How MediatR Pipeline Behaviours Works



TracingBehavior



```
public class LoggingBehavior<TRequest, TResponse> (ILogger<LoggingBehavior<TRequest, TResponse>
    : IPipelineBehavior<TRequest, TResponse>
    where TRequest : notnull, IRequest<TResponse>
    where TResponse : notnull
{
    0 references
    public async Task<TResponse> Handle(TRequest request, RequestHandlerDelegate<TResponse> next)
    {
        logger.LogInformation("[START] Handle request={Request} - Response={Response} - {RequestType} - {ResponseType}");
        var timer = new Stopwatch();
        timer.Start();

        var response = await next();
    }
}
```


MediatR Pipeline Behaviours and Fluent Validation Library

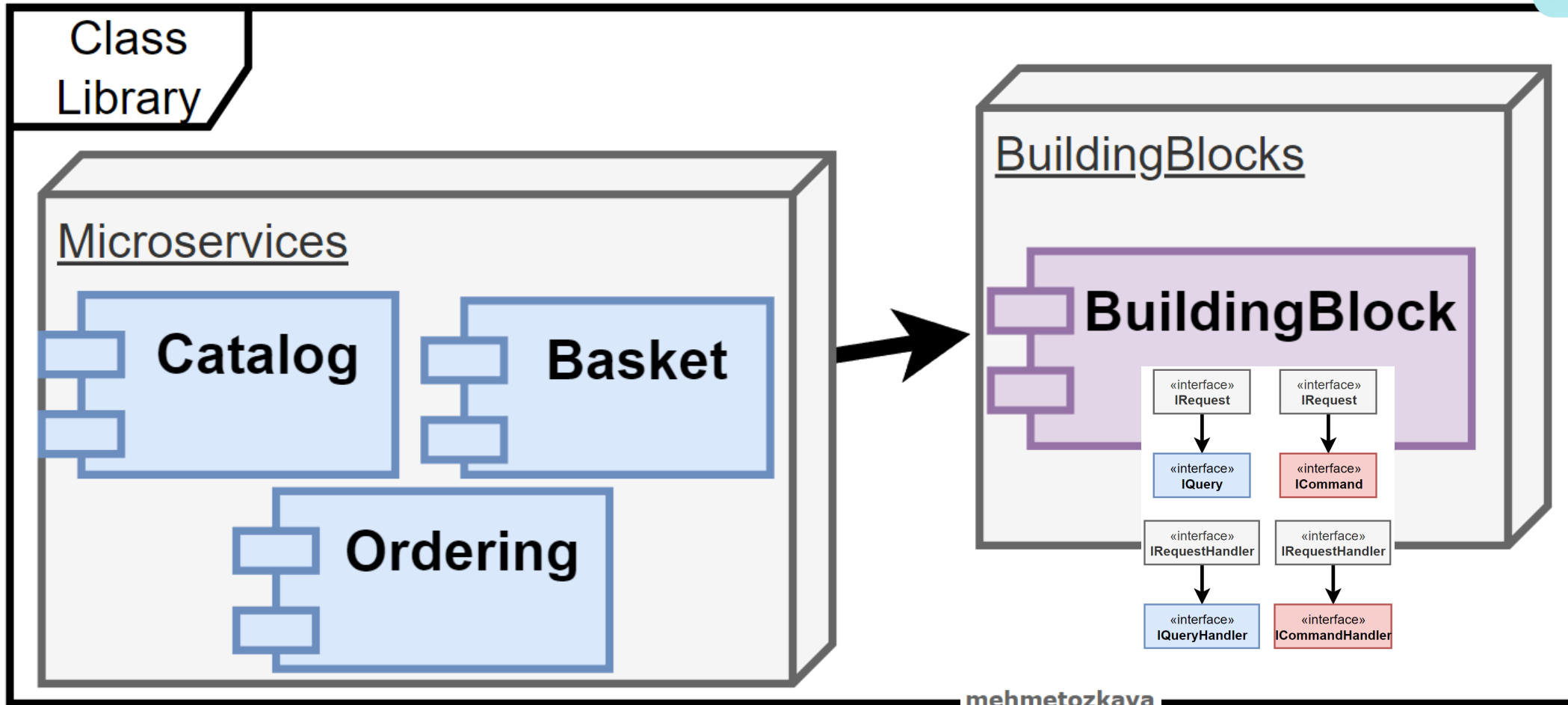
- **Fluent Validation** is a .NET library for building **strongly-typed validation rules**.
- Integrate **Fluent Validation** with **MediatR** to validate requests before they reach the actual handler within a pipeline behavior.
- Fluent Validation **define validation rules in separate classes**, using a fluent interface to specify conditions that each property of your models must satisfy.

```
public class CreateProductCommandValidator : AbstractValidator<CreateProductCommand>
{
    0 references
    public CreateProductCommandValidator()
    {
        RuleFor(x => x.Name).NotEmpty().WithMessage("Name is required");
        RuleFor(x => x.Category).NotEmpty().WithMessage("Category is required");
        RuleFor(x => x.ImageFile).NotEmpty().WithMessage("ImageFile is required");
        RuleFor(x => x.Price).GreaterThan(0).WithMessage("Price must be greater than 0");
    }
}
```



- Combining **MediatR** with **Fluent Validation** centralizes our **cross-cutting concerns** like validation, making our code cleaner and more maintainable.

Eshop Microservices uses BuildingBlocks Library



CQRS



mehmetozkaya

Seeding CatalogDb with Marten Initial Baseline Data

- **Seeding** is essential for **initializing the database** with baseline data. Marten provides a feature called **IInitialData** for this purpose.
- <https://martendb.io/documents/initial-data.html>
- Implement the **IInitialData** interface to seed our **CatalogDb** with **predefined products**.
- This interface allows **Marten** to **automatically populate** the database when it's **first initialized**.

```
public class InitialData: IInitialData
{
    private readonly object[] _initialData;

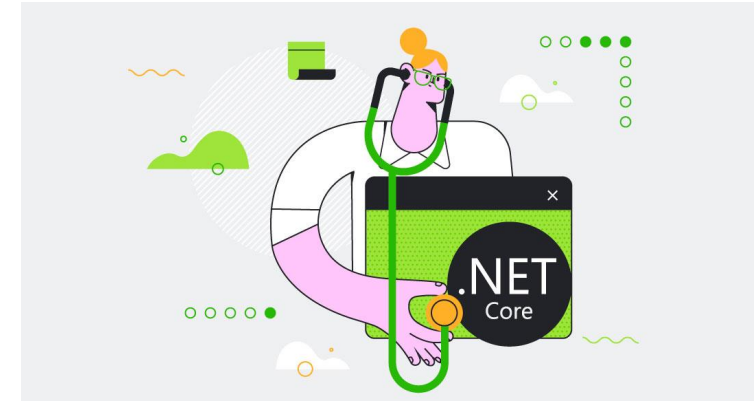
    public InitialData(params object[] initialData)
    {
        _initialData = initialData;
    }

    public async Task Populate(IDocumentStore store, CancellationToken cancella
    {
        await using var session = store.LightweightSession();
        // Marten UPSERT will cater for existing records
        session.Store(_initialData);
        await session.SaveChangesAsync();
    }
}

public static class InitialDatasets
{
    public static readonly Company[] Companies =
    {
        new Company { Id = Guid.Parse("2219b6f7-7883-4629-95d5-1a8a6c74b244") },
        new Company { Id = Guid.Parse("642a3e95-5875-498e-8ca0-93639ddfcbcd") },
    };
}
```

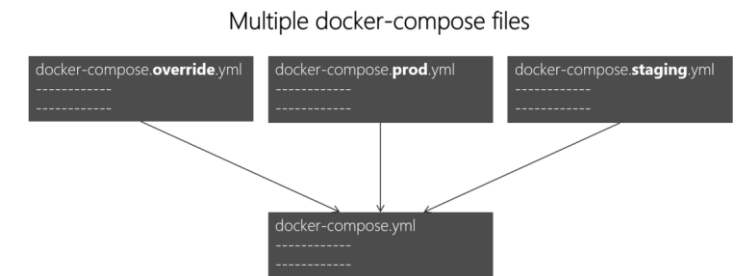
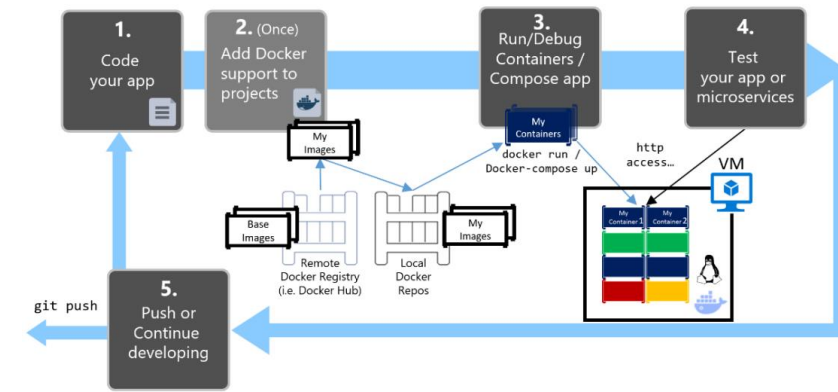
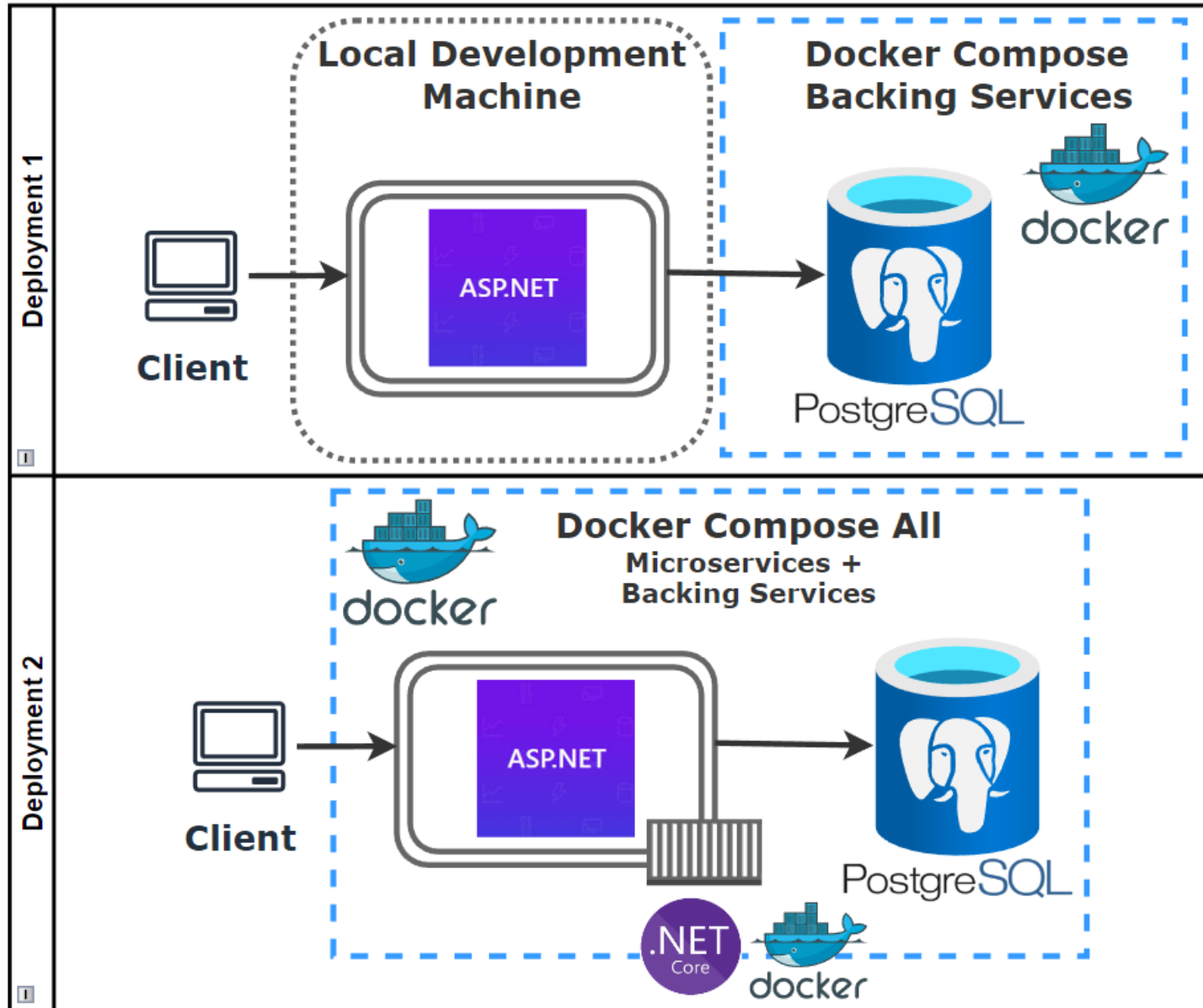
Health checks in ASP.NET Core

- **Health checks** in ASP.NET Core provide a way to **monitor** the **status** of your **application** and its **dependencies**.
- **Add health checks** to our **Catalog microservices** with checking the health of our **PostgreSQL database**.
- Health checks are exposed by an app as **HTTP endpoints**. Health check endpoints can be configured for various **real-time monitoring** scenarios.
- **Container orchestrator** may respond to a **failing health check** by **halting a rolling deployment** or restarting a container.
- **Load balancer** might react to an **unhealthy** app by **routing traffic** away from the **failing instance** to a **healthy instance**.



<https://blog.devart.com/how-to-implement-health-checks-in-asp-net-6.htm>

EShop Microservices Deployment Strategy

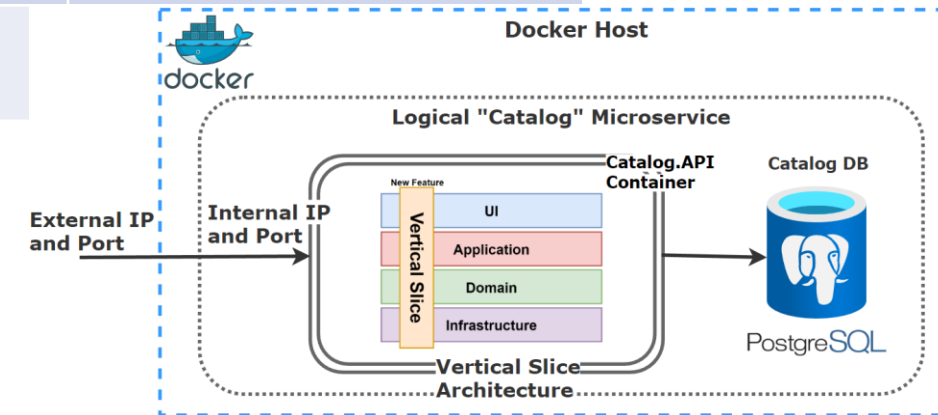


Port Numbers for Catalog.API Microservice

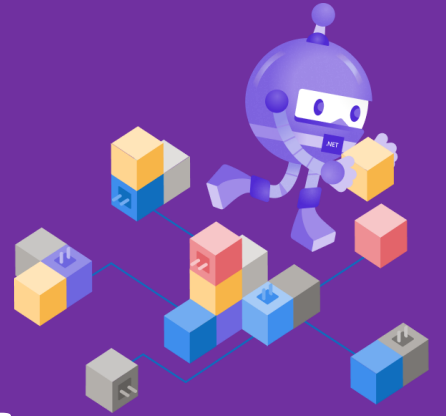


Microservices	Local Env	Docker Env	Docker Inside
Catalog	5000 – 5050	6000 – 6060	8080 – 8081
Basket	5001 – 5051	6001 – 6061	8080 – 8081
Discount	5002 – 5052	6002 – 6062	8080 – 8081
Ordering	5003 – 5053	6003 – 6063	

- ASP.NET Core **Ports** are listed as **http / https** for running application.



Basket Microservices with Vertical Slice Architecture and CQRS



Minimal APIs and latest features of .Net8 and C# 12

Vertical Slice Architecture with Feature folders

CQRS implementation using MediatR library

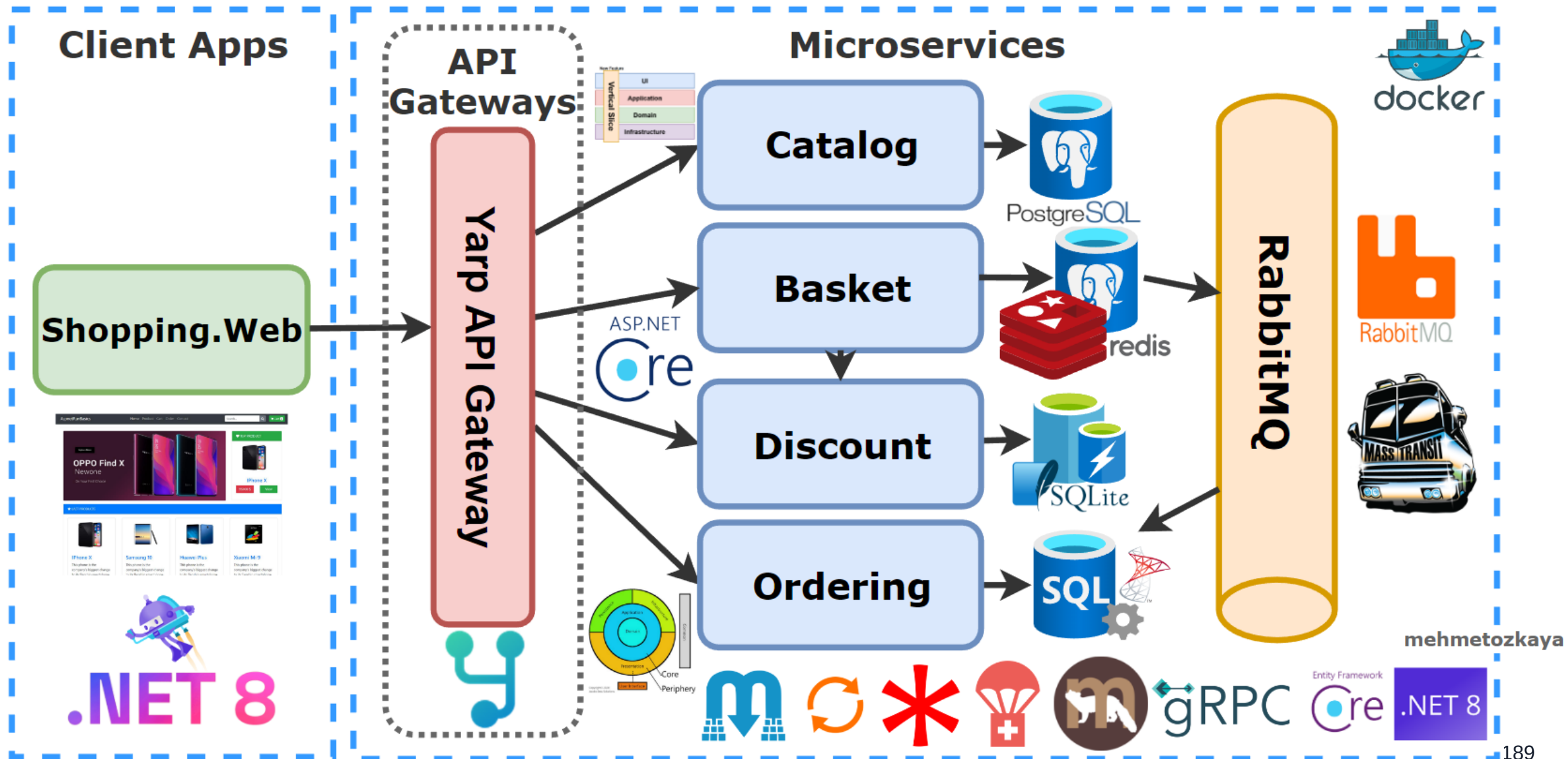
Repository Pattern Implementation over Marten and Redis Cache

Marten library for .NET Transactional Document DB on PostgreSQL

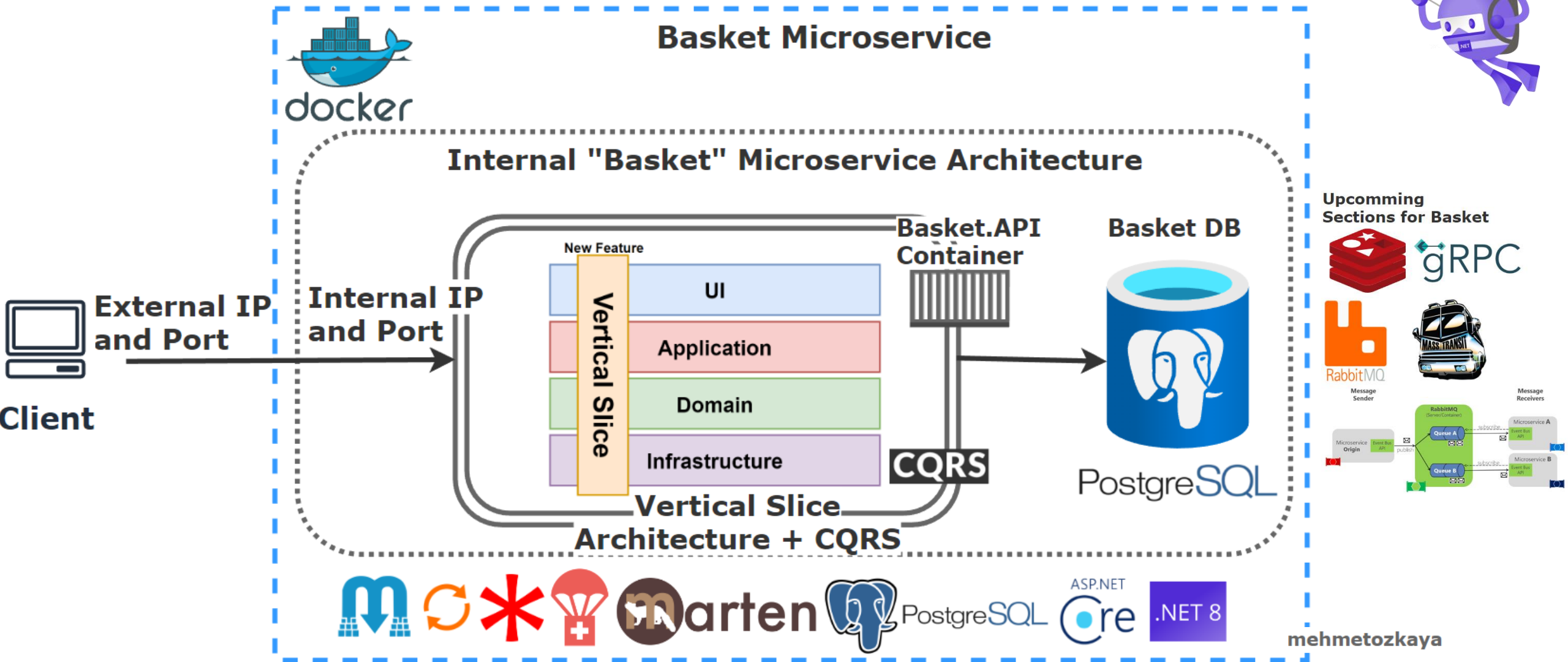
Mehmet Ozkaya



Big Picture – Course Project

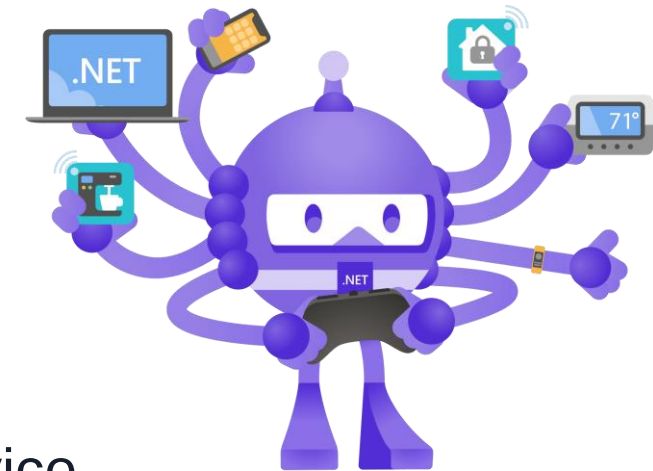


Developing Basket.API Microservices



Basket.API with Vertical Slice Architecture and CQRS

- **ASP.NET Core Minimal APIs** for building fast HTTP APIs-fully functioning REST endpoints and top-level Program.cs
- **Vertical Slice Architecture** implementation with Feature folders and single .cs file includes different classes in one file.
- **CQRS** implementation using **MediatR** library.
- **Marten** library for .NET Transactional Document DB on PostgreSQL
- **Repository Pattern** Implementation over Marten and Redis Cache
- **Carter** for Minimal Api endpoint definition
- **FluentValidation** to validate inputs and add MediatR validation pipeline
- **Dockerfile** and **docker-compose** file for running Basket microservice and PostgreSQL database in docker environment

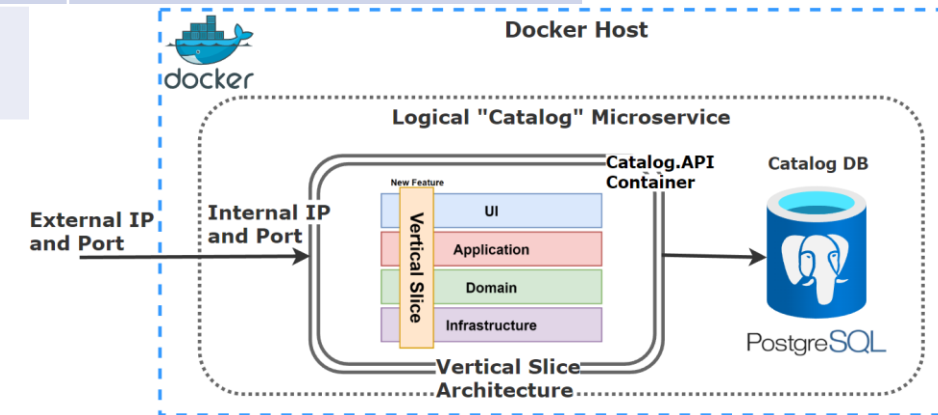


Port Numbers for Basket.API Microservice

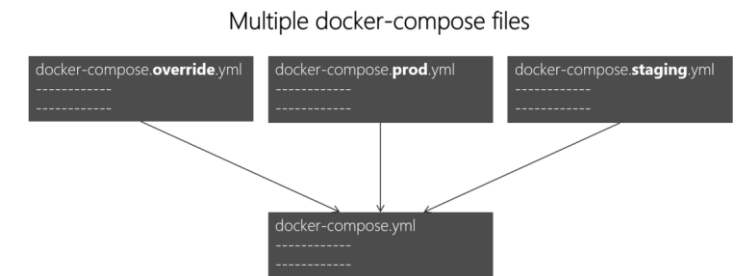
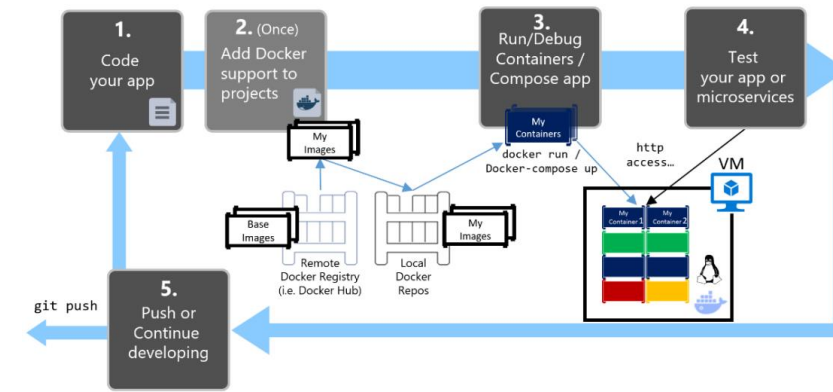
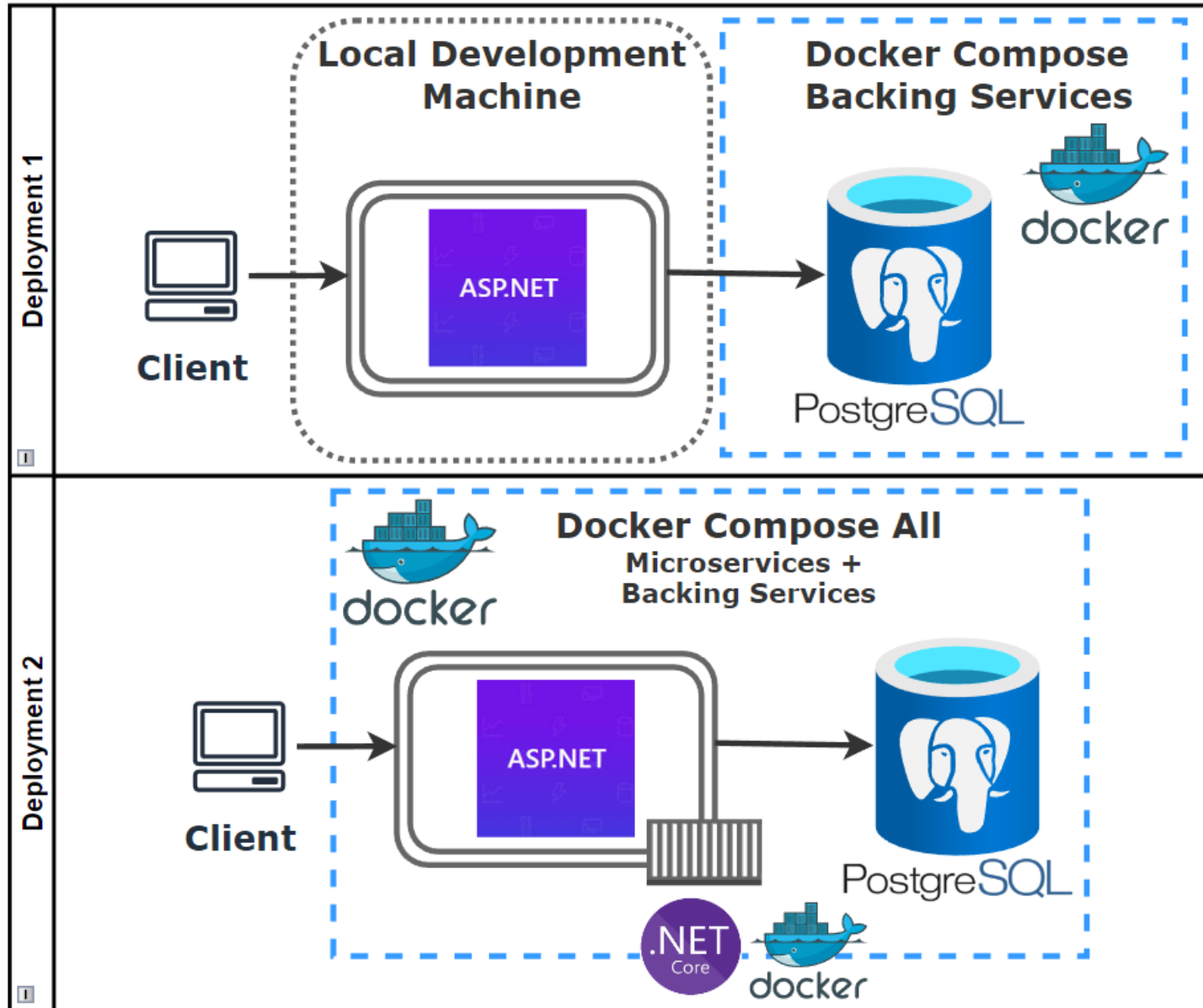


Microservices	Local Env	Docker Env	Docker Inside
Catalog	5000 – 5050	6000 – 6060	8080 – 8081
Basket	5001 – 5051	6001 – 6061	8080 – 8081
Discount	5002 – 5052	6002 – 6062	8080 – 8081
Ordering	5003 – 5053	6003 – 6063	

- ASP.NET Core **Ports** are **listed** as **http / https** for running application.

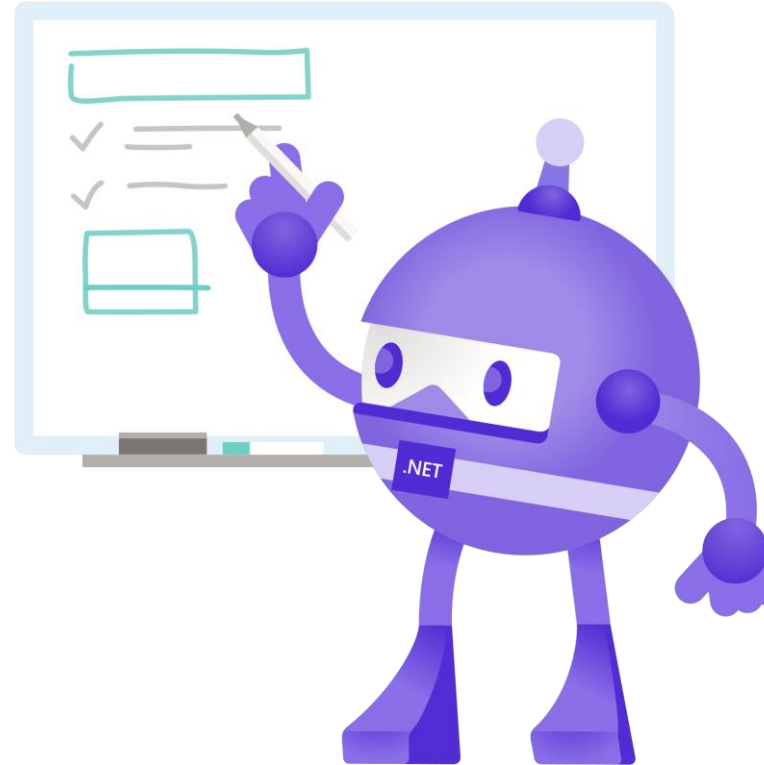


EShop Microservices Deployment Strategy



Domain Analysis of Basket Microservices

1. Domain Models
2. Application Use Cases
3. Rest API Endpoints
4. Underlying Data Structures



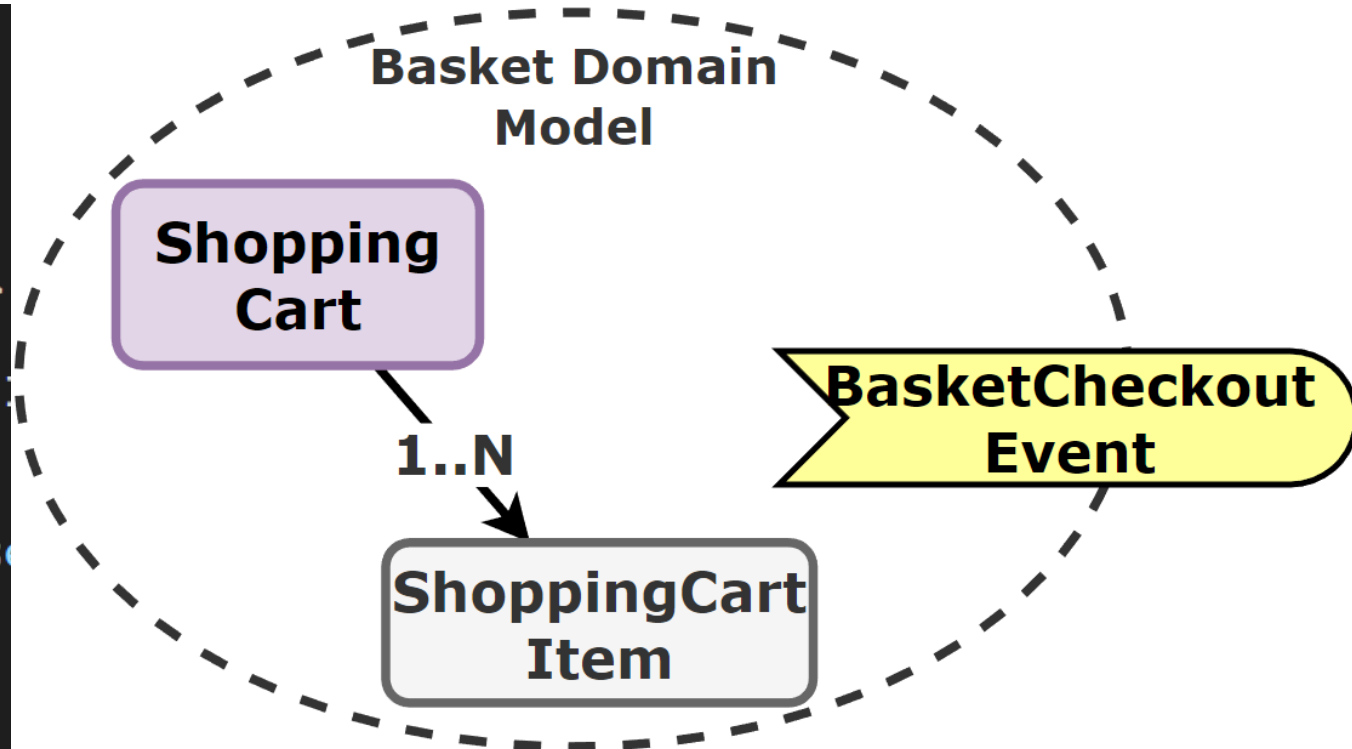
1. Domain Models of Basket Microservices



- Primary domain model is 'ShoppingCart', 'ShoppingCartItem', and 'Basket Checkout'

```
public class ShoppingCart
{
    5 references
    public string UserName { get;
    2 references
    public List<ShoppingCartItem>
    1 reference
    public decimal TotalPrice =>

    0 references
    public ShoppingCart(string user
    {
        UserName = userName;
    }
}
```



- Consider Domain event for basket checkout, leading to integration events.

2. Application Use Cases of Basket Microservices

CRUD Basket Operations:

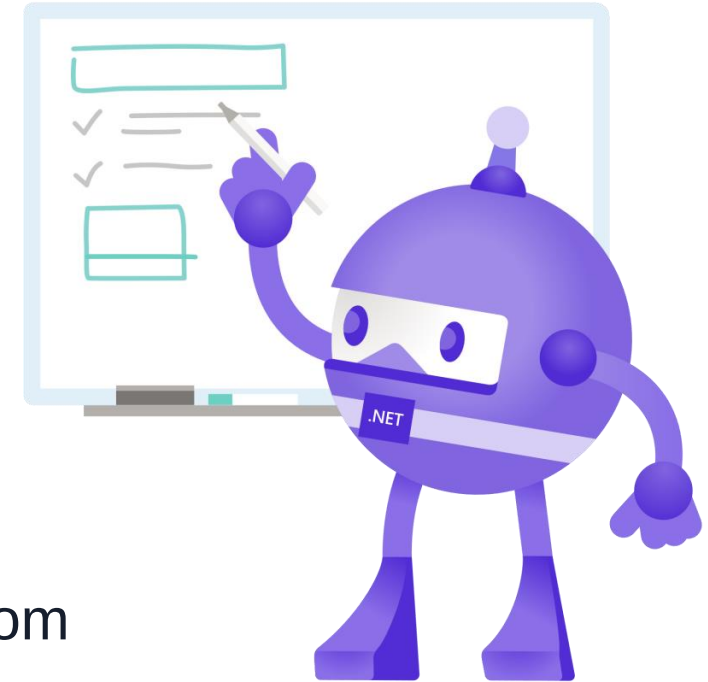
- Get ShoppingCart with Items
- Store (Upsert-Create and Update) ShoppingCart with Items that includes;
 - Add Item into ShoppingCart
 - Remove Item into ShoppingCart
 - Update ShoppingCartItem into ShoppingCart (increase Quantity)
- Delete ShoppingCart with Items

gRPC Basket Operations:

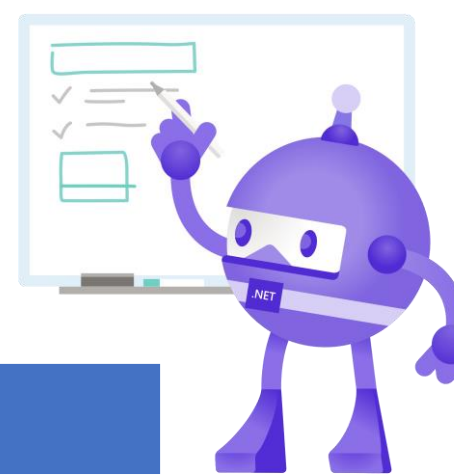
- When Store Basket: GetDiscount and deduct discount coupon from Item Price

Async Basket Operations:

- Checkout Basket and Publish event to RabbitMQ Message Broker



3. Rest API Endpoints of Basket Microservices



Method	Request URI	Use Cases
GET	/basket/{userName}	Get basket w/ username
POST	/basket/{userName}	Store Basket (insert-update)
DELETE	/basket/{userName}	Delete basket w/ username
POST	/basket/checkout	Checkout basket

4. Underlying Data Structures of Basket Microservices

Basket Microservices has 2 Datastore:

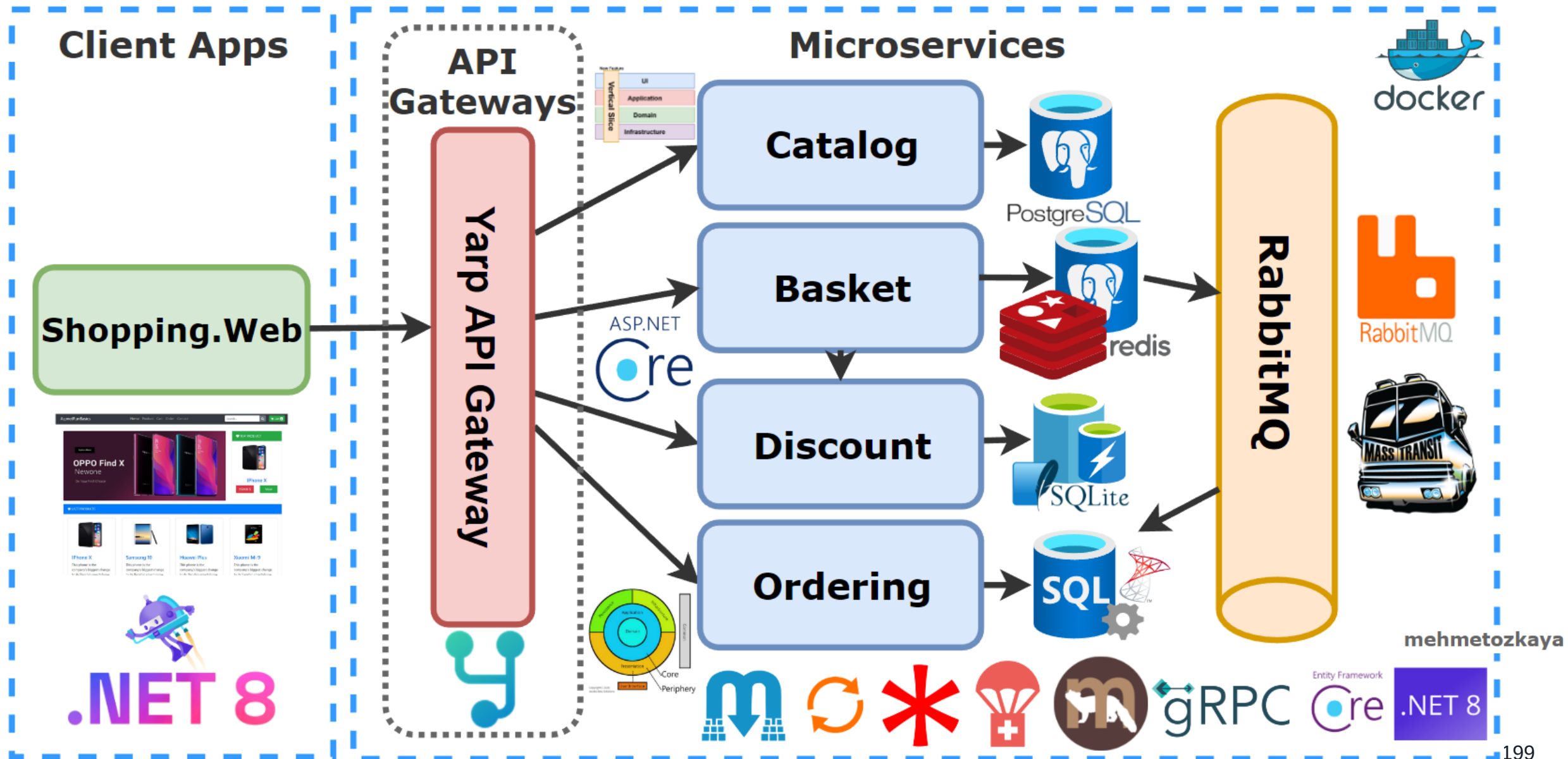
1. Marten Document Database

2. Redis Distributed Cache

- **Marten** is a powerful library that transforms **PostgreSQL** into a **.NET Transactional Document DB** using **PostgreSQL's JSON** column features.
- **Redis** a powerful in-memory data store and **distributed cache** which is good fit for microservices architectures.

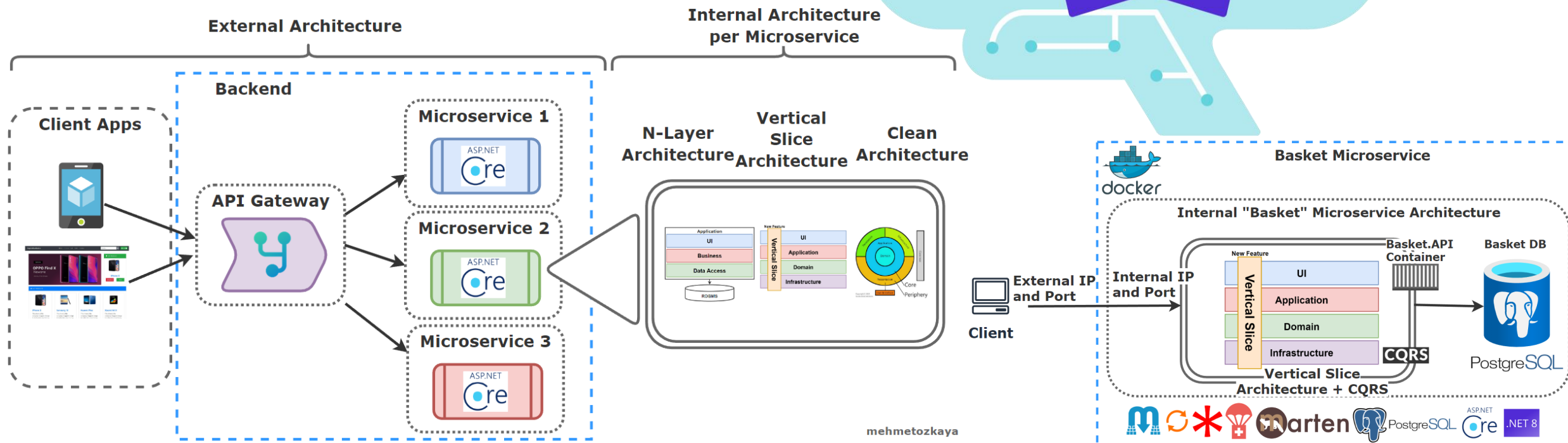
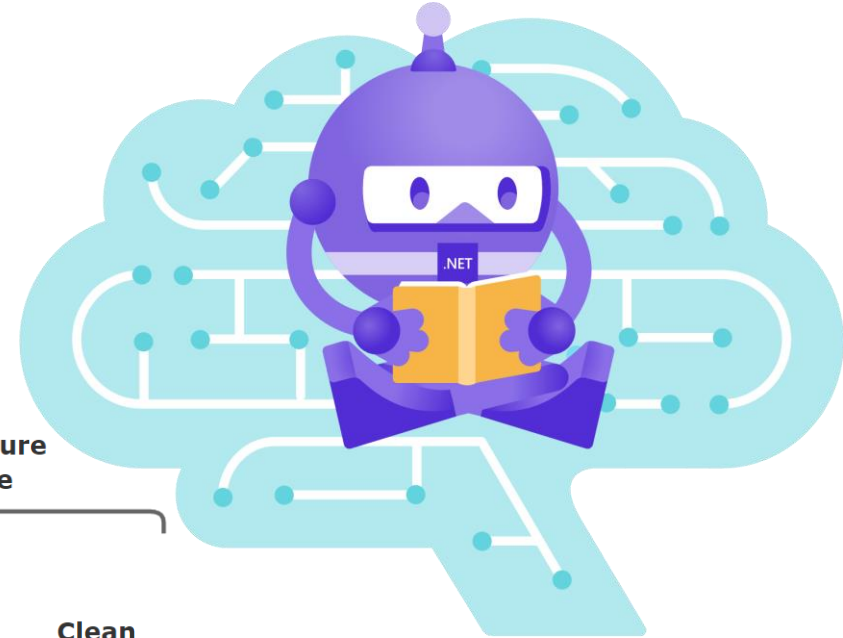


Big Picture – Course Project



Technical Analysis of Basket Microservices

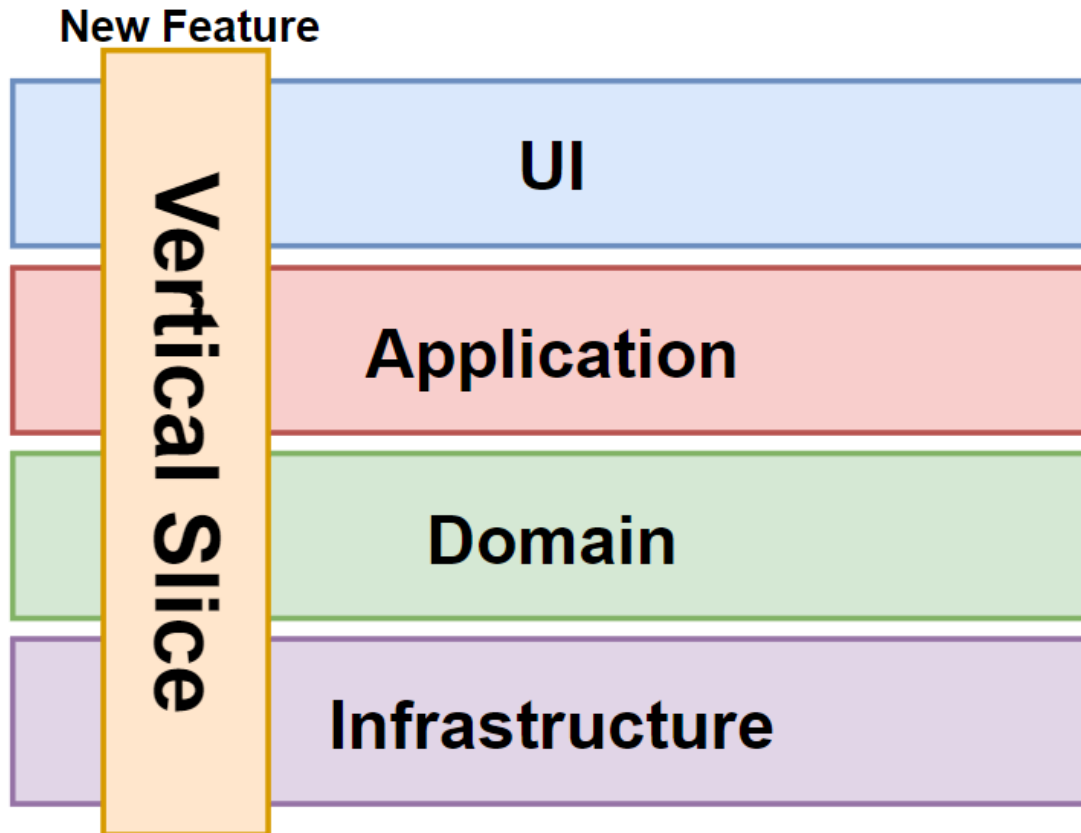
1. Application Architecture Style
2. Patterns & Principles
3. Libraries Nuget Packages
4. Project Folder Structure



1. Application Architecture Style of Basket Microservices

Vertical Slice Architecture

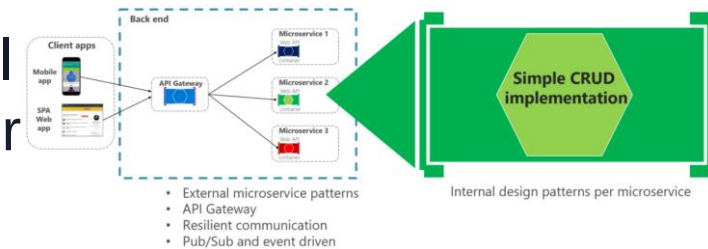
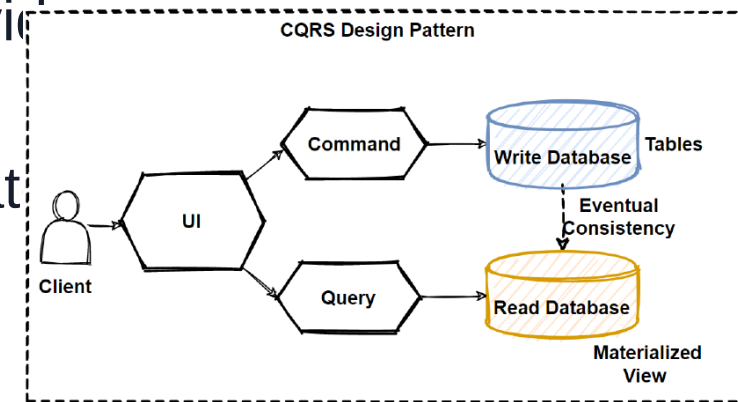
- Organizes our code into feature folders, each feature encapsulated in a single .cs file



2. Patterns & Principles of Basket Microservices



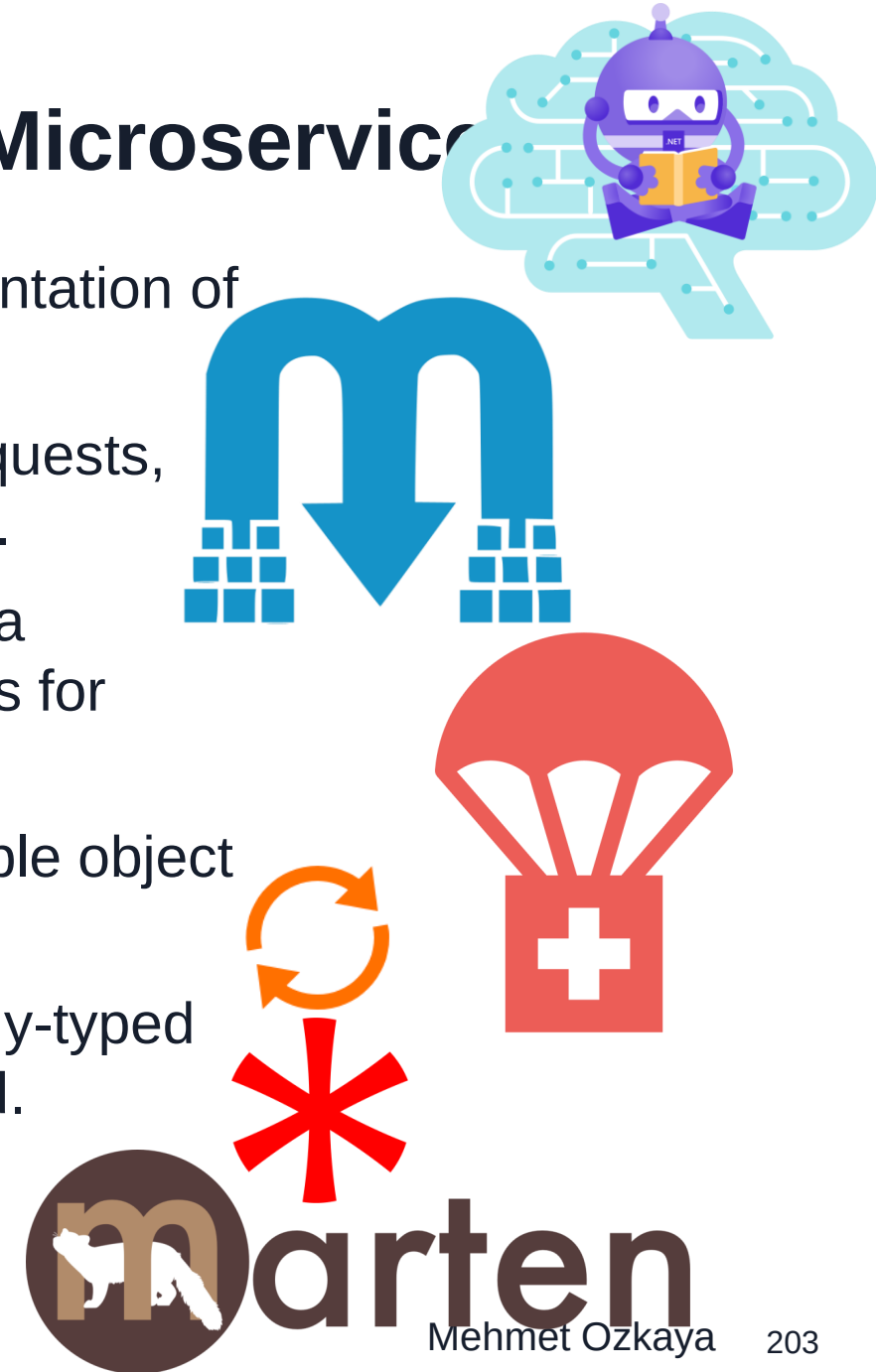
- **Repository Pattern:** DDD pattern to keep persistence concerns outside of the system's domain model. Provides an abstraction of data, that app can work with a simple abstraction interfaces.
- **CQRS Pattern:** Command Query Responsibility Segregation divides operations into commands (write) and queries (read).
- **Mediator Pattern:** Facilitates object interaction through a 'mediator' reducing direct dependencies and simplifying communications.
- **DI in ASP.NET Core:** Dependency Injection is a core feature, allowing us to inject dependencies.
- **Minimal APIs and Routing in ASP.NET 8:** ASP.NET 8's Minimal APIs simplify endpoint definitions, providing lightweight syntax for routing and handling HTTP requests.



3. Libraries Nuget Packages of Basket Microservice

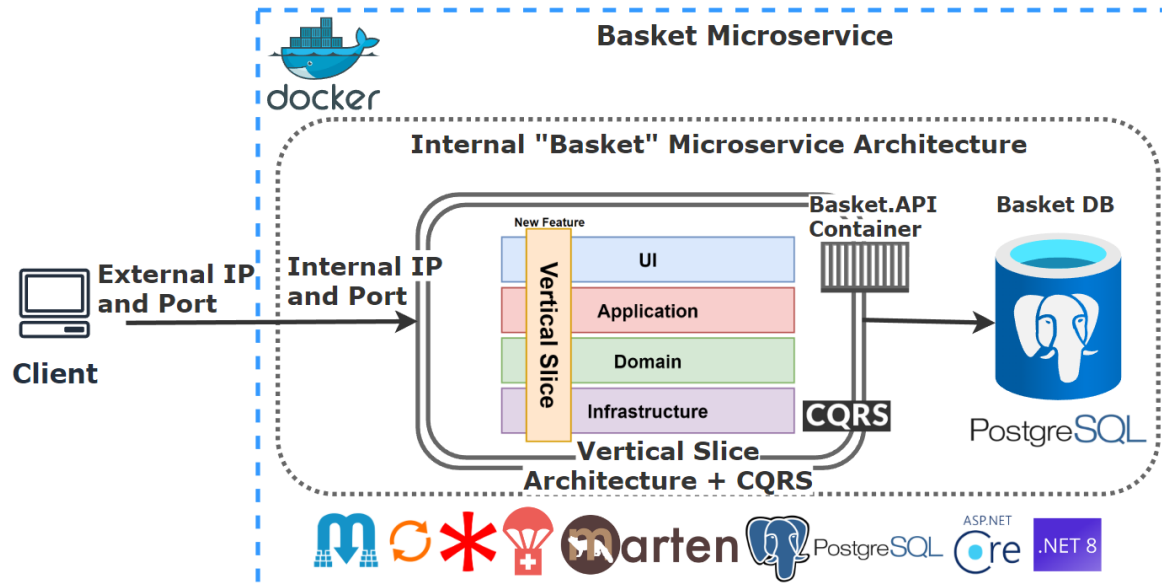
- **MediatR for CQRS:** MediatR library simplifies the implementation of the CQRS pattern.
- **Carter for API Endpoints:** Routing and handling HTTP requests, easier to define API endpoints with clean and concise code.
- **Marten for PostgreSQL Interaction:** Use PostgreSQL as a Document DB. It leverages PostgreSQL's JSON capabilities for storing, querying, and managing documents.
- **Mapster for Object Mapping:** Mapster is a fast, configurable object mapper that simplifies the task of mapping objects.
- **FluentValidation for Input Validation:** For building strongly-typed validation rules, ensure inputs are correct before processed.
- **Next Sections:** Scrutor, gRPC, StackExchange.Redis,

MassTransit



4. Project Folder Structure of Basket Microservices

- The project is organized into **Model**, **Features**, **Data**, and **Abstractions**.
- **Features** like **GetBasket** and **StoreBasket** have dedicated handlers and endpoint definitions.
- **Feature** folder will be **Basket**.
- **Data** folder and **Repository** objects manages database and cache interactions.

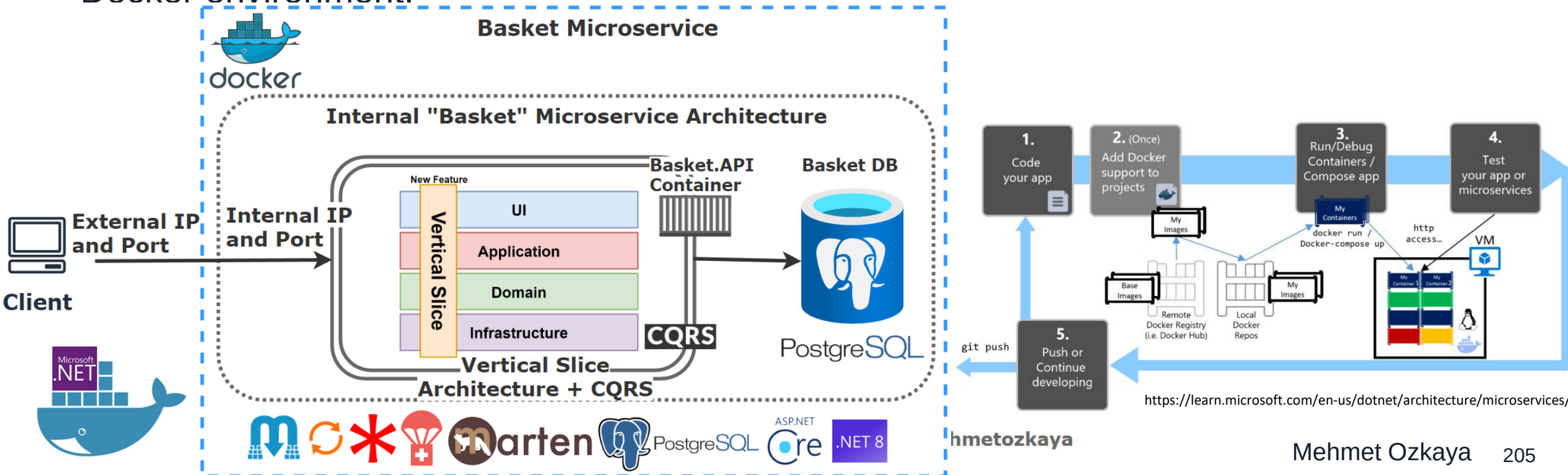


```
Basket.API
├── Connected Services
├── Dependencies
├── Properties
├── Basket
│   ├── CheckoutBasket
│   ├── DeleteBasket
│   └── GetBasket
│       ├── GetBasketEndpoints.cs
│       └── GetBasketHandler.cs
│   └── StoreBasket
├── Data
│   ├── BasketRepository.cs
│   ├── CachedBasketRepository.cs
│   └── IBasketRepository.cs
├── Dtos
├── Exceptions
├── GrpcServices
├── Models
│   ├── ShoppingCart.cs
│   └── ShoppingCartItem.cs
├── Protos
├── appsettings.json
├── Dockerfile
└── Program.cs
```

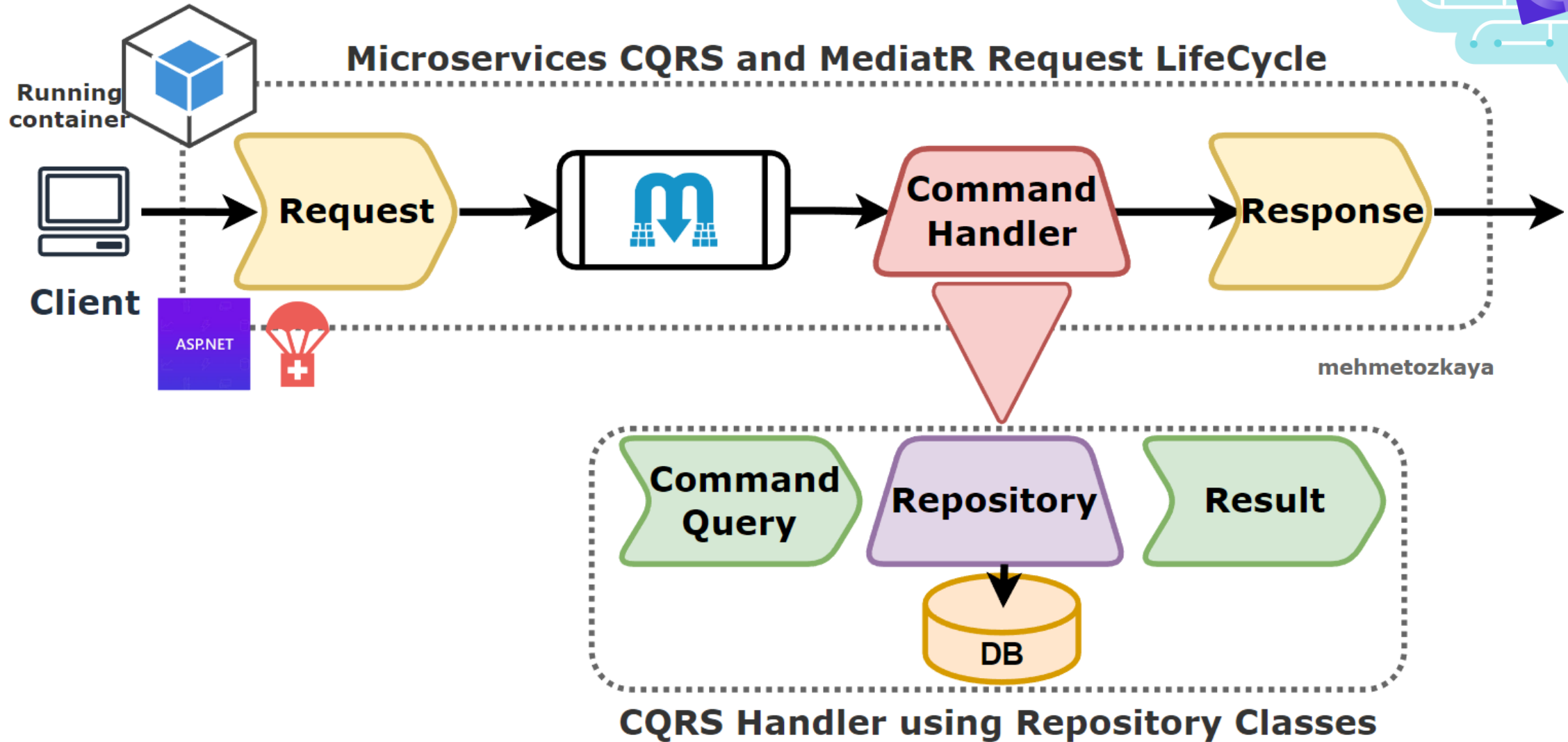


Deployment of Basket Microservices

- **Containerize** our service with **Docker**, ensuring deployment and integration with PostgreSQL
- Implement **Dockerfile** and **docker-compose** file for running Basket microservice and **PostgreSQL** database and **Redis** Cache in Docker environment.

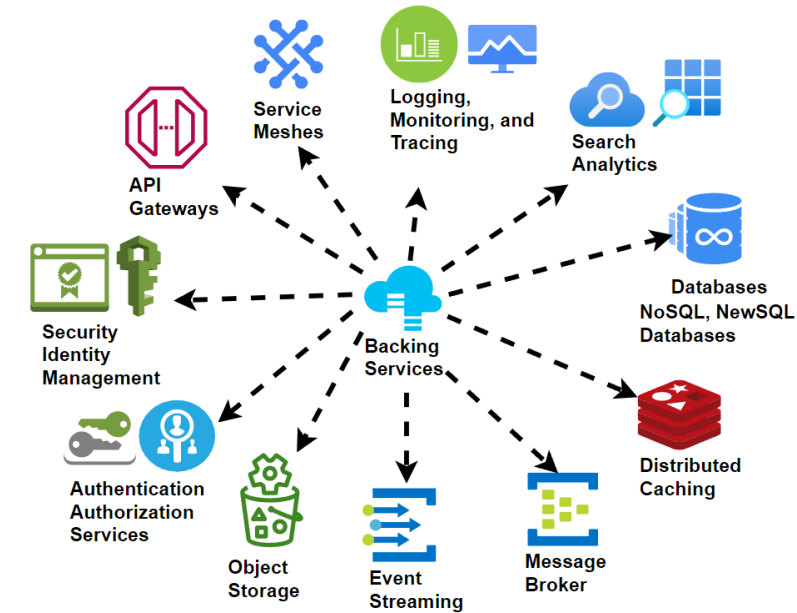


CQRS and MediatR Request LifeCycle

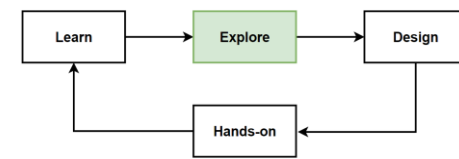
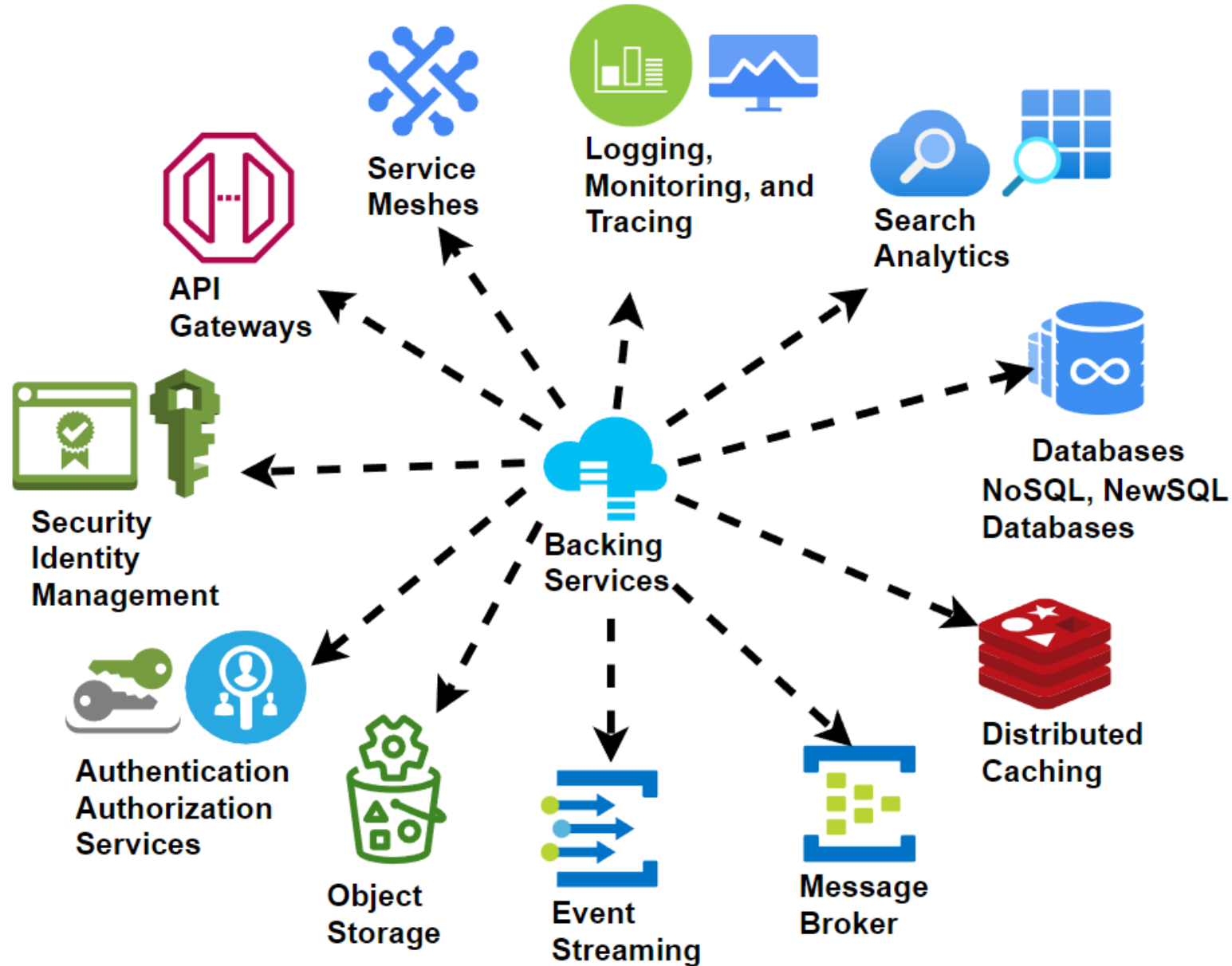


What is Backing Services for Cloud-Native Microservices

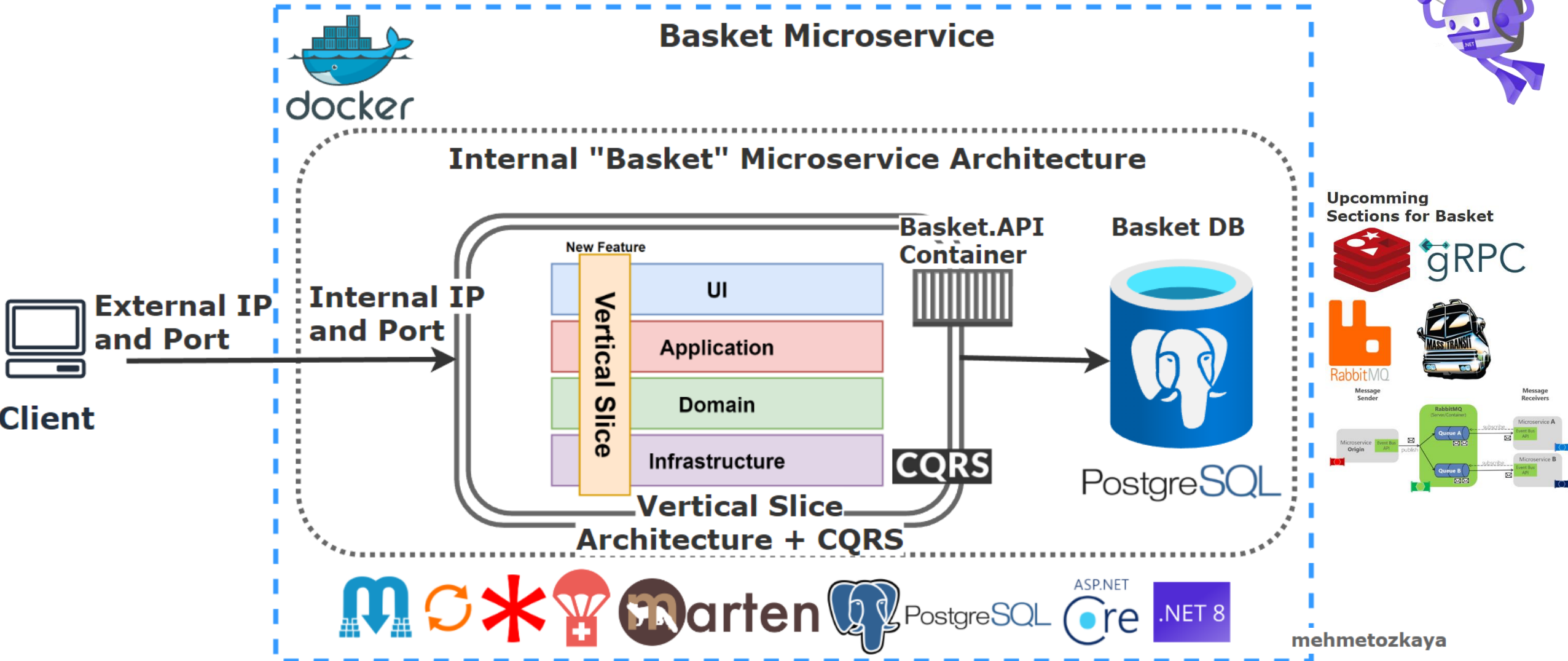
- **Backing services** for **cloud-native microservices** are the **external components** that microservices depend on for their operation.
- Provide support for **various functionalities**, such as **data storage, messaging, caching, and authentication**.
- **Backing services** like databases, messaging systems, and caching services are **treated as attached resources**.
- **Backing services** are external to the microservice and can be **swapped or replaced without changing** the microservice's core logic.
- **Decoupled from the microservices** themselves, promoting flexibility, scalability, and easier maintenance.



Backing Services for Cloud-Native Microservices

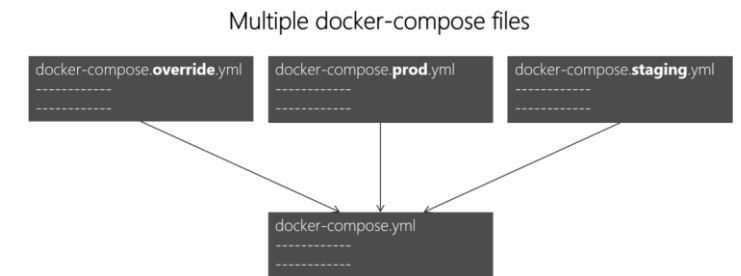
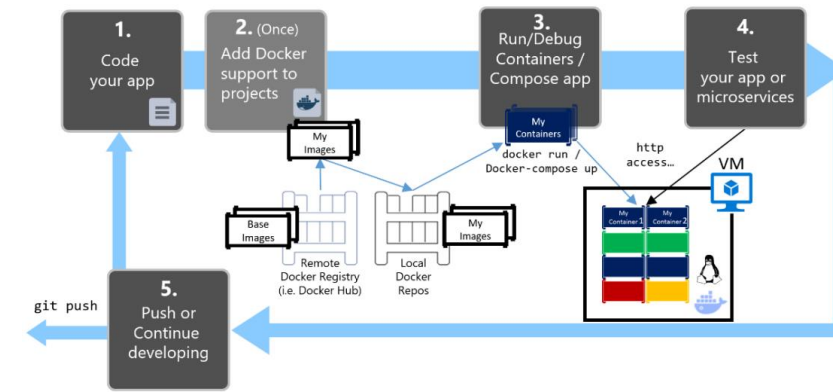
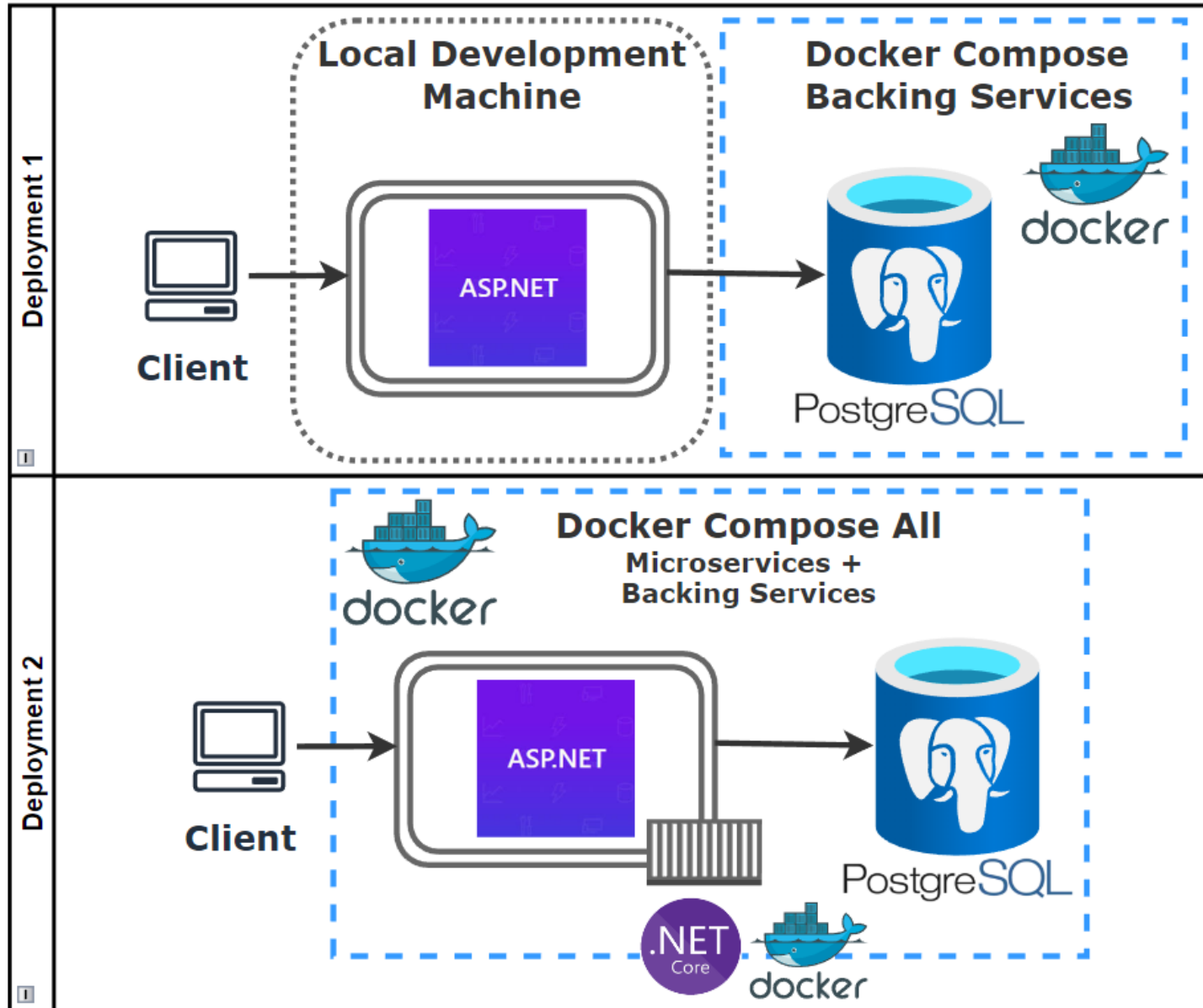


Developing Basket.API Microservices

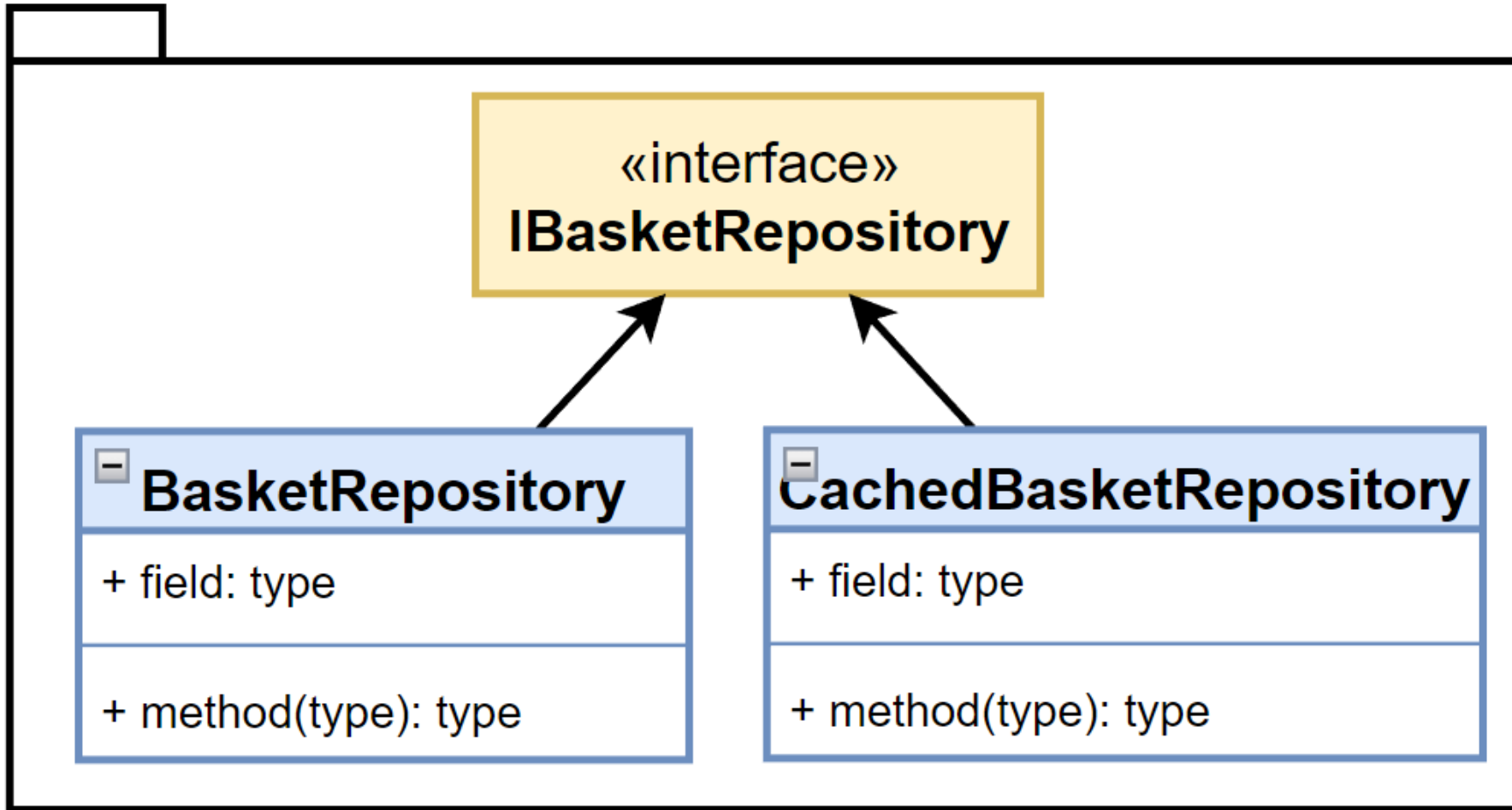


mehmetozkaya

EShop Microservices Deployment Strategy

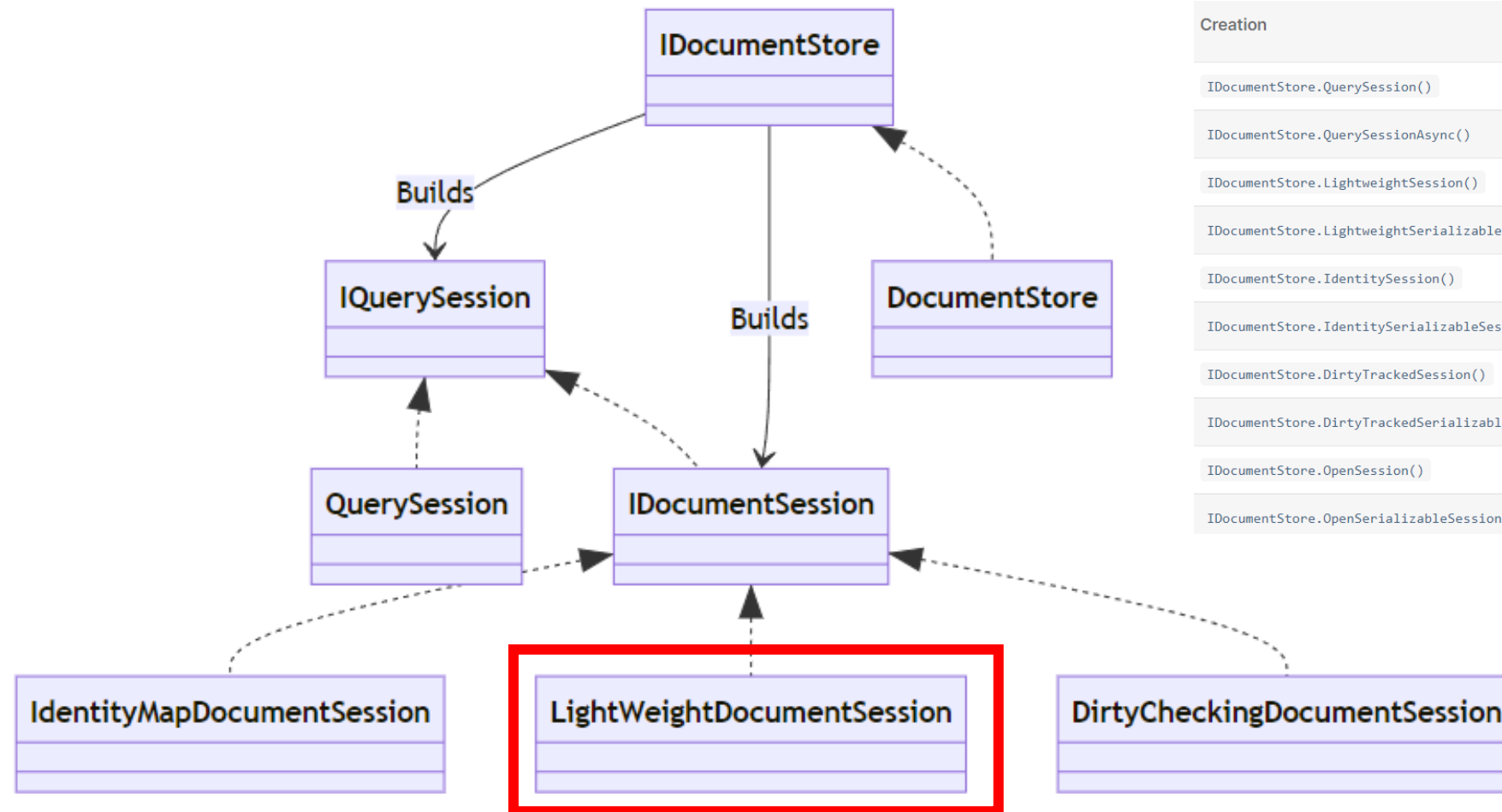


Basket.API Microservices Repository Class Diagram



- **Patterns:** Repository, Decorator and Cache-aside Pattern

Opening Sessions in Marten as Document DB



Creation	Read/Write	Identity Map	Dirty Checking
<code>IDocumentStore.QuerySession()</code>	Read Only	No	No
<code>IDocumentStore.QuerySessionAsync()</code>	Read Only	No	No
<code>IDocumentStore.LightweightSession()</code>	Read/Write	No	No
<code>IDocumentStore.LightweightSerializableSessionAsync()</code>	Read/Write	No	No
<code>IDocumentStore.IdentitySession()</code>	Read/Write	Yes	Yes
<code>IDocumentStore.IdentitySerializableSessionAsync()</code>	Read/Write	Yes	Yes
<code>IDocumentStore.DirtyTrackedSession()</code>	Read/Write	Yes	Yes
<code>IDocumentStore.DirtyTrackedSerializableSessionAsync()</code>	Read/Write	Yes	Yes
<code>IDocumentStore.OpenSession()</code>	Read/Write	Yes	No
<code>IDocumentStore.OpenSerializableSessionAsync()</code>	Read/Write	Yes	No

***Best practice**

<https://martendb.io/documents/sessions.html>

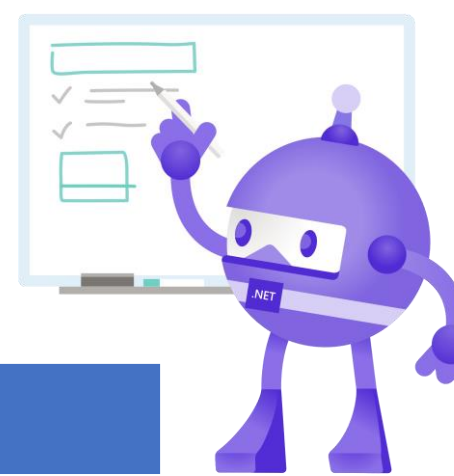


PostgreSQL



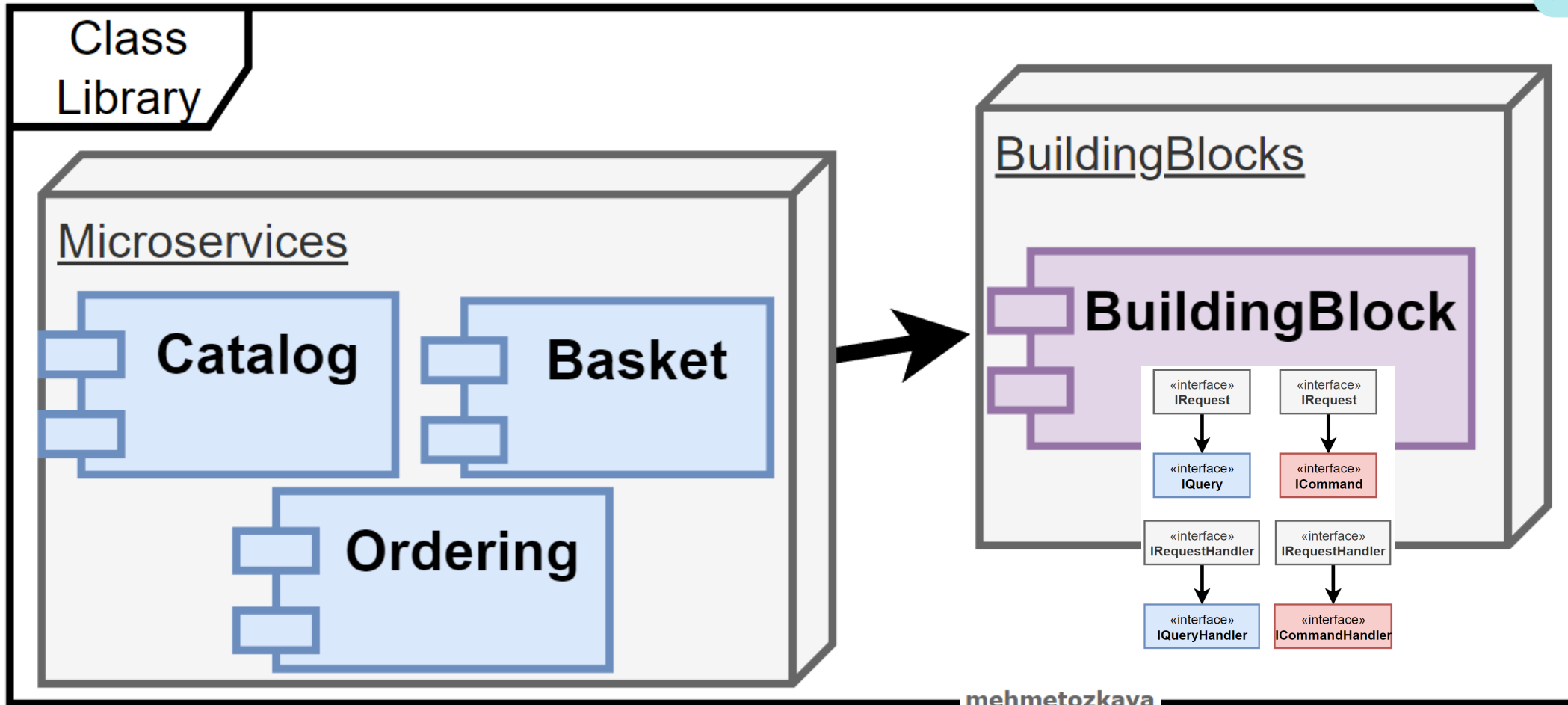
marten

3. Rest API Endpoints of Basket Microservices



Method	Request URI	Use Cases
GET	/basket/{userName}	Get basket w/ username
POST	/basket/{userName}	Store Basket (insert-update)
DELETE	/basket/{userName}	Delete basket w/ username
POST	/basket/checkout	Checkout basket

Eshop Microservices uses BuildingBlocks Library

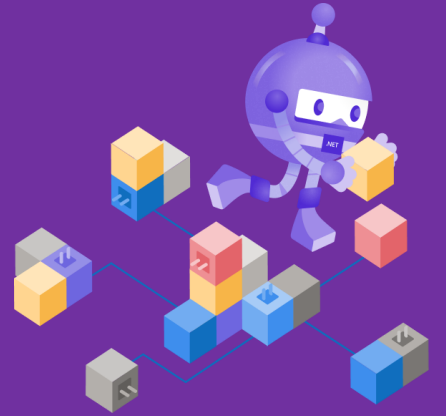


CQRS



mehmetozkaya

Develop Distributed Caching with Redis in Basket Microservices



Implement Proxy Pattern, Decorator Pattern and Scrutor Library

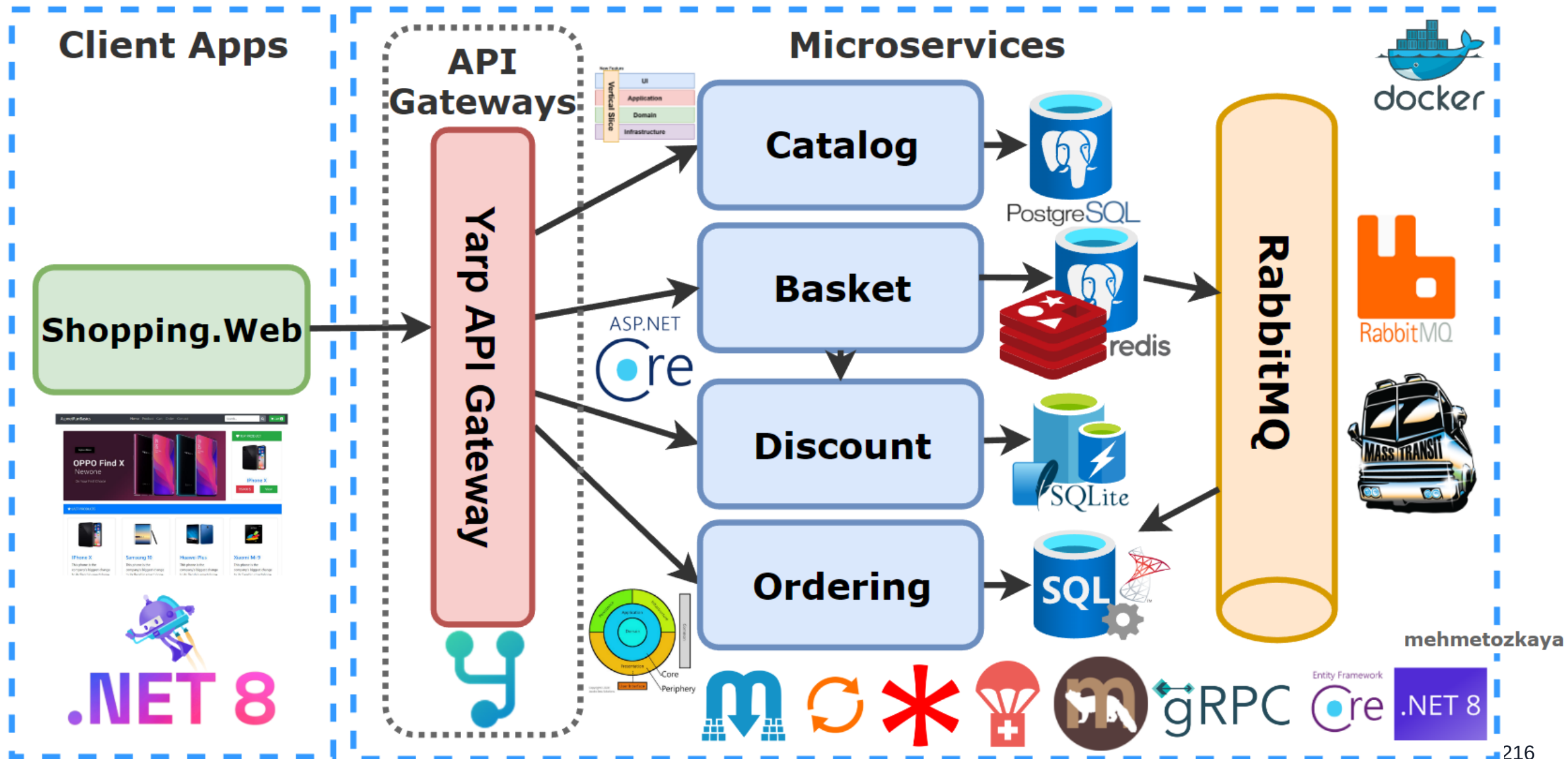
Implement Cache-aside Pattern / Cache invalidation

Develop Cached BasketRepository and Decorate w/ Scrutor Library

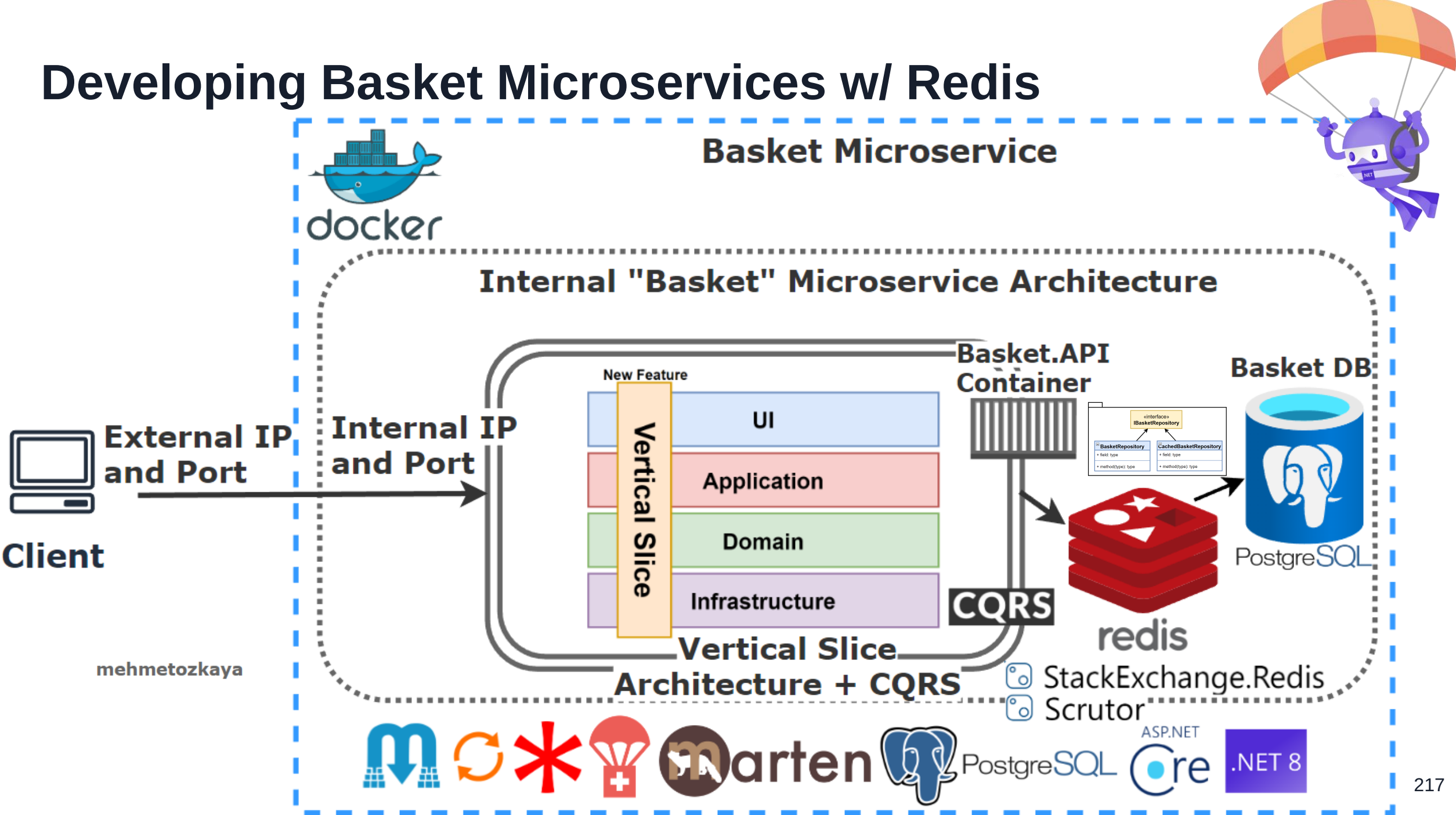
Setup Redis as a Distributed Cache using Docker-compose file for Multi-container docker environment



Big Picture – Course Project



Developing Basket Microservices w/ Redis



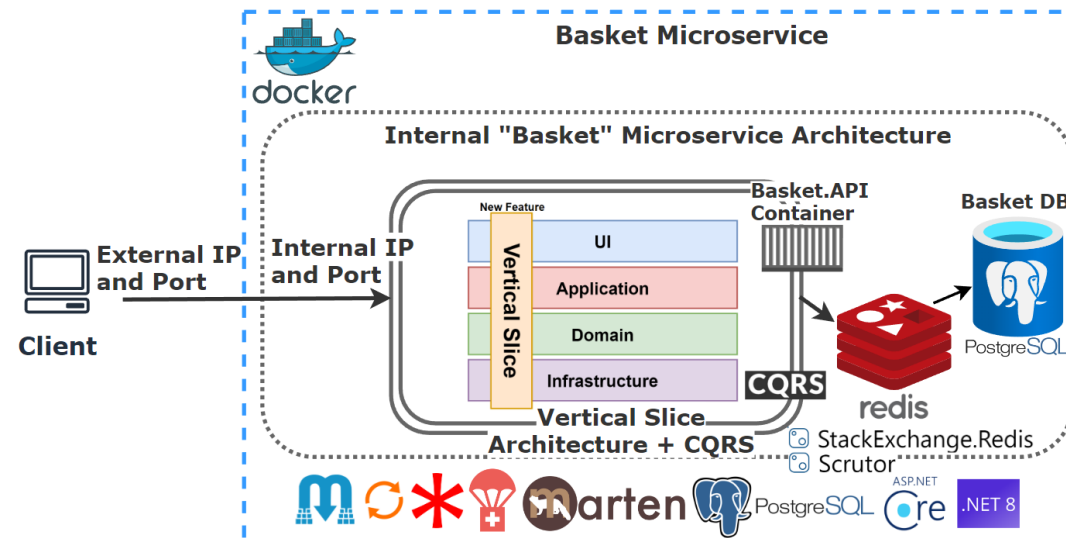
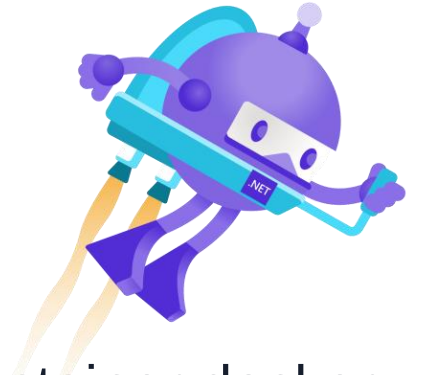
Why Choose Redis for Distributed Caching

- **Redis** is an **advanced key-value store** known for its high performance.
- It is often used for **caching**, session **storage**, **pub/sub systems**, and more.
- **Redis** offers **in-memory data storage**, resulting in fast data access.
- Its support for **various data structures** makes it versatile for different use cases.
- **Redis** is an **excellent choice for microservices architecture** primarily due to its inherent distributed characteristics.
- Enabling services to **access shared data quickly** and reducing the load on databases.



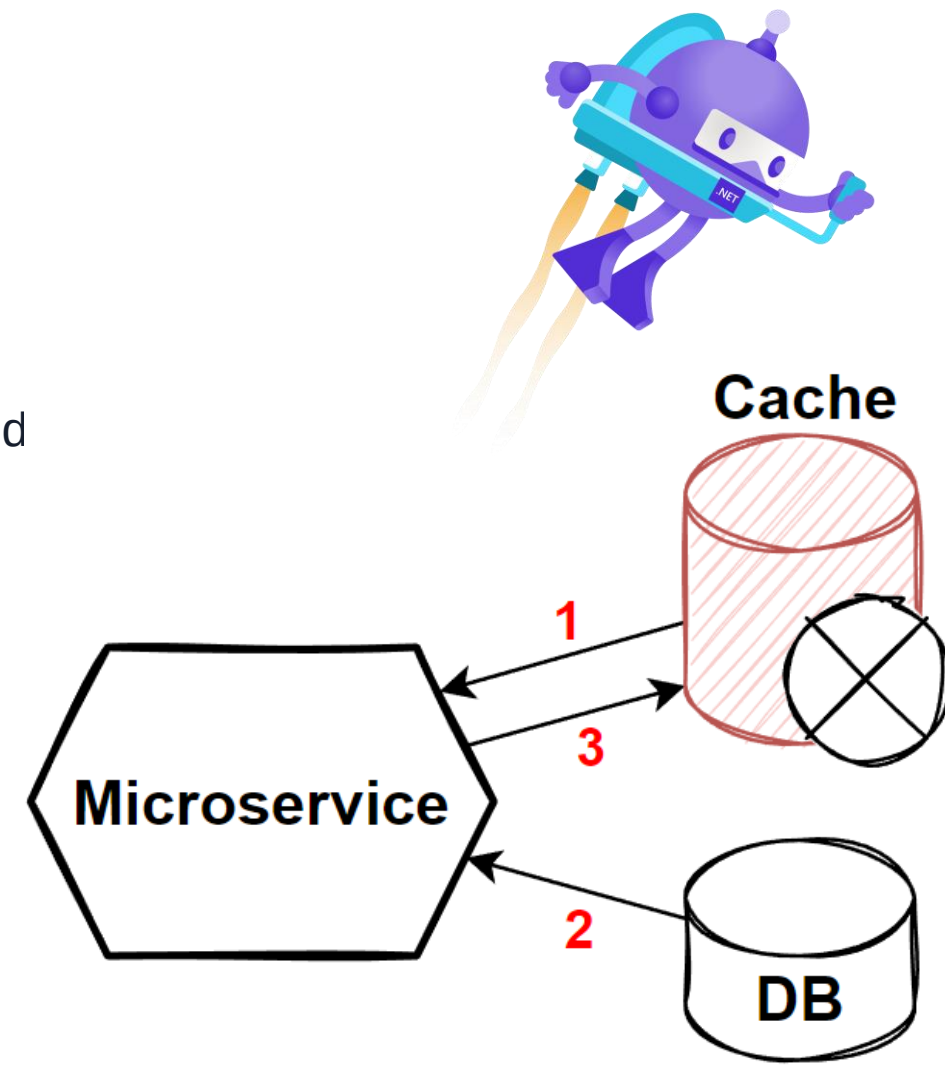
Distributed Caching with Redis in Basket Microservices

- Implement Proxy Pattern, Decorator Pattern and Scrutor Library
- Implement Cache-aside Pattern / Cache invalidation
- Develop Cached BasketRepository and Decorate w/ Scrutor Library
- Setup Redis as a Distributed Cache using Docker-compose file for Multi-container docker environment
- Integrate Redis into our Basket Microservices to cache user cart data
- Boost performance by reducing the load on our database and speeding up data retrieval.



Cache-Aside Pattern for Microservices

- **(1)** When a client needs to access data, it first checks to see if the data is in the cache.
- **(2)** If the data is in the cache, the client retrieves it from the cache and returns it to the caller.
- **(3)** If the data is not in the cache, the client retrieves it from the database, stores it in the cache, and then returns it to the caller.
- Some of caching systems provide **read-through** and **write-through/write-behind** operations. In these systems, **client application retrieves data over by the cache**.
- For not supported Caches, it's the **responsibility the applications use the cache and update the cache** if there is a **cache-miss**.
- **Microservices** good example to implement **Cache-Aside pattern**, it is common to use a **distributed cache** that is **shared across multiple services**.

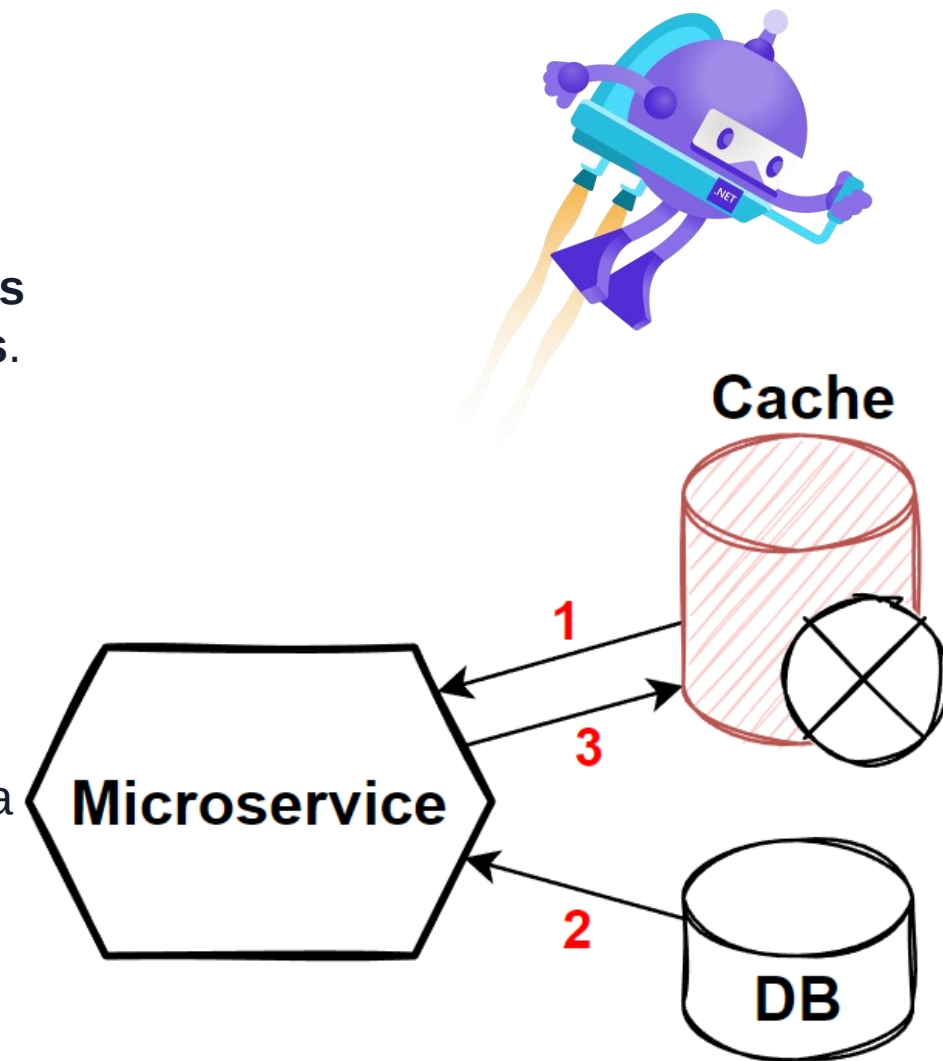


Cache-Aside Pattern for Microservices - 2

- **Cache-aside pattern** can improve performance of a microservices architecture, by **reducing the number of expensive database calls**.
- To use the Cache-aside pattern in a microservice, **need to implement a cache layer** in your service.
- Involve using a **Cache library** or framework, such as **Redis** or **Memcached**, or implementing a custom cache solution.

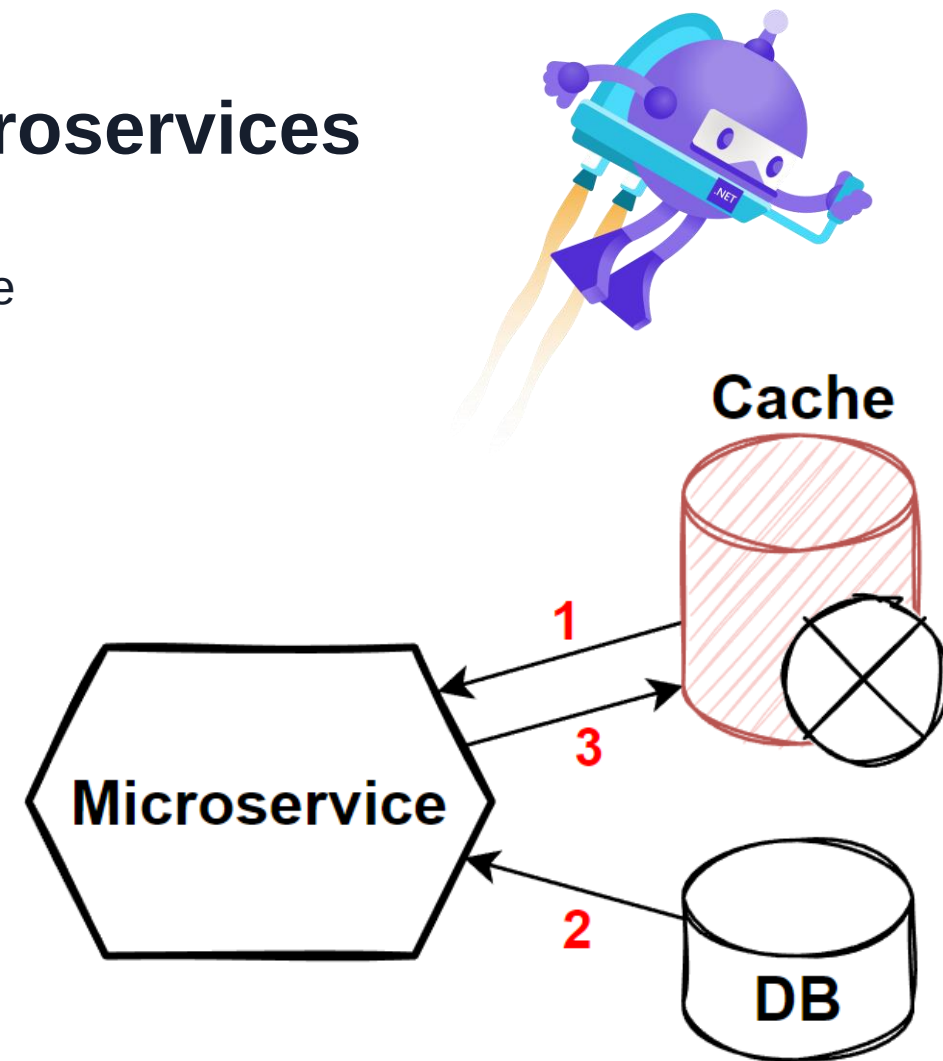
Process:

- When a service needs to access data, it first checks to see if the data is in the cache.
- If the data is in the cache, the service retrieves it from the cache and returns it to the caller.
- If the data is not in the cache, the service retrieves it from the database or other data store, stores it in the cache, and then returns it to the caller.



Drawbacks of Cache-Aside Pattern for Microservices

- **Cache** can introduce **additional complexity** and may not be suitable for all situations.
- The cache may **need to be invalidated** or **refreshed** when data is updated in the database or data store.
- This can **require additional coordination** between the microservices.
- The cache may introduce **additional latency** if it is located remotely from the microservices that are using it.



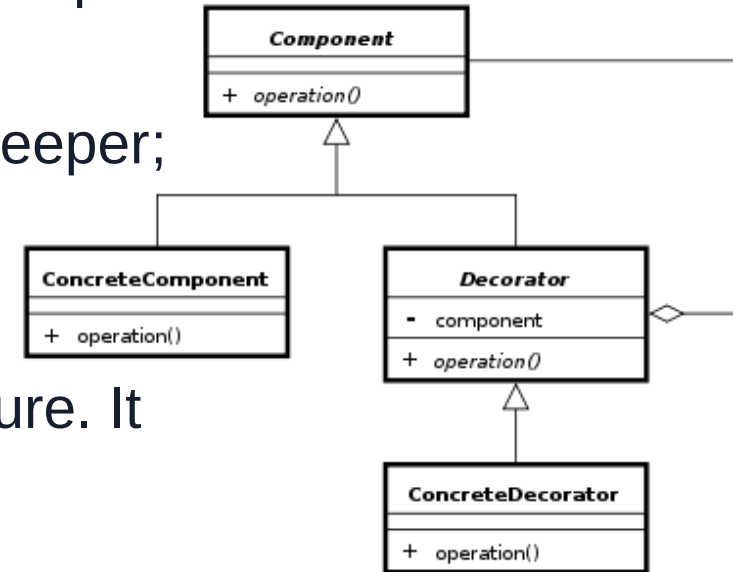
What is Proxy and Decorator Patterns ?

Proxy Pattern:

- Provides a placeholder for another object to control access to it. This pattern creates a proxy object that serves as an intermediary for requests intended for the original object.
- Lazy loading, controlling access, logging.. It's like having a gatekeeper; add extra behavior or checks before accessing the actual object.

Decorator Pattern:

- Dynamically adds behavior to an object without altering its structure. It involves a set of decorator classes that are used to extend the functionality of the original class without changing its code.
- Useful for adding functionalities to objects at runtime.
- For example, enhancing a window object with additional features like borders, scrollbars dynamically.



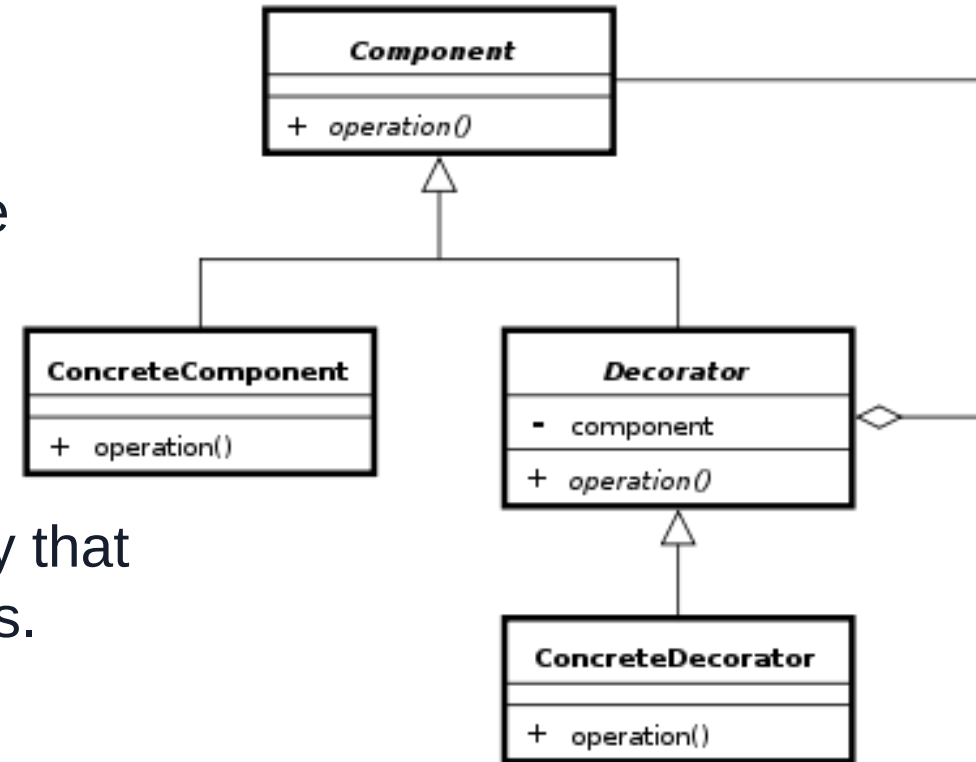
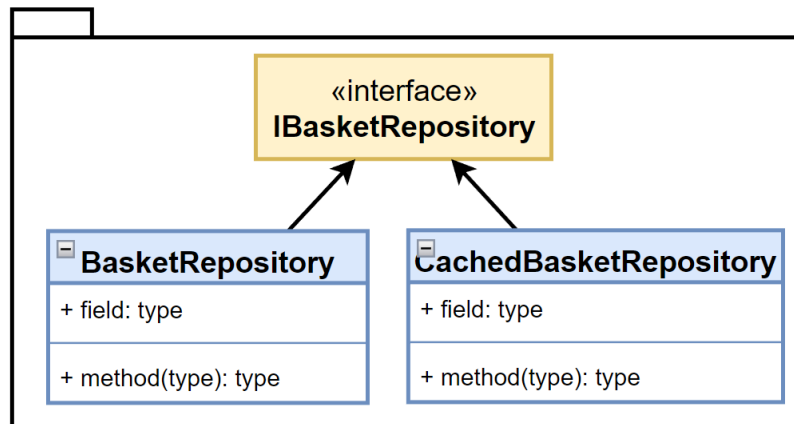
How Can We Use Decorator Pattern?

Implementation:

- Create abstract decorators that implement the same interface as the object they will extend. Then, concrete decorator classes add additional behavior.

Example:

- If you have a BasketRepository class for database operations, you can create a CachedBasketRepository that extends the BasketRepository with caching capabilities.



What is Scrutor Library? Why We Use It?

Scrutor Library:

- .NET library that extends the built-in IOC container of ASP.NET Core. It provides additional capabilities to scan and register services in a more flexible way.

Usage:

- Implementing patterns like Decorator in a clean and manageable way. It simplifies the process of service registration and decoration in ASP.NET Core applications.



Implement Decorator Pattern Using Scrutor Library

1. Define Interface & Base Implementation:

- Start with an interface, e.g., `IBasketRepository`, and a base implementation, e.g., `BasketRepository`.

2. Create Decorator:

- Create a decorator class like `CachedBasketRepository` that also implements `IBasketRepository` but adds caching logic.

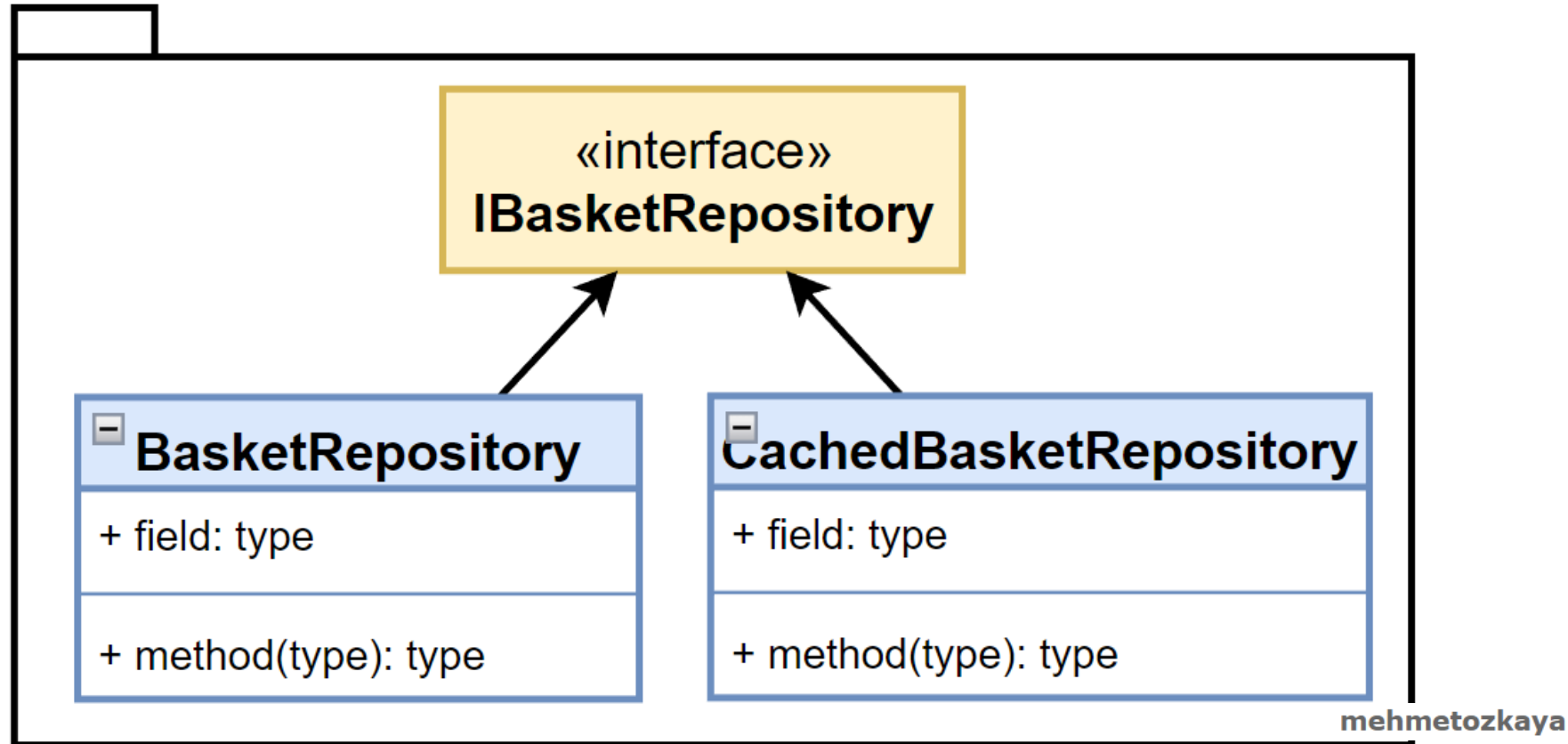
3. Register Services with Scrutor:

- Use Scrutor in `Program.cs` to first register the base service (`BasketRepository`) and then apply the decorator (`CachedBasketRepository`).

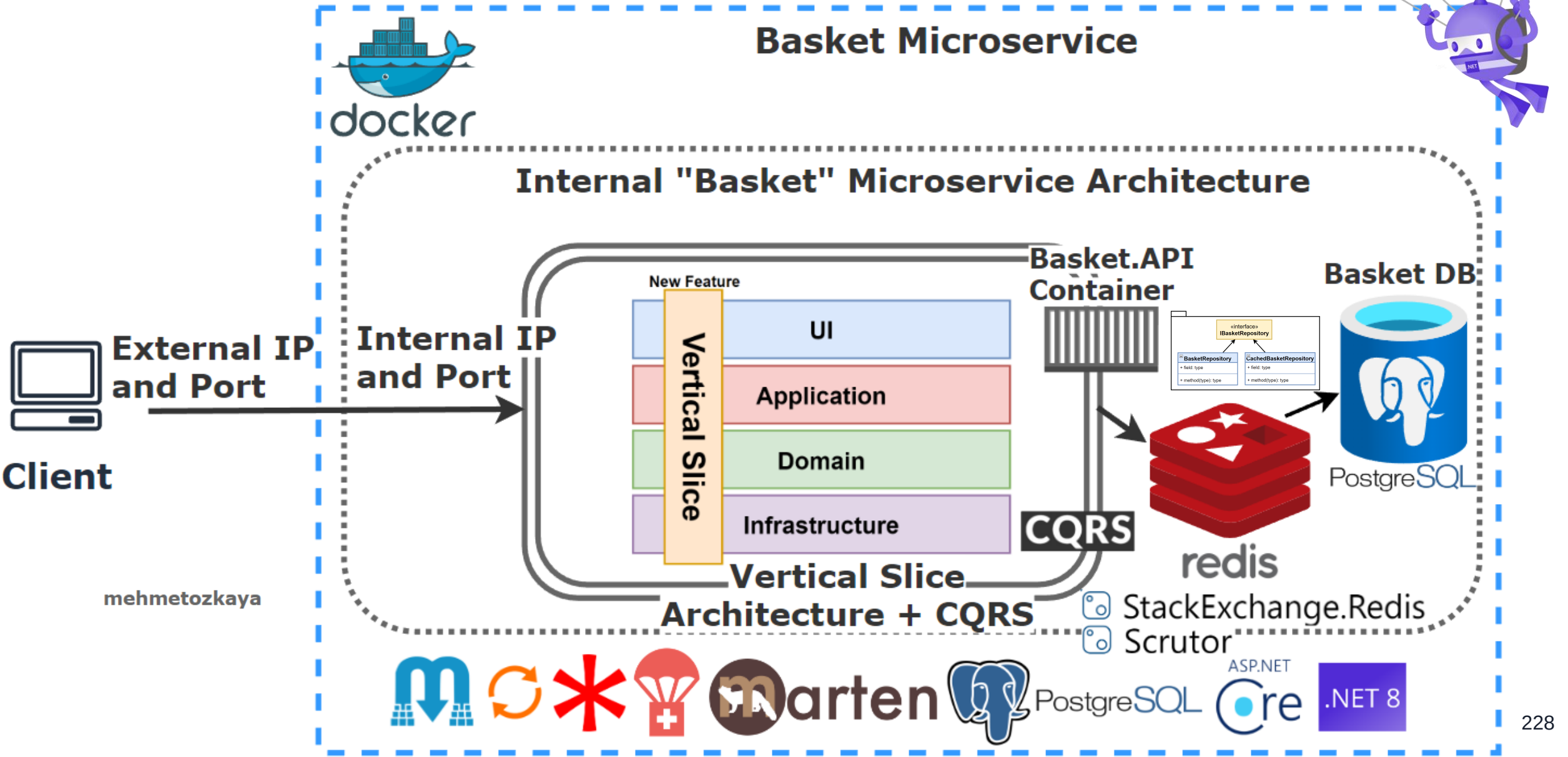


```
builder.Services.AddScoped<IBasketRepository, BasketRepository>();  
builder.Services.Decorate<IBasketRepository, CachedBasketRepository>();
```

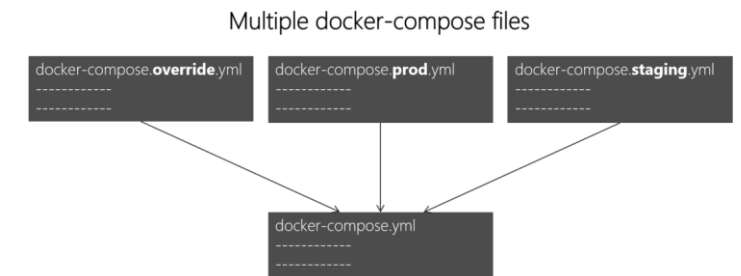
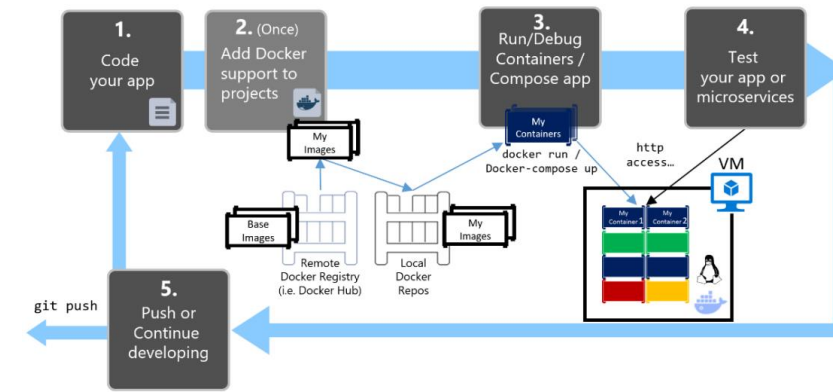
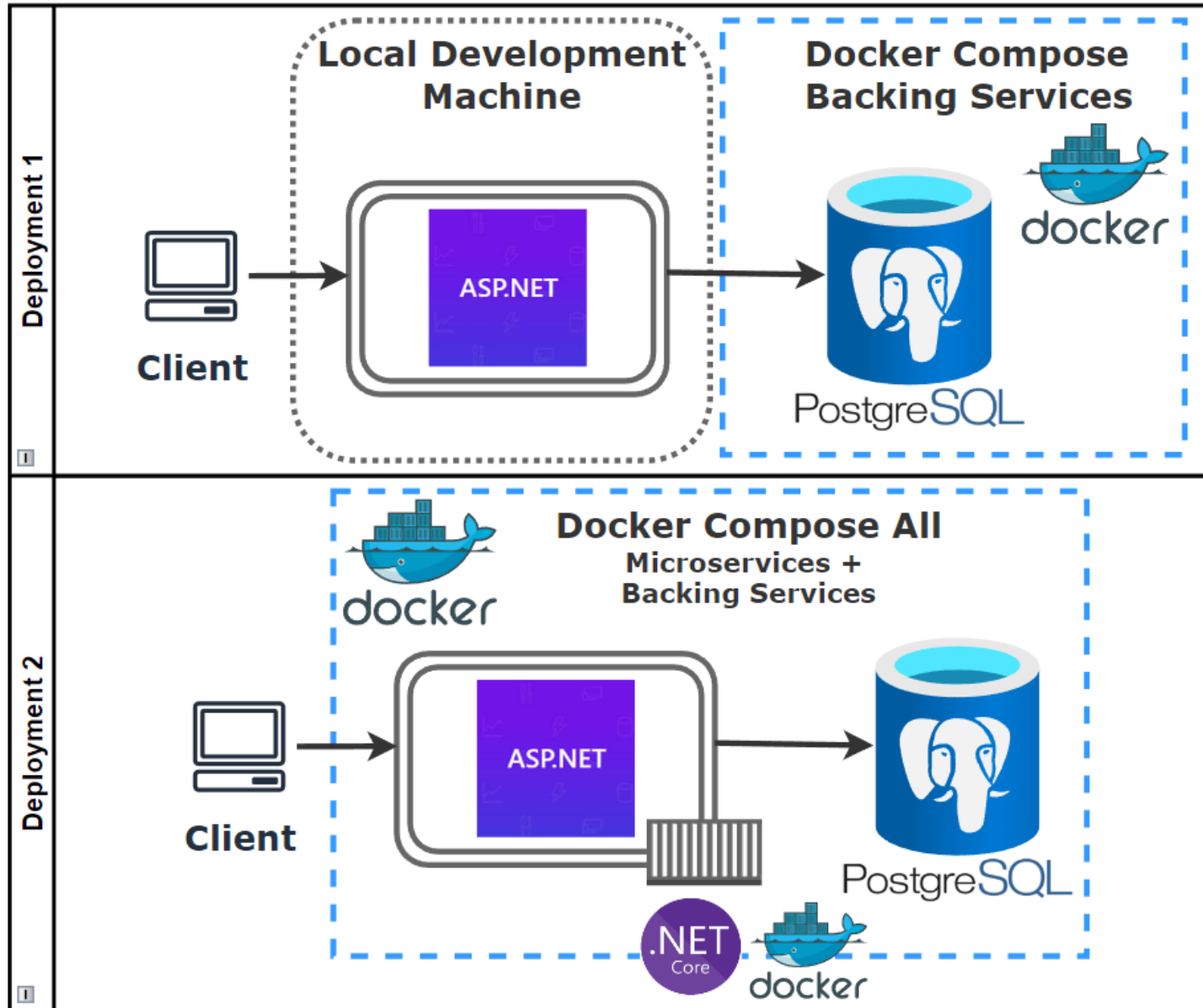
Developing Basket Microservices w/ Redis



Developing Basket Microservices w/ Redis



EShop Microservices Deployment Strategy

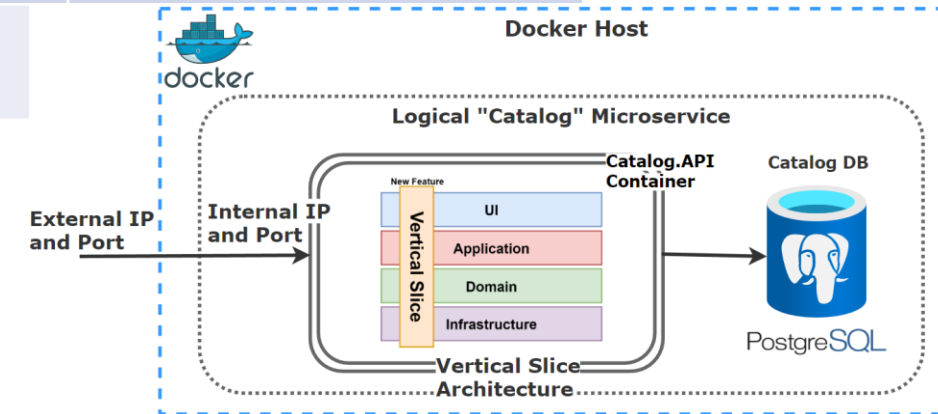


Port Numbers for Basket.API Microservice



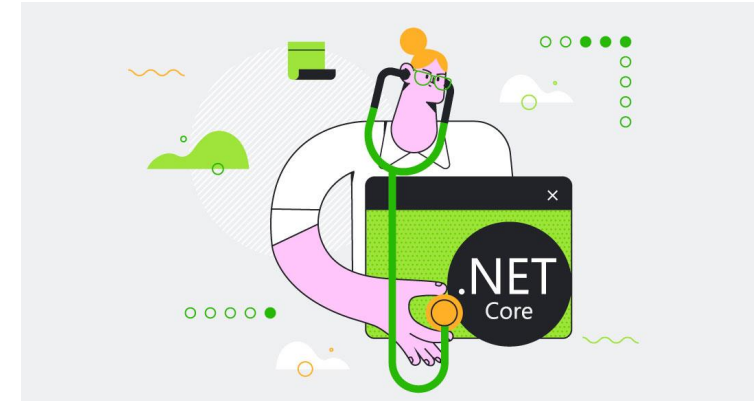
Microservices	Local Env	Docker Env	Docker Inside
Catalog	5000 – 5050	6000 – 6060	8080 – 8081
Basket	5001 – 5051	6001 – 6061	8080 – 8081
Discount	5002 – 5052	6002 – 6062	8080 – 8081
Ordering	5003 – 5053	6003 – 6063	

- ASP.NET Core **Ports** are **listed** as **http / https** for running application.



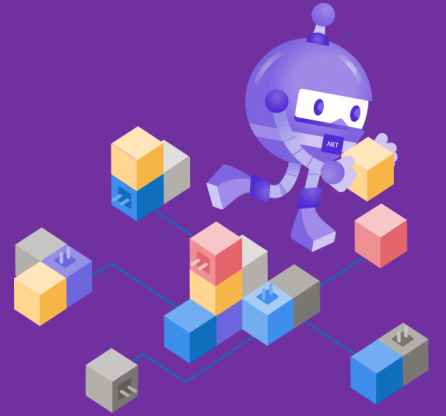
Health checks in ASP.NET Core

- **Health checks** in ASP.NET Core provide a way to **monitor** the **status** of your **application** and its **dependencies**.
- **Add health checks** to our **Basket microservices** with checking the health of our **PostgreSQL database** and **Redis Cache**.
- Health checks are exposed by an app as **HTTP endpoints**. Health check endpoints can be configured for various **real-time monitoring** scenarios.
- **Container orchestrator** may respond to a **failing health check** by **halting a rolling deployment** or restarting a container.
- **Load balancer** might react to an **unhealthy** app by **routing traffic** away from the **failing instance** to a **healthy instance**.



<https://blog.devart.com/how-to-implement-health-checks-in-asp-net-6.htm>

Microservices Synchronous Communications with gRPC



Microservices Communication Types: Synchronous or Asynchronous Communication

Microservices Synchronous Communications and Best Practices

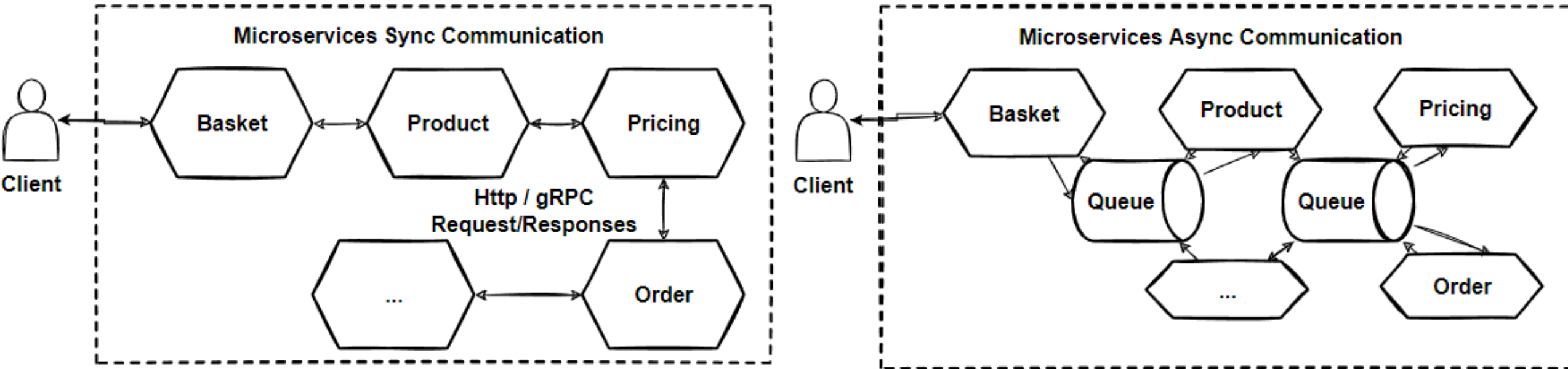
gRPC: High Performance Remote Procedure Calls

gRPC Usage in Microservices Communication

Mehmet Ozkaya

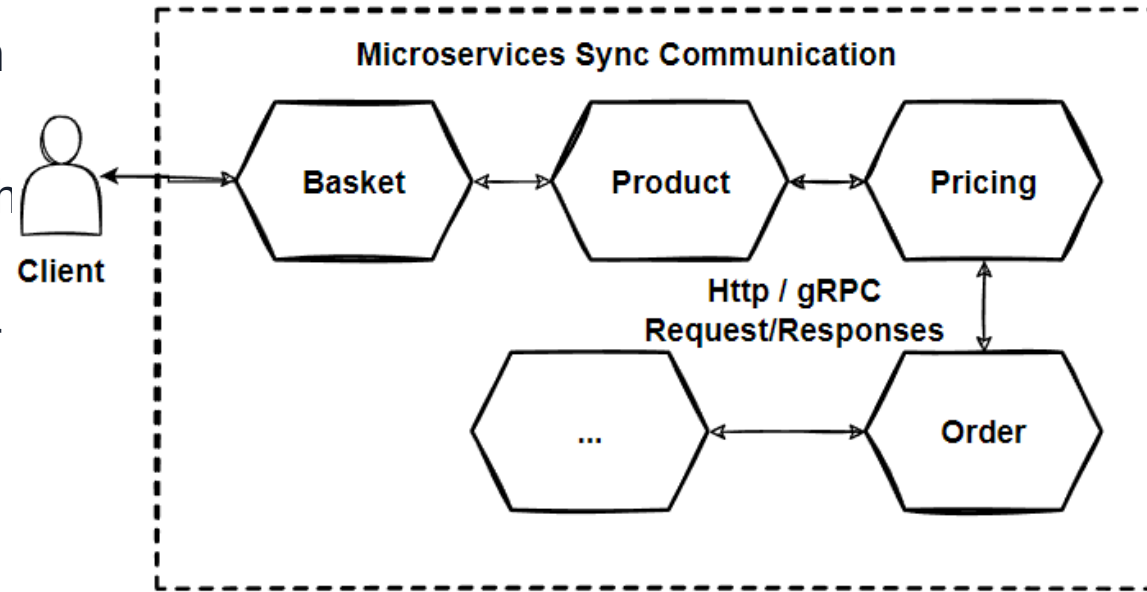


Microservices Communication Types - Sync or Async



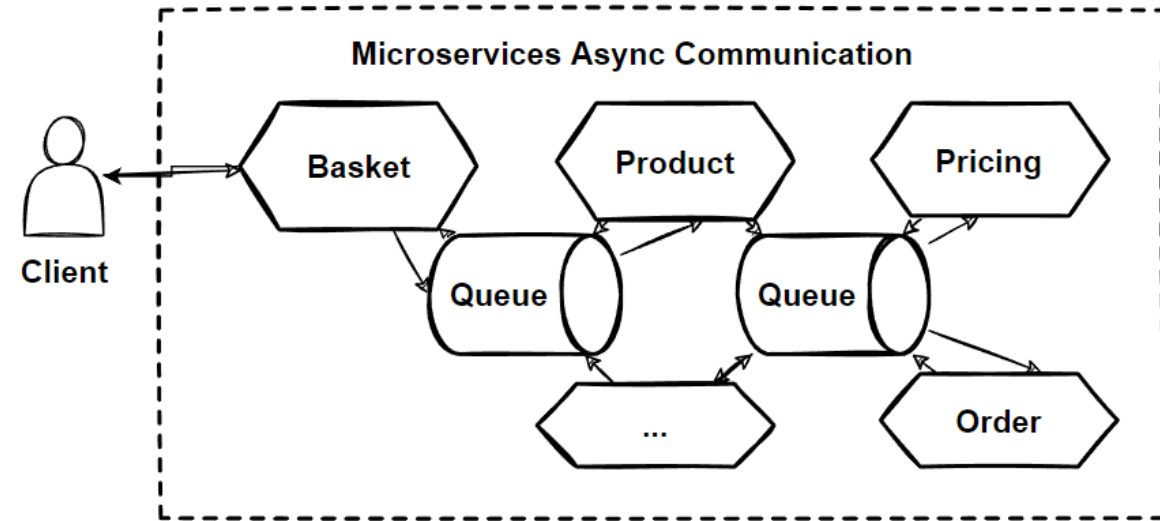
Microservices Synchronous Communication

- **Synchronous communication** is using **HTTP** or **gRPC** protocol for returning synchronous response.
- The client **sends a request** and **waits for a response** from the service.
- The client code **block their thread**, until the response reach from the server.
- The synchronous communication protocols can be **HTTP** or **HTTPS**.
- The client sends a request with using **http protocols** and waits for a response from the service.
- The client **call the server** and **block** client their operations.
- The client code will **continue** its task when it **receives** the HTTP server **response**.



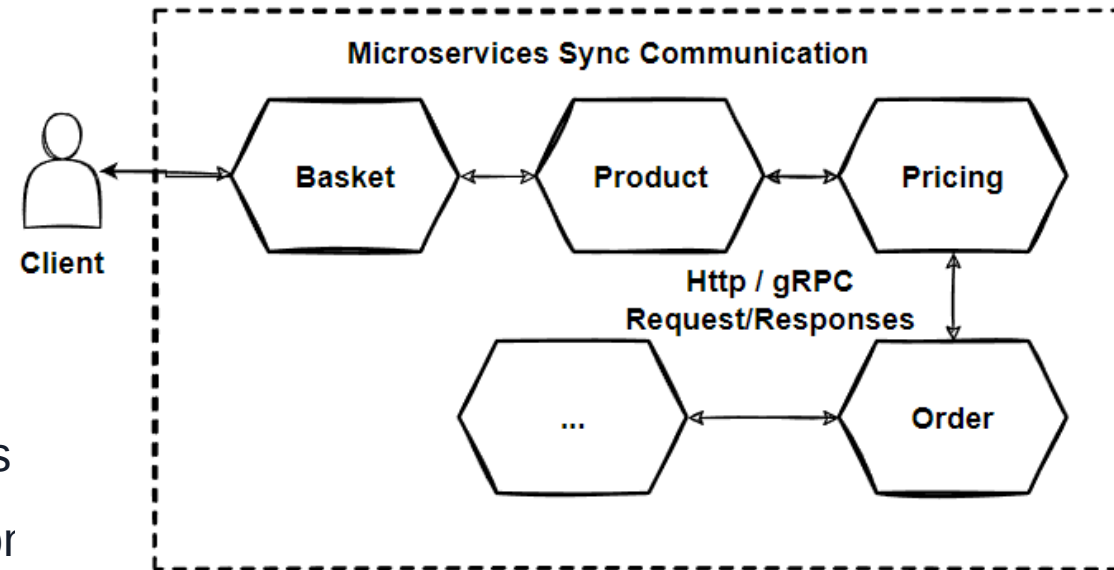
Microservices Asynchronous Communication

- The client sends a request but it **doesn't wait for a response** from the service.
- The client **should not have blocked** a thread while waiting for a response.
- **AMQP** (Advanced Message Queuing Protocol)
- Using AMQP protocols, the client **sends the message** with using message broker systems like **Kafka** and **RabbitMQ** queue.
- The message **producer does not wait** for a **response**.
- Message consume from the **subscriber** systems in async way, and no one waiting for response **suddenly**.
- **Asynchronous** communication also divided by 2:
 - **one-to-one(queue)**
 - **one-to-many (topic)**

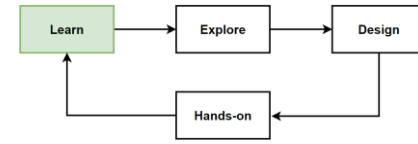


Microservices Synchronous Communications and Best Practices

- The client **sends a request** with using http protocols and **waits for a response** from the service.
 - The synchronous communication protocols can be **HTTP** or **HTTPS**.
 - **Request/response communication** with HTTP and REST Protocol (extends gRPC and GraphQL)
 - **REST HTTP APIs** when exposing from microservices
 - **gRPC APIs** when communicate internal microservices
 - **GraphQL APIs** when structured flexible data in microservices
 - **WebSocket APIs** when real-time bi-directional communication
-
- How can we design and exposing APIs with HTTP protocols for our microservices ?



gRPC: High Performance Remote Procedure Calls

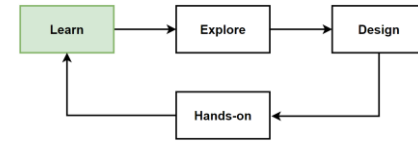


- **gRPC** is an open source **remote procedure call** (RPC) system developed at Google.
- gRPC is a framework to **efficiently connect services** and build distributed systems.
- It is **focused on high performance** and uses the **HTTP/2 protocol** to transport binary messages.
- It relies on the **Protocol Buffers** language to define service contracts.
- **Protocol Buffers (Protobuf)**, allow to define the interface to be used in service to service communication regardless of the programming language.
- It generates **cross-platform client** and **server bindings** for many languages.
- Most common **usage** scenarios include connecting services in **microservices style architecture**.



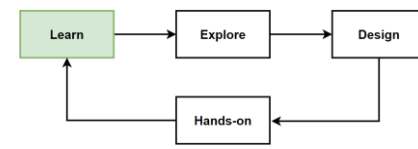
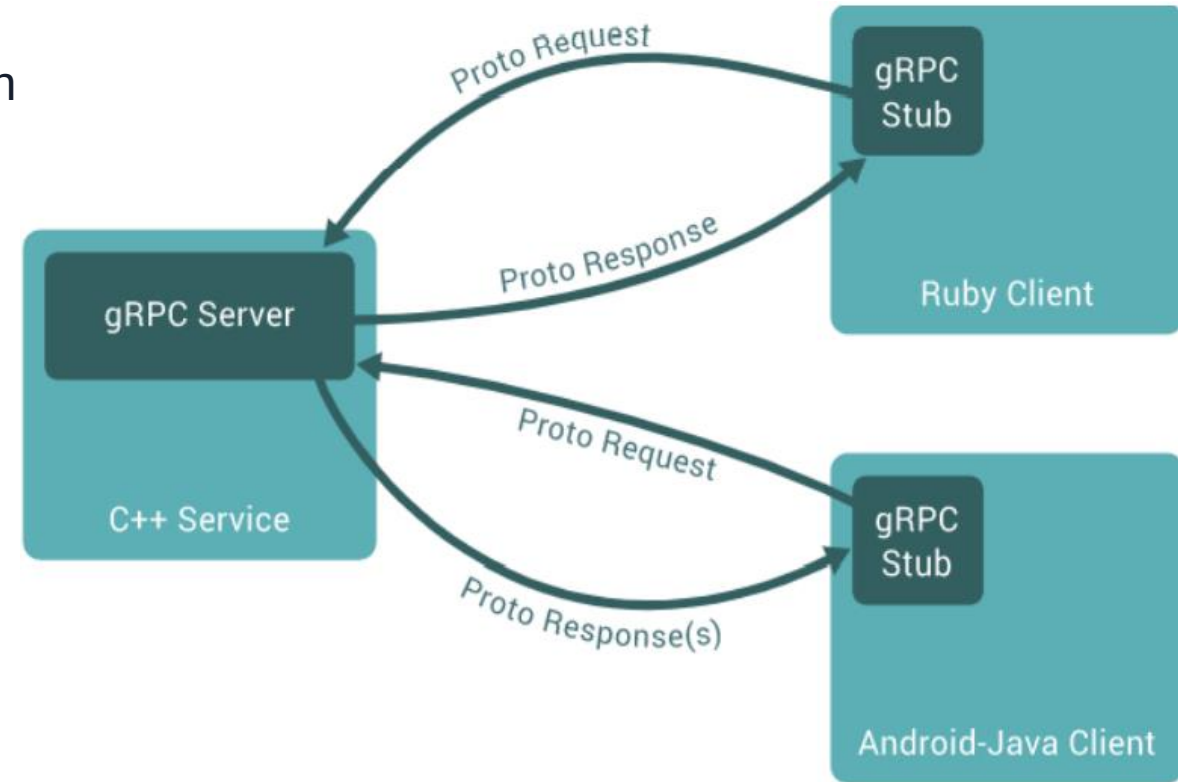
gRPC: High Performance Remote Procedure Calls

- **gRPC framework** allows developers to create services that can communicate with each other **efficiently** and **independently** with their **preferred programming language**.
- Once you **define** a **contract** with **Protobuf**, this contract used by each service to automatically **generate** the **code** that sets up the communication infrastructure.
- This feature simplifies the creation of service interaction and together with **high performance**, makes **gRPC** the **ideal framework** for creating **microservices**.

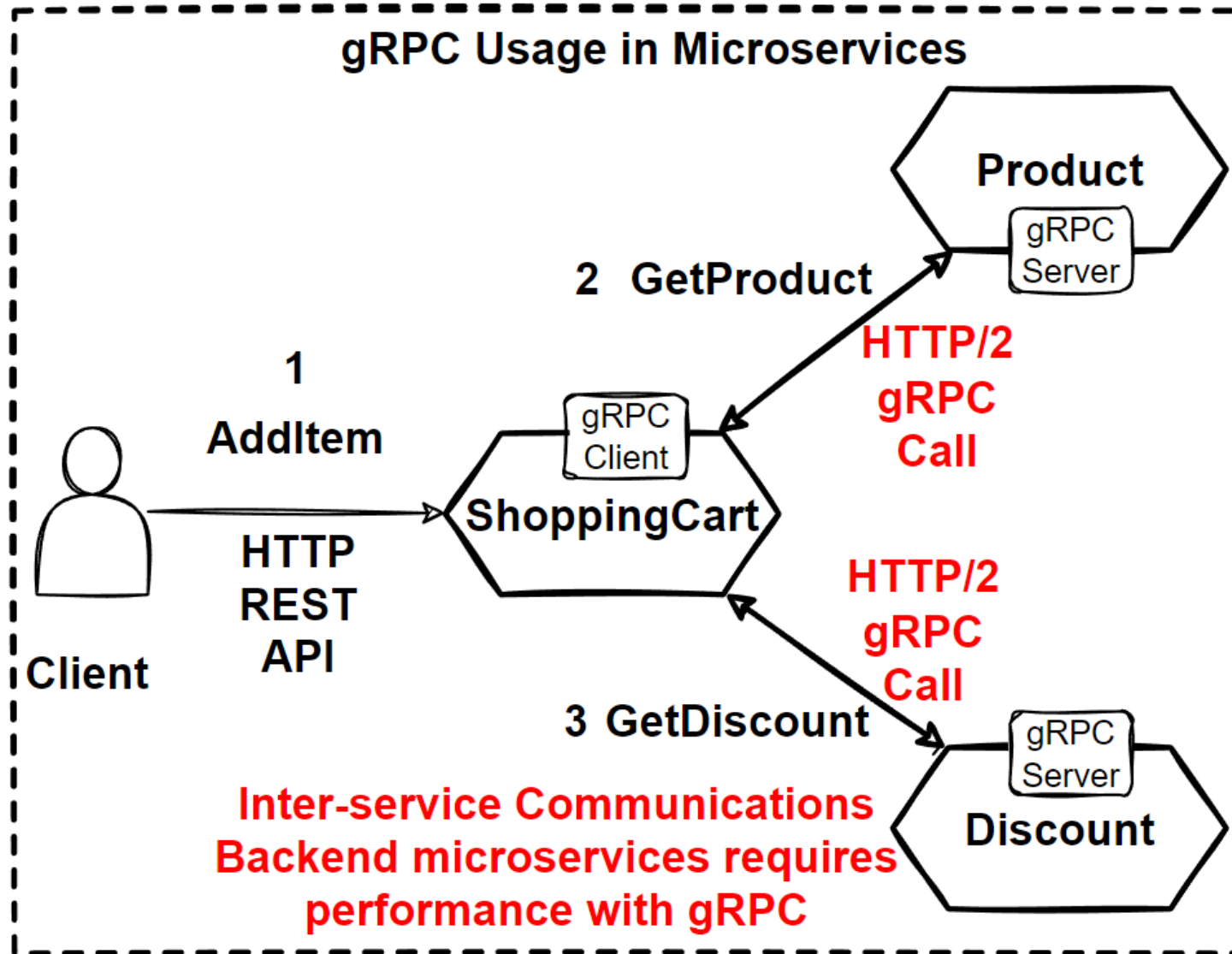
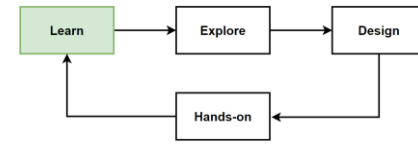


How gRPC works ?

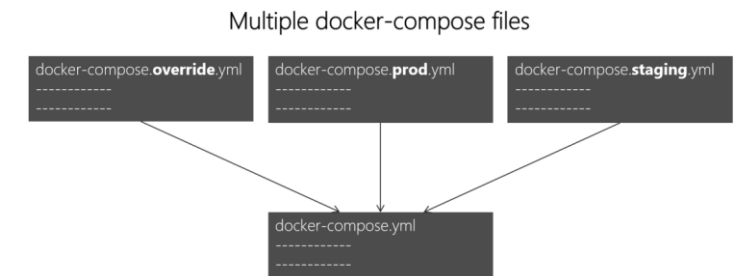
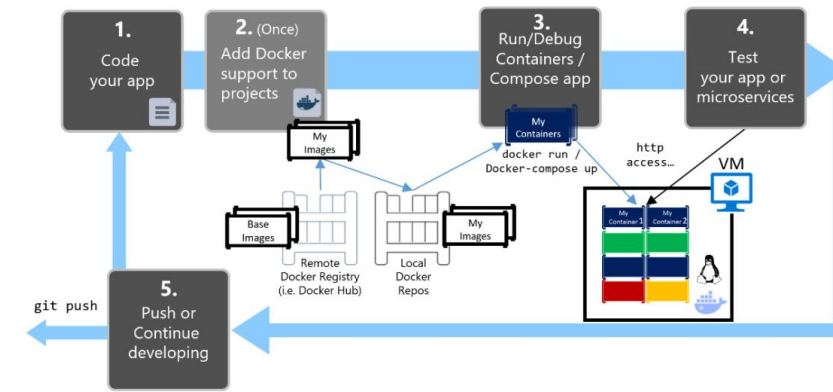
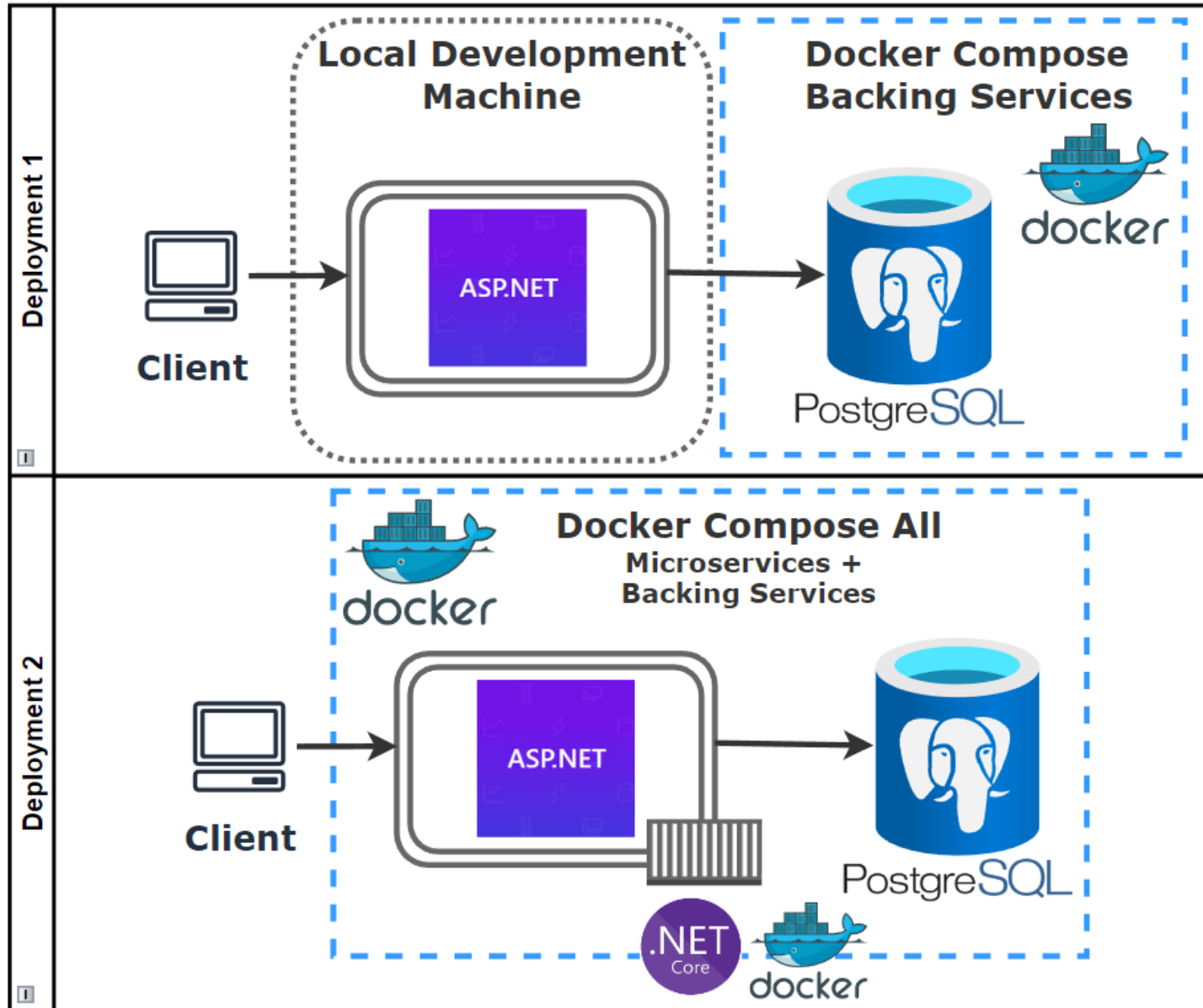
- **RPC** is a form of **Client-Server Communication** that uses a **function call** rather than a usual HTTP call.
- In **GRPC**, client application can **directly call a method** on a server application on a **different machine** like a local object.
- **gRPC** is based on the idea of **defining a service and methods**, and these can be called remotely with their parameters and return types.
- On the **server side**, the server implements this interface and runs a gRPC server to handle client calls.
- On the **client side**, the client has a stub that provides the same methods as the server.
- **gRPC clients** and **servers** can work and talk to each other in a different of environments.



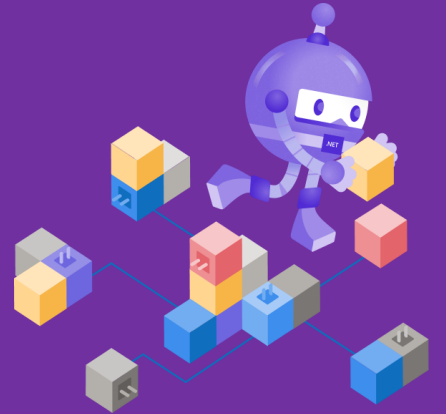
gRPC Usage in Microservices Communication



EShop Microservices Deployment Strategy



Develop Discount.Grpc Microservices for Microservices Grpc Communication



Build a Highly Performant inter-service gRPC Communication with
Discount and Basket Microservice

Exposing Grpc Services with creating Protobuf messages

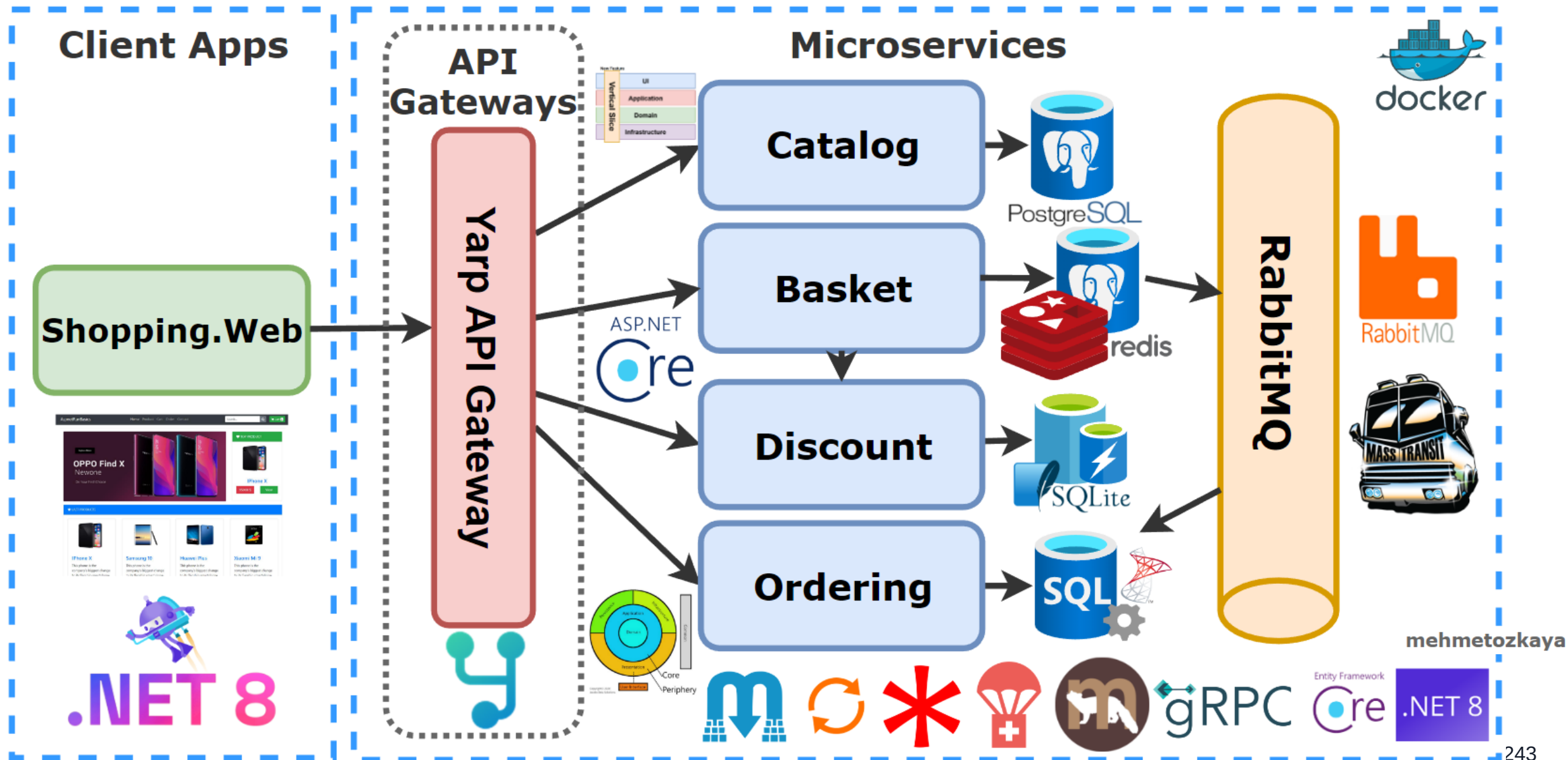
SQLite database connection and containerization

Entity Framework Core ORM - SQLite Data Provider and Migrations
simplify data access and ensure high performance

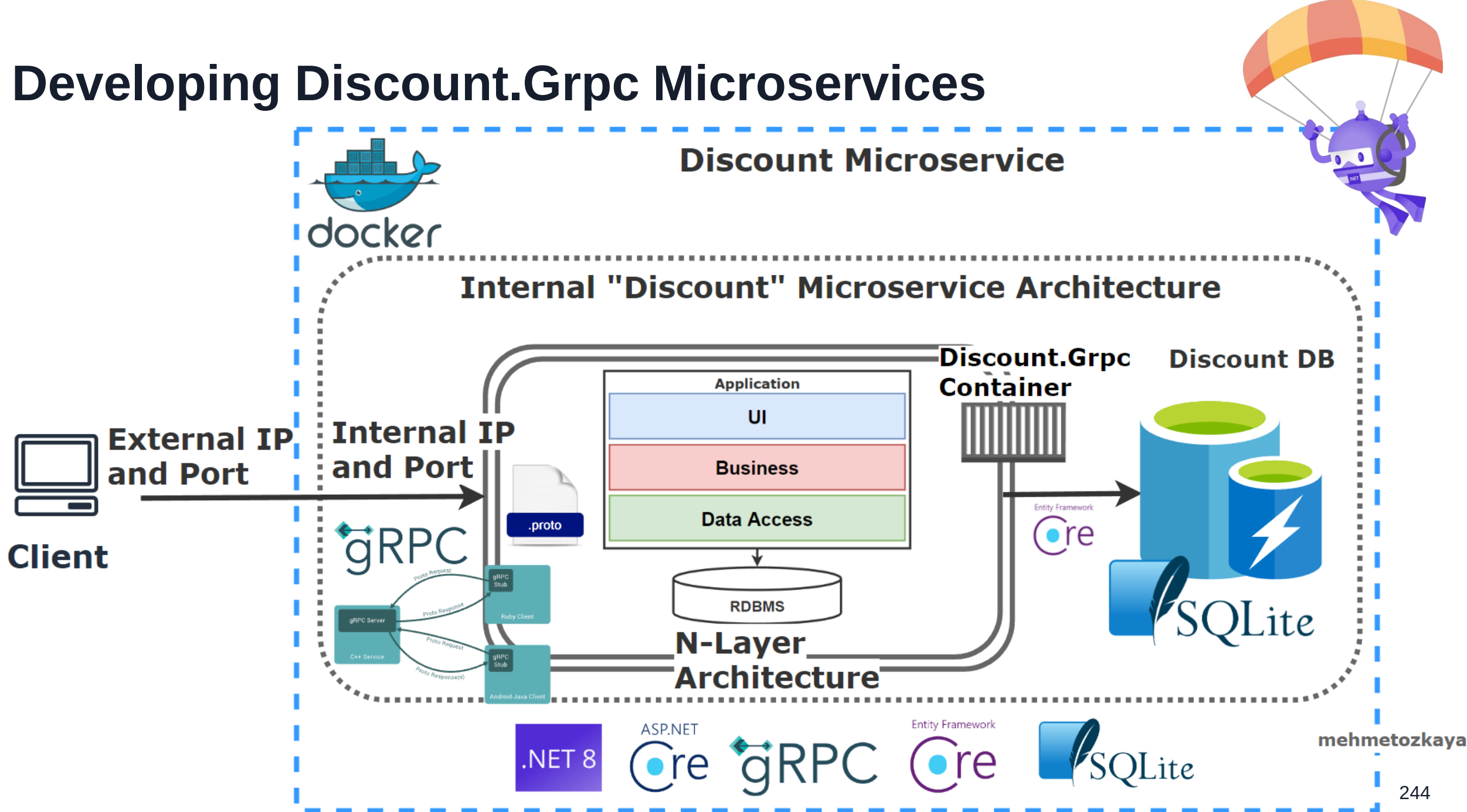
Mehmet Ozkaya



Big Picture – Course Project

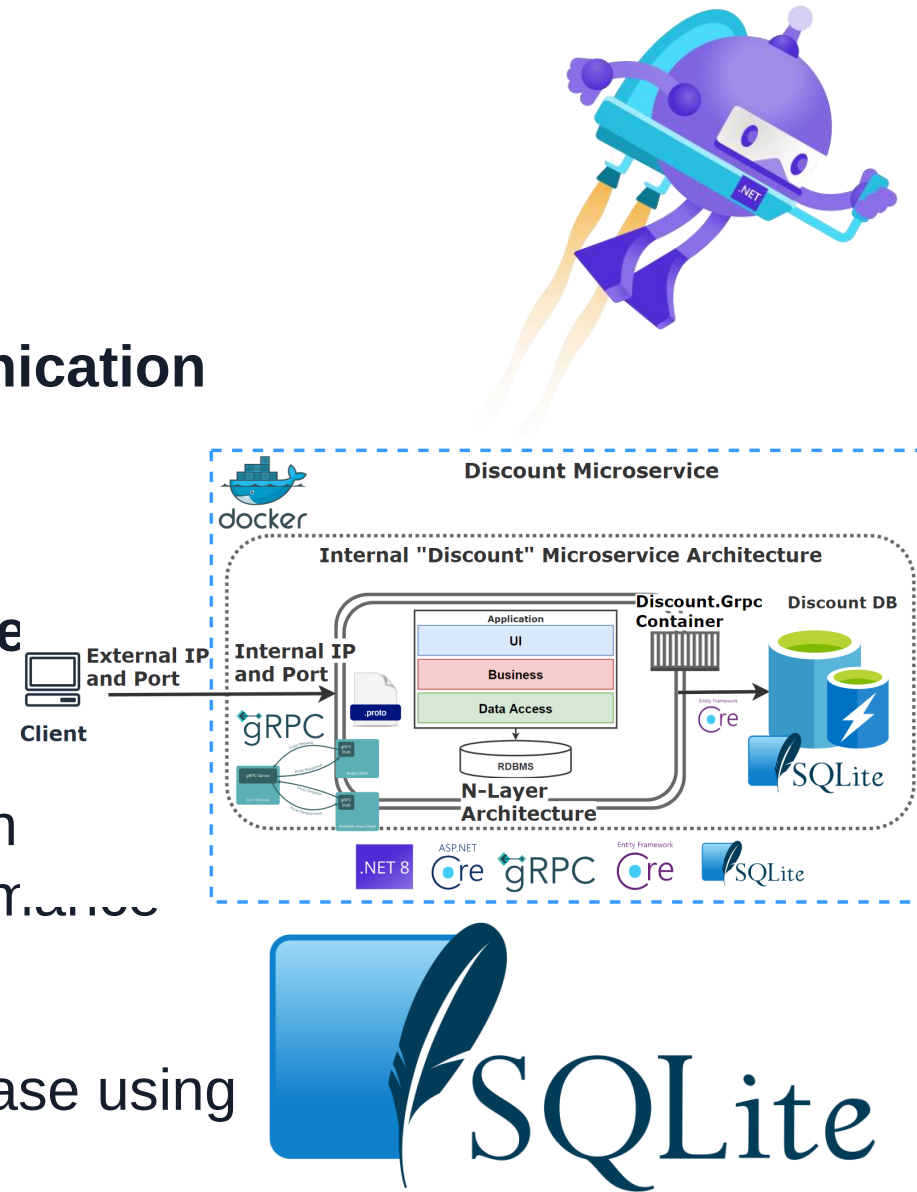


Developing Discount.Grpc Microservices

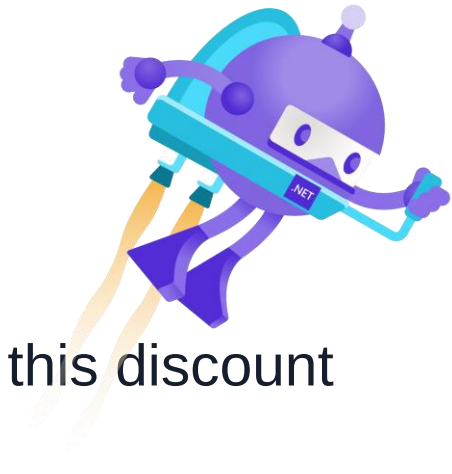


Discount.Grpc Microservices

- ASP.NET gRPC API application
- Build a **Highly Performant inter-service gRPC Communication** with **Discount** and **Basket** Microservice
- **gRPC** Communications, **Proto files** CRUD operations
- Exposing **Grpc Services** with creating **Protobuf message**
- **SQLite database** connection and containerization
- **Entity Framework Core ORM - SQLite Data Provider** and Migrations to simplify data access and ensure high performance
- **N-Layer Architecture** implementation
- **Containerize Discount** Microservices with **SQLite** database using Docker Compose



Main Idea of Discount.Grpc Microservices



Use Case: Add Item into SC

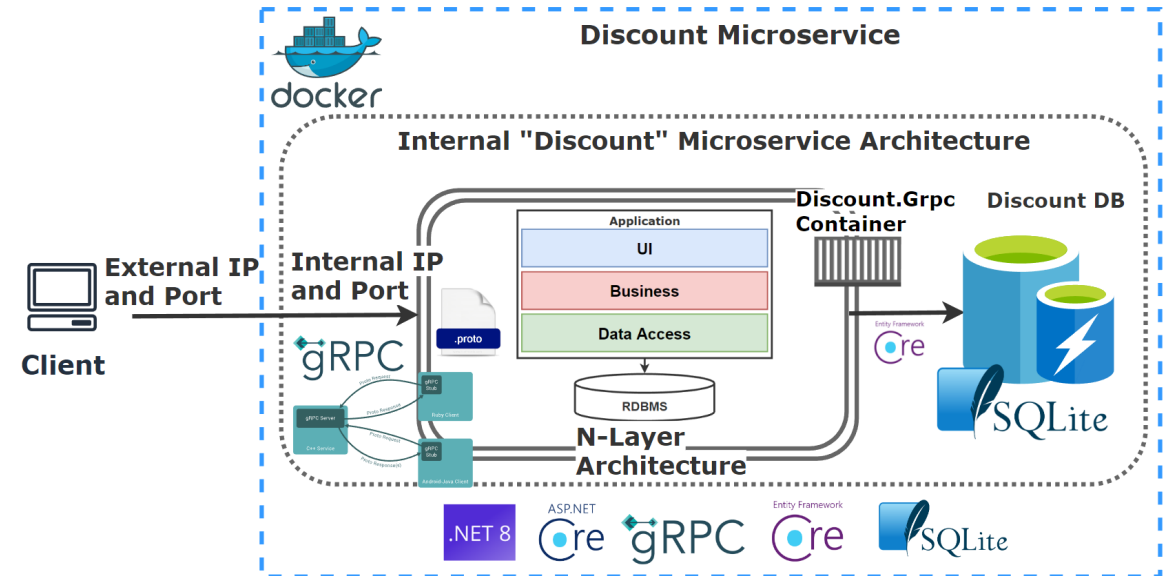
- When client add item into shopping cart, Basket microservice will consume this discount grpc services, to get latest discounts on that product item.

Main Idea: Be performant as much as possible

- Achieve maximum performance and reduce service invocation time.
- Ensuring the Discount microservice responds quickly to synchronous calls from the Basket service.

Architecture

- gRPC communication, SQLite database,
- N-Layer architecture, Ef Core SQLite provider

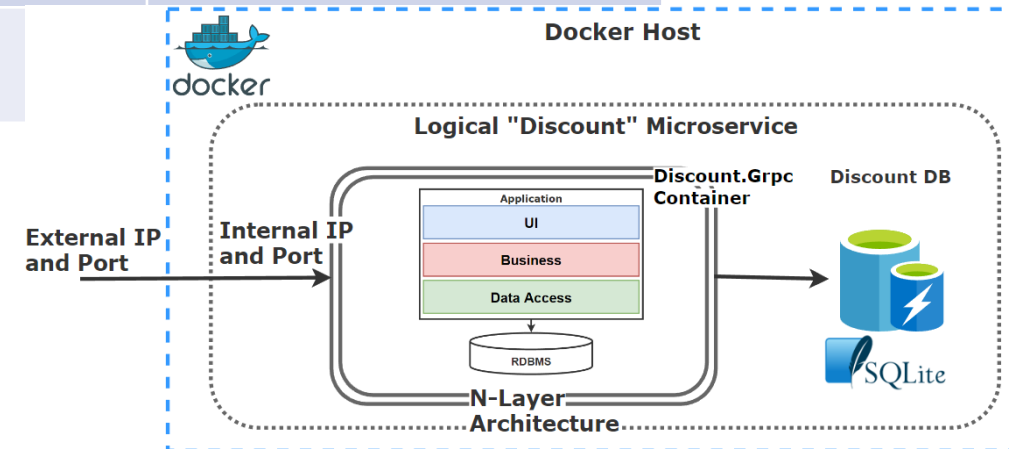


Port Numbers for Discount.Grpc Microservice



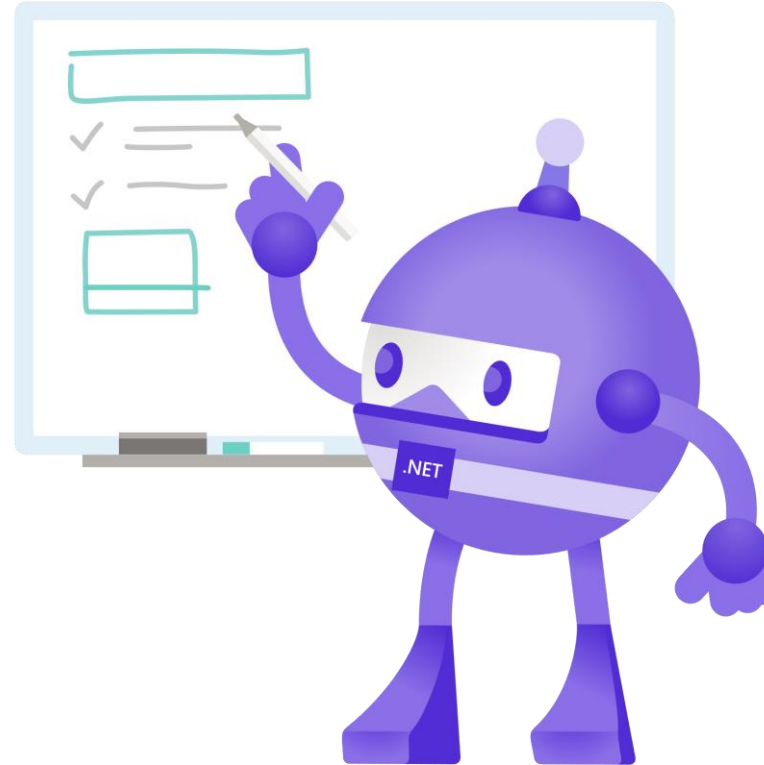
Microservices	Local Env	Docker Env	Docker Inside
Catalog	5000 – 5050	6000 – 6060	8080 – 8081
Basket	5001 – 5051	6001 – 6061	8080 – 8081
Discount	5002 – 5052	6002 – 6062	8080 – 8081
Ordering	5003 – 5053	6003 – 6063	

- ASP.NET Core **Ports** are listed as **http / https** for running application.



Domain Analysis of Discount.Grpc Microservices

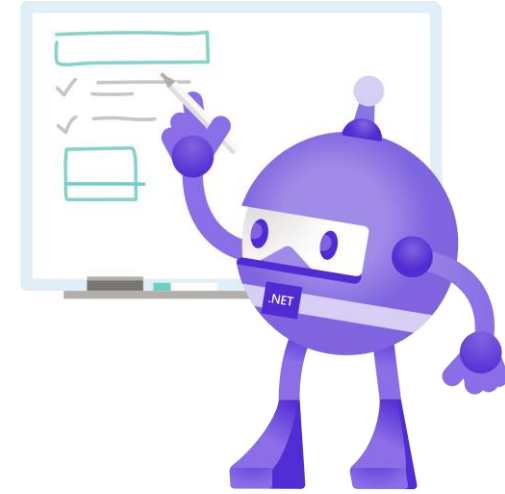
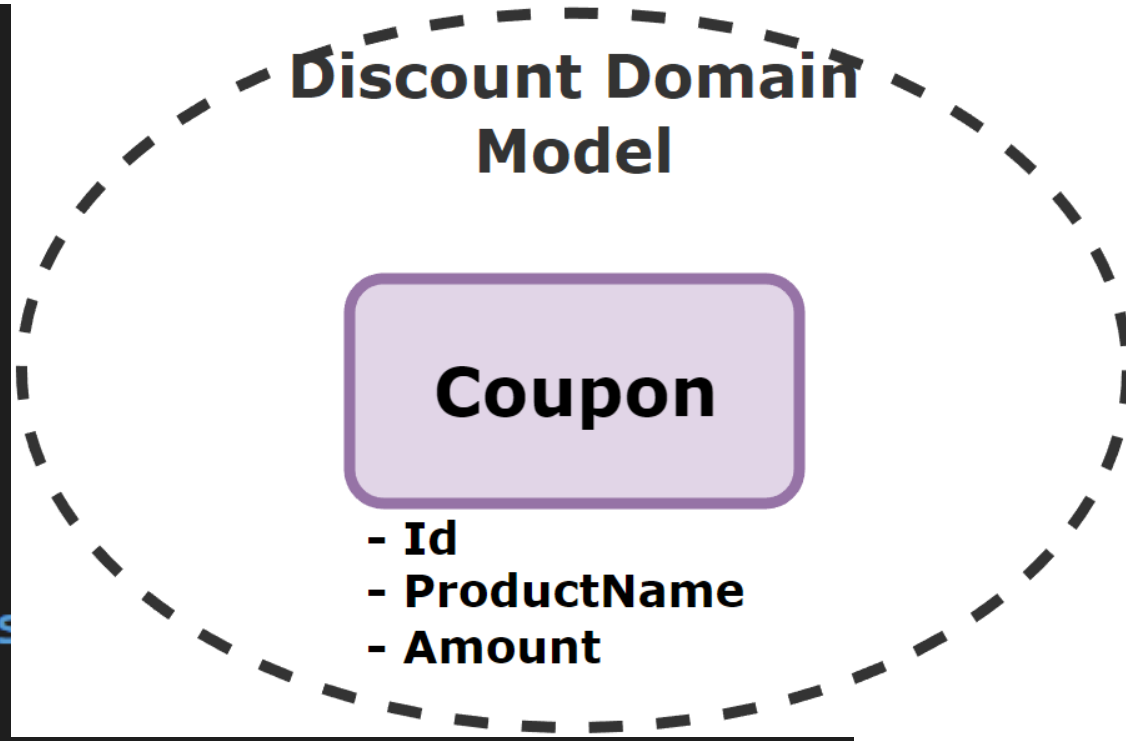
1. Domain Models
2. Application Use Cases
3. Rest API Endpoints
4. Underlying Data Structures



1. Domain Models of Discount Microservices

- Primary domain model is 'Coupon'

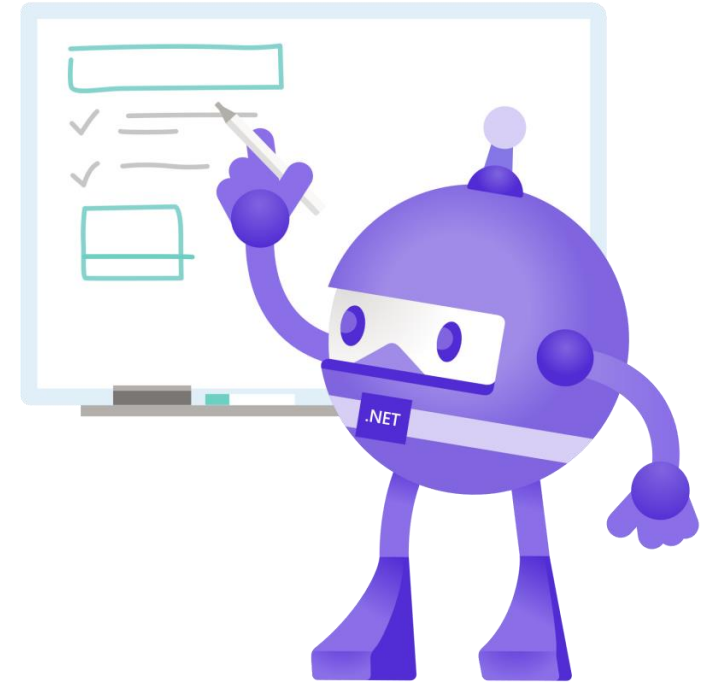
```
public class Coupon
{
    2 references
    public int Id { get; set; }
    8 references
    public string ProductName
    3 references
    public string Description
    4 references
    public int Amount { get; set; }
}
```



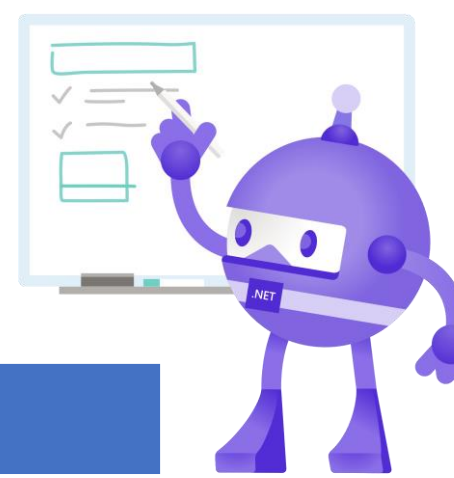
2. Application Use Cases of Discount Microservices

CRUD Discount Operations:

- Get Discount with ProductName
- Create Discount
- Update Discount
- Delete Discount



3. Grpc Endpoints of Discount Microservices



Method (gRPC)	Request URI	Use Cases
GetDiscount	GetDiscountRequest	Get discount w/ productname
CreateDiscount	CreateDiscountRequest	Create discount
UpdateDiscount	UpdateDiscountRequest	Update discount
DeleteDiscount	DeleteDiscountRequest	Delete discount

4. Underlying Data Structures of Discount Microservices

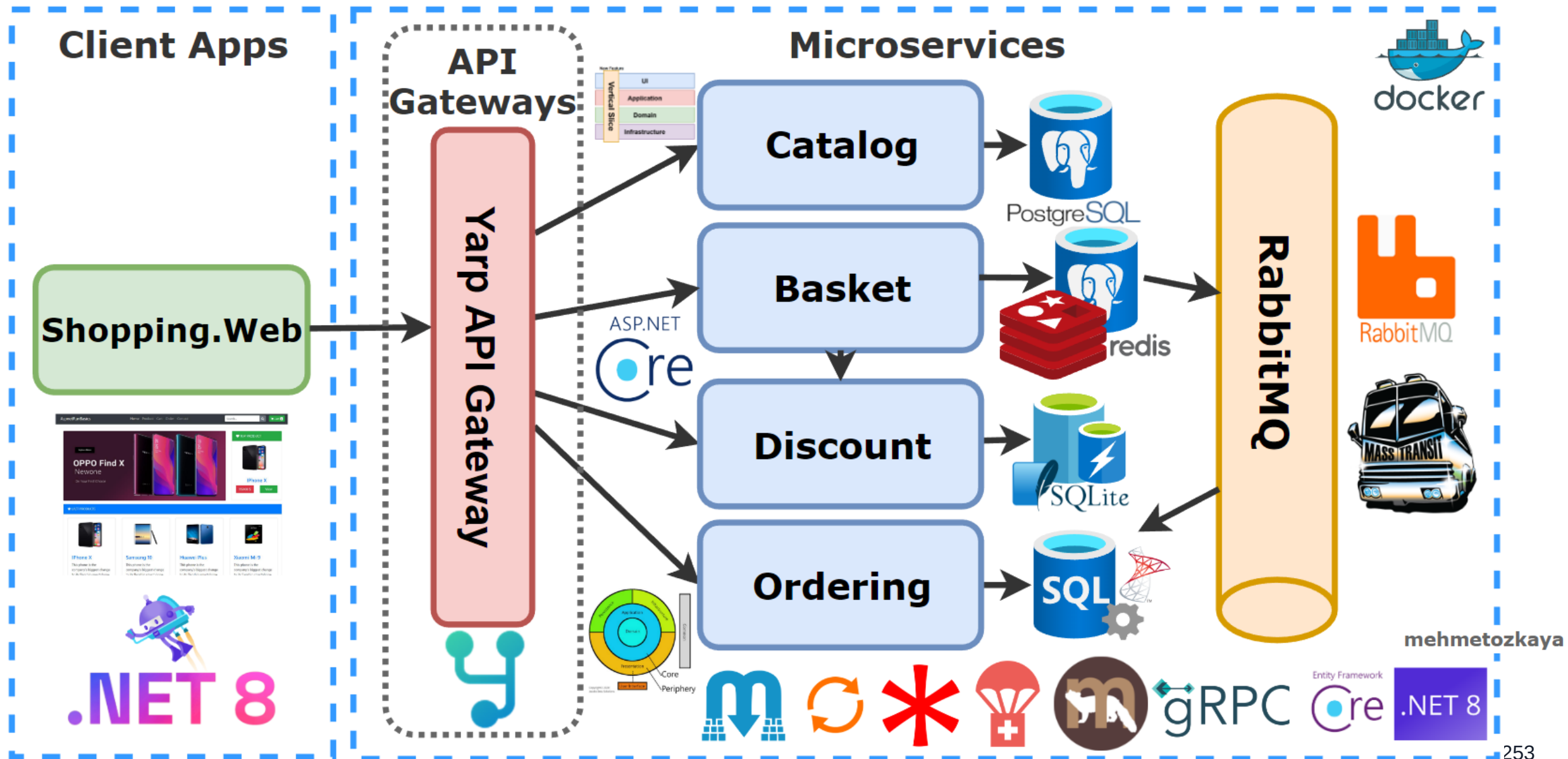
Basket Microservices has 1 Datastore:

1. **SQLite Database:** Store Coupon table in lite database

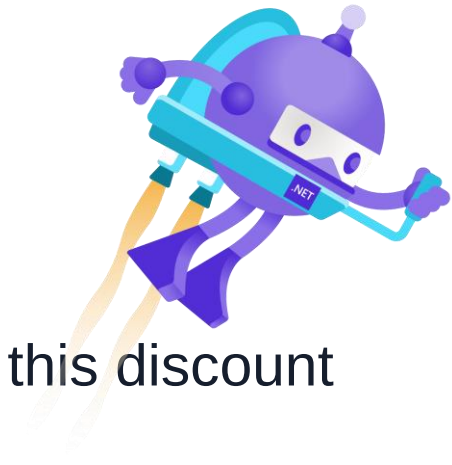


- **SQLite** is a C-language library that implements a **small, fast, self-contained, high-reliability, full-featured, SQL** database engine.
- Chosen for its **simplicity** and **efficiency**, especially for **small-scale data** like discounts. It's **embedded within** the **application**, reducing the need for additional infrastructure.
- **Advantages of SQLite:**
 - Embedded within the application, providing high performance.
 - Suitable for lightweight and straightforward operations.
 - Simplifies deployment without needing a separate database server.

Big Picture – Course Project



Main Idea of Discount.Grpc Microservices



Use Case: Add Item into SC

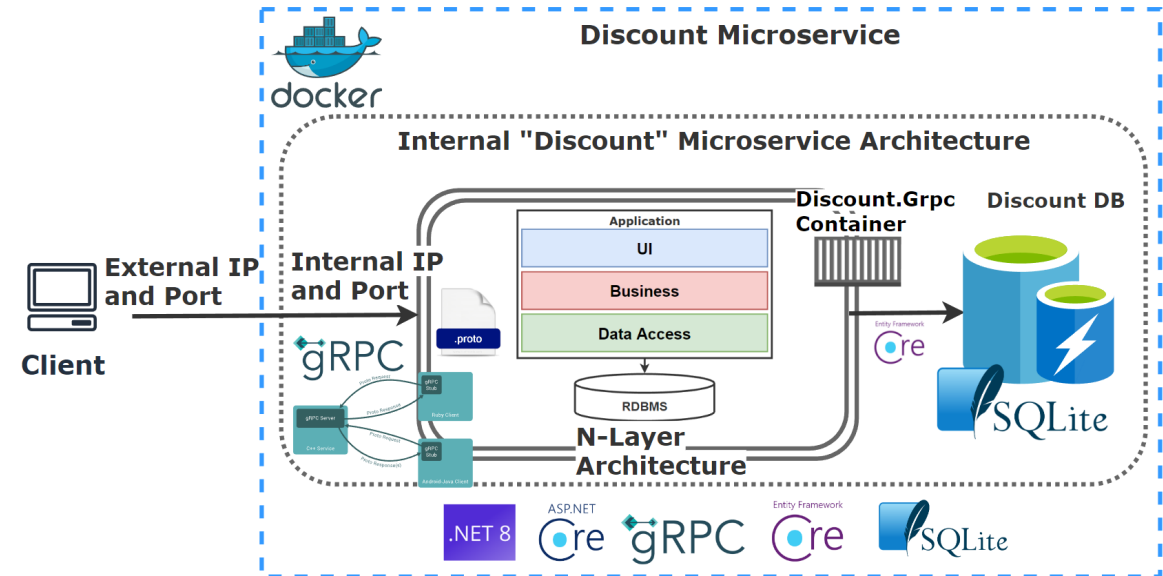
- When client add item into shopping cart, Basket microservice will consume this discount grpc services, to get latest discounts on that product item.

Main Idea: Be performant as much as possible

- Achieve maximum performance and reduce service invocation time.
- Ensuring the Discount microservice responds quickly to synchronous calls from the Basket service.

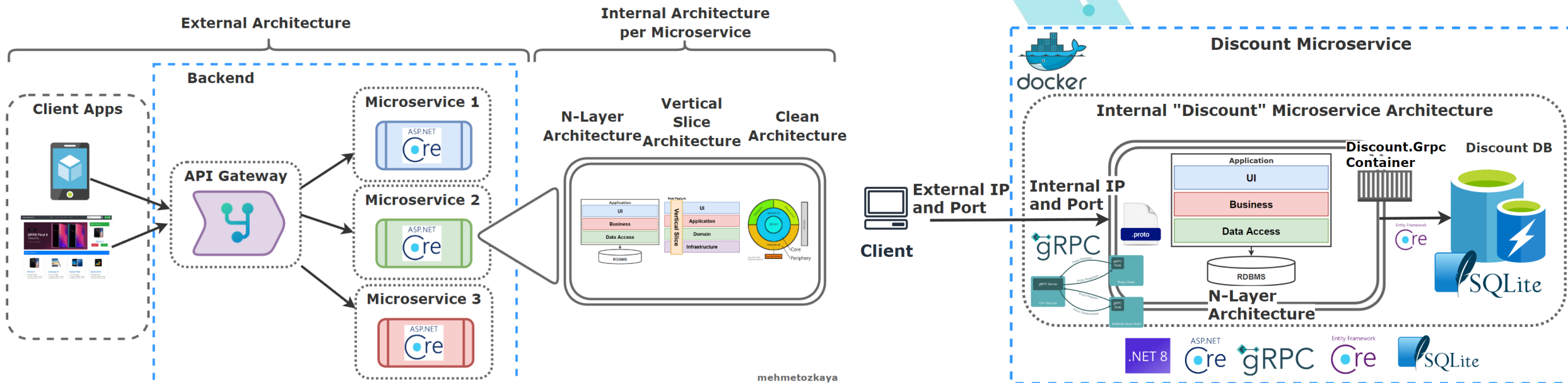
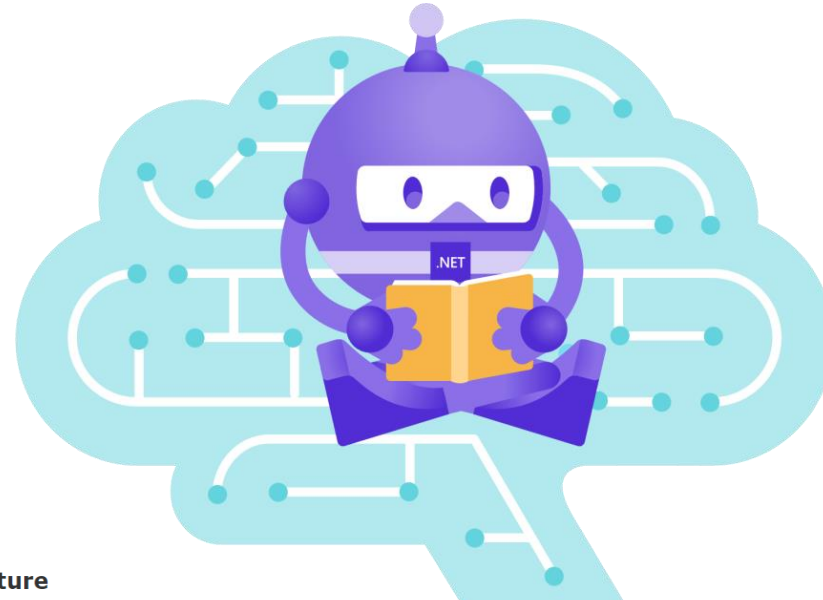
Architecture

- gRPC communication, SQLite database,
- N-Layer architecture, Ef Core SQLite provider



Technical Analysis of Discount Microservices

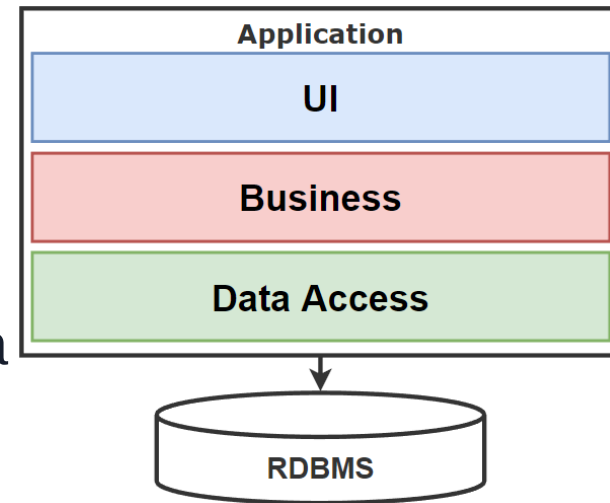
1. Application Architecture Style
2. Patterns & Principles
3. Libraries Nuget Packages
4. Project Folder Structure



1. Application Architecture Style of Discount Microservices

Traditional N-Layer Architecture

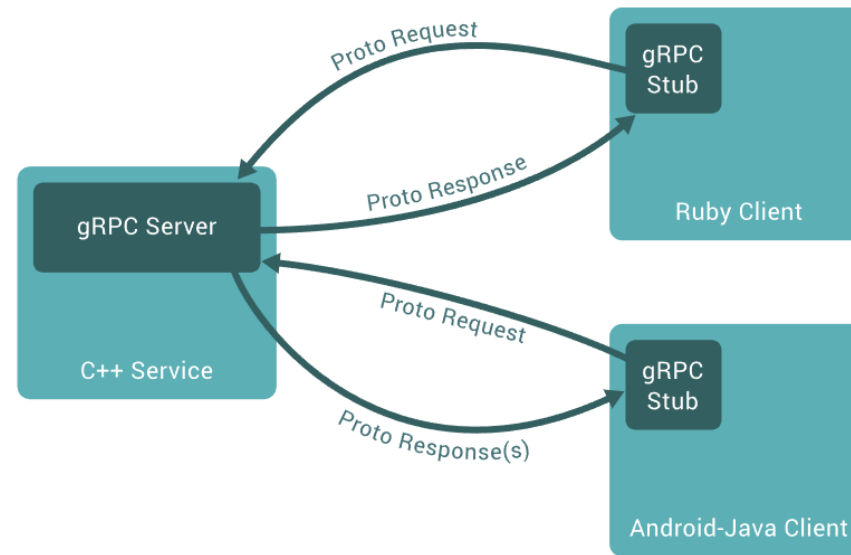
- **Data Access Layer:** Only database operations are performed on this layer. The task of this layer is to add, delete, update and extract data from the database.
- **Business Layer:** Implement business logics . Process the data taken by Data Access into the project. We do not use the Data Access layer directly in our applications.
- **Presentation Layer:** User interacts layer. Show the data to the user and to transmit the data from the user to the Business Layer and Data Access.



2. Patterns & Principles of Discount Microservices



- **gRPC ProtoBuf files Endpoints:** For service communication.
- **Entity Framework Core:** ORM for database operations.
- **SQLite Database:** Embedded SQL database for efficient data storage.



3. Libraries Nuget Packages of Discount Microservi



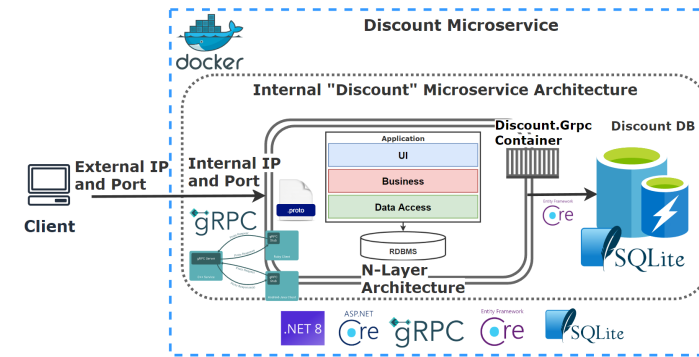
- **Generic Packages:** Mapster, FluentValidation.
- **gRPC:** Aspnetcore.Grpc.
- **Database:** Microsoft.EntityFrameworkCore.Sqlite, Microsoft.EntityFrameworkCore.Tools.



4. Project Folder Structure of Basket Microservices

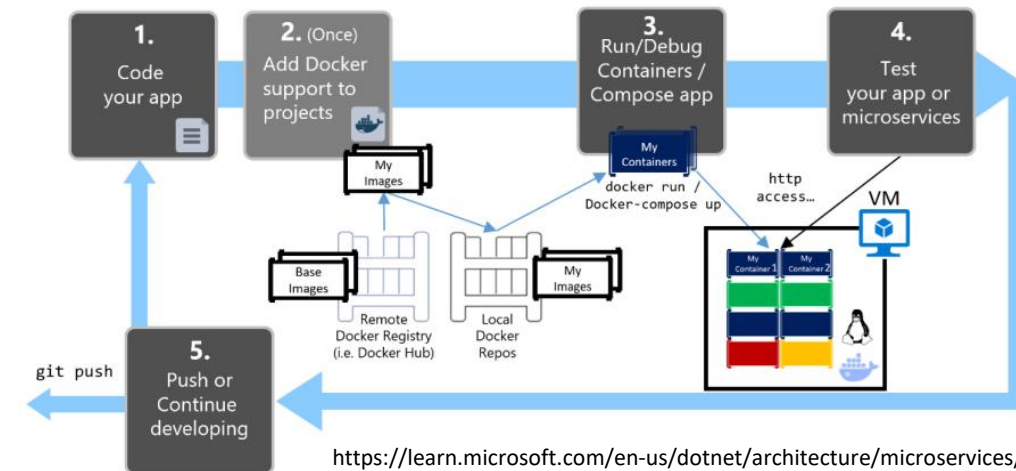
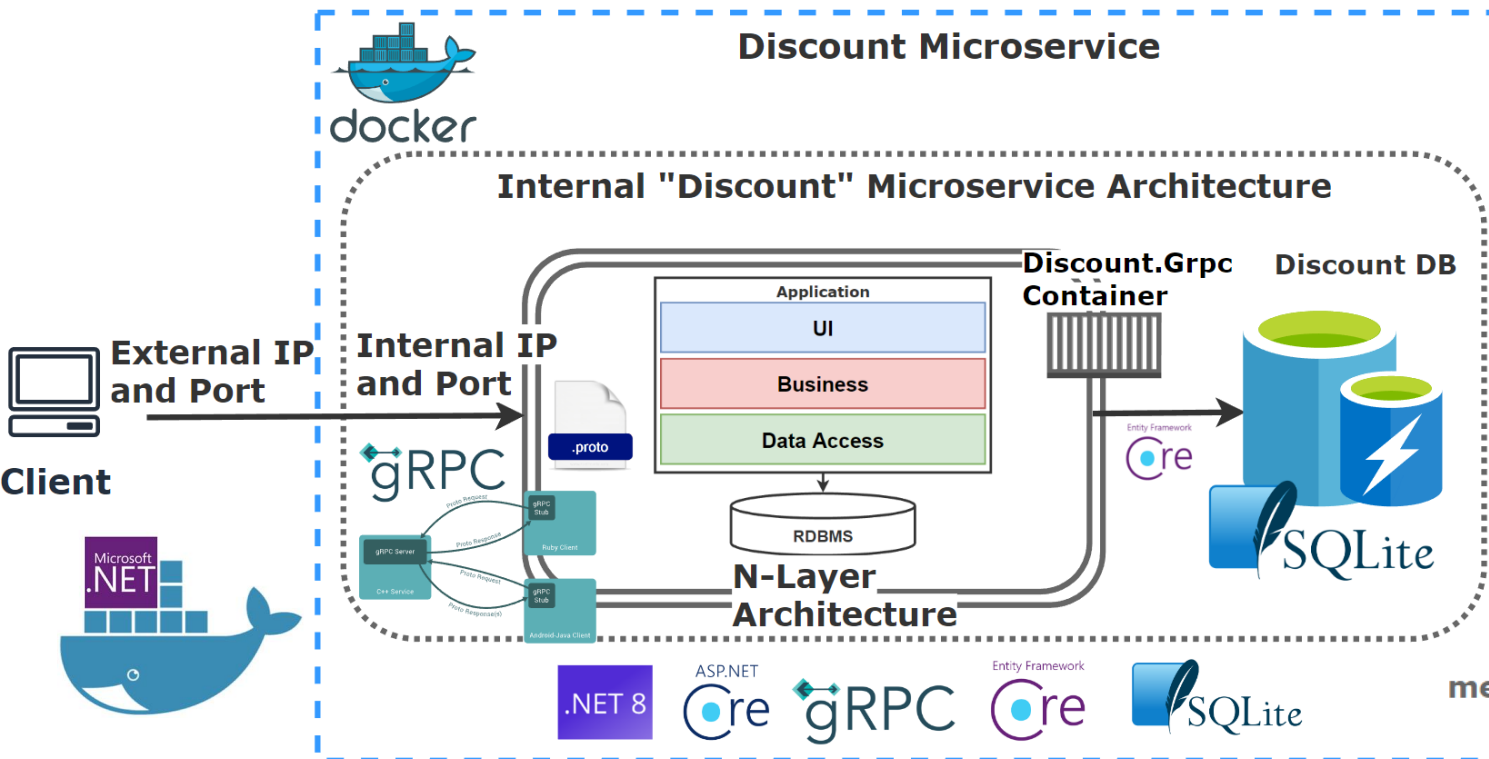
- **Models:** SQLite entity
 - **Data:** EF Core data context and Migration Extensions
 - **Service:** gRPC Services
 - **Protos:** gRPC exposing Apis
-
- Map to N-Layer Architecture
 - Models -- Domain Layer
 - Data -- Data Access Layer
 - Services -- Business Layer
 - Protos -- Presentation Layer

```
Discount
├── Discount.Grpc
│   ├── Connected Services
│   ├── Dependencies
│   ├── Properties
│   ├── Data
│   │   ├── DiscountContext.cs
│   │   └── Extensions.cs
│   ├── Migrations
│   ├── Models
│   │   └── Coupon.cs
│   ├── Protos
│   │   └── discount.proto
│   ├── Services
│   │   └── DiscountService.cs
│   ├── appsettings.json
│   ├── discountdb
│   ├── discountdb-shm
│   ├── discountdb-wal
│   ├── Dockerfile
│   └── Program.cs
```



Deployment of Discount Microservices

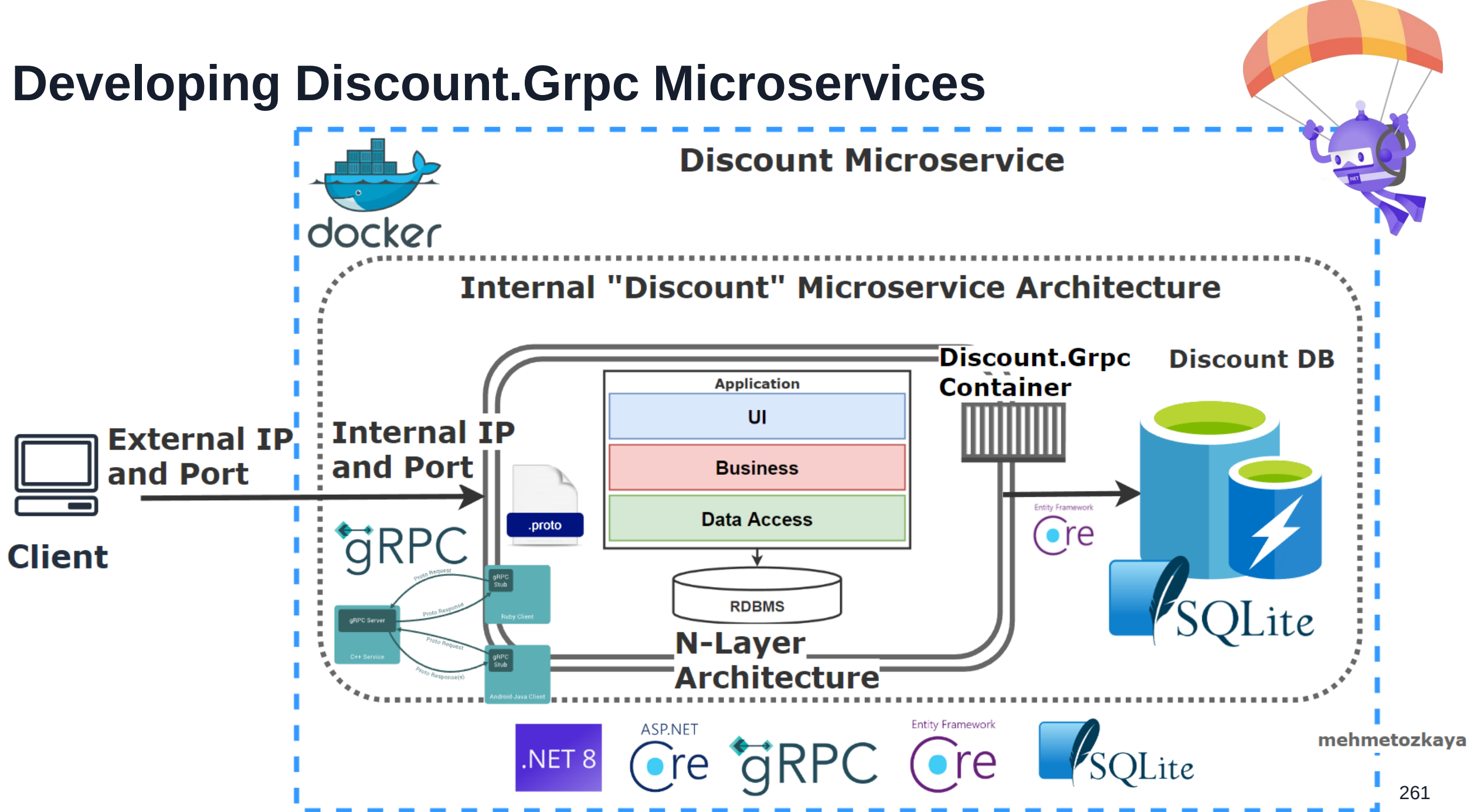
- **Containerize** Discount service with **Docker**, ensuring deployment and integration with SQLite
- Implement **Dockerfile** and **docker-compose** file for running **Discount** microservice and **SQLite** database in Docker environment.



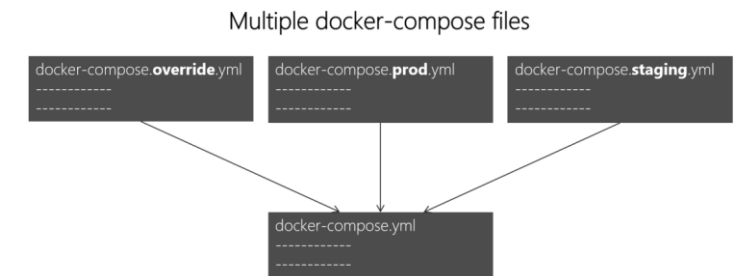
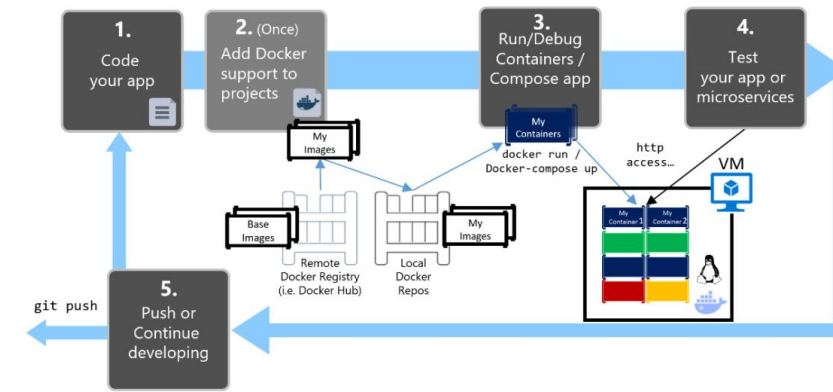
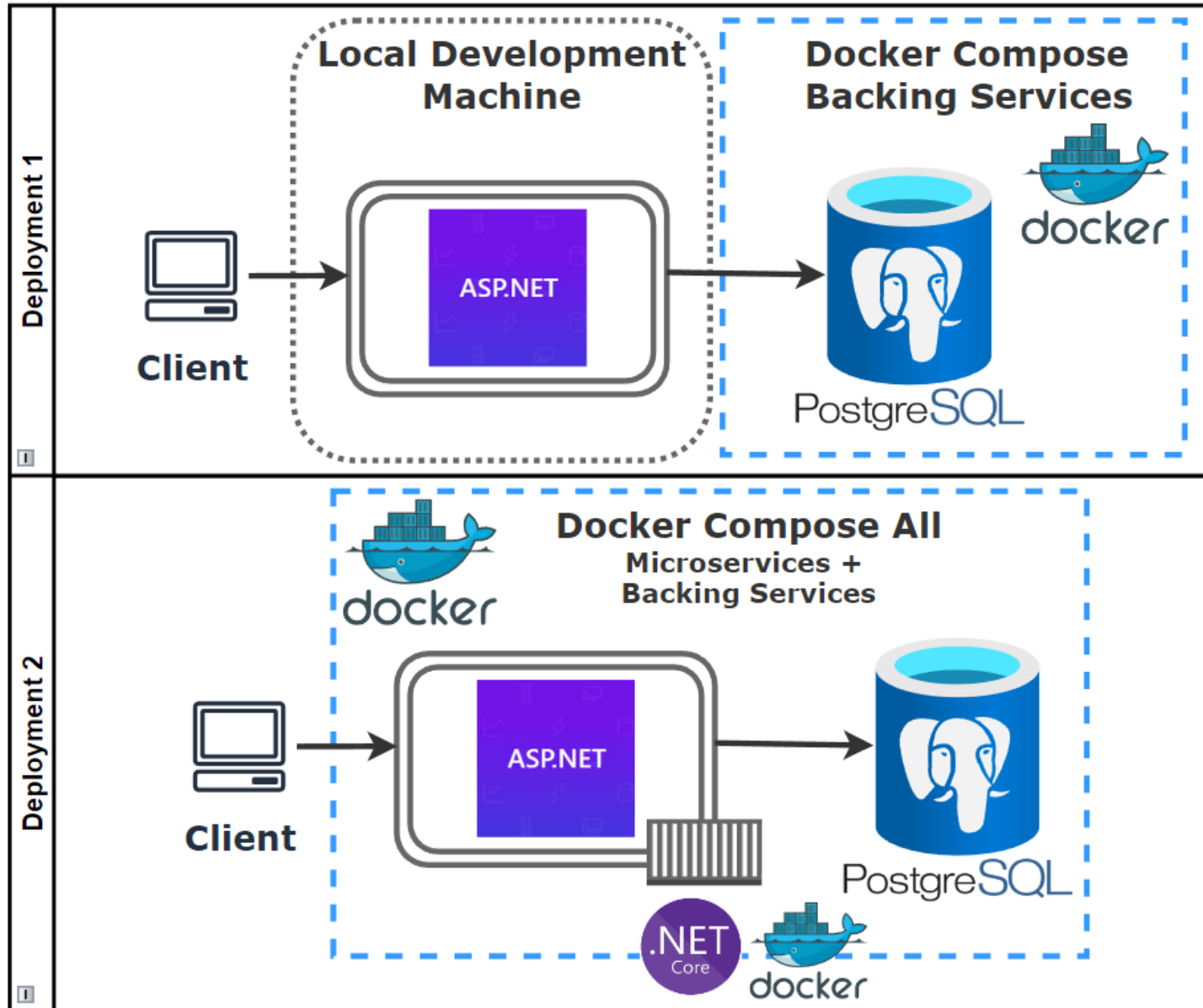
<https://learn.microsoft.com/en-us/dotnet/architecture/microservices/>

mehmetozkaya

Developing Discount.Grpc Microservices



EShop Microservices Deployment Strategy

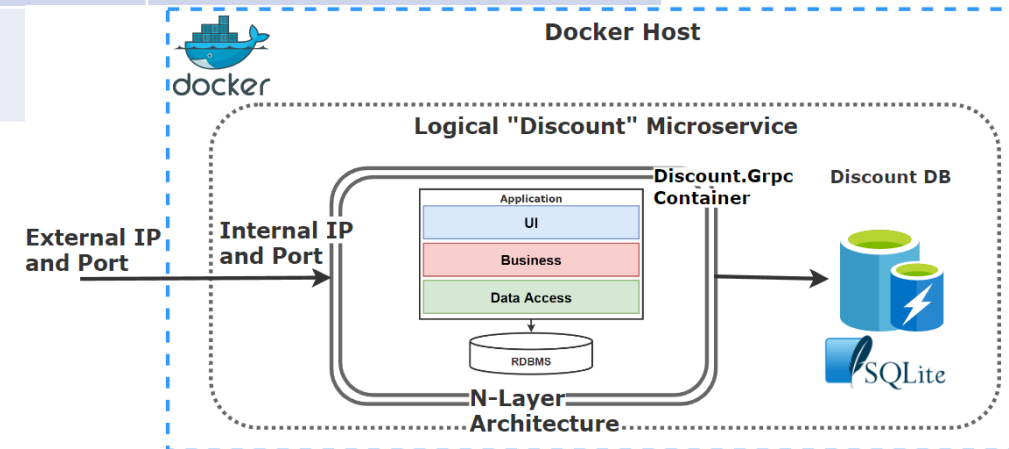


Port Numbers for Discount.Grpc Microservice



Microservices	Local Env	Docker Env	Docker Inside
Catalog	5000 – 5050	6000 – 6060	8080 – 8081
Basket	5001 – 5051	6001 – 6061	8080 – 8081
Discount	5002 – 5052	6002 – 6062	8080 – 8081
Ordering	5003 – 5053	6003 – 6063	

- ASP.NET Core **Ports** are listed as **http / https** for running application.



EF Core commands for VS and .Net CLI



Visual Studio

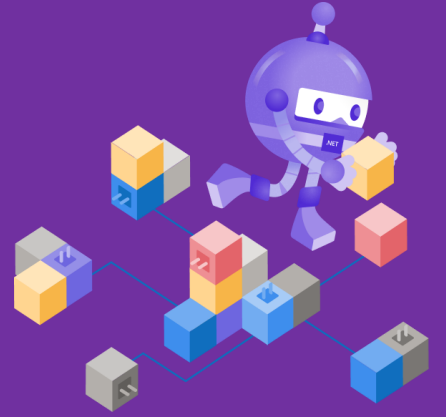
- Install-Package Microsoft.EntityFrameworkCore.Tools
- Add-Migration InitialCreate
- Update-Database



.Net CLI and Visual Studio Code

- dotnet tool install --global dotnet-ef
- dotnet add package Microsoft.EntityFrameworkCore.Design
- dotnet ef migrations add InitialCreate
- dotnet ef database update

Consuming Discount Grpc Service From Basket Microservice



Consume Discount Grpc Service From Basket Microservice

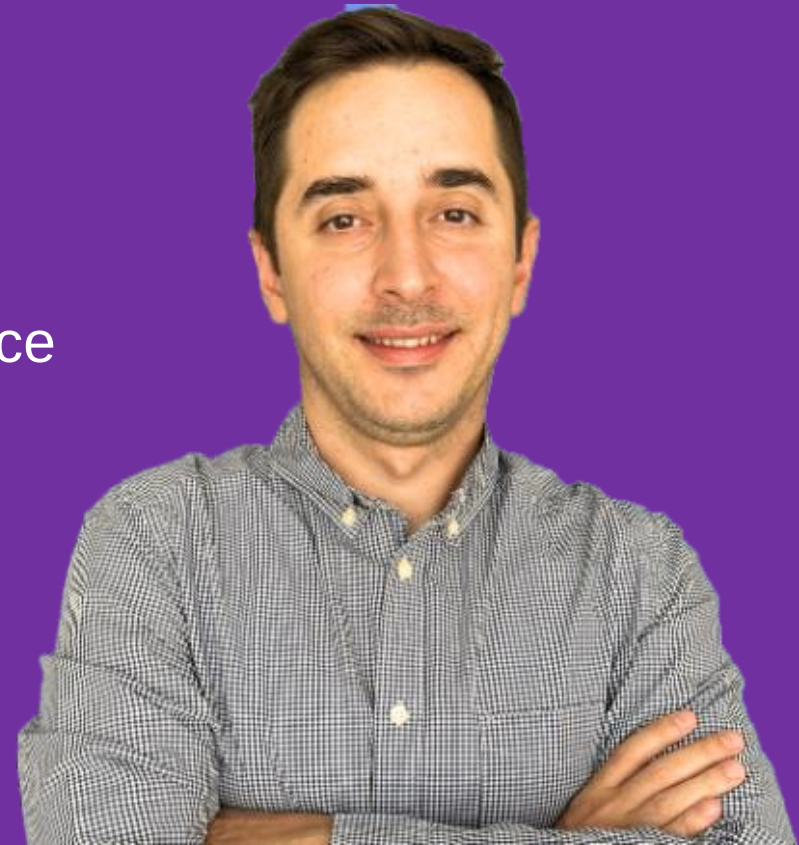
When Adding Cart Item into Shopping Cart To Calculate Final Price

Integration of gRPC Client and Service Registration

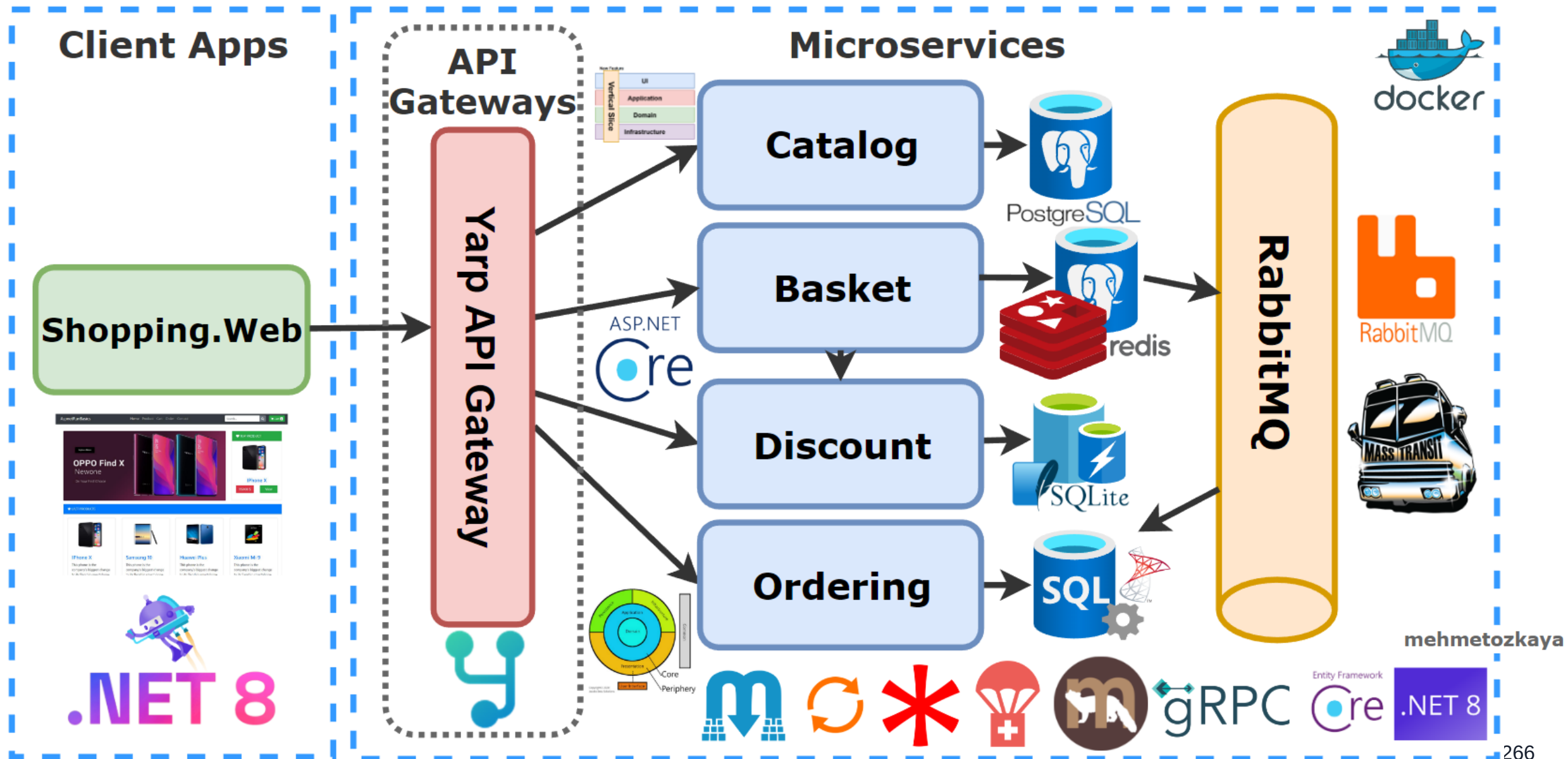
Consume Discount Service

Re-Containerization

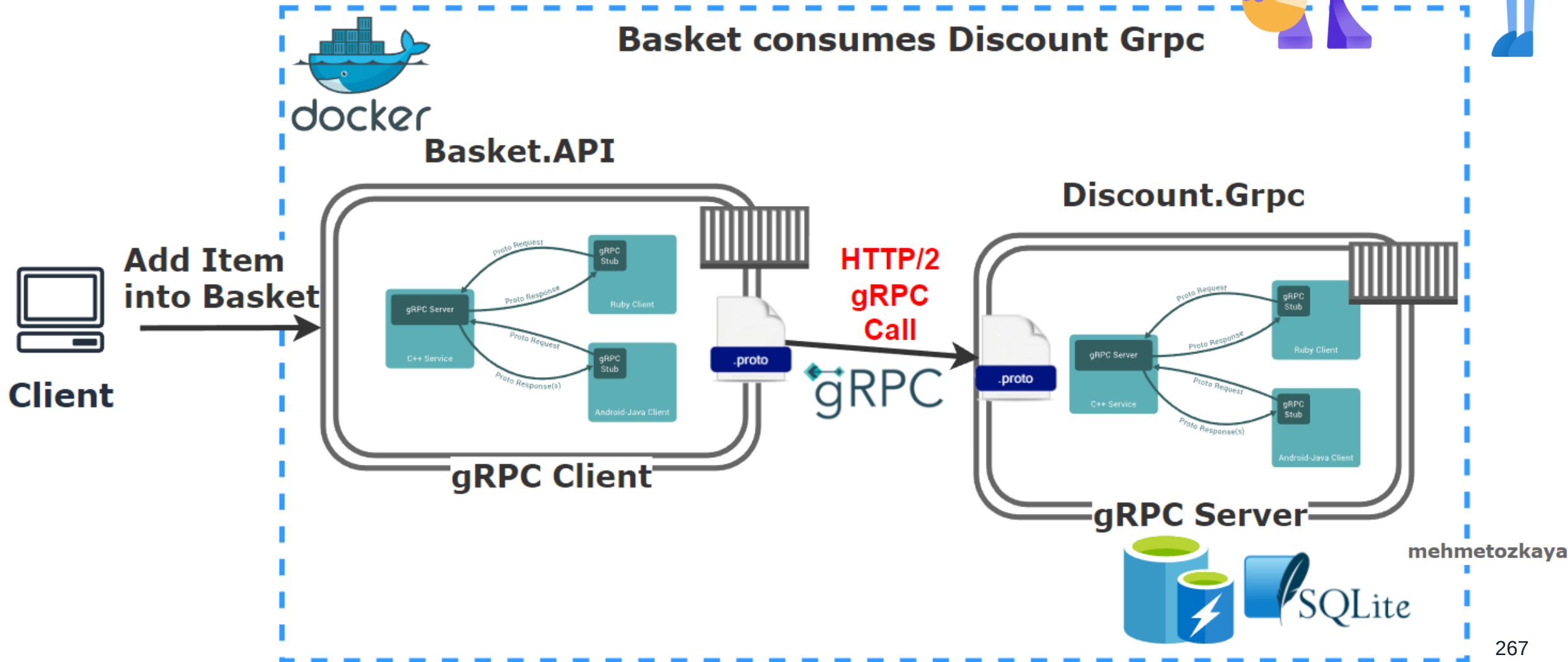
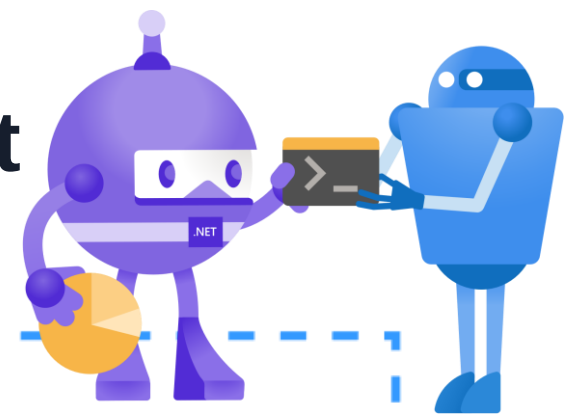
Mehmet Ozkaya



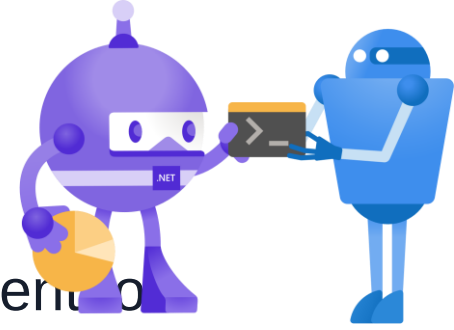
Big Picture – Course Project



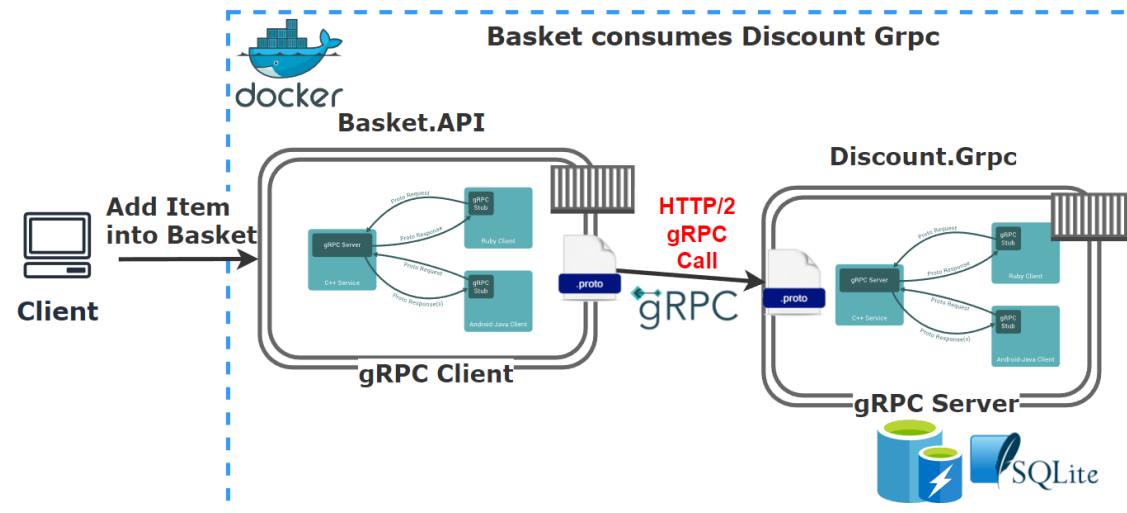
Consuming Discount Grpc Service From Basket



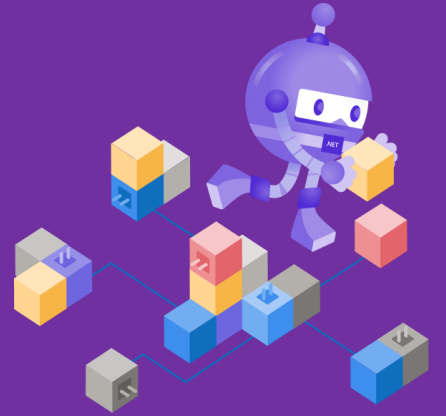
Consuming Discount Grpc Service From Basket



- **Integration of gRPC Client:** The Basket Microservice will act as a gRPC client to consume services exposed by the Discount Grpc Service.
- **Service Registration:** Modify the Program.cs in the Basket Microservice to include the Discount Grpc client.
- **Consume Discount Service:** Update the logic to apply these discounts to the cart items and calculate the final price accordingly.
- **Re-Containerization:** Update the Docker Compose setup to ensure the Basket Microservice can communicate with the Discount Grpc Service within the orchestrated environment.



Ordering Microservices with DDD, CQRS and Clean Architecture



Apply DDD, CQRS and Clean Architecture Patterns and Best Practices

Implement Design Principles: SOLID, KISS, YAGNI, SoC, DIP

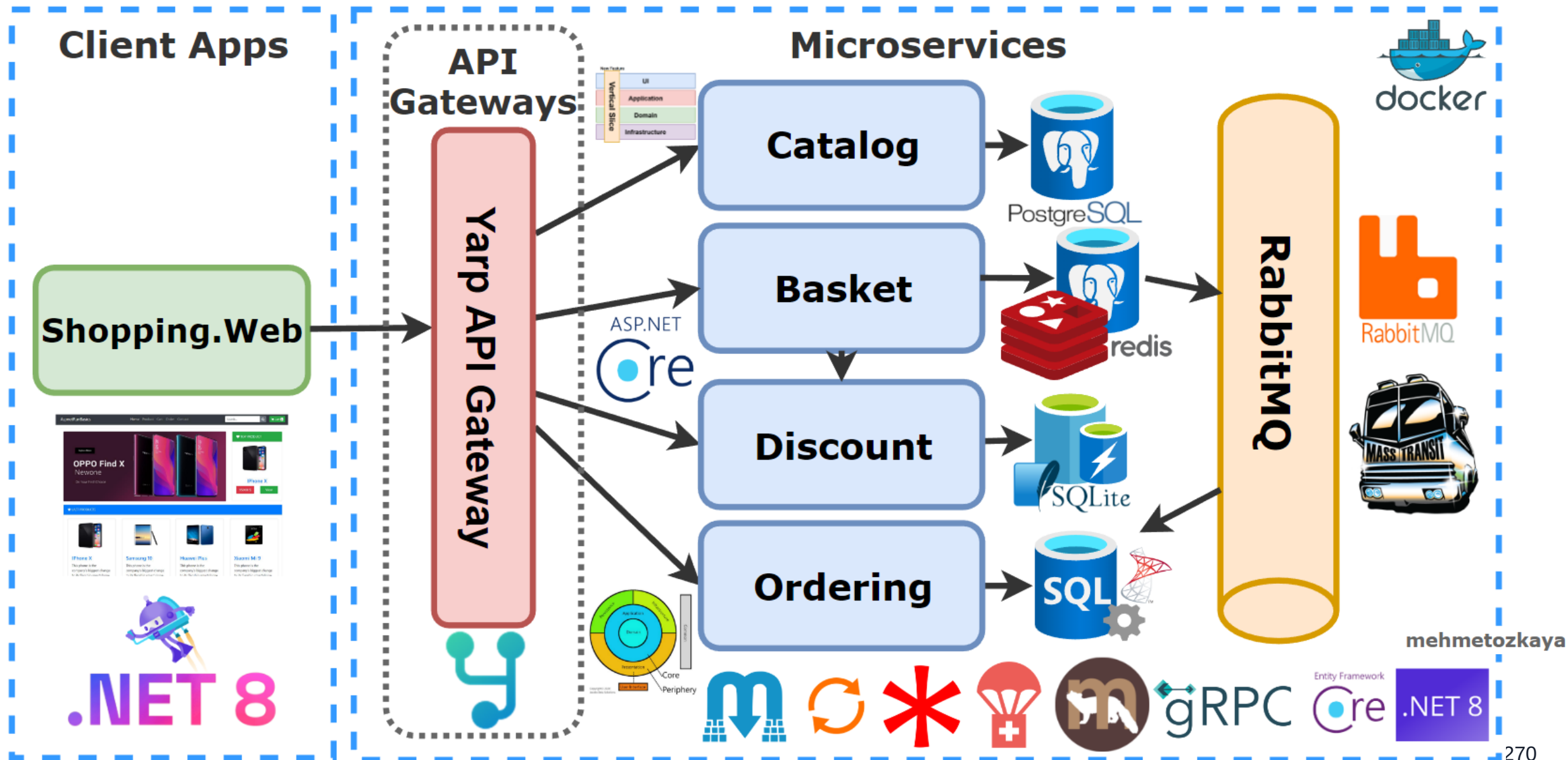
Develop Clean Architecture Layers: Domain, Application, Infrastructure and API

CQRS with MediatR library to implement the Command and Command
Handlers

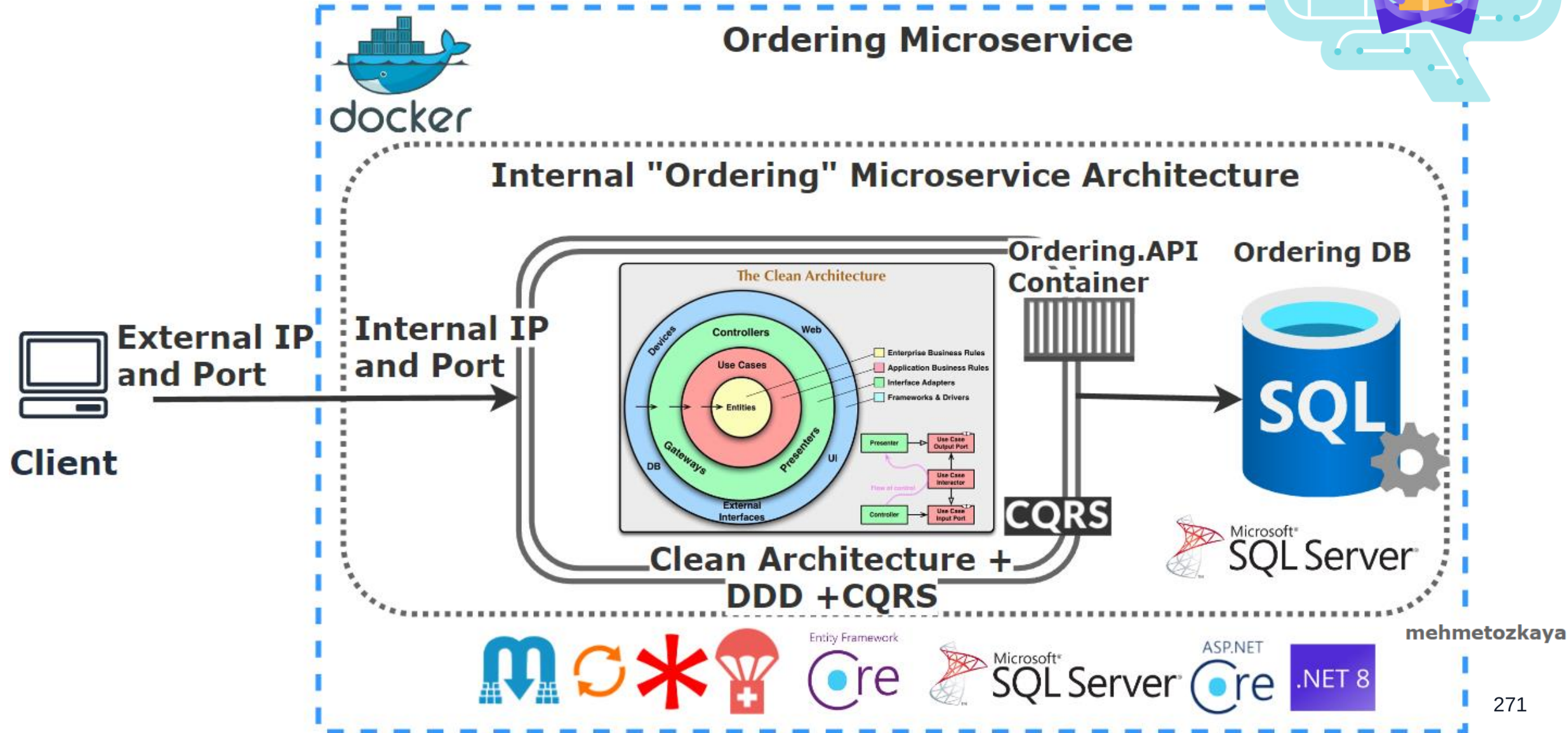
Mehmet Ozkaya



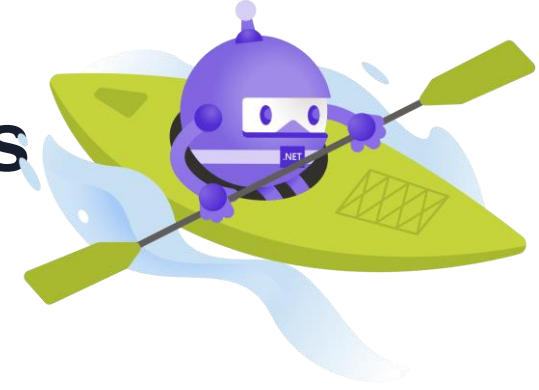
Big Picture – Course Project



Ordering Microservice w/ DDD, CQRS and Clean Architecture



Ordering Microservice Clean Architecture Layers



■ Domain Layer

- Tactical DDD implementations, including entities, value objects, and aggregates.
- Anemic-domain model vs Rich-domain model Entities in DDD
- Domain Events & Integration Events

■ Infrastructure Layer

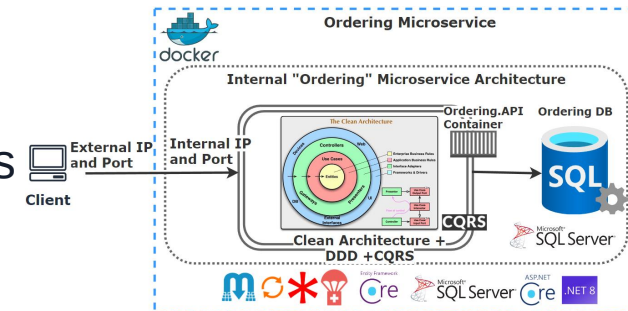
- Entity Framework Core 8 Code First Approach, Migrations and SQL Server Connection
- EF Core 8 Relations and DDD ValueObject Mapping with ComplexType and ComplexProperty
- Auto Migrate and Seed EF Core Entities to SQL Server DB when Application Startup
- EF Core Interceptors: SaveChangesInterceptor

■ Application Layer

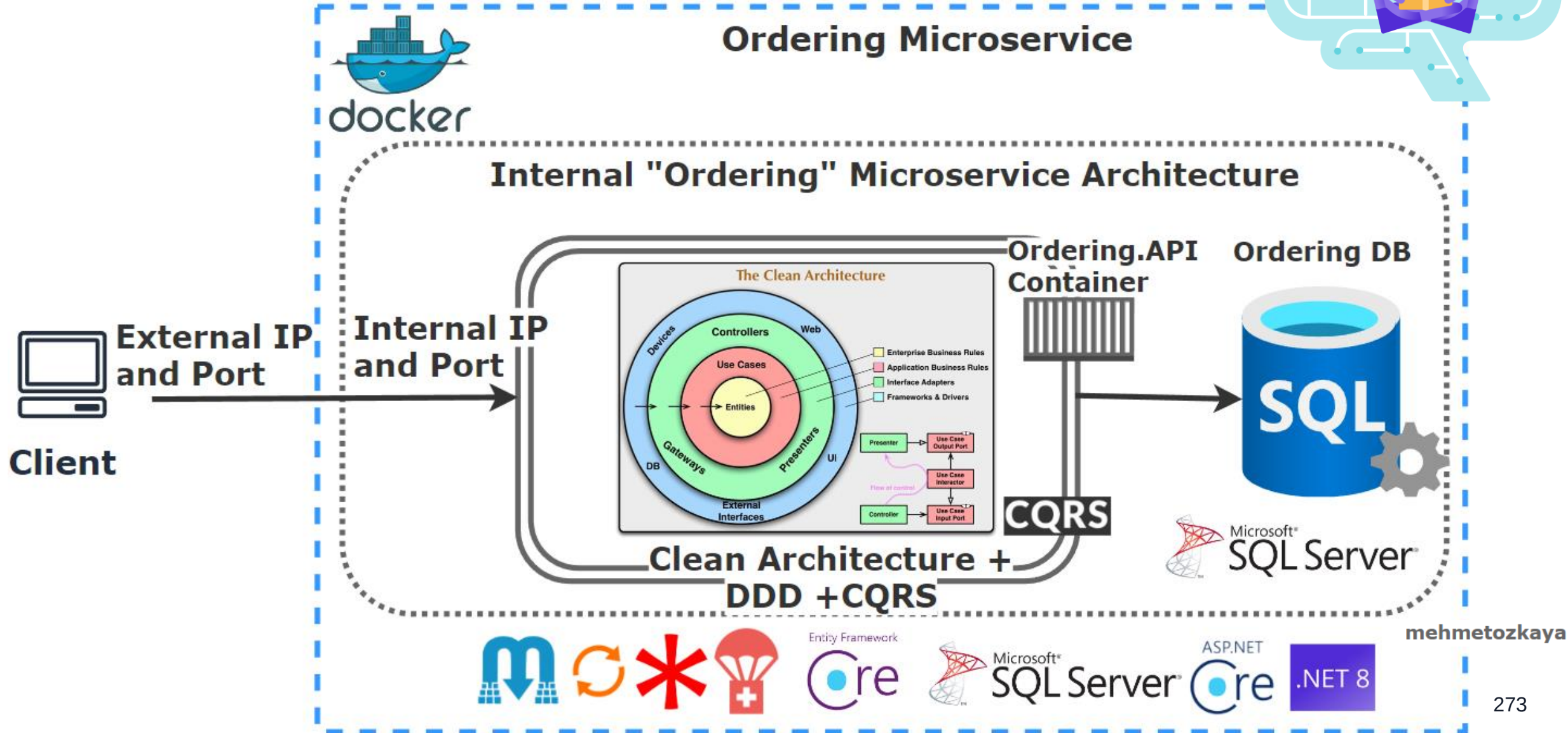
- CQRS with MediatR library to implement the Command and Command Handlers
- MediatR Pipeline Behaviours (Validation, Logging)
- Domain Event Handlers with MediatR INotificationHandler

■ API Layer

- Using Carter for Minimal API endpoint definition
- Cross-Cutting Concerns Custom Exception Handler in Ordering Microservice



Ordering Microservice w/ DDD, CQRS and Clean Architecture

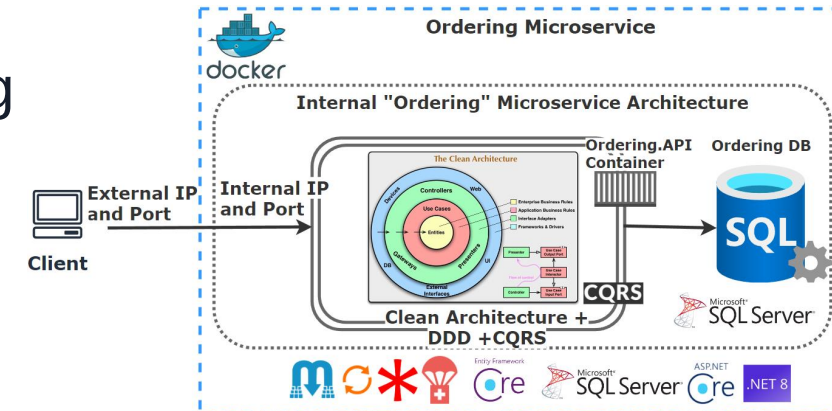


Port Numbers for Ordering.API Microservice



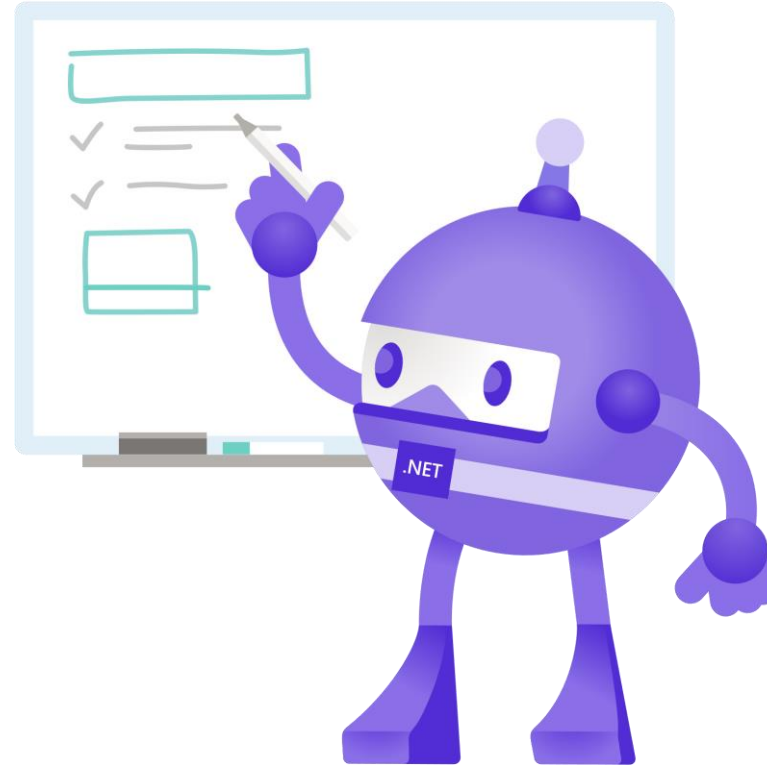
Microservices	Local Env	Docker Env	Docker Inside
Catalog	5000 – 5050	6000 – 6060	8080 – 8081
Basket	5001 – 5051	6001 – 6061	8080 – 8081
Discount	5002 – 5052	6002 – 6062	8080 – 8081
Ordering	5003 – 5053	6003 – 6063	8080 – 8081

- ASP.NET Core **Ports** are **listed** as **http / https** for running application.



Domain Analysis of Ordering Microservices

1. Domain Models
2. Application Use Cases
3. Rest API Endpoints
4. Underlying Data Structures

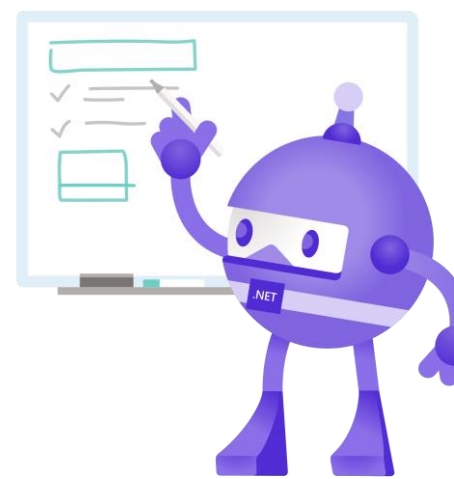
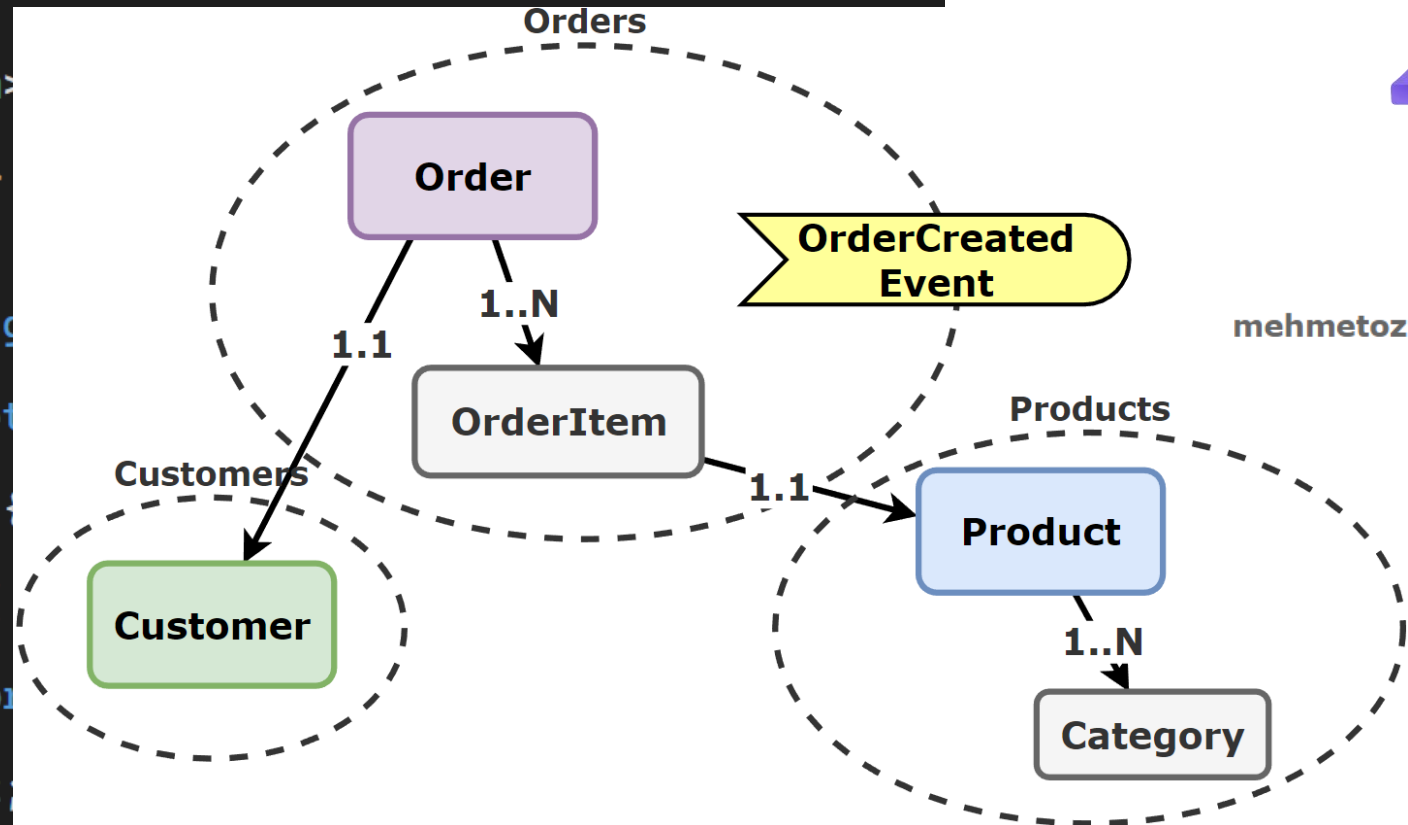


1. Domain Models of Ordering Microservices

- Primary domain model is 'Order' and 'OrderItem'

```
public class Order : Aggregate<OrderId>
{
    private readonly List<OrderItem>
    6 references
    public IReadOnlyList<OrderItem>

    5 references
    public CustomerId CustomerId { get; }
    9 references
    public OrderName OrderName { get; }
    10 references
    public Address ShippingAddress { get; }
    10 references
    public Address BillingAddress { get; }
    8 references
    public Payment Payment { get; }
    4 references
    public OrderStatus Status { get; }
```



mehmetozkaya

- Consider integration event for order created action.

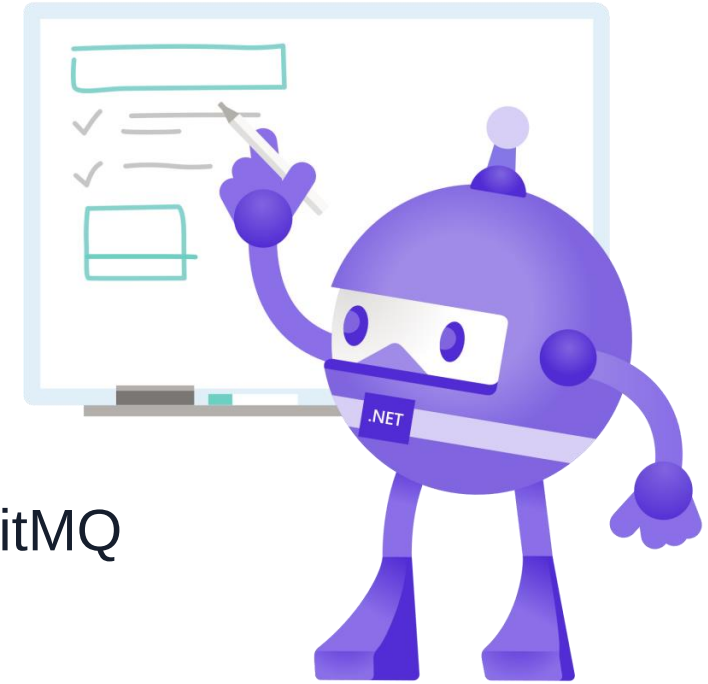
2. Application Use Cases of Ordering Microservices

CRUD Ordering Operations:

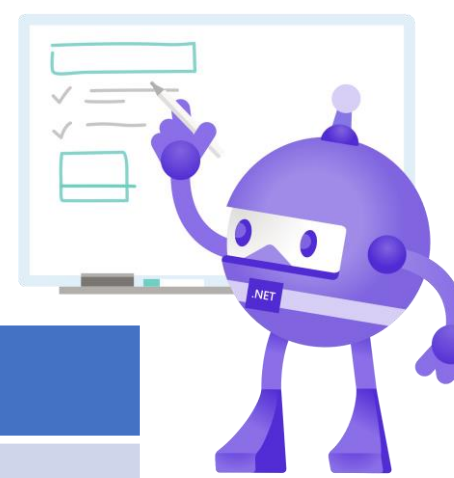
- Get Order with Items (can filter by Name and Customer)
- Create a new Order
- Update an existing Order
- Delete Order
- Add and Remove Item from Order

Asynchronous Ordering Operations:

- **Basket Checkout:** Consume Basket Checkout event from RabbitMQ using MassTransit
- **Order Fulfilment:** Perform order fulfilment operations (billing, shipment, notification..)
- Raise OrderCreated Domain Event that leads to integration event



3. Rest API Endpoints of Ordering Microservices



Method	Request URI	Use Cases
GET	/orders	Get Orders w/ Pagination
GET	/orders/{orderName}	Get Orders by OrderName
GET	/orders/customer/{customerId}	Get Orders by Customer
POST	/orders	Create a new Order
PUT	/orders	Update an existing Order
DELETE	/order/{id}	Delete Order with Id

- **AMQP:** Consume BasketCheckout Event from RabbitMQ using MassTransit

4. Underlying Data Structures of Ordering Microservices

Ordering Microservices has 1 Datastore:

1. SQL Server Relational Database

- A **relational database (SQL Server)** is used for storing Orders and OrderItems.
- **Entity Framework Core** is chosen as the ORM tool, applying Code-First Approach.
- The domain models are mapped to EF Core entities, considering DDD concepts like Aggregates and Value Objects.

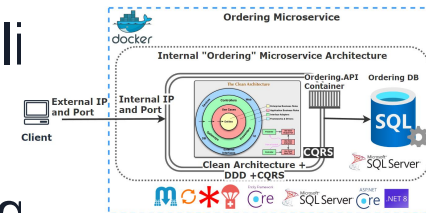


Ordering Microservice w/ DDD, CQRS and Clean Architecture

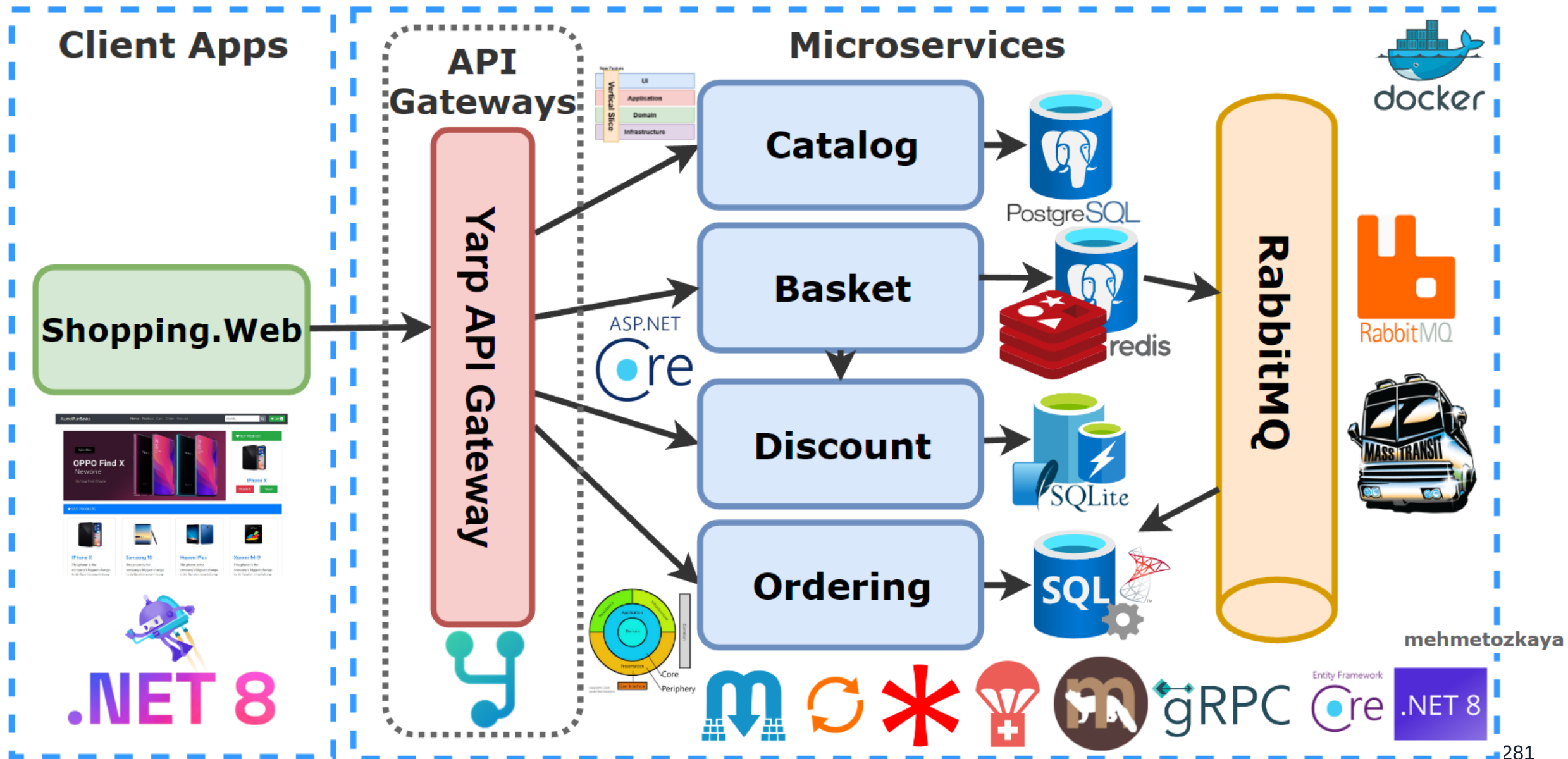
- Utilizes ASP.NET Core Minimal APIs, .NET 8, and C# 12 features.
- Embraces DDD, CQRS, and Clean Architecture patterns.
- Focuses on design principles like SOLID, KISS, YAGNI, SoC, DIP.

Clean Architecture Layers:

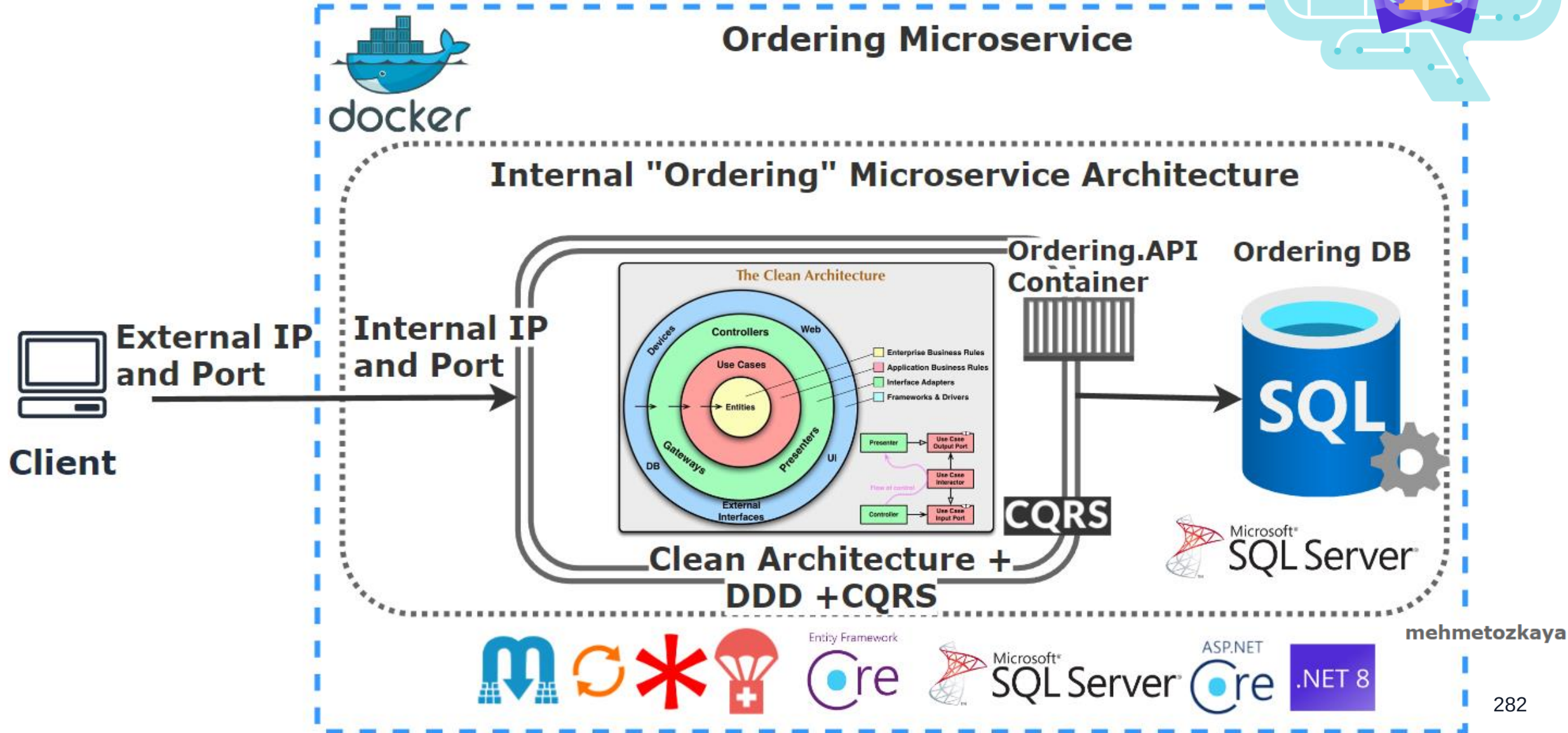
- **Domain Layer:** Tactical DDD implementations, including entities, value objects, and aggregates.
- **Infrastructure Layer:** Entity Framework Core for database operations, migrations, and mappings.
- **Application Layer:** Implementing CQRS using MediatR, domain event handling, and FluentValidation.
- **API Layer:** Using Carter for defining minimal API endpoints and implementing custom exception handling.
- Dockerfile and Docker-compose file for running Ordering microservice and SQL Server database in Docker environment



Big Picture – Course Project

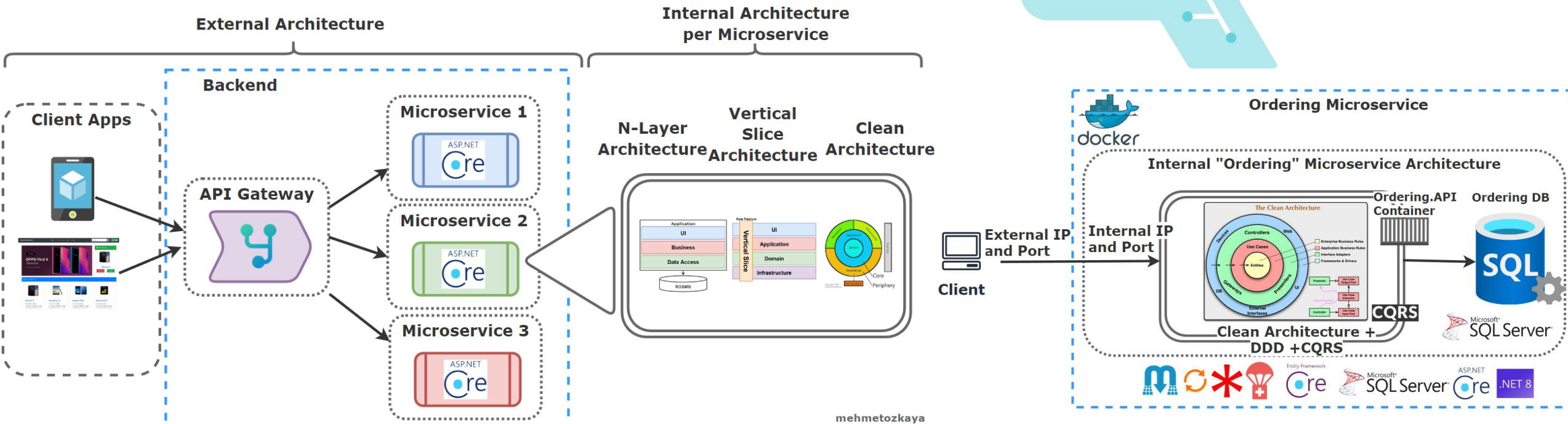


Ordering Microservice w/ DDD, CQRS and Clean Architecture



Technical Analysis of Ordering Microservices

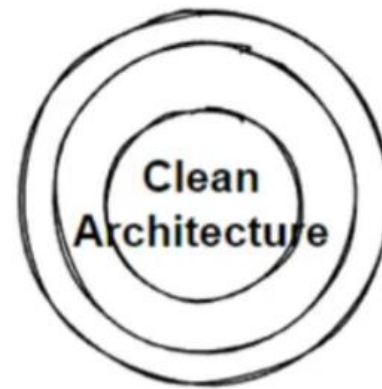
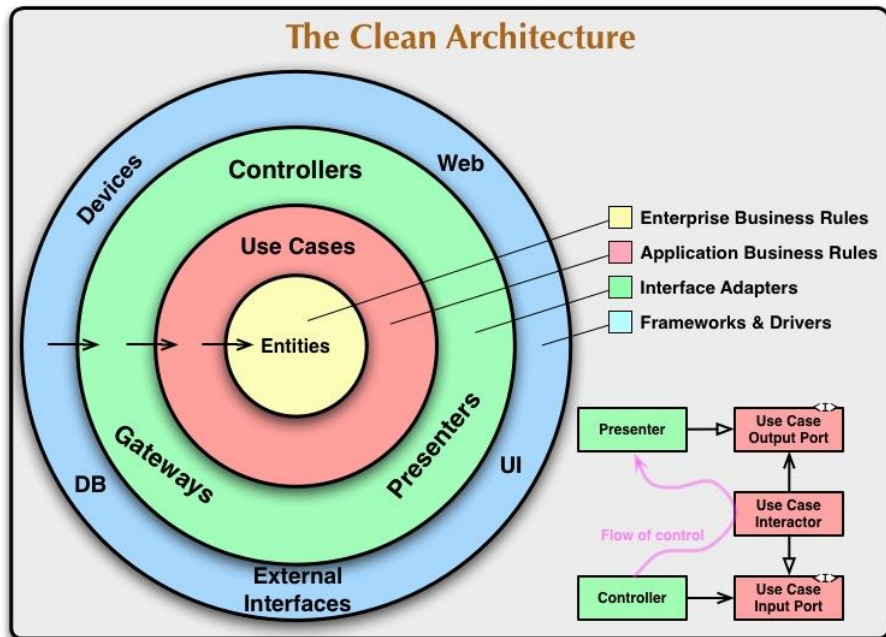
1. Application Architecture Style
2. Patterns & Principles
3. Libraries Nuget Packages
4. Project Folder Structure



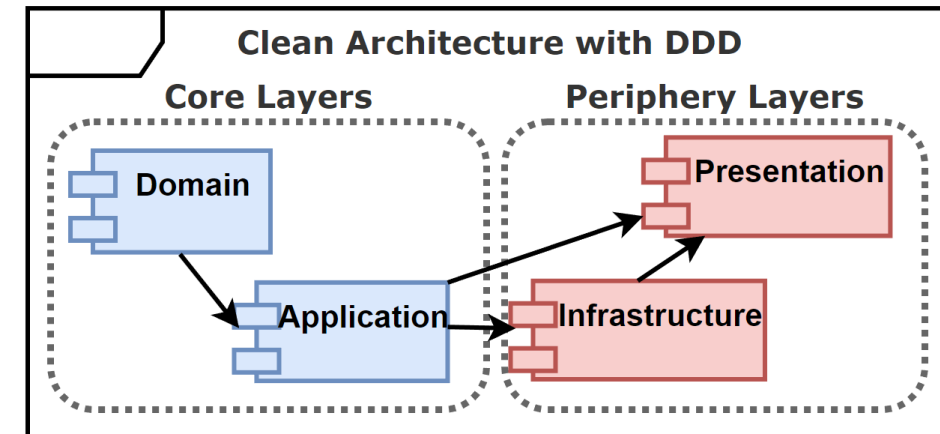
1. Application Architecture Style of Ordering Microservices

DDD, CQRS and Clean Architectures

- Fundamental architectural styles guiding the development of the Ordering microservices.
- Clean Architecture organized into 4 layers with Class Library project that reference according to clean architecture.



Domain Layer
Application Layer
Infrastructure Layer
Web UI Layer



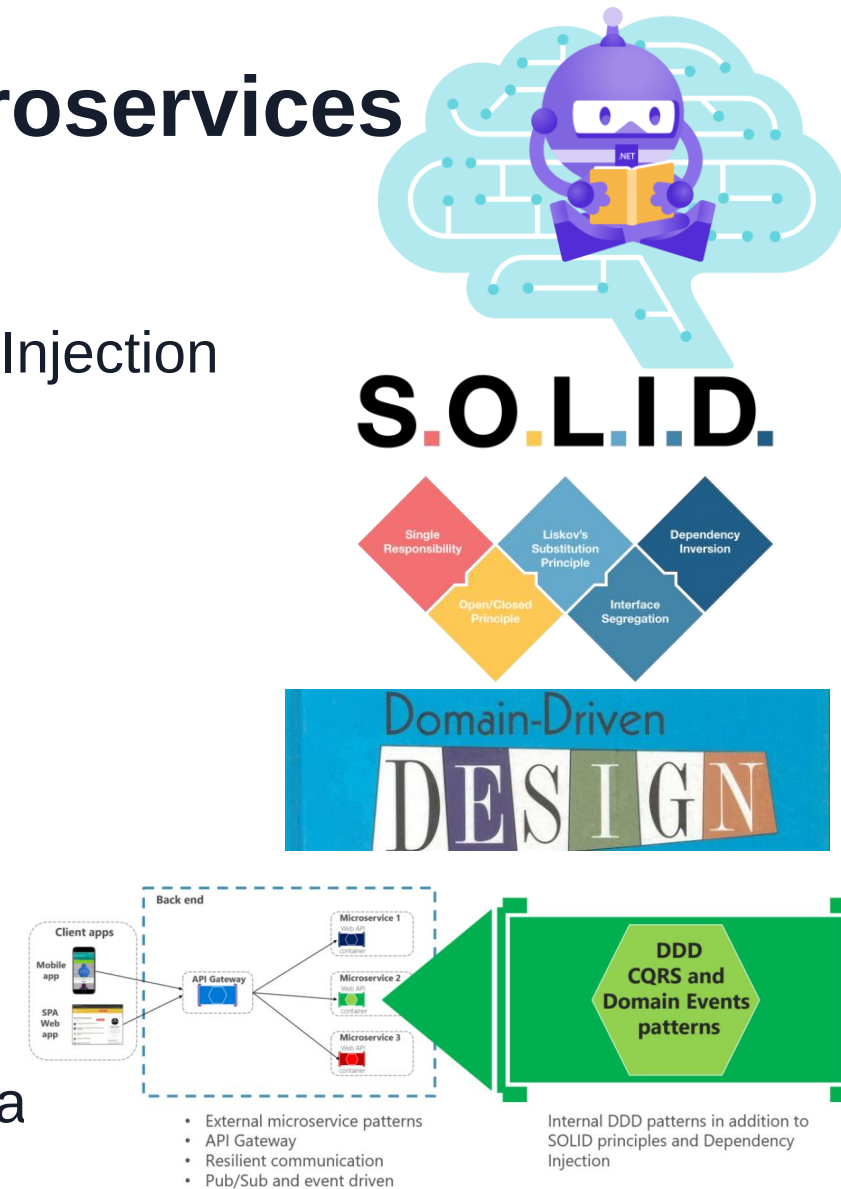
2. Patterns & Principles of Ordering Microservices

Common Principles

- SOLID, KISS, YAGNI, SoC, DIP principles and Dependency Injection are foundational.

Domain Layer Patterns

- Design a DDD-oriented microservice, DDD Tactical Patterns
- The Domain Entity pattern, Entity Base classes – SeedWork
- Anemic-domain model vs Rich-domain model
- The Value Object pattern
- The Aggregate pattern, The Aggregate Root or Root Entity pattern
- Strong Typed IDs Pattern
- Domain Events vs Integration Events



2. Patterns & Principles of Ordering Microservices

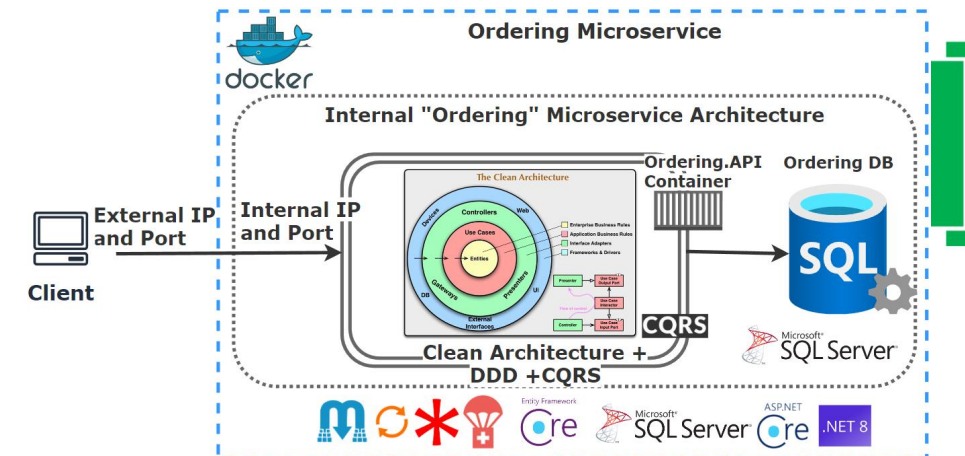
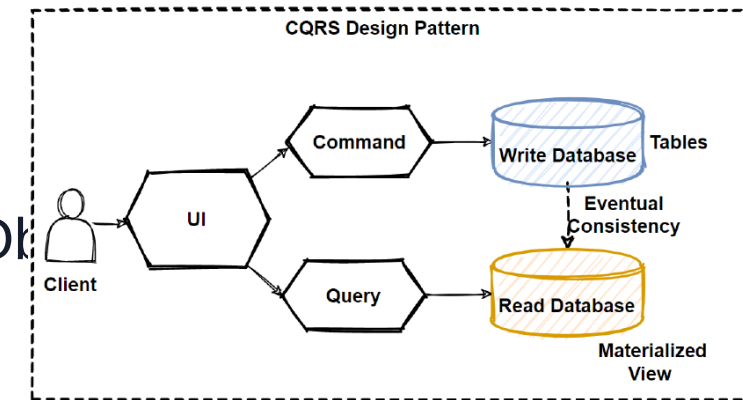


Infrastructure Data Layer Patterns

- Repository Pattern
- EF Core ORM - Code First Approach - Migrations - Seeding Database
- Value Object Complex Types, EF Aggregate Root Entities
- Entity Configurations w/ modelBuilder - DDD to EF Core Entity OK
- Raise & Dispatch Domain Events with EF Core and MediatR

Application UC Layer Patterns

- CQRS and CQS Pattern
- Command and Command Handler patterns
- Mediator Pattern, MediatR Pipeline Behaviours
- Fluent Validation, Logging Cross-cutting concerns



2. Patterns & Principles of Ordering Microservices

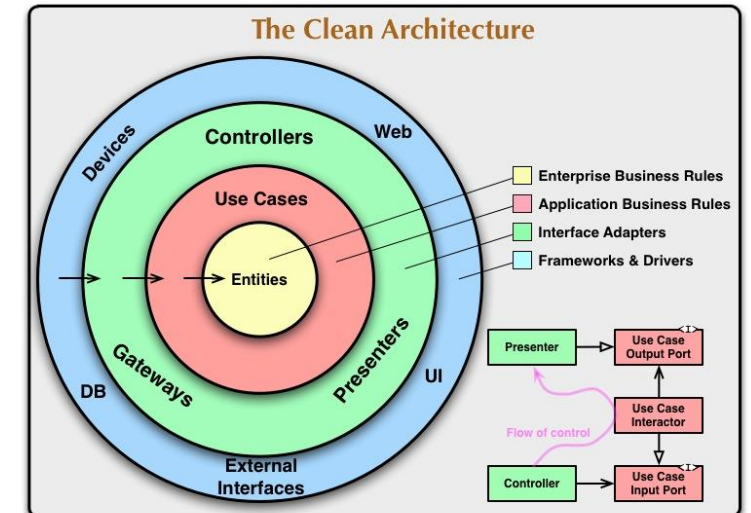
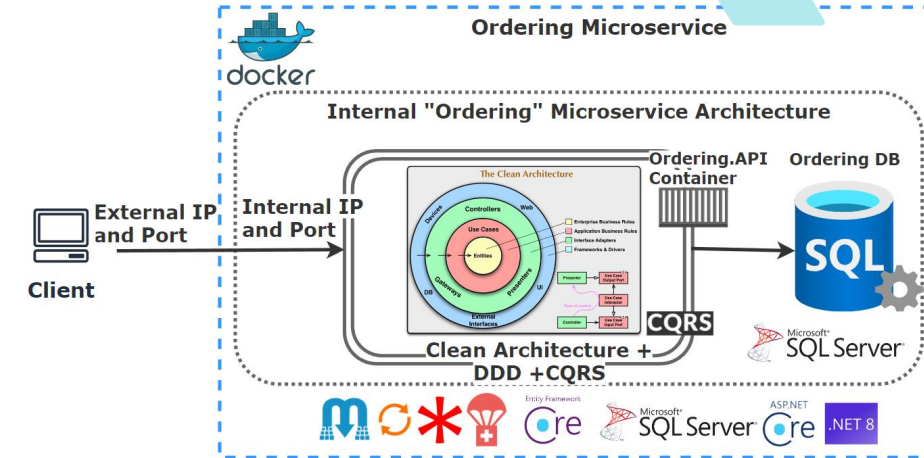
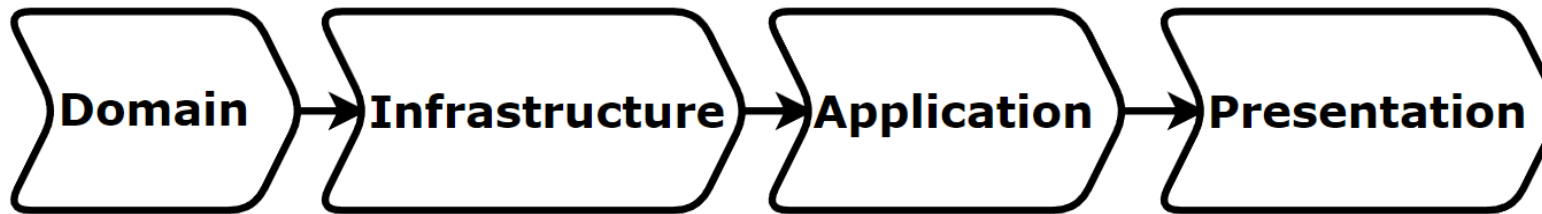


Presentation Api Layer Patterns

- Minimal APIs
- Asp.net DI, Routing with latest asp.net 8 features

SECTION FLOW

- Domain > Infrastructure > Application > Presentation
- Because Domain entities need to map EF.Core objects just after developed Domain Layer.



3. Libraries Nuget Packages of Ordering Microservi



Domain Layer Libraries:

- No Library

Infrastructure Data Libraries:

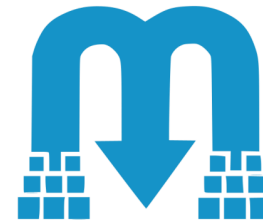
- Entity Framework Core packages for SQL Server and design tools

Entity Framework



Application UC Layer Libraries:

- Libraries from BuildingBlocks (e.g., MediatR, FluentValidation).
- Mapster for object mapping.



Presentation Api Layer Libraries:

- ASP.NET Core libraries.
- Carter for defining minimal API endpoints.



4. Project Folder Structure of Ordering Microservice

- Clean Architecture: 3 Class Library and 1 Web API: Ordering.Domain, Ordering.Application, Ordering.Infrastructure, Ordering.API

Ordering.Domain:

- Domain models like Order, OrderItem, Customer, Product.
- Value objects, events, exceptions, and base entity classes.

Ordering. Application:

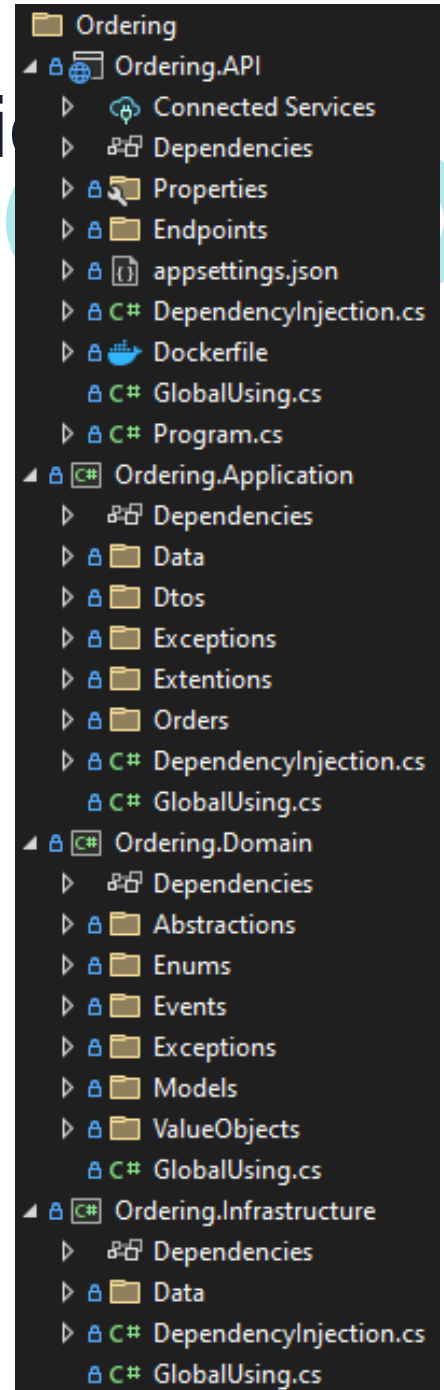
- CQRS commands and handlers organized by features.
- Dependency injection configurations.

Ordering. Infrastructure:

- Database context, configurations, migrations, and initial data seeding.

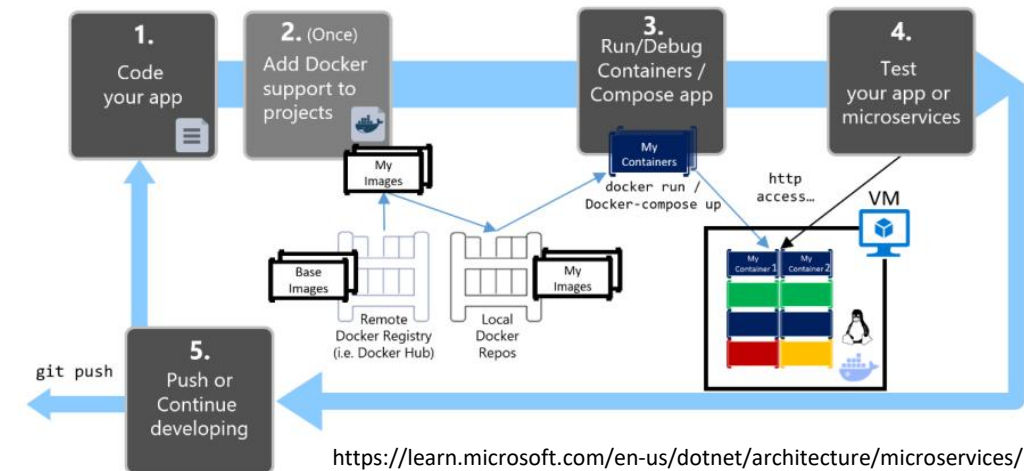
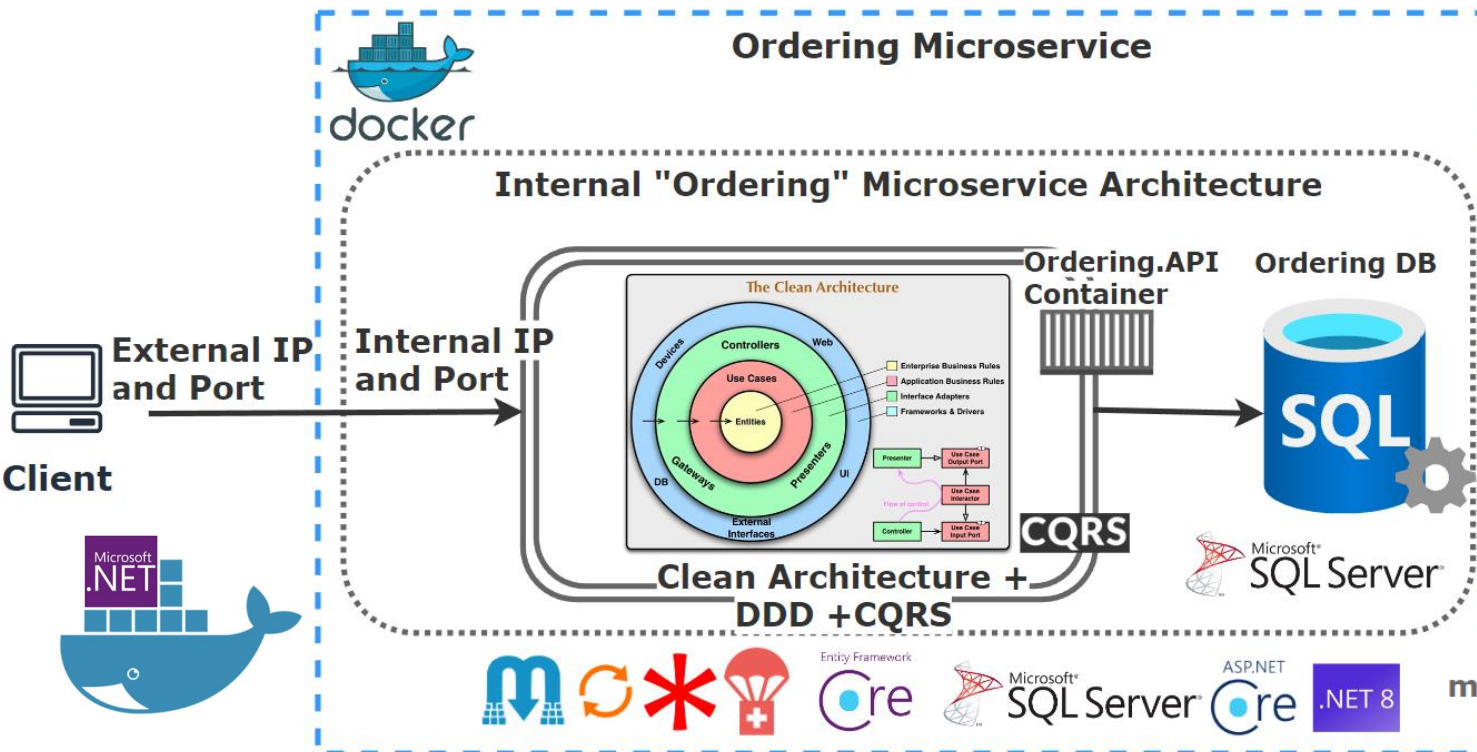
Ordering. API:

- API contracts and endpoint definitions.



Deployment of Ordering Microservices

- **Containerize** our Ordering service with **Docker**
- Implement **Dockerfile** and **docker-compose** file for running **Ordering** microservice and **SQL Server** database in Docker environment.



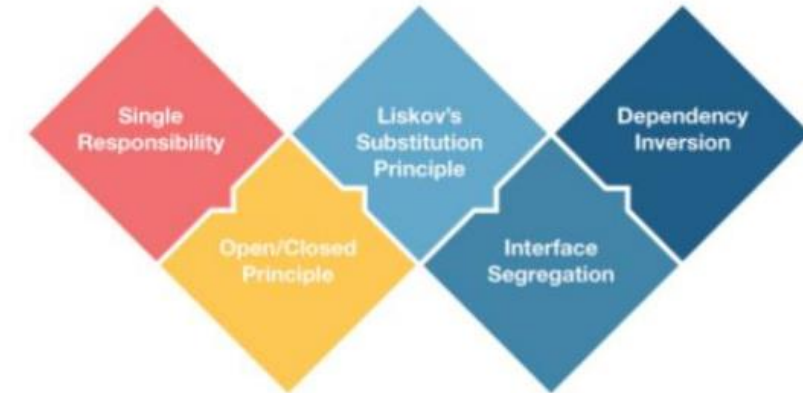
<https://learn.microsoft.com/en-us/dotnet/architecture/microservices/>

mehmetozkaya

Design principles - SOLID

- **Single Responsibility**
Each of your components or modules should responsible only one functionality.
- **Open-Closed Principle**
When we design the system, it should able to extend without changing existing architecture.
- **Liskov Substitution Principle**
Systems can be substitute each other easily. In our case we can use plug-in services that we can shift them easily.
- **Interface Segregation Principle**
States that no code should be forced to depend on methods it doesn't use.
- **Dependency Inversion Principle**
States that high-level modules should not depend on low-level modules; both should depend on abstractions.

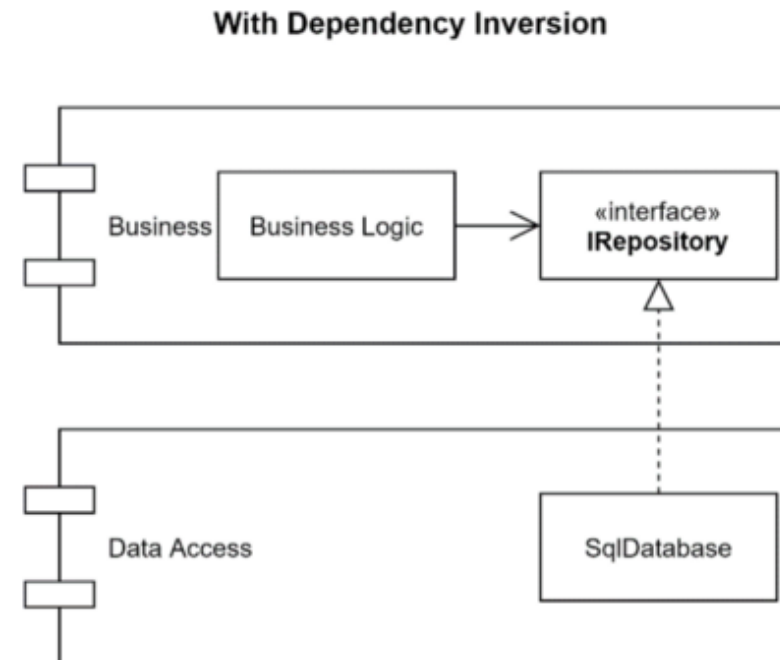
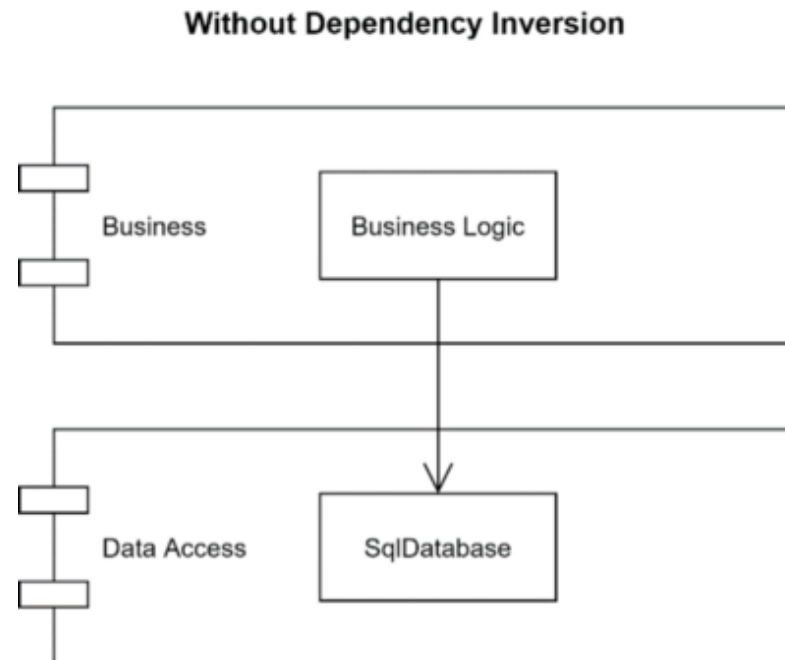
S.O.L.I.D.



<https://medium.com/bgl-tech/what-are-the-solid-design-principles-c61feff33685>

Dependency Inversion Principles (DIP)

- Provide to **break dependency** of classes by **inverting dependencies**, inject dependent classes via constructor.
- **Upper-level** modules or classes and **lower-level** classes **must not be dependent** on modules.
- **Lower-level** modules **must be dependent** on **higher-level** modules (interfaces of modules).



Design principles - Separation of Concerns (SoC)

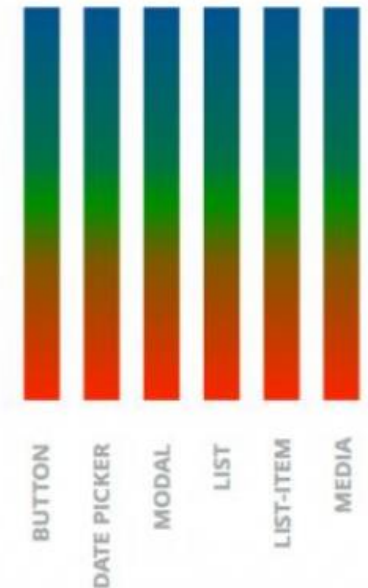
- **Separation of concerns(SOC)** is one of the core software design principle
- Separation of concerns is a design principle for separating a computer program into **distinct sections**
- **Isolate** the software application into separate sections, **manages complexity** by partitioning the software system
- **Distinguish between the concepts** of layer and tiers with certain responsibilities.
- Elements in the software should be **unique**
- Limits to allocate responsibilities
- Low-coupling, high-cohesion

Separation of Concerns

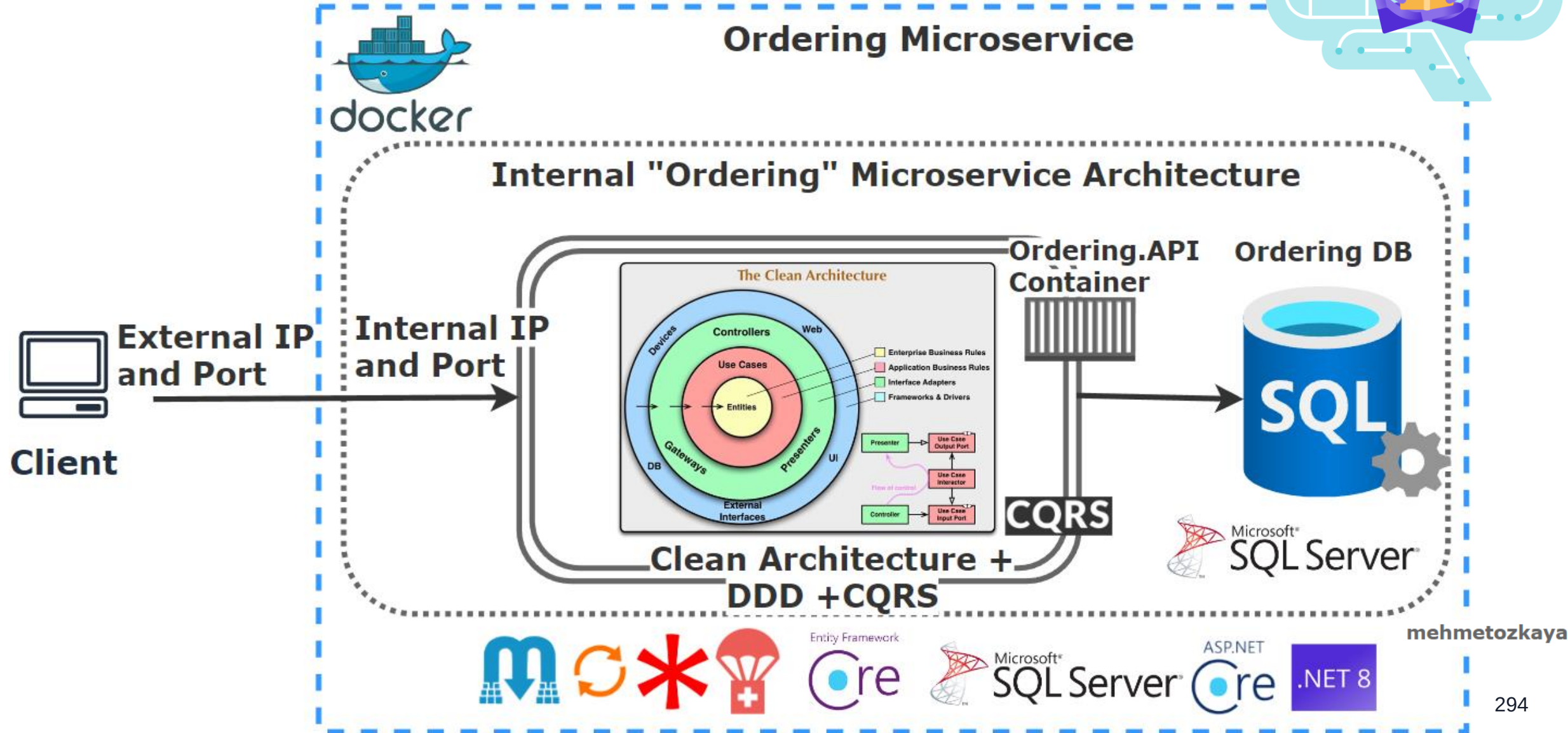


Separation of Concerns

(only, from a different point of view)



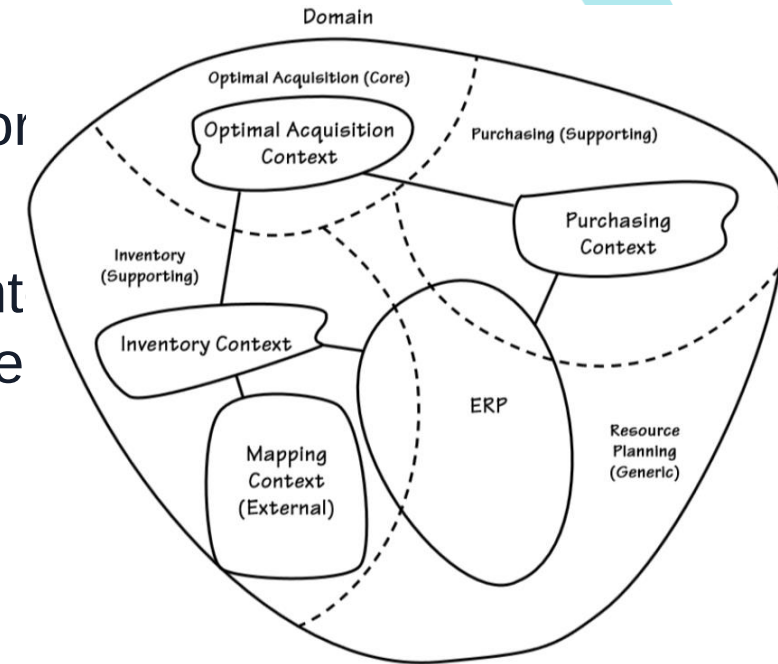
Ordering Microservice w/ DDD, CQRS and Clean Architecture



Domain Driven Design - DDD



- **Domain-Driven Design (DDD)** has been around since **Eric Evans** published his book about the subject in **2003**.
- **Domain Driven Design (DDD)** is not an improved technology or specific method.
- DDD is solving a **complex problem** is to **break the problem** into **smaller parts** and **focus on smaller problems** that are relative easy.
- A **complex domain** may **contain sub domains**. And some of **sub domains** can **combine** and **grouping** with each other for common rules and responsibilities.



DDD Types: Strategic and Tactical DDD

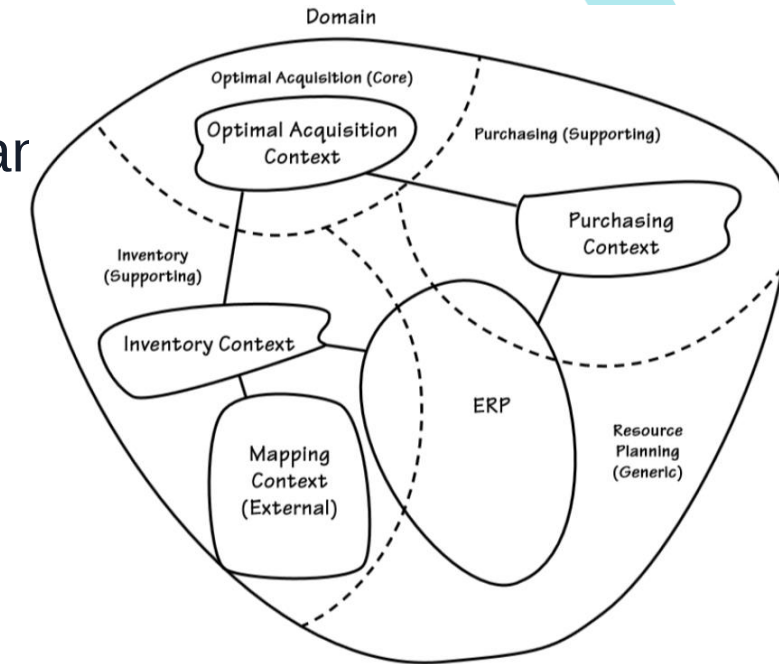


Strategic DDD:

- Understanding and modeling the business domain.
- It involves identifying the different domains, their subdomains, and how they interact with each other.

Tactical DDD:

- Implementation details and provides design patterns.
- Including patterns like Entities, Value Objects, Aggregates, etc.
- We'll focus on Strategic DDD and its key concepts.



DDD Concepts

- **DDD Concepts:** Domain, SubDomain, Ubiquitous Language, Bounded Context, Context Mapping

Domain:

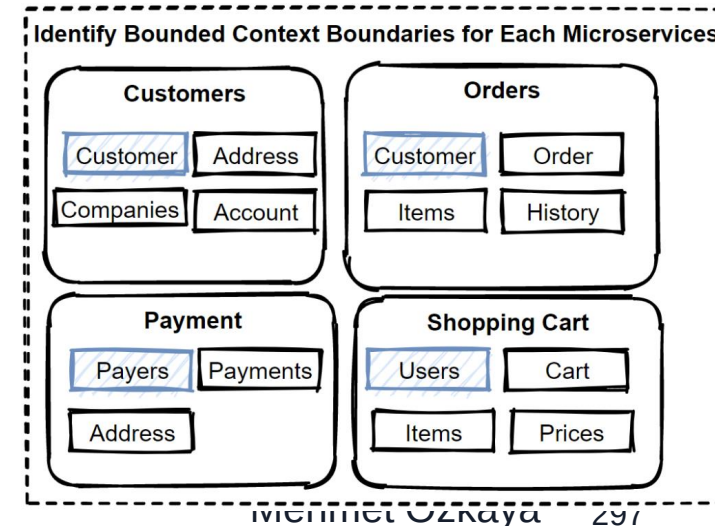
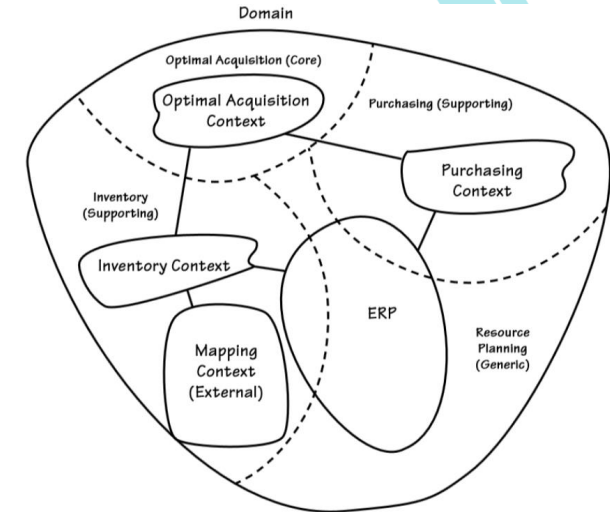
- The sphere of knowledge and activity around which the application logic revolves.
- Represents the problem space - the area of expertise or business need that the software addresses.

SubDomain:

- Represents a specific area of expertise within the overall domain.

Ubiquitous Language:

- Common language used by developers and domain experts to ensure clarity and consistency.
- Crucial for this language to be used in both discussions and in the codebase.



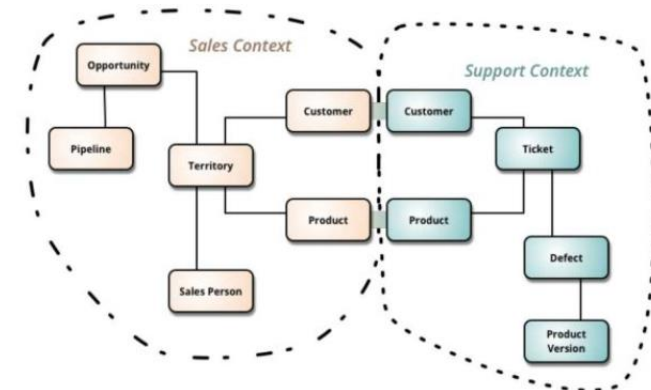
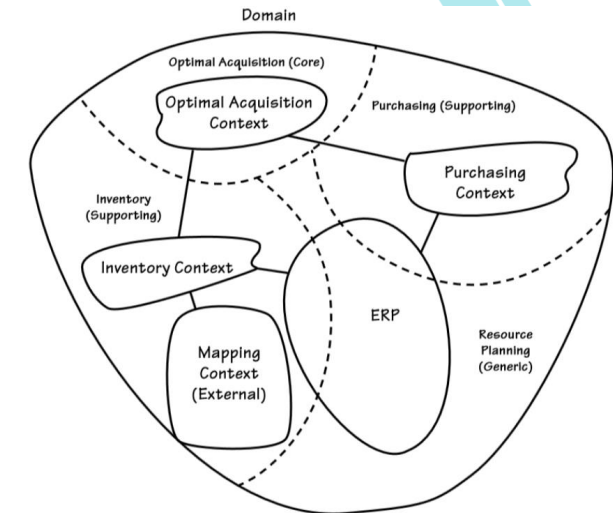
DDD Concepts - 2

Bounded Context:

- Grouping of closely related scopes that we can say logical boundaries.
- Logical boundary that represents the smaller problem particles of the complex domain that are self-consistent and as independent as possible.
- Each bounded context can have its database schema, its code models, and its team working on it.

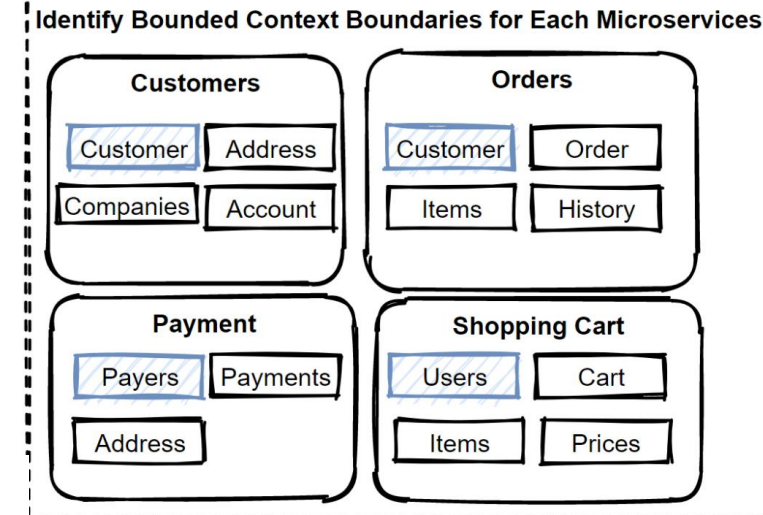
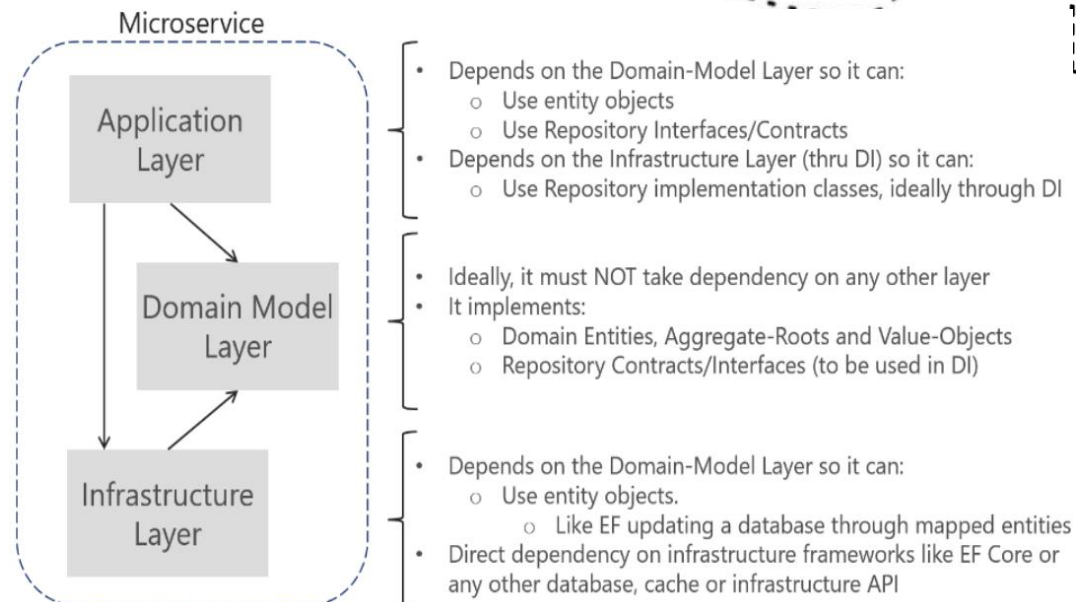
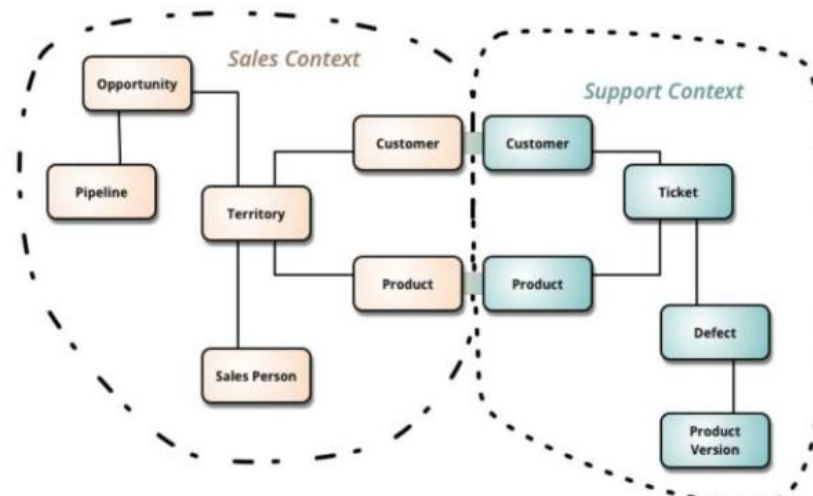
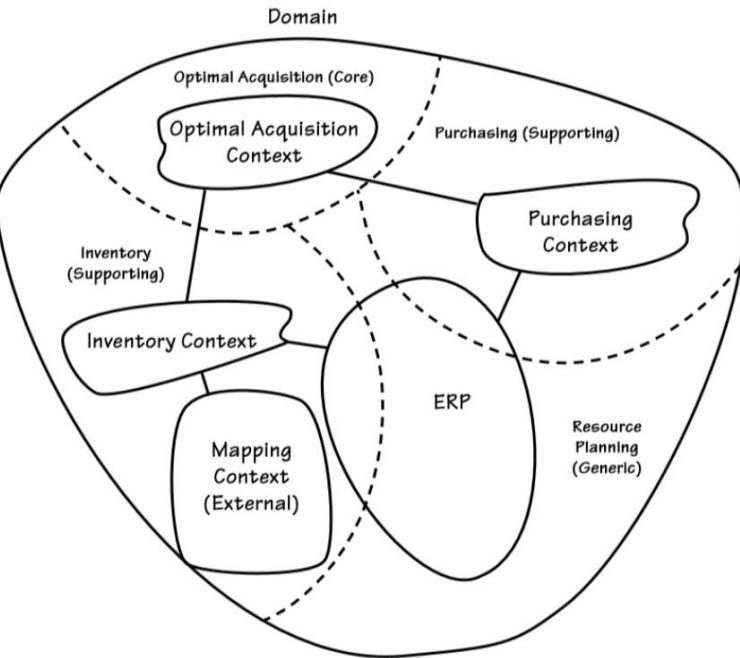
Context Mapping Pattern:

- Identify the whole bounded contexts in the application and with their logical boundaries.
- A way to define logical boundaries between domains.



DDD deals with large models by dividing them into different Bounded Contexts and being explicit about their interrelationships.

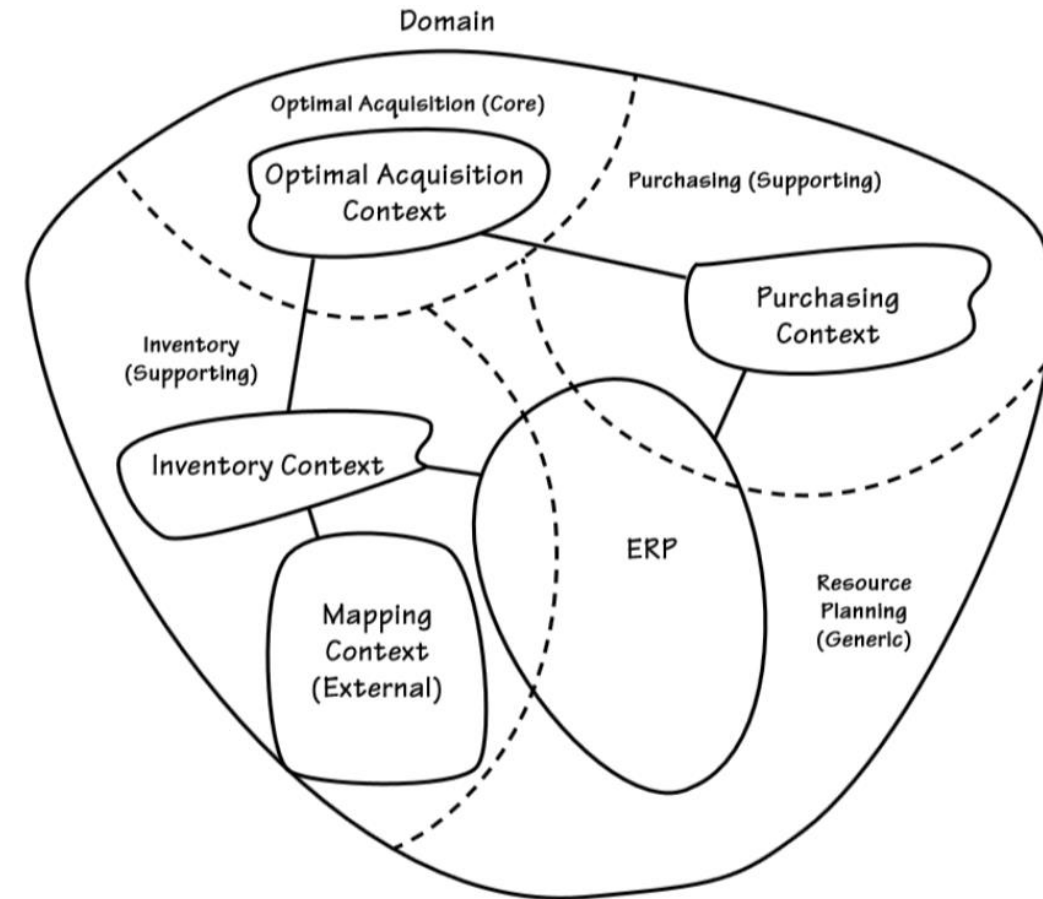
DDD Conclusion



<https://learn.microsoft.com/en-us/dotnet/architecture/microservices>

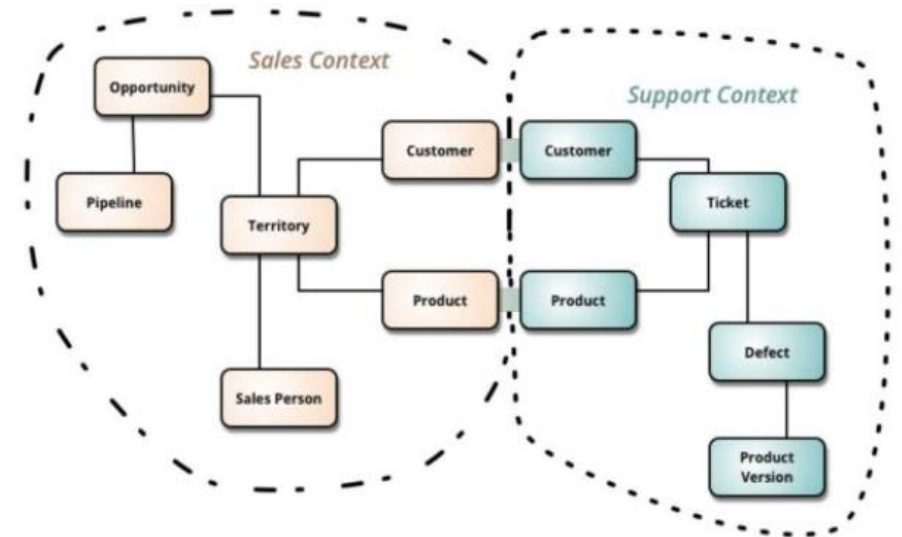
Bounded Context Pattern (Domain-Driven Design)

- **DDD - Bounded Context Pattern** which is one of the main pattern that we mainly use when **decomposing microservices**.
- **Domains** are require **high cooperation** and have a certain complexity by nature are called **collaborative domains**.
- DDD has **2 phases**, **Strategic** and **Tactical DDD**.
- **Strategic DDD**, we define the large-scale model of the system, defining to the business rules that allow designing loosely coupling units and the context map between them.
- **Tactical DDD** focuses on implementation and provides design patterns that we can use to build the software implementation.
- Include concepts such as **entity**, **aggregate**, **value object**, **repository**, and **domain service**.



Bounded Context Pattern (Domain-Driven Design)-2

- **DDD** domain defines its **own common language** and divides boundaries into specific, independent components. Common language is called **ubiquitous language**, and independent units are called **bounded context**.
- **DDD** is solving a **complex** problem is to break the problem into **smaller parts** and focus on smaller problems that are relatively easy.
- **A complex domain** may contain **sub domains**. And some of sub domains can **combine** and **grouping** with each other for **common rules** and **responsibilities**.
- **Bounded Context** is the grouping of closely related scopes that we can say **logical boundaries**.
- **Bounded context** is the **logical boundary** that represents the smaller problem particles of the complex domain that **are self-consistent** and as **independent** as possible.

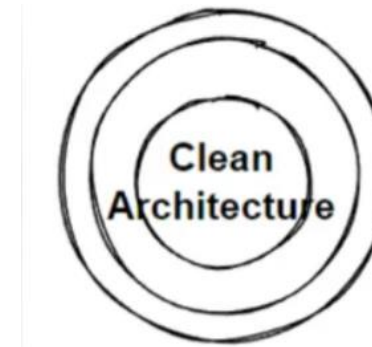
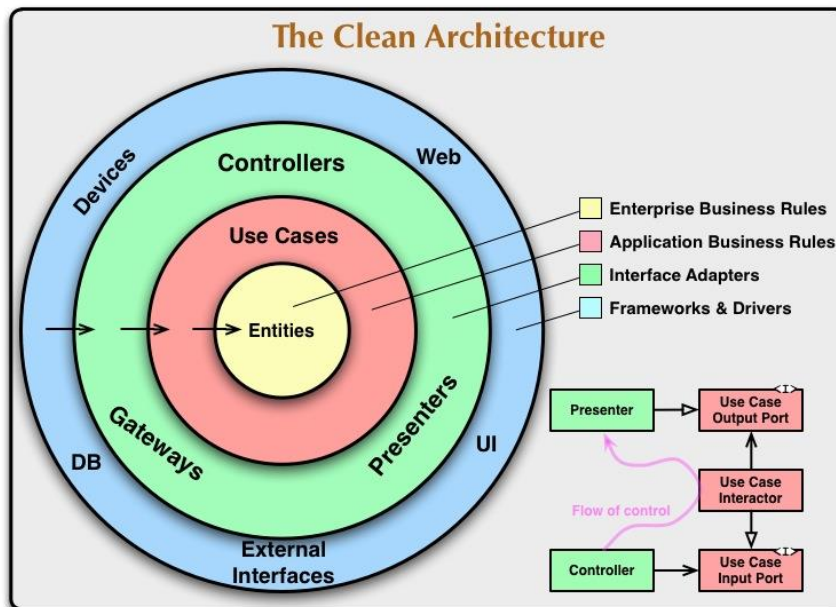


DDD deals with large models by dividing them into different Bounded Contexts and being explicit about their interrelationships.

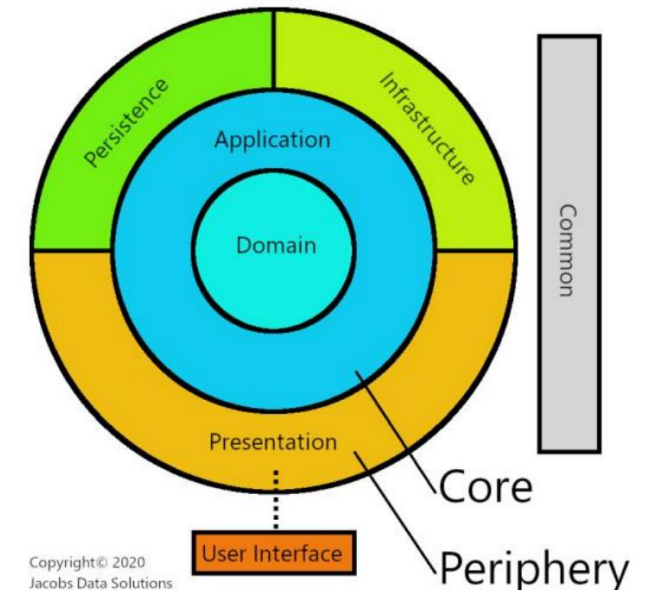
<http://martinfowler.com/bliki/BoundedContext.html>

Clean Architecture

- Introduced by **Robert C. Martin**, also known as **Uncle Bob**.
- Software architecture design that aims to **separate concerns** and **create systems** that are **independent of frameworks, UI, and databases**.
- Leads to systems that are more **maintainable**, **flexible**, and **adaptable** to changes.



Domain Layer
Application Layer
Infrastructure Layer
Web UI Layer



Key Principles of Clean Architecture

Independence of Frameworks:

- The system is not tightly coupled with a specific framework, which makes it adaptable to changes in frameworks and tools.

Testable:

- The business rules can be tested without the UI, database, web server or any external element.

UI Agnostic:

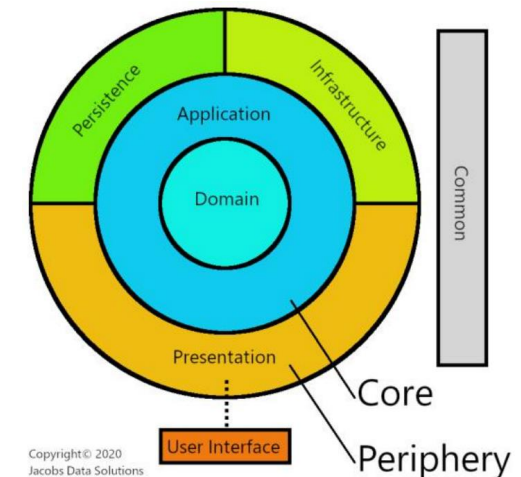
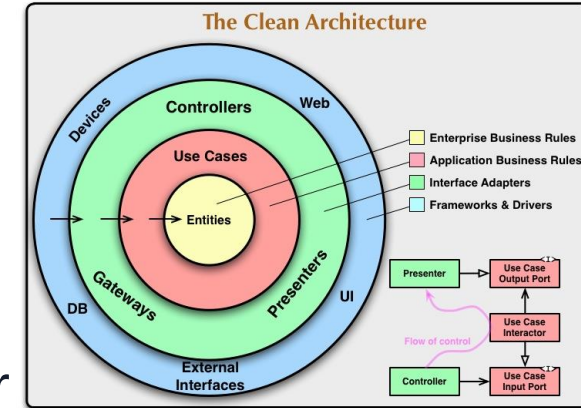
- The UI can change easily, without changing the rest of the system.

Database Agnostic:

- Business rules are not bound to the database, decoupled from the underlying data store.

External System Agnostic :

- Business rules don't know anything about the outside world, isolated.



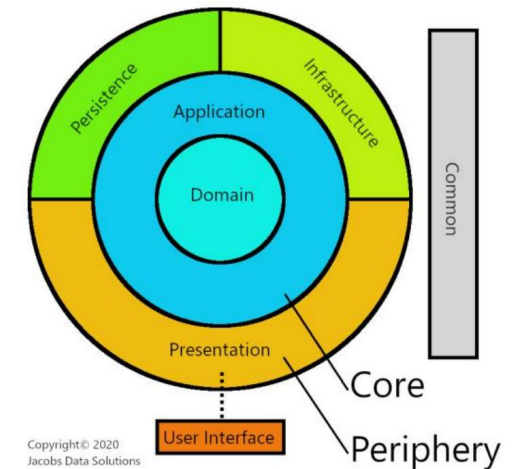
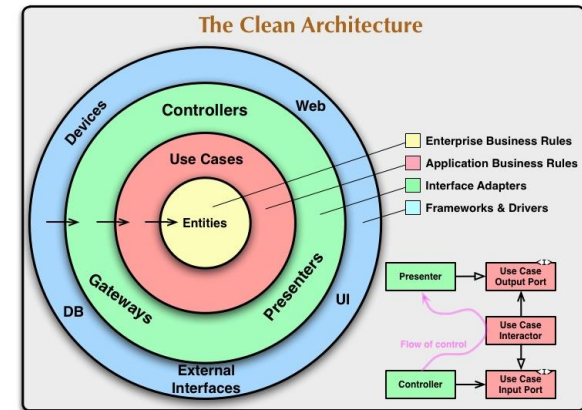
Clean Architecture Layers

Entities Layer (Domain Layer):

- Contains enterprise-wide business rules.
- Entities encapsulate the most general and high-level rules.
- Example: In an ordering microservice, entities might be Order, OrderItem, etc.

Use Cases Layer (Application Layer):

- Contains application-specific business rules.
- Encapsulates and implements all of the use cases of the system.
- Example: CreateOrder, CancelOrder, and UpdateOrderStatus use cases.



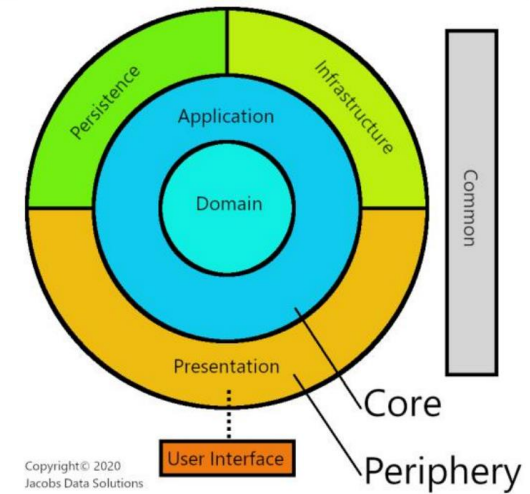
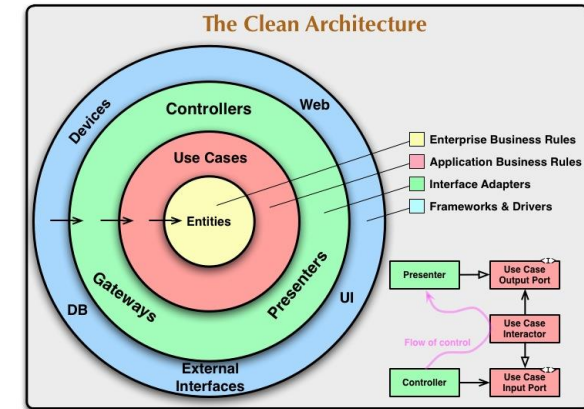
Clean Architecture Layers - 2

Interface Adapters Layer (Infrastructure Layer):

- Converts data from the format most convenient for the use cases and entities, to the format most convenient for external systems.
- Example: Mapping data from database models to domain entities.

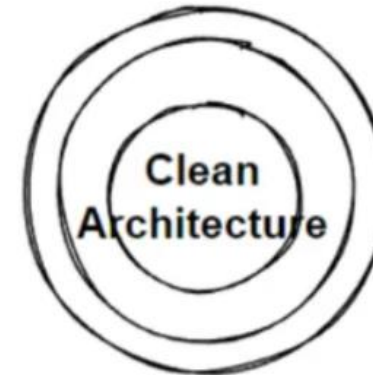
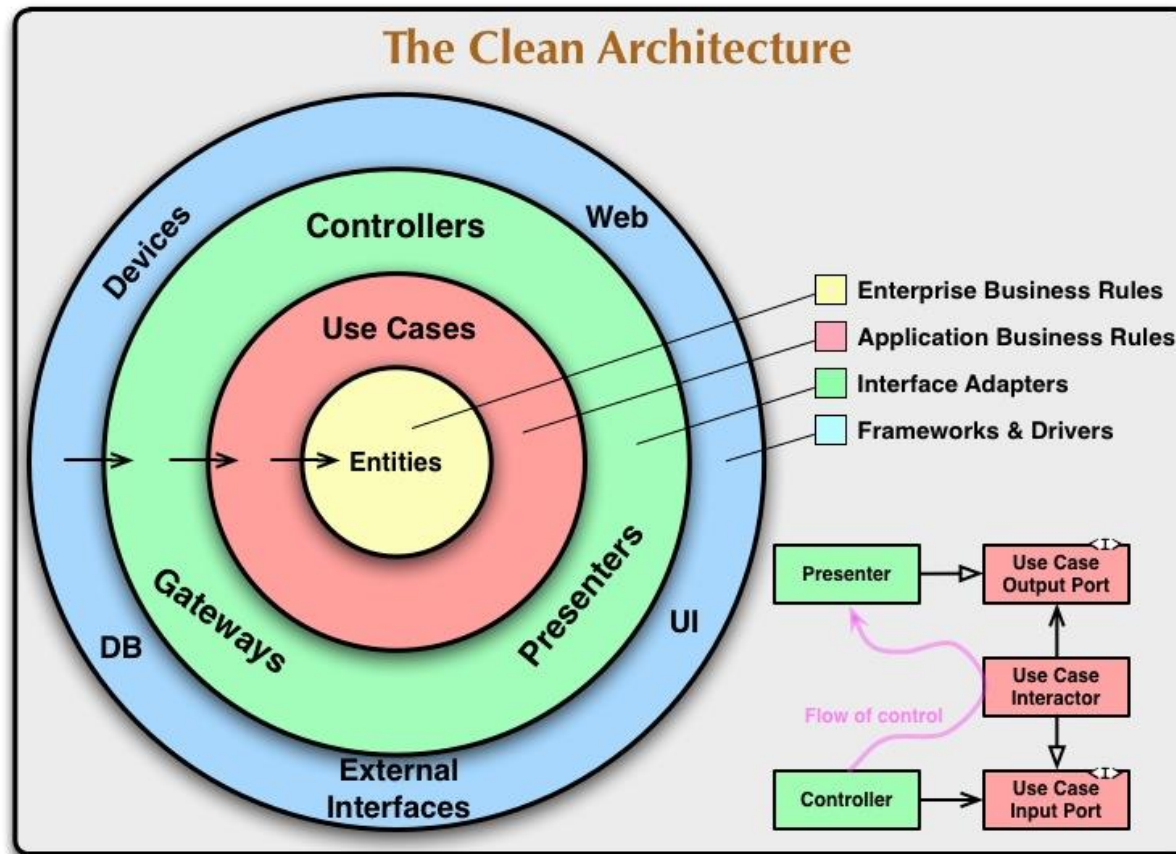
Frameworks and Drivers Layer (Infrastructure/External Concerns):

- The outermost layer consisting of frameworks and tools such as the Database, the Web Framework, etc.
- Generally includes the UI, database, external interfaces, etc.
- Example: REST controllers, database repositories.

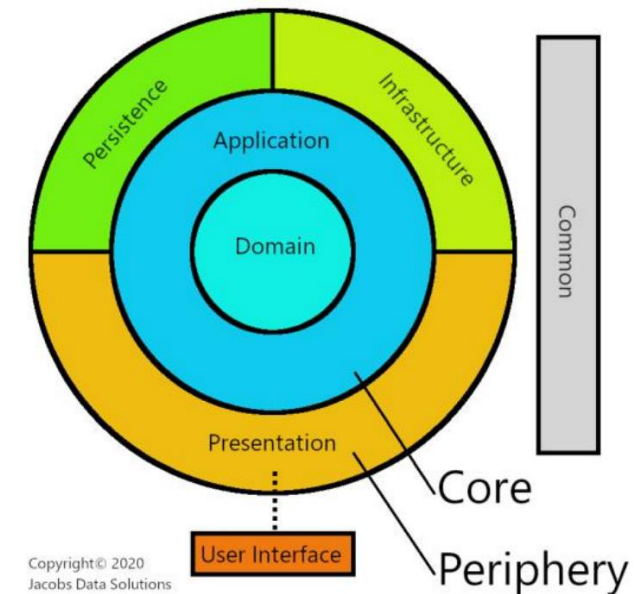


Clean Architecture

Clean Architecture: Modular, Adaptable, Testable and Maintainable

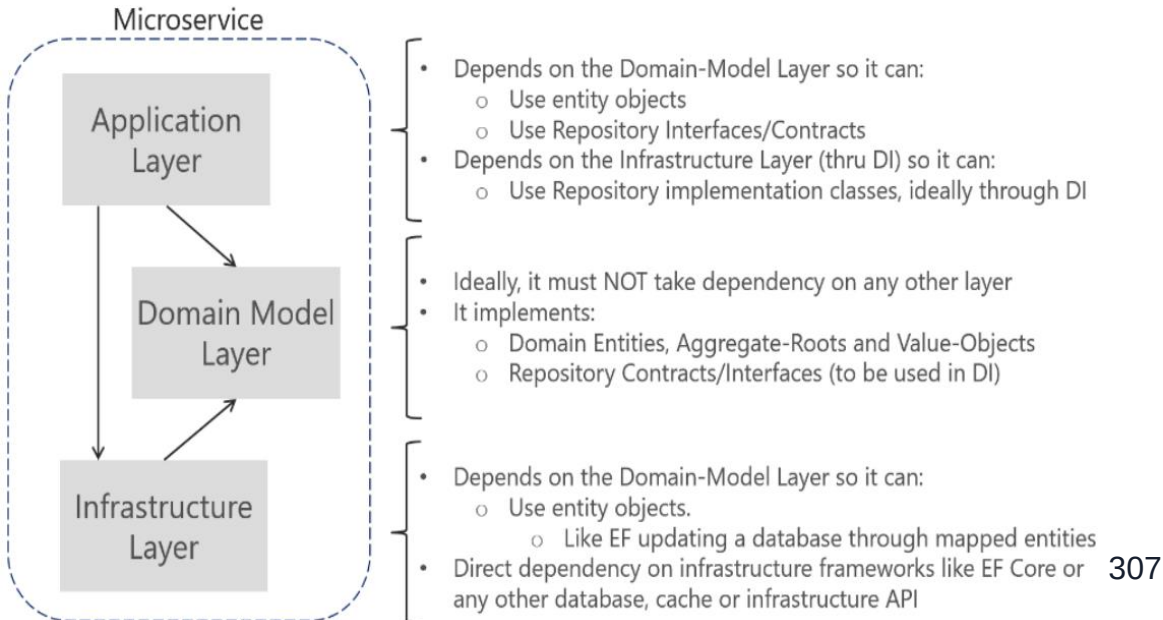
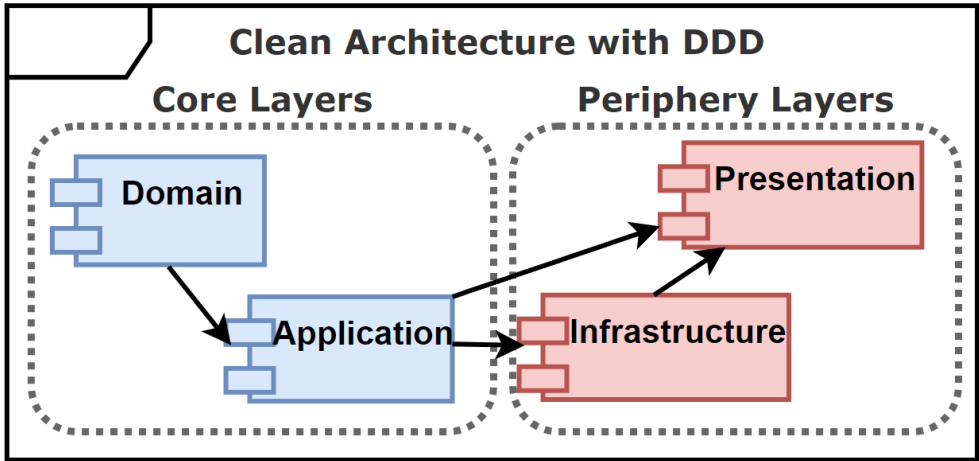
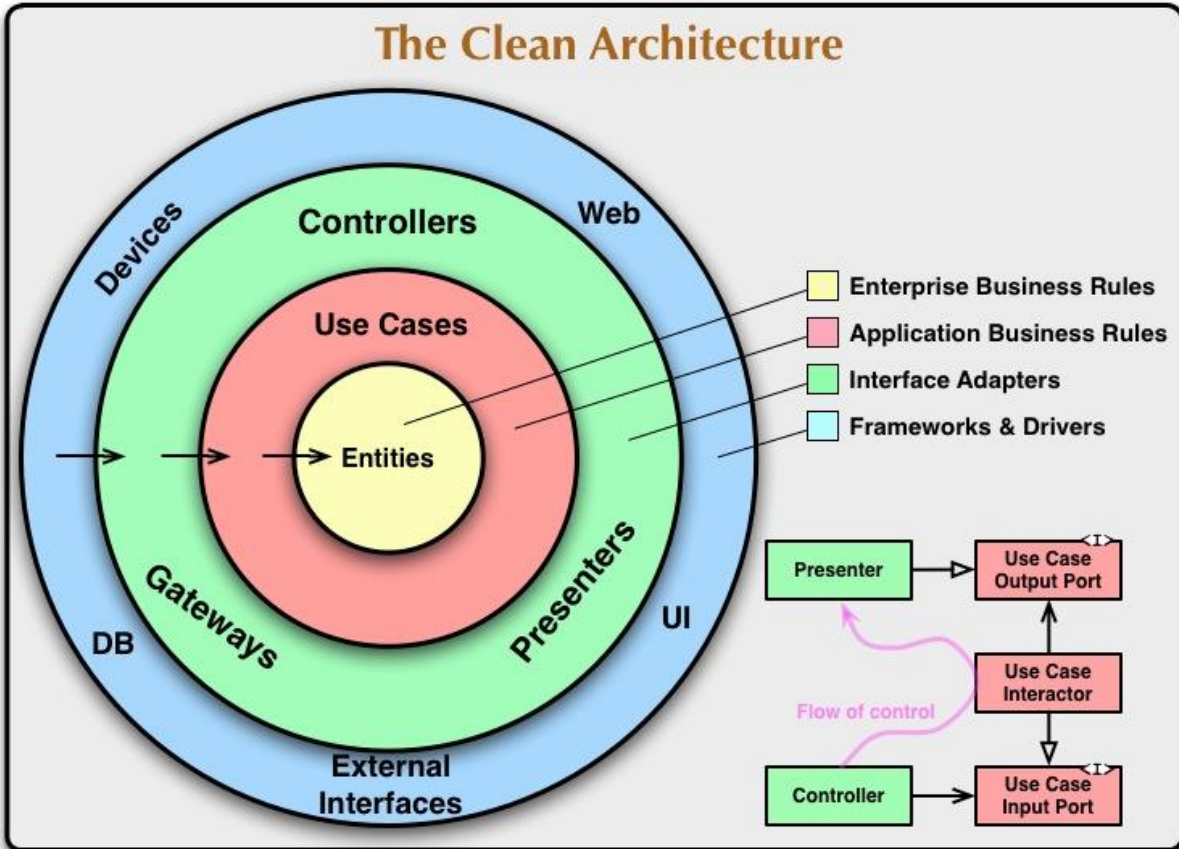


Domain Layer
Application Layer
Infrastructure Layer
Web UI Layer



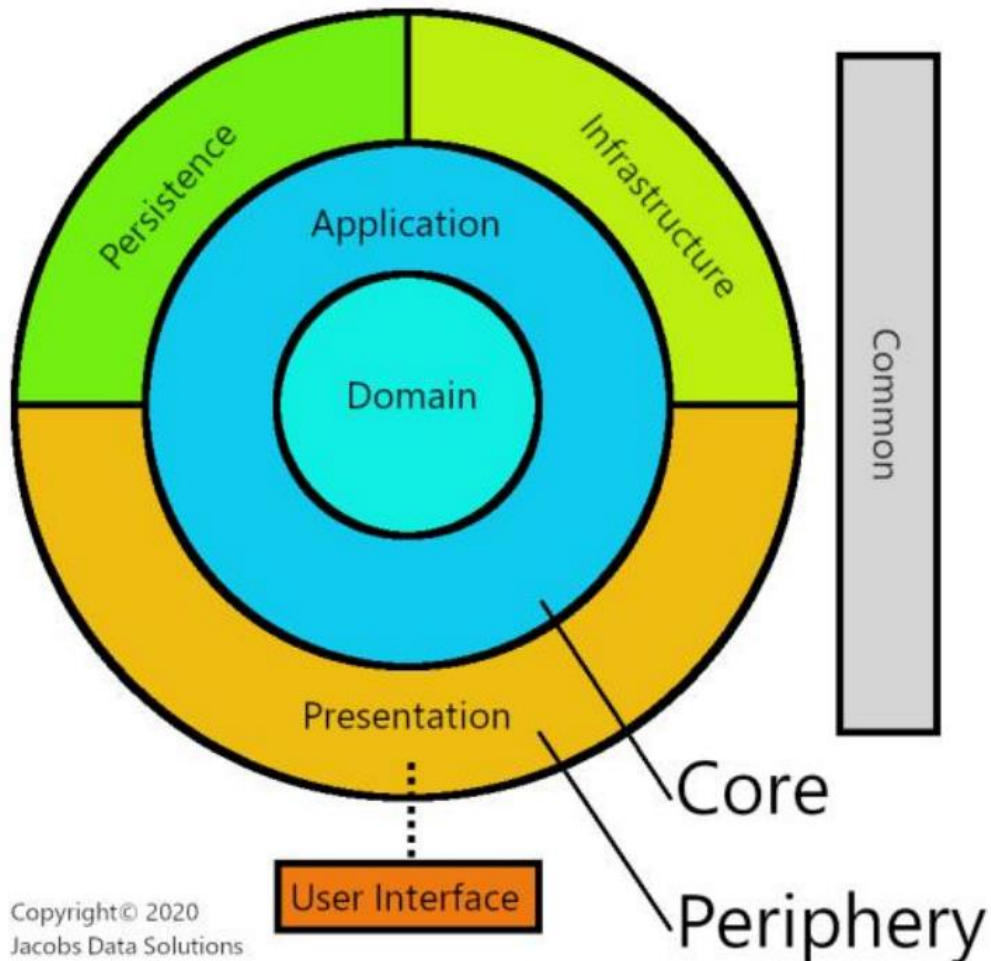
Clean Architecture with Domain Driven Design(DDD)

Clean Architecture: Modular, Adaptable, Testable and Maintainable

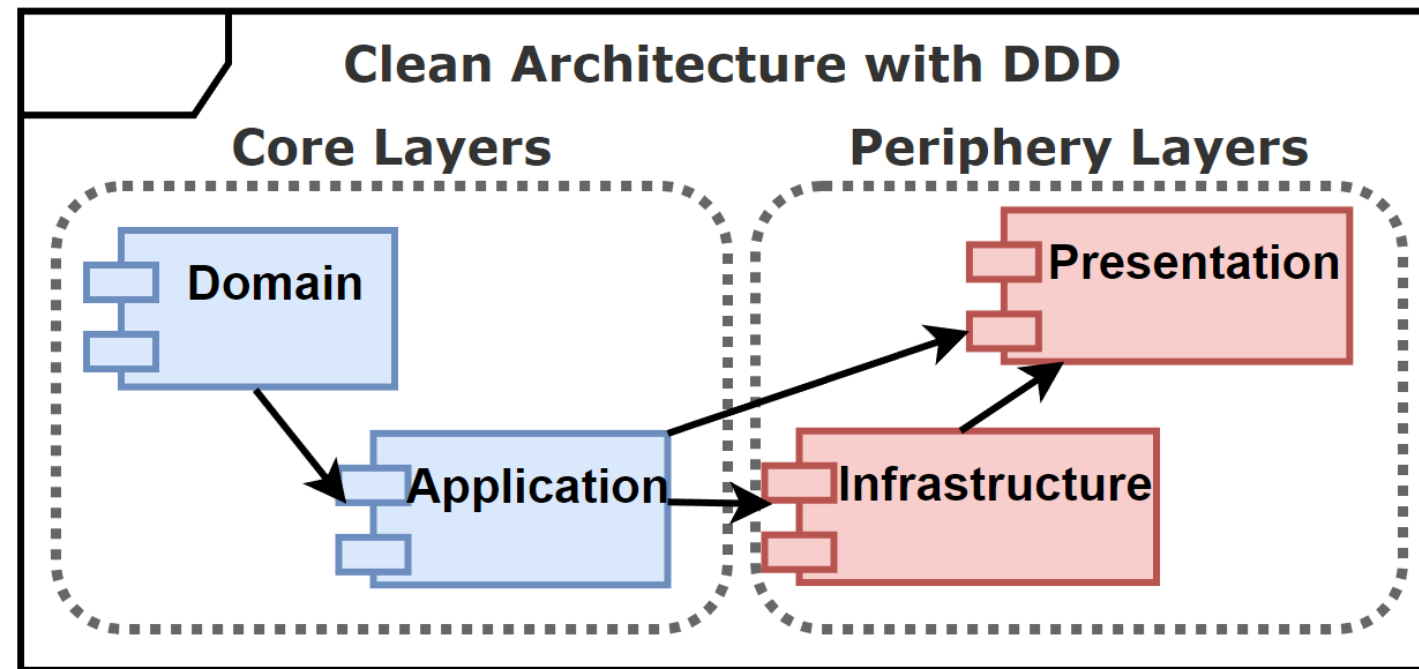


Integrating DDD in Clean Architecture Layers

Clean Architecture: Modular, Adaptable, Testable and Maintainable

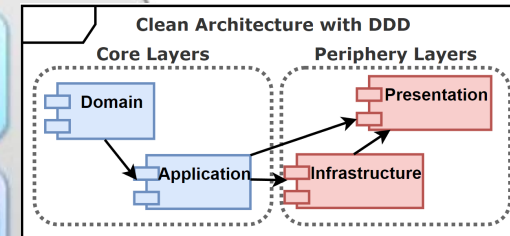
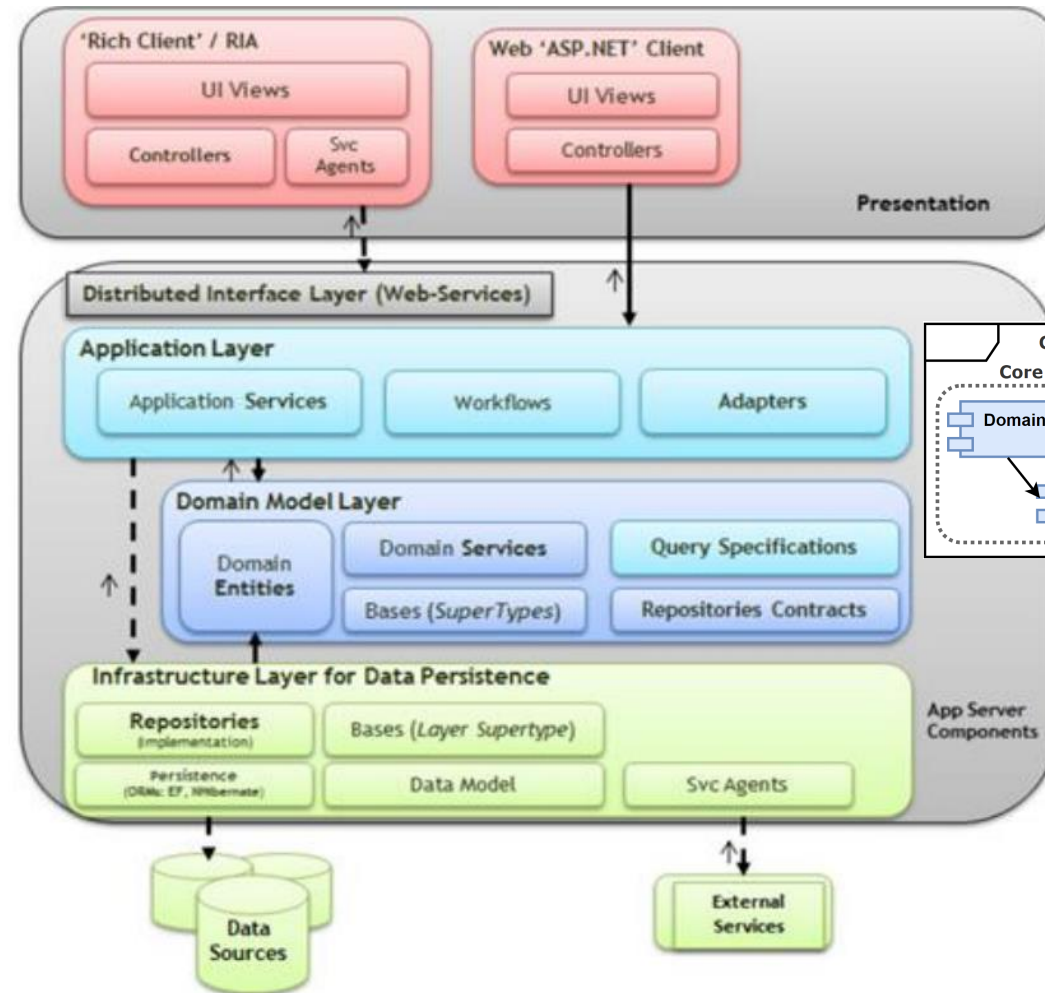
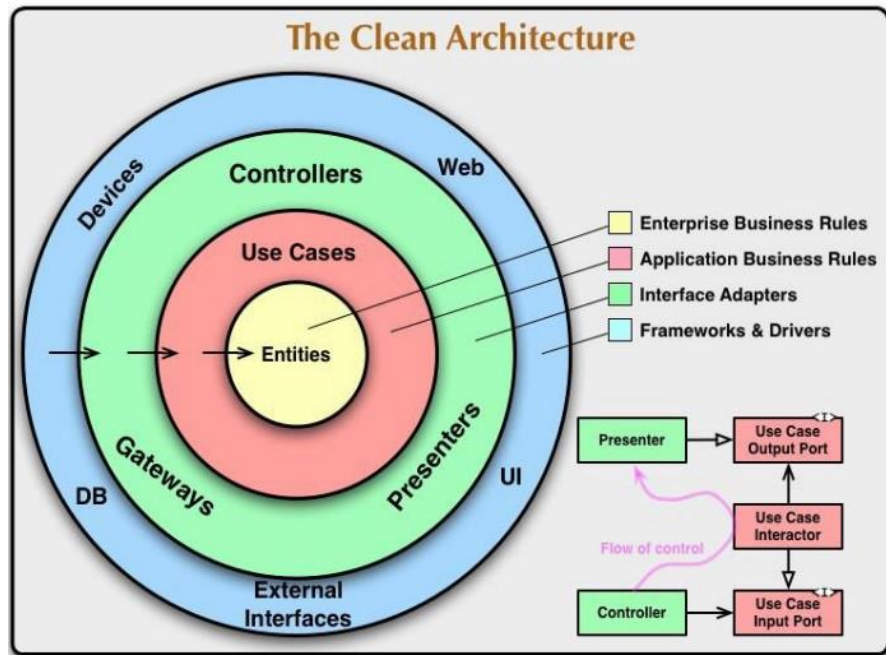


Copyright© 2020
Jacobs Data Solutions

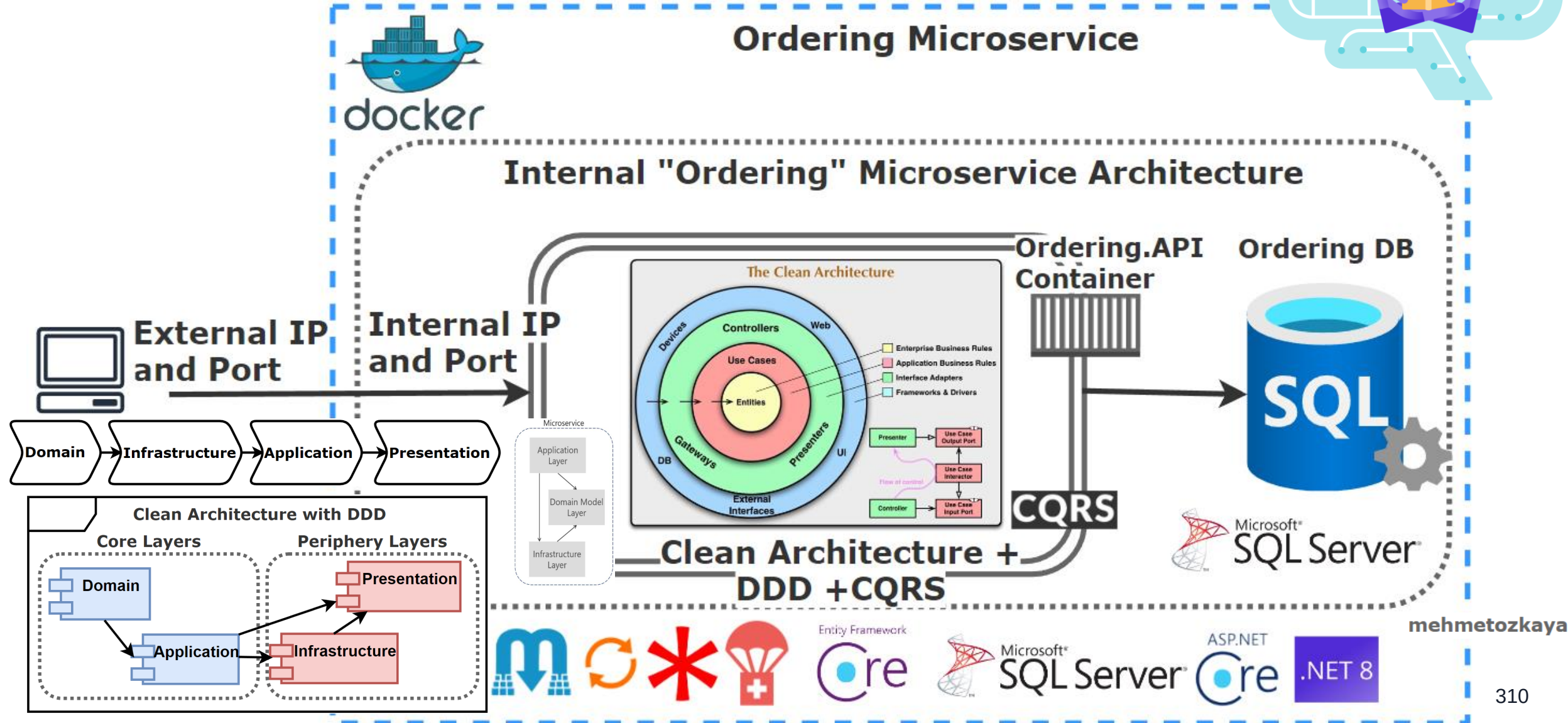


Clean Architecture with Domain Driven Design(DDD)

Clean Architecture: Modular, Adaptable, Testable and Maintainable



Ordering Microservice w/ DDD, CQRS and Clean Architecture



4. Project Folder Structure of Ordering Microservice

- Clean Architecture: 3 Class Library and 1 Web API: Ordering.Domain, Ordering.Application, Ordering.Infrastructure, Ordering.API

Ordering.Domain:

- Domain models like Order, OrderItem, Customer, Product.
- Value objects, events, exceptions, and base entity classes.

Ordering. Application:

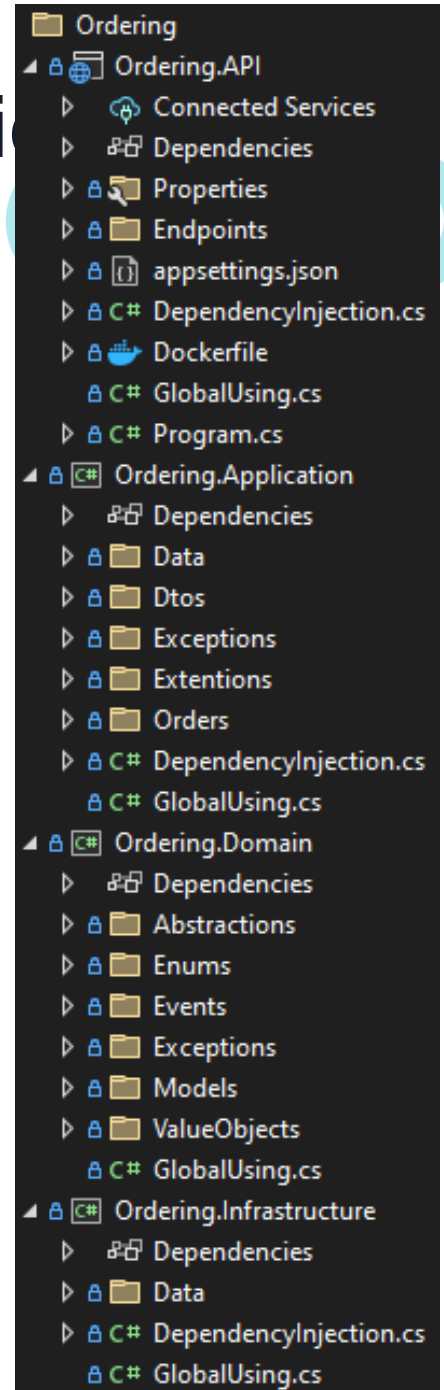
- CQRS commands and handlers organized by features.
- Dependency injection configurations.

Ordering. Infrastructure:

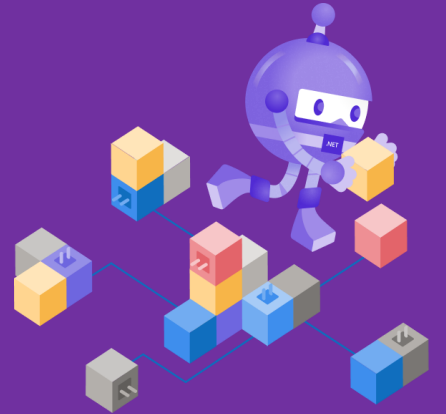
- Database context, configurations, migrations, and initial data seeding.

Ordering. API:

- API contracts and endpoint definitions.



Ordering.Domain Layer with Tactical Domain-Driven Design Patterns



Entities, Value Objects, Aggregates, Aggregate Roots Patterns

The Domain Entity Pattern

Develop Strongly typed IDs for Order Entities

Anemic-domain model vs Rich-domain model Entities in DDD

Domain Events in DDD and Domain vs Integration Events

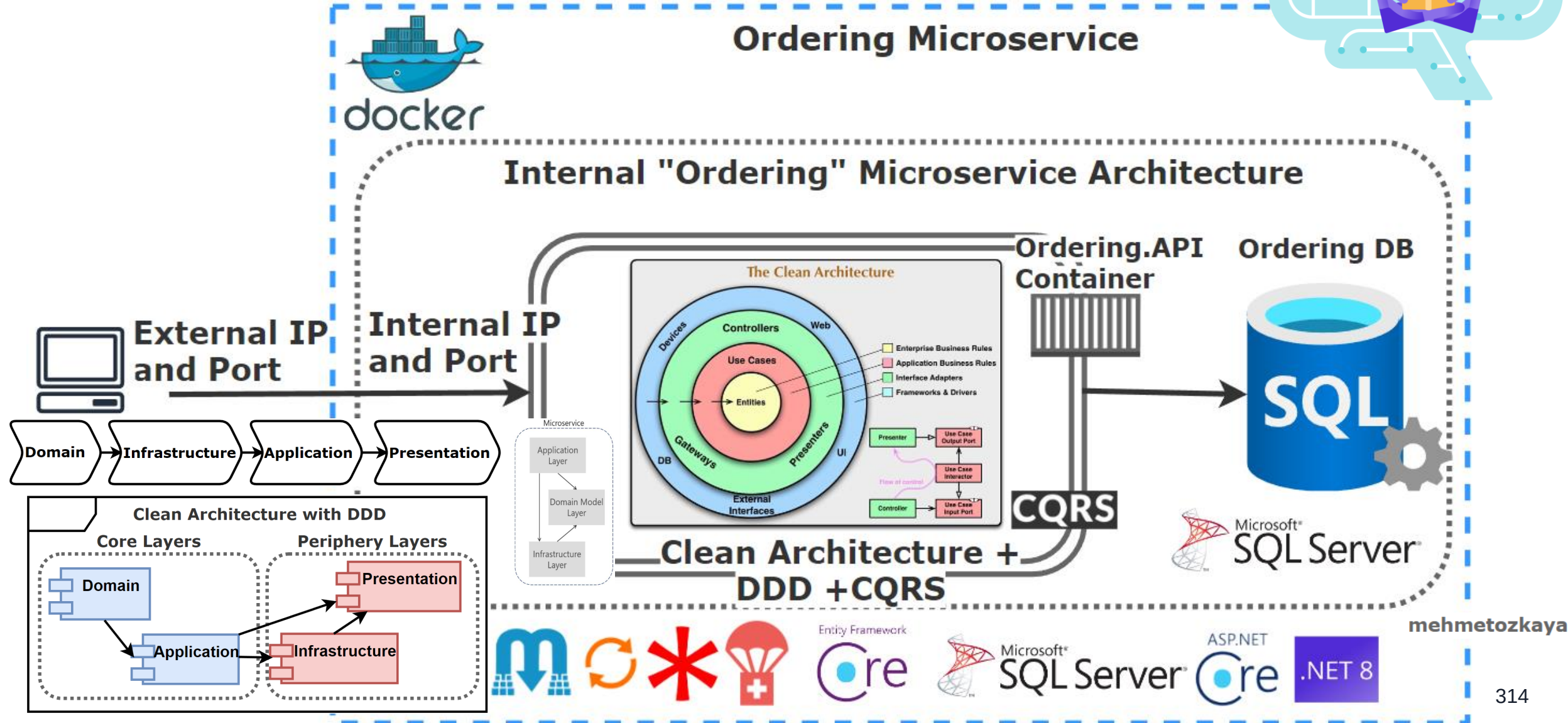
Mehmet Ozkaya



The diagram illustrates a microservices architecture for an e-commerce system. It is divided into three main sections: Client Apps, API Gateways, and Microservices.

- Client Apps:** Includes "Shopping.Web" and a mobile application icon, both utilizing ".NET 8".
- API Gateways:** Features the "Yarp API Gateway", which acts as the central point of entry for client requests.
- Microservices:** Consists of four core services:
 - Catalog:** Interacts with PostgreSQL.
 - Basket:** Interacts with PostgreSQL and Redis.
 - Discount:** Interacts with SQLite.
 - Ordering:** Interacts with SQL.
- Messaging:** RabbitMQ is used for asynchronous communication between the Basket and Ordering services.
- Additional Elements:**
 - A "Vertical Slice" diagram on the right shows layers: UI, Application, Domain, and Infrastructure.
 - Logos for various technologies are present at the bottom: ASP.NET Core, gRPC, Entity Framework, .NET 8, and others.
 - A "Mass Transit" bus icon is also visible.

Ordering Microservice w/ DDD, CQRS and Clean Architecture



DDD Types: Strategic and Tactical DDD

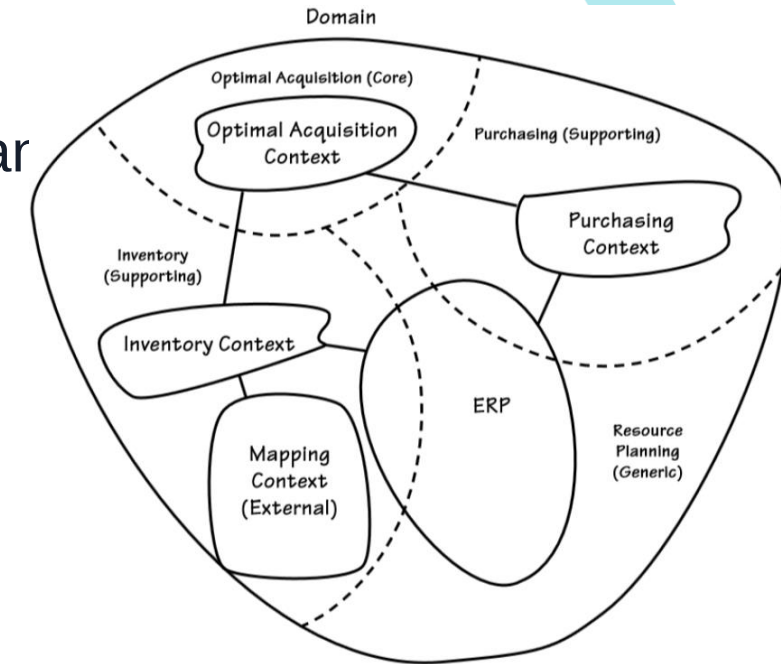


Strategic DDD:

- Understanding and modeling the business domain.
- It involves identifying the different domains, their subdomains, and how they interact with each other.

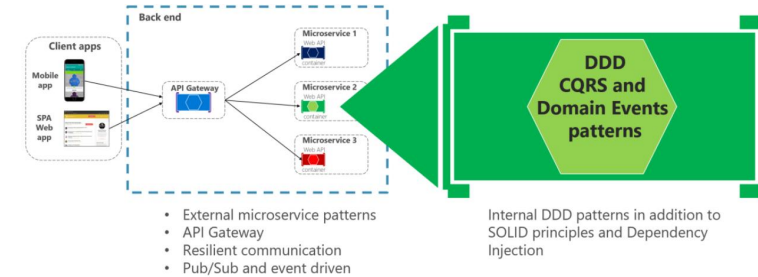
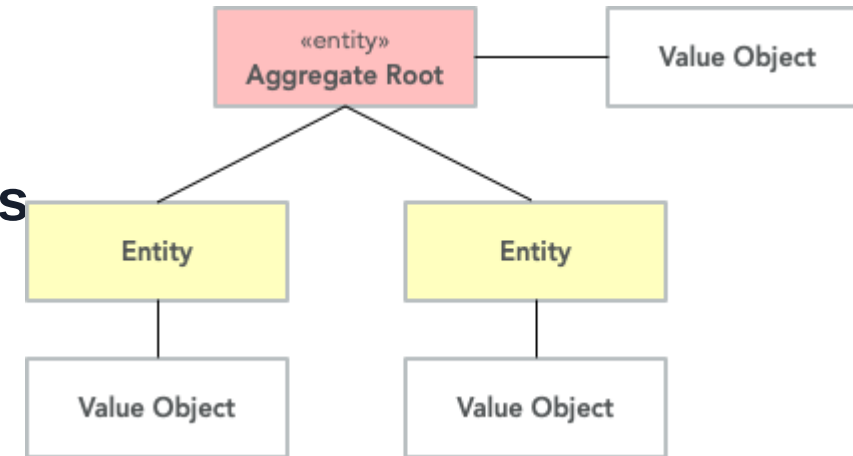
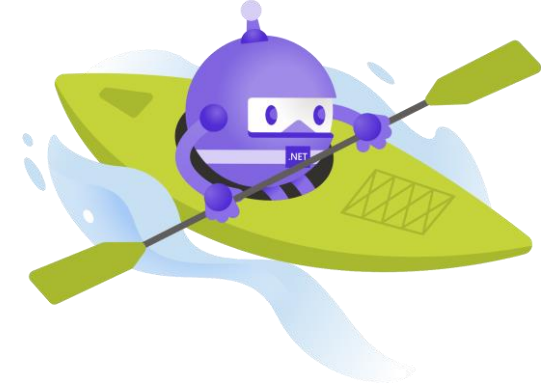
Tactical DDD:

- Implementation details and provides design patterns.
- Including patterns like Entities, Value Objects, Aggregates, etc.
- We'll focus on Strategic DDD and its key concepts.



Tactical Domain-Driven Design

- **Tactical DDD** is a set of patterns and principles that help in modeling complex software systems.
- Focuses on the **domain** and **domain logic**, importance of a model that reflects the business and its rules.
- **Entities, Value Objects, Aggregates, and Aggregate Roots**

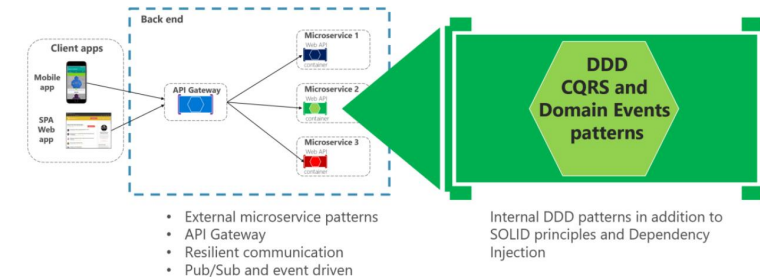
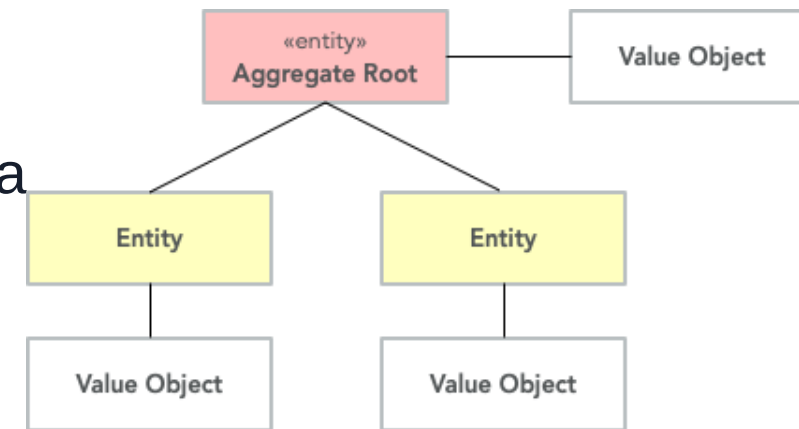
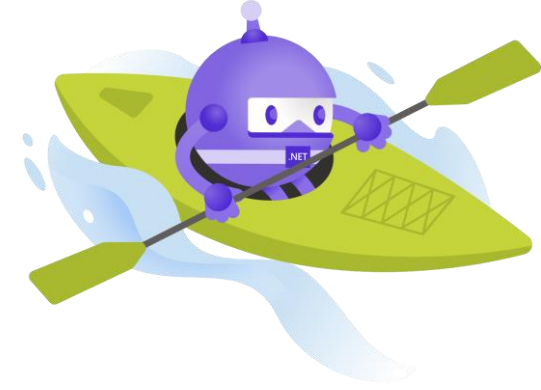


<https://learn.microsoft.com/en-us/dotnet/architecture/microservices>

Entities

- **Entity** is an object that is identified by its identity (ID), rather than its attributes.
- Identity **makes** each **entity unique**, even if other attributes are the same.
- **Entities** are used to represent objects in the system that have a **distinct identity** and lifecycle.
- **Example:** Ordering microservices, an Order can be an Entity.
- It is **uniquely identified** by an **OrderId**, even if other attributes (like date, total amount) are identical to another order.

```
public class Order : Entity
{
    0 references
    public OrderId Id { get; private set; }
    // Other properties and methods
}
```

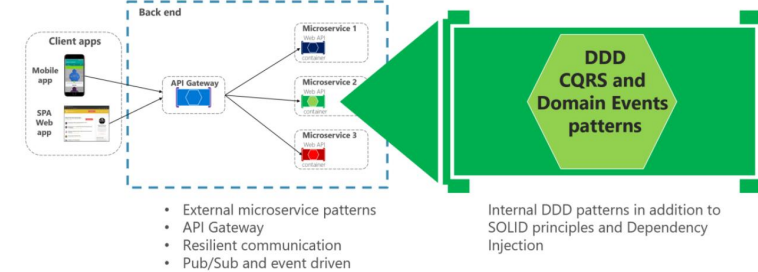
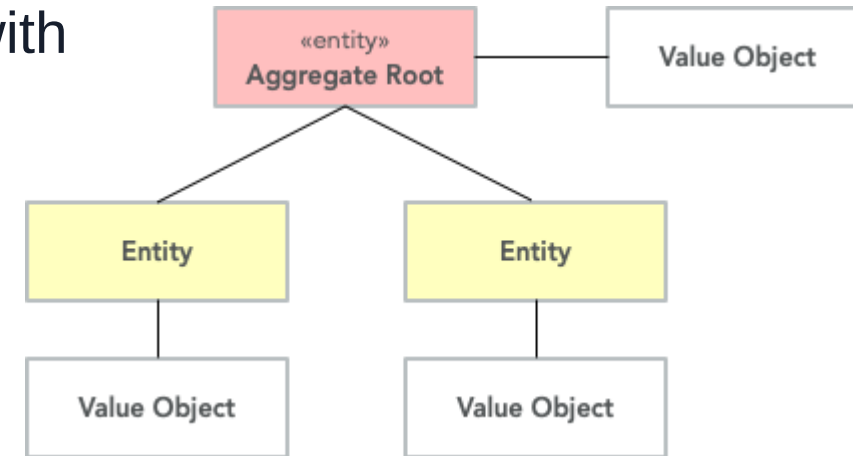
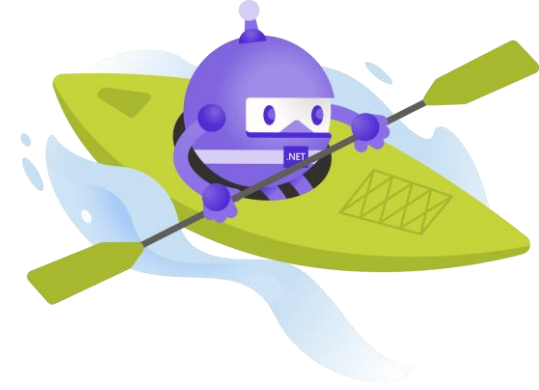


<https://learn.microsoft.com/en-us/dotnet/architecture/microservices>

Value Objects

- **Value Object** is an object that describes some characteristic or attribute but carries no concept of identity.
- **Value Objects** are used to describe aspects of the domain with **no conceptual identity**.
- They are **immutable** and are often used to **encapsulate complex attributes**.
- **Example:** An **Address** used in an Order might be a Value Object, as it is important for the order but does not define the order's identity.

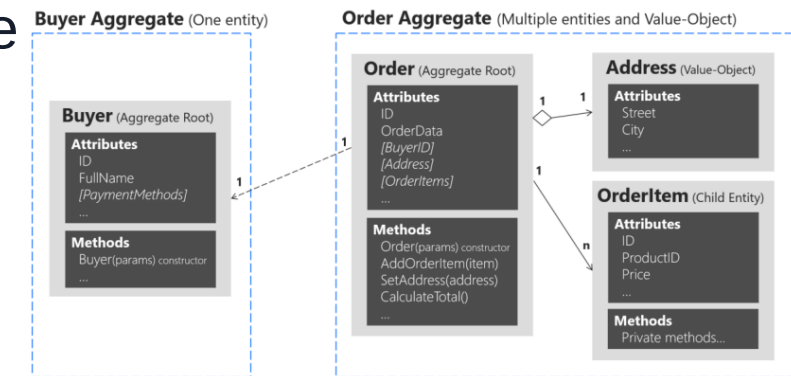
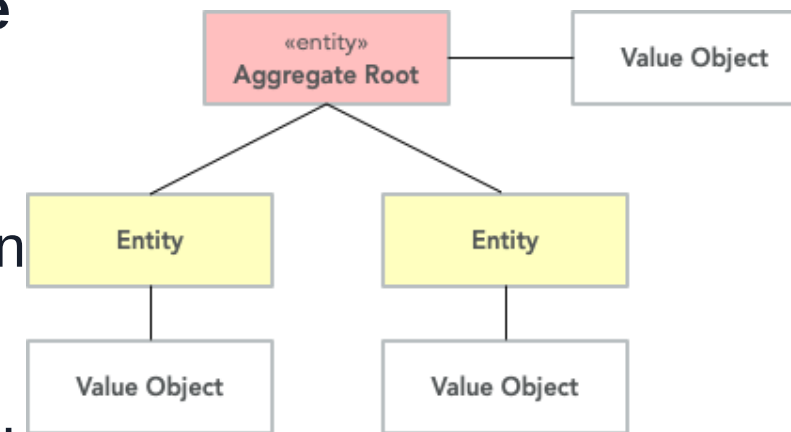
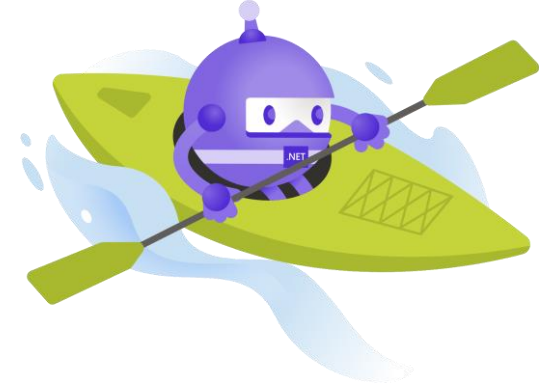
```
public class Address
{
    0 references
    public string Street { get; private set; }
    0 references
    public string City { get; private set; }
    // Other properties and constructor
}
```



<https://learn.microsoft.com/en-us/dotnet/architecture/microservices>

Aggregates

- **Aggregate** is a cluster of domain objects (Entities and Value Objects) that can be treated as a single unit.
- **Aggregate** will have one of its **components** be the **Aggregate Root**.
- Define a **boundary** around **related objects**. Operations within the boundary should maintain consistency of changes to data in an Aggregate.
- **Example: Order** can be an **Aggregate**, containing OrderItems, PaymentDetails, etc., within it. The consistency of an order (like total price calculation, stock validation) is maintained within the Aggregate boundaries.

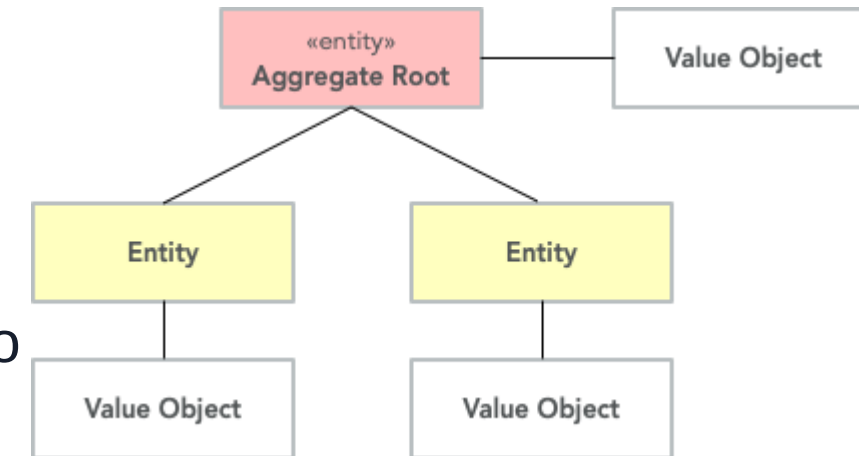
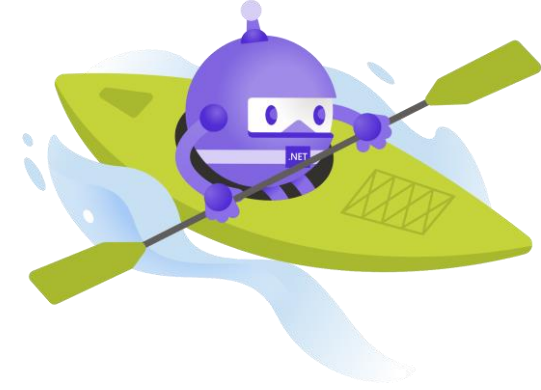


```
public class Order : Aggregate<OrderId>
{
    private readonly List<OrderItem> _orderItems = new();
    6 references
    public IReadOnlyList<OrderItem> OrderItems => _orderItems.AsReadOnly();
}
```

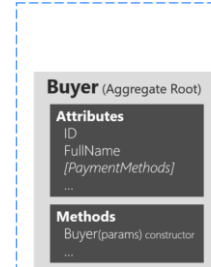
Aggregate Roots

- **Aggregate Root** is the main Entity in an Aggregate, through which external objects interact with the Aggregate.
- Provides a **gateway** to the **Aggregate**, ensuring that the **Aggregate's invariants** are enforced and consistency is maintained.
- **Example:** In the **Order Aggregate**, Order itself can be the Aggregate Root. External objects would interact with Order to affect changes within the Aggregate.

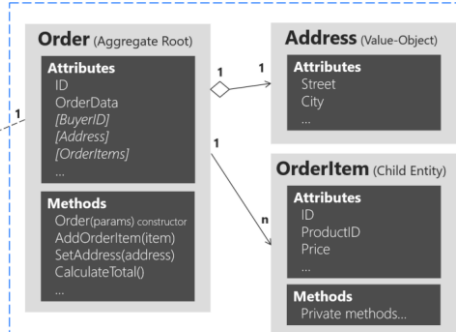
```
public class Order : Entity // Aggregate Root
{
    1 reference
    private List<OrderItem> _orderItems;
    0 references
    public IReadOnlyCollection<OrderItem> OrderItems => _orderItems.AsReadOnly();
    // Other properties and methods
}
```



Buyer Aggregate (One entity)

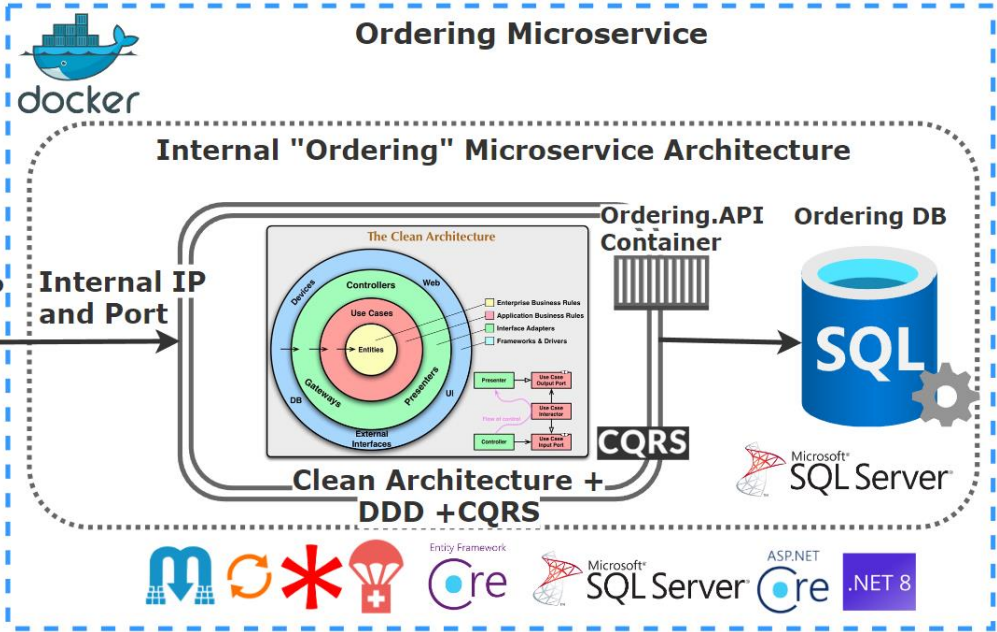
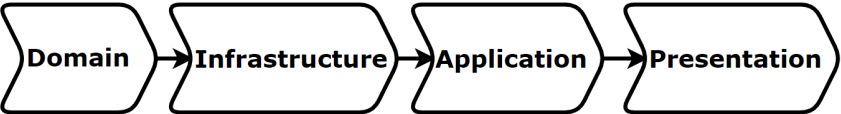
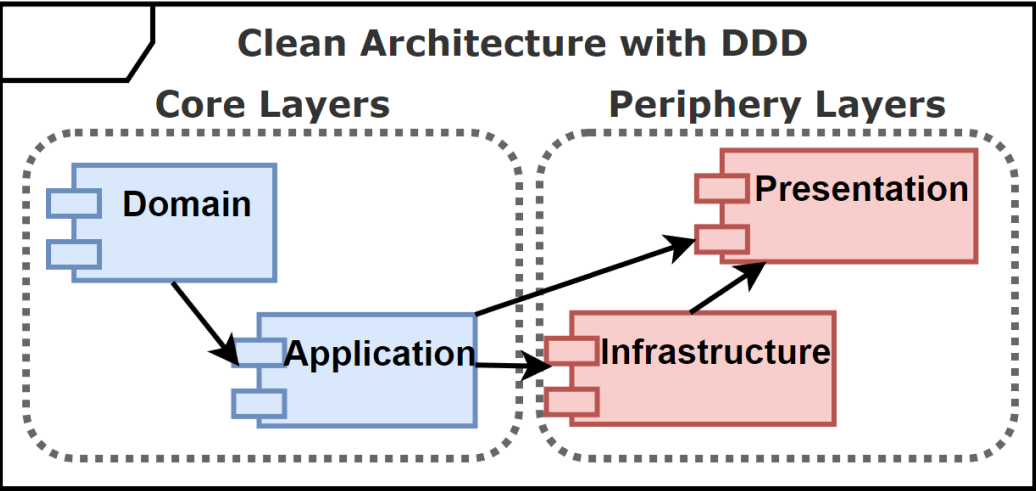
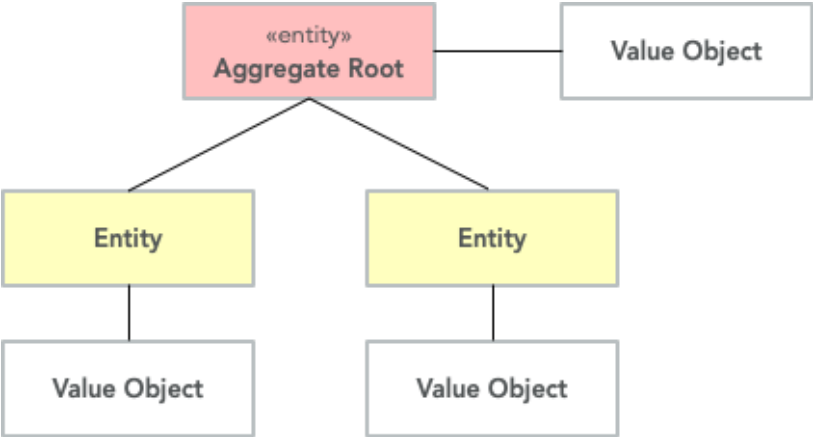
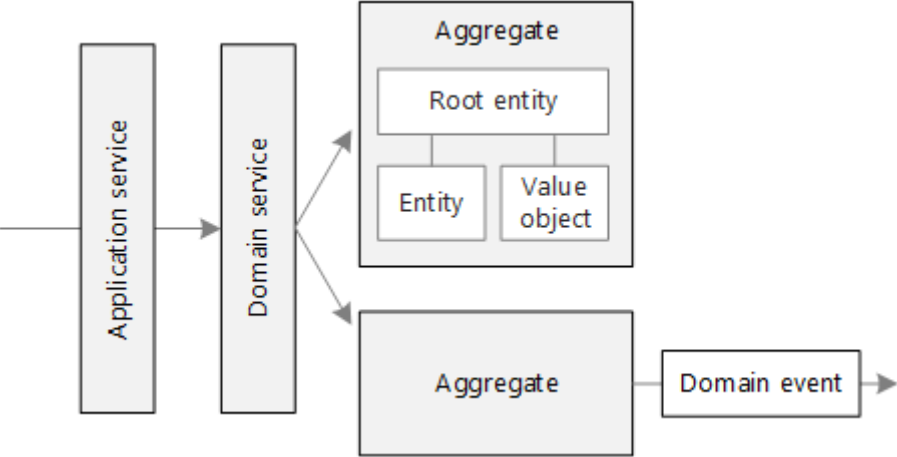
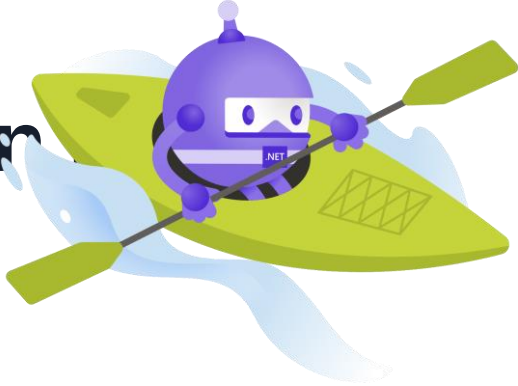


Order Aggregate (Multiple entities and Value-Object)



<https://learn.microsoft.com/en-us/dotnet/architecture/microservices>

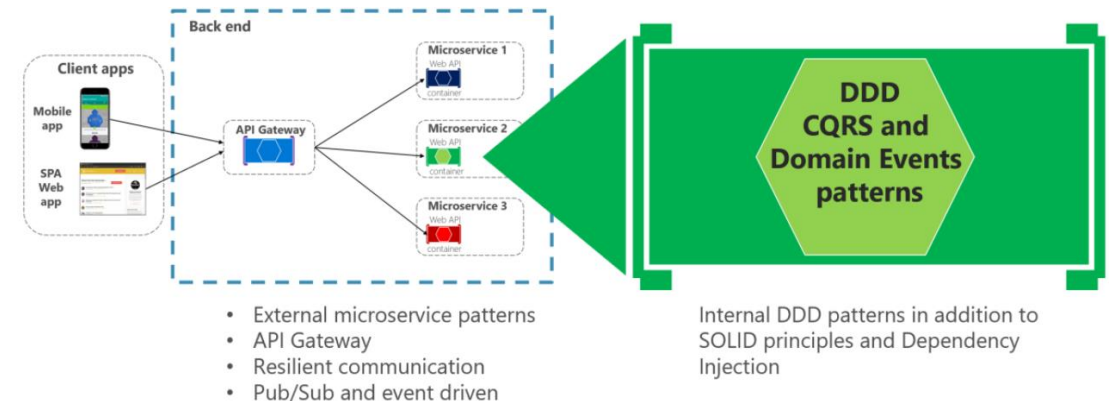
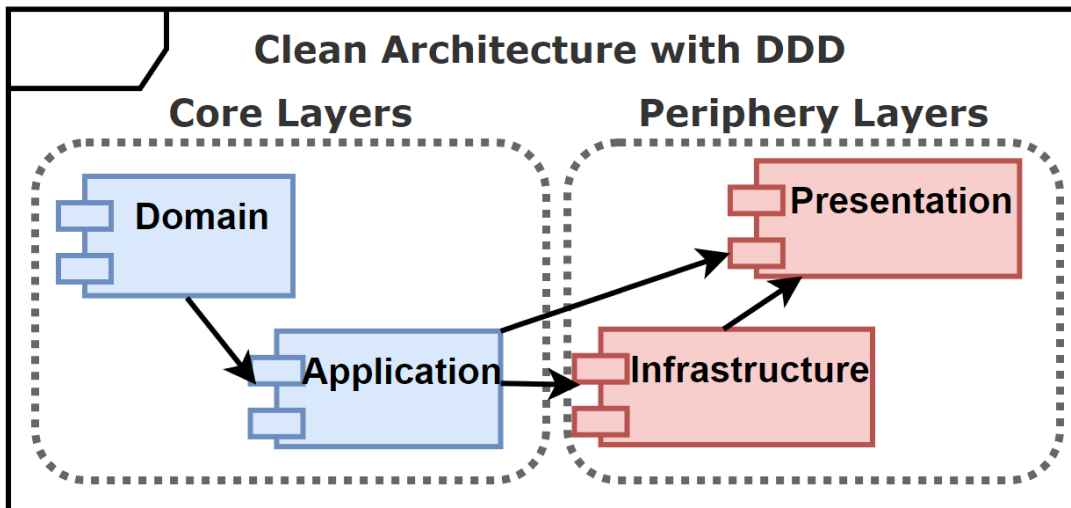
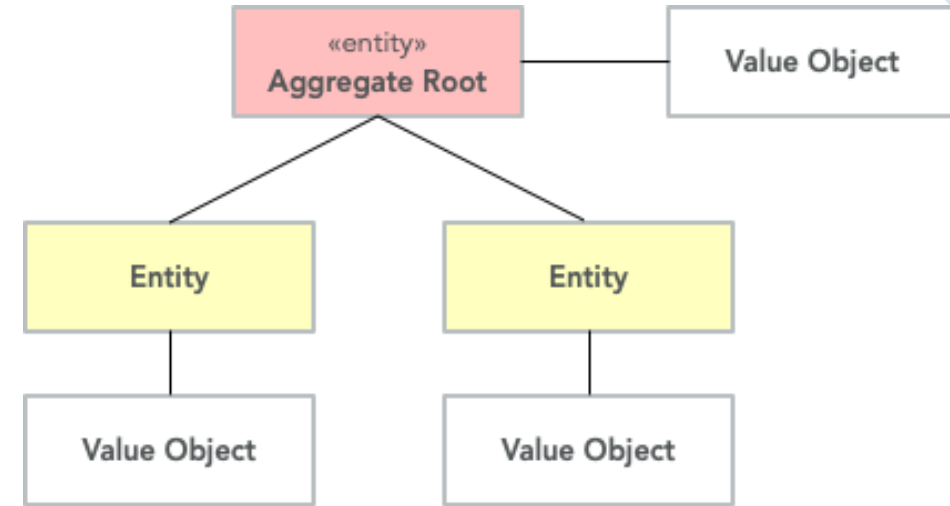
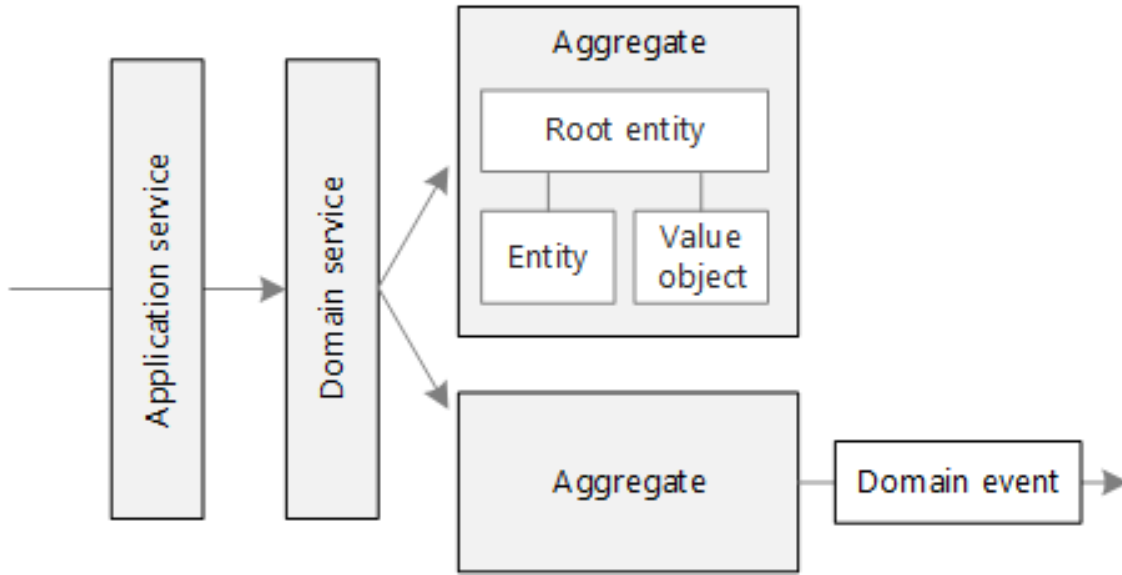
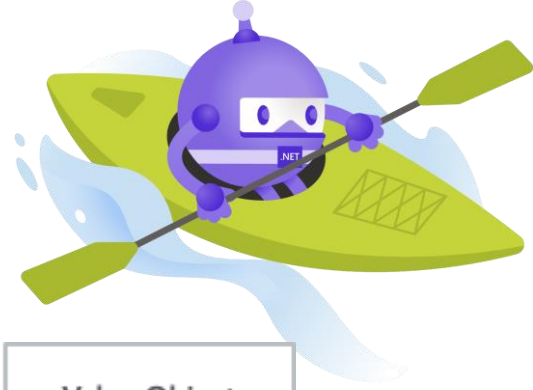
Applying These Concepts in the Ordering.Domain



DDD
CQRS
and
Domain Events
patterns

Internal DDD patterns in addition to
SOLID principles and Dependency
Injection

Tactical DDD Patterns in Ordering Microservices



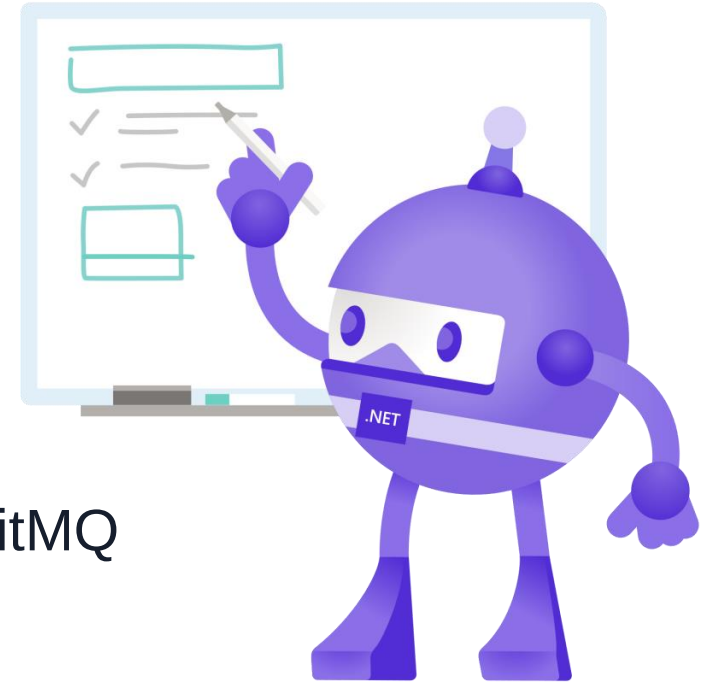
2. Application Use Cases of Ordering Microservices

CRUD Ordering Operations:

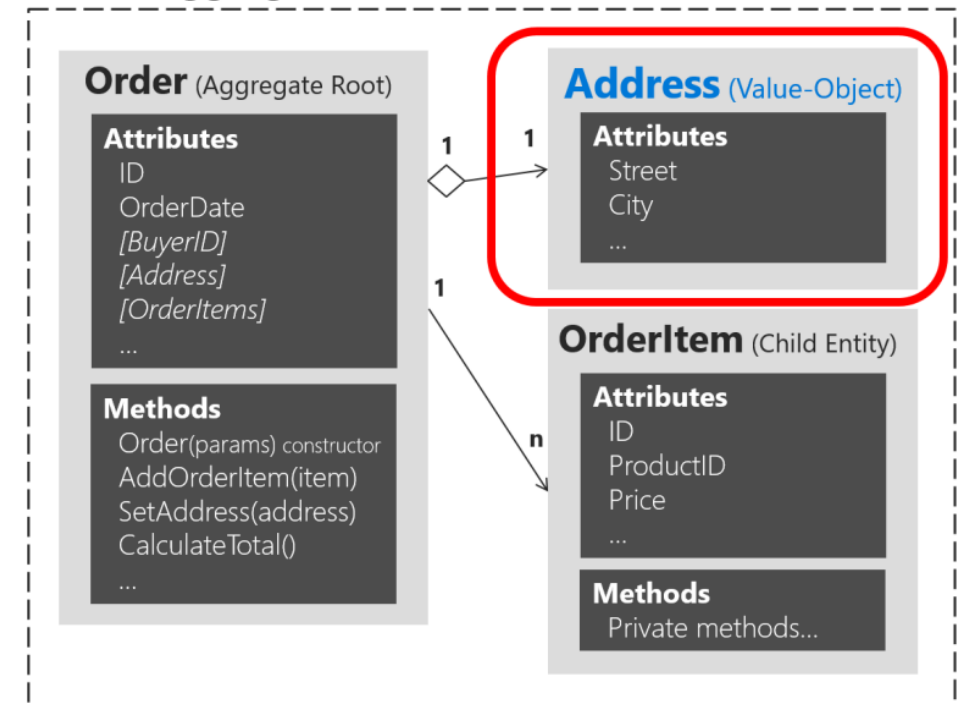
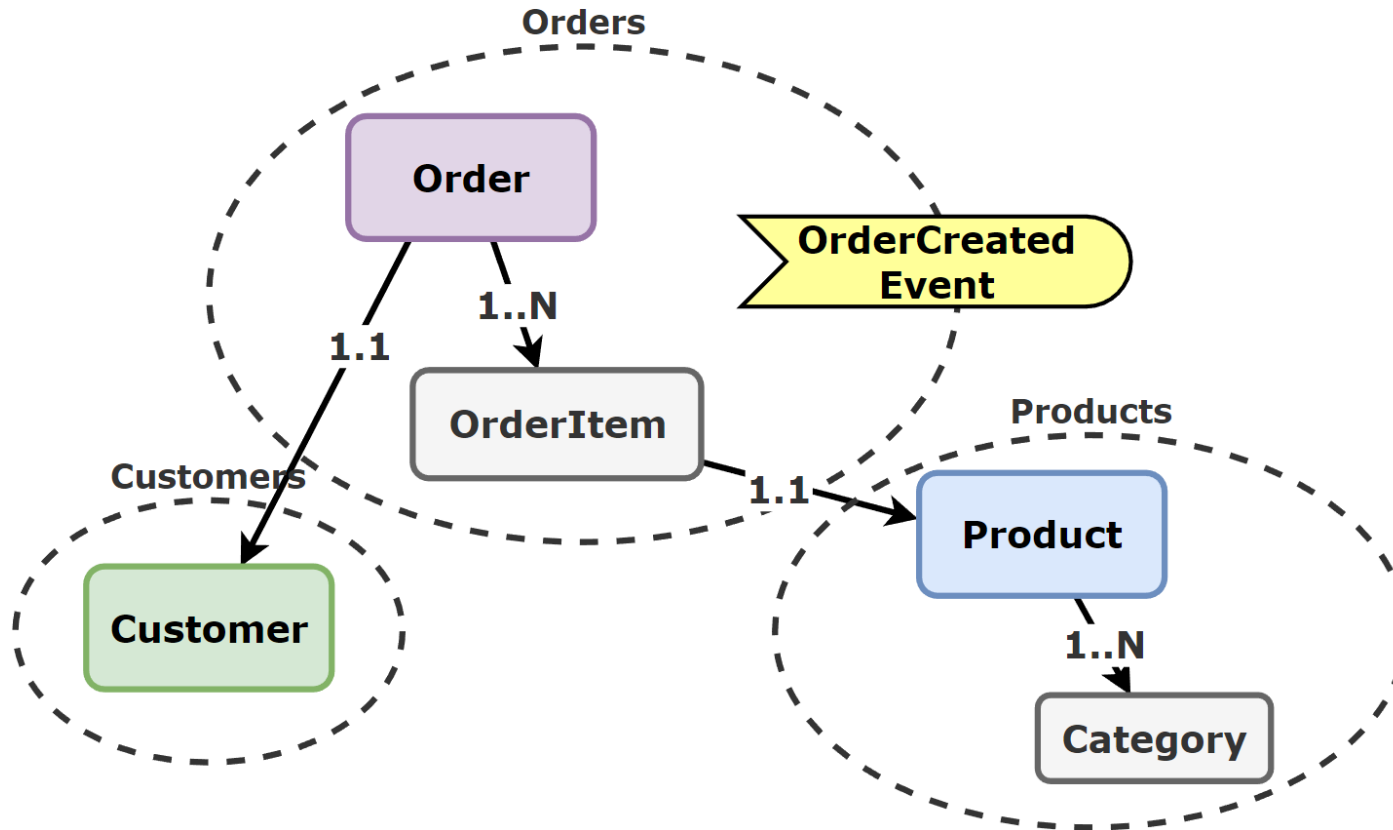
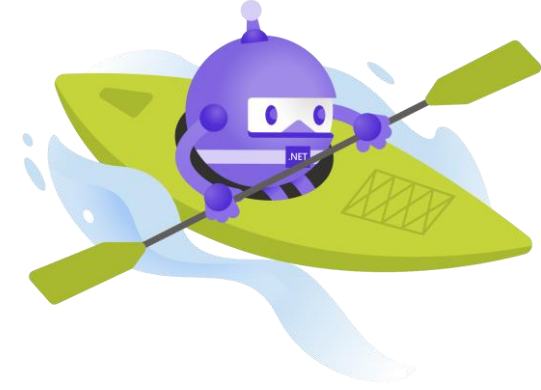
- Get Order with Items (can filter by Name and Customer)
- Create a new Order
- Update an existing Order
- Delete Order
- Add and Remove Item from Order

Asynchronous Ordering Operations:

- **Basket Checkout:** Consume Basket Checkout event from RabbitMQ using MassTransit
- **Order Fulfilment:** Perform order fulfilment operations (billing, shipment, notification..)
- Raise OrderCreated Domain Event that leads to integration event

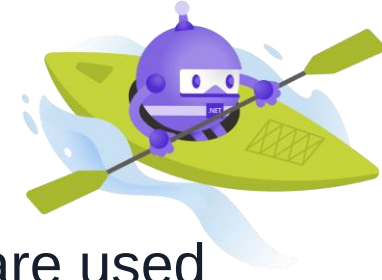


Domain Modeling for Ordering Microservices



<https://learn.microsoft.com/en-us/dotnet/architecture/microservices>

Primitive Obsession



- **Primitive Obsession** is a **code smell** where primitives (like string, int, Guid) are used for domain concepts, leading to ambiguity and errors.
- Using a **Guid** or **string** for an **orderId**, **customerId**, or **productId** makes it easy to mix these identifiers up, as they're all of the same type.
- How can we **solve** «**Primitive Obsession**» ?
- **Strongly Typed IDs**

```
public class CheckingAccount
{
    public int AccountNumber { get; set; }
    public string CustomerName { get; set; }
    public string Email { get; private set; }
    public string Address { get; set; }
    public int ZipCode { get; set; }
    public string City { get; set; }
    public string State { get; set; }
    public string Country { get; set; }
    public string SocialSecurityNumber { get; set; }
    public DateTime ActiveDate { get; set; }
    public string GetSSNLast4Digit()...
}
```

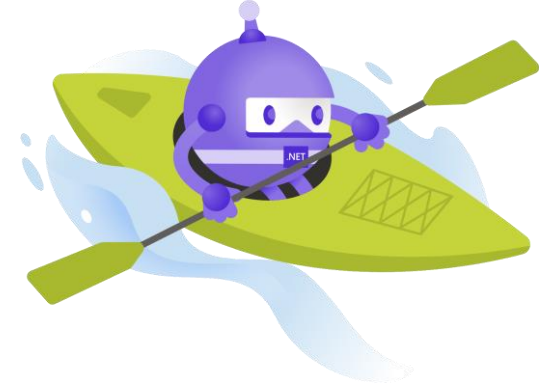


Adress/Locations

Personal Information
(PII)

Strong Typed IDs Pattern

- Creating **distinct types** for each kind of **ID** in your domain.
- This makes your code **more expressive** and **less error-prone**.
- It **clarifies which type of ID is expected** and prevents accidentally using one type of ID (like a productId) where another (like an orderId) is intended.



```
public class CheckingAccount
{
    public int AccountNumber { get; set; }
    public string CustomerName { get; set; }
    public string Email { get; private set; }
    public string Address { get; set; }
    public int ZipCode { get; set; }
    public string City { get; set; }
    public string State { get; set; }
    public string Country { get; set; }
    public string SocialSecurityNumber { get; set; }
    public DateTime ActiveDate { get; set; }
    public string GetSSNLast4Digit()...
```



Address/Locations

Personal Information
(PII)

```
public class CheckingAccount
{
    public int AccountNumber { get; set; }
    public string CustomerName { get; set; }
    public string Email { get; private set; }
    public Address Address { get; set; }
    public DateTime ActiveDate { get; set; }
    public SocialSecurity SocialSecurity { get; set; }
}
```



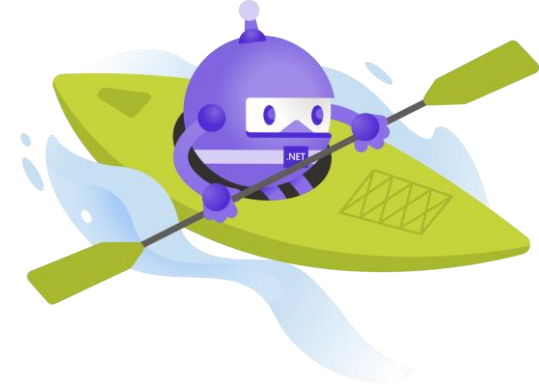
<https://blog.ndepend.com/code-smell-primitive-obsession-and-refactoring-recipes/>

Anemic-Domain Model Entity

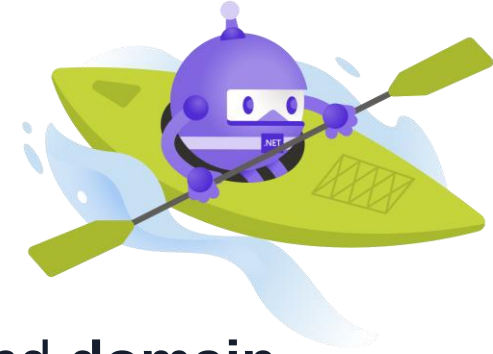
- **Entity** have **little** or **no business logic** (domain logic).
- Essentially **data structures** with **getters** and **setters**.
- But the **business rules** and **behaviors** are typically **implemented outside the entity**, often in service layers.

```
public class Order
{
    0 references
    public List<OrderItem> OrderItems { get; set; }
    // Other properties...
}
```

- **Order class** is **anemic** because it **only contains data** and lacks any domain logic or behaviors.



Rich-Domain Model Entity

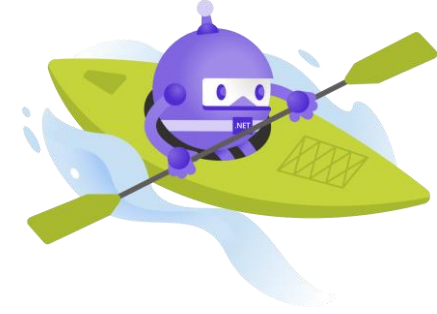


- Entities encapsulate both **data** and **behavior**.
- This model enriches entities with methods that **embody business rules** and **domain logic**.

```
public class Order : Aggregate<Guid>
{
    1 reference
    private readonly List<OrderItem> _orderItems = new();
    0 references
    public IReadOnlyList<OrderItem> OrderItems => _orderItems.AsReadOnly();
    0 references
    public void AddOrderItem(OrderItem item)
    {
        // Domain logic to add an item
    }
    0 references
    public void RemoveOrderItem(OrderItemId itemId)
    {
        // Domain logic to remove an item
    }
    // Other properties...
}
```

- **Order** is a **rich domain model** as it includes methods **AddOrderItem** and **RemoveOrderItem** which **encapsulate the business logic** for manipulating the order items.

Anemic-domain vs Rich-domain Model Entities

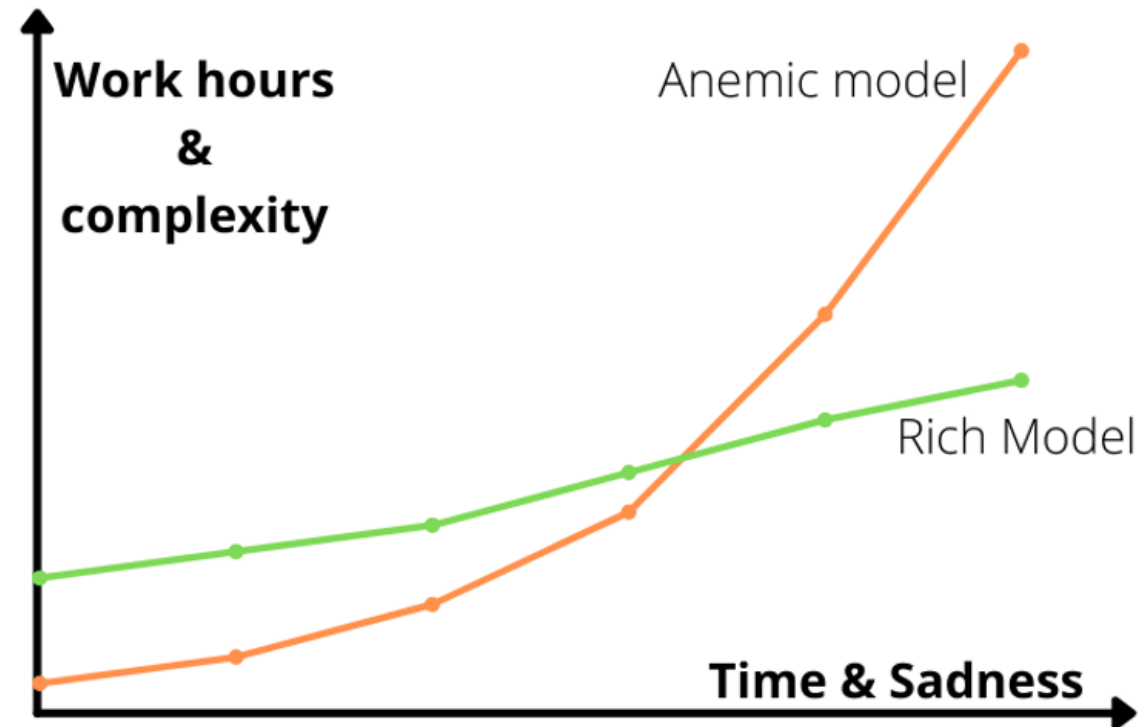


Anemic Domain Model:

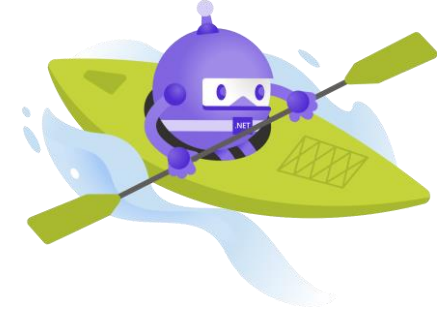
- Business logic is scattered across the application, often in services.
- Entities are simple data containers.
- Lead to issues with maintainability and understanding the business logic.

Rich Domain Model:

- Encapsulates business logic within the entities.
- More closely represents the real-world business domain.
- Can be more complex to design but leads to a more maintainable and cohesive model.



Order Example with Rich Domain Model

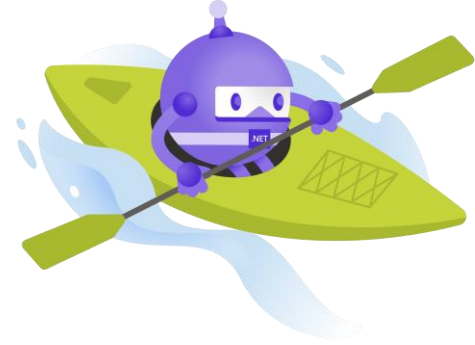


```
public class Order : Aggregate<OrderId>
{
    3 references
    private readonly List<OrderItem> _orderItems = new();

    // Encapsulated method to add an item
    0 references
    public void AddOrderItem(OrderItem item)
    {
        // Business logic for adding an item
        _orderItems.Add(item);
    }

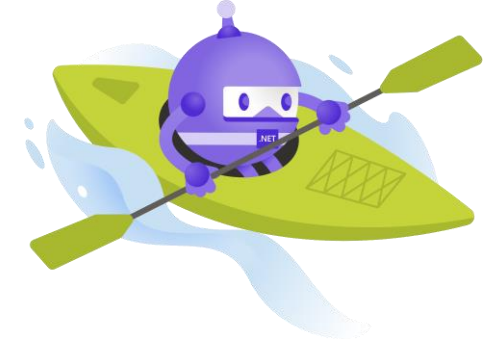
    // Encapsulated method to remove an item
    0 references
    public void RemoveOrderItem(OrderItemId itemId)
    {
        // Business logic for removing an item
        var item = _orderItems.FirstOrDefault(i => i.Id == itemId)
        if (item != null)
        {
            _orderItems.Remove(item);
        }
    }
}
```

Rich-domain model for Entities and ValueObjects

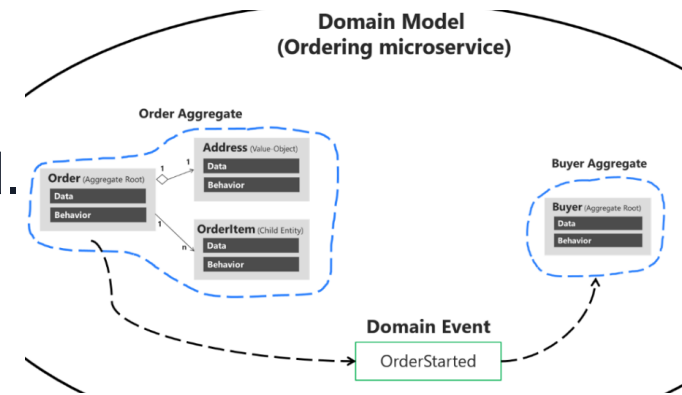


- **Entities** > **Create** Static Method
- **ValueObjects** > **Of** Static Method

Domain Event in DDD



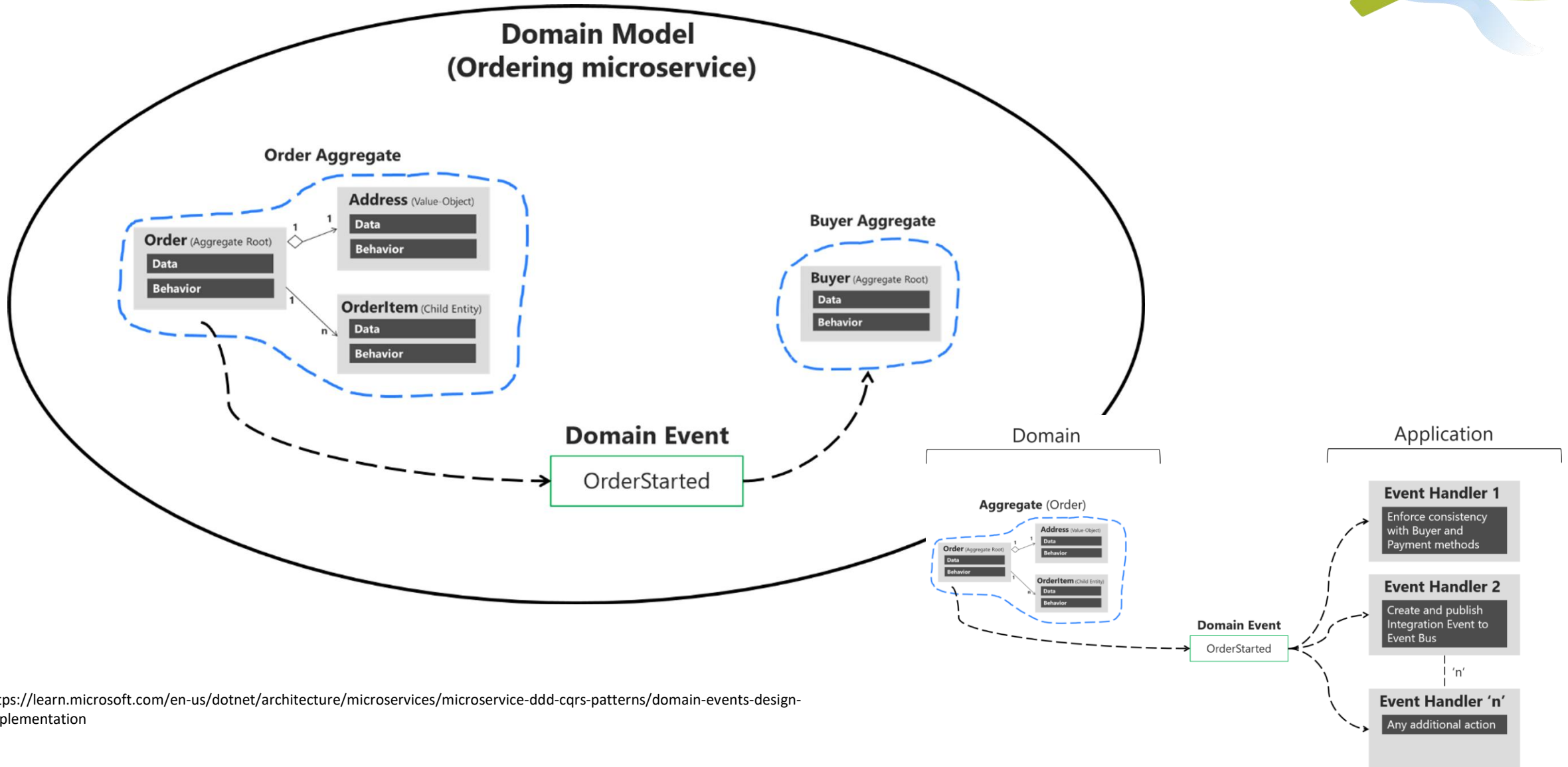
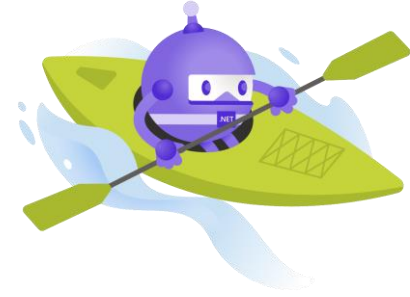
- **Domain Events** represent something that **happened** in the **past** and the other parts of the **same service boundary** same domain need to react to these changes.
- **Domain Event** is a **business event** that occurs within the **domain model**. It often represents a side effect of a domain operation.
- **Achieve consistency** between **aggregates** in the same domain.
- When an order is placed, an OrderPlaced event might be triggered.
- Trigger **side effects** or **notify other parts** of the system about changes within the domain.

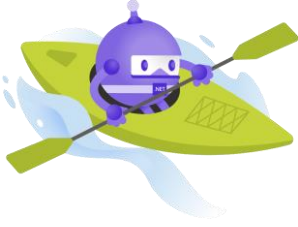


How to Use Domain Events in DDD?

- **Encapsulate** the event details and **dispatch** them to interested parties.
- **Communicate** changes **within the domain** to external handlers which may perform actions based on these events.

Eshop Domain Events in Ordering Microservices





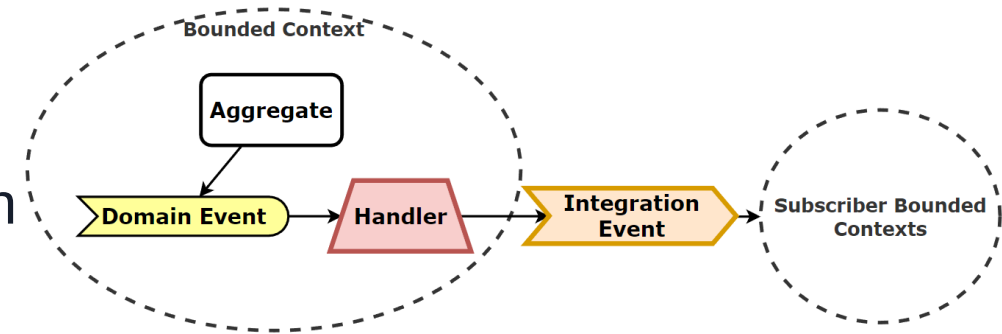
Domain vs Integration Events

Domain Events:

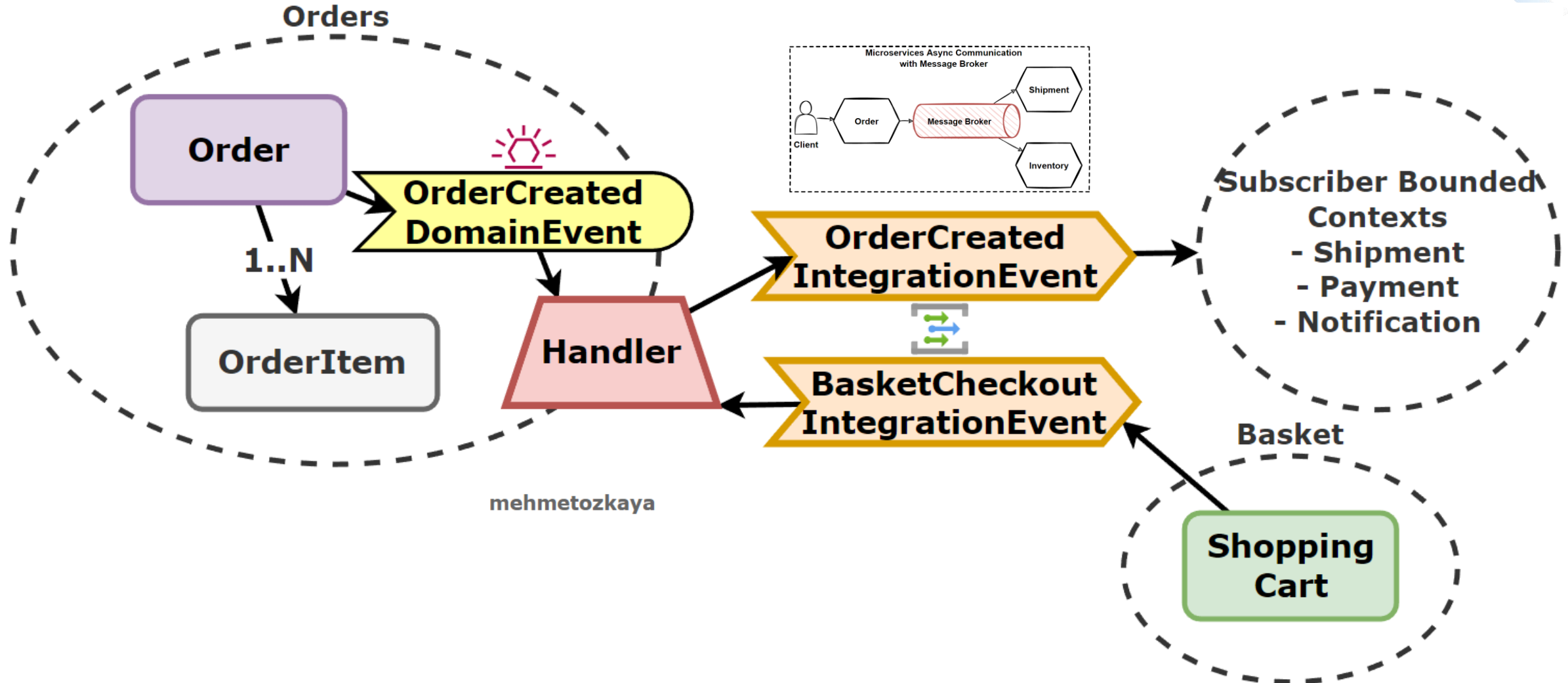
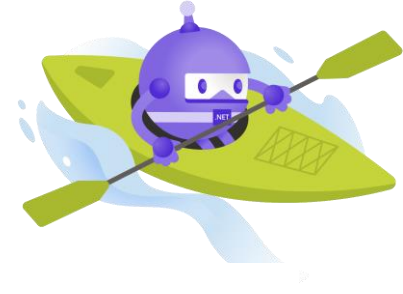
- Published and consumed within a single domain. Strictly within the boundary of the microservice/domain context.
- Indicate something that has happened within the aggregate.
- In-process and synchronously, sent using an in-memory message bus.
- **Example:** OrderPlacedEvent.

Integration Events:

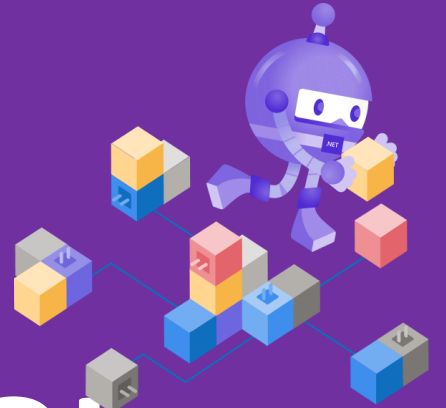
- Used to communicate state changes or events between contexts or microservices.
- Overall system's reaction to certain domain events.
- Asynchronously, sent with a message broker over a queue.
- **Example:** After handling OrderPlacedEvent, an OrderPlacedIntegrationEvent might be published to a message broker like RabbitMQ, then consumes by other microservices.



Eshop Domain and Integration Events



Ordering.Infrastructure Layer with EF Core Code First and SQL Server



EF Core 8 Features, Code First Approach, Migrations and SQL Server

EF Core DbContext Object for Storing Entities

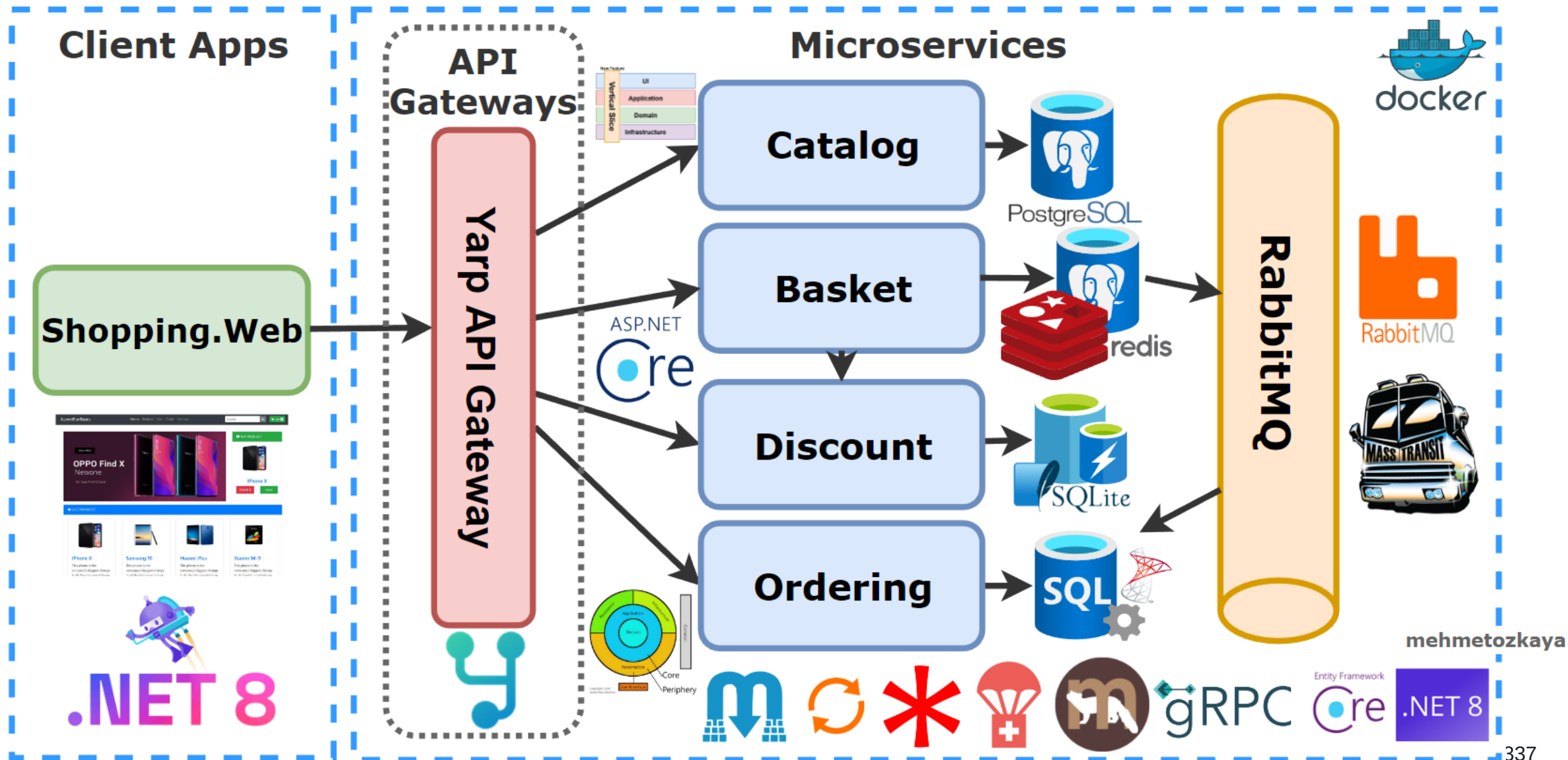
Mapping DDD Objects to EF Entities

Order Entity Configurations w/ ComplexTypes

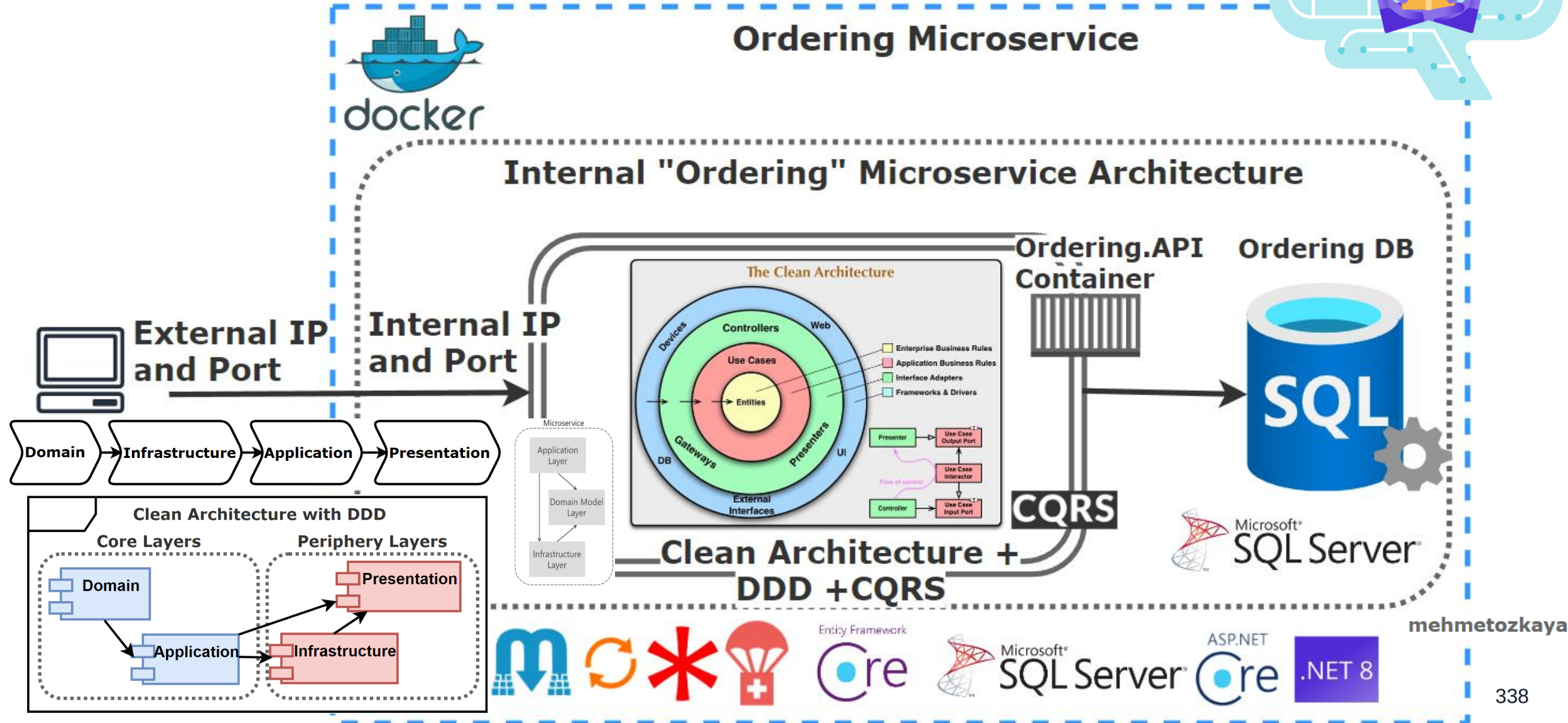
Auto Migrate and Seed EF Core Entities to SQL Server DB



Big Picture – Course Project



Ordering Microservice w/ DDD, CQRS and Clean Architecture



Ordering.Infrastructure Layer with EF Core



- EF Core 8 Features, Code First Approach, Migrations and SQL Server
- Develop EF Core DbContext Object for Storing Entities
- Mapping DDD Objects to EF Entities
- Develop Order Entity Configurations w/ modelBuilder and ComplexTypes in Ordering.Infrastructure
- Auto Migrate and Seed EF Core Entities to SQL Server DB
- Raise & Dispatch Domain Events with EF Core and MediatR

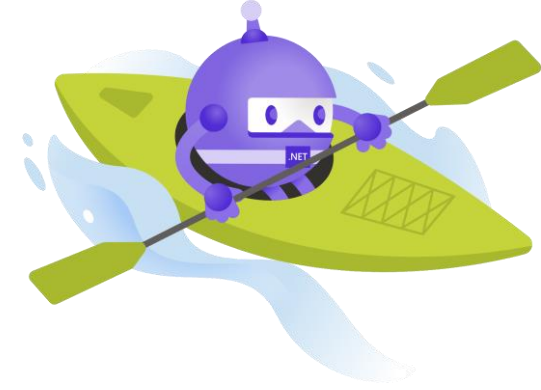
Main Target of Infra Layer:

- Mapping Domain Objects (Aggregate, Entity, ValueObjects) to EF Core Entities and Migrate tables into SQL Server Database.
- *Value Object Complex Types, EF Aggregate Root Entities

Entity Framework



Folder Structure of Infra Layer



```
└─ Ordering.Infrastructure
  └─ Dependencies
    └─ Data
      └─ Configurations
        ├── CustomerConfiguration.cs
        ├── OrderConfiguration.cs
        ├── OrderItemConfiguration.cs
        └── ProductConfiguration.cs
      └─ Extensions
        ├── DatabaseExtensions.cs
        └── InitialData.cs
      └─ Interceptors
        ├── AuditableEntityInterceptor.cs
        └── DispatchDomainEventsIntercep
      └─ Migrations
        ├── 20231127104453_InitialCreate.cs
        ├── ApplicationDbContextModelSn
        └── ApplicationDbContext.cs
    └─ DependencyInjection.cs
    └─ GlobalUsing.cs
```

Entity Framework Core



- **Entity Framework (EF) Core** is a lightweight, extensible, open source and cross-platform data access technology.
- **EF Core** can serve as an **object-relational mapper (O/RM)**
- Enables **.NET developers** to work with a **database** using **.NET objects**.
- **Eliminates** the need for most of the **data-access code** that typically needs to be written.
- EF Core **supports many database engines**.

Entity Framework



Entity Framework Core NuGet Packages for DB Providers

- EF Core supports **different database systems** through the use of "database providers".
- **Each system** has its **own database provider**, which is shipped as NuGet package.
- Applications should **install** one or more of these **provider package**

Database system	Package
SQL Server and SQL Azure	Microsoft.EntityFrameworkCore.SqlServer ↗
SQLite	Microsoft.EntityFrameworkCore.Sqlite ↗
Azure Cosmos DB	Microsoft.EntityFrameworkCore.Cosmos ↗
PostgreSQL	Npgsql.EntityFrameworkCore.PostgreSQL ↗ *
MySQL	Pomelo.EntityFrameworkCore.MySql ↗ *
EF Core in-memory database**	Microsoft.EntityFrameworkCore.InMemory ↗



EF Core Tools

- Tooling for EF Core migrations and reverse engineering (scaffolding) from an existing database has 2 way:

1- .NET Core CLI:

- **dotnet-ef** and **Microsoft.EntityFrameworkCore.Design** for cross-platform command line tooling
 - `dotnet tool install --global dotnet-ef`
 - `dotnet add package Microsoft.EntityFrameworkCore.Design`

2- Package Manager Console (PMC) in Visual Studio:

- **Microsoft.EntityFrameworkCore.Tools** for PowerShell tooling that works in the Visual Studio **Package Manager Console**
 - `Install-Package Microsoft.EntityFrameworkCore.Tools`



Getting Started with EF Core in Sample Console Application



Steps:

- Create new project
- Install Entity Framework Core
- Create the model entity classes and dbcontext
- Create the database
- Perform CRUD operations using EF Core and linq codes

Create new project

- `dotnet new console -o EFGetStarted`
- Select Console App with the C#

Entity Framework



EF Core in Sample Console Application

Install Entity Framework Core



- To install NuGet packages, you can use the .NET Core command-line interface (CLI), or the Visual Studio Package Manager Console.

1- .NET Core CLI:

- **dotnet-ef** and **Microsoft.EntityFrameworkCore.Design** for platform command line tooling
 - `dotnet tool install --global dotnet-ef`
 - `dotnet add package Microsoft.EntityFrameworkCore.Design`
 - `dotnet add package Microsoft.EntityFrameworkCore.SqlServer`

Entity Framework



2- Package Manager Console (PMC) in Visual Studio:

- **Microsoft.EntityFrameworkCore.Tools** for PowerShell tooling that works in the Visual Studio **Package Manager Console**
 - `Install-Package Microsoft.EntityFrameworkCore.Tools`
 - `Install-Package Microsoft.EntityFrameworkCore.SqlServer`

EF Core in Sample Console Application

Create the model Entity classes and DbContext



```
public class Blog
{
    0 references
    public int BlogId { get; set; }
    0 references
    public string Url { get; set; }

    0 references
    public List<Post> Posts { get; } = new();
}

2 references
public class Post
{
    0 references
    public int PostId { get; set; }
    0 references
    public string Title { get; set; }
    0 references
    public string Content { get; set; }
```

```
public class BloggingContext : DbContext
{
    0 references
    public DbSet<Blog> Blogs { get; set; }
    0 references
    public DbSet<Post> Posts { get; set; }

    2 references
    public string DbPath { get; }

    0 references
    public BloggingContext()
```

Entity Framework



EF Core in Sample Console Application

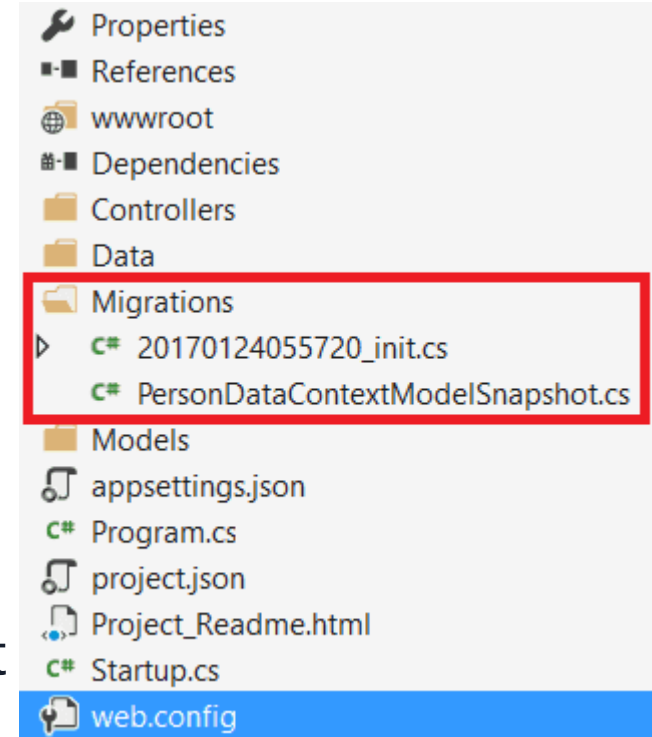
Create the Database

1- .NET Core CLI:

- **dotnet-ef** and **Microsoft.EntityFrameworkCore.Design** for cross-platform command line tooling
 - `dotnet tool install --global dotnet-ef`
 - `dotnet add package Microsoft.EntityFrameworkCore.Design`
 - `dotnet ef migrations add InitialCreate`
 - `dotnet ef database update`

2- Package Manager Console (PMC) in Visual Studio:

- **Microsoft.EntityFrameworkCore.Tools** for PowerShell tooling that works in the Visual Studio **Package Manager Console**
 - **Install-Package Microsoft.EntityFrameworkCore.Tools**
 - **Add-Migration InitialCreate**
 - **Update-Database**



EF Core in Sample Console Application

Perform CRUD operations



```
using var db = new BloggingContext();

// Create
db.Add(new Blog { Url = "http://blogs.msdn.com/adonet" });
db.SaveChanges();

// Read
var blog = db.Blogs
    .OrderBy(b => b.BlogId)
    .First();

// Update
blog.Url = "https://devblogs.microsoft.com/dotnet";
blog.Posts.Add(
    new Post { Title = "Hello World", Content = "I wrote an app using EF Core!" });
db.SaveChanges();

// Delete
db.Remove(blog);
db.SaveChanges();
```

Entity Framework



EF Core in Asp.Net: appsettings.json and DbContext



```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "SchoolContext": "Server=(localdb)\\mssqllocaldb;Database=SchoolContext"
  }
}
```

```
public class SchoolContext : DbContext
{
    0 references
    public SchoolContext (DbContextOptions<SchoolContext> options)
        : base(options)
    {
    }

    0 references
    public DbSet<Student> Students { get; set; }
    0 references
    public DbSet<Enrollment> Enrollments { get; set; }
    0 references
    public DbSet<Course> Courses { get; set; }
```

EF Core in Asp.Net: Program.cs



```
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddRazorPages();

builder.Services.AddDbContext<SchoolContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("SchoolContext")));
```

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "SchoolContext": "Server=(localdb)\\mssqllocaldb;Database=SchoolContext
  }
}
```

Entity Framework



EF Core in Asp.Net: Auto Migrate and Seed Database



```
using (var scope = app.Services.CreateScope())
{
    var services = scope.ServiceProvider;

    var context = services.GetRequiredService<SchoolContext>();
    context.Database.EnsureCreated();
    DbInitializer.Initialize(context);
}
```

```
static class DbInitializer
```

```
0 references
```

```
public static void Initialize(SchoolContext context)
```

```
{
```

```
    // Look for any students.
```

```
    if (context.Students.Any())
```

```
    {
```

```
        return; // DB has been seeded
```

```
    }
```

```
    var students = new Student[]
```

```
    {
```

```
        new Student{FirstMidName="Carson", LastName="Alexander",
```

```
        new Student{FirstMidName="Meredith", LastName="Alonso",
```

```
    };
```

```
    context.Students.AddRange(students);
```

```
    context.SaveChanges();
```

Entity Framework



EF Core Developments Steps for Ordering.Infrastructure



1- Install EF Core Nuget Packages:

- Microsoft.EntityFrameworkCore.Tools for PowerShell tooling that works in the Visual Studio Package Manager Console
 - Install-Package Microsoft.EntityFrameworkCore.Tools
 - Install-Package Microsoft.EntityFrameworkCore.SqlServer

2- Develop DbContext Object:

- Adding Entity Objects into Ef Core DbContext object

```
public class ApplicationDbContext : DbContext, IApplicationDbContext
{
    0 references
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options) : base(options)
    {
        3 references
        public DbSet<Customer> Customers => Set<Customer>();
        3 references
        public DbSet<Product> Products => Set<Product>();
        12 references
        public DbSet<Order> Orders => Set<Order>();
        1 reference
        public DbSet<OrderItem> OrderItems => Set<OrderItem>();
    }
}
```

Entity Framework



EF Core Developments Steps for Ordering.Infrastructure - continue

3- Configure Domain Entities to EF Core Entities for SQL Server:

- Using Ef Core modelBuilder for mapping DDD Entities to EF Core Entities

```
public class OrderConfiguration : IEntityTypeConfiguration<Order>
{
    0 references
    public void Configure(EntityTypeBuilder<Order> builder)
    {
        builder.HasKey(o => o.Id);

        builder.Property(o => o.Id).HasConversion(
            orderId => orderId.Value,
            dbId => OrderId.Of(dbId));
    }
}
```

4- Asp.net Core Setup & Configurations:

- appsettings.json Database connection string

```
"ConnectionStrings": {
  "Database": "Server=localhost;Database=OrderDb;User Id=sa;Password=SwN12345678;"
},
```



Entity Framework



EF Core Developments Steps for Ordering.Infrastructure - continue



5- Register EF Core DbContext to Asp.Net:

- Register into DependencyInjection.cs

```
services.AddDbContext<ApplicationDbContext>((sp, options) =>
{
    options.AddInterceptors(sp.GetServices<ISaveChangesInterceptor>());
    options.UseSqlServer(connectionString);
});
```

6- Create Migrations and create Database w/ Commands:

- Install-Package Microsoft.EntityFrameworkCore.Tools
- Add-Migration InitialCreate
- Update-Database

```
└─ Migrations
  └─ C# 20231127104453_InitialCreate.cs
  └─ C# ApplicationDbContextModelSn
```

Entity Framework



EF Core Developments Steps for Ordering.Infrastructure - continue



7- Create Database w/ Migrate method:

- Register into DependencyInjection.cs

```
using var scope = app.Services.CreateScope();

var context = scope.ServiceProvider.GetRequiredService<ApplicationDbContext>();

context.Database.MigrateAsync().GetAwaiter().GetResult();

await SeedAsync(context);
```

8- Seed Database:

- Create Data/DbInitializer.cs

```
if (!await context.Orders.AnyAsync())
{
    await context.Orders.AddRangeAsync(InitialData.OrdersWithItems);
    await context.SaveChangesAsync();
}
```

Entity Framework



EF Core Developments Steps for Ordering.Infrastructure - continue



9- Dispatch DomainEvents w/ EF Core Interceptor:

- Use EF Core SaveChangesInterceptor

```
internal class DispatchDomainEventsInterceptor(IMediator mediator) : SaveChangesIntercept
{
    0 references
    public override InterceptionResult<int> SavingChanges(DbContextEventData eventData, I
    {
        DispatchDomainEvents(eventData.Context).GetAwaiter().GetResult();

        return base.SavingChanges(eventData, result);
    }
}
```

10- Test all from local w/ containerize SQL Server in Docker:

- Docker SQL Server Setup - Backing Services

```
if (!await context.Orders.AnyAsync())
{
    await context.Orders.AddRangeAsync(InitialData.OrdersWithItems);
    await context.SaveChangesAsync();
}
```

Entity Framework



EF Core 8 Relations w/ Has/With Pattern

- **Relationships** with other entities **one-to-one**, **one-to-many**, or **many-to-many**.
- EF Core handles these relationships using the **Has/With** pattern.
- **Relationships** can be configured using **HasMany/WithOne** methods for one-to-many and **HasOne/WithMany** for many-to-one relationships.

One-to-Many Relationships:

- **Order** and **OrderItem** have a **one-to-many relationship** (an order can have multiple items) configured using EF Core Fluent API as follows:

```
builder.Entity<Order>()  
    .HasMany(o => o.OrderItems)  
    .WithOne()  
    .HasForeignKey(oi => oi.OrderId);
```

- Each **OrderItem** references an **Order** through a **foreign key OrderId**.



Entity Framework



ValueObject Mapping with ComplexType and ComplexProperty



- EF Core 8, "**Complex Types**" are introduced to better support **value objects** in **DDD**.
- **Complex type** is an object that does not have a primary key and is used to represent a set of properties in an entity.

Example of Complex Types (Value Objects):

- **Address** can be a **complex type** representing the shipping and billing addresses for an order. And Configuring **Complex Types** in **OnModelCreating**:

Entity Framework

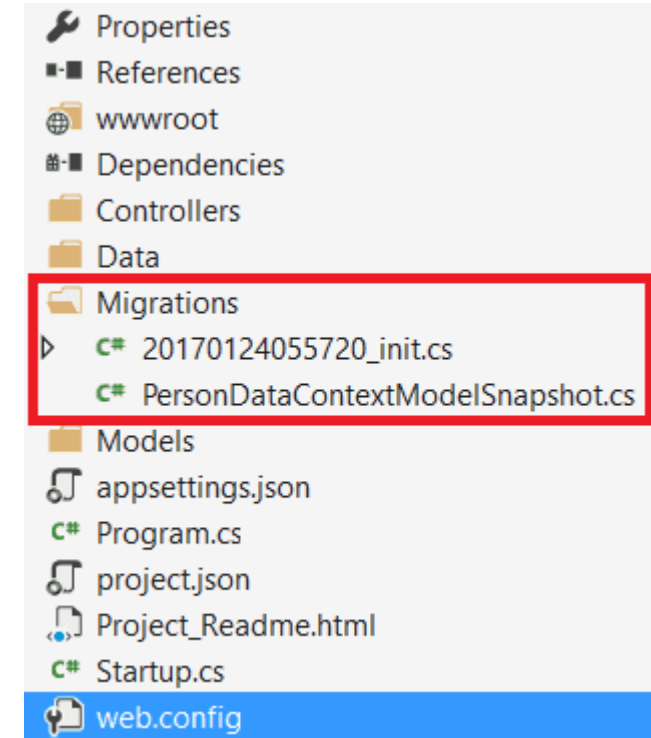


```
[ComplexType]
0 references
public class Address
{
    0 references
    public required string Line1 { get; set; }
    0 references
    public string? Line2 { get; set; }
    0 references
    public required string City { get; set; }
    0 references
    public required string Country { get; set; }
    0 references
    public required string PostCode { get; set; }
}
```

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Order>(b =>
    {
        b.ComplexProperty(e => e.BillingAddress);
        b.ComplexProperty(e => e.ShippingAddress);
    });
}
```

What is EF Core Migrations ?

- **Entity Framework Core Migrations** are a powerful feature for managing **database schema changes** in an application's lifecycle.
- Enable a developer to **evolve** a **database schema** in a controlled and **consistent way**, in line with the domain model changes.
- **Data models change** as features get implemented: **new entities** or properties are **added** and **removed**, and database schemas need to be changed accordingly.
- **Migrations** provide a way to **incrementally update** the **database** schema so that it stays in sync with the application's data model.
- Useful in **team environments** and **continuous deployment** scenarios where database schema **changes** are **frequent** and need to be applied systematically.



Entity Framework



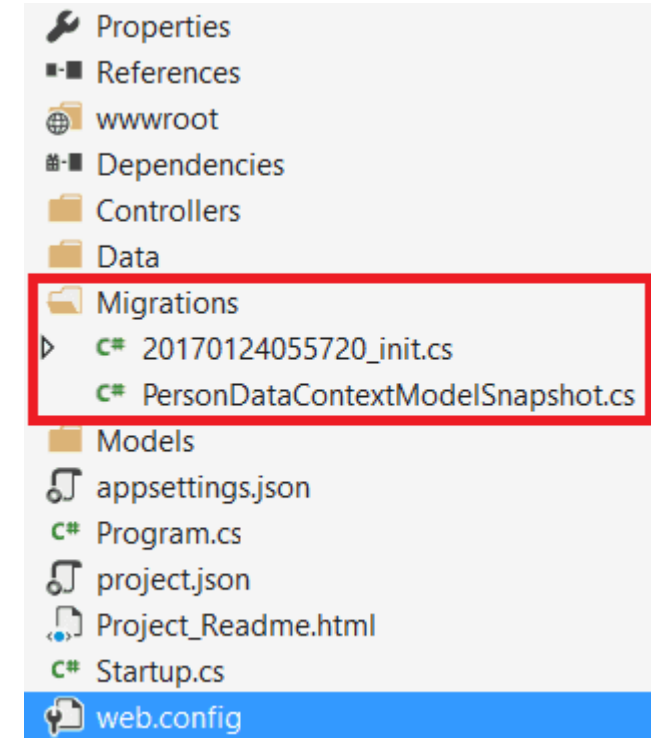
How EF Core Migrations works ?

1- Model Change Occurs:

- When a data model change occurs, developers use EF Core tools to create a corresponding migration that describes the necessary database updates.
- EF Core compares the current model against a snapshot of the previous model to find differences and generates migration source files.

2- Migration classes generated:

- Once a new migration has been generated, it can be applied to a database in various ways.
- EF Core records all applied migrations in a special history table, allowing it to know which migrations have been applied and which haven't.

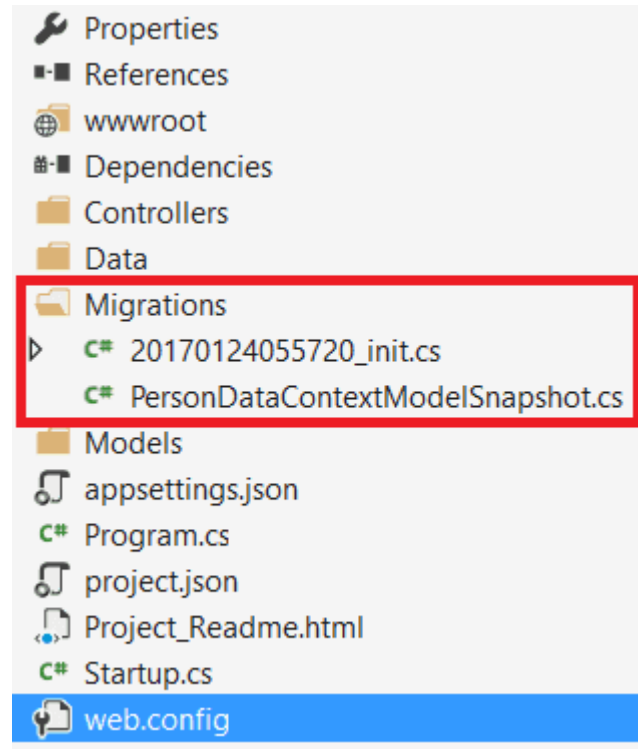


Entity Framework



EF Core Code-First Approach

- **Database creation** and **schema updates** are driven by the entity classes and **DbContext configuration** in the application.
- First, we should **develop Entity classes** and **DbContexts** after that run **EF Migration Commands**.



Entity Framework

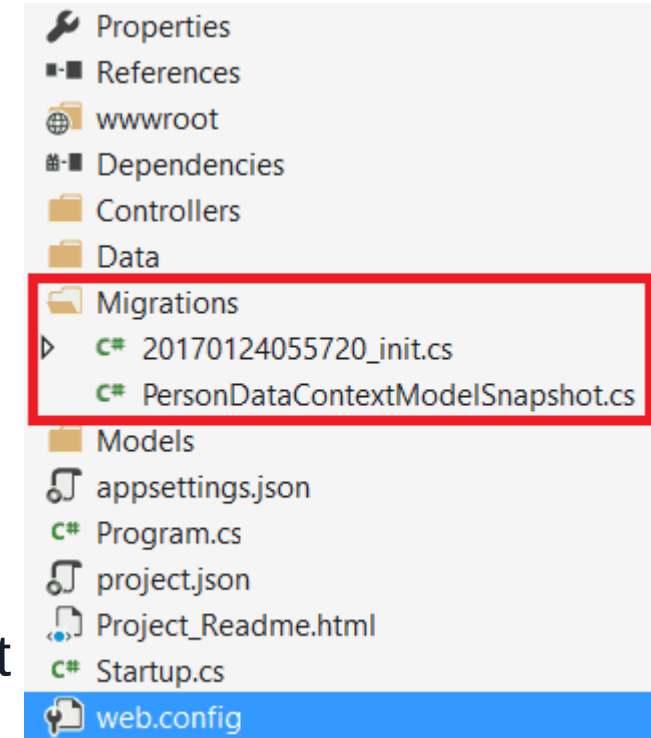


EF Migration Commands to Create the Database



1- .NET Core CLI:

- **dotnet-ef** and **Microsoft.EntityFrameworkCore.Design** for cross-platform command line tooling
 - `dotnet tool install --global dotnet-ef`
 - `dotnet add package Microsoft.EntityFrameworkCore.Design`
 - `dotnet ef migrations add InitialCreate`
 - `dotnet ef database update`



2- Package Manager Console (PMC) in Visual Studio:

- **Microsoft.EntityFrameworkCore.Tools** for PowerShell tooling that works in the Visual Studio **Package Manager Console**
 - `Install-Package Microsoft.EntityFrameworkCore.Tools`
 - `Add-Migration InitialCreate`
 - `Update-Database`



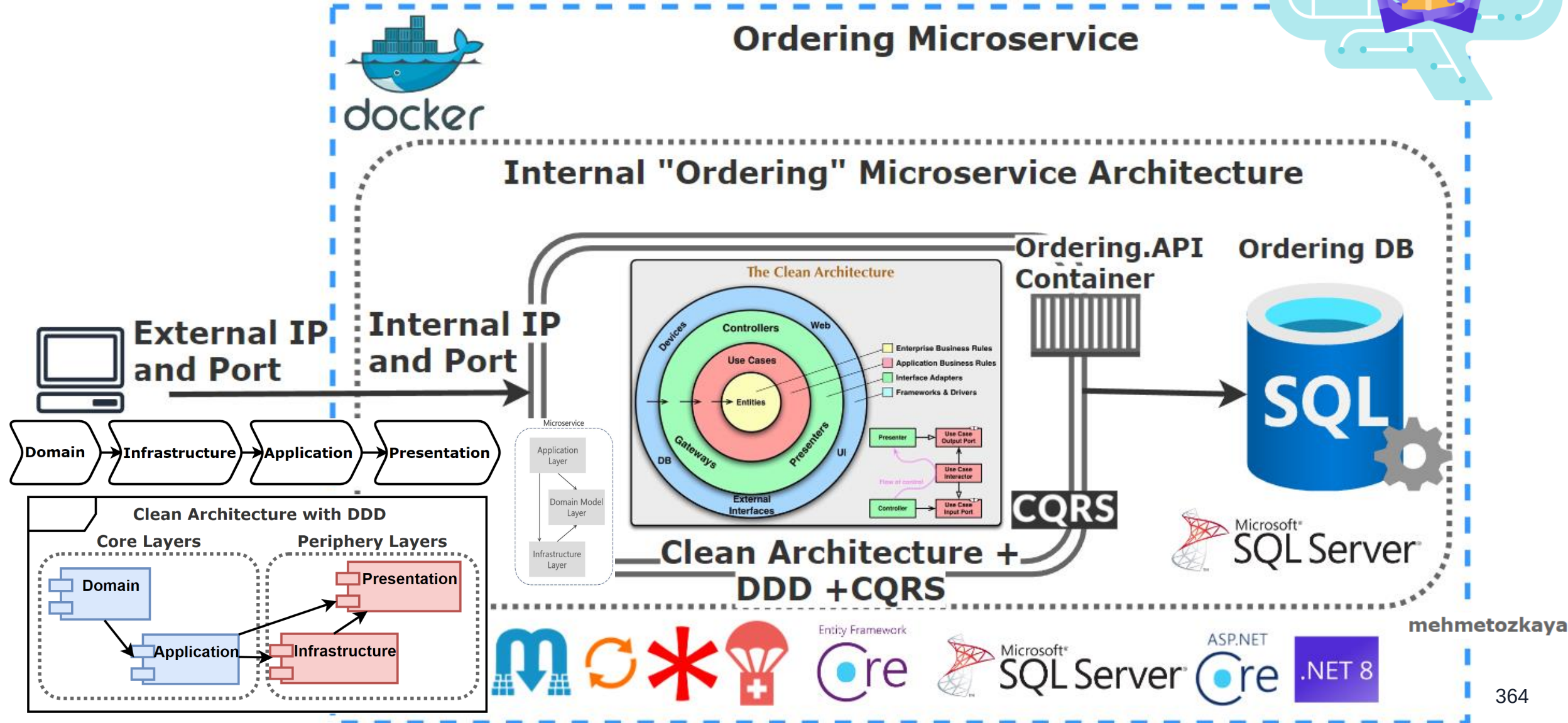
EF Migration Add-Migration Command Parameters



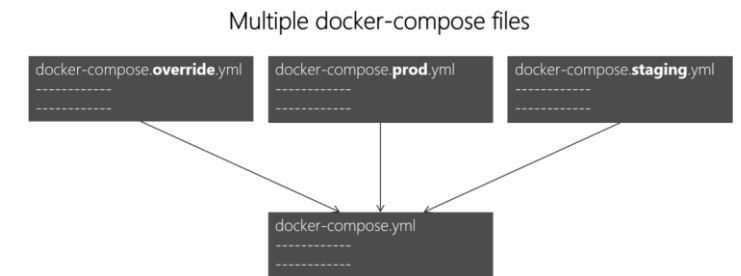
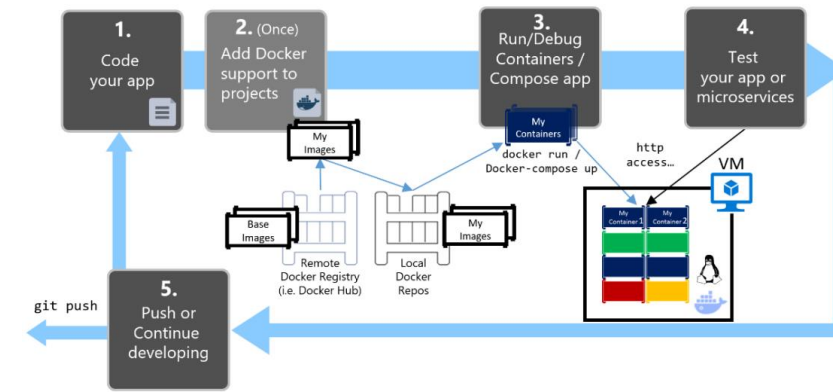
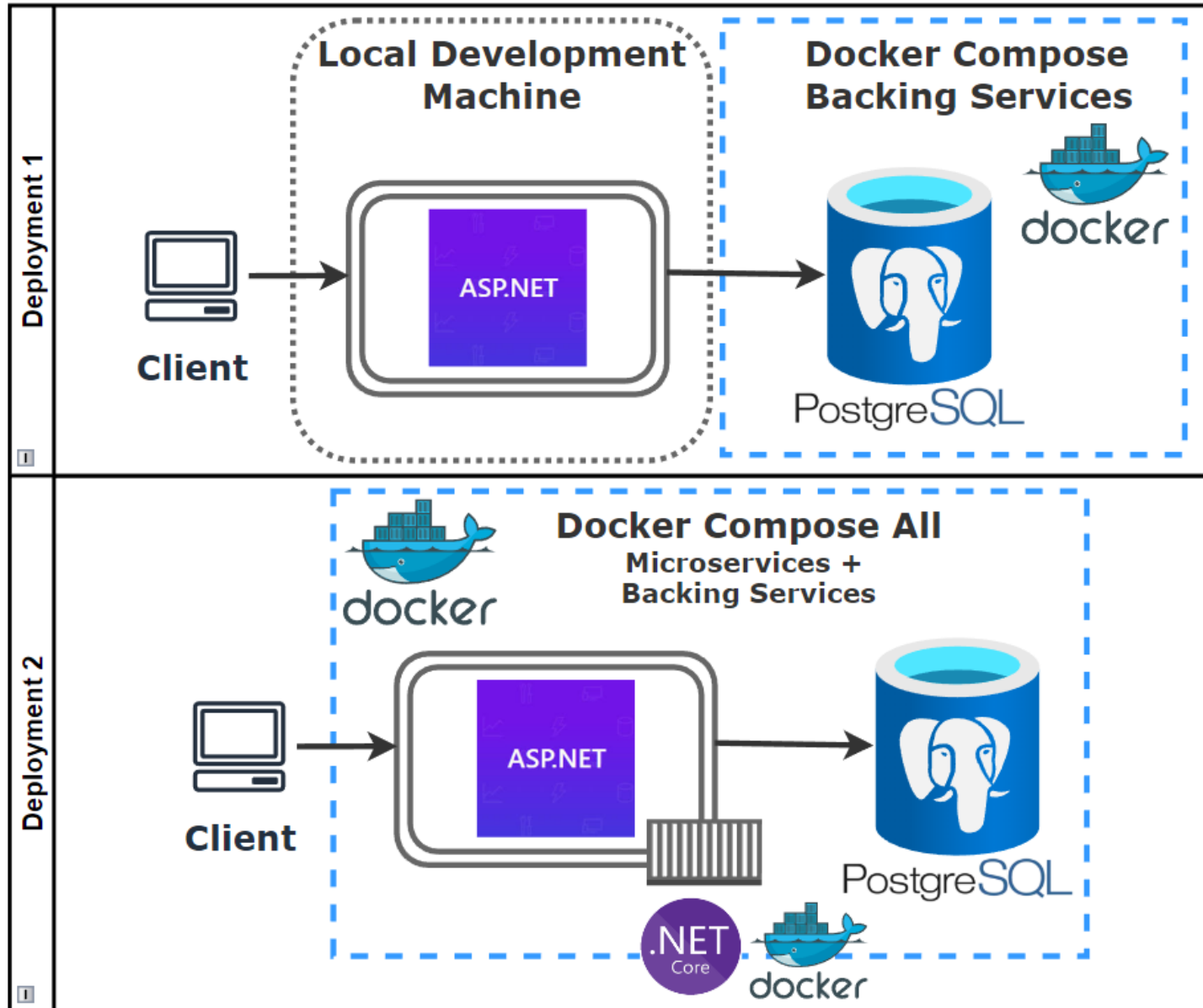
Option	Description
-Name	The name of the migration you want to create. While the <code>-Name</code> parameter is there, it's often simpler to directly specify the migration name as the first parameter. For example, <code>Add-Migration AddingEFExtensions</code> will create a migration named AddingEFExtensions .
-OutputDir	The output directory relative to the project where migration files will be created. For example, <code>Add-Migration AddingEFExtensions -OutputDir MyMigrationDirectoryName</code> will create the migration file in the MyMigrationDirectoryName directory.
-Namespace	The namespace to use for the generated classes in the migration file. If omitted, the following namespace <code>[ProjectNamespace].[OutputDirectoryName]</code> will be used. For example, <code>Add-Migration AddingEFExtensions -Namespace Z.MyMigrationNamespace</code> will add classes in the Z.MyMigrationNamespace namespace.
-Context	The context class to use to generate the migration file, it can be either the <code>name</code> or the <code>fullname</code> including namespaces. For example, <code>Add-Migration AddingEFExtensions -Context Z.MyContextName</code> will generate a migration for the Z.MyContextName context.
-Project	The project containing the context used to generate the migration file. If omitted, the default project in the Package Manager Console is used. For example, <code>Add-Migration AddingEFExtensions -Project Z.MyProjectName</code> will generate a migration for the context in the Z.MyProjectName project.
-StartupProject	The project with the configurations and settings. If omitted, the startup project of your solution is used. For example, <code>Add-Migration AddingEFExtensions -StartupProject Z.MyStartupProjectName</code> will make a migration using the configurations and settings of the Z.MyStartupProjectName project.

Special Consideration for Different Assemblies: Use `–OutputDir` Parameter

Ordering Microservice w/ DDD, CQRS and Clean Architecture



EShop Microservices Deployment Strategy



EF Core Developments Steps for Ordering.Infrastructure - continue



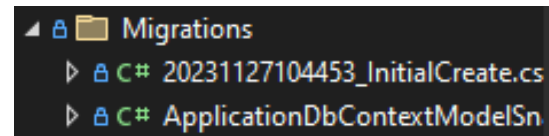
5- Register EF Core DbContext to Asp.Net:

- Register into DependencyInjection.cs

```
services.AddDbContext<ApplicationDbContext>((sp, options) =>
{
    options.AddInterceptors(sp.GetServices<ISaveChangesInterceptor>());
    options.UseSqlServer(connectionString);
});
```

6- Create Migrations and create Database w/ Commands:

- Install-Package Microsoft.EntityFrameworkCore.Tools
- Add-Migration InitialCreate
- Update-Database



Entity Framework



EF Core Developments Steps for Ordering.Infrastructure - continue



7- Create Database w/ Migrate method: (Auto Migrate EF Entities)

- Register into DependencyInjection.cs

```
using var scope = app.Services.CreateScope();

var context = scope.ServiceProvider.GetRequiredService<ApplicationDbContext>();

context.Database.MigrateAsync().GetAwaiter().GetResult();

await SeedAsync(context);
```

8- Seed Database:

- Create Data/DbInitializer.cs

```
if (!await context.Orders.AnyAsync())
{
    await context.Orders.AddRangeAsync(InitialData.OrdersWithItems);
    await context.SaveChangesAsync();
}
```

Entity Framework



EF Core Interceptors: SaveChangesInterceptor for Auditing Entities

- **Interceptors** in EF Core enable the interception, modification, or suppression of **EF Core operations**.
- This includes **low-level database operations** such as executing a command, as well as **higher-level operations**, such as calls to `SaveChanges`.

SaveChanges Interception:

- The **SaveChanges** and **SaveChangesAsync** interception points are used to execute **custom logic when saving changes** to the database.
- These interception points are defined by the **ISaveChangesInterceptor** interface.
- **EF Core** provides a **SaveChangesInterceptor** base class with no-op (no operation) methods as a convenience.



Entity Framework



Use Case: SaveChanges interception for Auditing



- Interception of **SaveChanges** can be used to create an independent **audit record** of changes.
- This is useful for **maintaining a history** of who changed an entity and when.
- **Before saving** changes to the database, you can **iterate** through the **changed entities** in the **DbContext** and **log** or **store** the **audit information**, like timestamps or user identifiers.



Entity Framework



Registering Interceptors

How to Register Interceptors ?

- Interceptors are registered using **AddInterceptors** when configuring a **DbContext** instance.
- Common approach is usually done in the **OnConfiguring** method of the **DbContext**.
- **Example Code:** Registering a SaveChangesInterceptor

```
public class ExampleContext : BlogsContext
{
    0 references
    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    => optionsBuilder.AddInterceptors(new TaggedQueryCommandInterceptor());
}
```



Entity Framework



Implementing a Custom SaveChangesInterceptor



- Implement a custom interceptor by extending **SaveChangesInterceptor** or implementing **ISaveChangesInterceptor**.
- **Override** methods like **SavingChanges** and **SavedChanges** to execute custom logic.
- **Example Code:** Implementing Auditing in an Interceptor

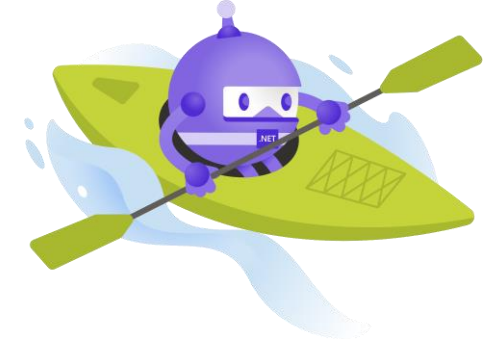
```
public class MySaveChangesInterceptor : SaveChangesInterceptor
{
    0 references
    public override InterceptionResult<int> SavingChanges(DbContextEventData eventData)
    {
        var context = eventData.Context;
        foreach (var entry in context.ChangeTracker.Entries().Where(e => e.State ==
        {
            // Example: Set audit fields based on entity state
            if (entry.Entity is AuditableEntity entity)
            {
                if (entry.State == EntityState.Added)
                {
                    entity.CreatedAt = DateTime.UtcNow;
                    entity.CreatedBy = // get current user
                }
            }
        }
    }
}
```



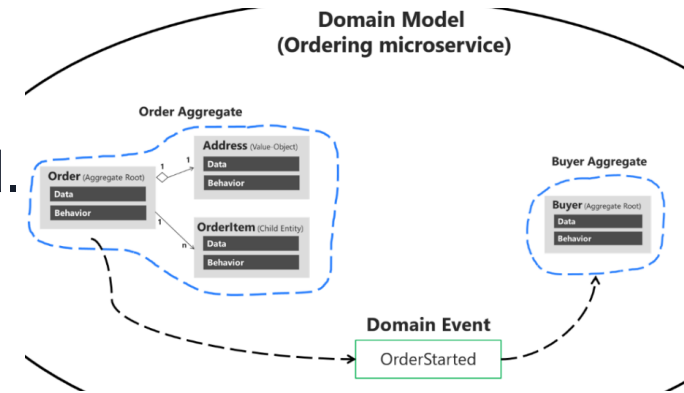
Entity Framework



Domain Event in DDD



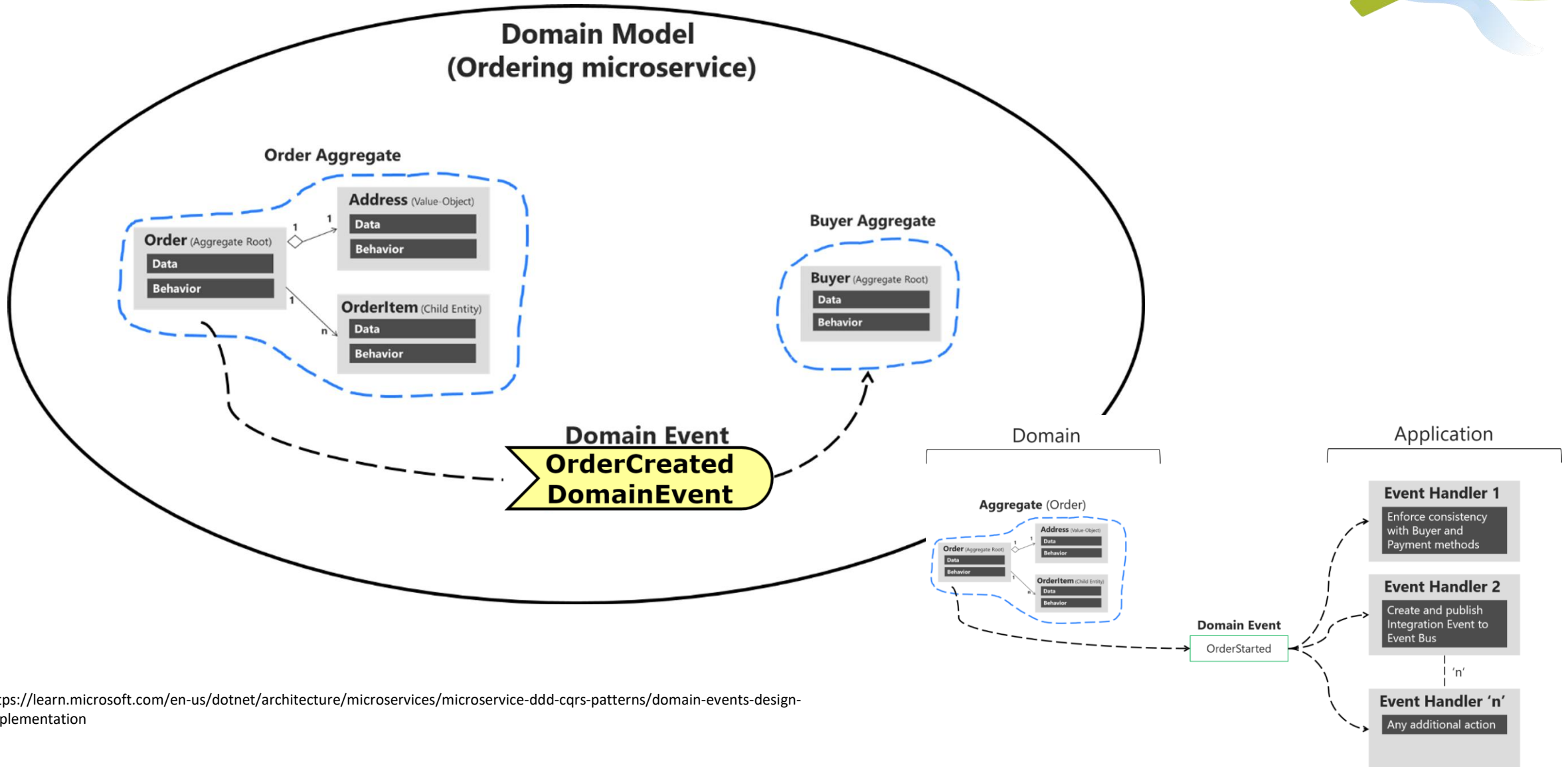
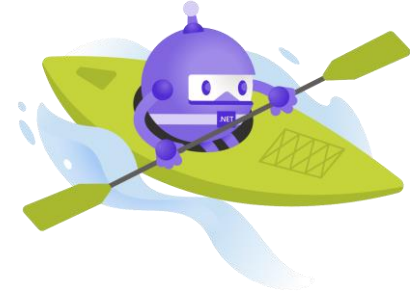
- **Domain Events** represent something that **happened** in the **past** and the other parts of the **same service boundary** same domain need to react to these changes.
- **Domain Event** is a **business event** that occurs within the **domain model**. It often represents a side effect of a domain operation.
- **Achieve consistency** between **aggregates** in the same domain.
- When an order is placed, an OrderPlaced event might be triggered.
- Trigger **side effects** or **notify other parts** of the system about changes within the domain.



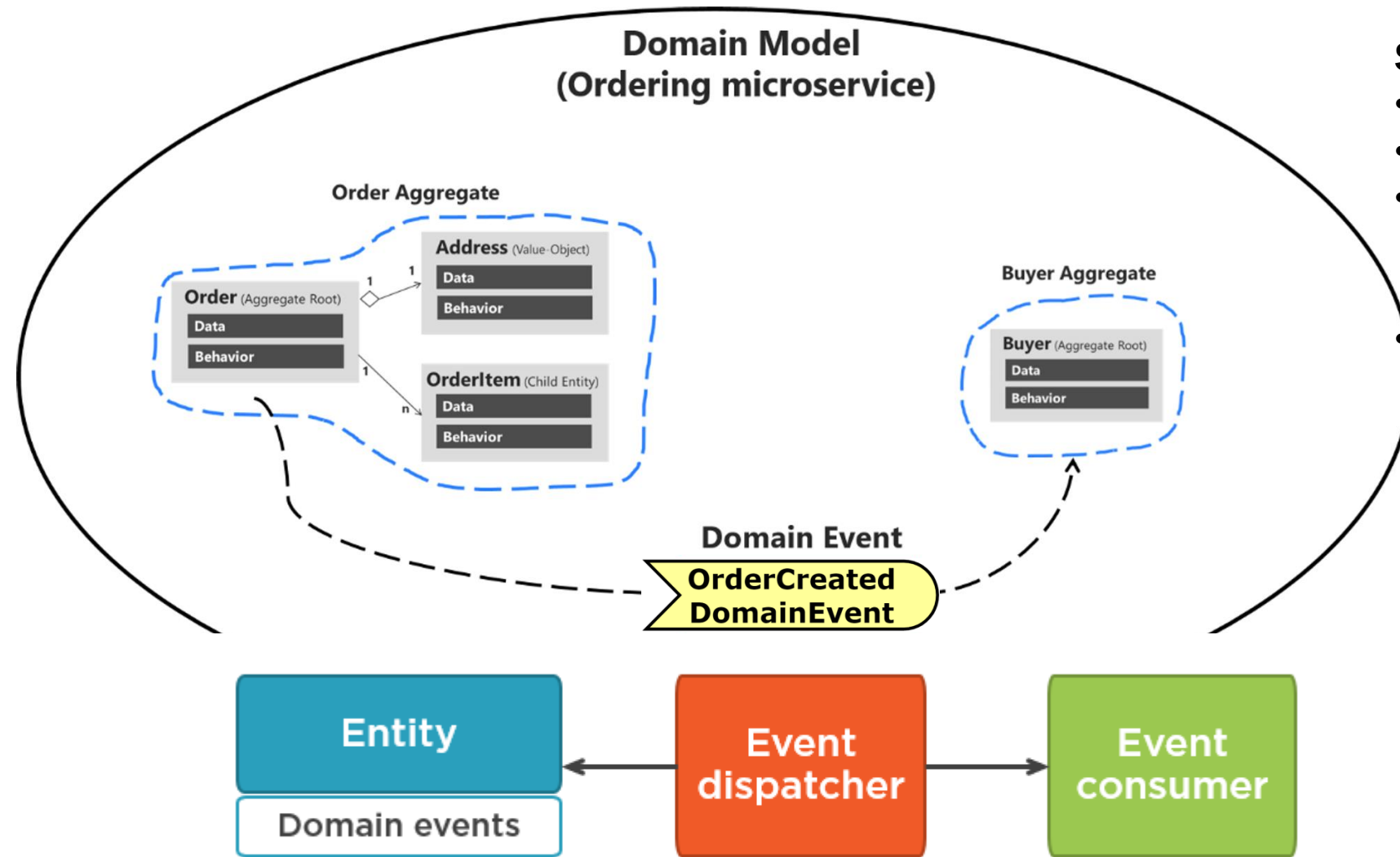
How to Use Domain Events in DDD?

- **Encapsulate** the event details and **dispatch** them to interested parties.
- **Communicate** changes **within the domain** to external handlers which may perform actions based on these events.

Eshop Domain Events in Ordering Microservices



Dispatch Domain Events w/ EF Core SaveChangesInterceptor

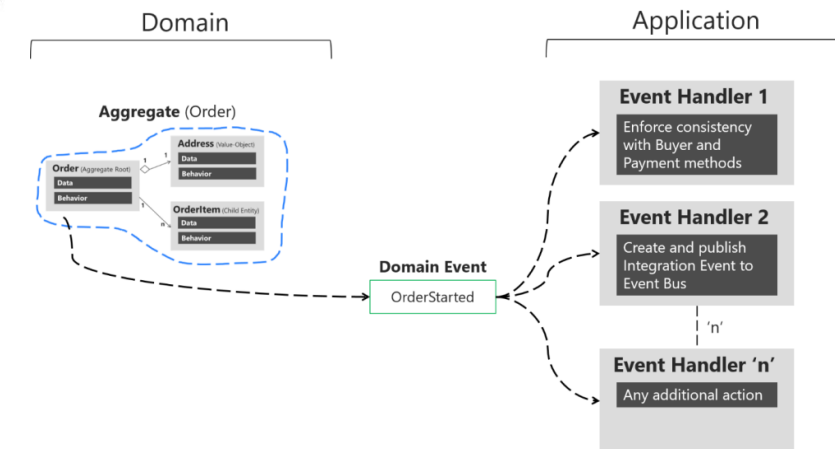


Steps dispatch Events:

- Create a SaveChangesInterceptor
- Identify and Dispatch Domain Events
- public class

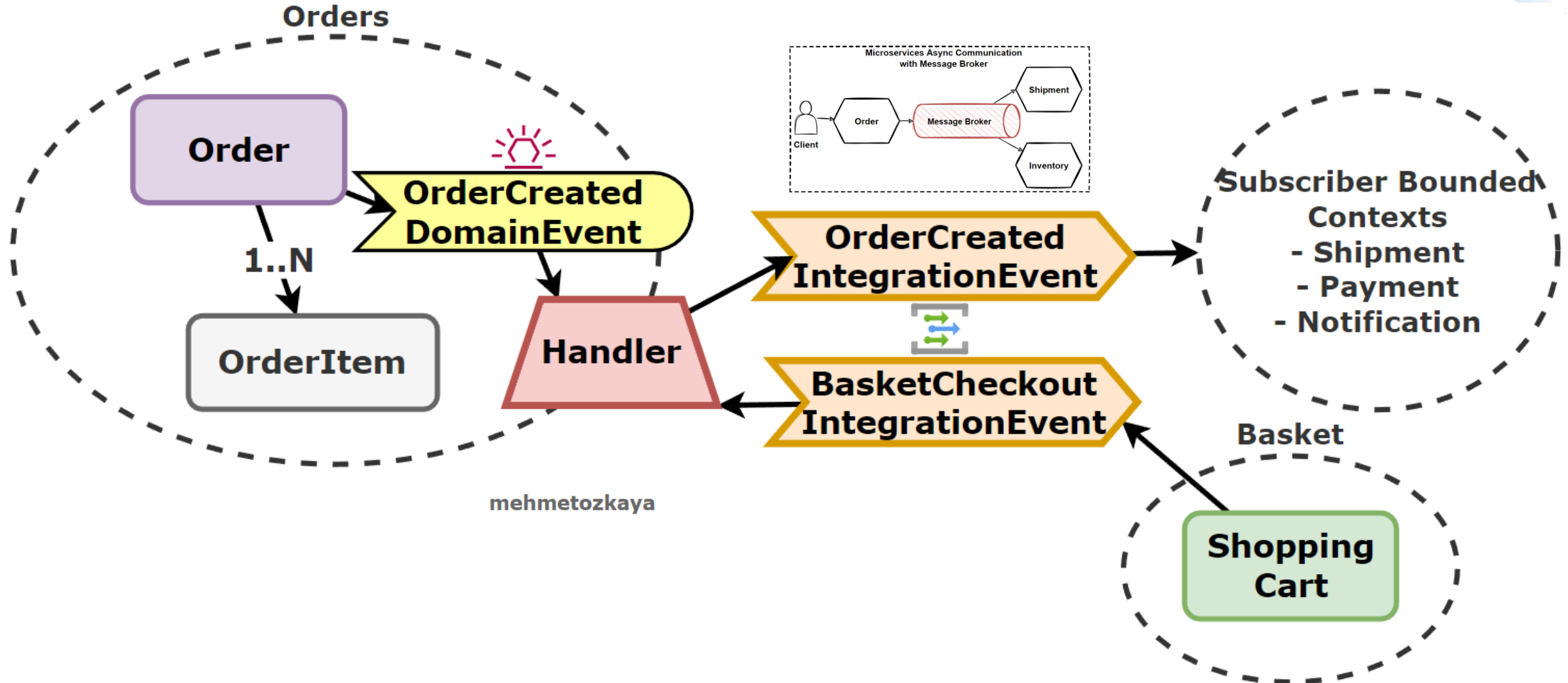
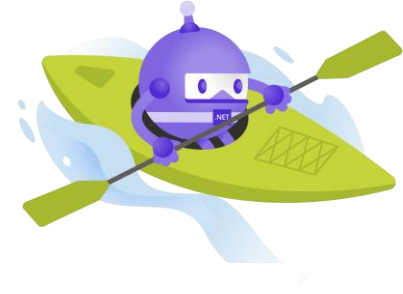
DomainEventDispatcherInterceptor :
SaveChangesInterceptor

- Registering the Interceptor into DependencyInjection class

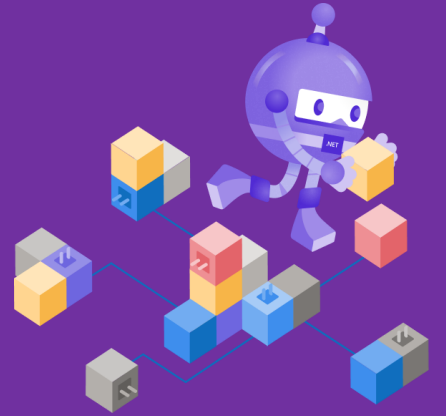


Best Practice to dispatch Events:
EF Core SaveChangesInterceptor

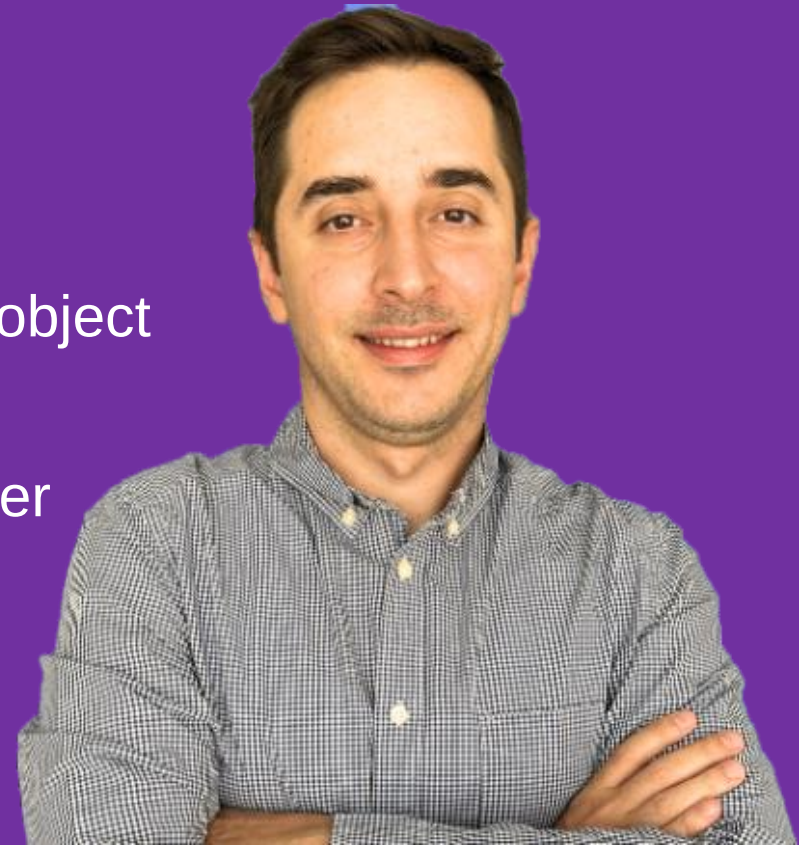
Eshop Domain and Integration Events



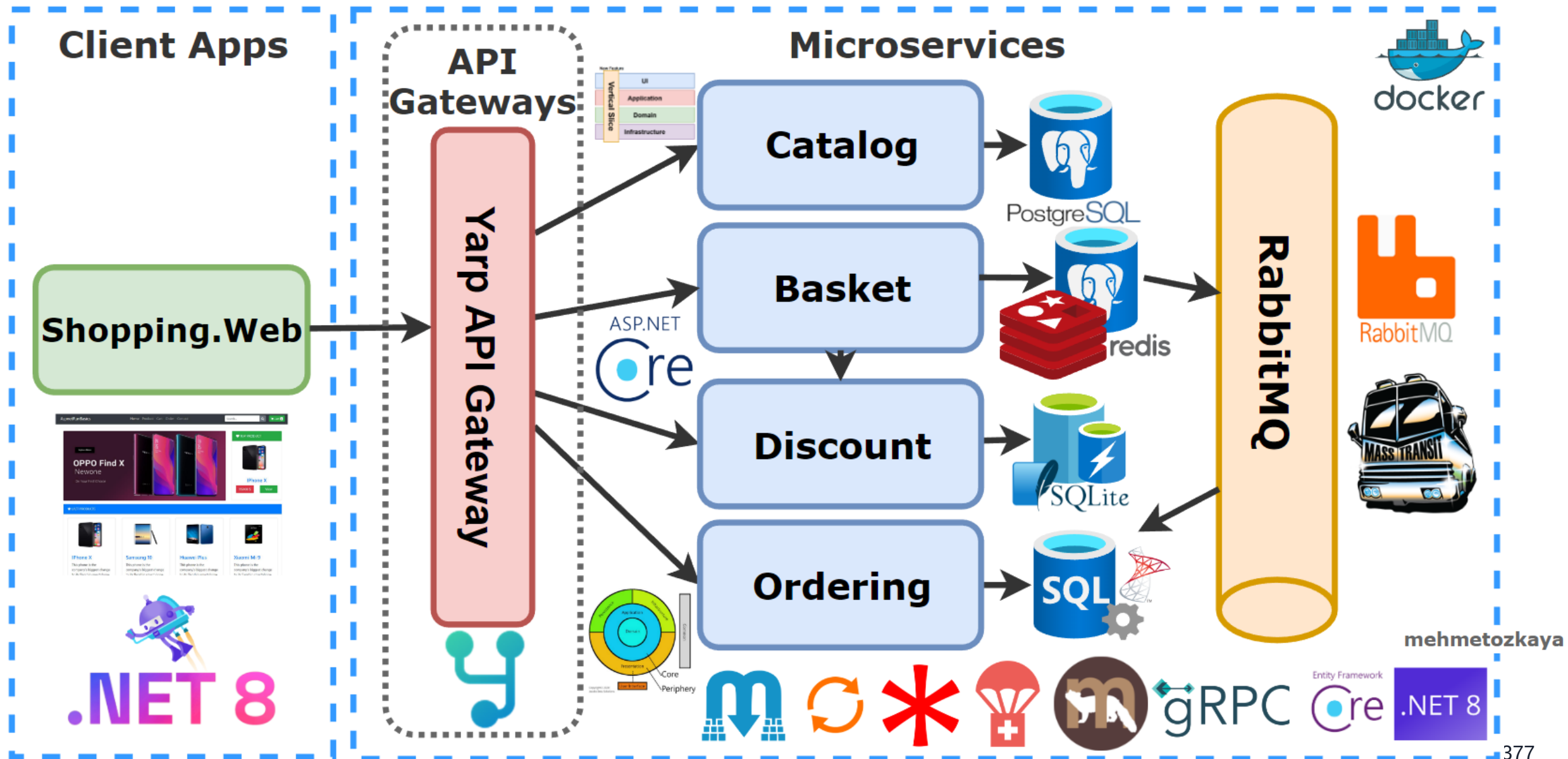
Ordering.Application Layer with CQRS and MediatR



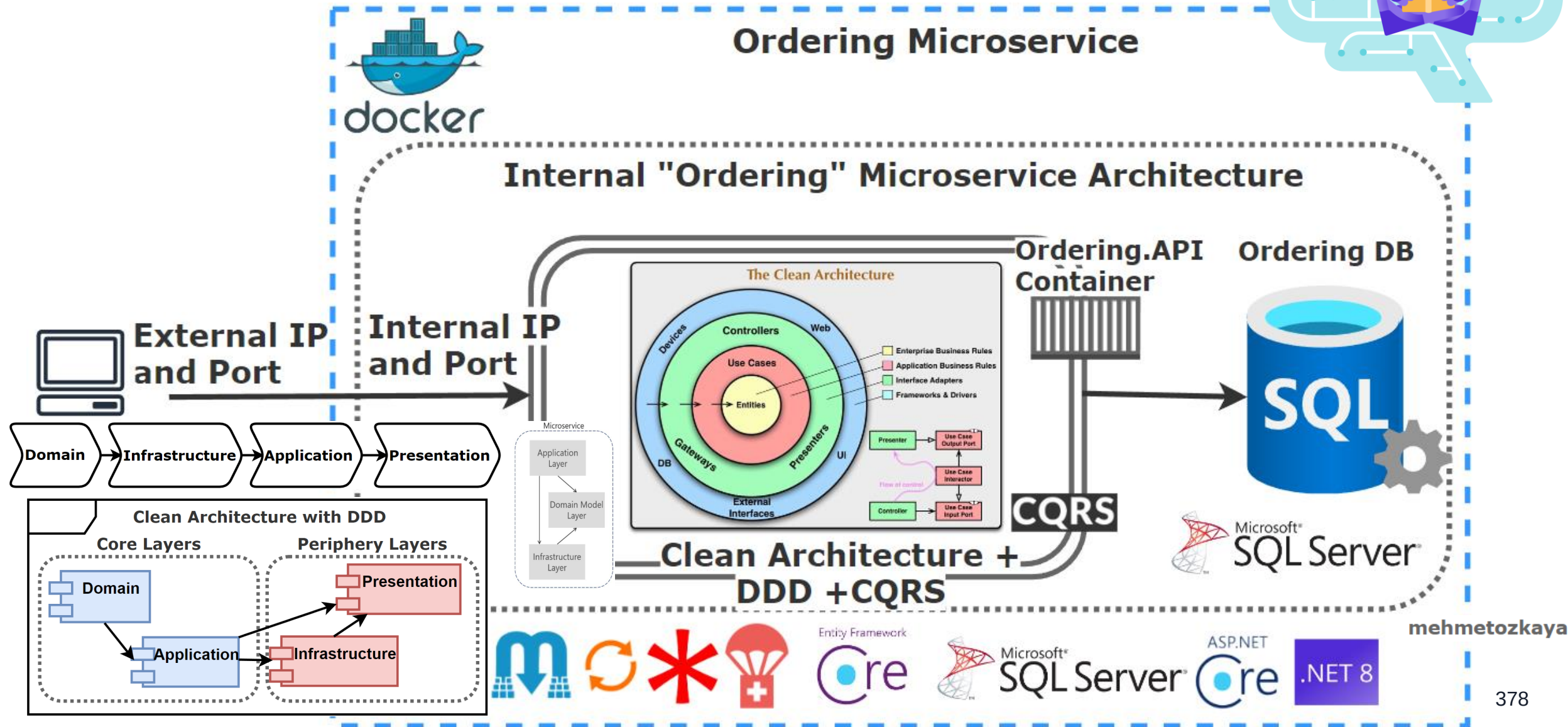
- Develop Orders CRUD OrderCommand and Handler
- Develop IApplicationDbContext for injection EFCore DB Context object
- Develop MediatR Pipeline Behaviours (Validation, Logging)
- Develop Domain Event Handlers with MediatR INotificationHandler
- Develop Order Queries with CQRS and MediatR



Big Picture – Course Project



Ordering Microservice w/ DDD, CQRS and Clean Architecture



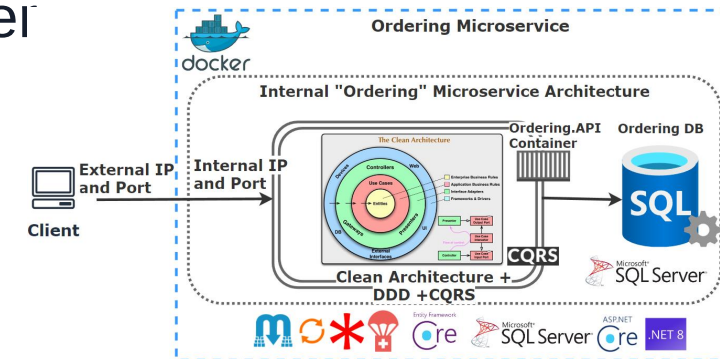
Ordering.Application Layer with CQRS and MediatR



- Apply DDD, CQRS and Clean Architecture Patterns
- Develop Orders CRUD OrderCommand and Handler with CQRS
- Implement the command process pipeline with a Mediator Pattern using MediatR
- Develop IApplicationDbContext for injection EFCore DB Context object
- Develop MediatR Pipeline Behaviours (Validation, Logging) for Cross-cutting concerns
- Develop Domain Event Handlers with MediatR INotificationHandler
- Develop Order Queries with CQRS and MediatR

Main Target of Infra Layer:

- Implementing application logic and use cases with CQRS and MediatR patterns

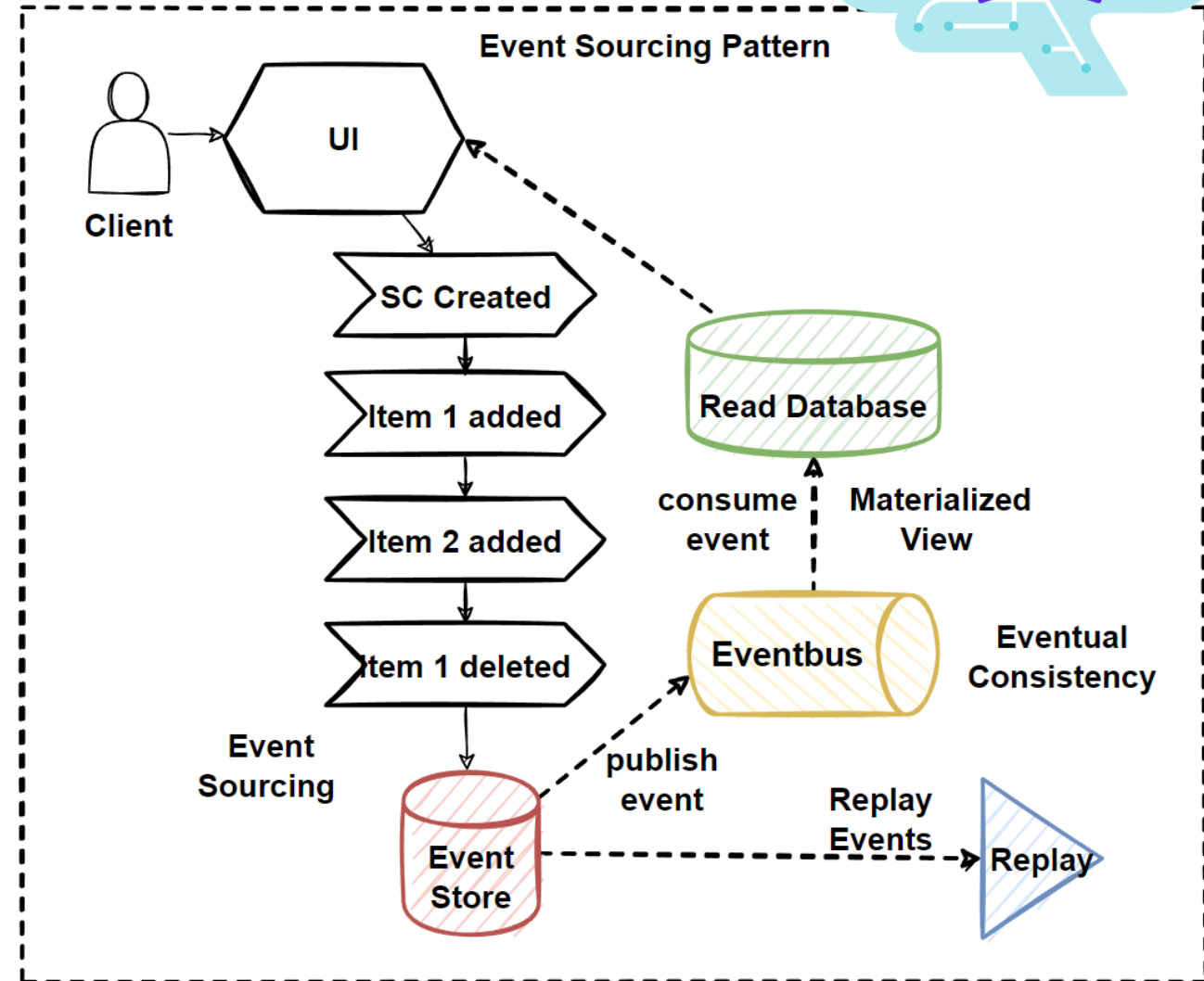


Fundamental Patterns of Application Layers

- CQRS Pattern
- Event Sourcing Pattern
- CQRS with Event Sourcing Pattern
- Eventual Consistency Principle

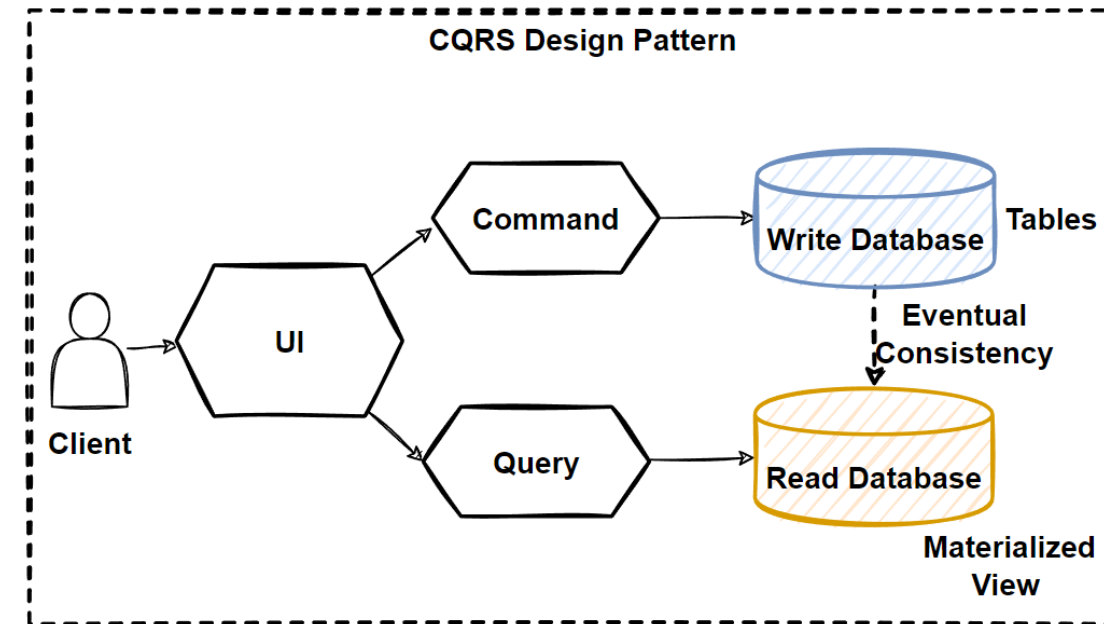
After that;

- Design Ordering Microservice Application Layer and Web
- Implement the Command and Command Handler patterns with MediatR



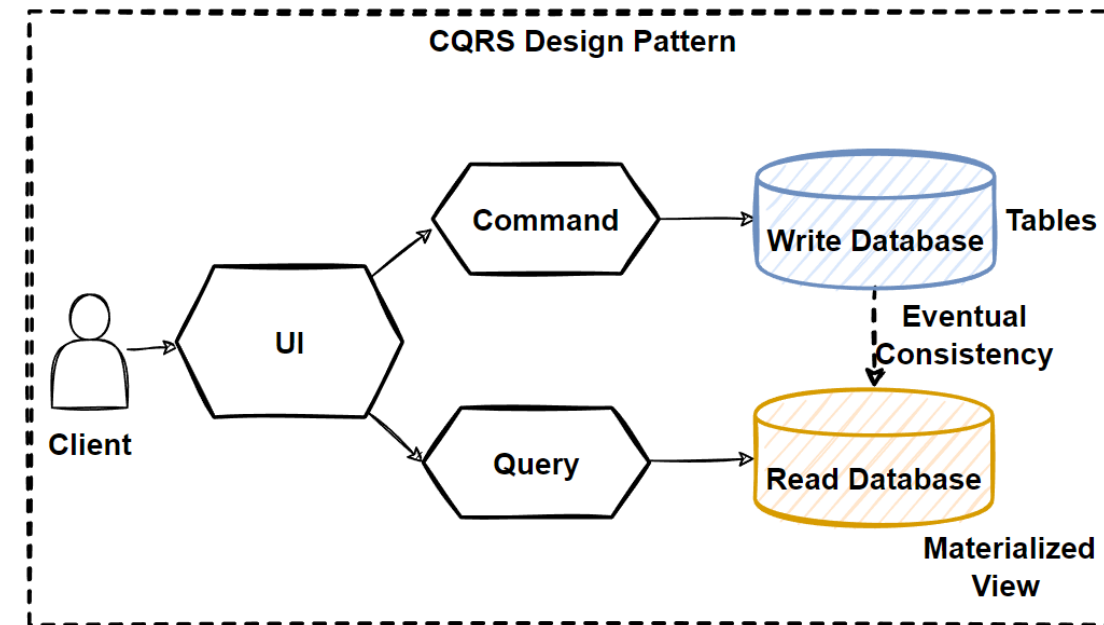
CQRS - Command Query Responsibility Segregation

- **CQRS design pattern** in order to avoid **complex queries** to get rid of **inefficient joins**.
- **Separates read and write operations** with separating databases.
- **Commands**: changing the state of data into application.
- **Queries**: handling complex join operations and returning a result and don't change the state of data into application.
- Large-scaled **microservices architectures** needs to manage **high-volume data requirements**.
- **Single database** for services can **cause bottlenecks**.
- Uses both **CQRS** and **Event Sourcing** patterns to improve application performance.
- **CQRS** offers to **separates read and write data** that provide to maximize query performance and scalability.



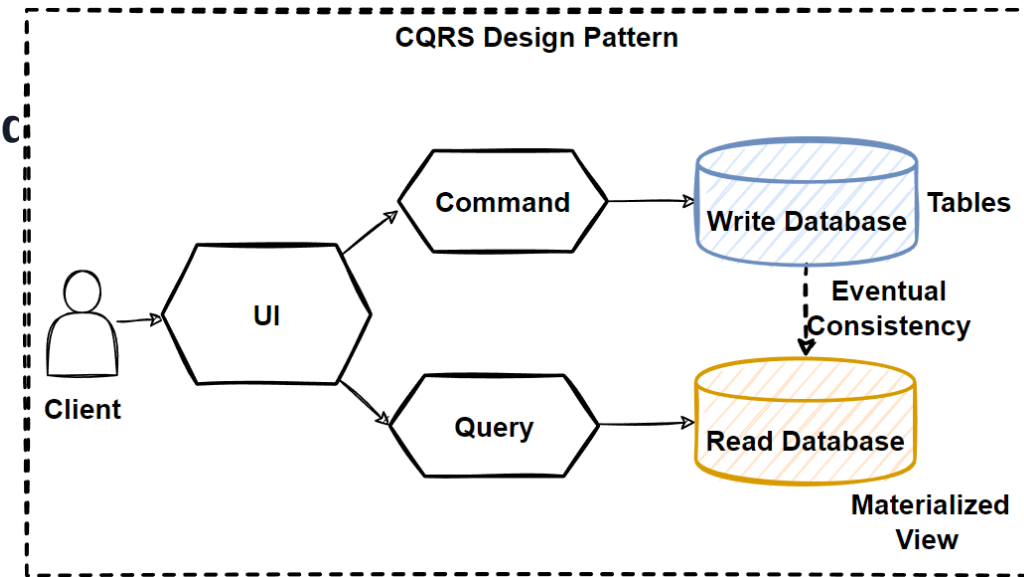
CQRS – Read and Write Operations

- **Monolithic** has **single database** is both working for **complex join queries**, and also perform **CRUD operations**.
- When **application** goes **more complex**, this **query** and **CRUD** operations will become **un-manageable situation**.
- Application required some **query** that **needs** to **join** more than **10 table**, will **lock** the **database** due to **latency** of **query computation**.
- Performing **CRUD operations** need to make **complex validations** and **process long business logics**, will cause to **lock database** operations.
- **Reading** and **writing database** has different approaches, define different strategy.
- «**Separation of concerns**» principles: separate reading database and the writing database with 2 database.
 - **Read database** uses No-SQL databases with denormalized data.
 - **Write database** uses Relational databases with fully normalized and supports strong data consistency.



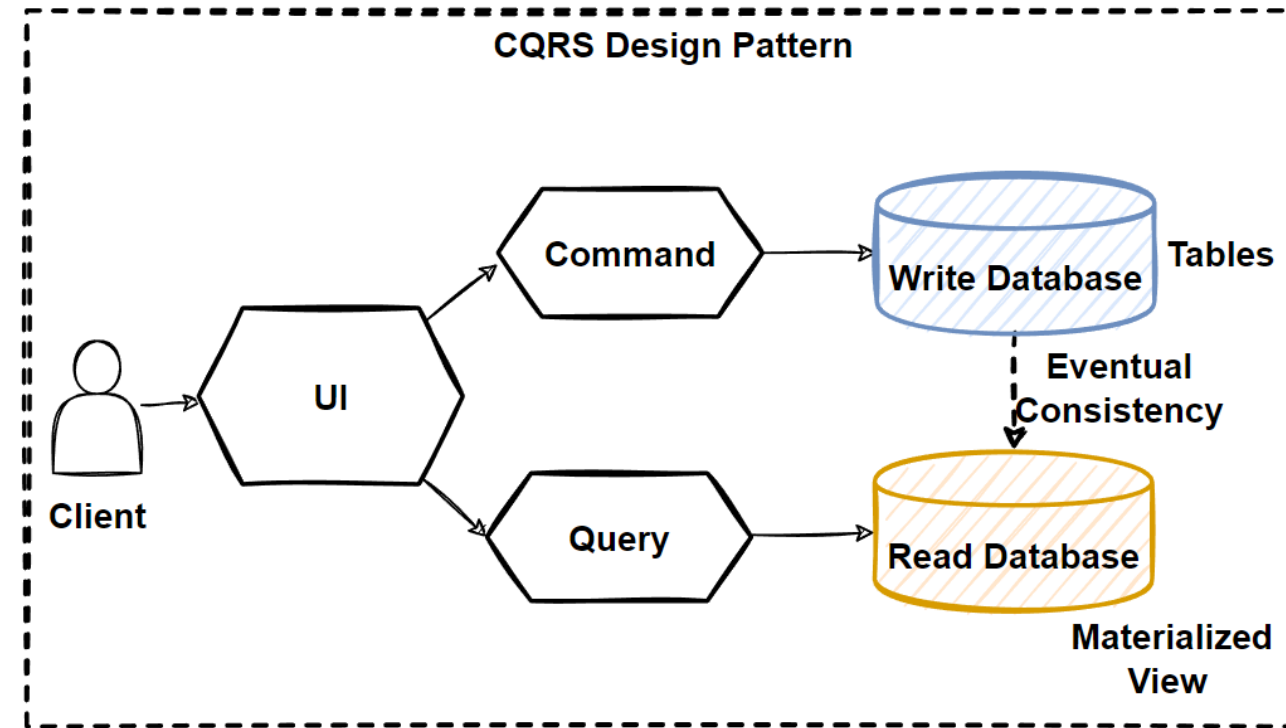
CQRS – Read and Write Operations - 2

- If our **application** is **mostly reading use cases** and not writing so much, it is **read-incentive application**.
- **Read** and **write operations** are **asymmetrical** and has very different performance and scale requirements.
- To **improve query performance**, the read operation perform **queries** from a **highly denormalized materialized views** to **avoid** expensive repetitive **table joins** and **table locks**.
- **Write operation** which is **command operation**, can perform commands into separate **fully normalized relational database**.
- Supporting **ACID transactions** and **strong data consistency**.
- Commands: **task-based operations** like "add item into shopping cart" or "checkout order".
- **Commands** can be handle with message broker systems that provide to process commands in async way.
- **Queries:** never modify the database, always return the JSON data with DTO objects.



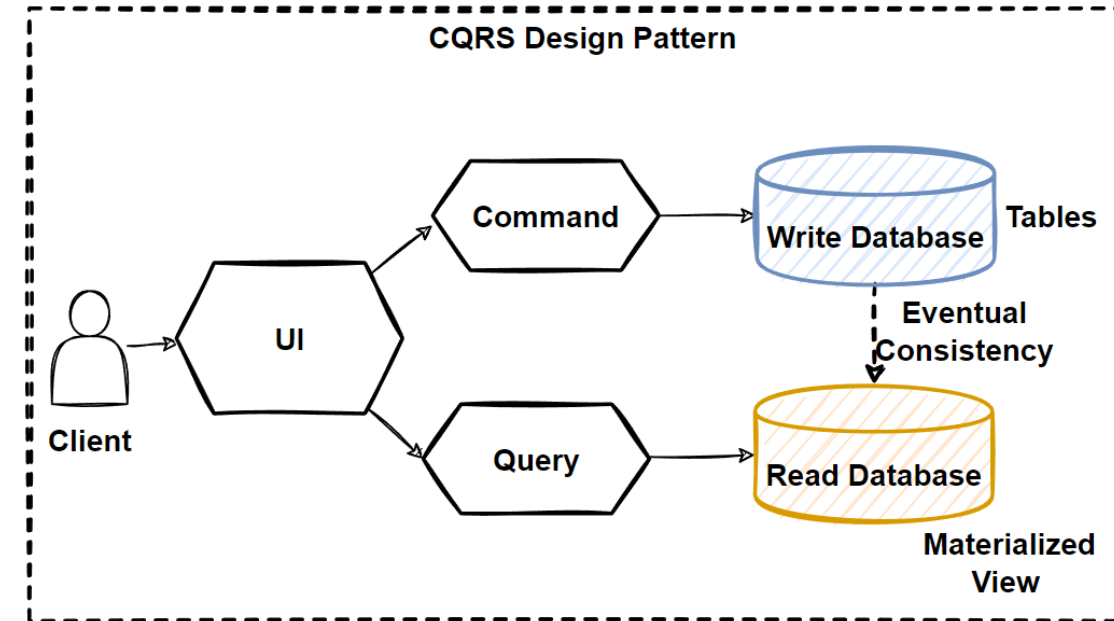
CQRS – Synchronization with Read and Write DB

- Keep sync both read and write databases.
- Publishes an event that subscribe from **Read Database** and **update** the **read table** accordingly.
- **Synchronization** handles with async communication using message brokers.
- This creates **Eventual Consistency principle**.
- The **Read** database **eventually synchronizes** from the **Write** database.
- **Some lag** in the process due to **async communication** with message brokers that applies publish/subscribe pattern.
- Welcome to **Eventual consistency**.



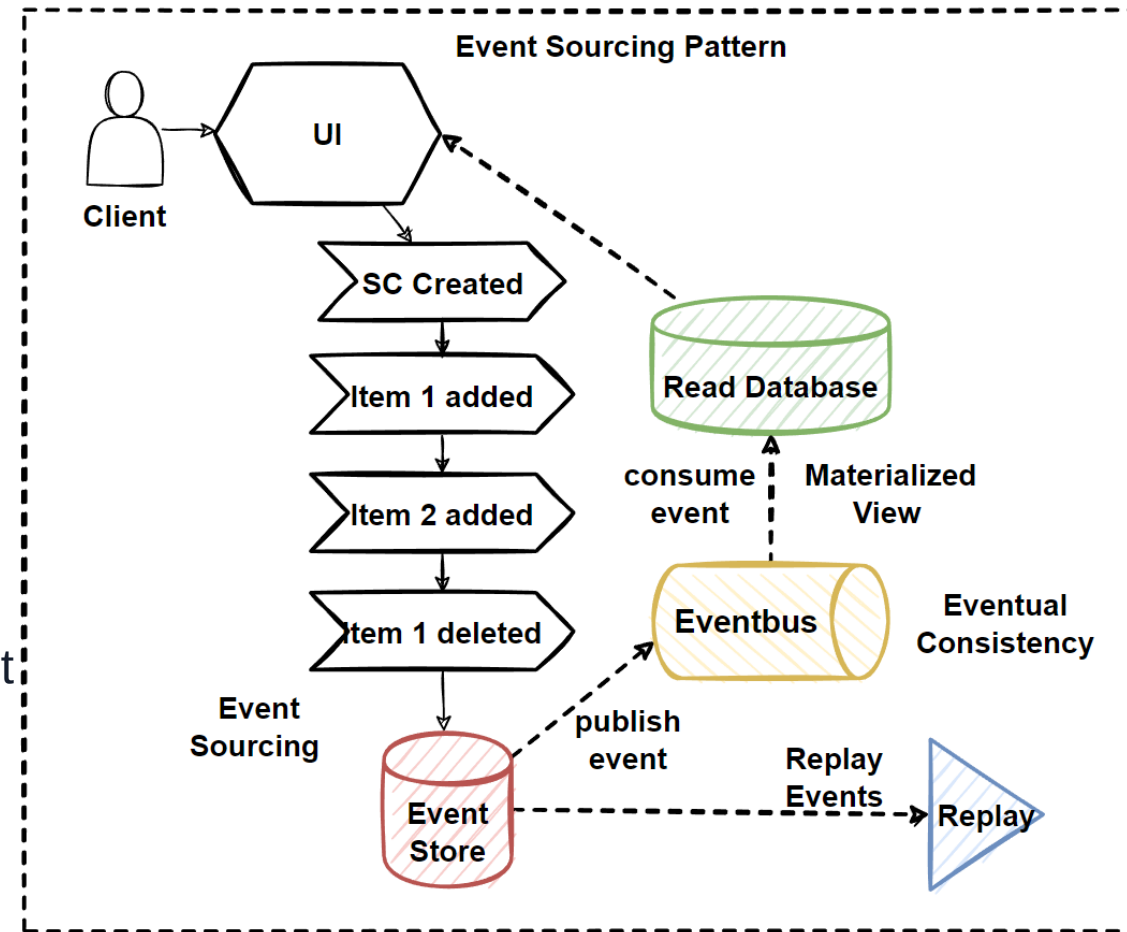
Best Practices for CQRS

- **Best practices to separate read and write database with 2 database physically.**
- **Read-intensive** that means **reading more than writing**, can define **custom data schema to optimized for queries.**
- **Materialized View Pattern** is good example to implement reading databases.
- **Avoid complex joins** and mappings with **pre-defined fine-grained data for query operations.**
- **Use different database** for reading and writing database types.
- Using **No-SQL** document database **for reading** and using **Relational** database **for CRUD** operations.



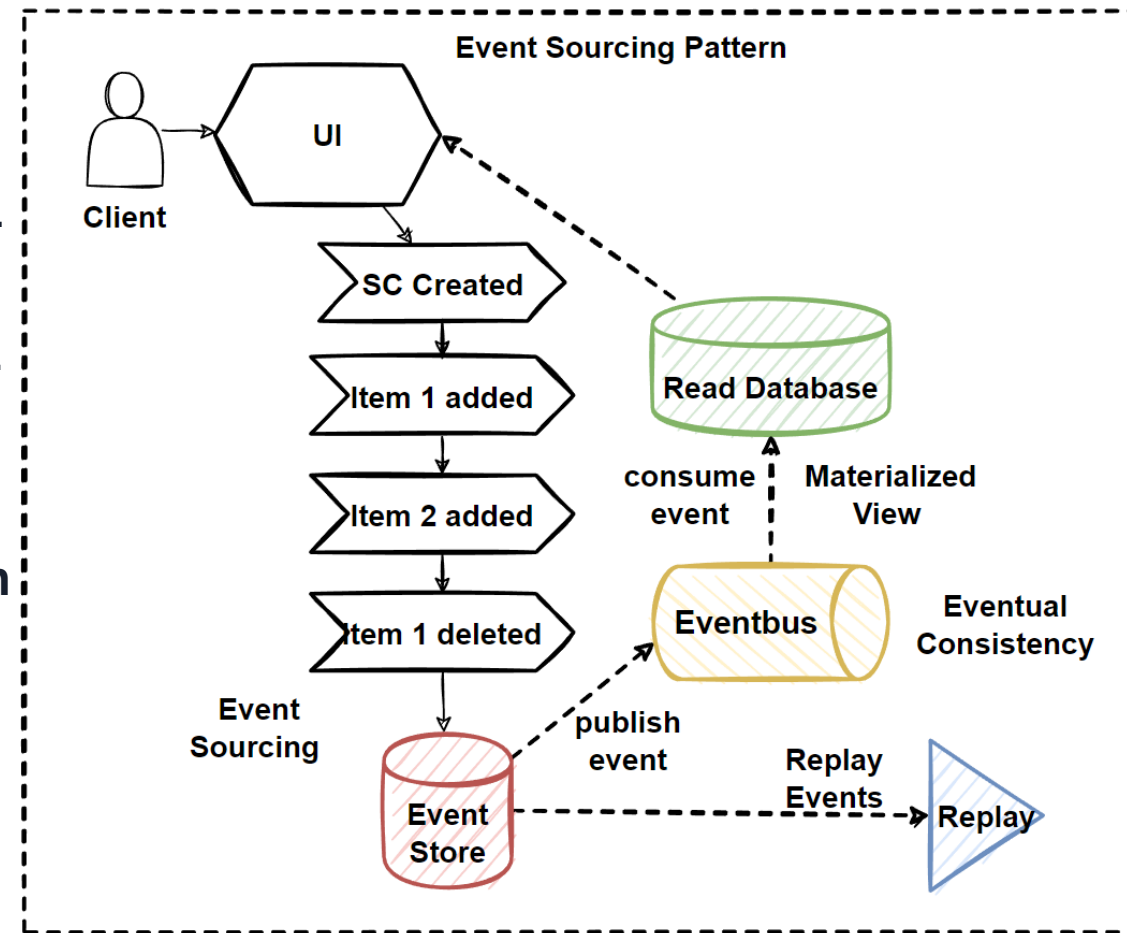
Event Sourcing Pattern

- Most applications **saves data** into databases with **the current state of the entity**. I.e. user change the email address table, email field updated with the latest updated one. Always know the **latest status** of the **data**.
- In large-scaled architectures, **frequent update database** operations can **negatively impact database performance, responsiveness, and limits of scalability**.
- **Event Sourcing pattern** offers to persist each action that affects to data into **Event Store database**. And call all these **action** as a **event**.
- **Instead of saving latest status** of data into database, Event Sourcing pattern offers to **save all events into database** with **sequential ordered** of data events.
- This **events database** called **Event Store**.



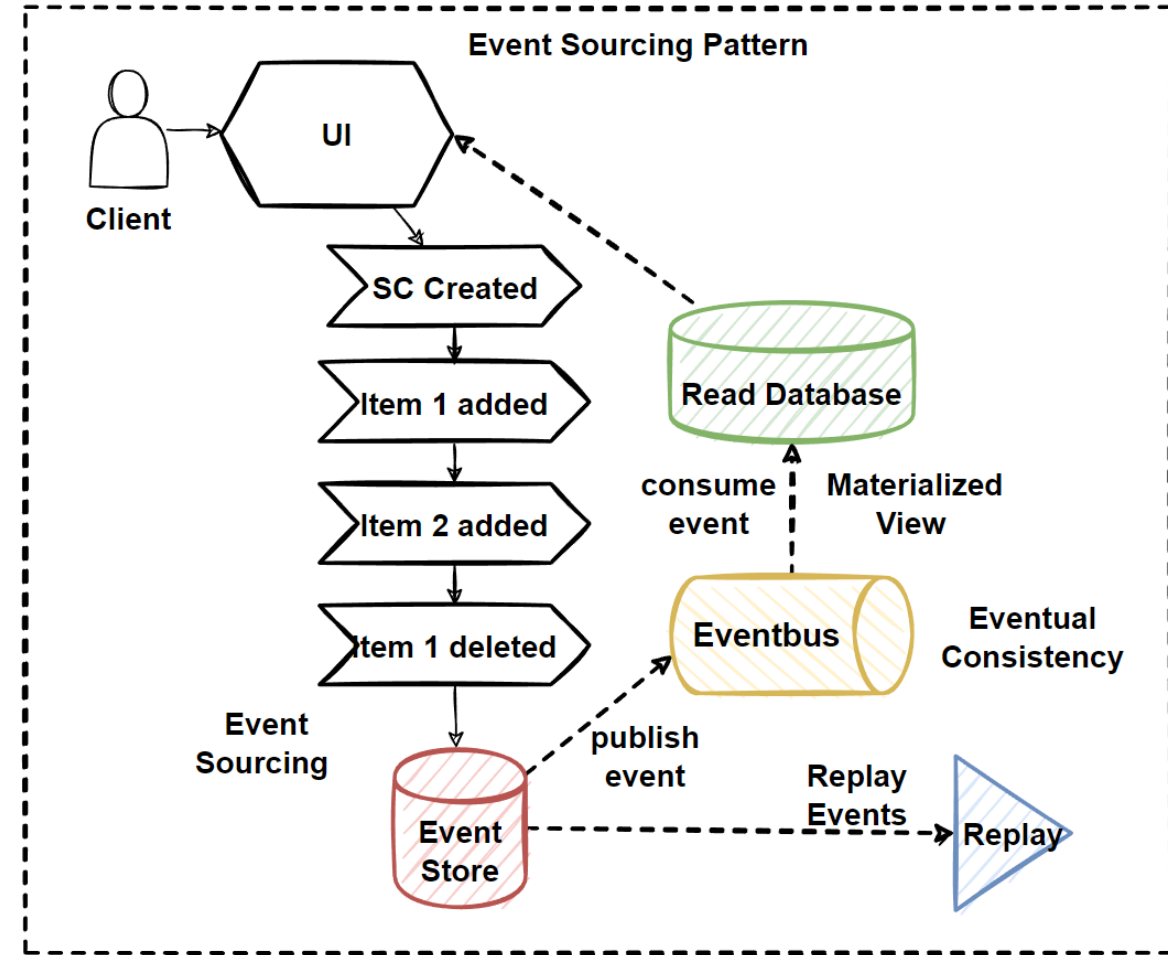
Event Sourcing Pattern - 2

- Instead of overriding the data into table, It create a new record for each change to data, and it becomes sequential list of past events.
- Event Store database become the source-of-truth of data.
- Sequential event list using for generating Materialized Views that represents final state of data to perform queries.
- Event Store convert to read database with following the Materialized Views Pattern.
- Convert operation can handle by publish/subscribe pattern with publish event with message broker systems.
- Event list gives ability to replay events at given certain timestamp.
- It is able to re-build latest status of data with replaying events.

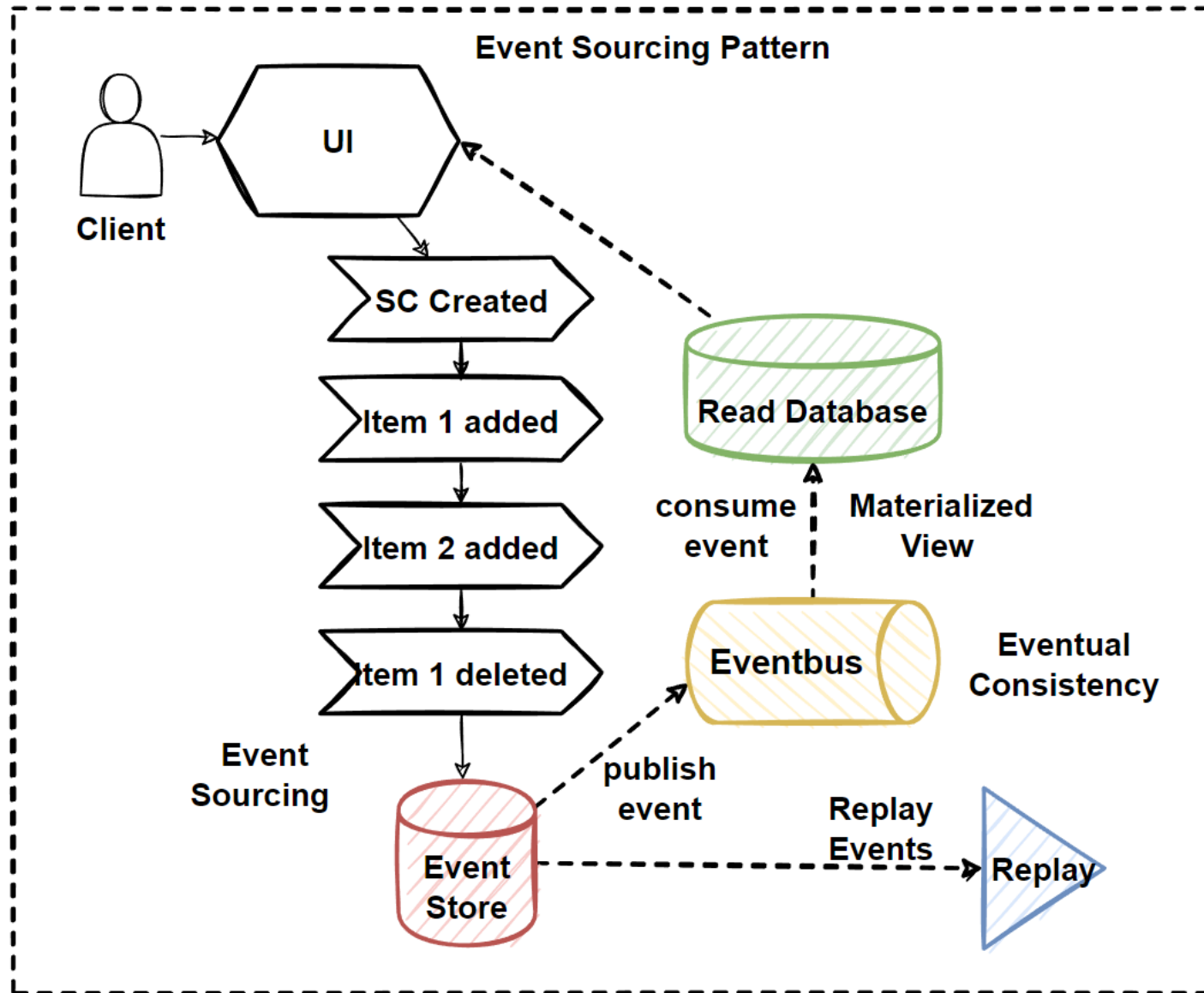


CQRS with Event Sourcing Pattern

- **CQRS** pattern is mostly **using with the Event Sourcing** pattern.
- **Store events** into the **write database**; **source-of-truth events** database.
- **Read database** of CQRS pattern provides **materialized views** of the data with **denormalized tables**.
- **Materialized views** read database **consumes events** from write database **convert them** into **denormalized views**.
- The **writing database** is **never save status** of data only events actions are stored.
- **Store history of data** and able to **reply any point of time** in order to **re-generate status** of data.
- System can **increased query performance** and **scale databases independently**.

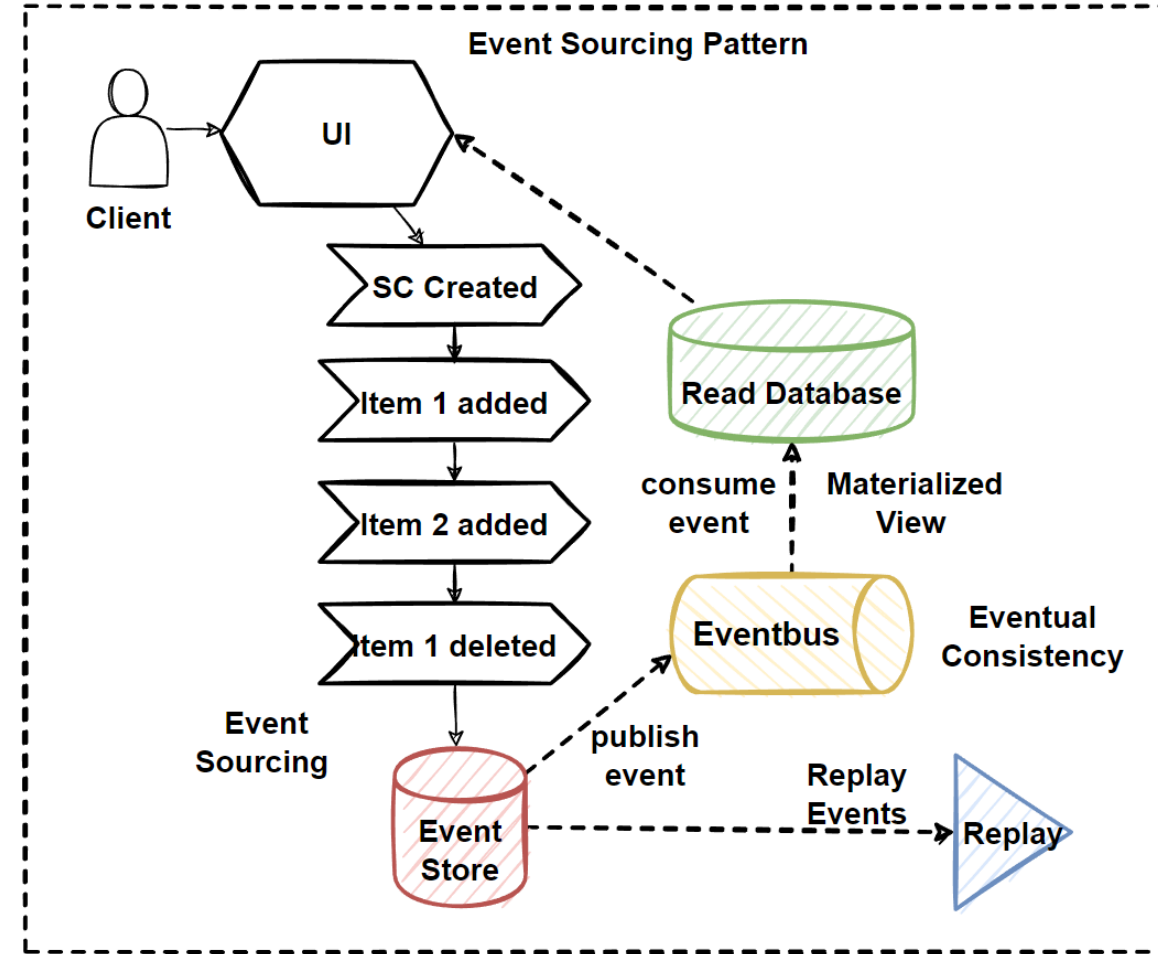


CQRS with Event Sourcing Pattern



Eventual Consistency Principle

- **CQRS** with **Event Sourcing Pattern** leads **Eventual Consistency**.
- **Eventual Consistency** is especially used for systems that **prefer high availability to strong consistency**.
- The system will **become consistent after a certain time**.
- We called this latency is a **Eventual Consistency Principle** and offers to be **consistent after a certain time**.
- There are **2 type of "Consistency Level"**:
- **Strict Consistency**: When we save data, the data should affect and seen immediately for every client.
- **Eventual Consistency**: When we write any data, it will take some time for clients reading the data.

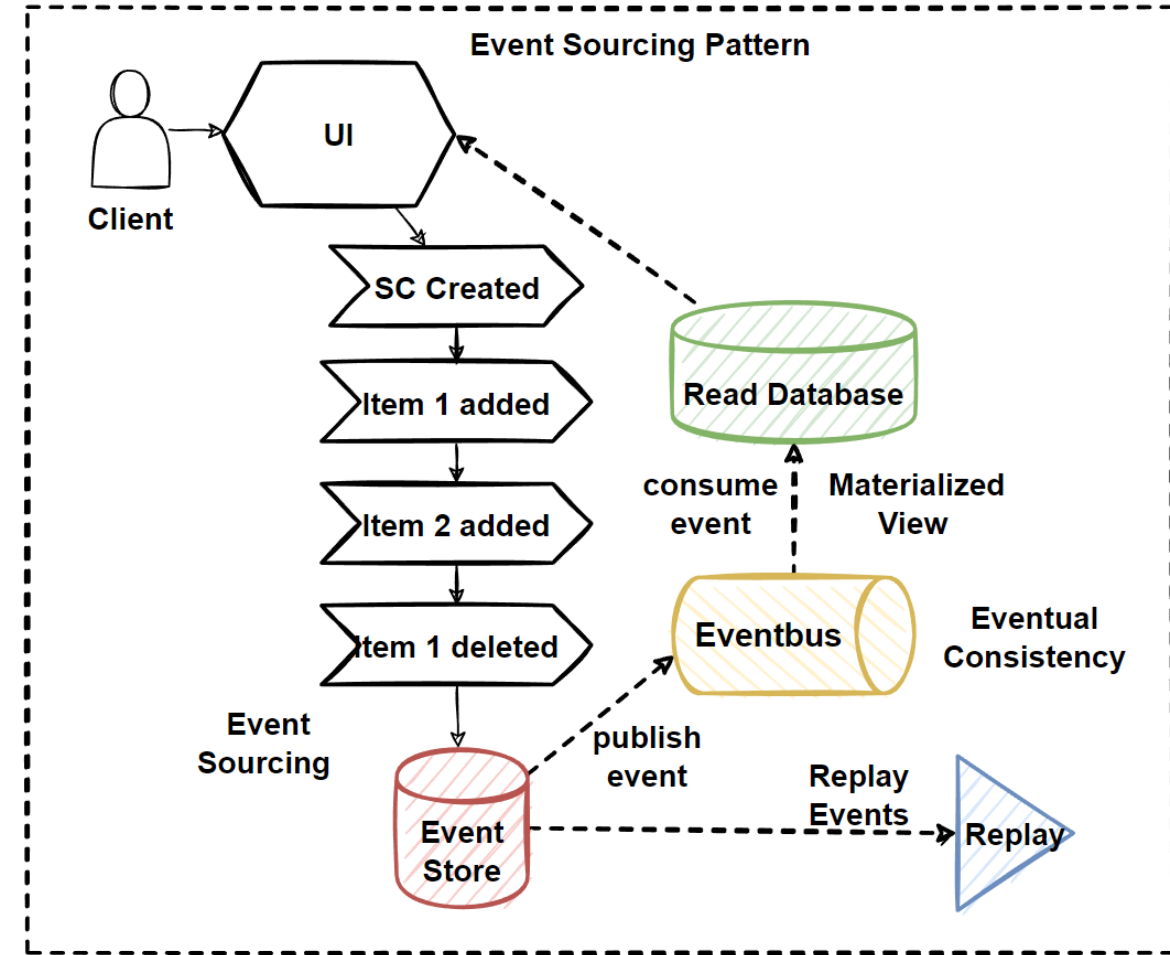


Eventual Consistency Principle - 2

CQRS Design Pattern and Event Sourcing patterns:

1. When user perform any action into application, this will save actions **as a event into event store**.
2. Data will **convert to Reading database** with following the publish/subscribe pattern with using message brokers.
3. Data will be **denormalized into materialized view database** for querying from the application.

We call this process is a **Eventual Consistency Principle**.



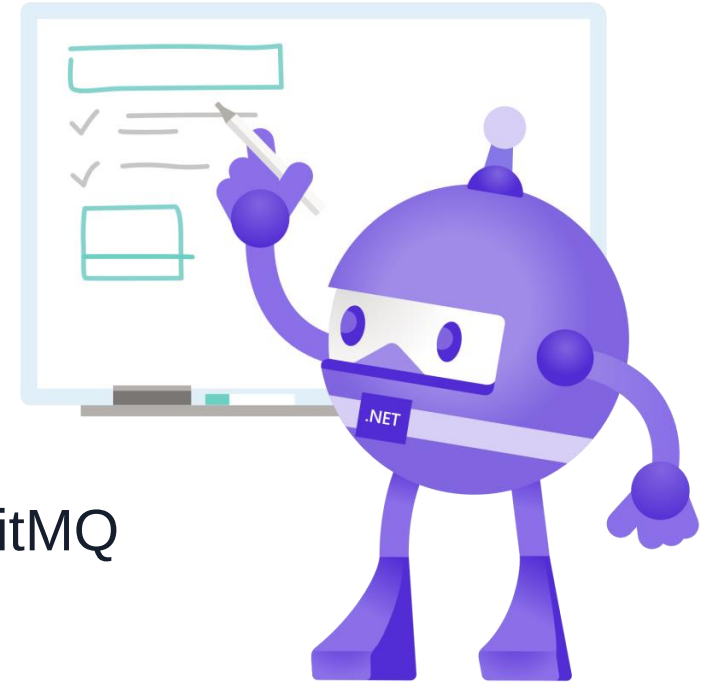
2. Application Use Cases of Ordering Microservices

CRUD Ordering Operations:

- Get Order with Items (can filter by Name and Customer)
- Create a new Order
- Update an existing Order
- Delete Order
- Add and Remove Item from Order

Asynchronous Ordering Operations:

- **Basket Checkout:** Consume Basket Checkout event from RabbitMQ using MassTransit
- **Order Fulfilment:** Perform order fulfilment operations (billing, shipment, notification..)
- Raise OrderCreated Domain Event that leads to integration event



2. Patterns & Principles of Ordering Microservices

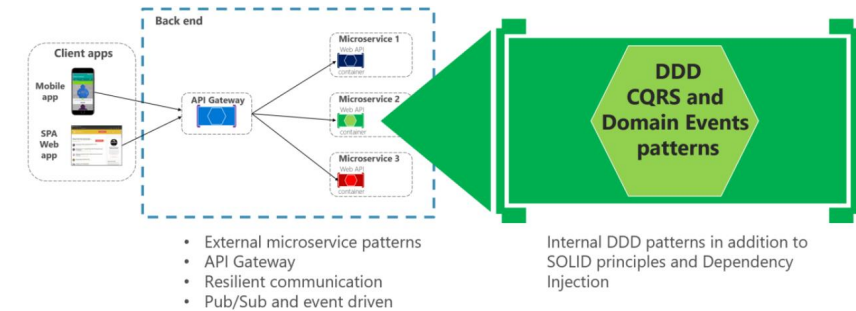
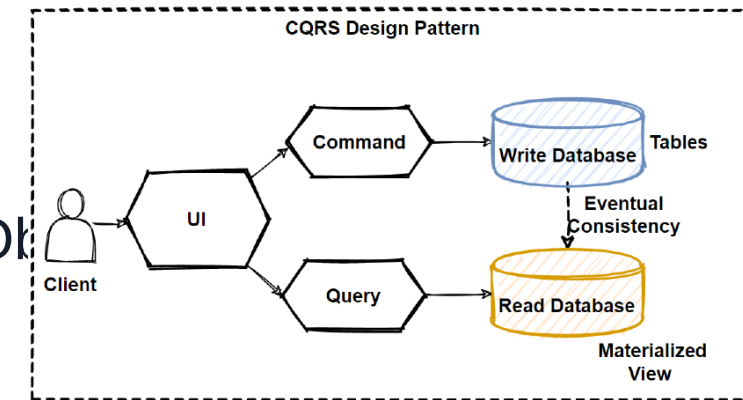


Infrastructure Data Layer Patterns

- Repository Pattern
- EF Core ORM - Code First Approach - Migrations - Seeding Database
- Value Object Complex Types, EF Aggregate Root Entities
- Entity Configurations w/ ModelBuilder - DDD to EF Core Entity OK
- Raise & Dispatch Domain Events with EF Core and MediatR

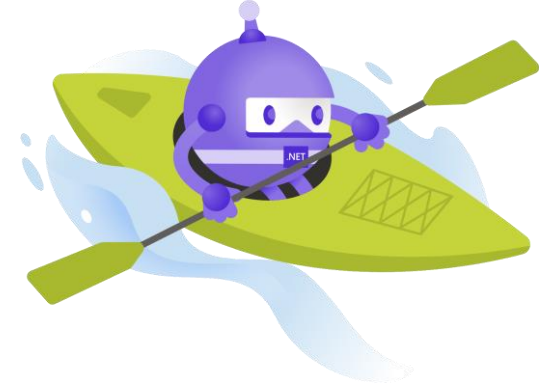
Application UC Layer Patterns

- CQRS and CQS Pattern
- Command and Command Handler patterns
- Mediator Pattern, MediatR Pipeline Behaviours
- Fluent Validation, Logging Cross-cutting concerns



Folder Structure of Application Layer

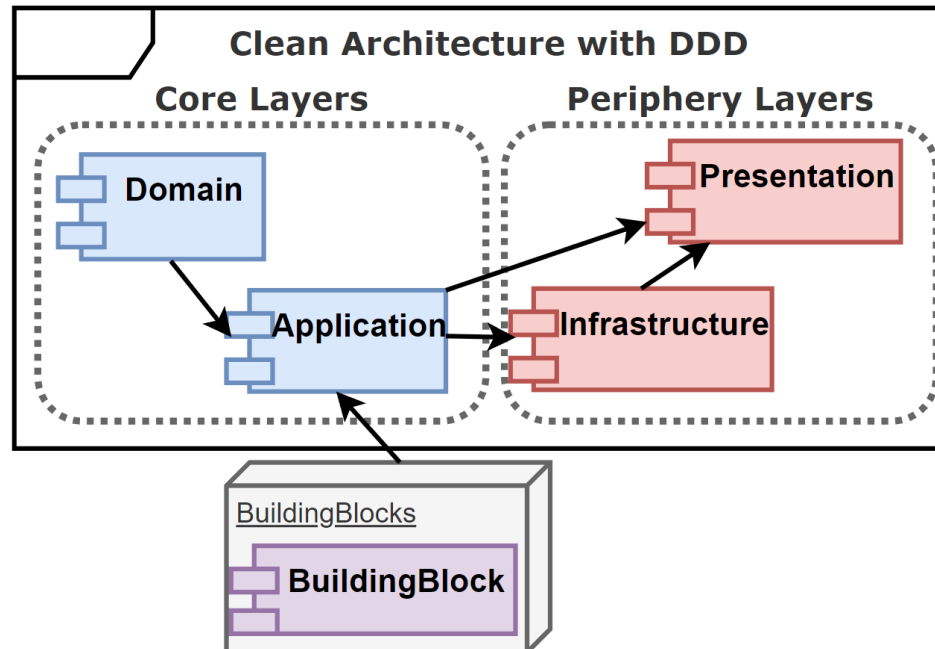
```
Ordering.Application
├── Dependencies
├── Data
│   ├── IApplicationDbContext.cs
├── Dtos
├── Exceptions
│   ├── OrderNotFoundException.cs
├── Extentions
│   ├── OrderExtentions.cs
├── Orders
│   ├── Commands
│   │   ├── CreateOrder
│   │   │   ├── CreateOrderCommand.cs
│   │   │   ├── CreateOrderHandler.cs
│   │   ├── DeleteOrder
│   │   ├── UpdateOrder
│   ├── EventHandlers
│   │   ├── Domain
│   │   │   ├── OrderCreatedEventHandler.cs
│   │   │   ├── OrderUpdatedEventHandler.cs
│   │   ├── Integration
│   │   │   ├── BasketCheckoutEventHandler.cs
│   ├── Queries
│   │   ├── GetOrderByName
│   │   ├── GetOrders
│   │   ├── GetOrdersByCustomer
├── DependencyInjection.cs
└── GlobalUsing.cs
```



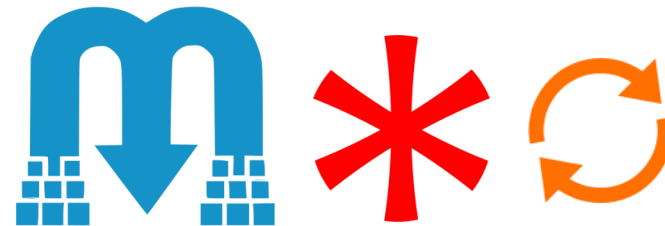
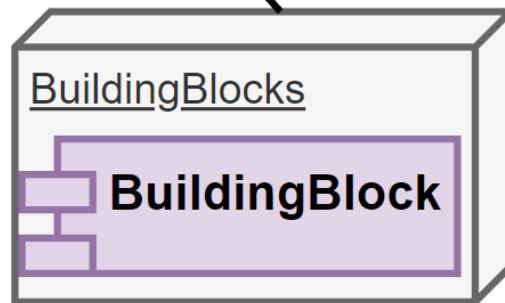
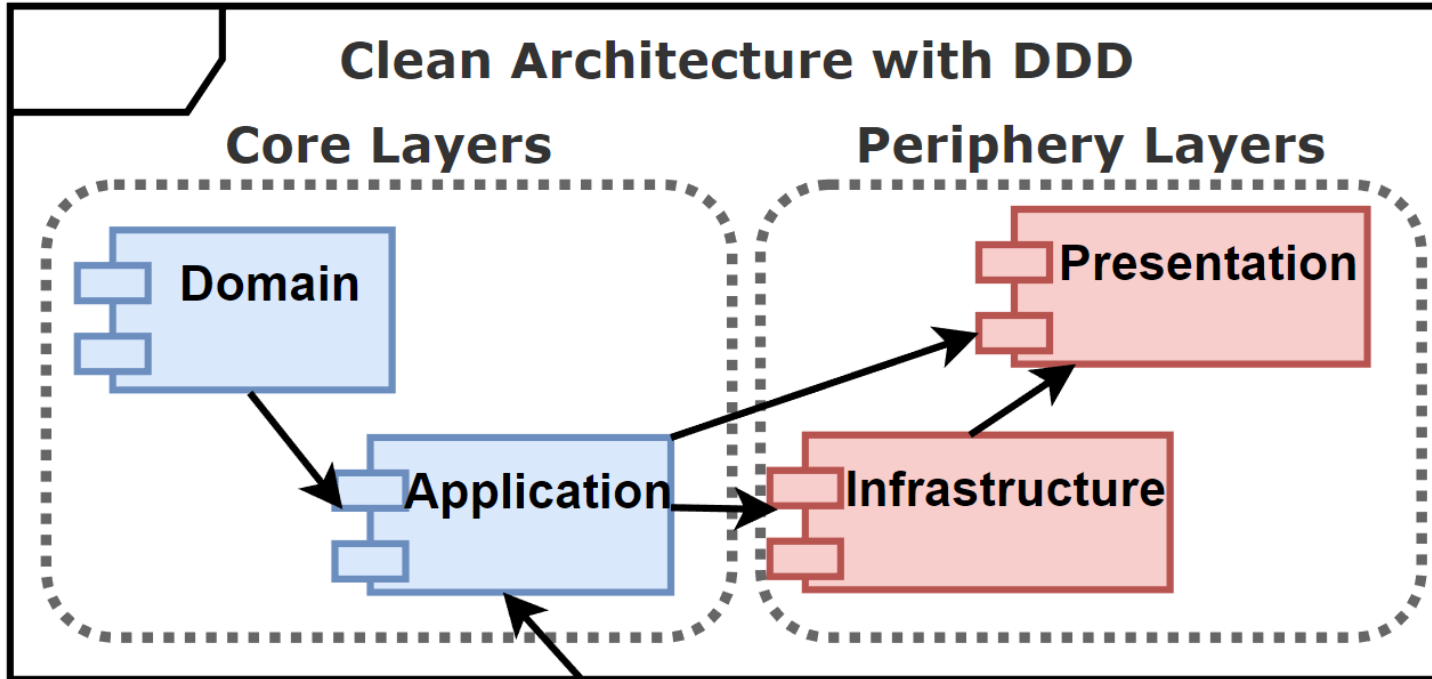
Libraries Nuget Packages of Application Layer

Application UC Layer Libraries:

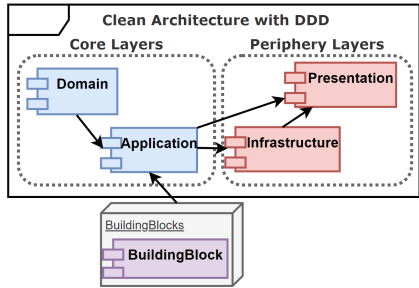
- Libraries from BuildingBlocks (e.g., MediatR, FluentValidation).
 - MediatR.Extensions.Microsoft.DependencyInjection
 - FluentValidation
 - FluentValidation.DependencyInjectionExtensions
 - Mapster for object mapping.
 - Microsoft.Extensions.Logging.Abstractions



BuildingBlocks Common Libraries for Application Layer

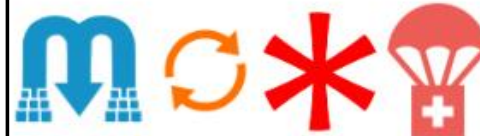
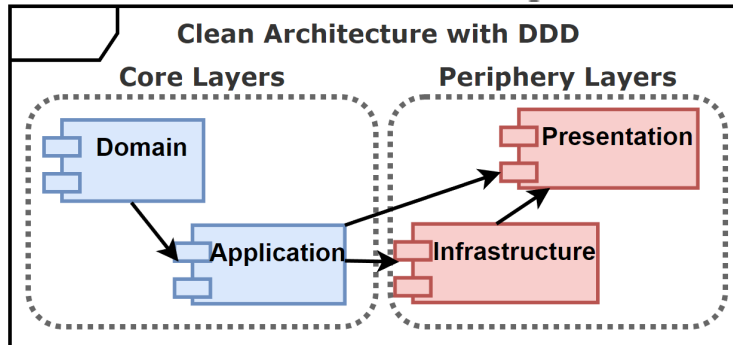
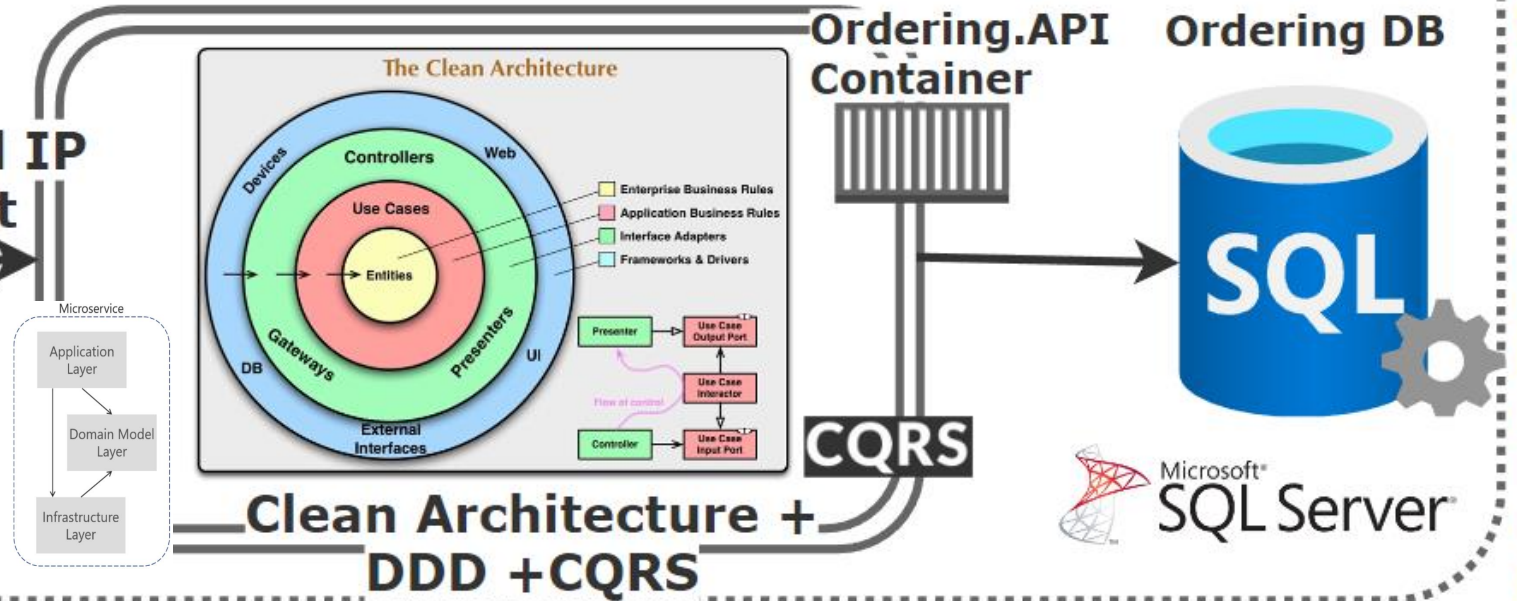
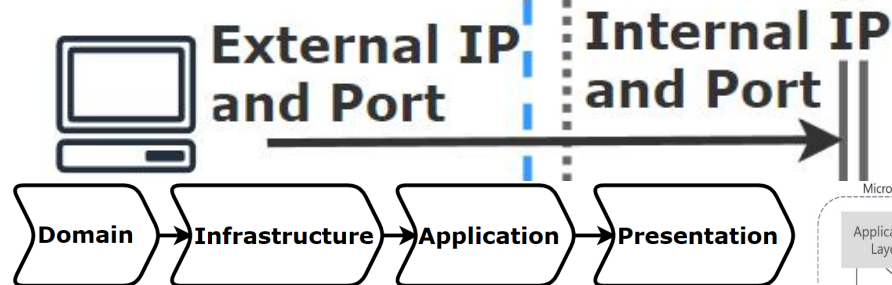


Ordering Microservice w/ DDD, CQRS and Clean Architecture

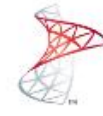


Ordering Microservice

Internal "Ordering" Microservice Architecture



Entity Framework



Microsoft SQL Server

ASP.NET

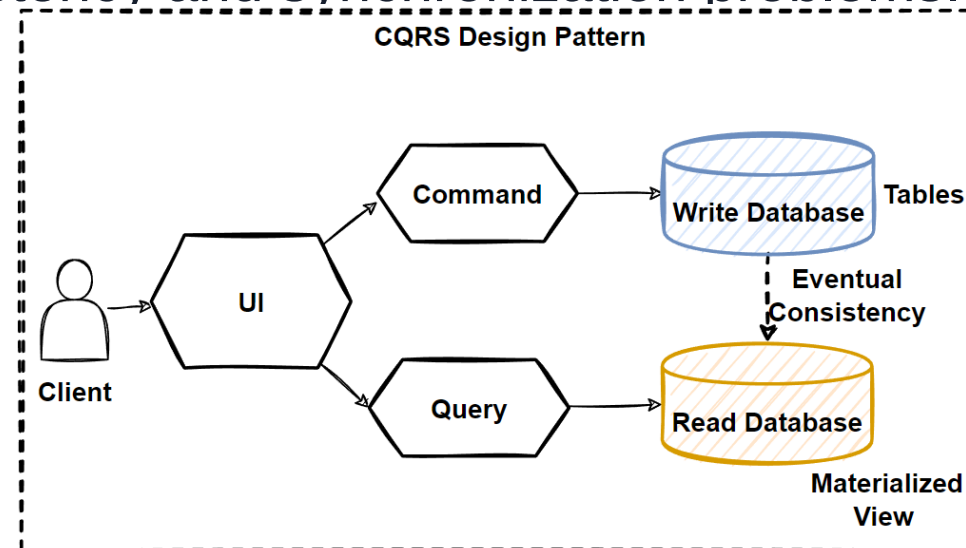


.NET 8

mehmetozkaya

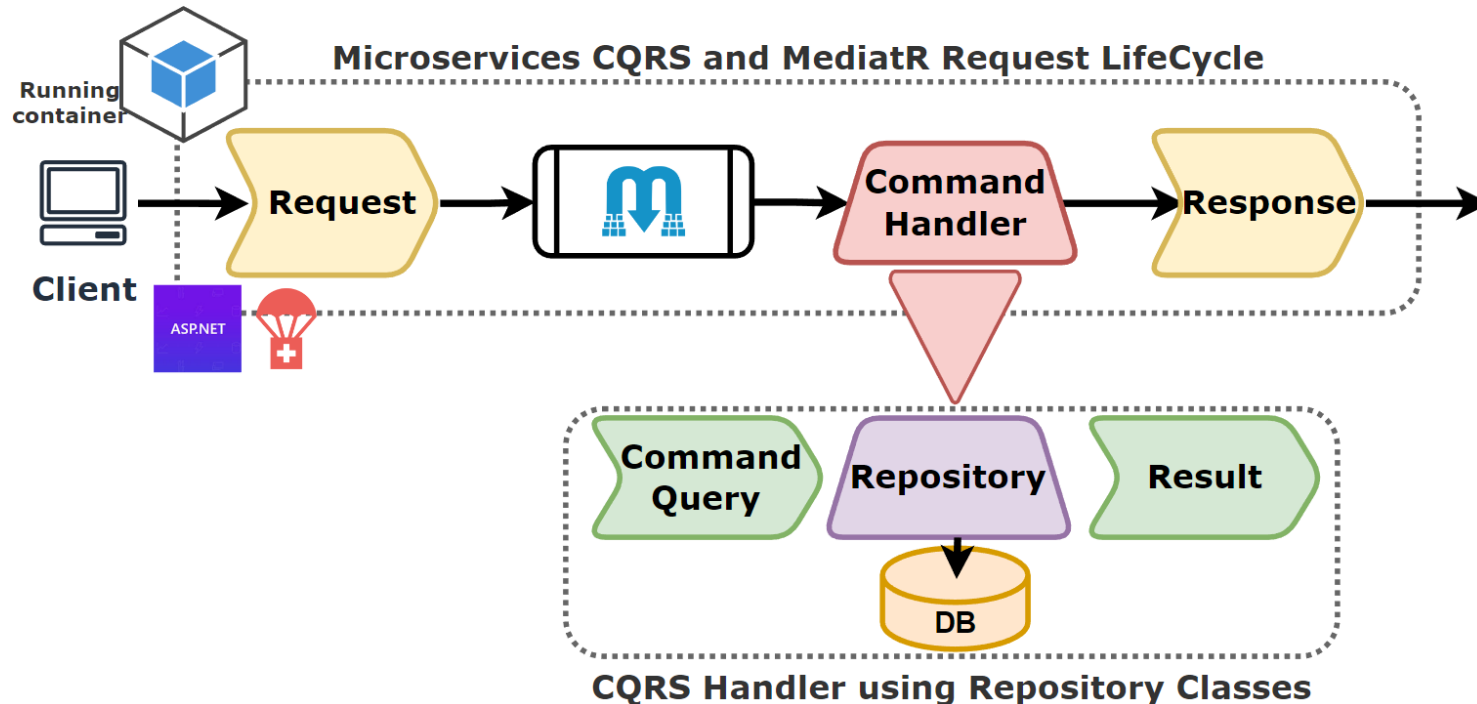
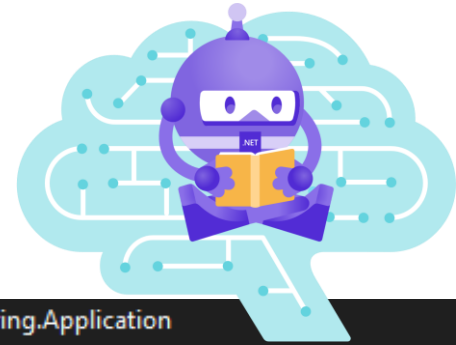
Logical and Physical Implementation of CQRS

- **Logical Implementation of CQRS:** Splitting Operations, Not Databases. Separate the read (query) operations from the write (command) operations at the code level, but not necessarily at the database level.
- Even though the same database is used, the paths for reading and writing data are distinct.
- **Physical Implementation of CQRS:** Separate Databases. Splitting the read and write operations not just at the code level but also physically using separate databases.
- Introduces data consistency and synchronization problems.



Command and Command Handler in CQRS Ordering.Application Layer

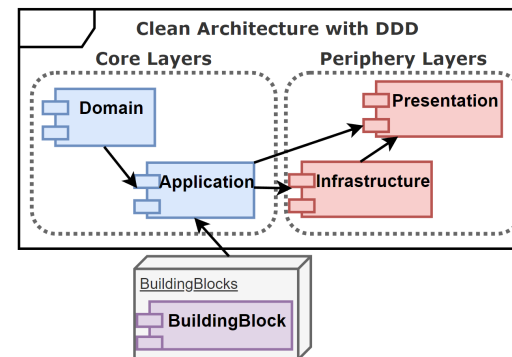
- **Commands:** Change the state of the system. encapsulates the data required for a specific action.
- **Command Handlers:** Each command has a corresponding handler. The handler interacts with the domain model and persists changes.
- Executes business logic and updates the state through repositories.



```
Ordering.Application
├── Dependencies
├── Data
│   └── IApplicationDbContext.cs
├── Dtos
├── Exceptions
│   └── OrderNotFoundException.cs
├── Extensions
│   └── OrderExtensions.cs
├── Orders
│   ├── Commands
│   │   ├── CreateOrder
│   │   │   ├── CreateOrderCommand.cs
│   │   │   └── CreateOrderHandler.cs
│   │   ├── DeleteOrder
│   │   └── UpdateOrder
│   ├── EventHandlers
│   │   ├── Domain
│   │   │   ├── OrderCreatedEventHandler.cs
│   │   │   └── OrderUpdatedEventHandler.cs
│   │   └── Integration
│   │       └── BasketCheckoutEventHandler.cs
│   └── Queries
│       ├── GetOrderByName
│       ├── GetOrders
│       └── GetOrdersByCustomer
├── DependencyInjection.cs
└── GlobalUsing.cs
```

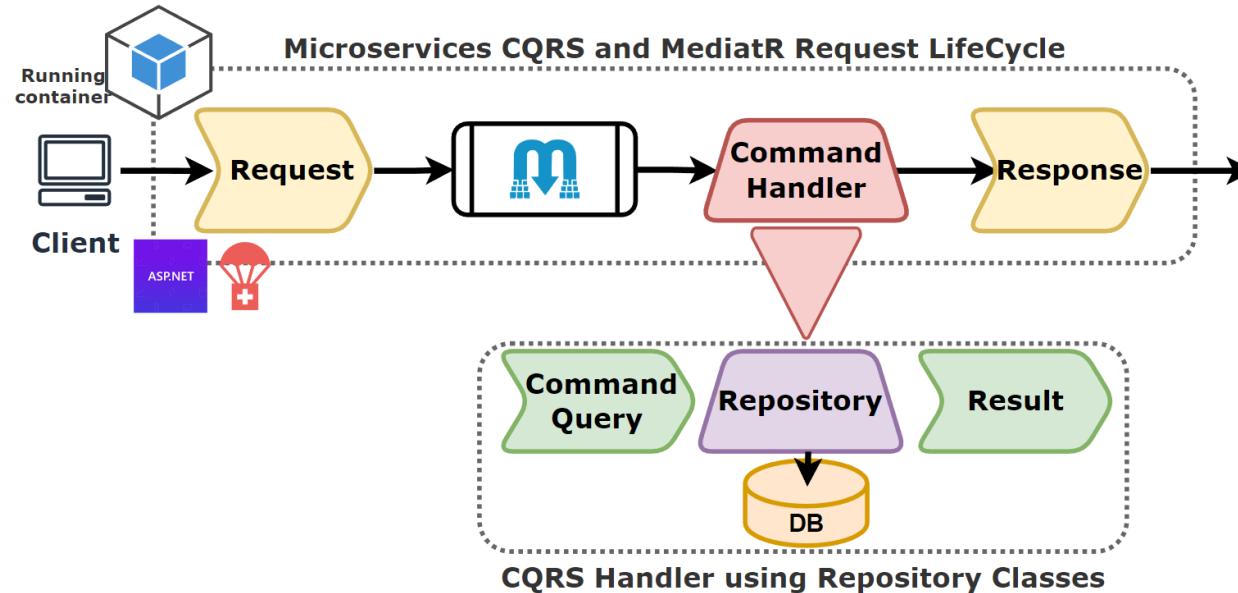
CQRS Design Pattern With MediatR Library

- **MediatR** is a popular .NET library used to implement the mediator pattern, which is a great fit for CQRS.
- **IRequest interface** is used to define a request, which can be either a command or a query. The return type of the request can be specified as a generic parameter.
- **Handler** inherits from **IRequestHandler<TRequest, TResponse>**, where TRequest is the type of the command or query, and TResponse is the return type.
- To make the distinction between commands and queries clearer, you can define two custom interfaces:
 ICommand and **IQuery**.
 - `public interface ICommand<TResult> : IRequest<TResult> { }`
 - `public interface IQuery<TResult> : IRequest<TResult> { }`



MediatR and ASP.NET Core Minimal Apis

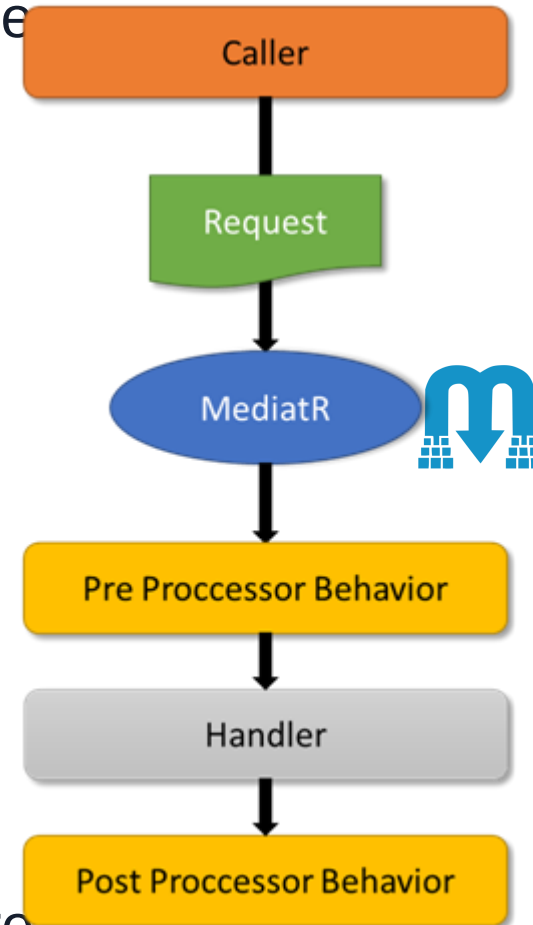
- ASP.NET Core using MediatR, minimal apis act as the **entry point** for handling HTTP requests. Instead of implementing the business logic directly, these actions **delegate** the **responsibility** to MediatR. (**REPR pattern**)



- Minimal Api** receives a request, it **creates** a **command** or **query** object and sends it to **MediatR**. MediatR then **dispatches** this object to the **appropriate handler**.

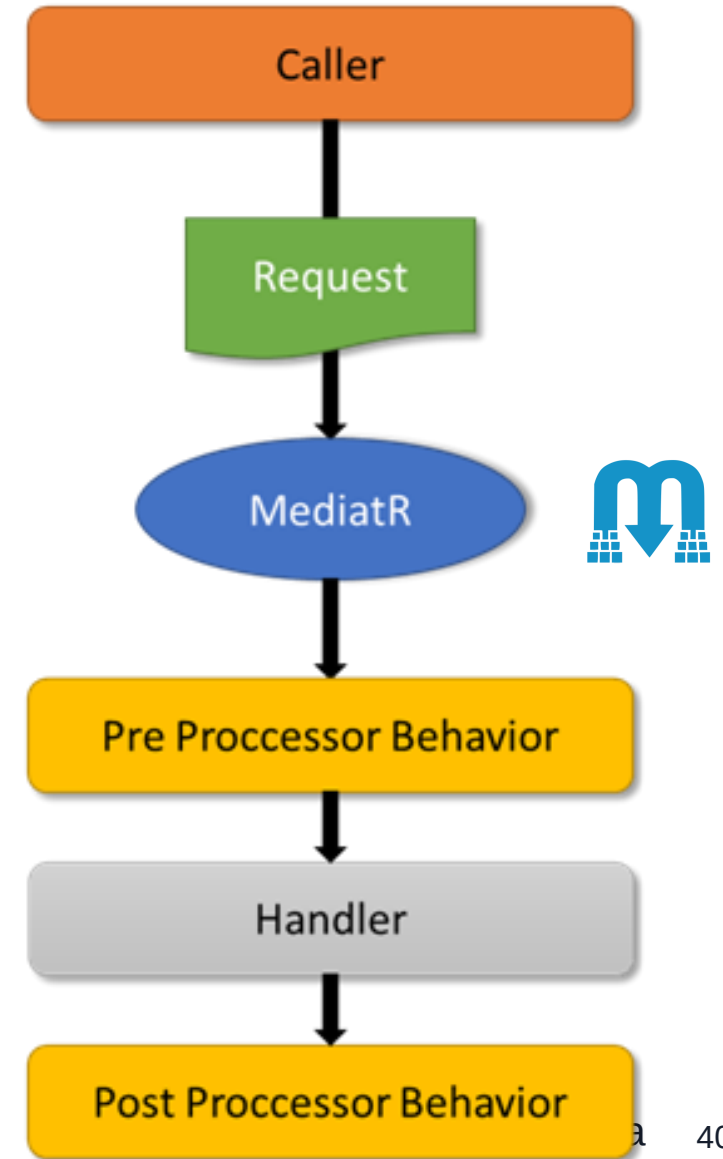
Mediator Pattern and Pipeline Behaviours

- **Mediator pattern** useful in complex or enterprise-level applications where request processing often involves more than just business logic.
- **Handling a request** might require additional steps like **logging**, **validation**, **auditing**, and applying security checks. These are known as cross-cutting concerns.
- **MediatR** provides a **mediator pipeline** where these cross-cutting concerns can be inserted transparently.
- Pipeline **coordinates the request handling**, ensuring that all necessary steps are executed in the right order.
- In MediatR, pipeline behaviors are used to **implement cross-cutting concerns**.
- Wrap around the **request handling**, allowing you to **execute logic** before and after the actual handler is called.



EShop Microservices Pipeline Behaviours

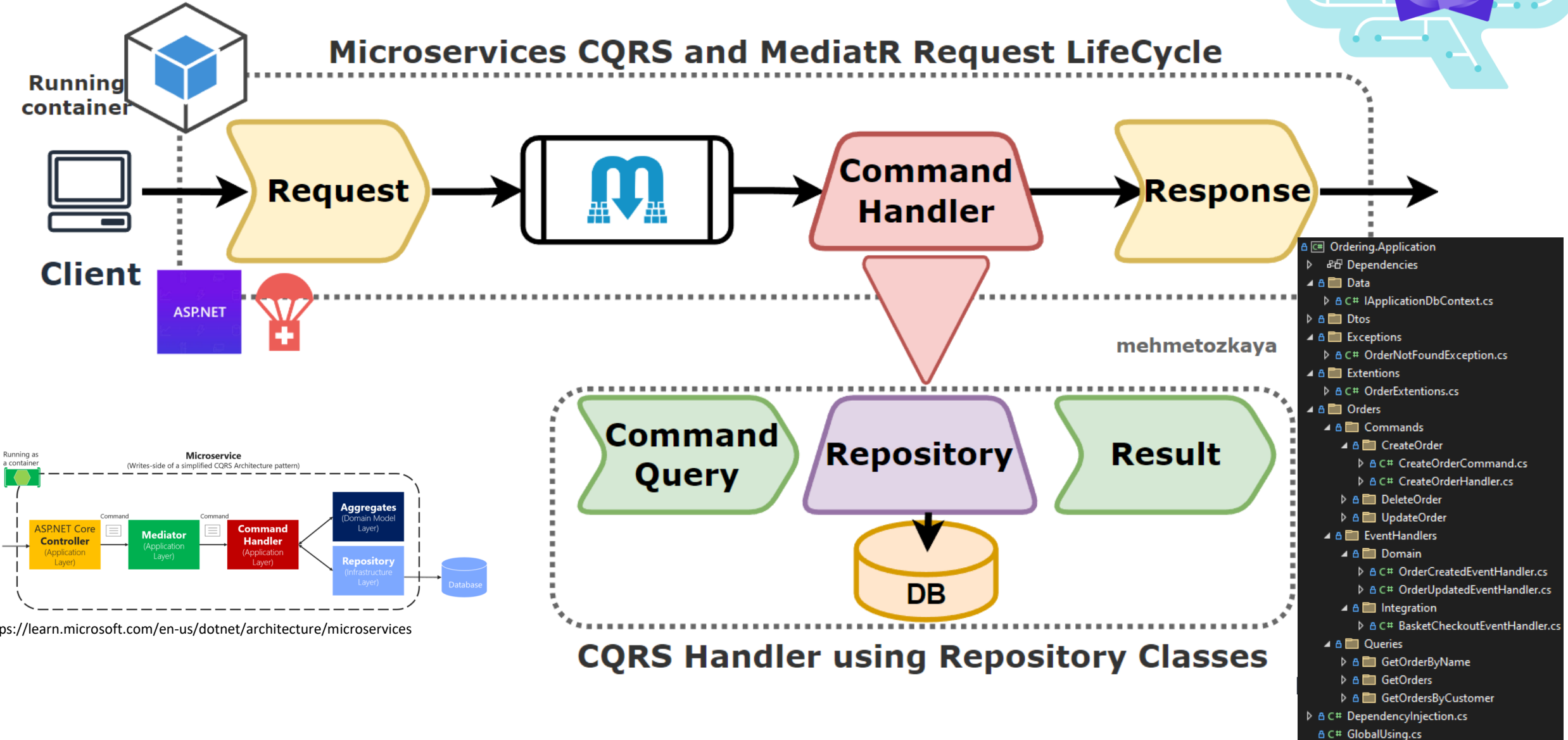
- **LogBehavior:** A behavior that logs details about the handling of a request.
- **ValidatorBehavior:** A behavior that validates incoming requests before they reach the handler.
- Every request processed by MediatR in the application is logged and validated consistently.
- Using MediatR with the mediator pattern in ASP.NET Core applications provides a structured and clean way to handle complex request processing.



CQRS and MediatR Request LifeCycle

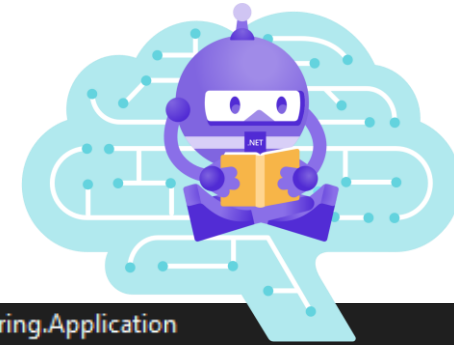
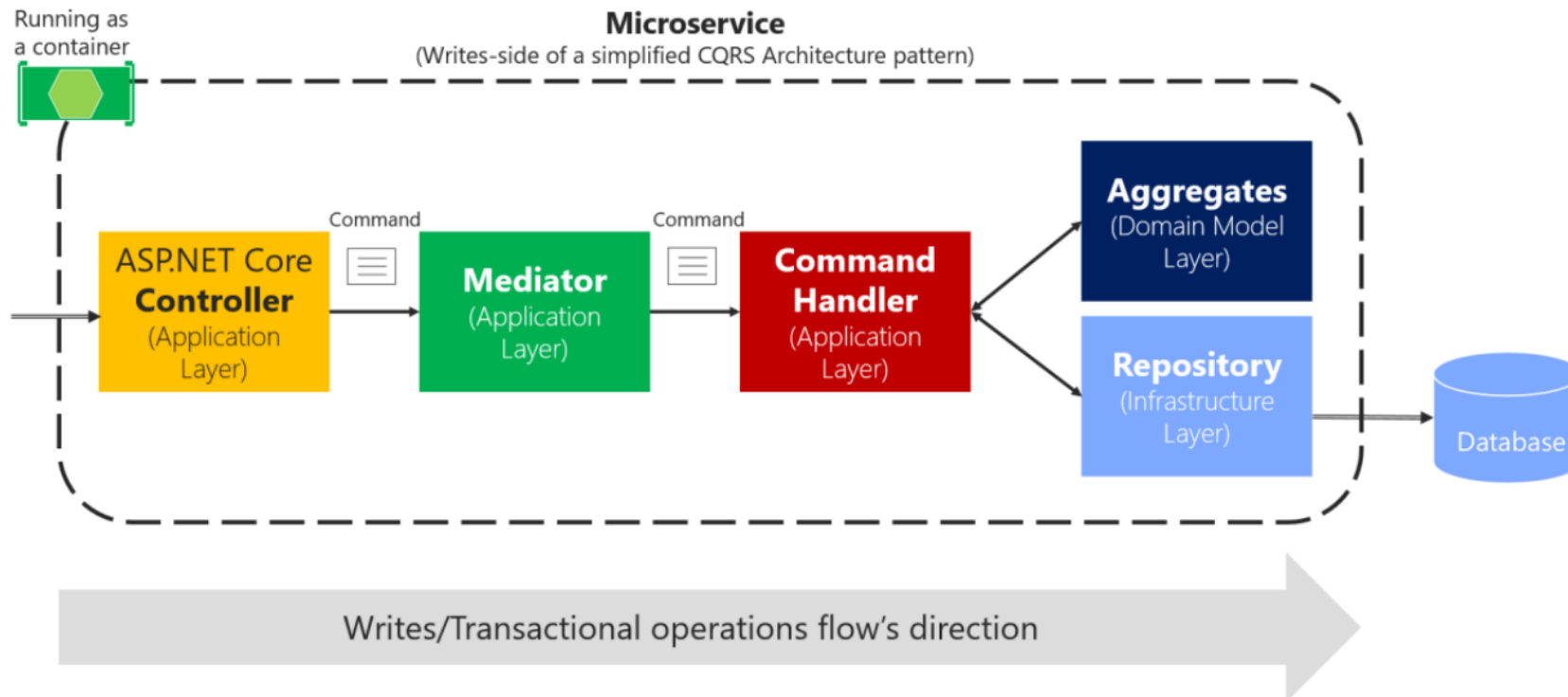


Microservices CQRS and MediatR Request LifeCycle



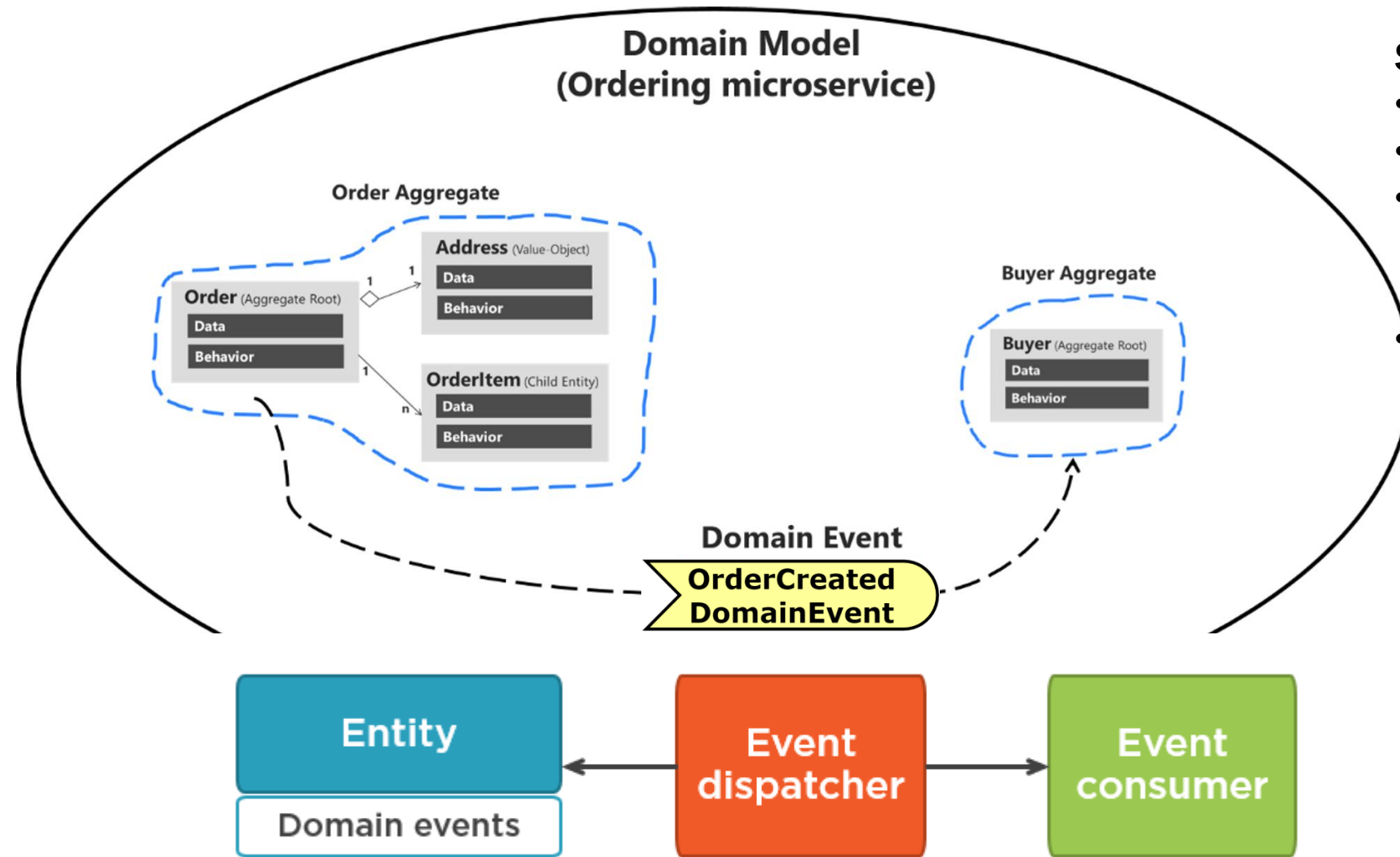
Command and Command Handler in CQRS Ordering.Application Layer

- **Commands:** Change the state of the system. encapsulates the data required for a specific action.
- **Command Handlers:** Each command has a corresponding handler.. The handler interacts with the domain model and persists changes.
- Executes business logic and updates the state through repositories.



```
Ordering.Application
├── Dependencies
├── Data
│   └── IApplicationDbContext.cs
├── Dtos
├── Exceptions
│   └── OrderNotFoundException.cs
├── Extentions
│   └── OrderExtentions.cs
├── Orders
│   ├── Commands
│   │   ├── CreateOrder
│   │   │   ├── CreateOrderCommand.cs
│   │   │   └── CreateOrderHandler.cs
│   │   ├── DeleteOrder
│   │   └── UpdateOrder
│   ├── EventHandlers
│   │   ├── Domain
│   │   │   ├── OrderCreatedEventHandler.cs
│   │   │   └── OrderUpdatedEventHandler.cs
│   │   └── Integration
│   │       └── BasketCheckoutEventHandler.cs
│   └── Queries
│       ├── GetOrderByName
│       ├── GetOrders
│       └── GetOrdersByCustomer
├── DependencyInjection.cs
└── GlobalUsing.cs
```

Dispatch Domain Events w/ EF Core SaveChangesInterceptor

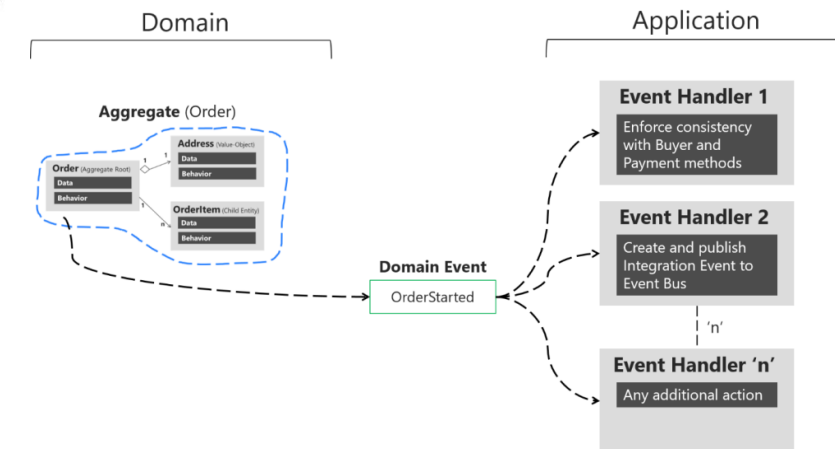


Steps dispatch Events:

- Create a SaveChangesInterceptor
- Identify and Dispatch Domain Events
- public class

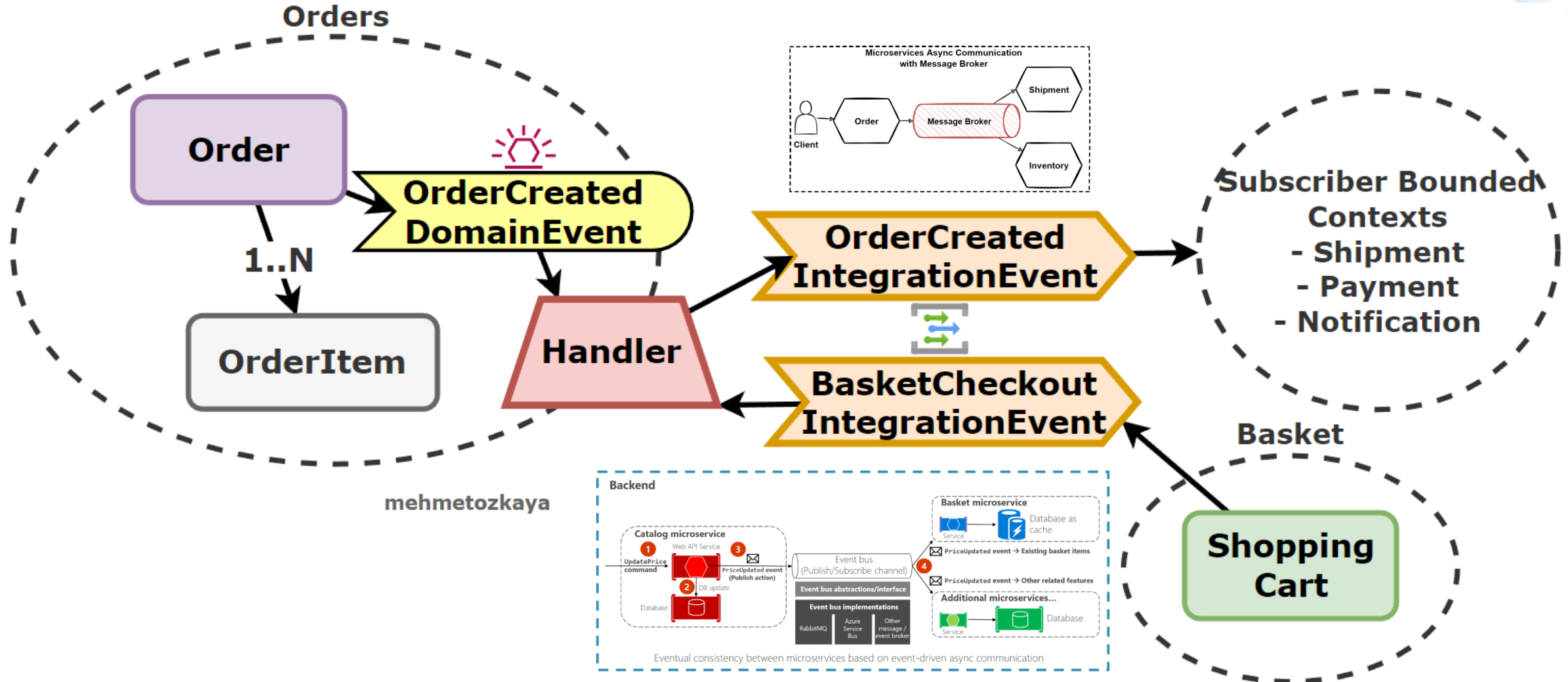
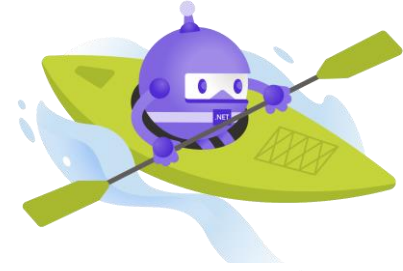
DomainEventDispatcherInterceptor :
SaveChangesInterceptor

- Registering the Interceptor into DependencyInjection class



Best Practice to dispatch Events:
EF Core SaveChangesInterceptor

Domain Events Leads to Integration Events



<https://learn.microsoft.com/en-us/dotnet/architecture/microservices>

Pagination with EF Core



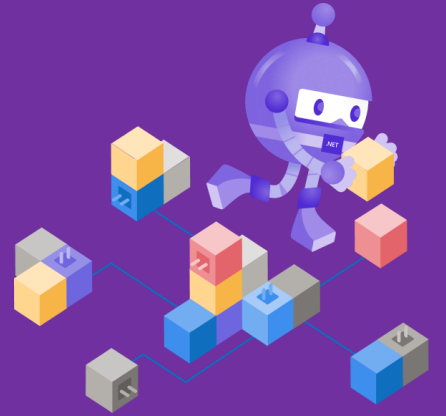
- **EF Core** supports pagination through the Skip and Take LINQ methods.
- Allow us to **retrieve only a subset of data** corresponding to a specific page.
- **Skip Method:** Skips the first n elements of a sequence.
- **Take Method:** Returns a specified number of contiguous elements from the start of a sequence.
- **Pagination** through a combination of the **Skip** and **Take** methods, which are part of **LINQ**.
- Dividing the **data** into "**pages**" and retrieving only a specific page.



Pagination



Ordering.API Layer Exposing Minimal Api Endpoints with Carter and REPR Pattern



Implement REPR Pattern for CQRS and MediatR Request LifeCycle
Developing Order CRUD Endpoints for REST APIs in Ordering.API Layer
Developing Get Order Endpoints for REST APIs in Ordering.API Layer
Cross-Cutting Concerns Custom Exception Handler
Health Checks for Ordering Microservices with EF.Core and SQL Server
Testing Ordering.API Endpoints with CQRS and Clean Architecture

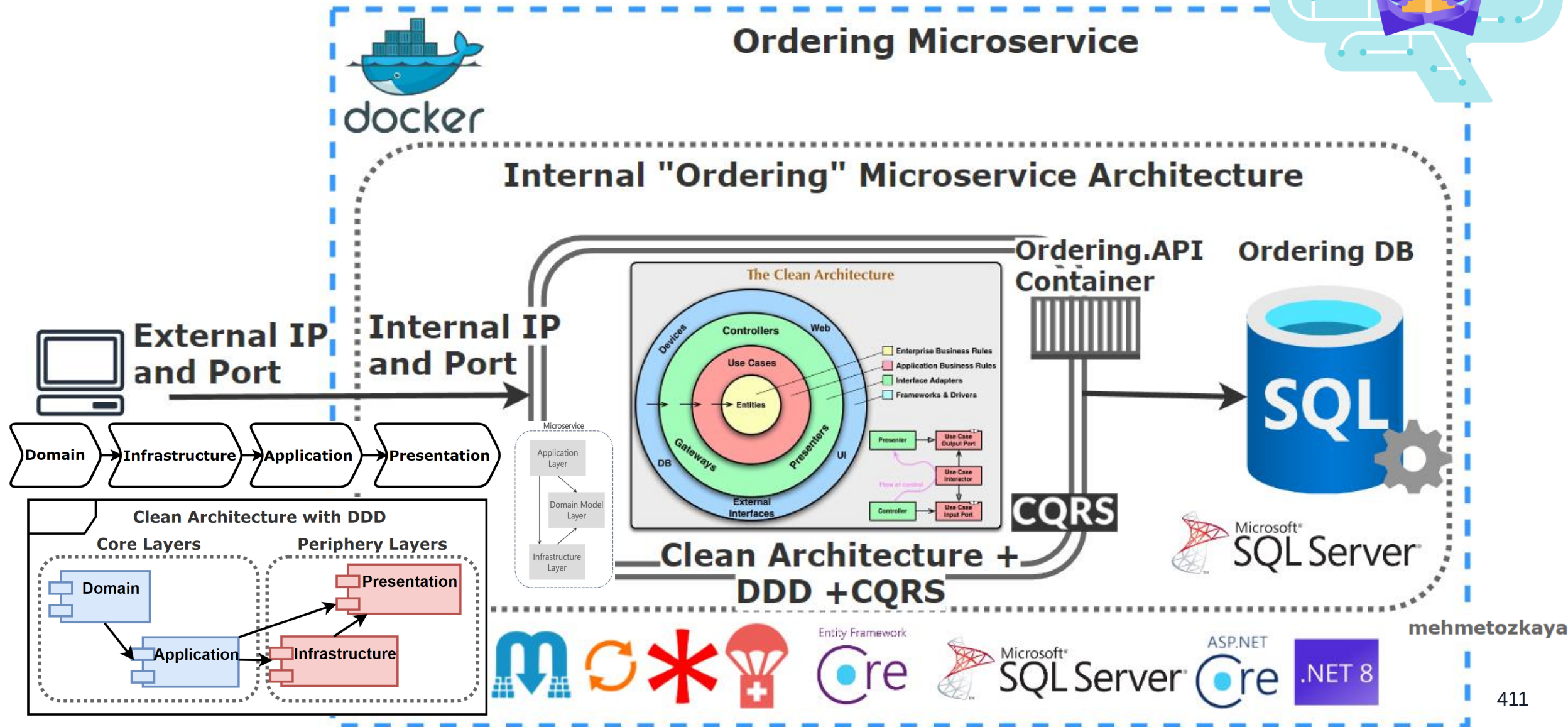
Mehmet Ozkaya



The diagram illustrates a microservices architecture for an e-commerce application. It is divided into three main sections: Client Apps, API Gateways, and Microservices.

- Client Apps:** Includes "Shopping.Web" and a mobile application interface.
- API Gateways:** A "Yarp API Gateway" acts as the central point of entry, routing requests to the microservices.
- Microservices:**
 - Catalog:** Interacts with PostgreSQL.
 - Basket:** Interacts with PostgreSQL and Redis.
 - Discount:** Interacts with SQLite.
 - Ordering:** Interacts with SQL.
- Message Broker:** RabbitMQ is used for asynchronous communication between the Basket and Ordering services.
- Technology Stack:** The bottom of the diagram features logos for .NET 8, ASP.NET Core, gRPC, Entity Framework, and Docker.

Ordering Microservice w/ DDD, CQRS and Clean Architecture



Ordering.API Layer Exposing Minimal Api Endpoints with Carter

- Develop Ordering.API Layer with Exposing Minimal REST APIs w/ Carter following Clean Architecture
- Implement REPR design pattern for CQRS and MediatR Request LifeCycle
- Developing Order CRUD Endpoints for REST APIs in Ordering.API Layer
- Developing Get Order Endpoints for REST APIs in Ordering.API Layer
- Cross-Cutting Concerns Custom Exception Handler in Ordering Microservice
- Health Checks for Ordering Microservices with EF.Core and SQL Server
- Testing Ordering.API Endpoints with CQRS and Clean Architecture Layers



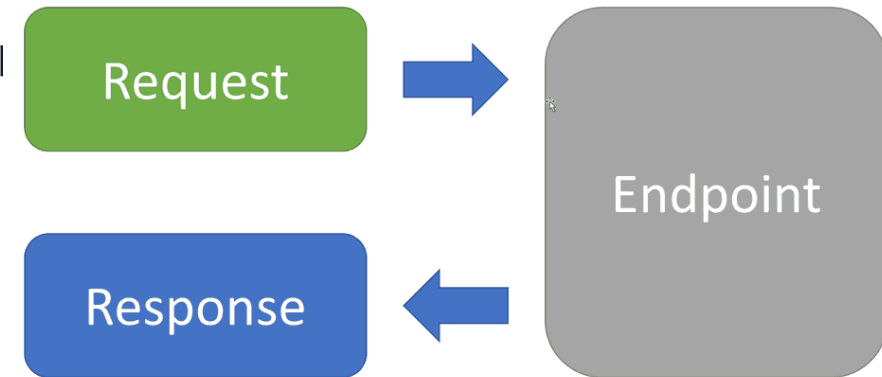
Main Target of API Layer:

- Exposes minimal REST APIs, serving as the entry point for external requests.

REPR Design Pattern



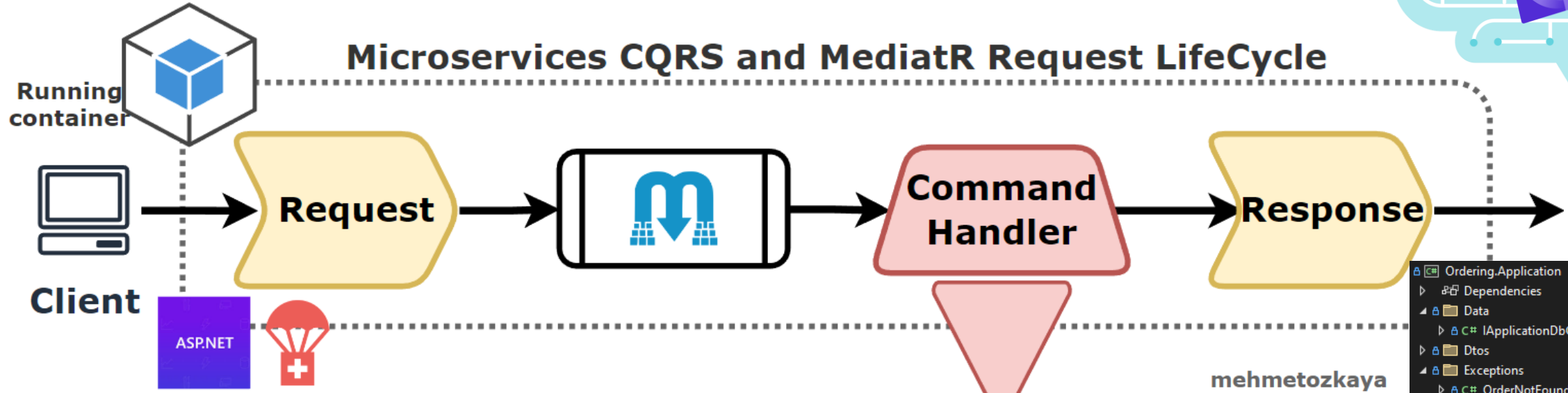
- Developing **API Endpoints** implementing a simple **Request EndPoint Response** flow.
- **REPR** stands for **Request, Endpoint, Response** – a pattern simplifying the development of **REST API endpoints**.
- **REPR Design Pattern** defines web API endpoints as having three components:
 - **Request** - Data structure the endpoint expects.
 - **Endpoint** - Logic the endpoint performs given a request.
 - **Response** - Data structure the endpoint returns.
- Simplified concepts to enable the development REST API endpoints and enforcing the **Single Responsibility Principle** for endpoints.
- **Developers** need to **care** about are the **Request** and **Response**.



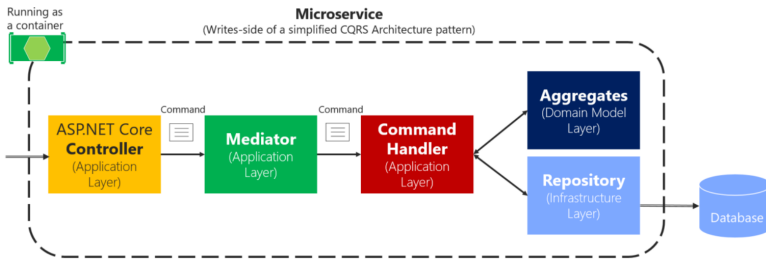
CQRS and MediatR Request LifeCycle



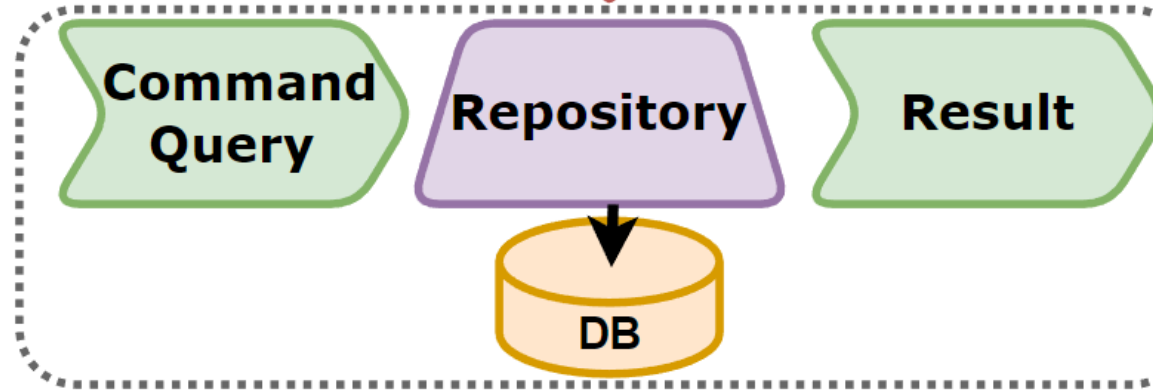
Microservices CQRS and MediatR Request LifeCycle



mehmetozkaya



<https://learn.microsoft.com/en-us/dotnet/architecture/microservices>



CQRS Handler using Repository Classes

```
Ordering.Application
├── Dependencies
├── Data
│   ├── IApplicationDbContext.cs
├── Dtos
├── Exceptions
│   ├── OrderNotFoundException.cs
├── Extensions
│   ├── OrderExtensions.cs
├── Orders
│   ├── Commands
│   │   ├── CreateOrder
│   │   │   ├── CreateOrderCommand.cs
│   │   │   ├── CreateOrderHandler.cs
│   │   ├── DeleteOrder
│   │   ├── UpdateOrder
├── EventHandlers
│   ├── Domain
│   │   ├── OrderCreatedEventHandler.cs
│   │   ├── OrderUpdatedEventHandler.cs
│   ├── Integration
│   │   ├── BasketCheckoutEventHandler.cs
├── Queries
│   ├── GetOrderByName
│   ├── GetOrders
│   ├── GetOrdersByCustomer
├── C# DependencyInjection.cs
├── C# GlobalUsing.cs
```

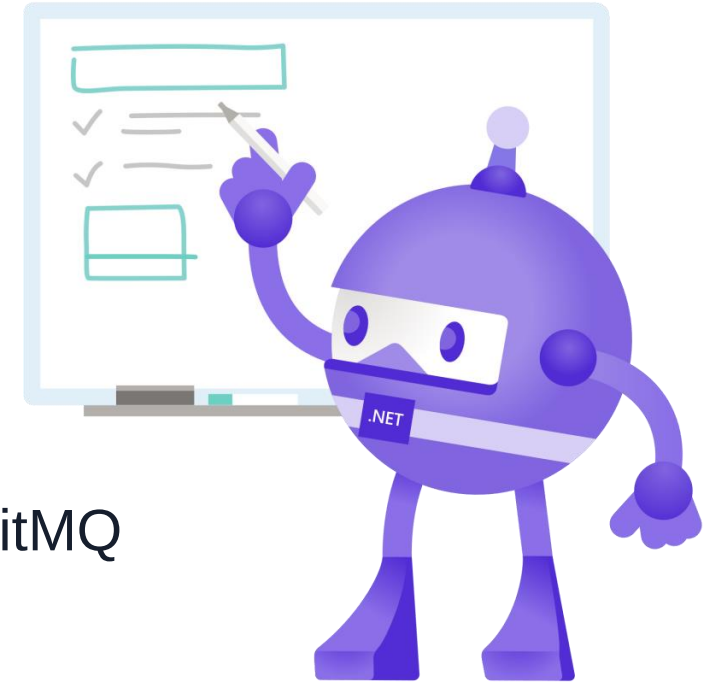
2. Application Use Cases of Ordering Microservices

CRUD Ordering Operations:

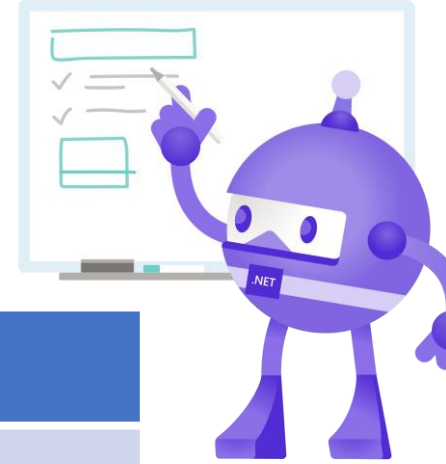
- Get Order with Items (can filter by Name and Customer)
- Create a new Order
- Update an existing Order
- Delete Order
- Add and Remove Item from Order

Asynchronous Ordering Operations:

- **Basket Checkout:** Consume Basket Checkout event from RabbitMQ using MassTransit
- **Order Fulfilment:** Perform order fulfilment operations (billing, shipment, notification..)
- Raise OrderCreated Domain Event that leads to integration event



3. Rest API Endpoints of Ordering Microservices



Method	Request URI	Use Cases
GET	/orders	Get Orders w/ Pagination
GET	/orders/{orderName}	Get Orders by OrderName
GET	/orders/customer/{customerId}	Get Orders by Customer
POST	/orders	Create a new Order
PUT	/orders	Update an existing Order
DELETE	/order/{id}	Delete Order with Id

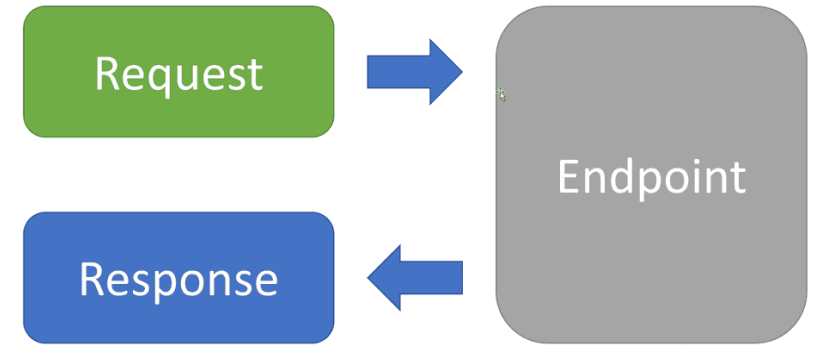
- AMQP: Consume BasketCheckout Event from RabbitMQ using MassTransit

2. Patterns & Principles of Ordering Microservices



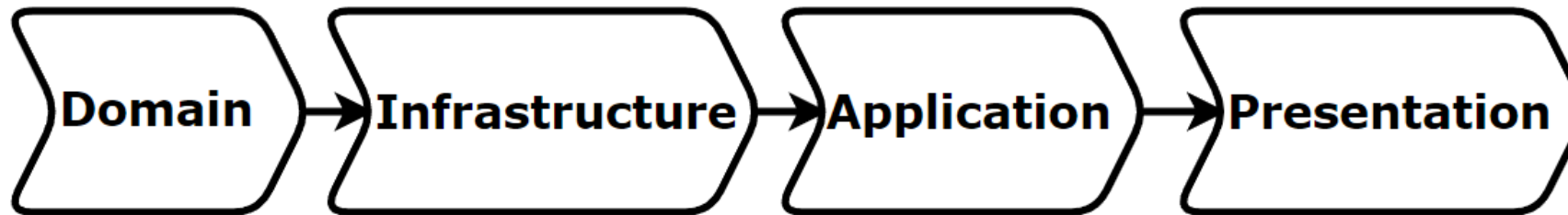
Presentation Api Layer Patterns

- Minimal APIs
- REPR Design Pattern
- Asp.net DI, Routing with latest Asp.net 8 features

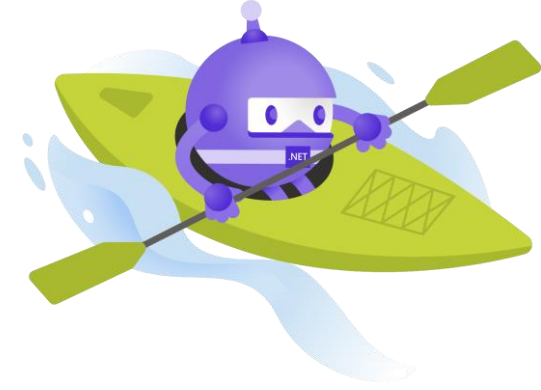


SECTION FLOW

- Domain > Infrastructure > Application > Presentation



Folder Structure of API Layer



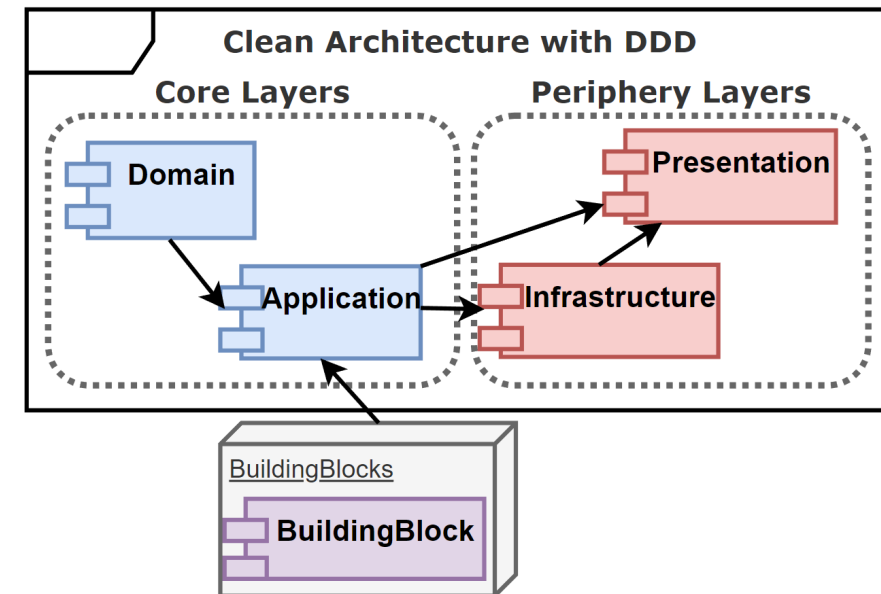
```
└─ Ordering
  └─ Ordering.API
    ├── Connected Services
    ├── Dependencies
    ├── Properties
    └─ Endpoints
      ├── C# CreateOrder.cs
      ├── C# DeleteOrder.cs
      ├── C# GetOrders.cs
      ├── C# GetOrdersByCustomer.cs
      ├── C# GetOrdersByName.cs
      └── C# UpdateOrder.cs
    ├── appsettings.json
    ├── C# DependencyInjection.cs
    ├── Dockerfile
    ├── C# GlobalUsing.cs
    └── C# Program.cs
```


Libraries Nuget Packages of Application Layer

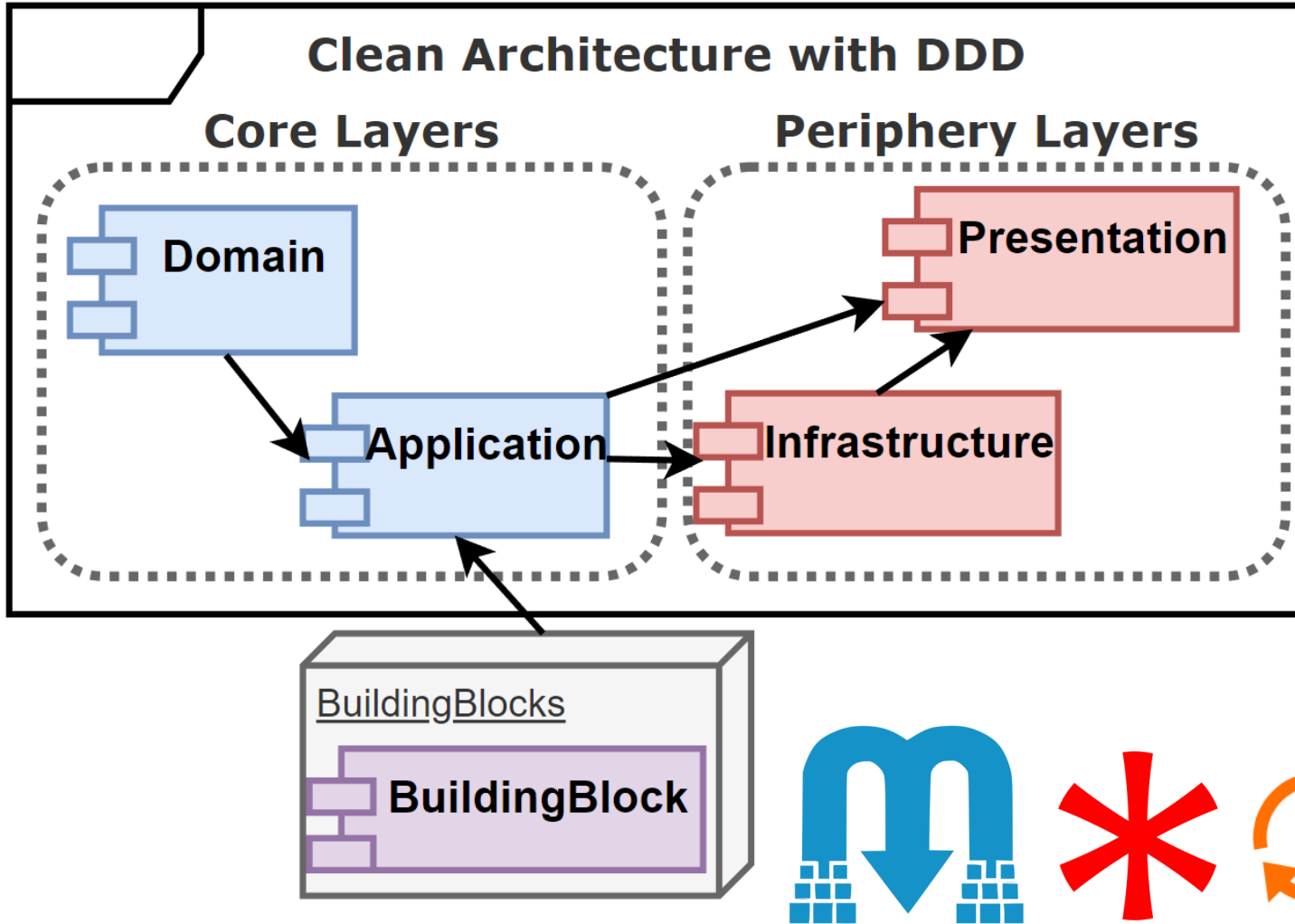


Presentation Api Layer Libraries:

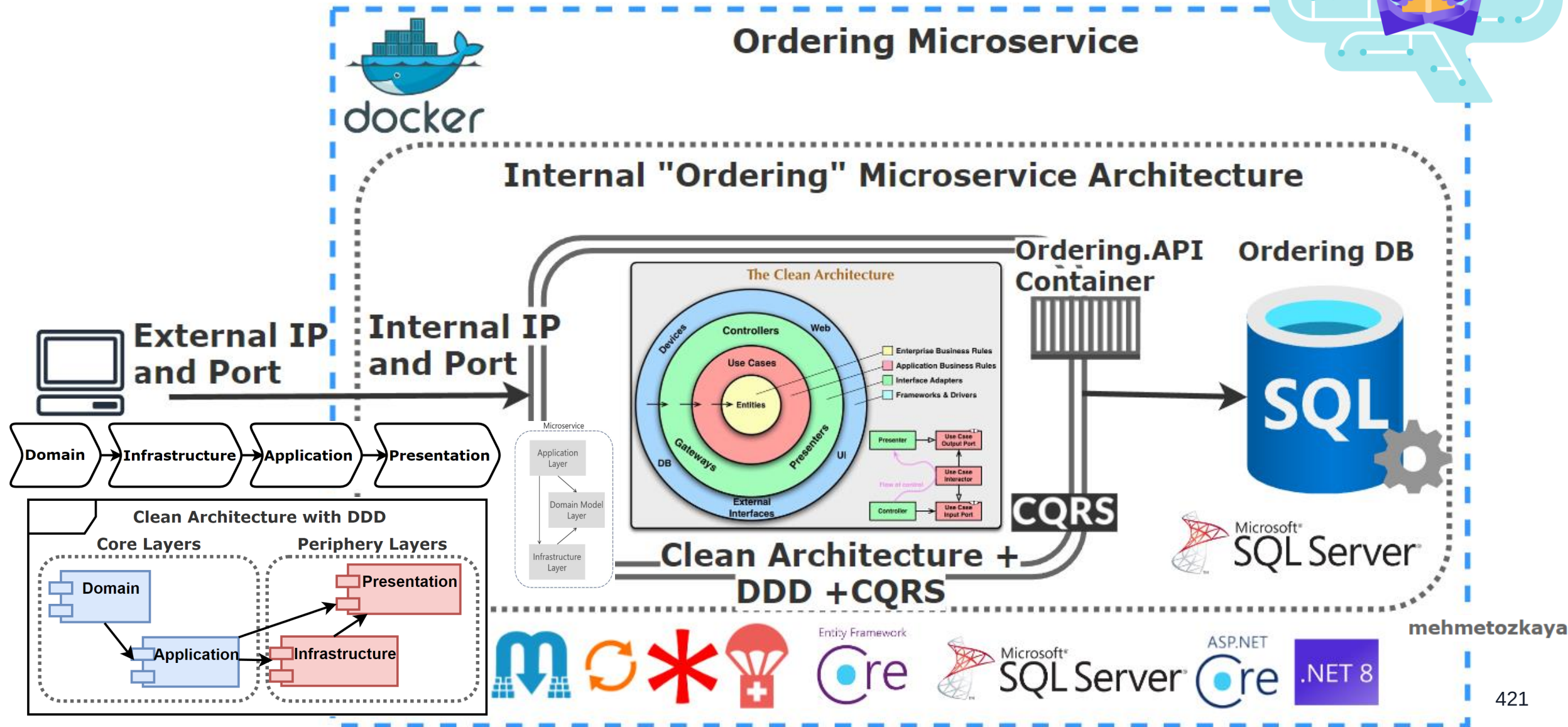
- ASP.NET Core libraries.
- Carter for defining Minimal API endpoints.
- Building Blocks comes from Application Layer reference that includes MediatR, Mapster and other common libraries.



BuildingBlocks Common Libraries for API Layer

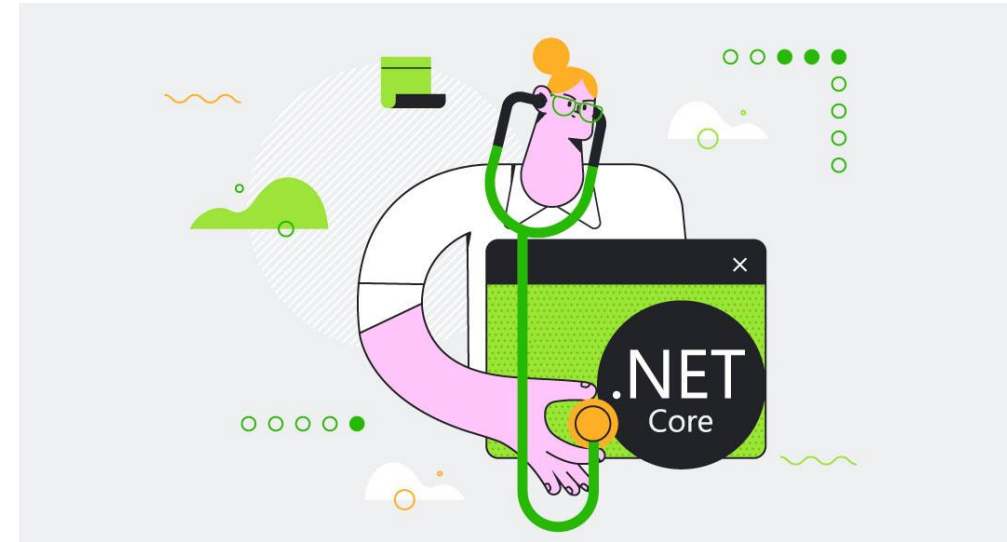


Ordering Microservice w/ DDD, CQRS and Clean Architecture



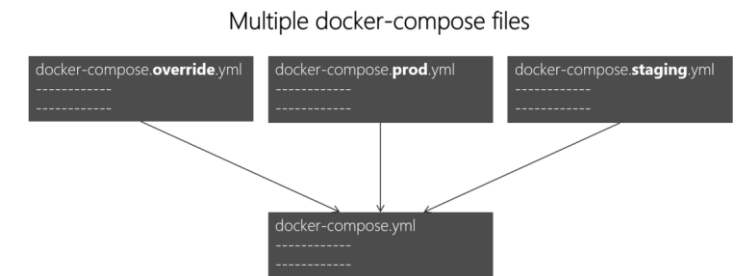
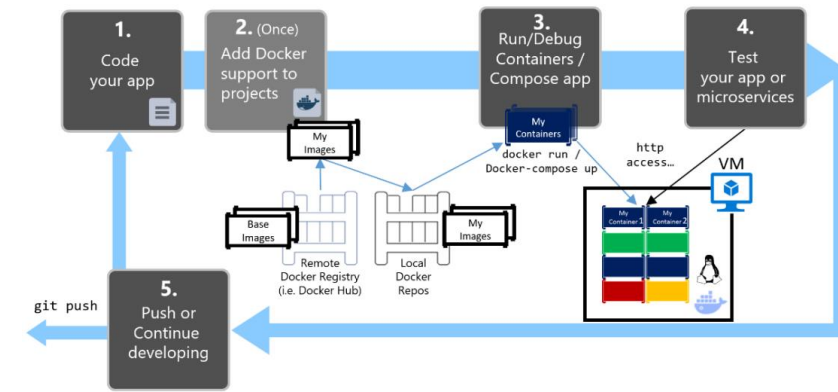
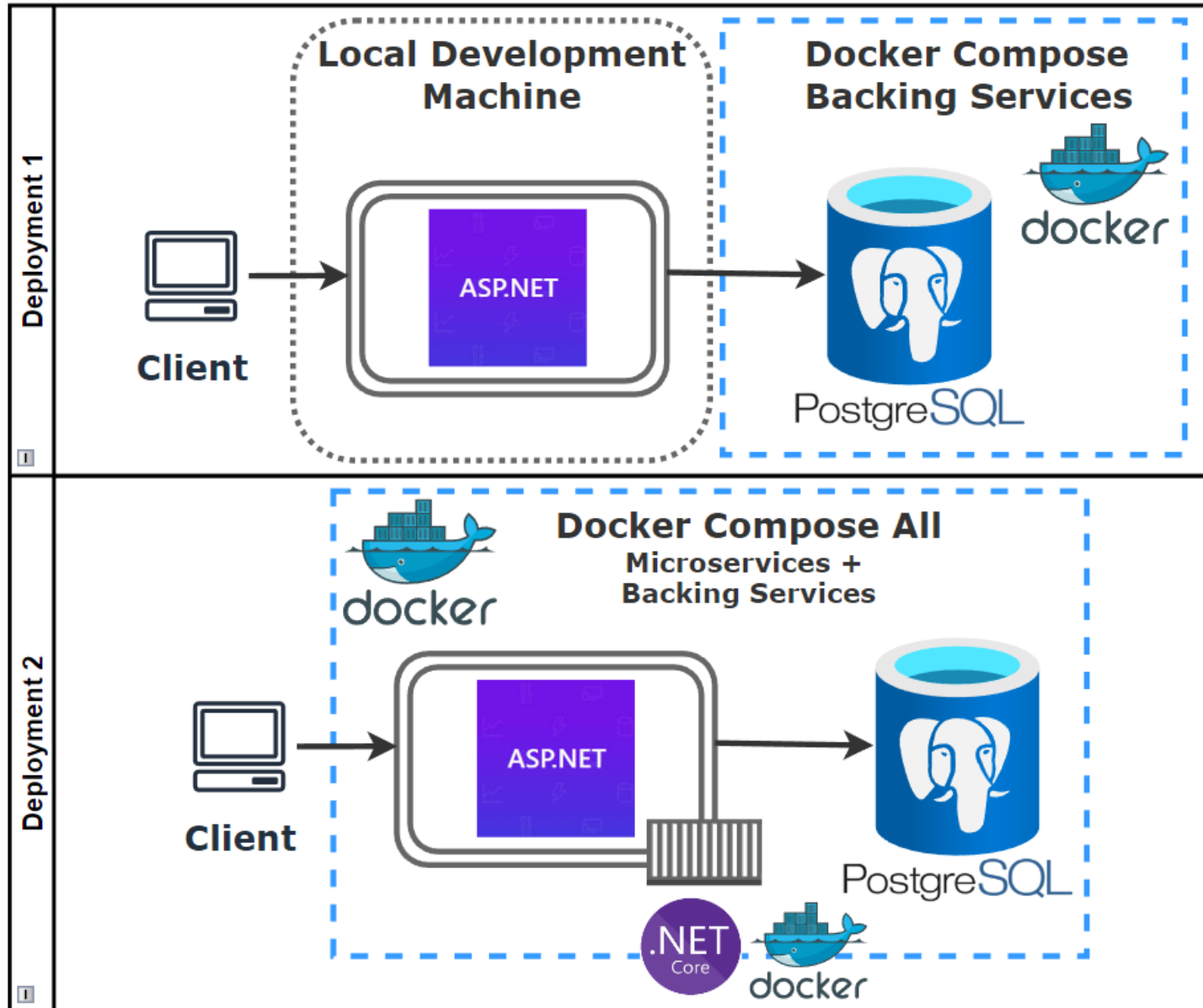
Health checks in ASP.NET Core

- Health checks in ASP.NET Core provide a way to monitor the status of your application and its dependencies.

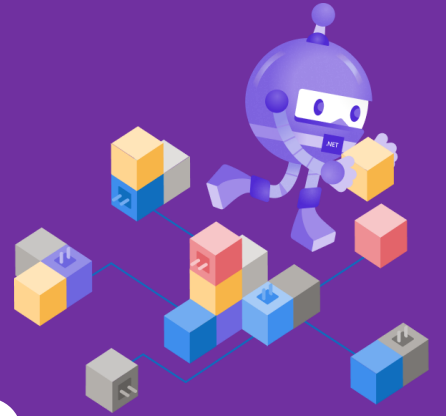


<https://blog.devart.com/how-to-implement-health-checks-in-asp-net-6.html>

EShop Microservices Deployment Strategy



Microservices Async Communication w/ RabbitMQ & MassTransit

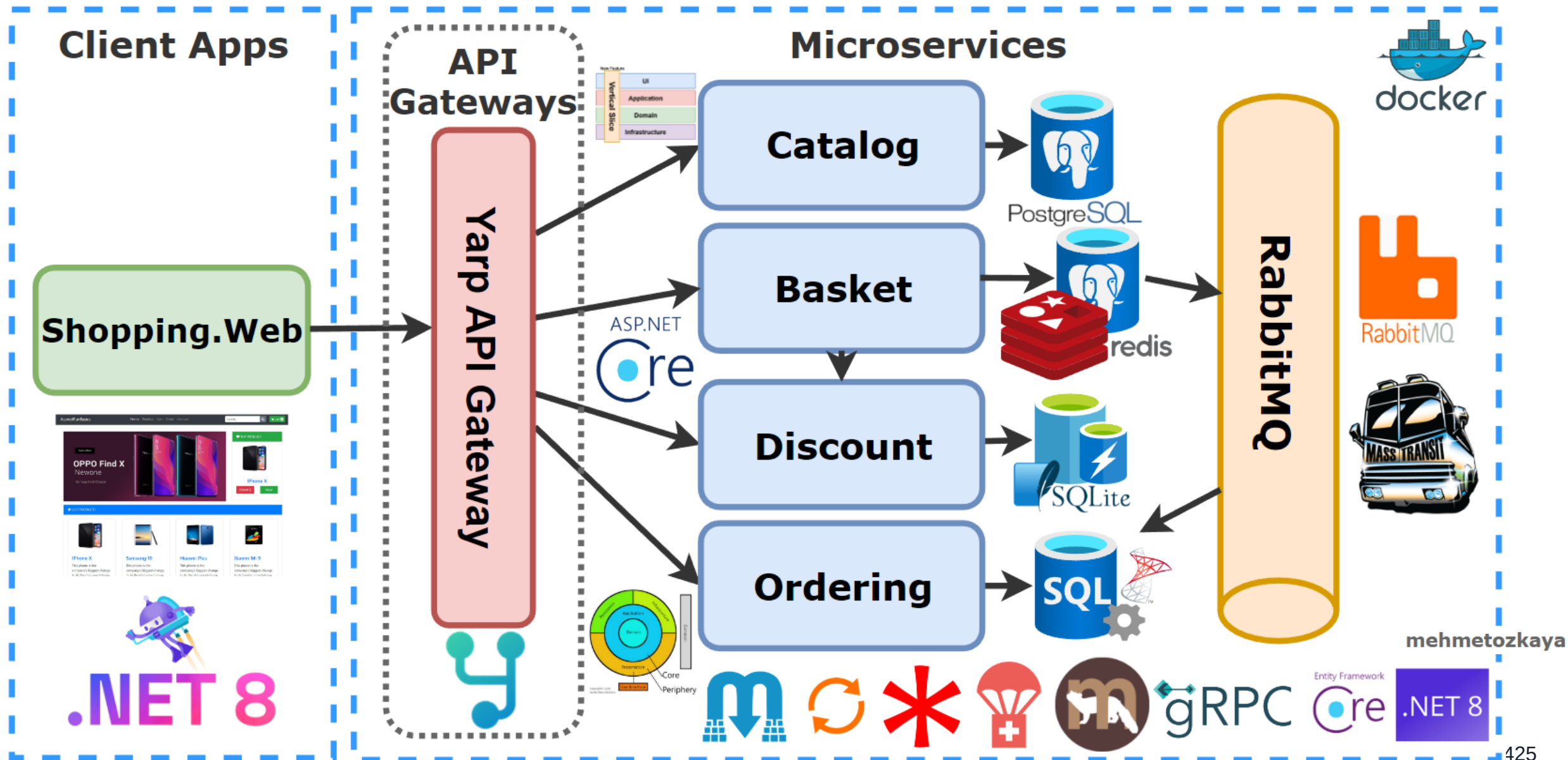


Checkout Order use cases Between Basket-Ordering Microservices
Using RabbitMQ Publish/Subscribe Topic Exchange Model
Using MassTransit for abstraction over RabbitMQ Message-Broker
Publishing BasketCheckout event queue from Basket microservices
Subscribing event from Ordering microservices
Create RabbitMQ BuildingBlocks.Messaging library

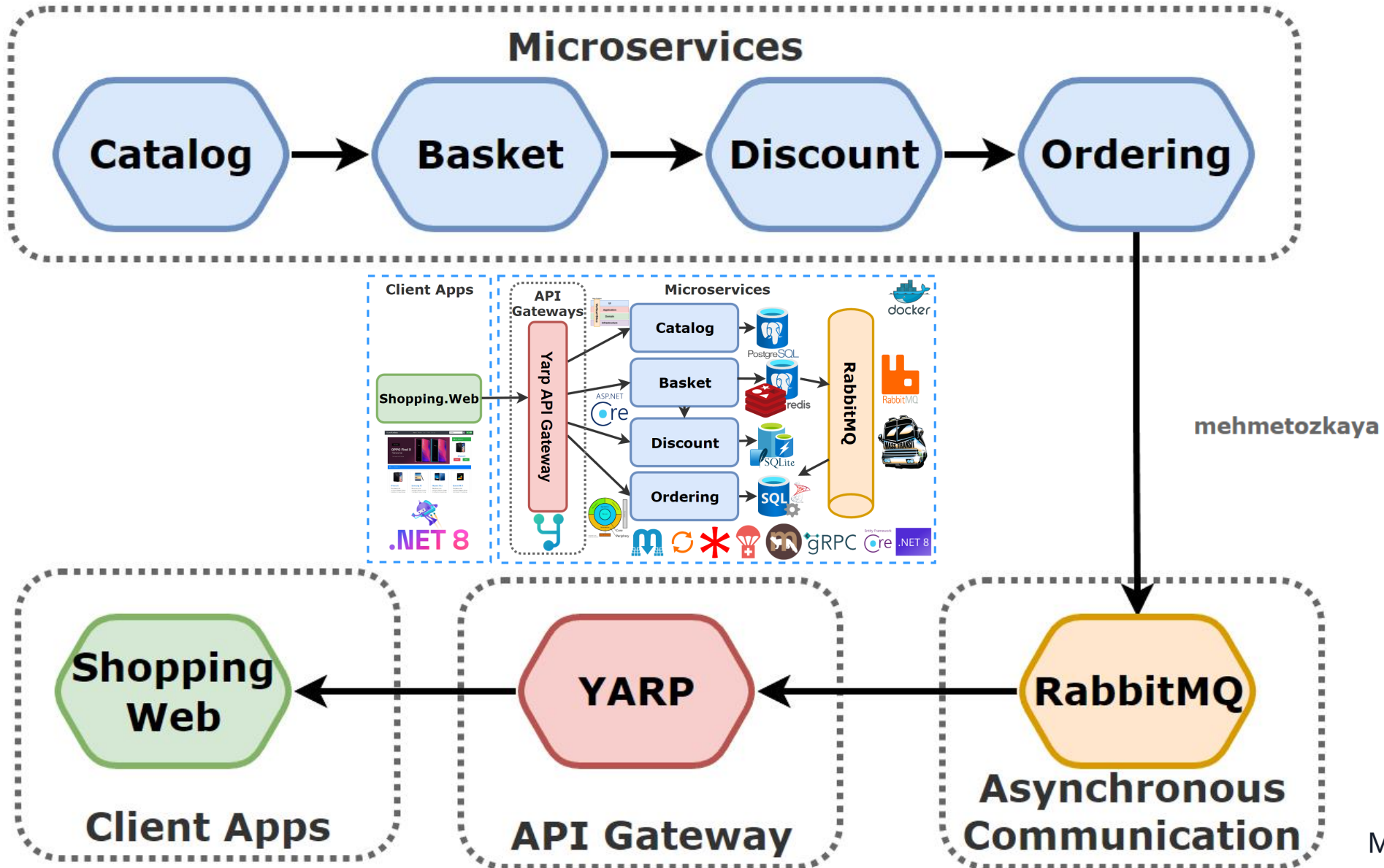
Mehmet Ozkaya



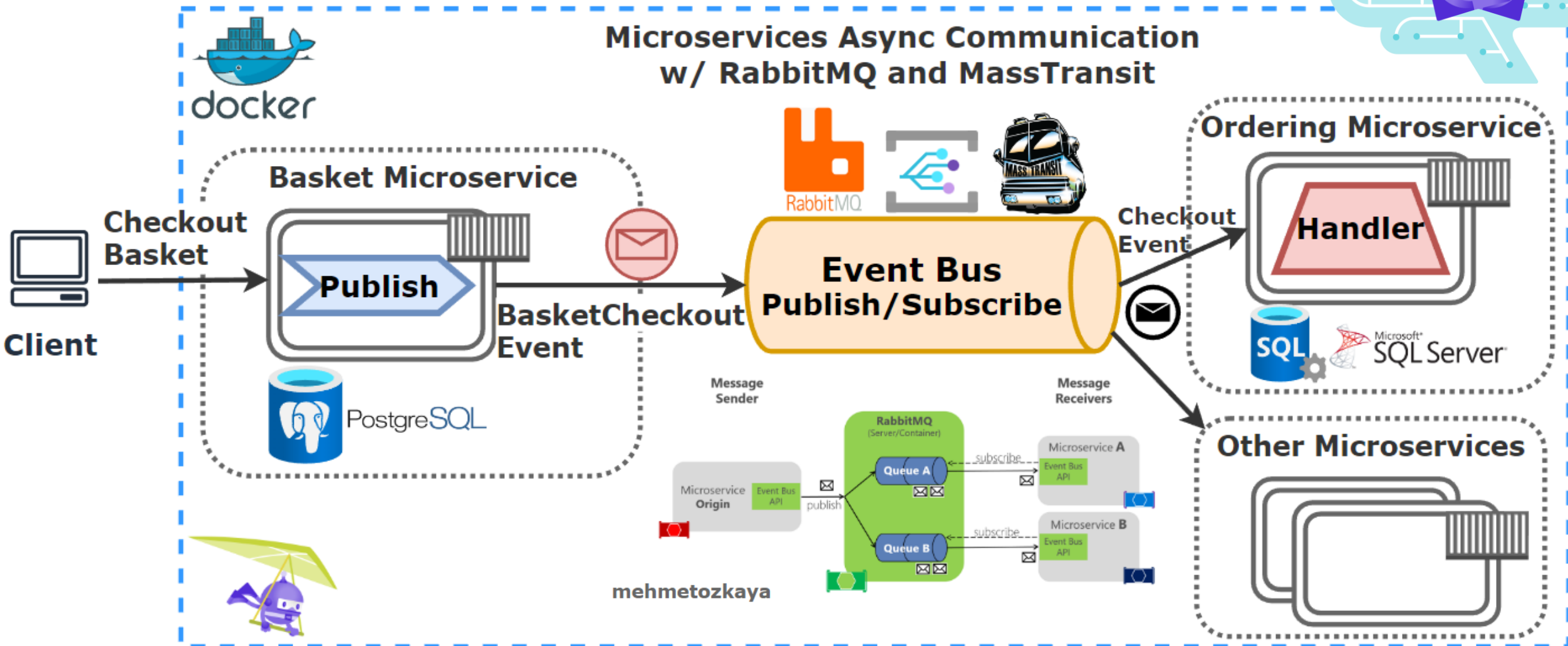
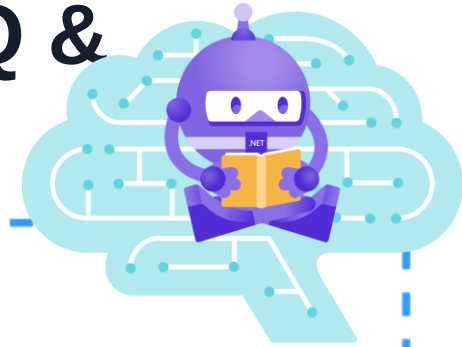
Big Picture – Course Project



How to Follow the Course (The Course Flow)

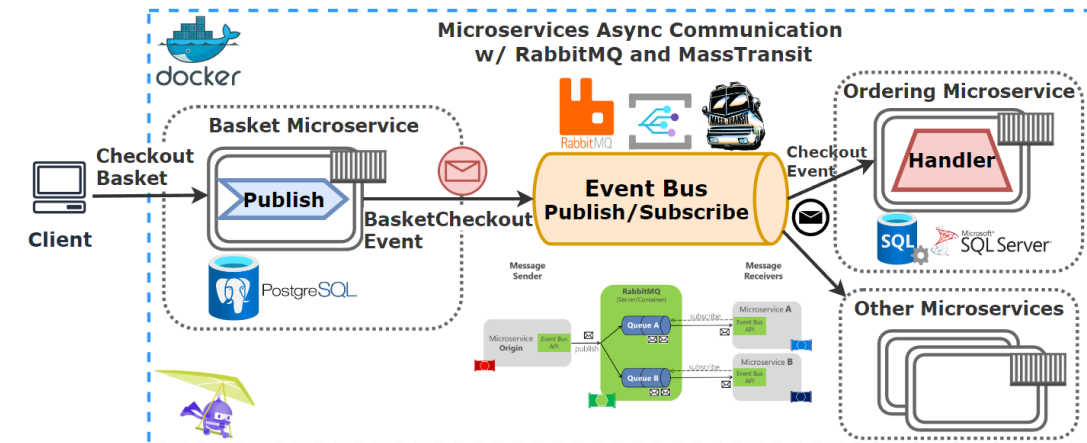
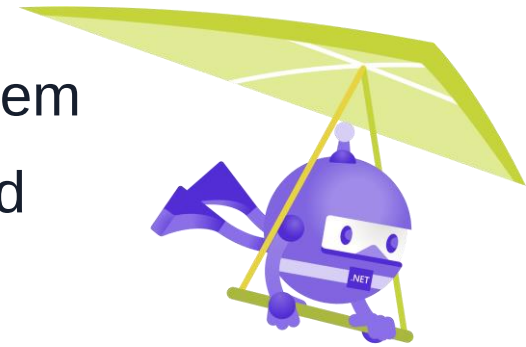


Microservices Async Communication w/ RabbitMQ & MassTransit



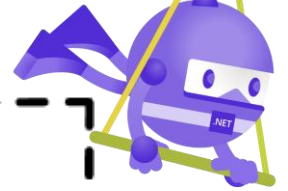
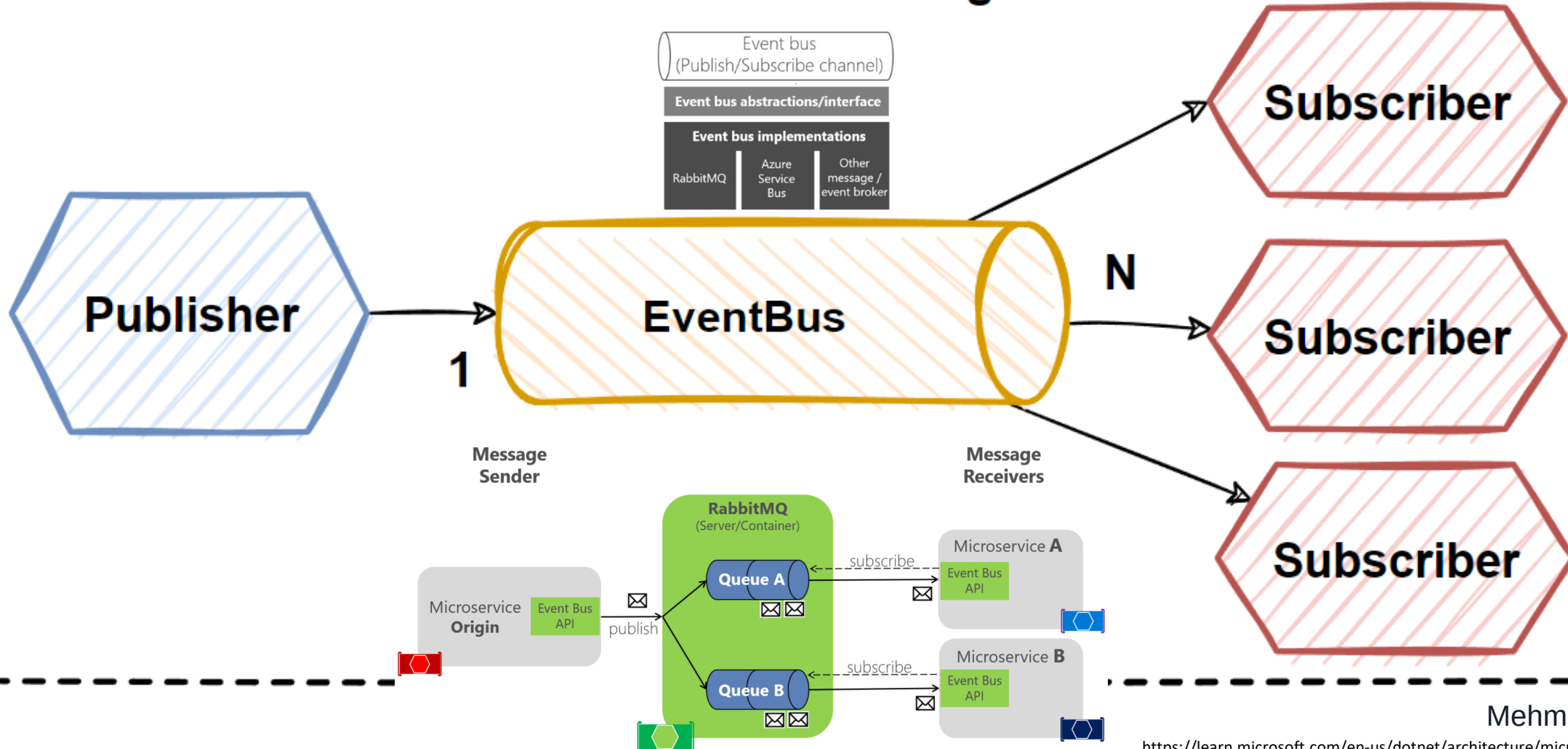
Microservices Async Communication w/ RabbitMQ & MassTransit

- **Async** Microservices **Communication** with **RabbitMQ** Message-Broker Service
- Using **RabbitMQ Publish/Subscribe** Topic Exchange Model
- Using **MassTransit** for **abstraction** over RabbitMQ Message-Broker system
- Publishing **BasketCheckout** event queue from **Basket** microservices and **Subscribing** this event from **Ordering** microservices
- Create RabbitMQ **BuildingBlocks.Messaging** library and add references Microservices
- **Containerize** RabbitMQ Message Queue system with Basket and Ordering microservices using **Docker Compose**

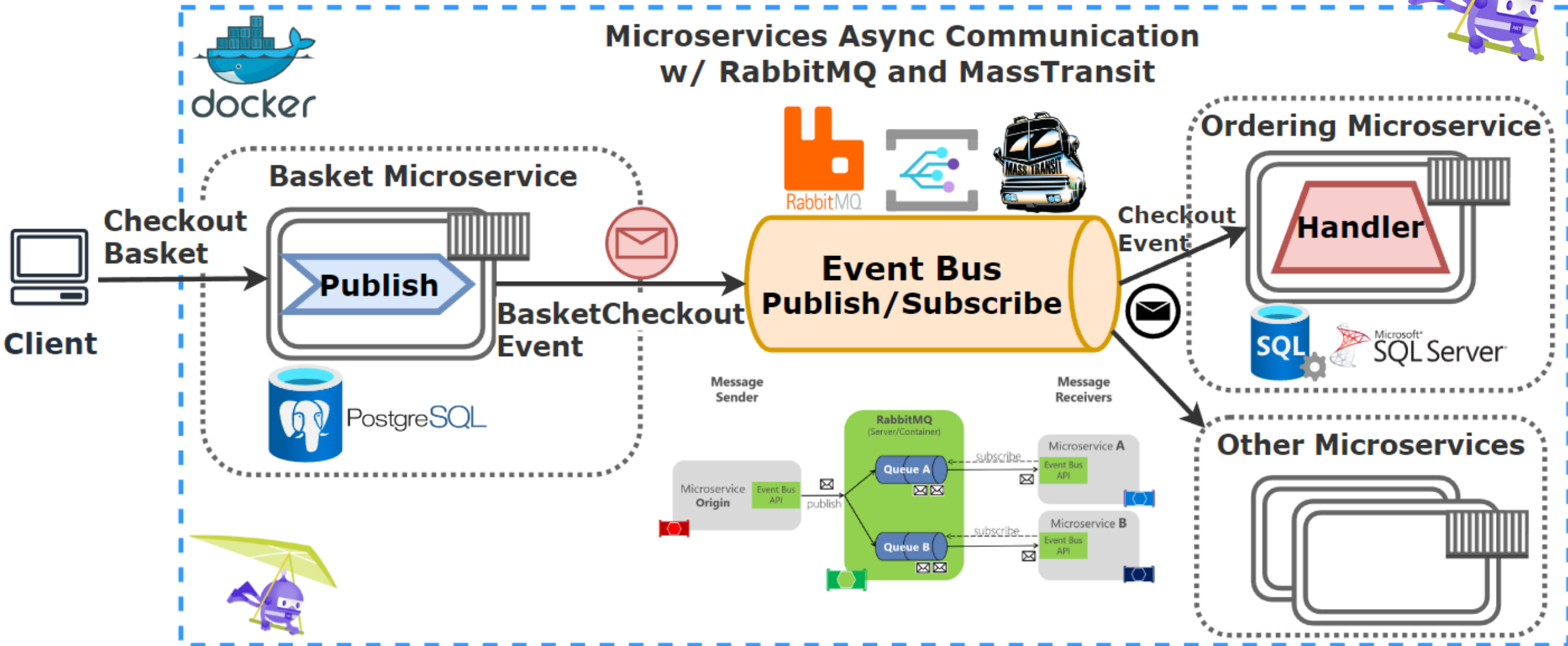


Pub/Sub RabbitMQ Architecture

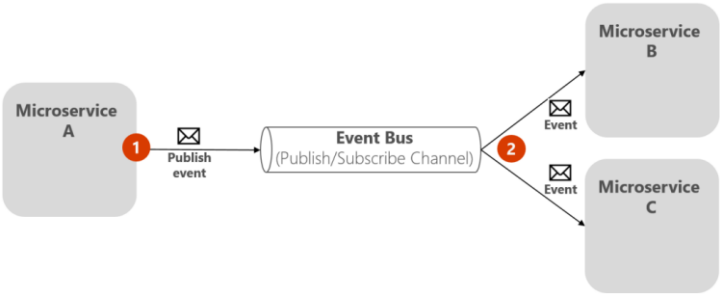
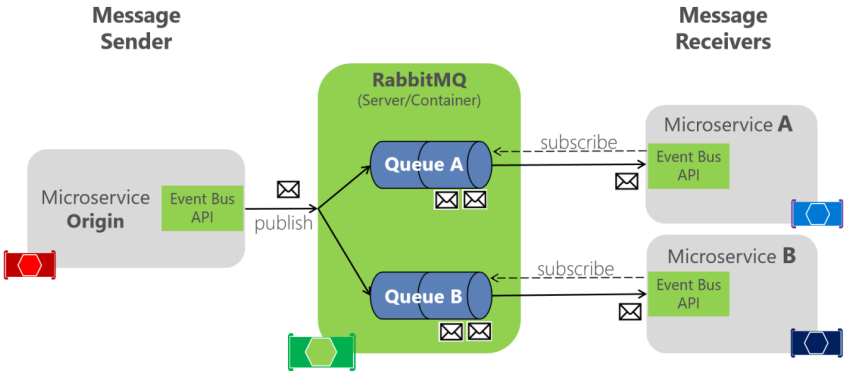
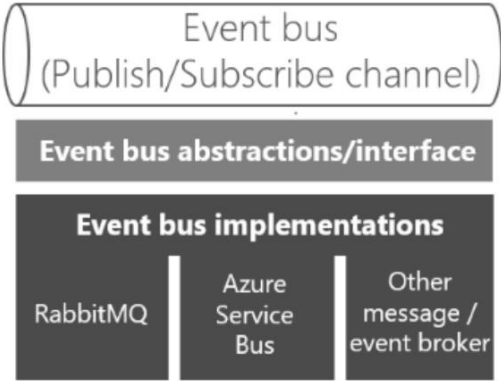
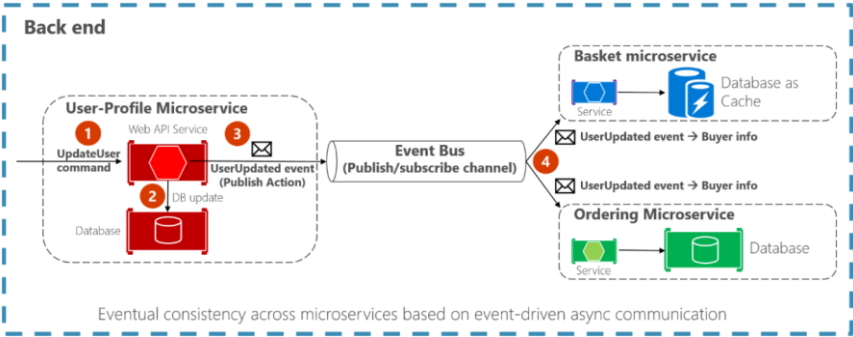
Publish-Subscribe Design Pattern



Publisher/Subscriber of BasketCheckout Event w/ Basket and Ordering Microservices

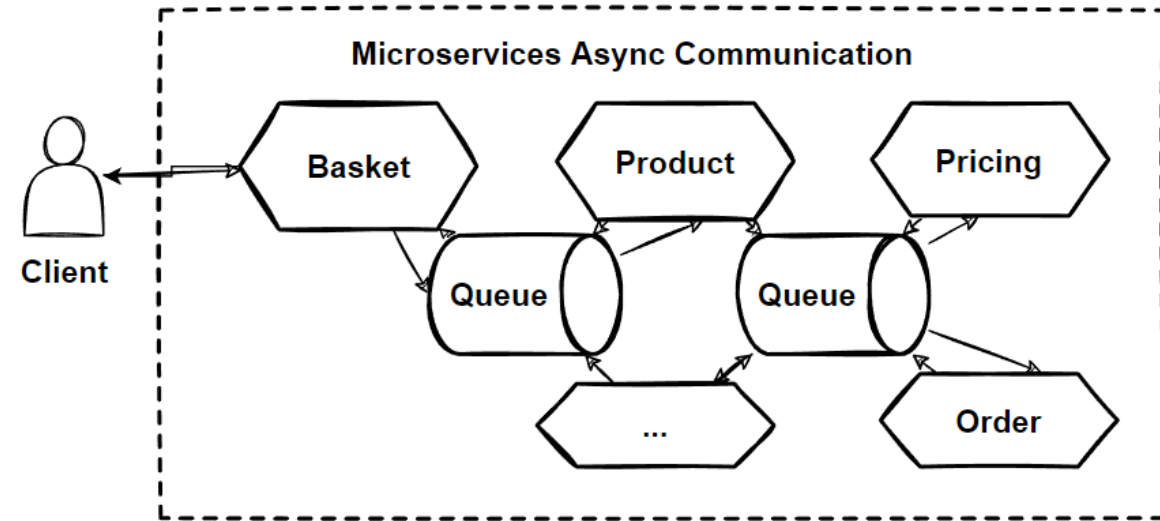


Microservices Async Communication w/ RabbitMQ & MassTransit



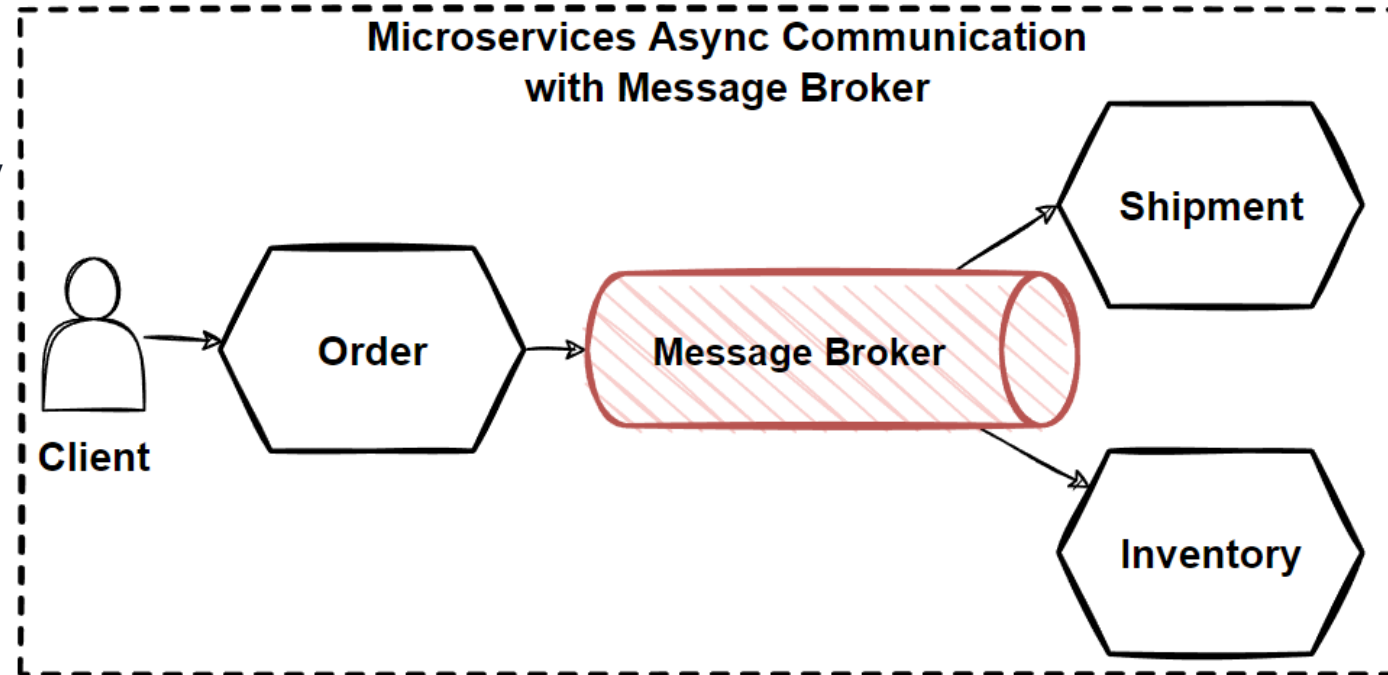
Microservices Asynchronous Communication

- The client sends a request but it **doesn't wait for a response** from the service. The client **should not have blocked** a thread while waiting for a response.
- **AMQP** (Advanced Message Queuing Protocol)
- Using AMQP protocols, the client **sends** the **message** with using message broker systems like **Kafka** and **RabbitMQ** queue.
- The message **producer does not wait** for a **response**.
- Message consume from the **subscriber** systems in async way, and no one waiting for response **suddenly**.
- If there is **busy interactions** in communication across multiple microservices, then use **asynchronous messaging platforms**.

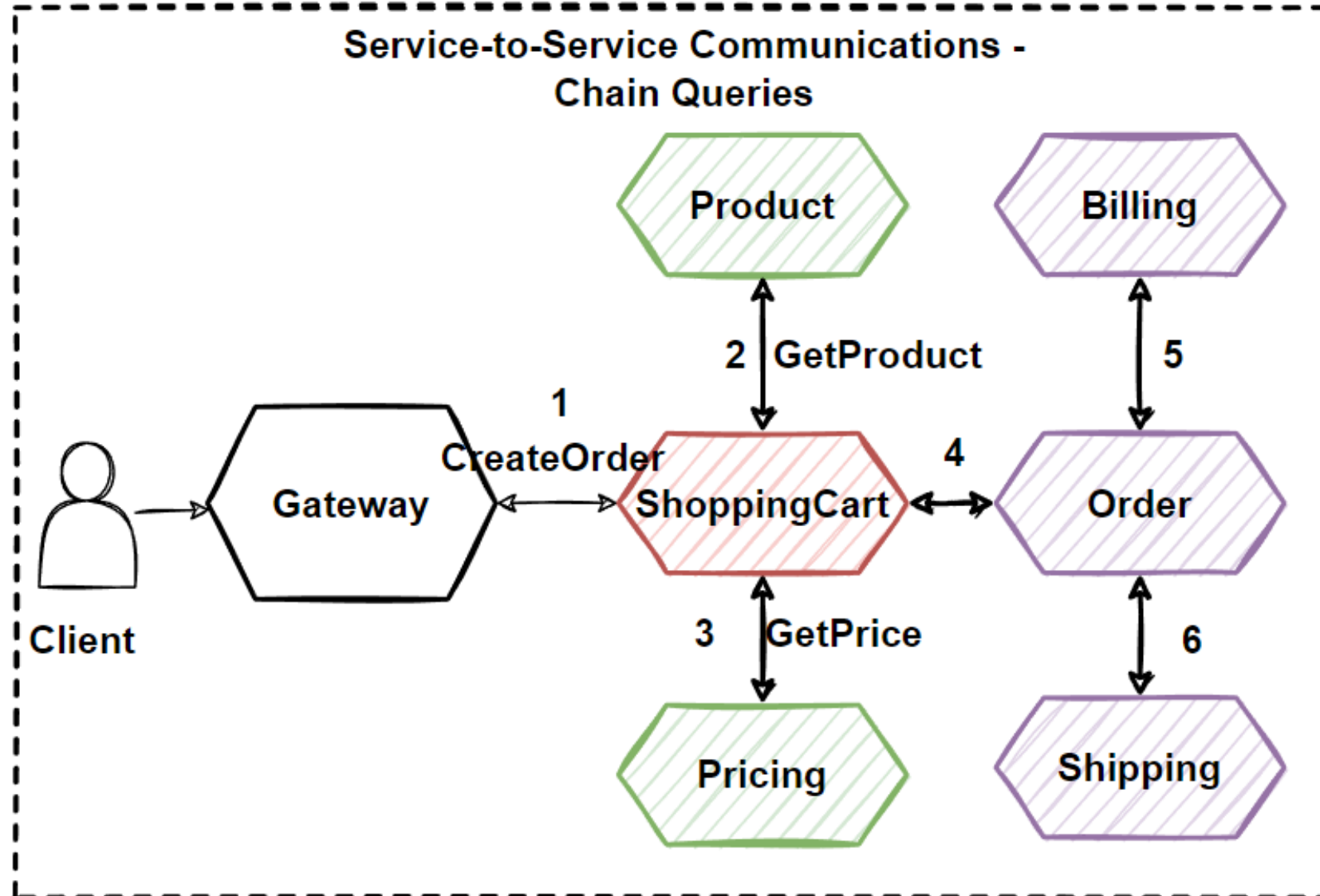


Microservices Asynchronous Communication-2

- **Message brokers** are responsible for handling the message sent by the producer service in async messaging-based communication.
- If the **consumer service** is **down** at the moment, the broker might be configured to **retry** as long as necessary for successful delivery.
- Messages can be **persisted if required** or stored only in memory.
- Message broker is **responsible** for **delivering** the message.
- No longer necessary for both microservices to be **up and running** for successful communication.
- Async messaging provides **loosely couple communication**.



Problem: Chain of request and highly coupled dependent microservices



Benefits of Asynchronous Communication

- **New Subscriber Services**

Adding new services is very simple. We can easily subscribe to message that we want to receive. The producer doesn't need to be know about subscribers, we can remove and add subscribers without affecting producer service.

- **Scalability**

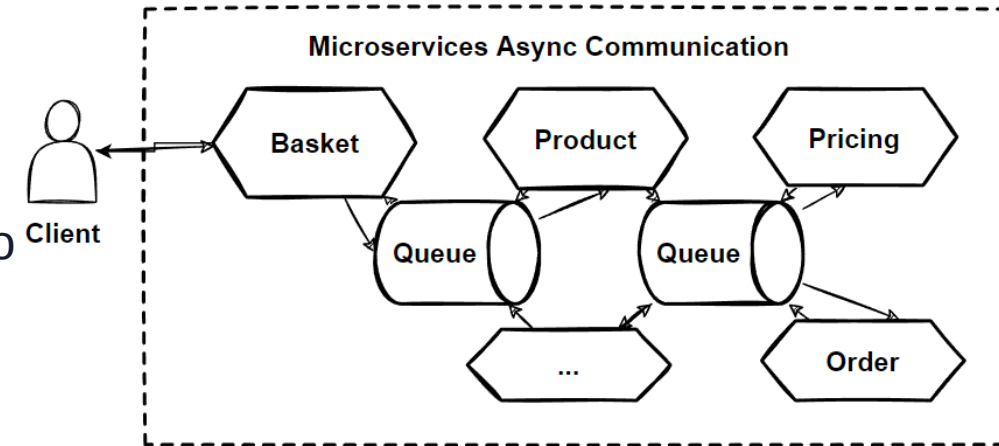
With async communications we can easier to manage scalability issues, can scale producer, consumer and message broker system independently. Scale services according to incoming messages into event bus system. Kubernetes KEDA Auto-scalers.

- **Event-driven Microservices**

With async communication, we can provide event-driven architectures which is best way to communicate between microservices.

- **Retry mechanisms**

Brokers can retry to sending message and keep trying automatically without any custom solutions.



Challenges of Asynchronous Communication

- **Single Point of Failure - Message Broker**

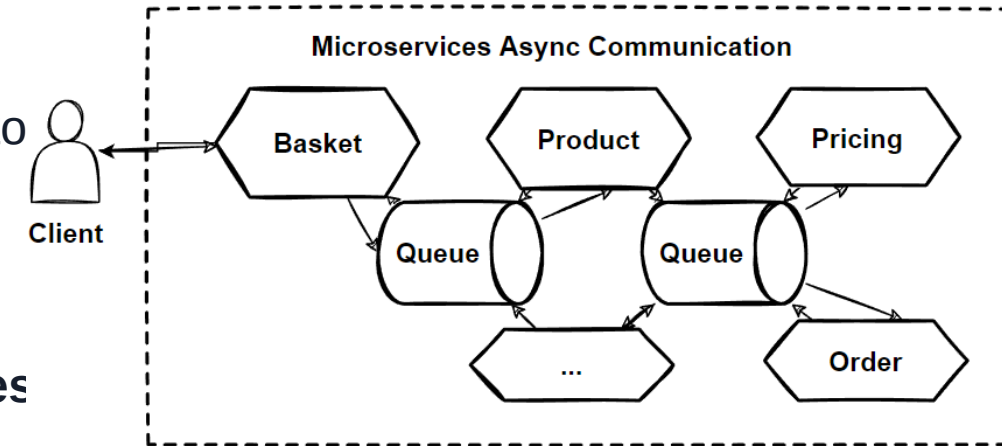
The message broker becomes a single point of failure. We should not rely of all communication with a single node of message brokers, instead we should scale it and use hybrid communication with sync and async in your cases.

- **Debugging**

difficult to debug issues with async communication, it can be hard to trace the flow of a single operation across service boundaries. debugging of the flow and the payload of events takes so many times and hard to debug at the same time.

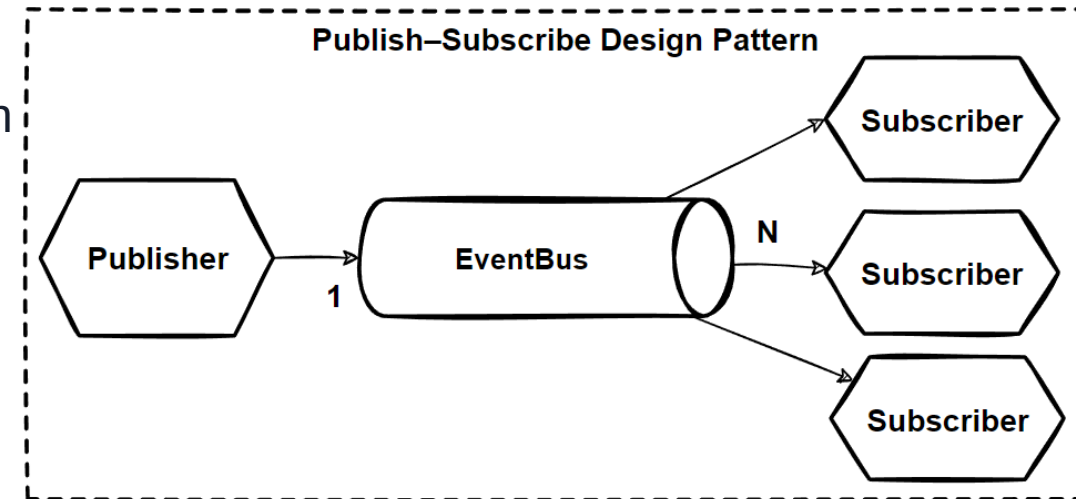
- **At-least-once delivery and Not Guarantee an order of messages**

Mostly Brokers use at-least-once delivery and not Guarantee order of messages. Should embrace these message delivery mechanism with applying idempotency consumers and not designing FIFO requires cases.



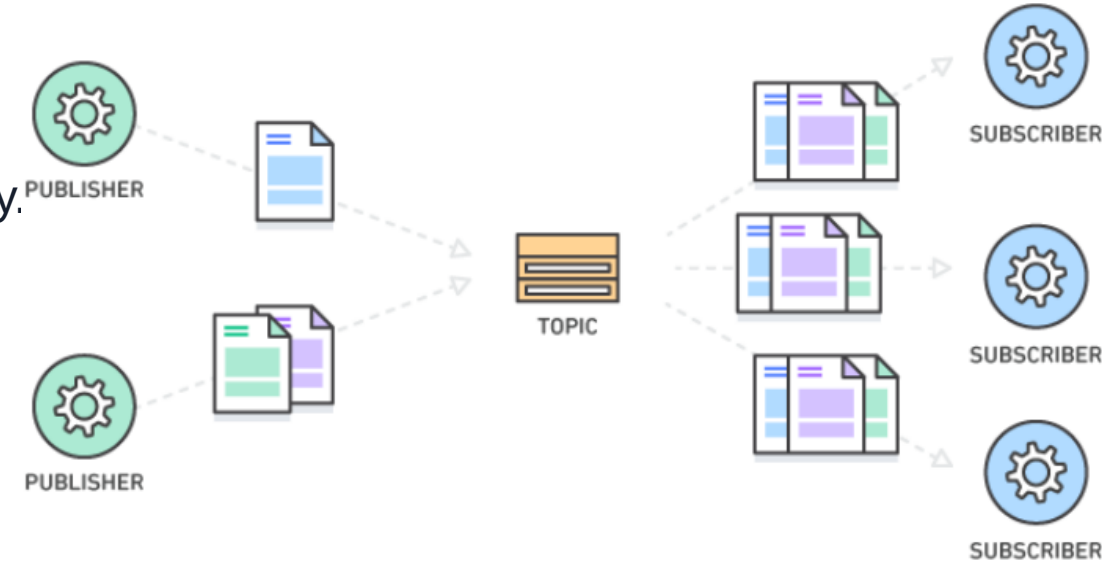
Fan-Out & Message Filtering with Publish/Subscribe Pattern

- **Fan-out** is a messaging pattern; ‘fanned out’ to **multiple destination** in parallel.
- Each of destinations can **work** and **process** this messages in **parallel**.
- **Publisher/subscriber model** to **define a topic** which is **logical access point** to enabling message communication with asynchronously.
- Publisher **sends the message to the topic**, message is immediately **fanned out** to all **subscribers** of this topic.
- Each service can operate and **scale independently** and individually that **completely decoupled** and **asnychronously**.
- The publisher and the subscribers **don't need to know** who is publishing / consuming this message that is broadcasting.
- Deliver the same message to **multiple receivers** is to use the **Fanout Publish/Subscribe Messaging Pattern**.

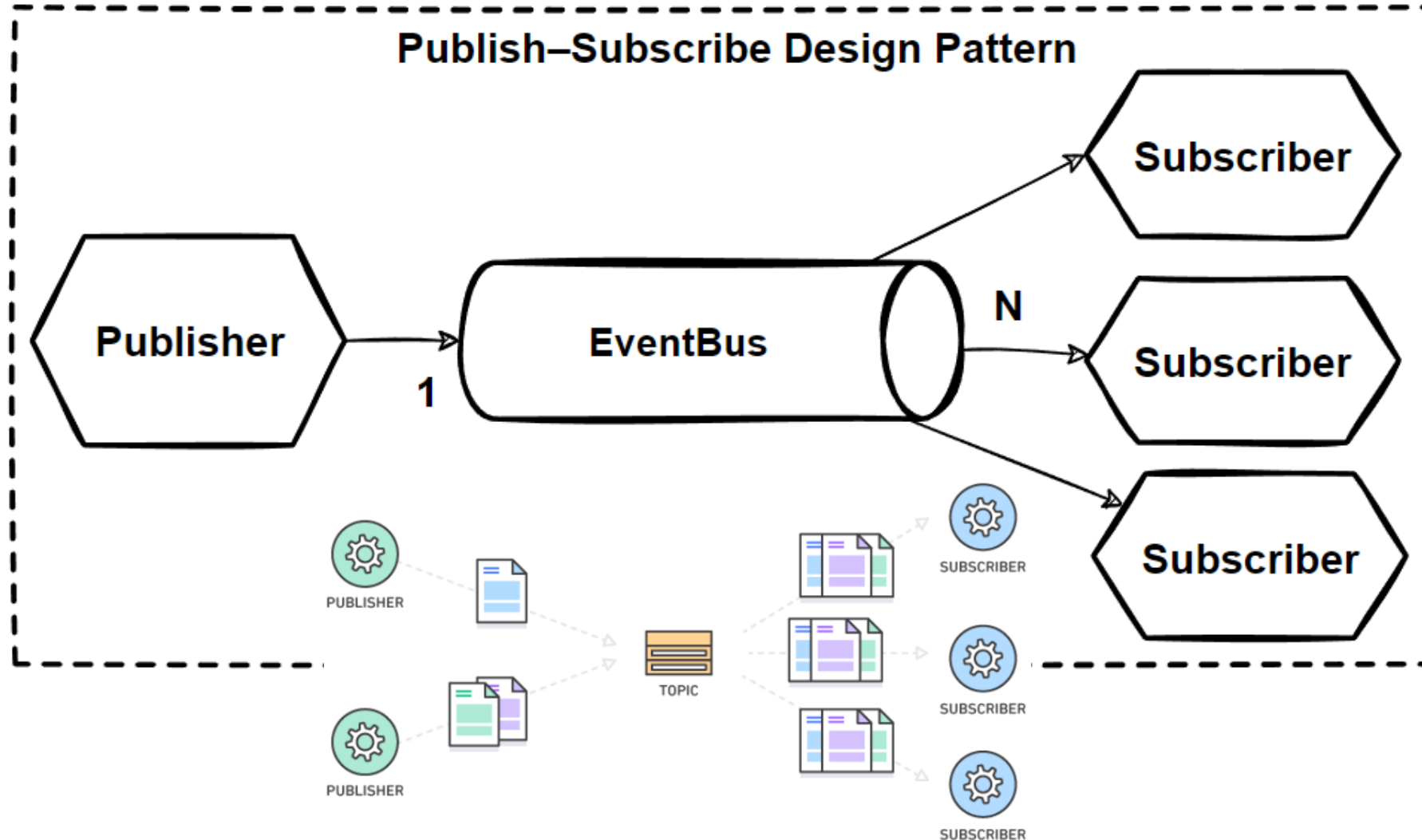


Publish/Subscribe Messaging Pattern

- **Publish/subscribe messaging** is a form of **asynchronous** service-to-service communication.
- Any message **published** to a **topic** is **immediately received** by all of the **subscribers** to the **topic**.
- Enable **event-driven architectures**, and **decouple applications** to increase performance, reliability and scalability.
- Applications are **decoupled** into **smaller, independent building blocks** that are easier to develop, deploy and maintain.
- Publish/Subscribe (Pub/Sub) messaging provides **instant event notifications** for these distributed applications.
- A **message topic** provides a **lightweight mechanism** to broadcast **asynchronous event notifications**.
- All components that subscribe to the topic receive every message, unless a **message filtering policy** is set by the subscriber.



Publish/Subscribe Messaging Pattern-2

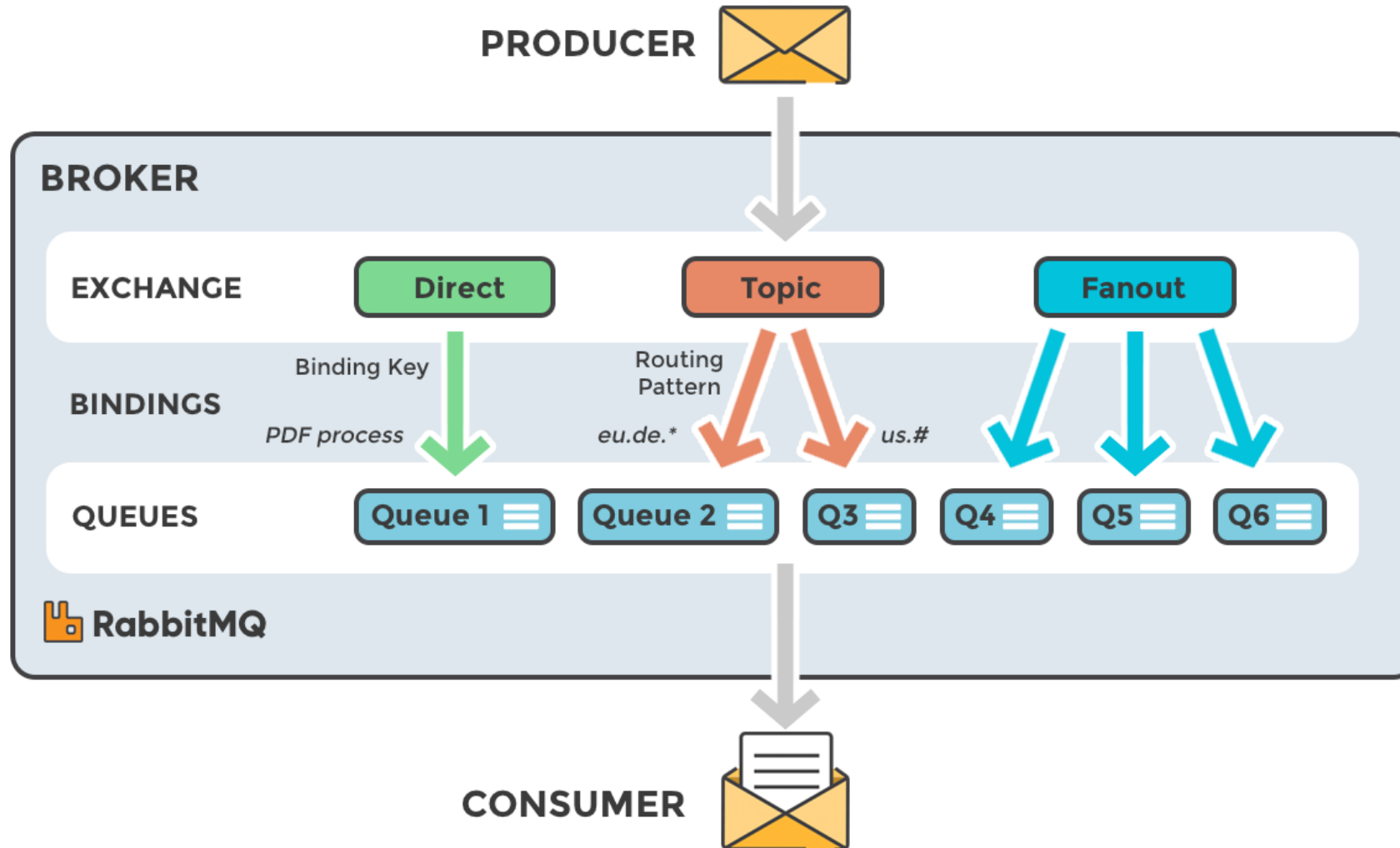


What is RabbitMQ ?

- **RabbitMQ** is a **message broker software** that implements the **Advanced Message Queuing Protocol (AMQP)**.
- It allows applications to communicate with each other by **sending** and **receiving messages** through **queues**.
- **RabbitMQ** is a **message queuing system** that transmit a message received from any source to another source.
- Similar ones can be listed as **Apache Kafka, Msmq, Microsoft Azure Service Bus, Kestrel, ActiveMQ**.
- All **transactions** can be **listed in a queue** until the source to be transmitted gets up.
- It allows to **send** and **receive** messages **asynchronously**.
- **RabbitMQ's** support for **multiple operating systems** and open source code is one of the most preferred reasons.
- Main Components of RabbitMQ: **Producer, Queue, Consumer, Message, Exchange, Binding** and **FIFO**.



RabbitMQ Components: Producer, Queue, Consumer, Message, Exchange, Binding



RabbitMQ Queue Properties

- **Queue Name**

The name of the queue we have defined.

- **Durable**

Determines the lifetime of the queue. If we want persistence, we have to set it true.

- **Exclusive**

Contains information whether the queue will be used with other connections.

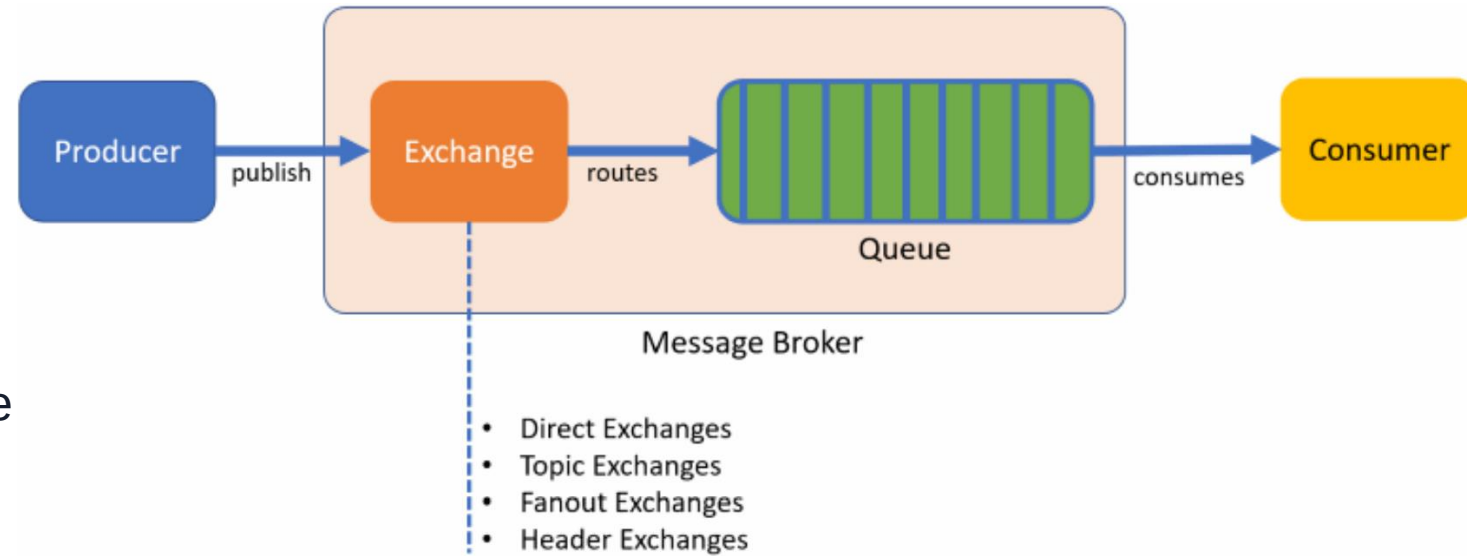
- **AutoDelete**

Contains information about deletion of the queue with the data sent to the queue passes to the consumer side.



RabbitMQ Exchange Types

- **RabbitMQ** provides **four types** of **exchanges**: Direct, Fanout, Topic, and Headers.
- **Direct Exchange**
Uses of a single queue is being addressed. The most appropriate queue is reached with the relevant direct exchange.
- **Topic Exchange**
Messages are sent to different queues according to their subject. Incoming message is classified and sent to the related queue.
- Variation of the Publish / Subscribe pattern.
Topic Exchange should be used to determine what kind of message they want to receive.



<https://www.rabbitmq.com/tutorials/amqp-concepts.html>

RabbitMQ Exchange Types-2

- **Fanout Exchange**

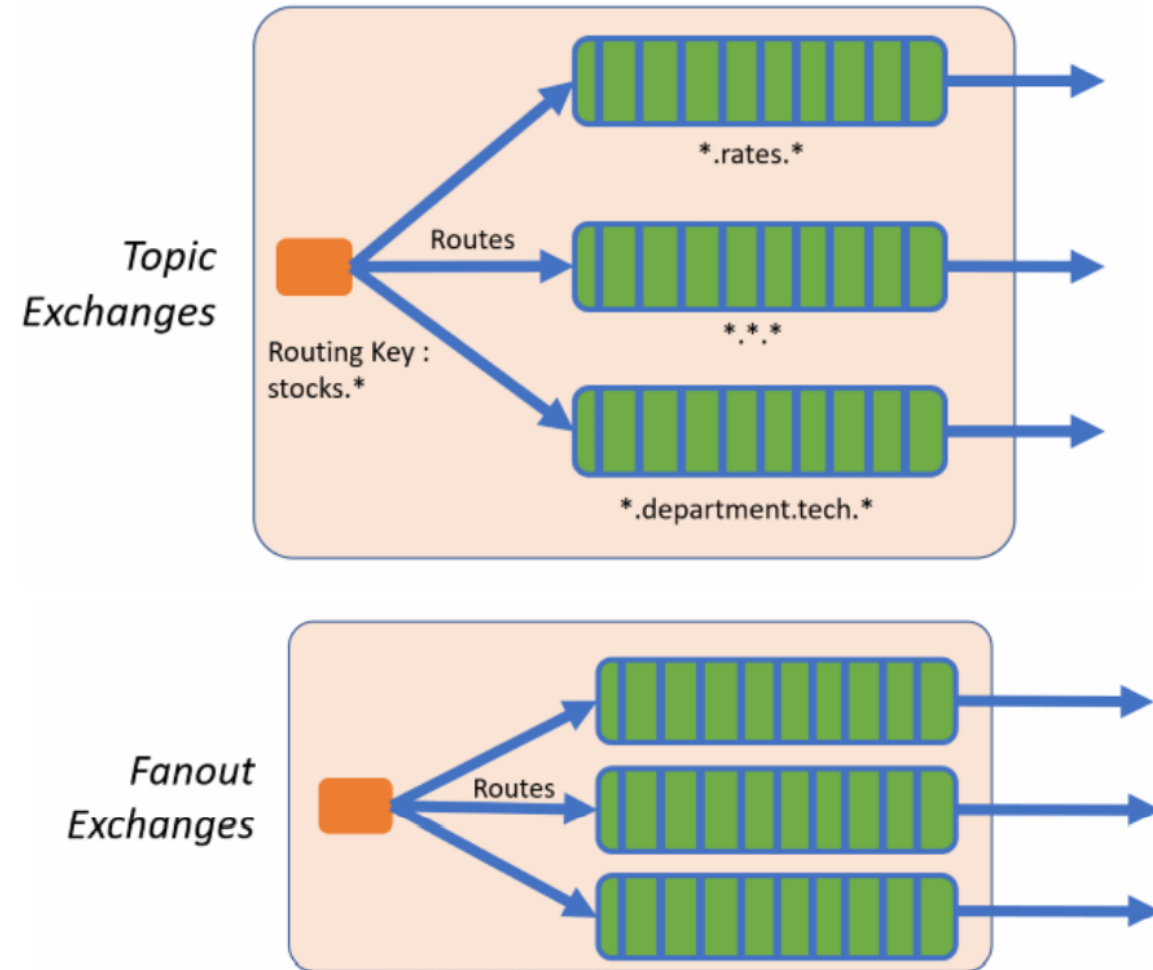
Used in situations where the message should be sent to more than one queue. Especially applied in Broadcasting systems.

- Mainly used for games for global announcements.

- **Headers Exchange**

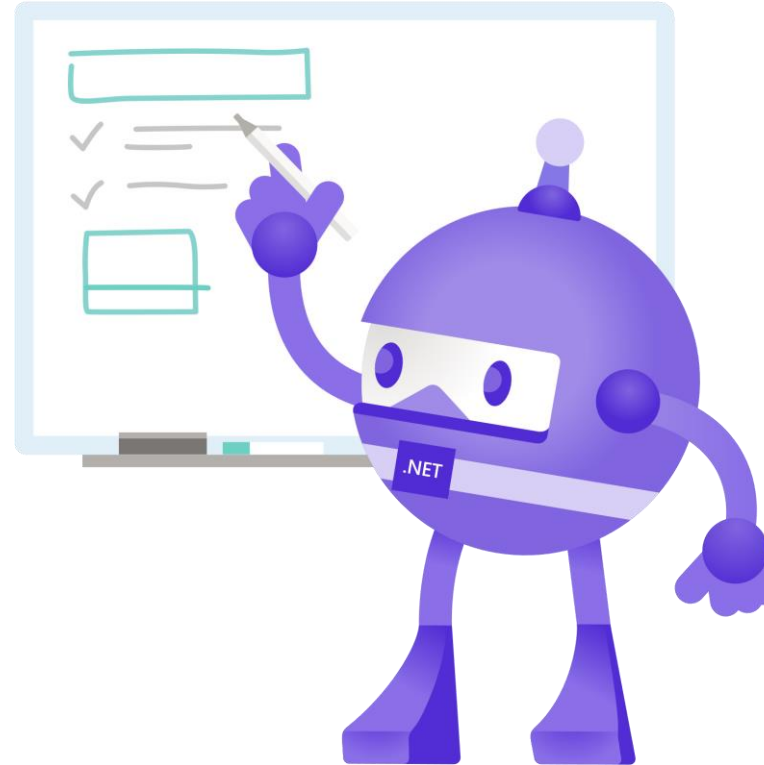
Guided by the features added to the header of the message. Routing-Key used in other models is not used.

- Transmits to the correct queue with a few features and descriptions in message headers.
- The attributes on the header and the attributes on the queue must match each other's values.



Domain Analysis of Async Communication between Basket and Ordering Microservices

1. Event Models
2. Application Use Cases
3. Rest API Endpoints
4. Underlying Data Structures



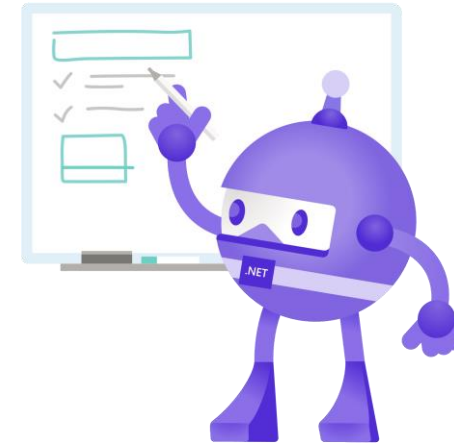
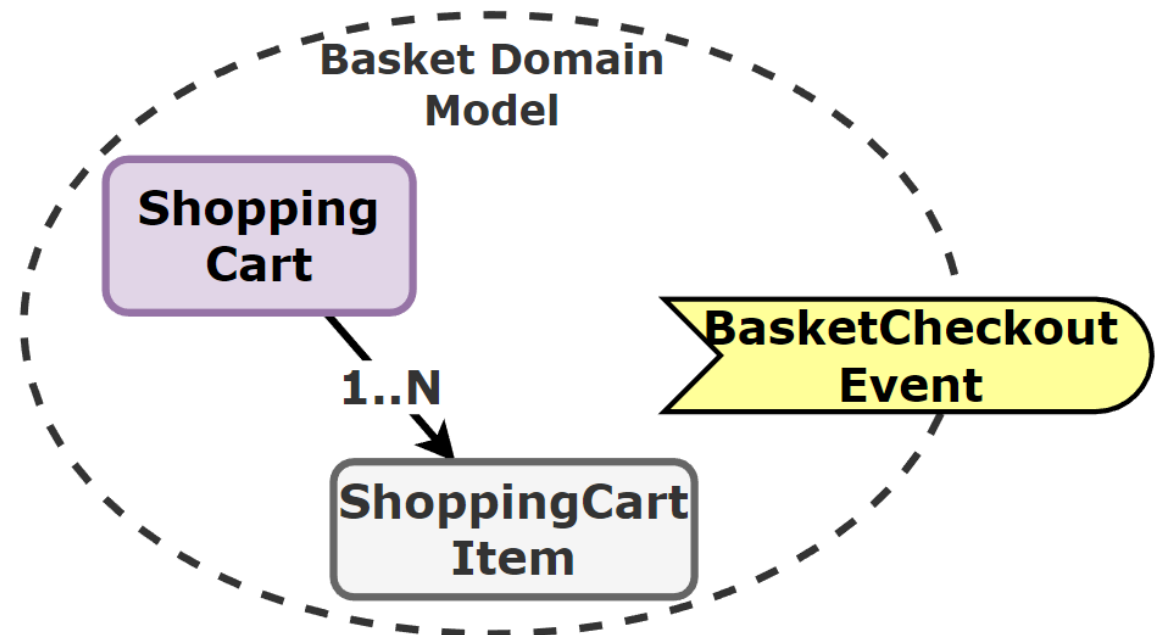
1. Event Models of BasketCheckout Event

- Primary domain model is 'BasketCheckout'

```
public class BasketCheckout
{
    public string UserName { get; set; }
    public decimal TotalPrice { get; set; }

    // BillingAddress
    public string FirstName { get; set; }
    public string LastName { get; set; }
    public string EmailAddress { get; set; }
    public string AddressLine { get; set; }
    public string Country { get; set; }
    public string State { get; set; }
    public string ZipCode { get; set; }

    // Payment
    public string CardName { get; set; }
    public string CardNumber { get; set; }
    public string Expiration { get; set; }
    public string CVV { get; set; }
    public int PaymentMethod { get; set; }
}
```



- Consider Domain event for basket checkout, leading to integration events.

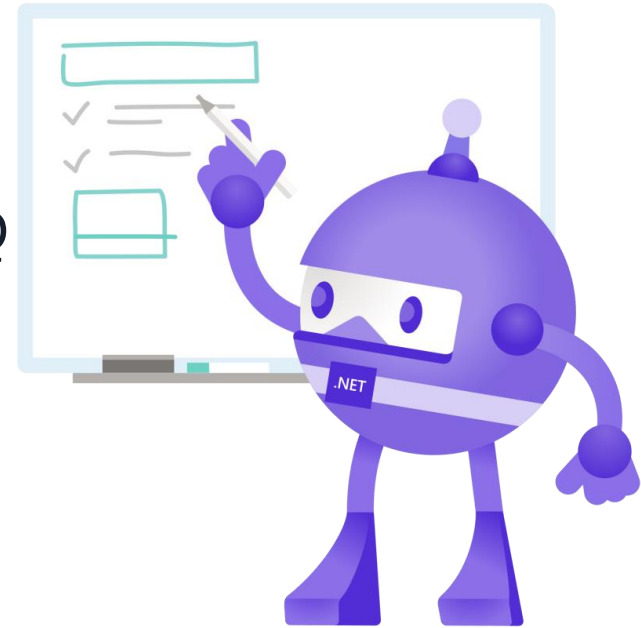
2. Application Use Cases of Basket Microservices

Async Basket Operations:

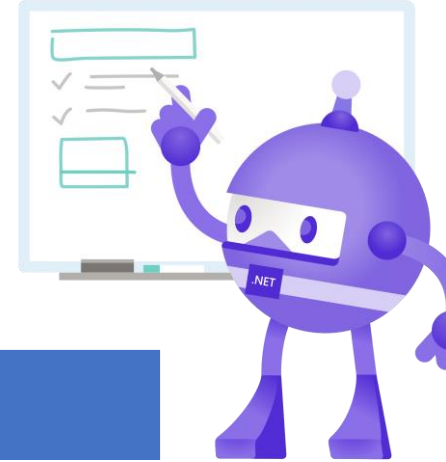
- Checkout Basket and Publish event to RabbitMQ Message Broker

Asynchronous Ordering Operations:

- **Basket Checkout:** Consume Basket Checkout event from RabbitMQ using MassTransit
- **Order Fulfilment:** Perform order fulfilment operations (billing, shipment, notification..)
- Raise OrderCreated Domain Event that leads to integration event



3. Rest API Endpoints of Basket Microservices



Method	Request URI	Use Cases
POST	/basket/checkout	Checkout basket

Ordering AMQP Protocol

- AMQP: Consume BasketCheckout Event from RabbitMQ using MassTransit

4. Underlying Data Structures of Async Communications

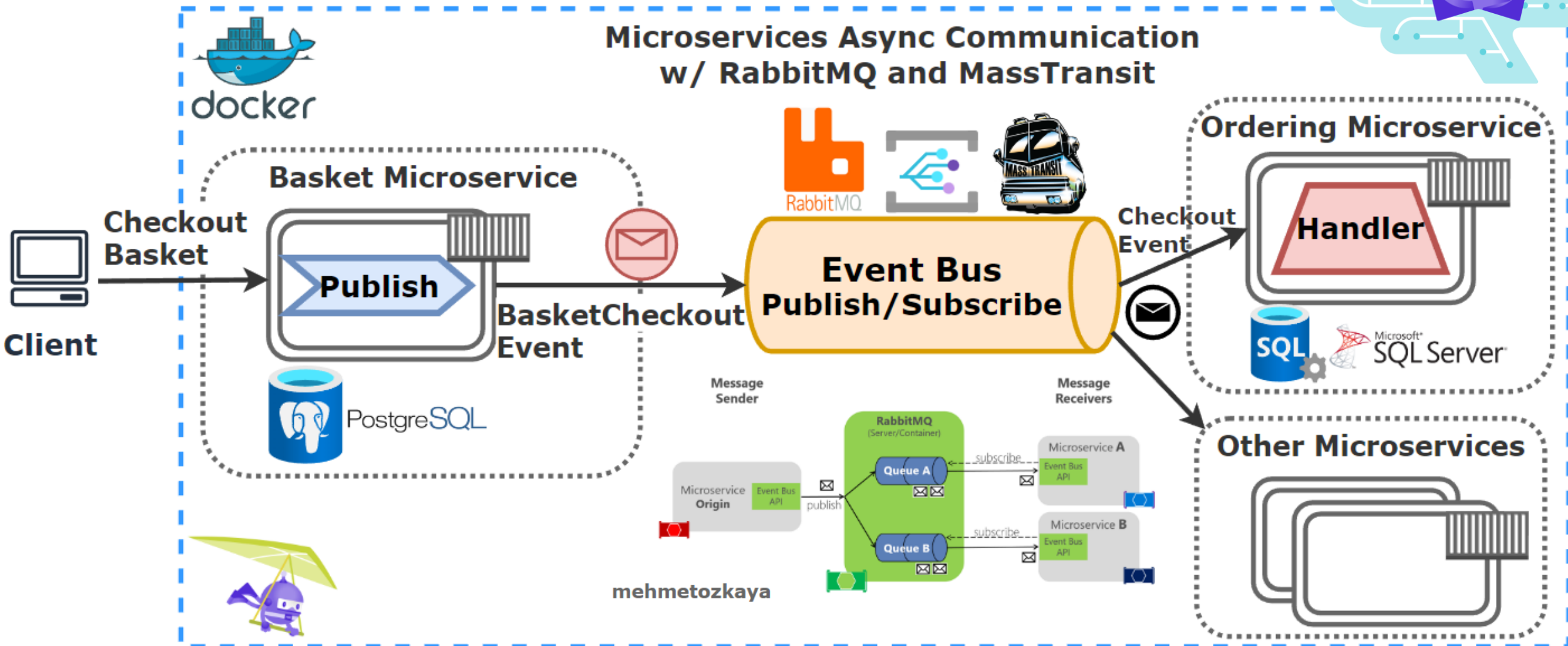
For Async Communicatin Between Microservices has 1 Backing Services:

1. RabbitMQ Distributed Message Broker

- **RabbitMQ** allows for robust, efficient, and **scalable asynchronous communication**. Facilitates **pub/sub messaging** between **Basket** and **Ordering microservices**.
- Message Broker for both Basket and Ordering microservices connect for pub/sub messaging is RabbitMQ.



Microservices Async Communication w/ RabbitMQ & MassTransit

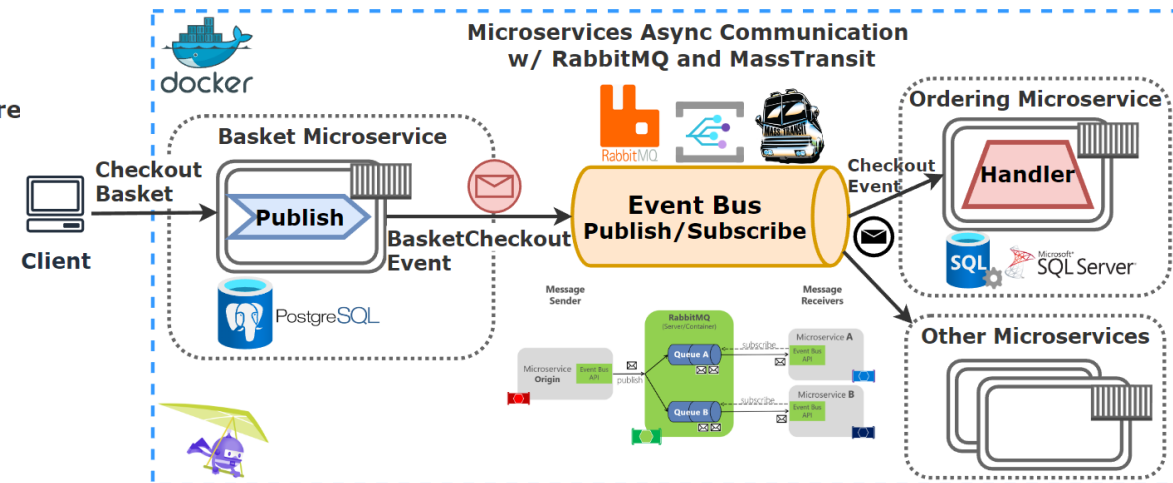
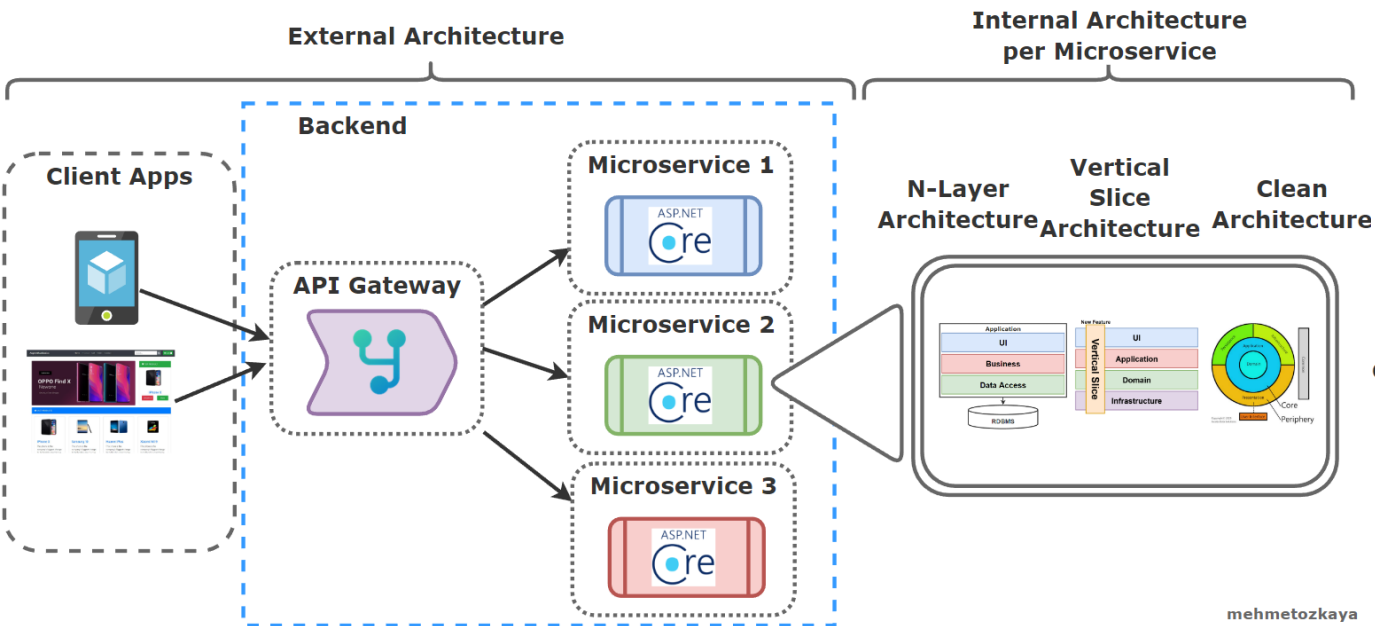


The diagram illustrates a microservices architecture for an e-commerce application. It is divided into three main sections: Client Apps, API Gateways, and Microservices.

- Client Apps:** Includes "Shopping.Web" and a mobile application icon, both utilizing ".NET 8".
- API Gateways:** Features the "Yarp API Gateway", which acts as the central point of entry for client requests.
- Microservices:** Consists of four core services:
 - Catalog:** Interacts with PostgreSQL and includes layers for UI, Application, Domain, and Infrastructure.
 - Basket:** Utilizes ASP.NET Core and Redis for session management.
 - Discount:** Uses SQLite for data storage.
 - Ordering:** Relies on SQL for database operations.
- Messaging:** RabbitMQ is used for asynchronous communication between the microservices.
- Other Technologies:** The architecture leverages Docker for containerization, RabbitMQ for messaging, and various .NET 8 components like gRPC, Entity Framework, and Core.

Technical Analysis of Async Communication between Basket and Ordering Microservices

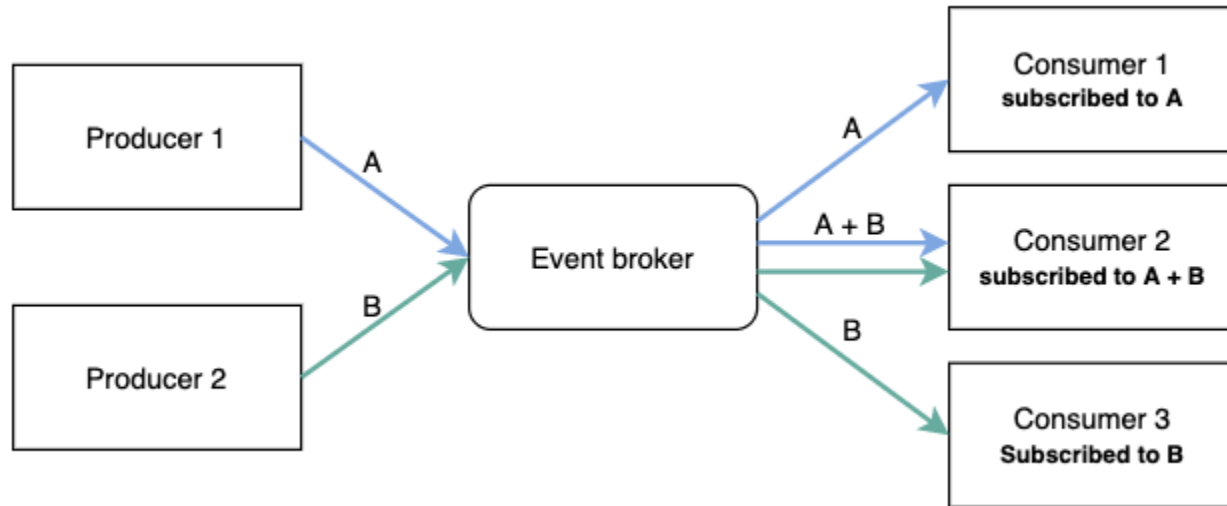
1. Application Architecture Style
2. Patterns & Principles
3. Libraries Nuget Packages
4. Project Folder Structure



1. Application Architecture Style of Async Communication

Event-driven Microservices Architecture

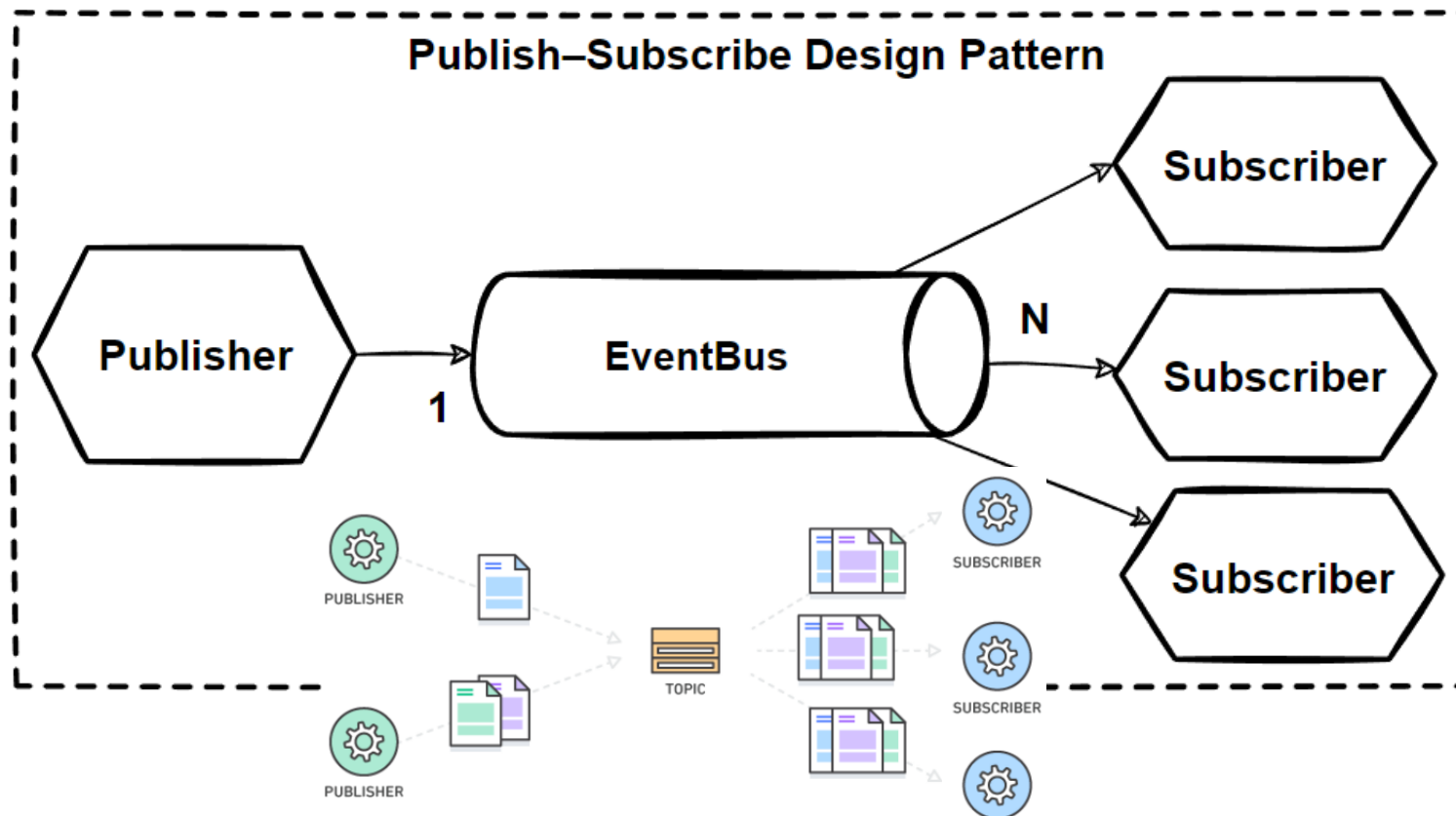
- Focuses on events as the primary source of communication between different microservices.
- Enhances decoupling, scalability, and responsiveness.



2. Patterns & Principles of Async Communication



- **Publish/Subscribe Pattern**
- A core pattern enabling event-driven architecture.
- One service (publisher) sends messages/events without knowing the receivers (subscribers). Subscribers listen and react to these events.



Class Library Architecture of Async Communication

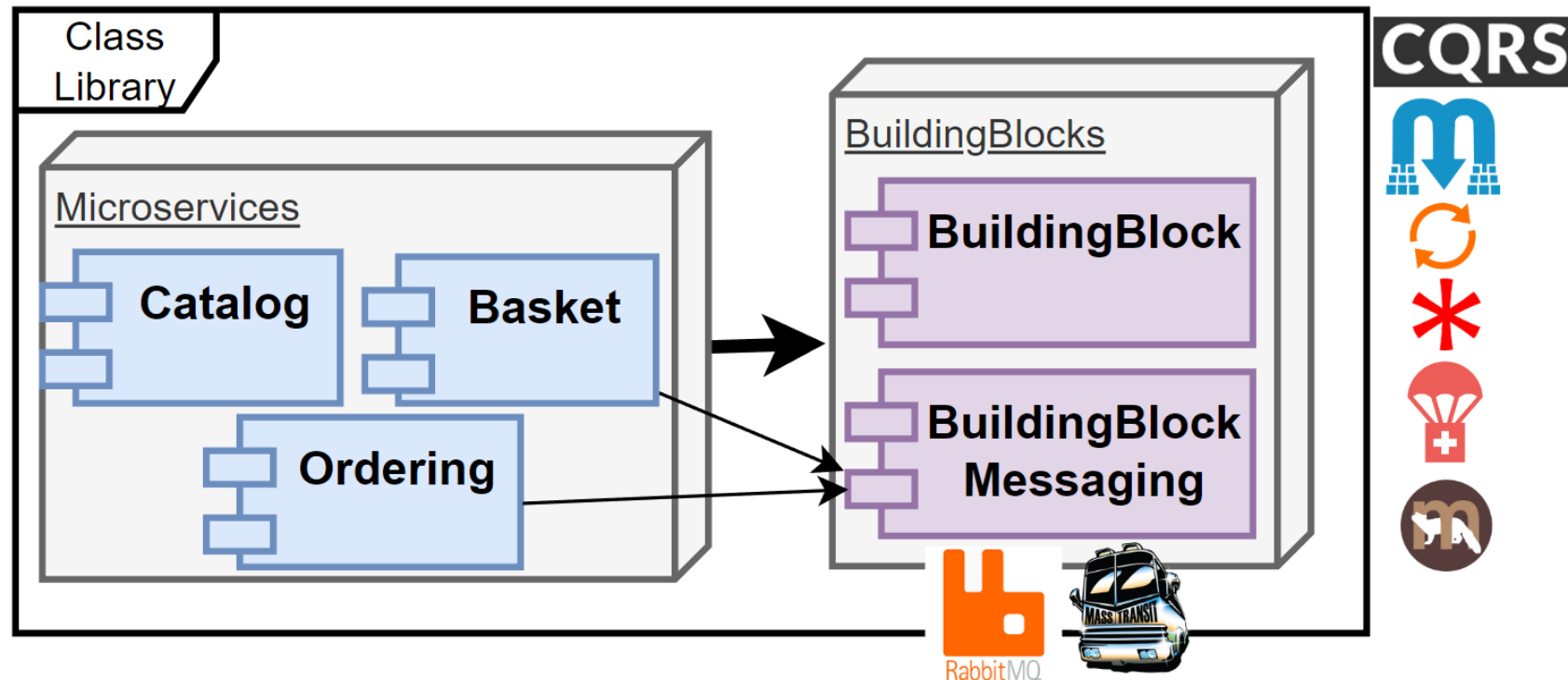


BuildingBlocks

- Contains common classes and logic shared across services.

BuildingBlocks.Messaging

- Dedicated class library for async communication over RabbitMQ, containing common integration events and MassTransit configurations.



3. Libraries Nuget Packages of Async Communication



- **MassTransit:** An abstraction over message brokers for .NET, simplifying the implementation of asynchronous messaging.
- **MassTransit.RabbitMQ:** Specific integration for RabbitMQ with MassTransit.



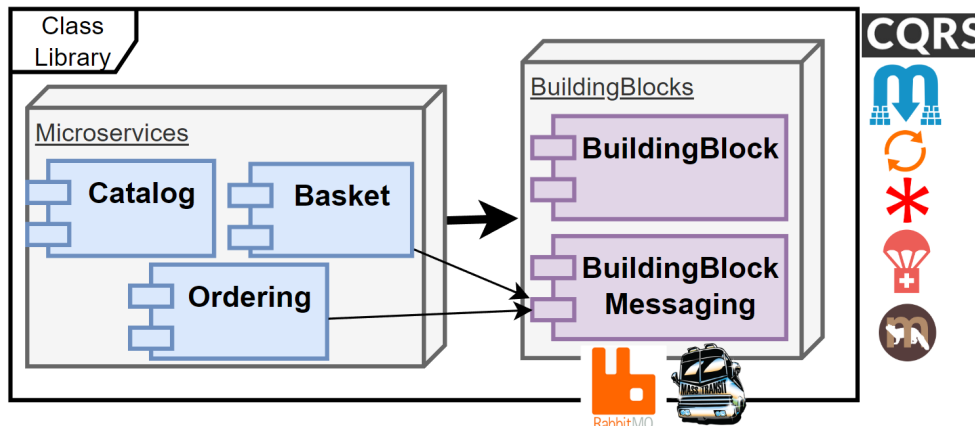
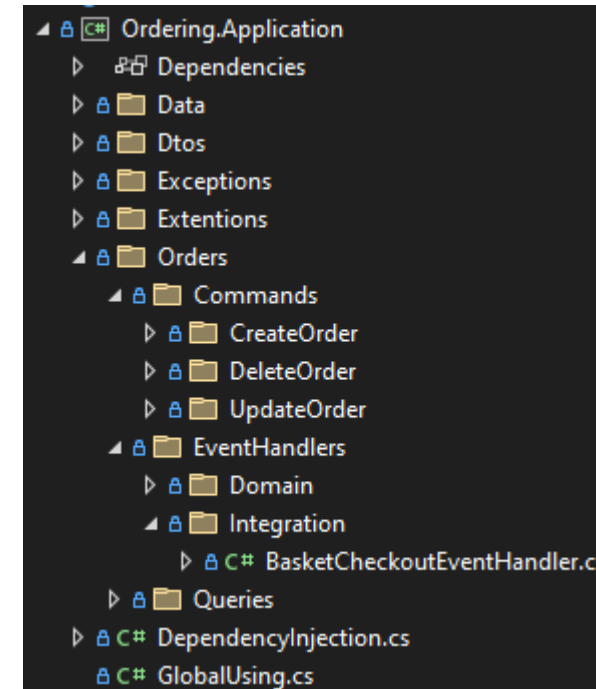
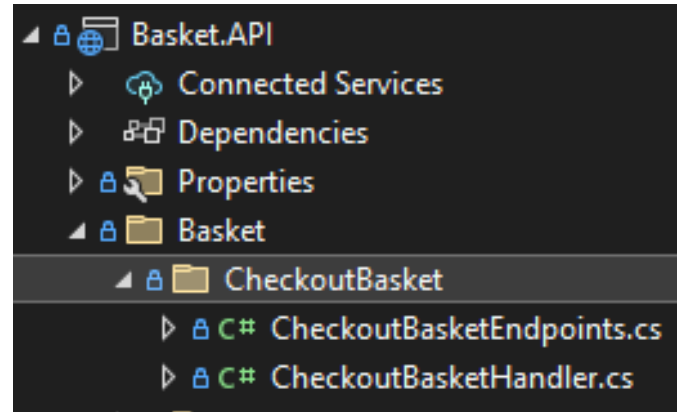
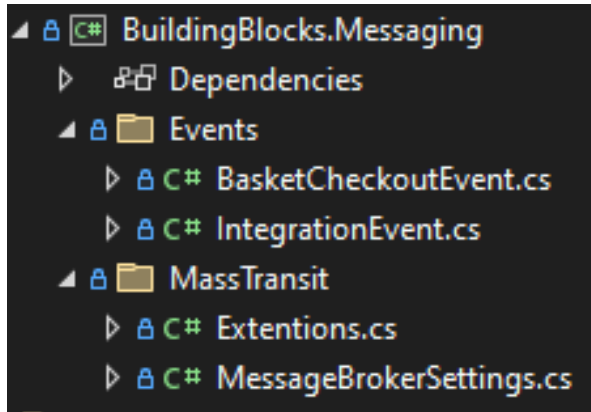
 RabbitMQ



4. Project Folder Structure of Async Communication

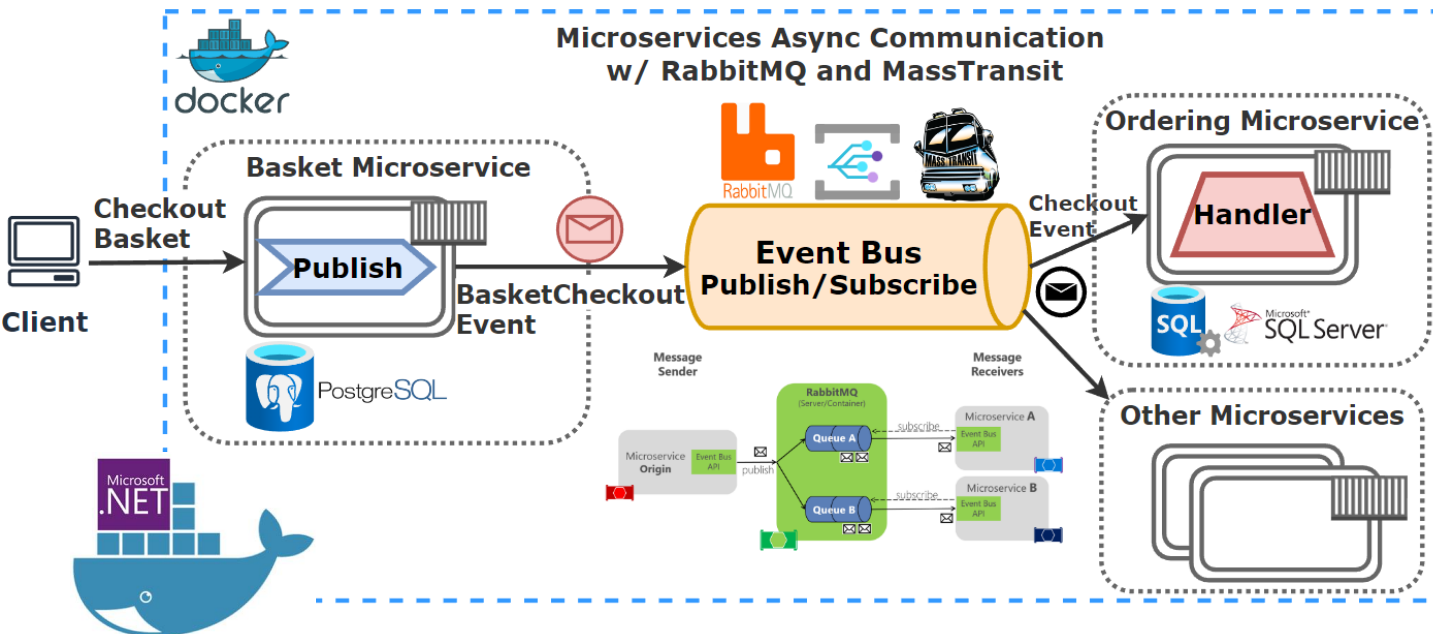


- BuildingBlocks.Messaging
- Basket.API – CheckoutBasket
- Ordering.Application - IntegrationEventHandlers

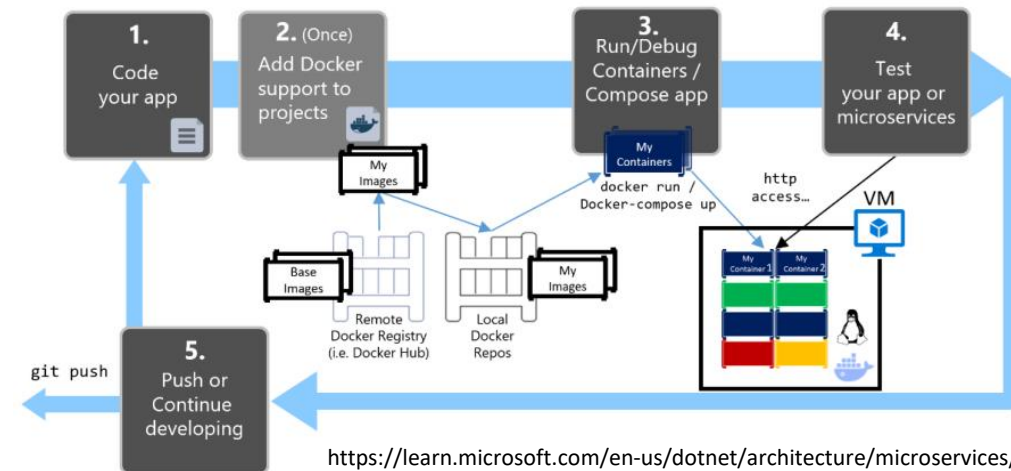


Deployment of Async Communication

- Add **RabbitMQ Image Container** into multi-container Docker-compose file
- Implement **Dockerfile** and **docker-compose** file for running Basket and Ordering microservice and **RabbitMQ Message Broker** in Docker environment.



mehmetozkaya

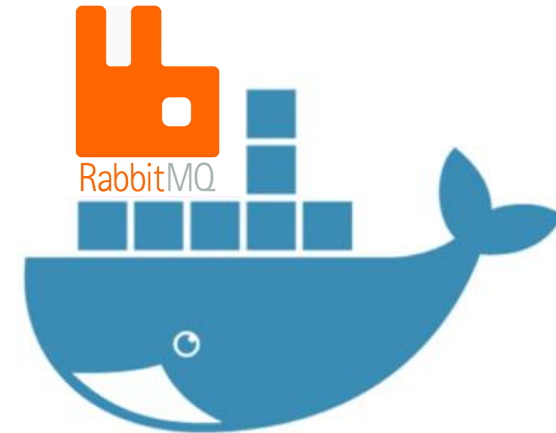
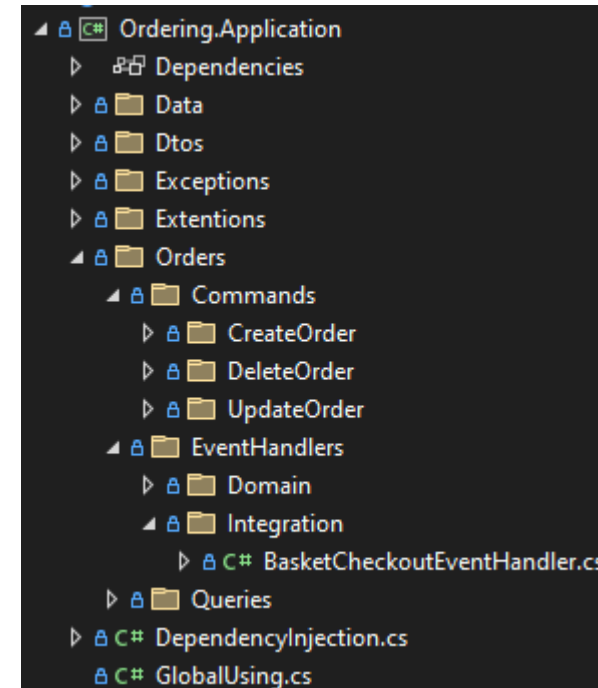
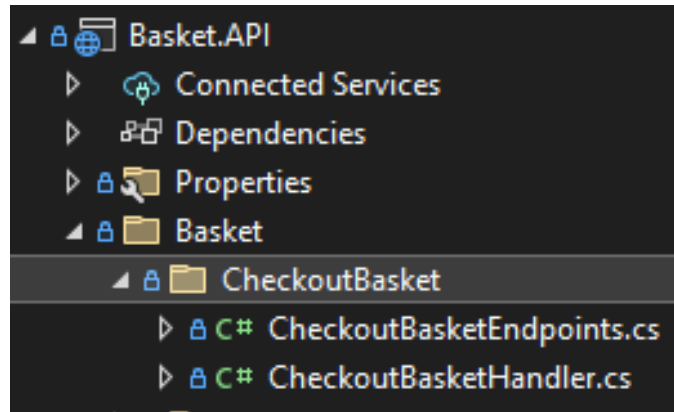
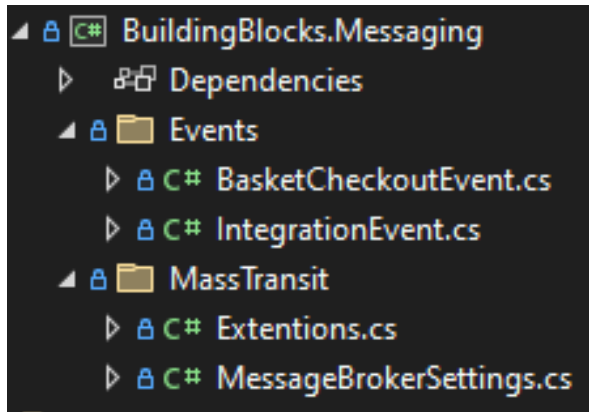


<https://learn.microsoft.com/en-us/dotnet/architecture/microservices/>

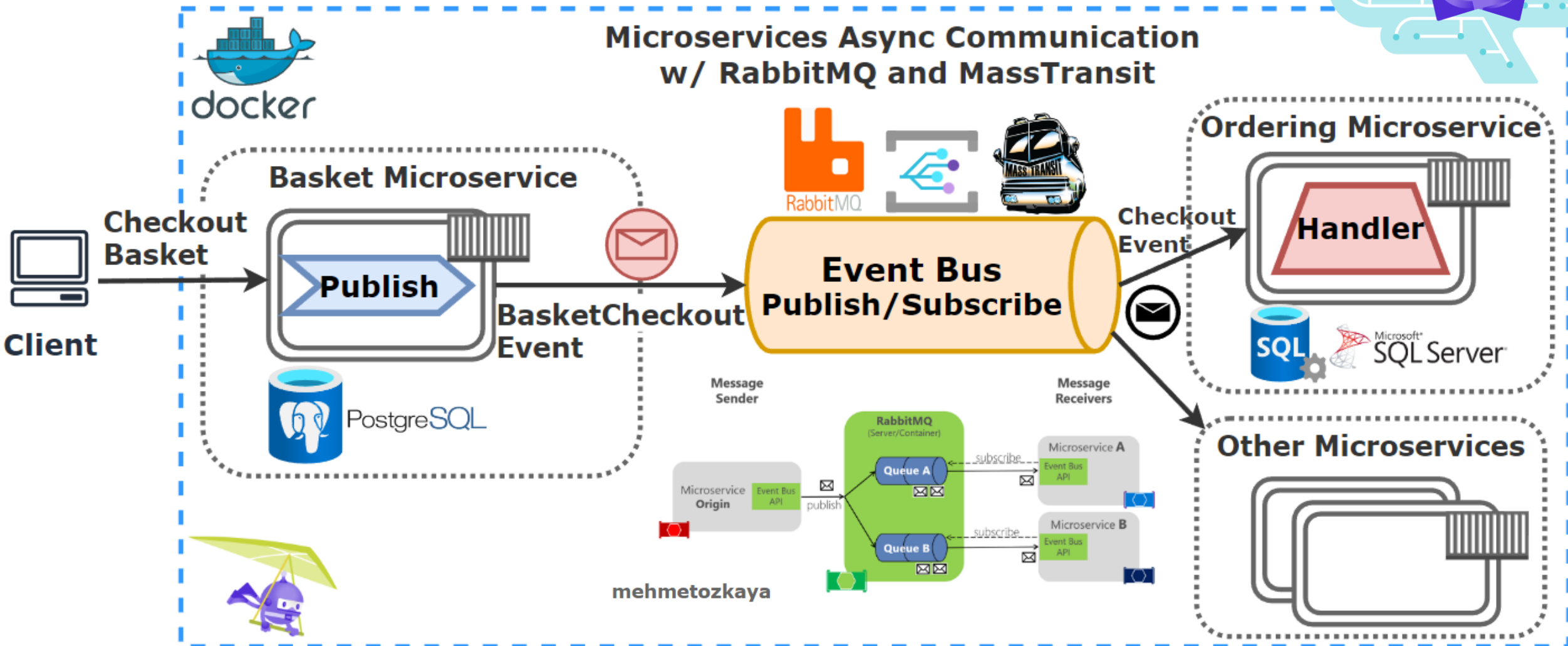
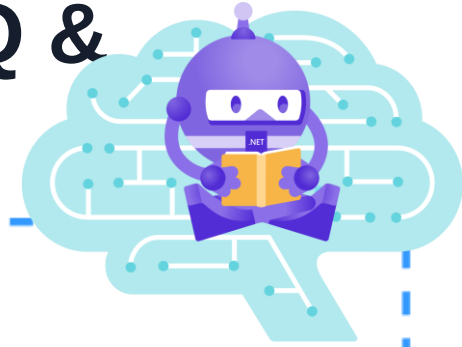
Steps of Developing Async Communication between Basket and Ordering Microservices



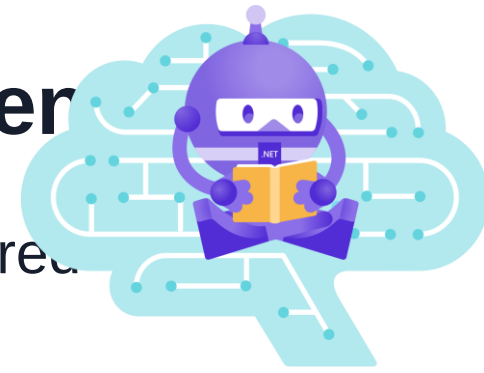
1. BuildingBlocks.Messaging Shared Class Library
2. Basket.API to Publish BasketCheckout Event
3. Ordering Microservices to Subscribe and Consume BasketCheckout Event
4. Docker-compose Add RabbitMQ Image in Multi-container Environment



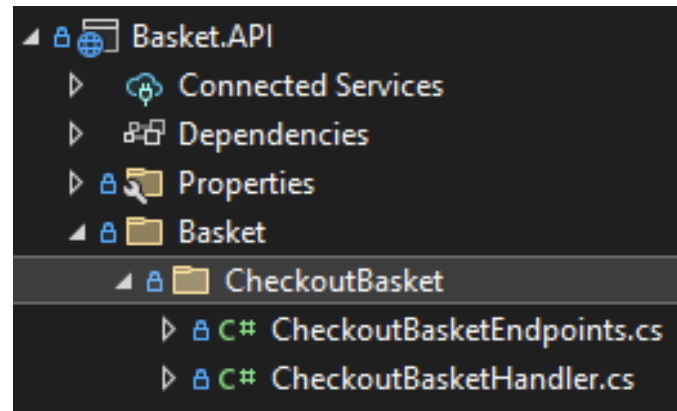
Microservices Async Communication w/ RabbitMQ & MassTransit



Sub Steps of Basket Publish BasketCheckout Event



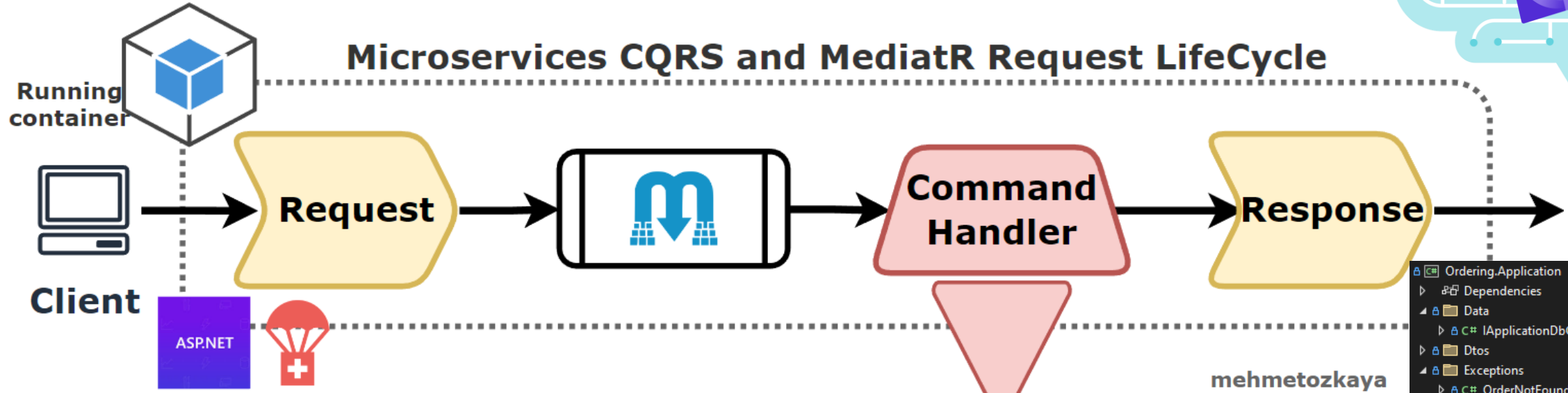
1. Project Reference to BB.Messaging in Basket.API project (to access shared events and MassTransit extensions)
2. Register MassTransit RabbitMQ packages into Basket.API DI in Program.cs file
3. Expose HTTP POST method for Basket Checkout operation in Basket.API Project
4. Develop Basket Checkout Handler Method to publish event to RabbitMQ via MassTransit library



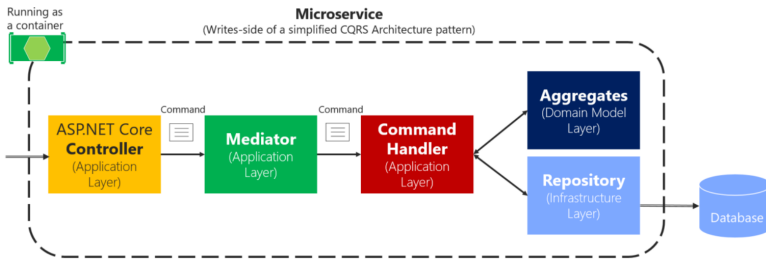
CQRS and MediatR Request LifeCycle



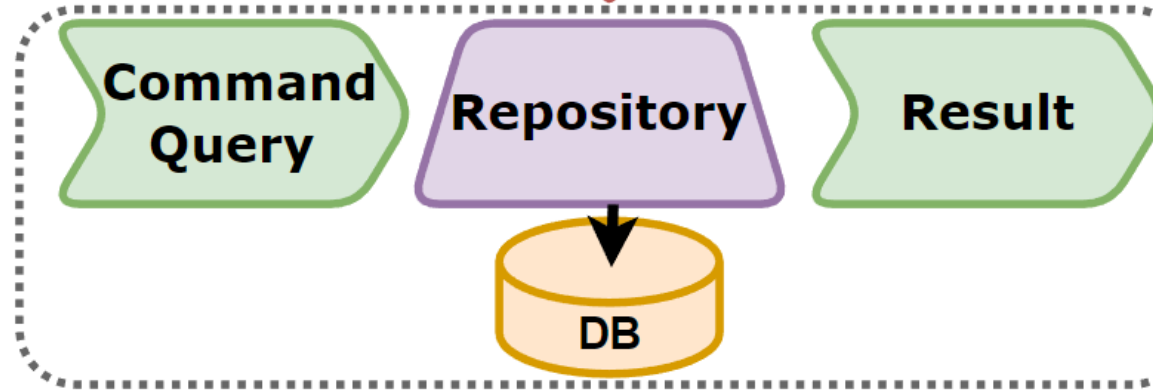
Microservices CQRS and MediatR Request LifeCycle



mehmetozkaya



<https://learn.microsoft.com/en-us/dotnet/architecture/microservices>



CQRS Handler using Repository Classes

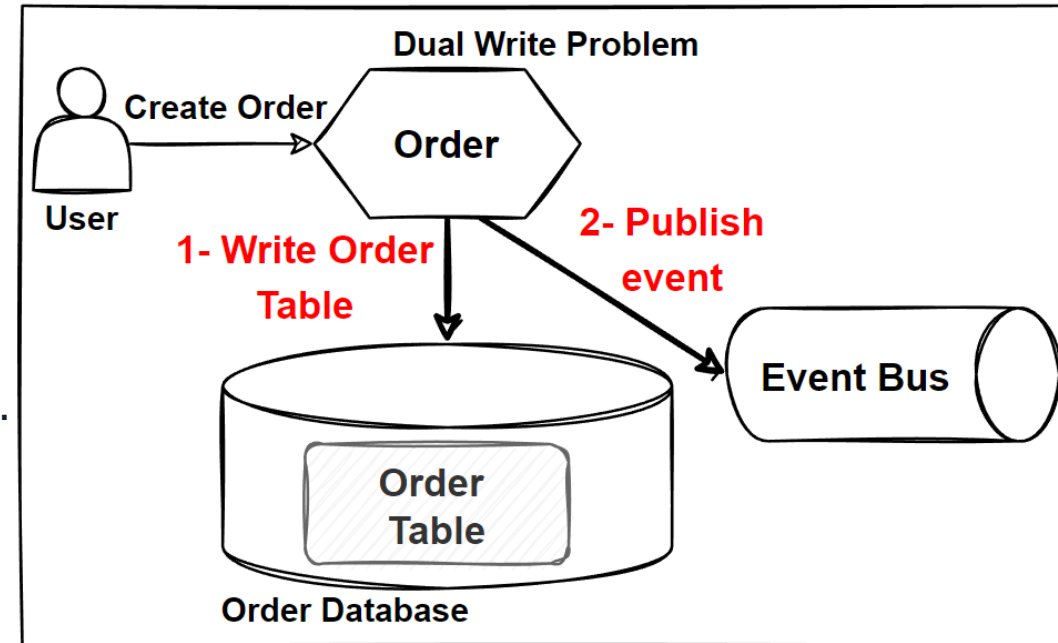
```
Ordering.Application
├── Dependencies
├── Data
│   ├── IApplicationDbContext.cs
├── Dtos
├── Exceptions
│   ├── OrderNotFoundException.cs
├── Extensions
│   ├── OrderExtensions.cs
├── Orders
│   ├── Commands
│   │   ├── CreateOrder
│   │   │   ├── CreateOrderCommand.cs
│   │   │   ├── CreateOrderHandler.cs
│   │   ├── DeleteOrder
│   │   ├── UpdateOrder
├── EventHandlers
│   ├── Domain
│   │   ├── OrderCreatedEventHandler.cs
│   │   ├── OrderUpdatedEventHandler.cs
├── Integration
│   ├── BasketCheckoutEventHandler.cs
├── Queries
│   ├── GetOrderByName
│   ├── GetOrders
│   ├── GetOrdersByCustomer
├── DependencyInjection.cs
├── GlobalUsing.cs
```

Problem: Dual Write Problem

- When application **needs to change data** in **two different systems**, i.e. a database and a message queue, if one of the writes fails, it can result in inconsistent data.
- Happens when you use a **local transaction** with **each of the external systems** operations.
- I.e. app needs to **persist data** in the **database** and **send a message** to **Kafka** for notifying other systems.
- **If one of these two operations fails**, the **data** will be **inconsistent** data and these two systems becomes inconsistent.

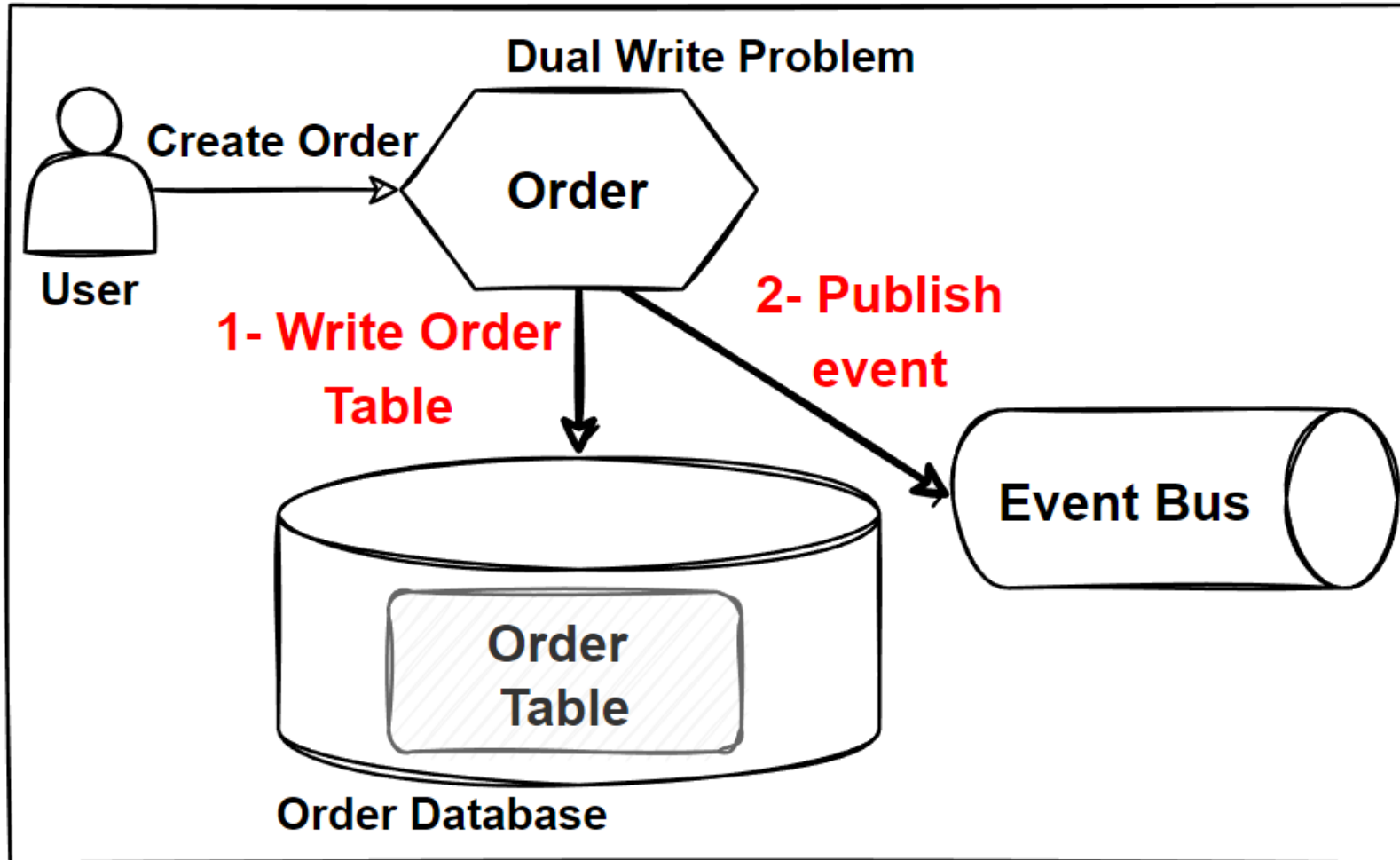
Problems

- Data loss or corruption.
- Difficult to resolve without proper error handling and recovery mechanisms.
- Dual writes can be hard to detect and fix.



Example of Dual-write problem: Create Order

- 1- Change data in Order database with creating new record
- 2- Send an order_created event to the EventBus like Apache Kafka.



Run Transactions

- 1, 2 or
- 2, 1
- Both becomes dual writes

How to avoid dual write problems in microservices ?

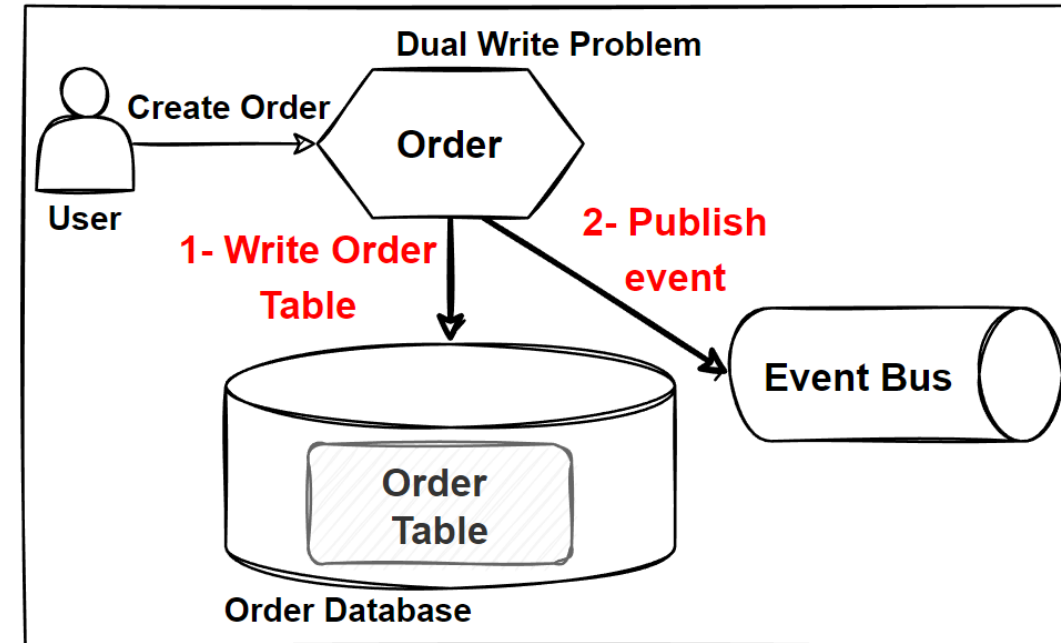
- **Monolith** applications use the **2 phase commit** protocol.
- It **splits the commit** process of the transaction into 2 steps and ensures the **ACID principles** for all systems.
- **Can't use 2-phase commit** transactions when building **microservices**.
- These transactions **require locks** and **don't scale well**.
- Need all systems to be **up and running** at the same time.

Solutions

- Transactional Outbox Pattern
- CDC - Change Data Capture

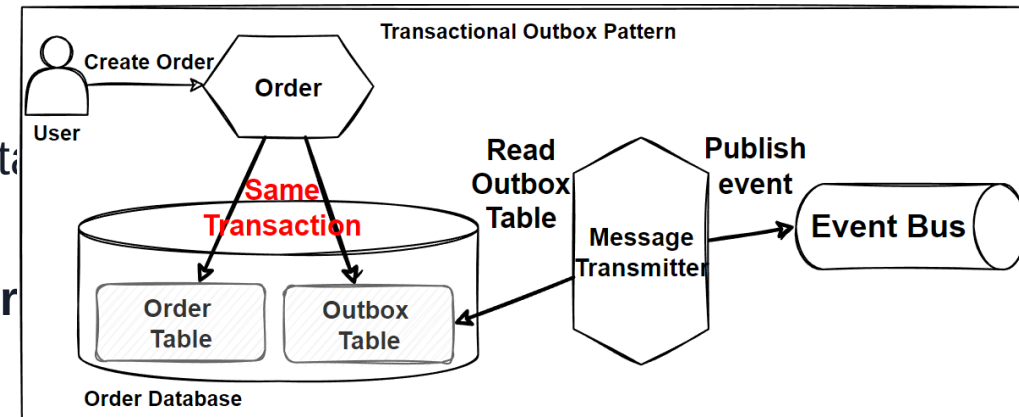
Best Practice

- Use Red Hat **Apache Kafka** and **CDC** using **Debezium** in event-driven applications.
- Use New databases like **CockroachDB** which has built-in **Change Data Capture** feature.



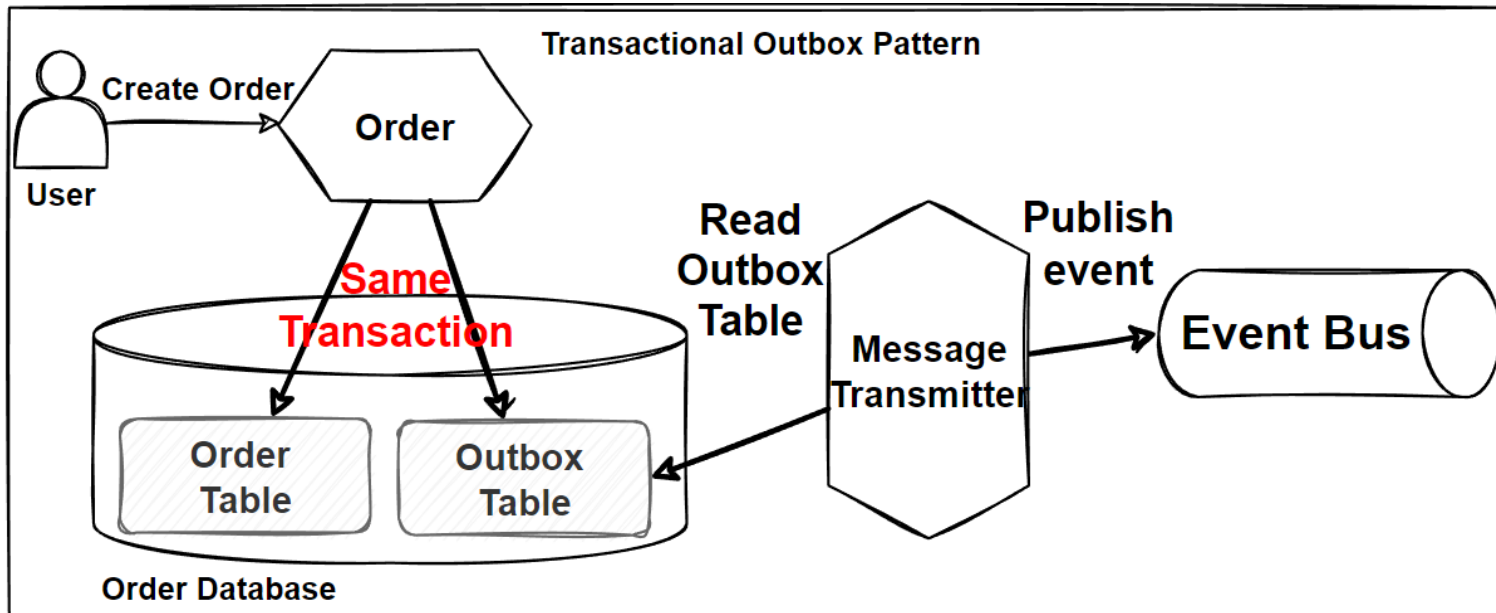
Transactional Outbox Pattern

- The idea is to have an “**Outbox**” **table** in the microservice’s database. It provides to publish events reliably.
- **Dual write problem** happens when application needs to change data in two different systems.
- Instead of sending the data to two separate locations, **send a single transaction** that will **store two separate copies** of the data on the database.
- **One copy** is **stored** in the relevant **database table**, and the **other copy** is **stored** in an **outbox table** that will publish to event bus.
- When API **publishes event messages**, it doesn’t directly send them, Instead, the **messages** are **persisted** in a **database table**.
- After that, **a job publish events** to message broker system in predefined time intervals.
- **Events** are **not written directly** to a event bus, it is written to a table in the “**Outbox**” role of the service.



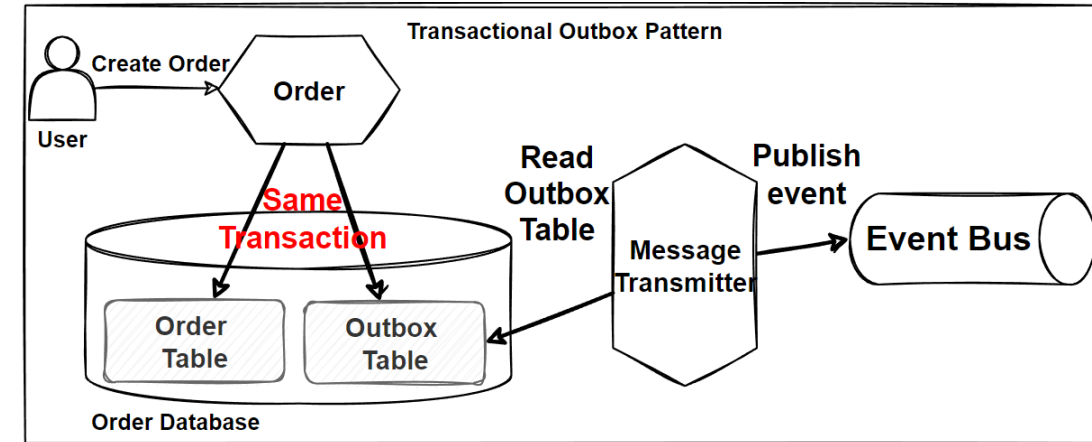
Transactional Outbox Pattern - 2

- Transaction **performed before the event** and the **event written** to the **outbox table** are part of the **same transaction**.
- When a **new order is added** to the system, the process of adding the order and writing the **Order_Created event** to the **Outbox table** is done **in the same transaction** to ensure the event is saved to the database.
- If one of the **process is fail**, this will **rollback the whole operations** with following ACID principles.
- The second step is to **receive these events** written to the **Outbox table** by an **independent service** and **write** them to the **Event bus**. Another service **listen** and **polls the Outbox table** records and publish events.



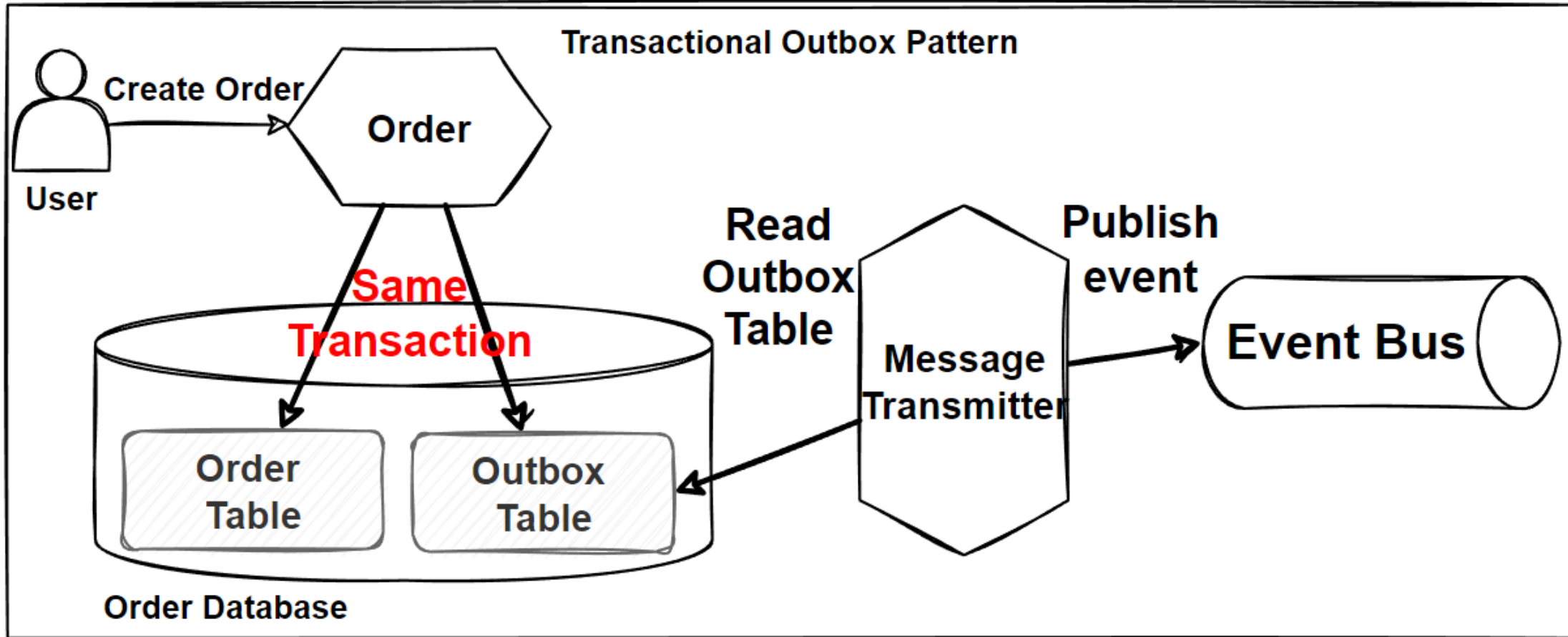
Transactional Outbox Pattern in Microservices

- **Microservice** provides an **outbox table** within its **database**. Outbox table will **include** all the **events**.
- There will be a **CDC (change data capture) plugin** that **reads** the **commit log** of the **outbox table** and **publish the events** to the relevant queue.
- It provides that **messages** are **reliably delivered** from a **microservice** to **another microservice** even if the transaction that triggered the message fails.
- It involves **storing** the **message** in a **local "Outbox" table** within the microservice, that message sent to the consumer after the transaction is committed.
- **Outbox pattern** can be used to ensure that **messages** are **delivered consistently**, even if the microservice that sent the message is **unavailable** or **experiencing errors**.
- Useful for **communicating important information** or updates between services.



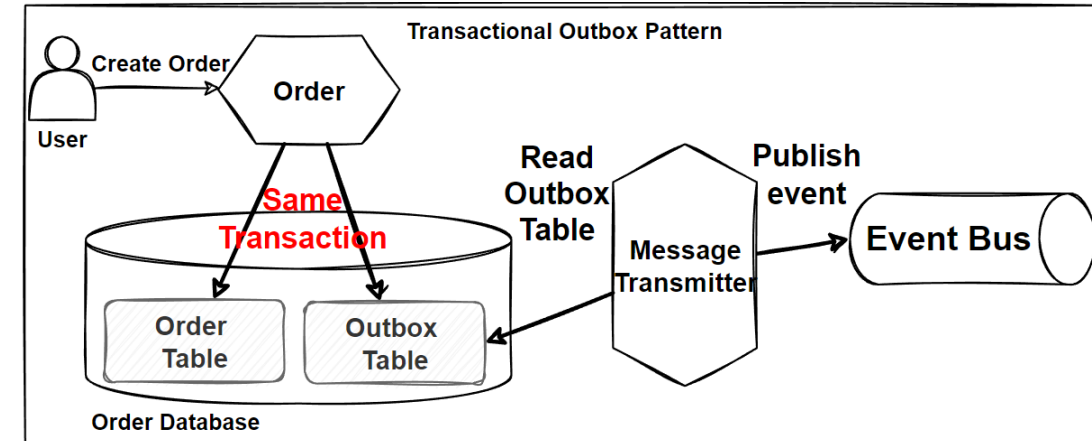
Customer Places an Order Use Case

Transactional Outbox pattern in Microservices



Why Use Outbox Pattern ?

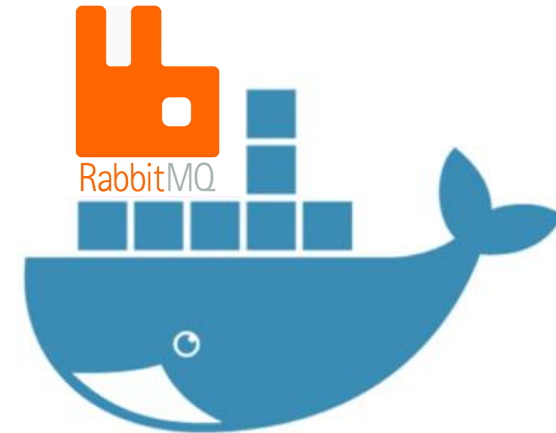
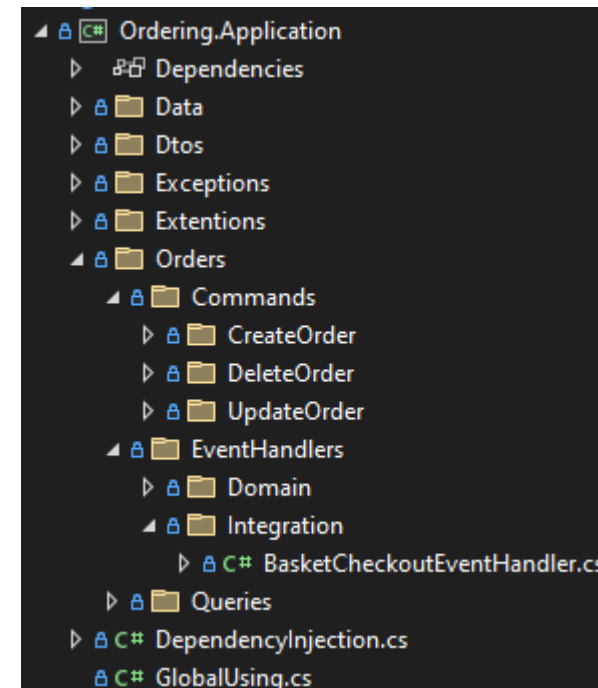
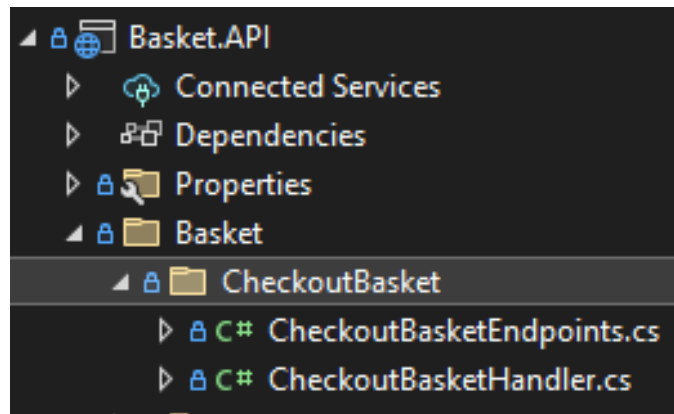
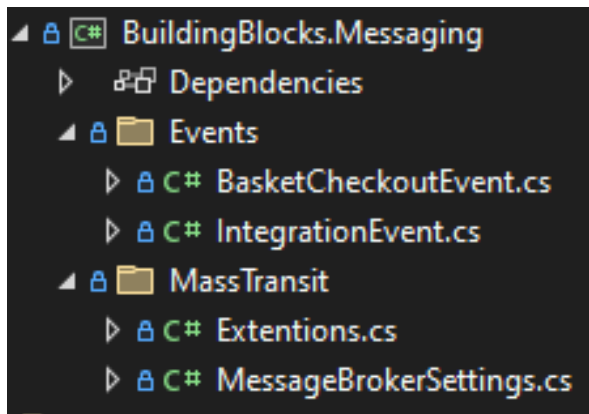
- When **working with critical data** that **need to consistent** and **need to accurate** to catch all requests.
- When the **database update** and **sending of the message** should be **atomic** to make sure **data consistency**.
- For example the **order sale transactions**, because they are about **financial business**. Thus, the calculations **must be correct 100%**.
- To access this **accuracy**, must be sure that our system is not **losing any event messages**.
- **The Outbox Pattern** should be applied this kind of cases.

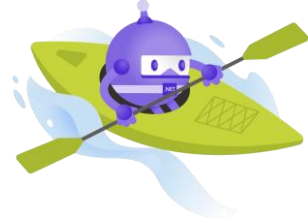


Steps of Developing Async Communication between Basket and Ordering Microservices



1. BuildingBlocks.Messaging Shared Class Library
2. Basket.API to Publish BasketCheckout Event
3. Ordering Microservices to Subscribe and Consume BasketCheckout Event
4. Docker-compose Add RabbitMQ Image in Multi-container Environment





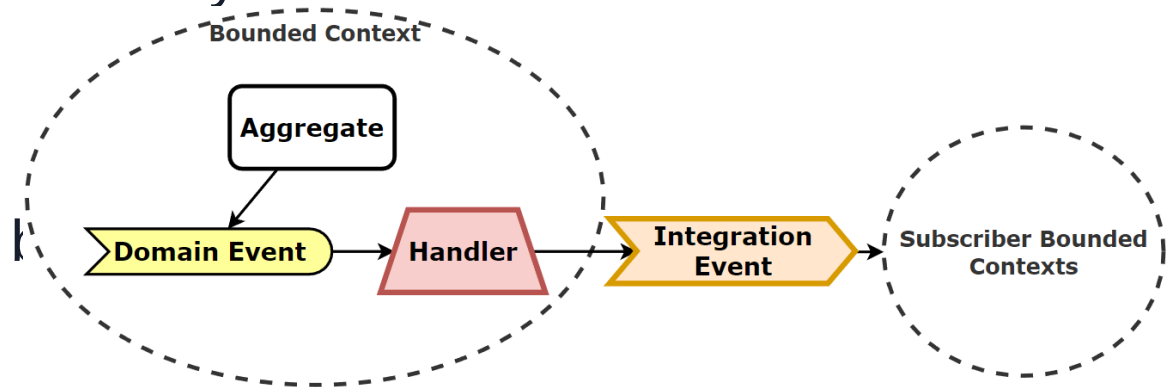
Domain vs Integration Events

Domain Events:

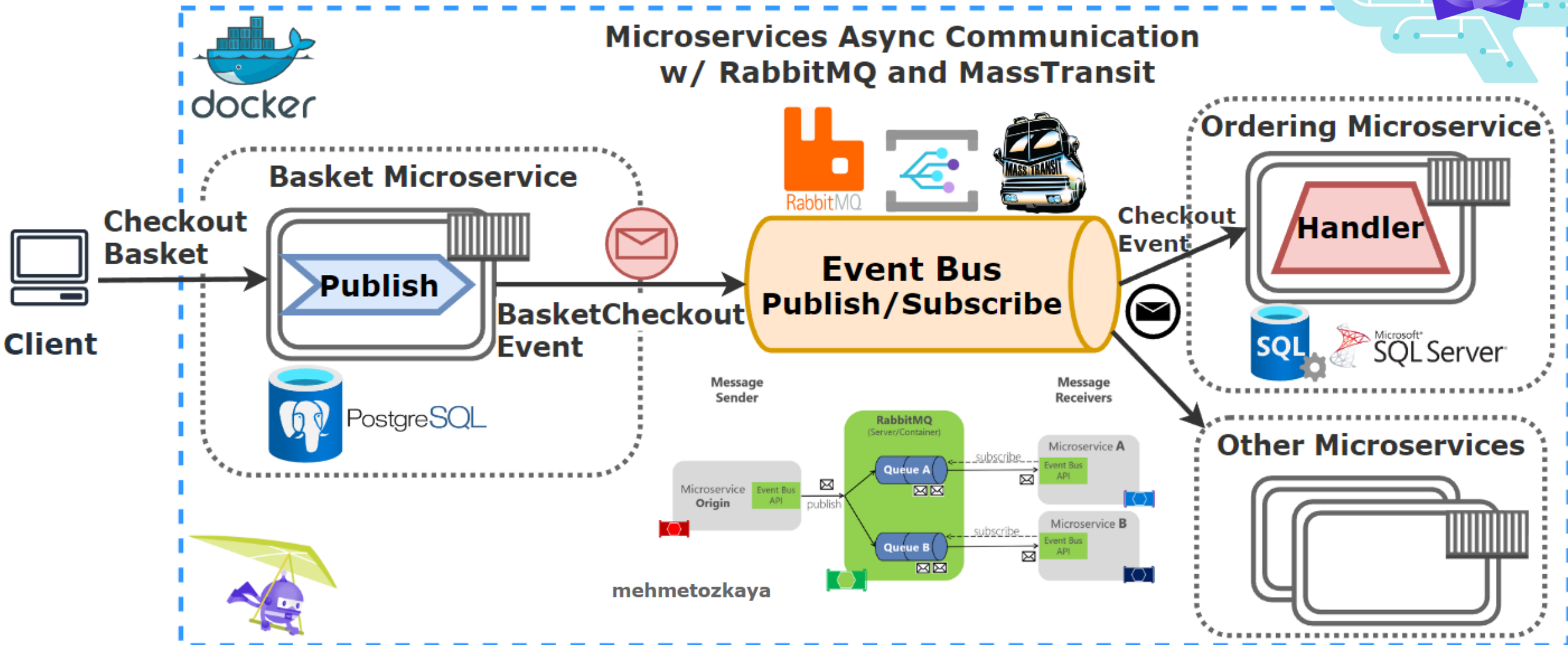
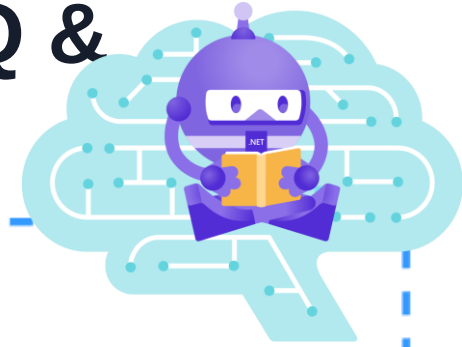
- Published and consumed within a single domain. Strictly within the boundary of the microservice/domain context.
- Indicate something that has happened within the aggregate.
- In-process and synchronously, sent using an in-memory message bus. **Example:** OrderCreatedEvent.

Integration Events:

- Used to communicate state changes or events between bounded contexts or microservices.
- Overall system's reaction to certain domain events.
- Asynchronously, sent with a message broker over a queue.
- **Example:** After handling Created, an OrderCreatedIntegrationEvent published to a message broker like RabbitMQ, then consumes by other microservices.

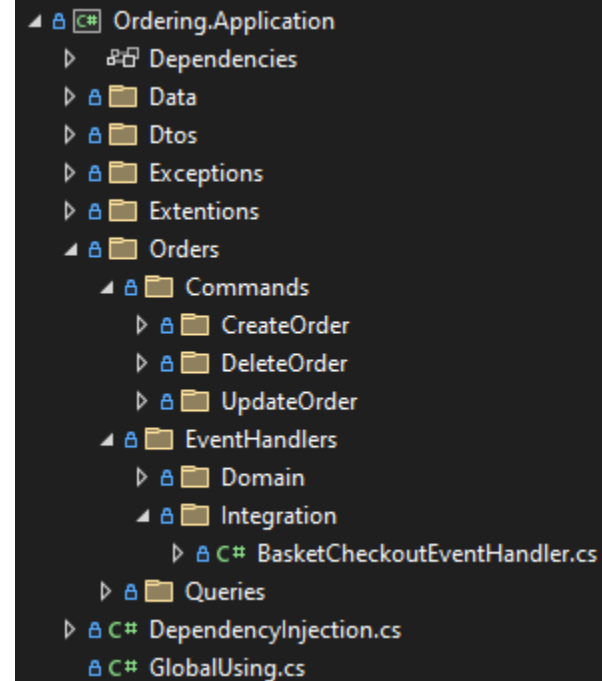


Microservices Async Communication w/ RabbitMQ & MassTransit



Sub Steps of Ordering Subscribe-Consume BasketCheckout

1. Project Reference to BB.Messaging in Ordering.Application project (to access shared events and MassTransit extensions)
2. Register MassTransit RabbitMQ packages into Ordering.Application DI in DependencyInjection.cs file
3. Develop Basket Checkout Event Consumer integration event handler class to perform checkout and order create operation
4. BONUS- Publish OrderCreated Integration event for order fulfillment microservices and implement SAGA Pattern



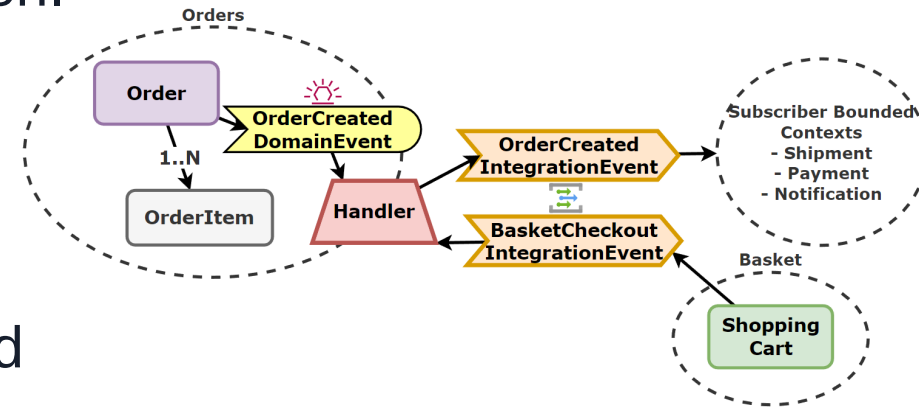
EShop Microservices Domain and Integration Events

1- BasketCheckout Integration Events:

- BasketCheckout is an integration event that published from Basket microservices to RabbitMQ.
- Subscribe-Consume by Ordering Microservices from RabbitMQ using MassTransit to perform async event-driven.

2- OrderCreated Domain Event:

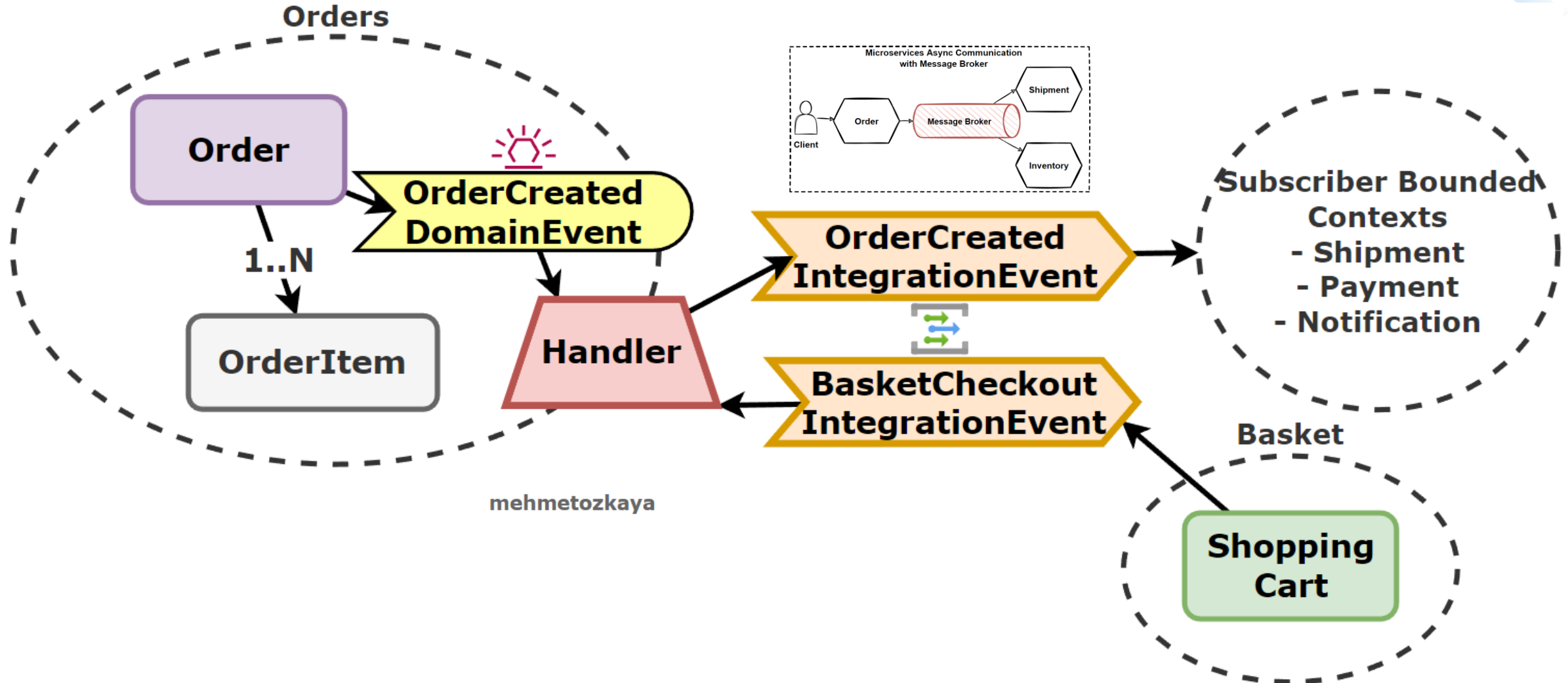
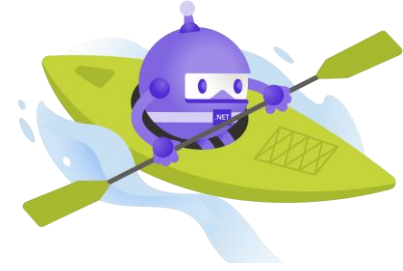
- When consumes BasketCheckout Integration Event into Handler class, it will create a new Order into Ordering ms.
- When a new Order is created, it will raise an OrderCreated domain event.



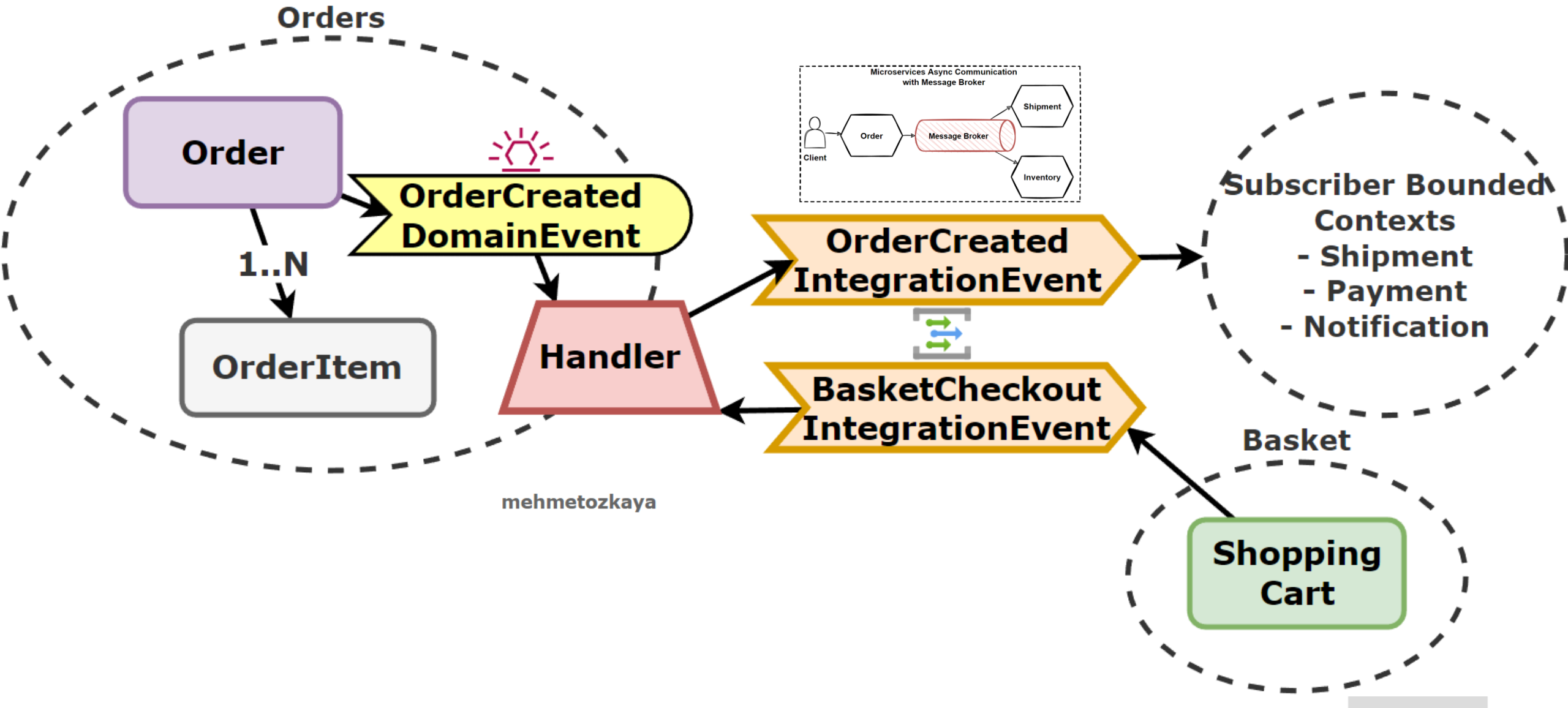
3- OrderCreated Integration Event for Order Fullfilment:

- Once the OrderCreated domain event is handled, an OrderCreatedIntegrationEvent is published for order fullfilment microservices and implement SAGA Pattern.

Eshop Domain and Integration Events

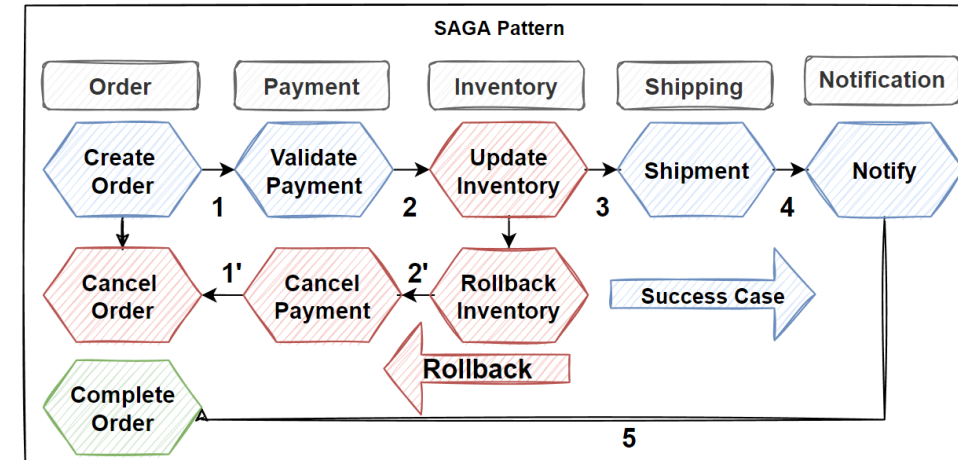


Order Creation Flow



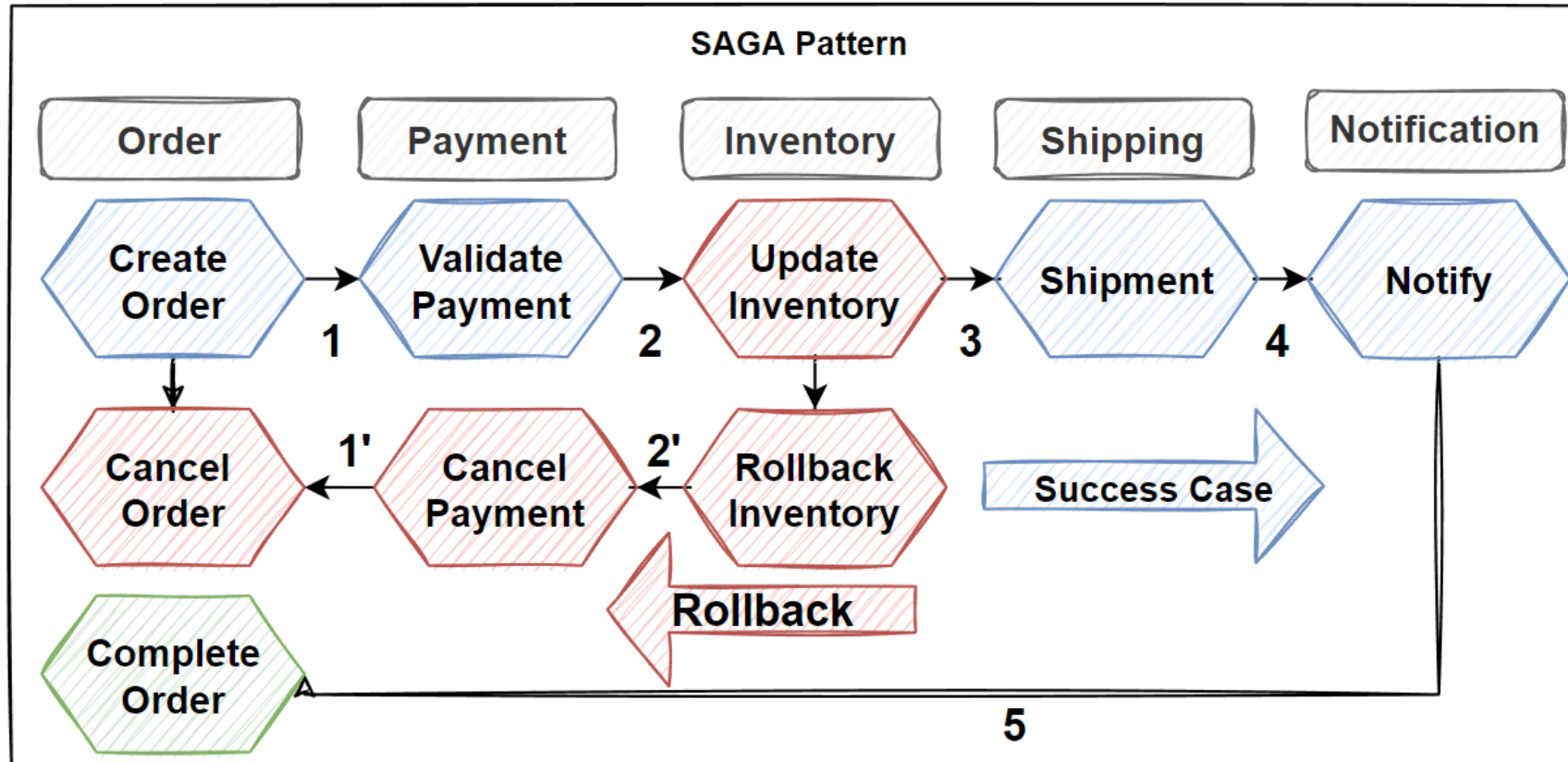
Saga Pattern for Distributed Transactions

- **Saga design pattern** is provide to **manage data consistency** across microservices in **distributed transaction** cases.
- **Saga** offers to **create a set of transactions** that **update microservices sequentially**, and **publish events** to **trigger the next transaction** for the next microservices.
- If **one of the step is failed**, than saga patterns **trigger to rollback transactions**, do **reverse operations** with **publishing rollback events** to previous microservices.
- **Publish/subscribe pattern** with brokers or **API composition**.
- **SAGA pattern** manage **long-running transactions** that involve multiple microservices which is a **series of local transactions** that work together to achieve **E2E use case**.
- Useful in distributed systems, where **multiple microservices** need to **coordinate their actions**.
- Ensure that the **overall transaction is either completed successfully**, or is **rolled back** to its **initial state**. (compensating transaction)



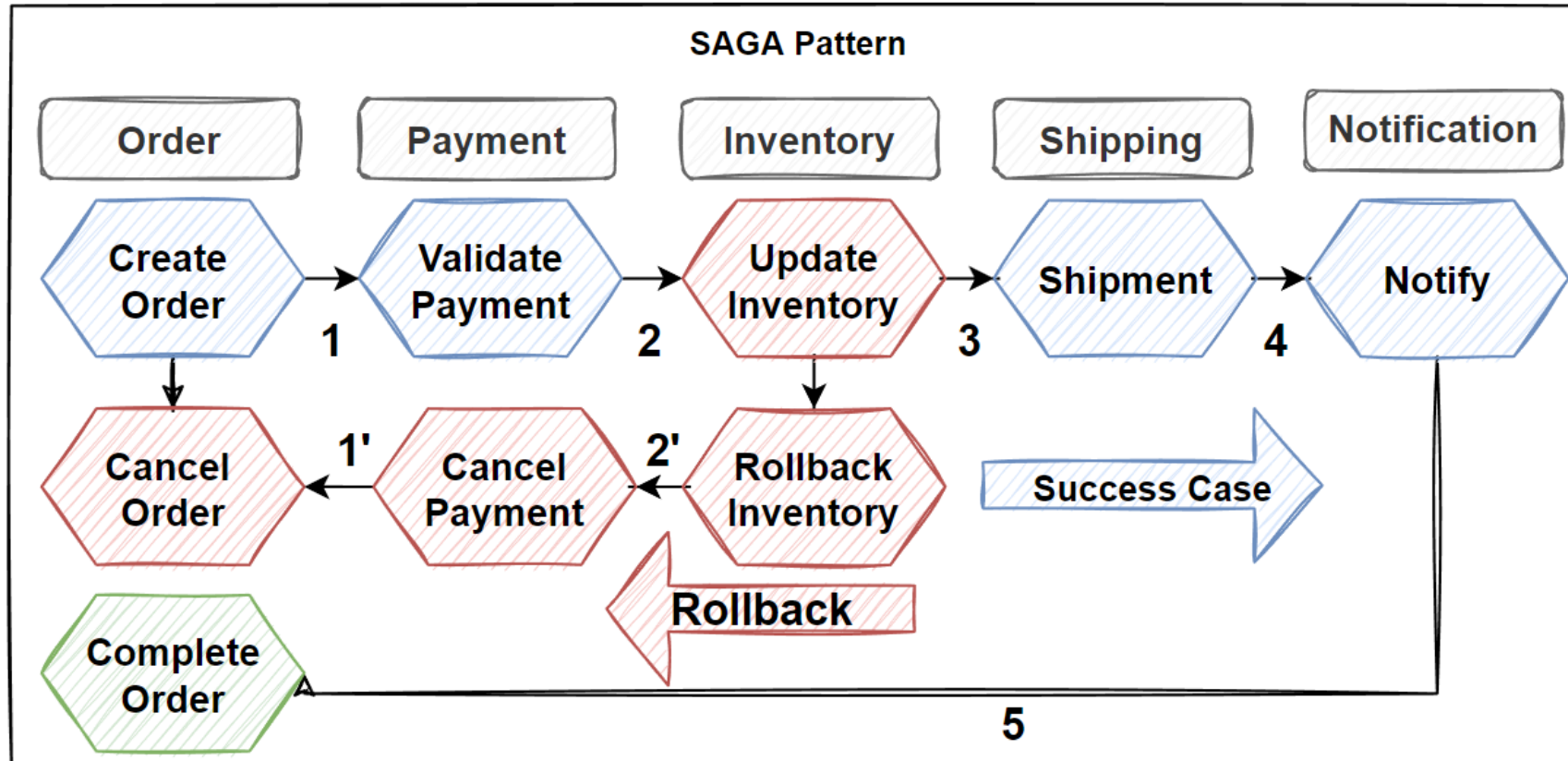
Saga Pattern for Distributed Transactions - 2

- Saga pattern provides transaction management with **using a sequence of local transactions** of microservices. And **grouping these local transactions** and **sequentially invoking** one by one.
- Each **local transaction updates the database** and **publishes an event** to trigger the **next local transaction**. If **one of the step is failed**, than saga patterns **trigger to rollback transactions**.



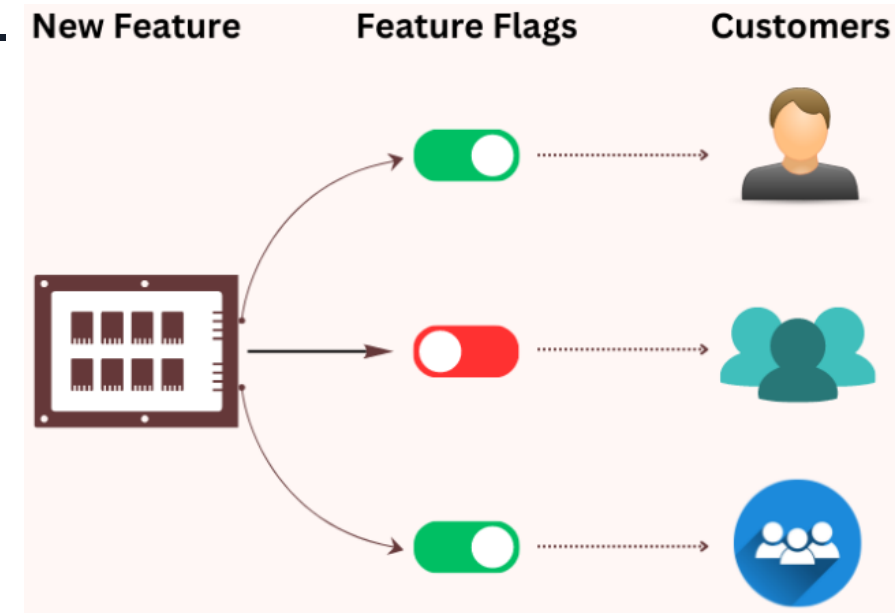
Types of Saga Implementation

- There are **two type of saga** implementation ways:
- **Choreography-based SAGA Implementation**
- **Orchestration-based SAGA Implementation**



Aspnet Feature Management for Feature Flags

- **Feature Flags** are a powerful technique, allowing teams to modify system behavior without changing code.
- They **enable functionalities** to be **turned on** or **off** dynamically, providing **flexibility** in deployment and testing.
- **ASP.NET Core** supports **feature management**, integrating seamlessly with the existing configuration system.
- **Microsoft.FeatureManagement.AspNetCore**: A NuGet package that offers a streamlined approach to implement feature flags.
- Why Use Feature Flags for "**OrderFulfillment**"?
- **Problem**: Unnecessary event-messages in RabbitMQ and premature triggering of order fulfillment processes.
- **Solution**: Implement a feature flag for the OrderFulfillment event, allowing it to be enabled or disabled based on the readiness of other microservices.

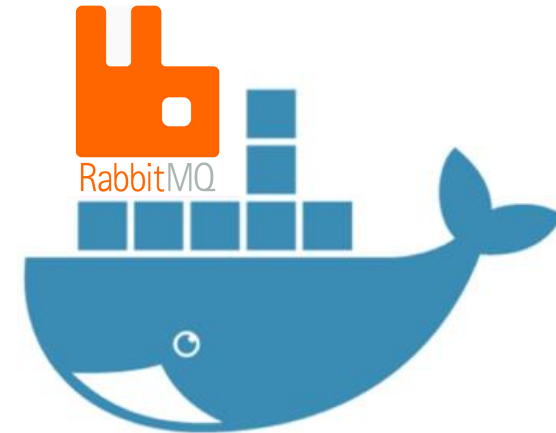
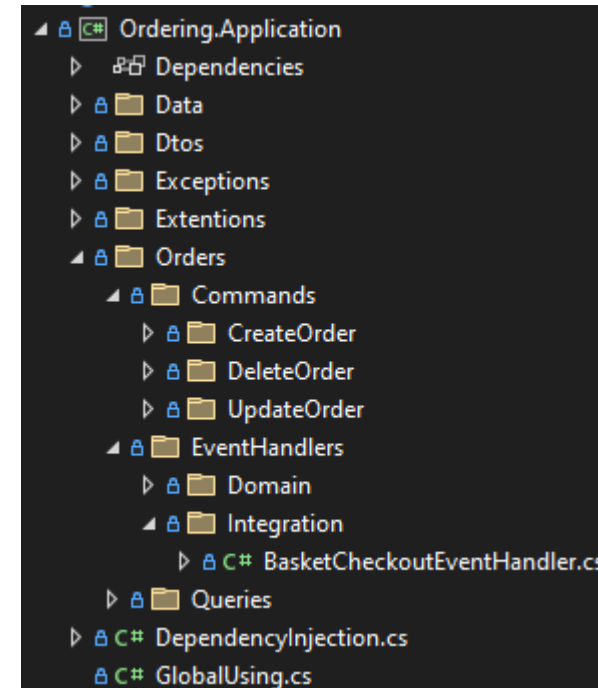
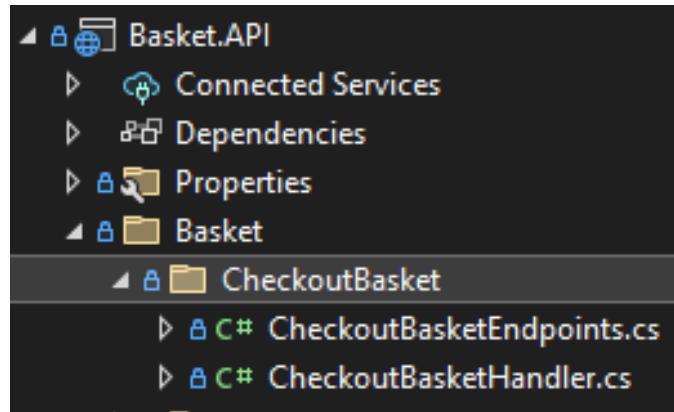
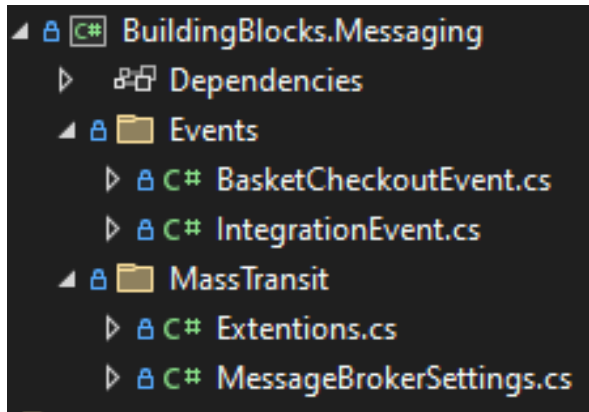


<https://www.ezzylearning.net/tutorial/implement-features-management-in-asp-net-core-app>

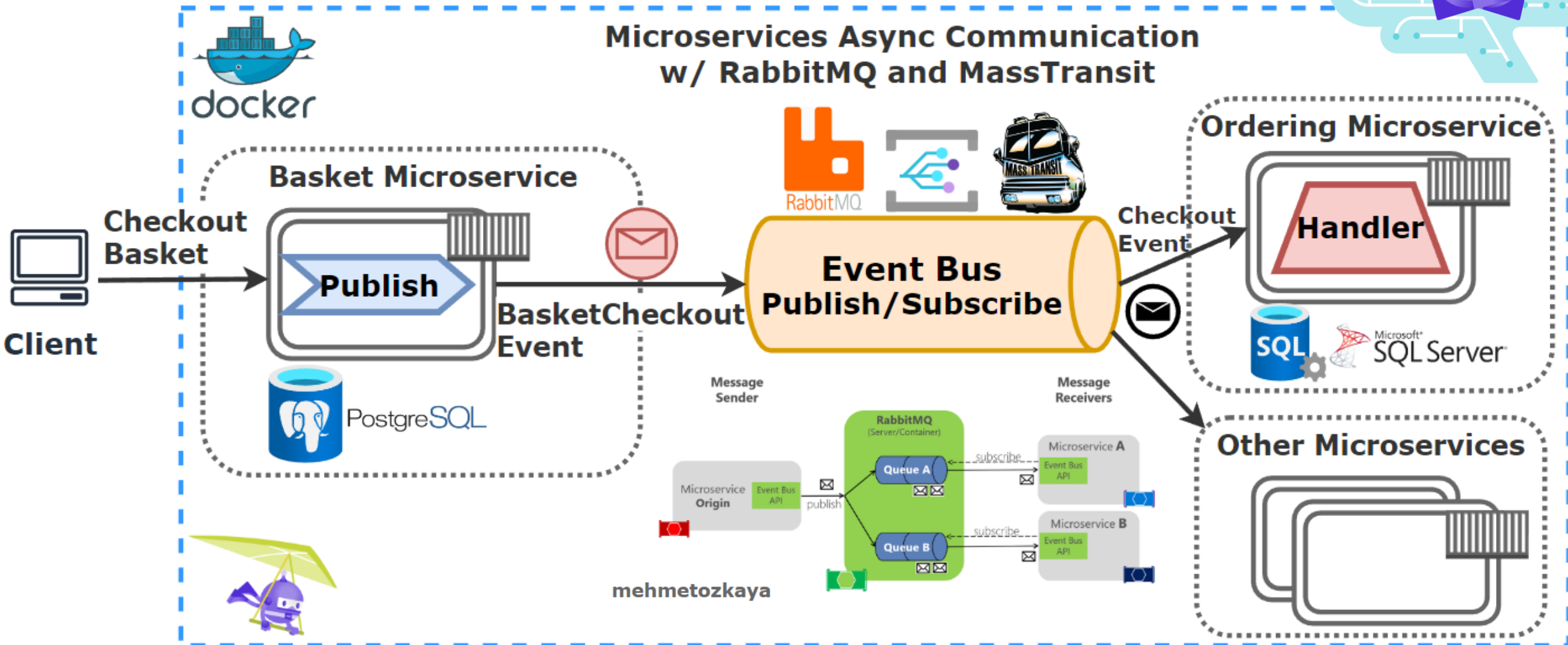
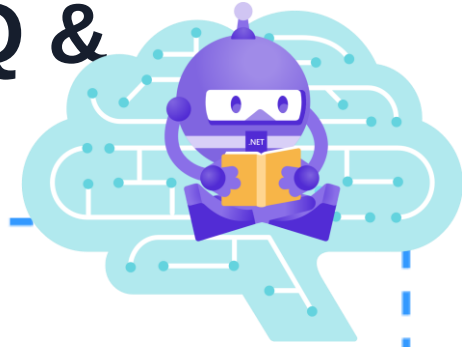
Steps of Developing Async Communication between Basket and Ordering Microservices



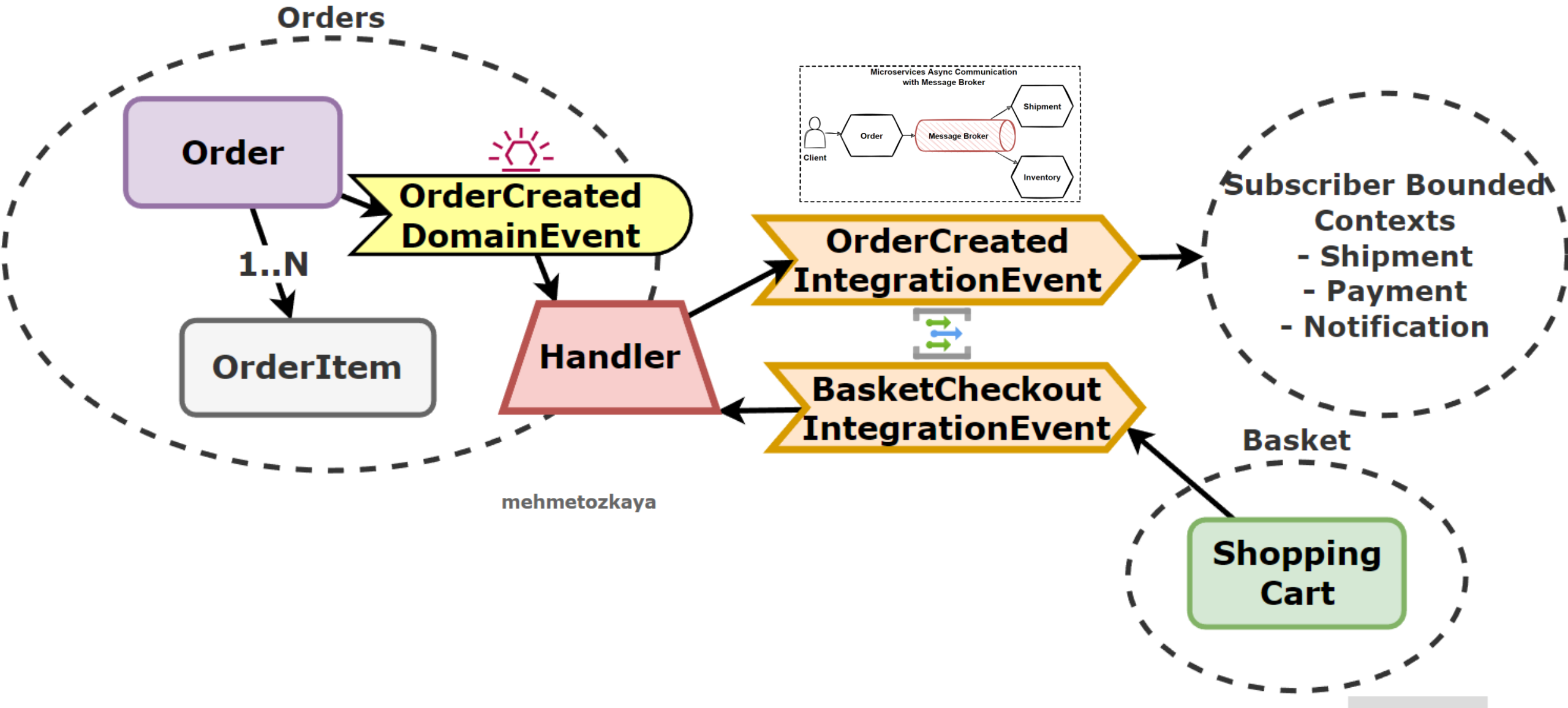
1. BuildingBlocks.Messaging Shared Class Library
2. Basket.API to Publish BasketCheckout Event
3. Ordering Microservices to Subscribe and Consume BasketCheckout Event
4. Docker-compose Add RabbitMQ Image in Multi-container Environment



Microservices Async Communication w/ RabbitMQ & MassTransit

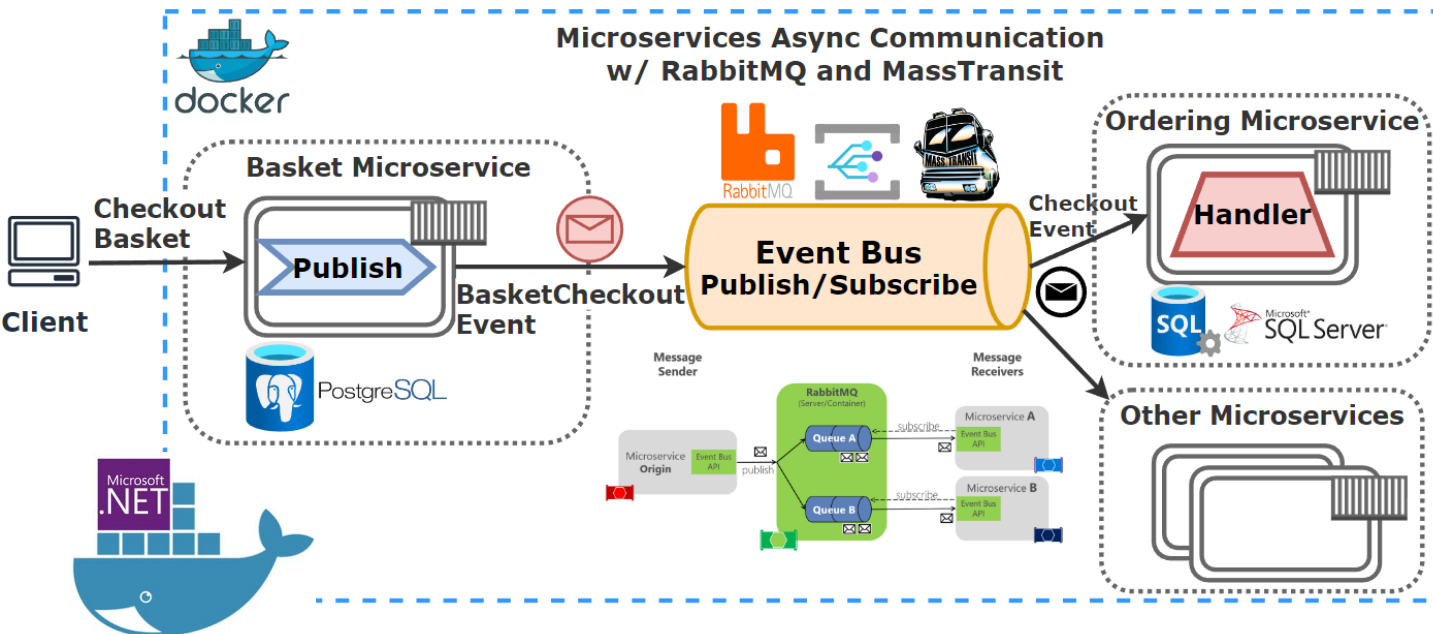


Order Creation Flow

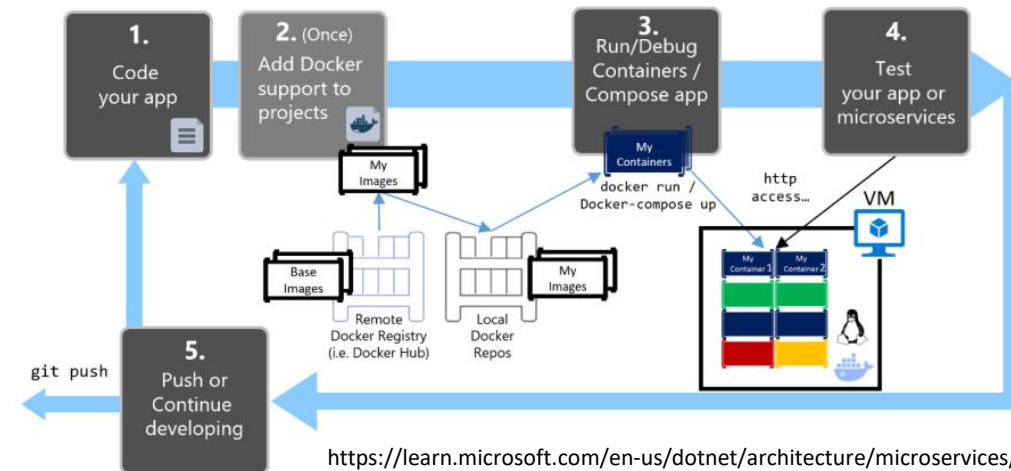


Deployment of Async Communication

- **Containerize** our service with **Docker**, ensuring deployment and integration with **RabbitMQ**
- Implement **Dockerfile** and **docker-compose** file for running Basket and Ordering microservice and **RabbitMQ Message Broker** in Docker environment.

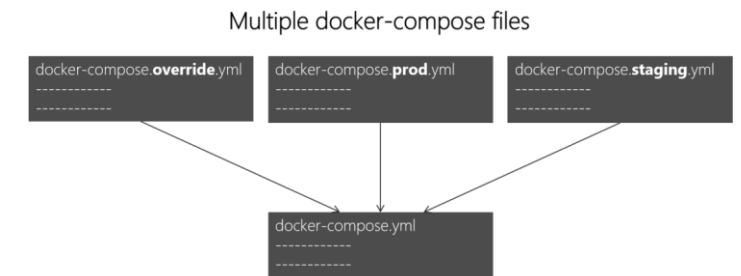
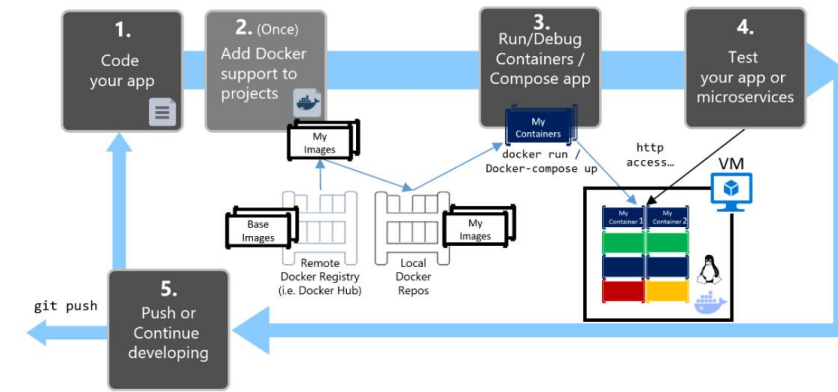
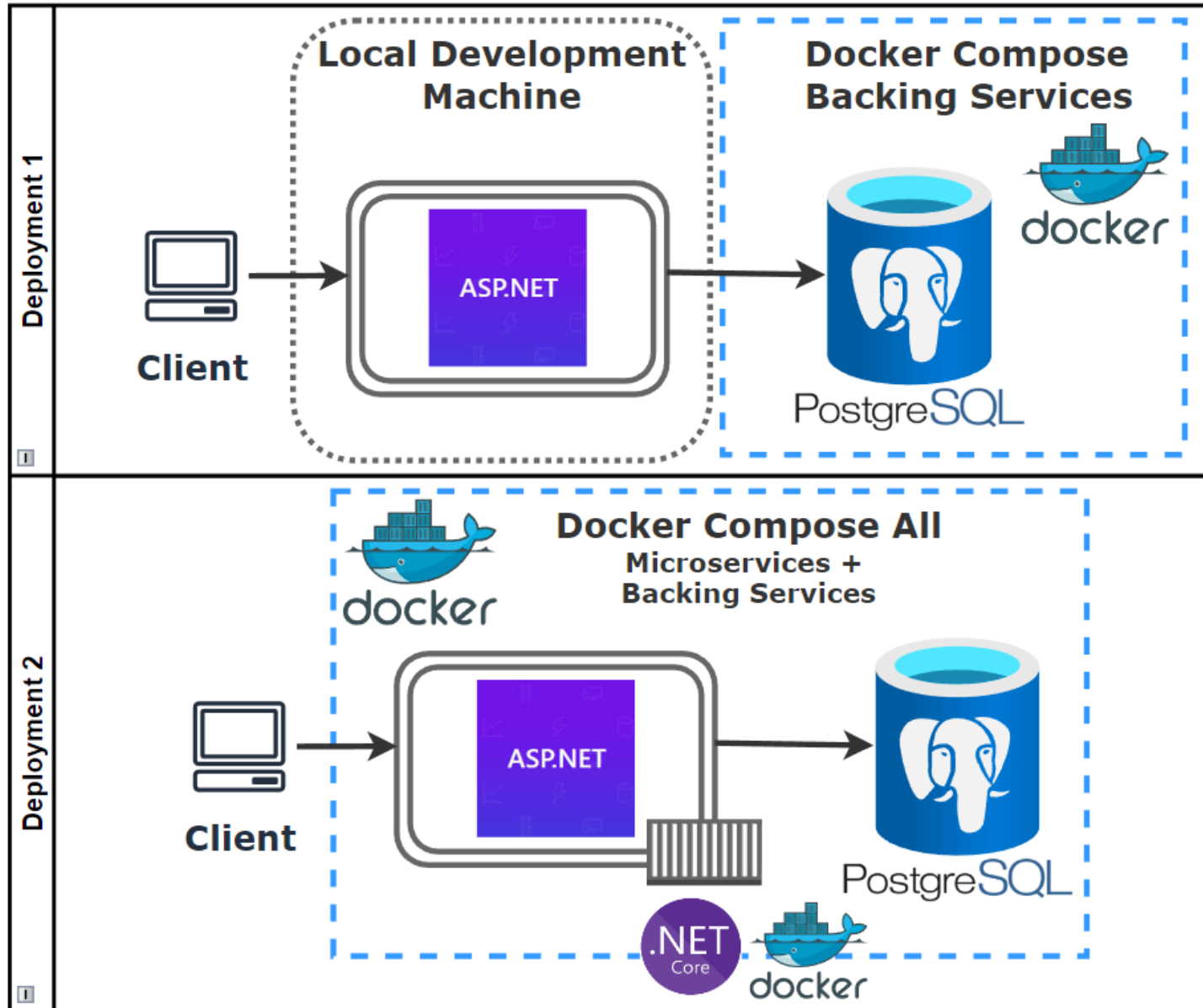


mehmetozkaya

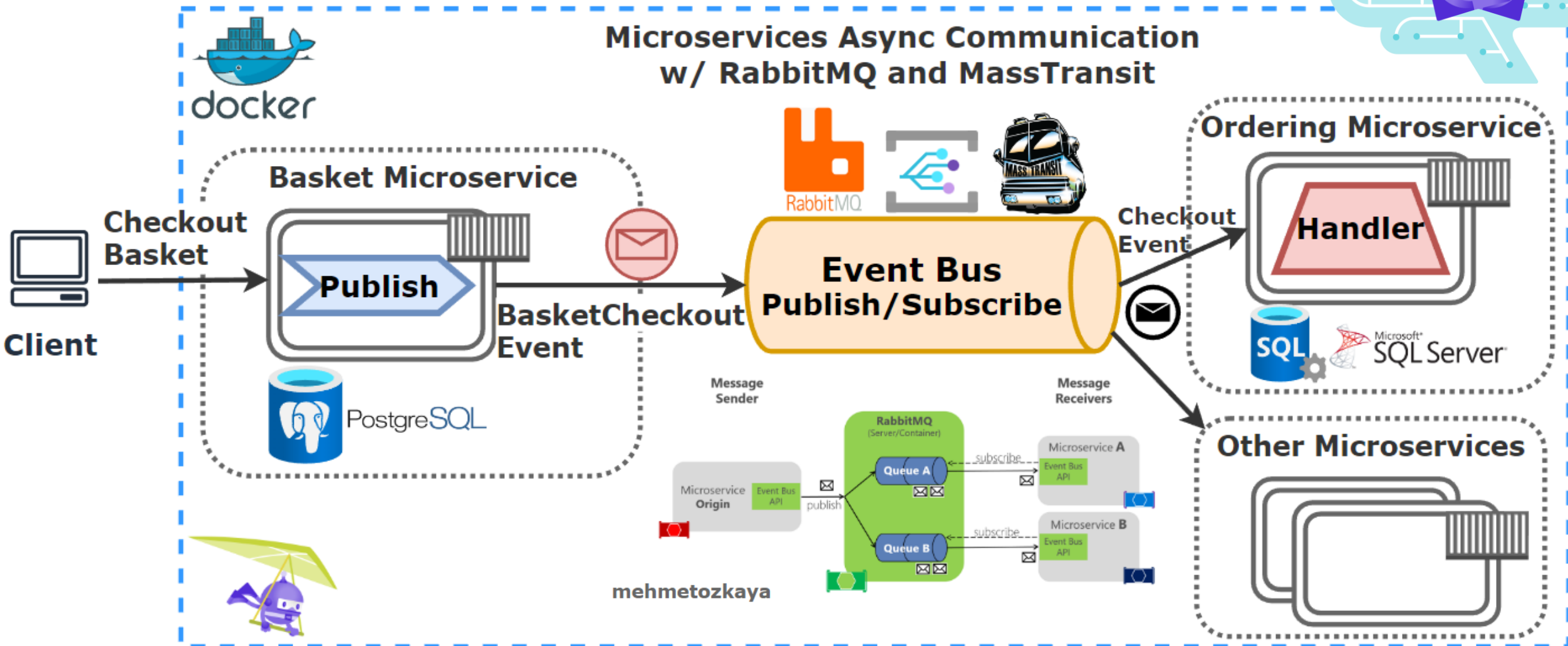


<https://learn.microsoft.com/en-us/dotnet/architecture/microservices/>

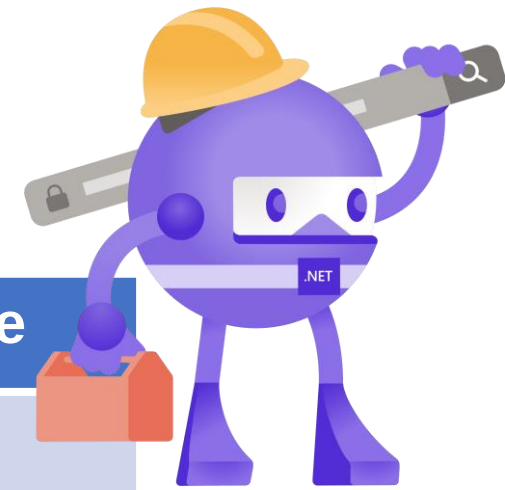
EShop Microservices Deployment Strategy



Microservices Async Communication w/ RabbitMQ & MassTransit

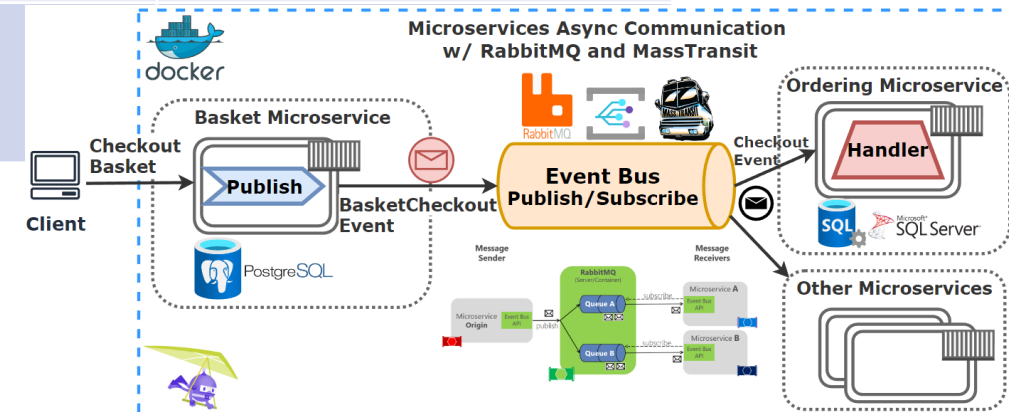


Port Numbers for Async Communication



Microservices	Local Env	Docker Env	Docker Inside
Catalog	5000 – 5050	6000 – 6060	8080 – 8081
Basket	5001 – 5051	6001 – 6061	8080 – 8081
Discount	5002 – 5052	6002 – 6062	8080 – 8081
Ordering	5003 – 5053	6003 – 6063	8080 – 8081
RabbitMQ	5672:5672 15672:15672	5672:5672 15672:15672	

- ASP.NET Core **Ports** are **listed** as **http / https** for running application.



API Gateways with YARP Reverse Proxy applying Gateway Routing Pattern

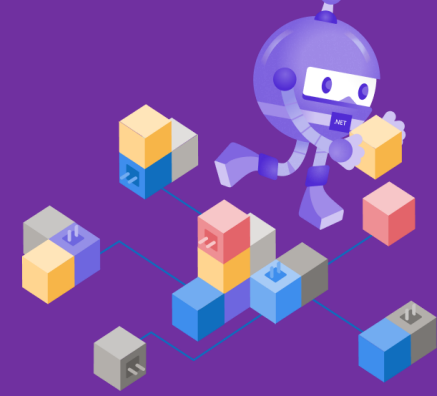
Gateway Routing, API Gateway Pattern, BFF Backend for Frontend
YARP as a Reverse Proxy

Implement API Gateway using YARP Reverse Proxy in YarpApiGateway

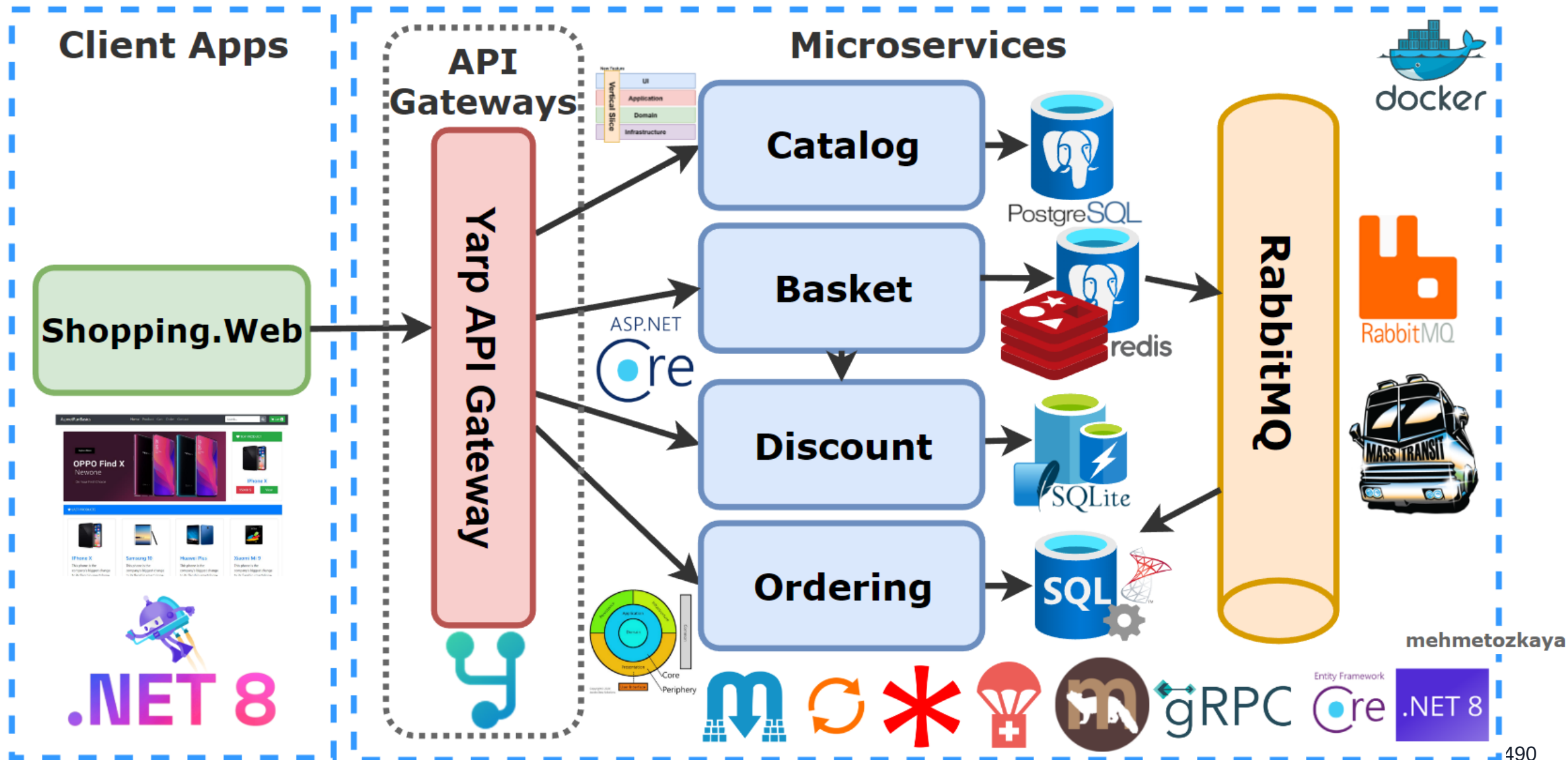
Rate Limiting to YARP Reverse Proxy as an API Gateway

Containerize and Orchestrate YarpApiGateway as an API Gateway
Docker-Compose

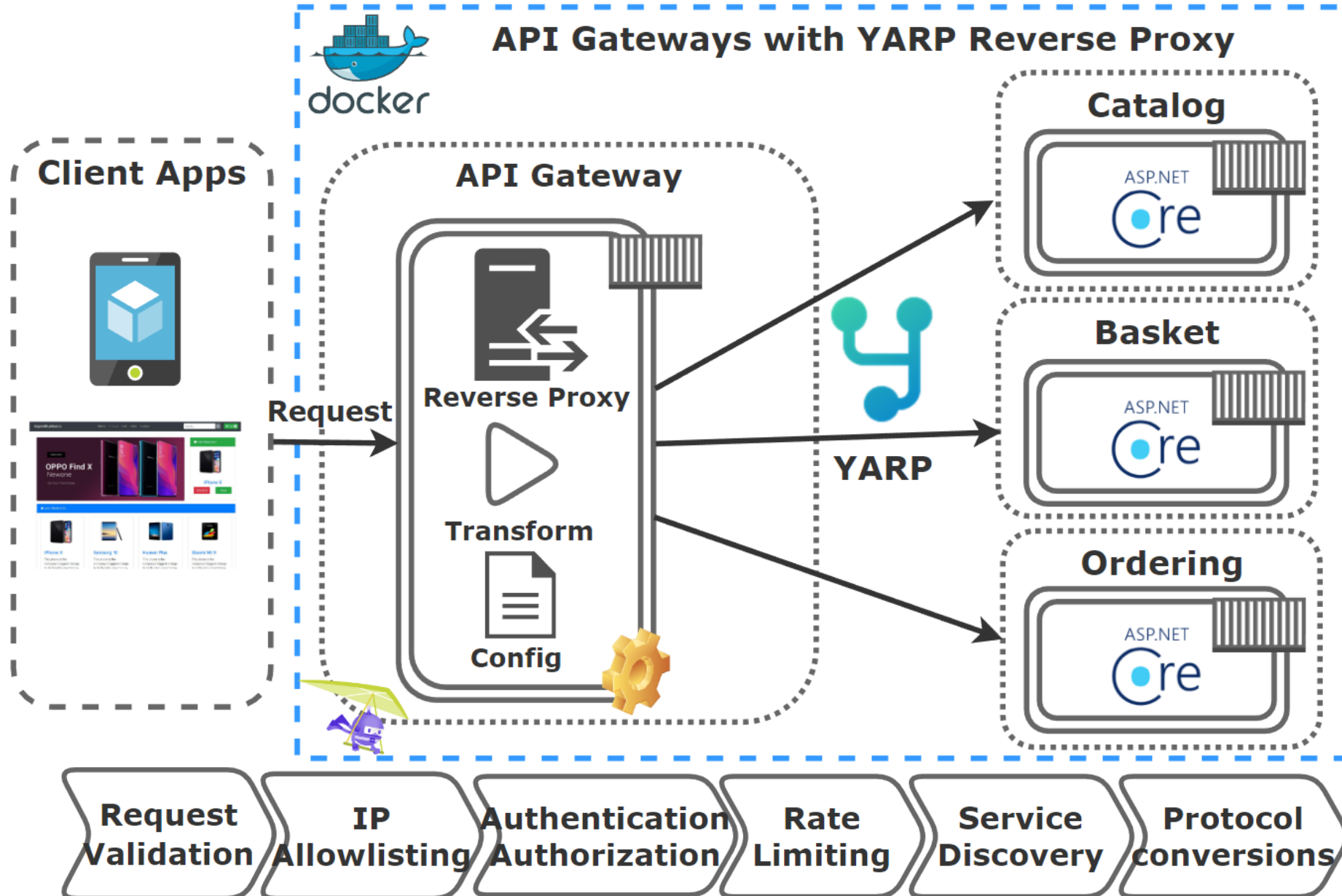
Mehmet Ozkaya



Big Picture – Course Project

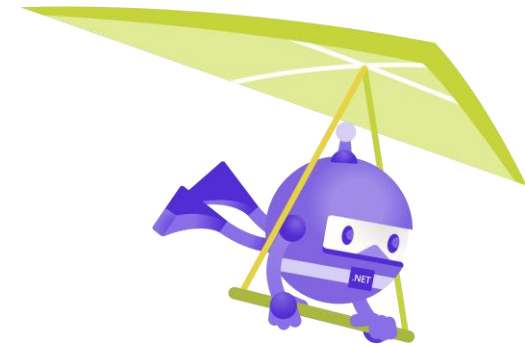
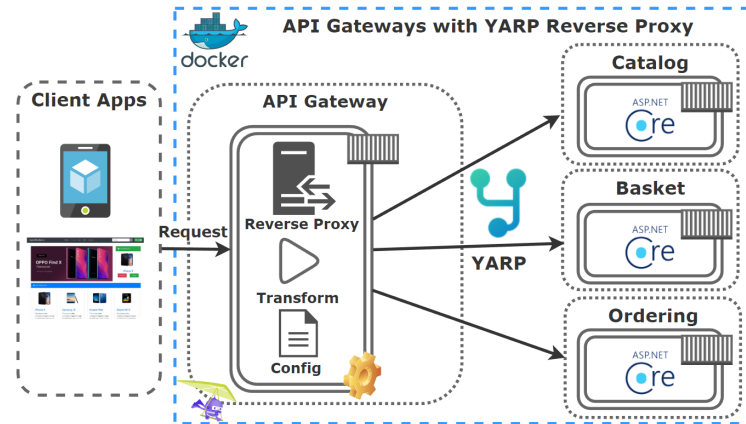


API Gateways with YARP Reverse Proxy



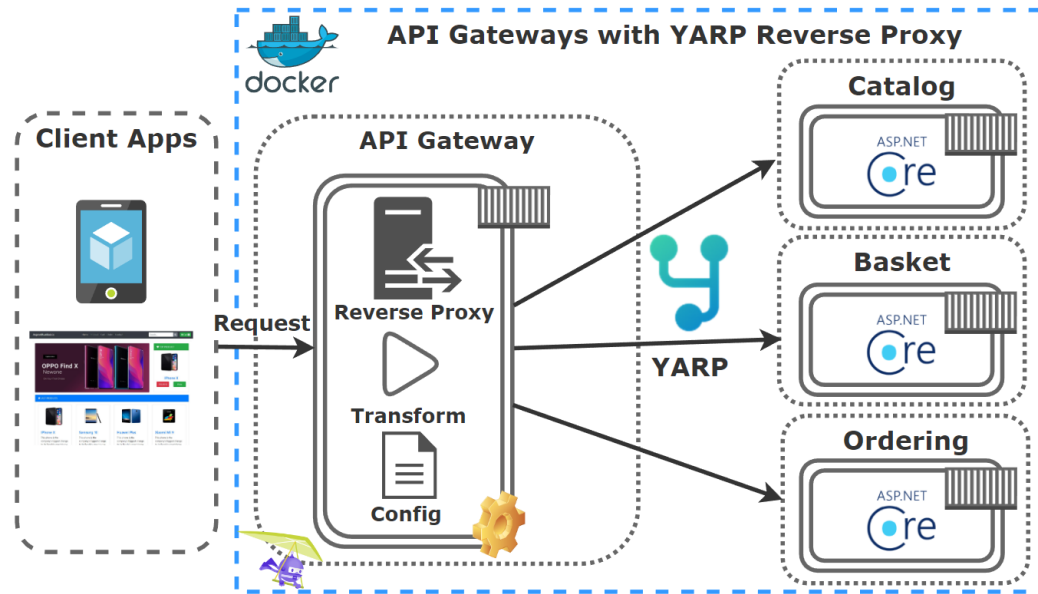
API Gateways with Yarp Reverse Proxy

- Gateway Routing Pattern
- API Gateway Pattern
- BFF Backend for Frontend Pattern
- Yarp as a Reverse Proxy
- Implement API Gateway using Yarp Reverse Proxy in YarpApiGateway
- Rate Limiting to Yarp Reverse Proxy as an API Gateway
- Containerize and Orchestrate YarpApiGateway as an API Gateway in Docker-Compose
- *Analyze and architect the API Gateway using YARP, understanding how it fits into our existing microservices setup with Catalog, Basket, and Ordering services.

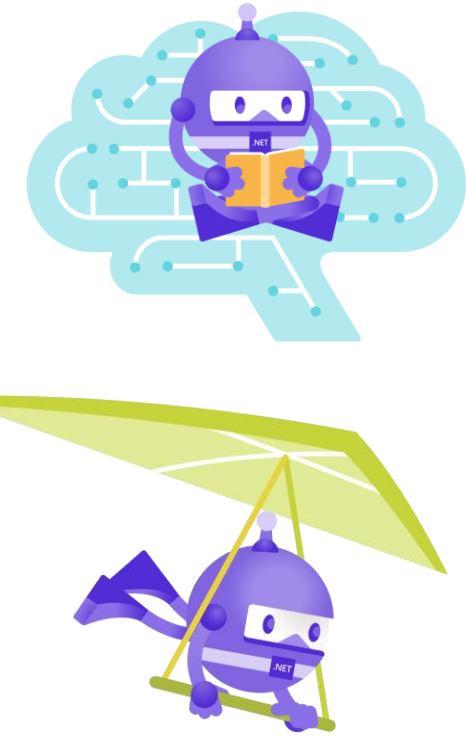


Steps of Developing Yarp API Gateway

1. Creating a New Web Project
2. Adding YARP Reverse Proxy
3. Configuring Routes
4. Testing the Gateway
5. Rate Limiting
6. Dockerization
7. Docker-compose Orchestration Testing

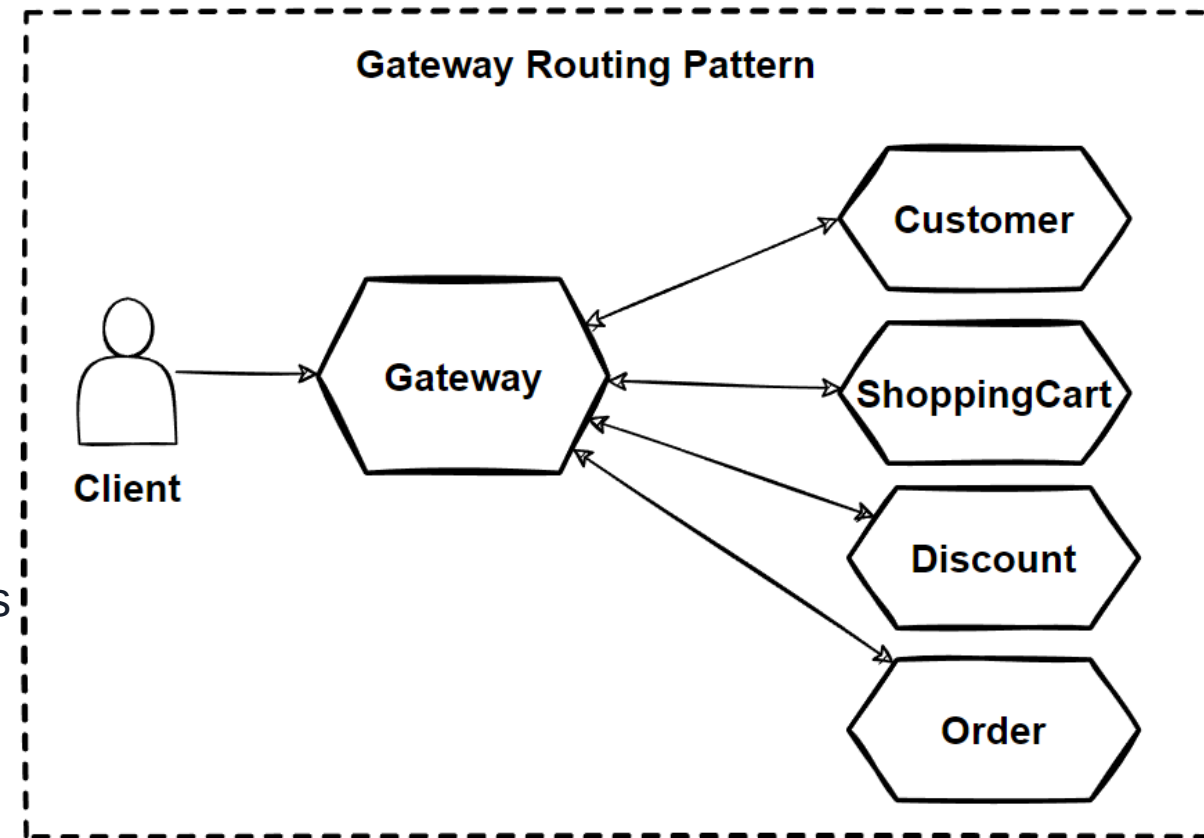


Yarp API Gateway routing operations over the Catalog, Basket and Ordering microservices.



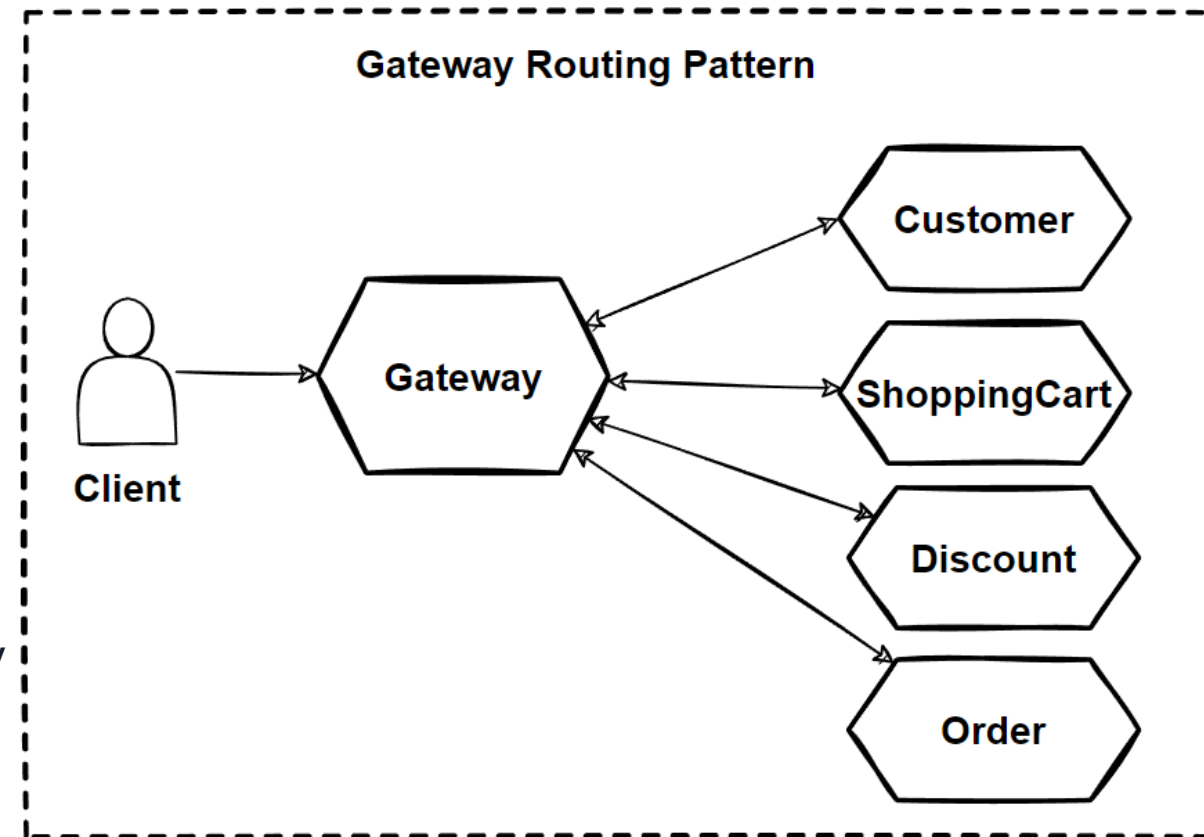
Gateway Routing pattern

- **Route requests to multiple microservices** with exposing a single endpoint.
- Useful when expose multiple services on a **single endpoint** and **route** them to **internal backend microservices** based on the request.
- The client needs to **consume several microservices**, **Gateway Routing pattern** offers to create a new endpoint that handle the request and route this request for each service.
- **E-commerce application** might provide services such as search Customers, Shopping cart, discount, and order history.
- **If one of microservices are changed**, the **Client doesn't know anything** and not need to change any code on client side, the only changes will be configuration routing changes.



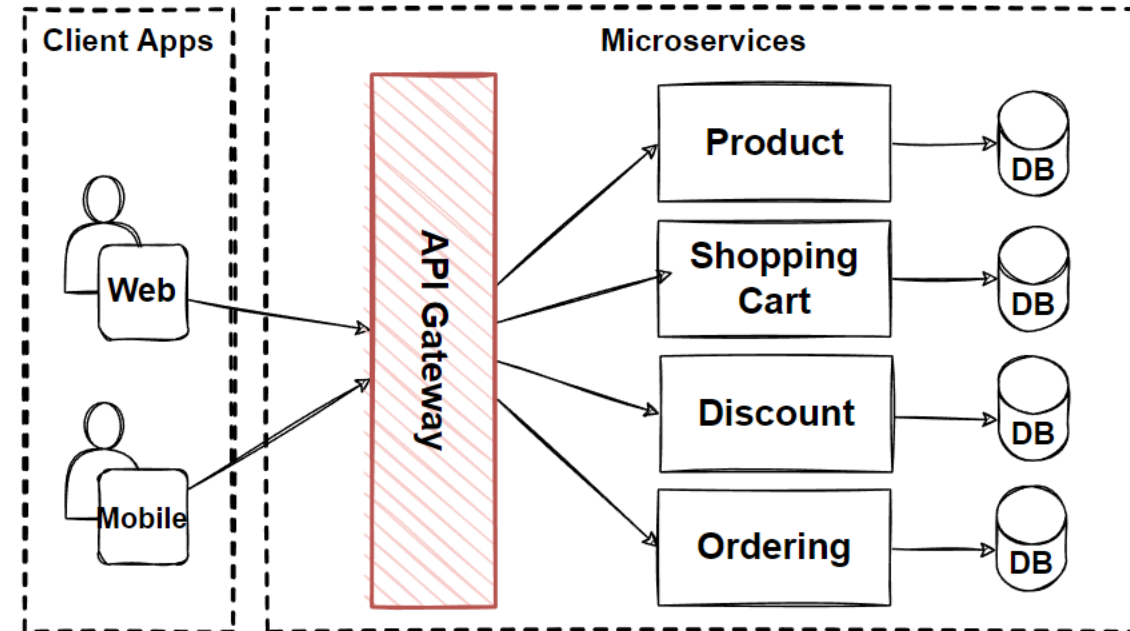
Gateway Routing pattern -2

- It **abstracts your backend microservices** from the client applications, with allowing to **keep client codes simple**, even backend services are changed on behind the API Gateway.
- **Deploy microservices APIs with blue/green or canary deployments** with multiple API versions of same microservices, you can **gradually shift your request** to new API with using the Gateway Routing Pattern.
- **Gateway Routing Pattern** gives you the **flexibility** to use different API versions routing to the requests.
- If new version of API has got exceptions, it can be quickly **reverted back** by making only a configuration change.
- Use **Application Layer 7 routing** to route the request to the internal services.



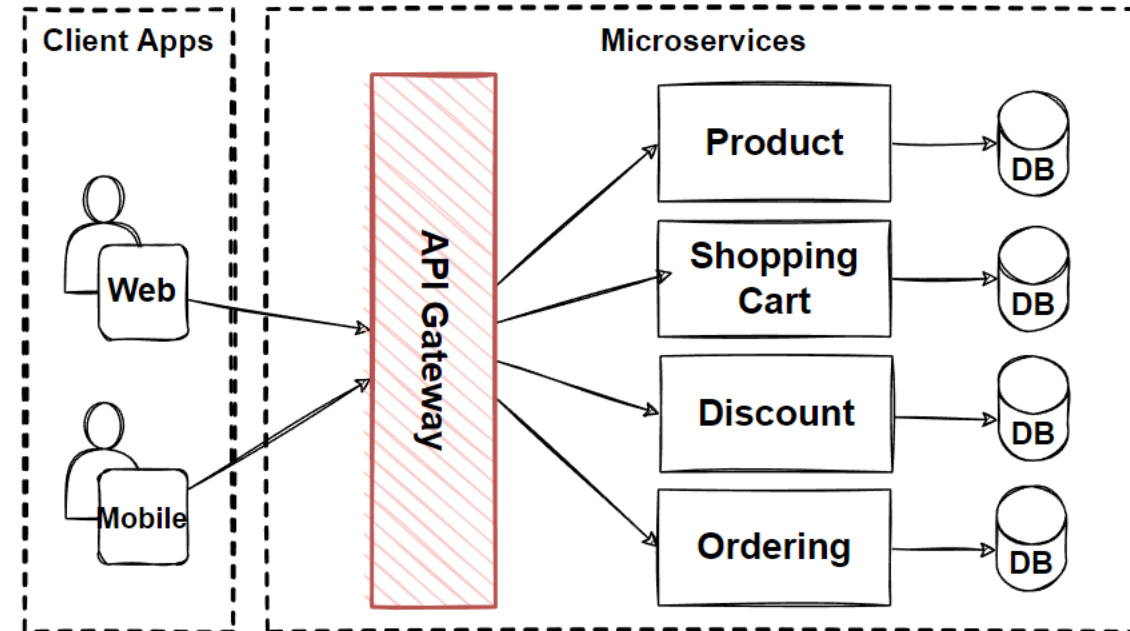
API Gateway Pattern

- **API Gateway** is a **single point of entry** to the client applications, sits between the **client** and **multiple backend**.
- API Gateways **manage routing** to internal microservices and able to **aggregate several microservice** request in 1 response and handle **cross-cutting concerns**.
- Recommended when design **complex large** microservices-based applications with **multiple client** applications.
- Similar to the **facade pattern** from object-oriented design, but it is a distributed system **reverse proxy or gateway routing** for using in synchronous communication.
- The pattern provides a **reverse proxy to redirect or route requests** to your internal microservices endpoints.
- API Gateway provides a **single endpoint** for the client applications, and it **maps the requests** to internal microservices.



API Gateway Pattern - Summarized

- API gateway **locate between** the **client** apps and the **internal microservices**.
- Working as a **reverse proxy** and **routing requests** from clients to backend services and provide **cross-cutting concerns** like authentication, SSL termination, and cache.
- Several client applications connect to single API Gateway possible to **single-point-of-failure risk**.
- If these client applications increase, or adding more logic to **business complexity** in API Gateway, it would be **anti-pattern**.
- Best practices is **splitting** the **API Gateway** in multiple services or multiple smaller API Gateways: **BFF-Backend-for-Frontend Pattern**.
- Should **segregated based** on **business boundaries** of the client applications.

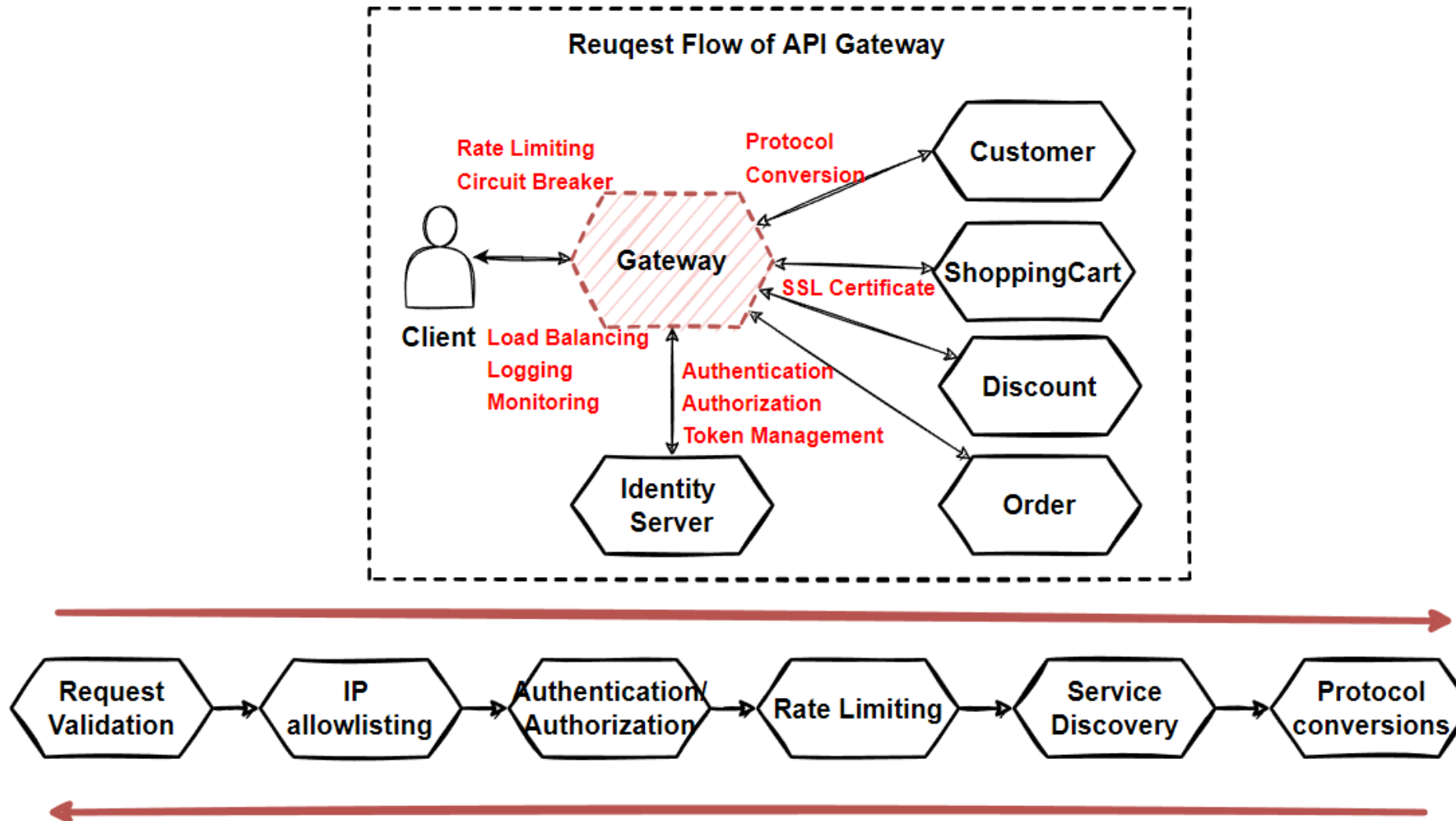


Main Features of API Gateway Pattern

- **Reverse Proxy and Gateway Routing**
Reverse proxy to redirect requests to the endpoints of the internal microservices. Using Layer 7 routing for HTTP requests for redirections. Decouple client applications from the internal microservices. Separating responsibilities on network layer and abstracting internal operations.
- **Requests Aggregation and Gateway Aggregation**
Aggregate multiple internal microservices into a single client request. Client application sends a single request to the API Gateway and it dispatches several requests to the internal microservices and then aggregates the results and sends back to the client application in 1 single response. Reduce chattiness communication.
- **Cross-cutting Concerns and Gateway Offloading**
Best practice to implement cross-cutting functionality on the API Gateways. Cross-cutting functionalities can be; Authentication and authorization, Service discovery, Response caching, Retry policies, Circuit Breaker, Rate limiting and throttling, Load balancing, Logging, tracing, IP allowlisting.

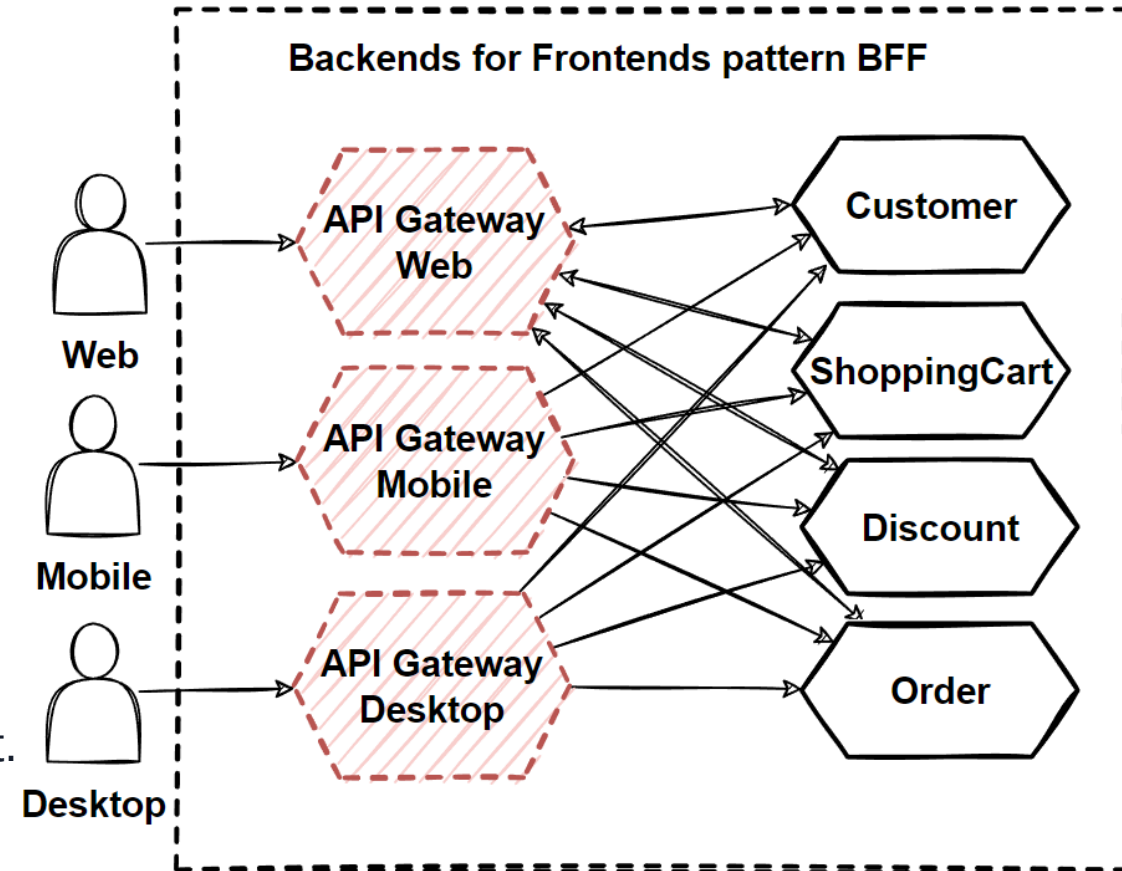
Routing	Authentication
Request Aggregation	Authorization
Service Discovery with Consul & Eureka	Throttling
Load Balancing	Logging, Tracing
Correlation Pass-Through	Headers/Query String Transformation
Quality of Service	Custom Middleware

Request Flow of API Gateway Pattern

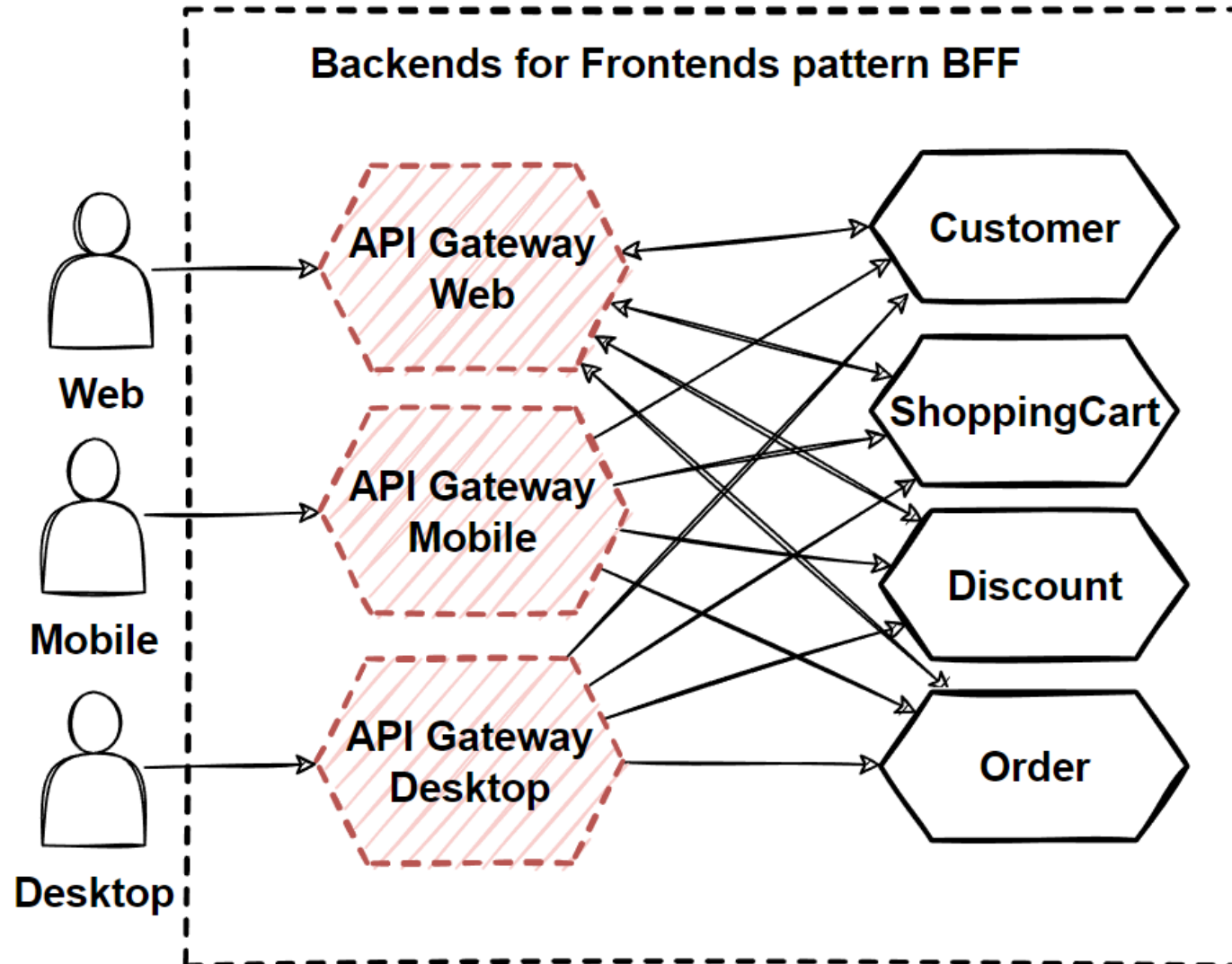


Backends for Frontends Pattern - BFF

- **Backends for Frontends** pattern basically **separate API Gateways** as per the **specific frontend** applications.
- Single API Gateway makes a **single-point-of failure**.
- BFF offers to create several API Gateways and **grouping** the **client applications** according to their boundaries.
- A **single** and **complex** API Gateway can be **risky** and becoming a **bottleneck** into your architecture.
- Larger systems often expose multiple API Gateways by **grouping client type** like mobile, web and desktop functionality.
- **Create several API Gateways** as per user interfaces to provide to best match the needs of the frontend environment.
- **BFF Pattern** is to provide client applications has a **focused interfaces** that connects with the internal microservices.

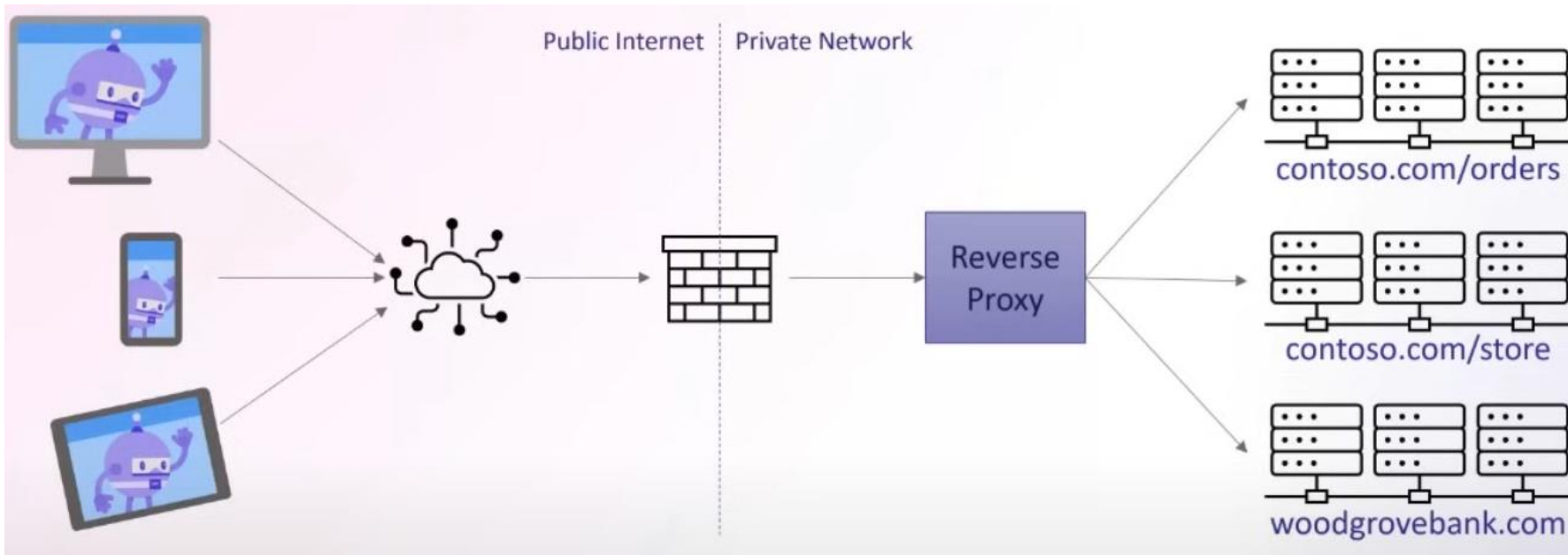


Backends for Frontends Pattern - BFF



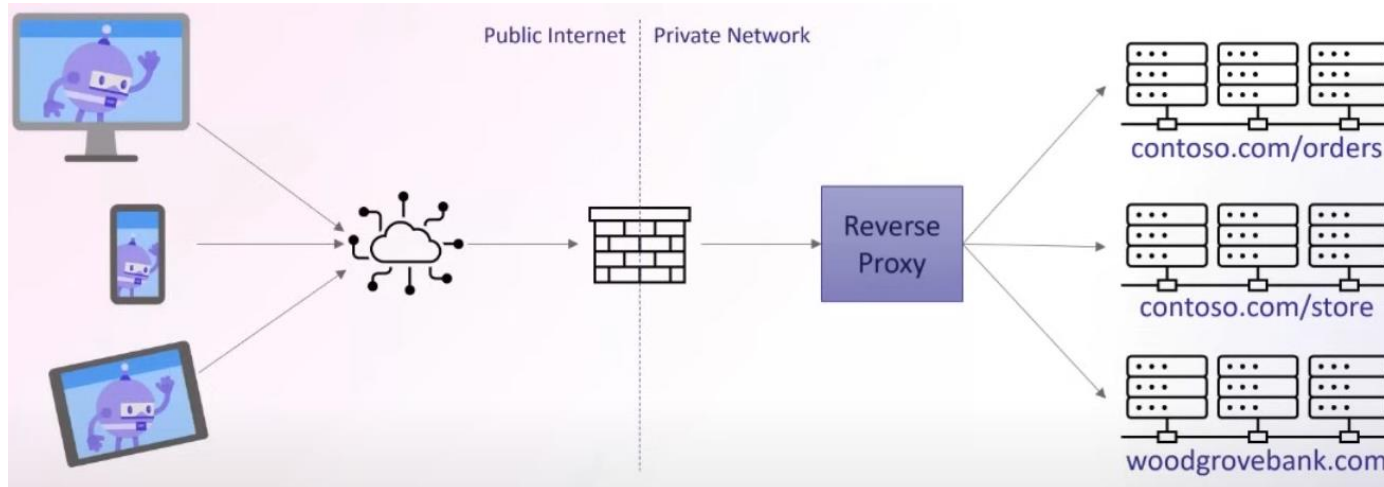
What is Reverse Proxy

- A **reverse proxy** is a server that sits in front of web servers and forwards client requests to these web servers.
- Implements **Gateway Routing** and **API Gateway** Patterns.
- It **acts as an intermediary** for requests from clients seeking resources from servers that provide those resources.
- **Functions:** Load Balancing, Offloading, Security

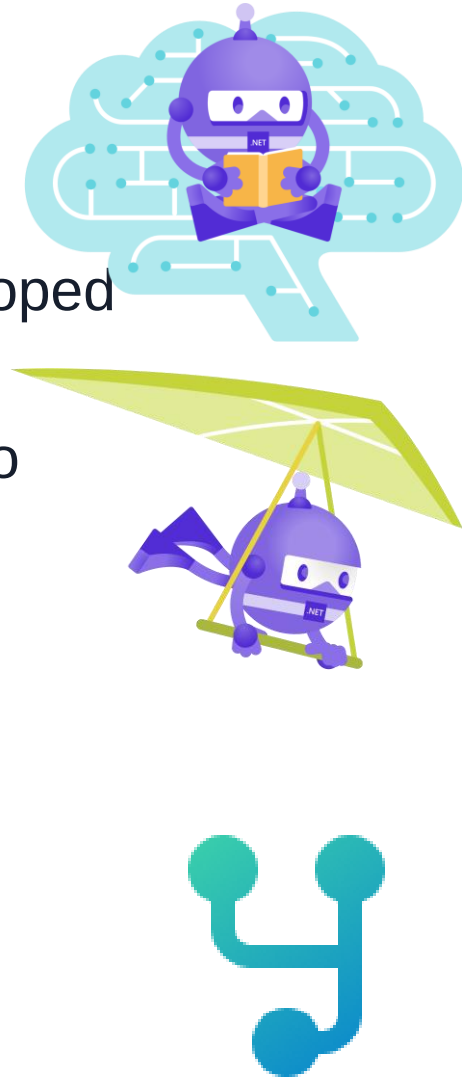


Microsoft Reverse Proxy: YARP

- **YARP** is a **lightweight**, highly customizable **reverse proxy solution** developed by Microsoft, tailored for **.NET applications**.
- **Designed to integrate** seamlessly with **.NET Core**, making it easy to add to existing .NET projects.
- Simplify the **routing of requests** to different backend services, offering capabilities for **request transformation**, **load balancing**, and more.
- **GitHub:** <https://github.com/microsoft/reverse-proxy>
- **Docs:** <https://microsoft.github.io/reverse-proxy/articles/getting-started.html>

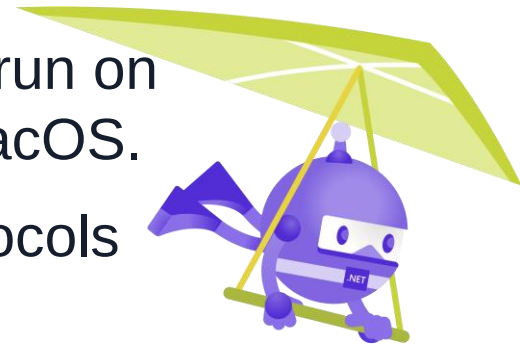


<https://www.youtube.com/watch?v=P8y8NAroVKK&t=845s>
<https://www.dotnetconf.net/>

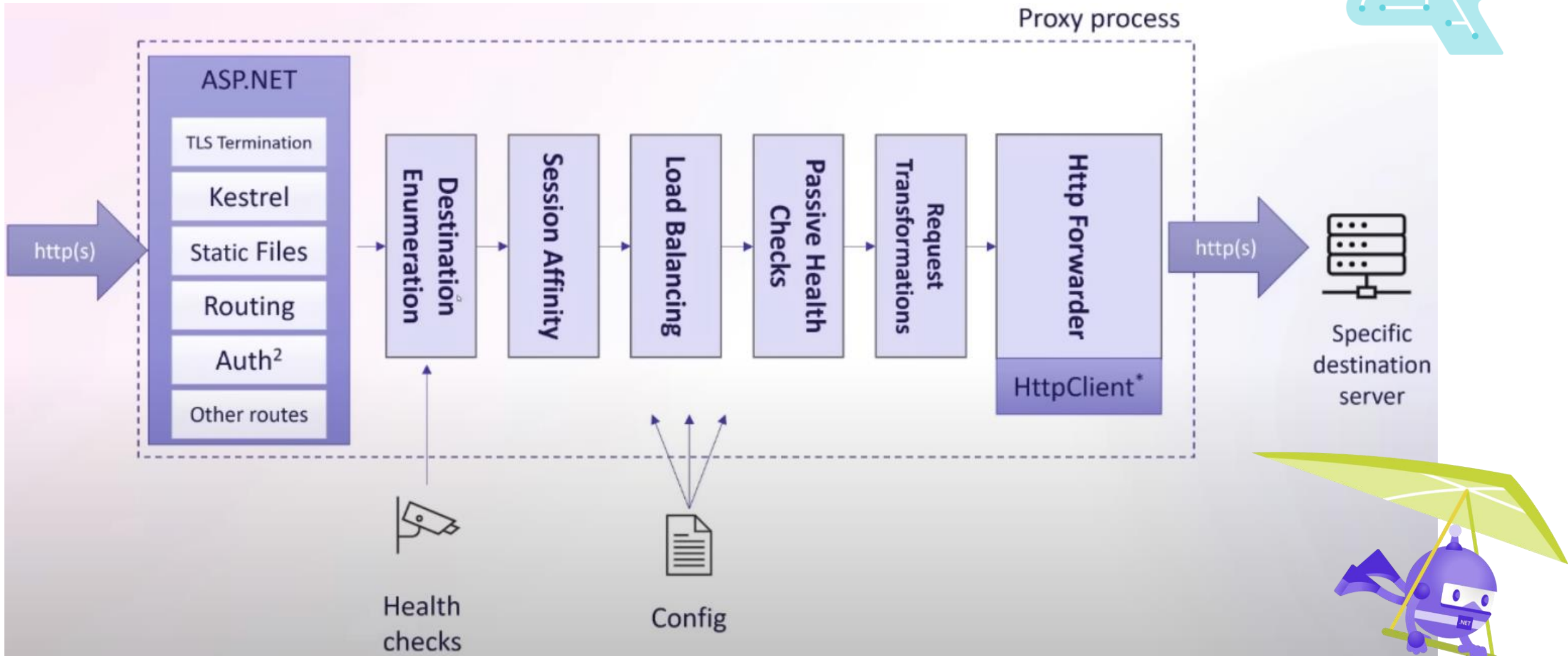


Microsoft Reverse Proxy: YARP Features

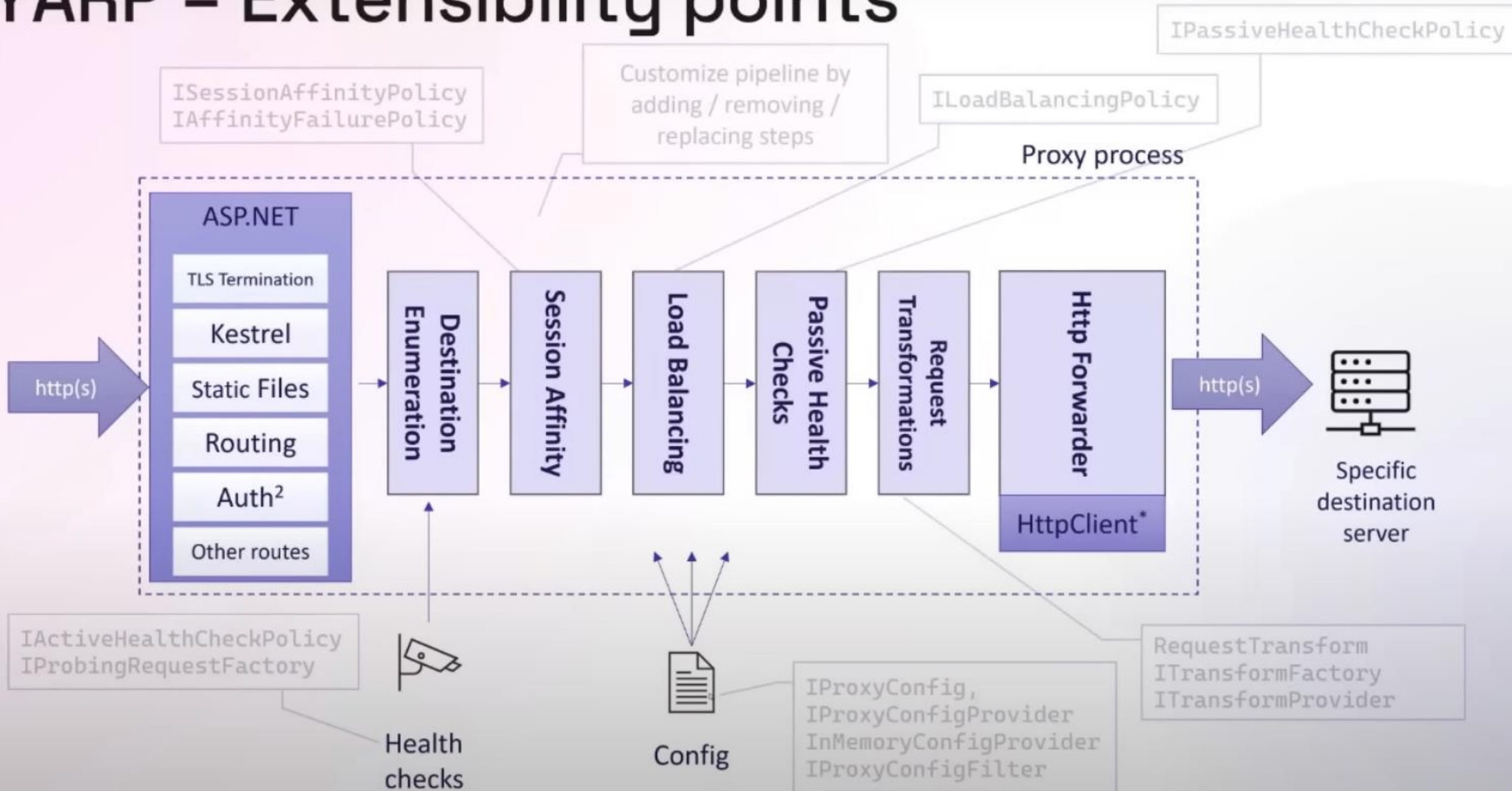
- **Customizable Routing Rules:** Allows detailed specification of how requests should be routed to backend services.
- **Cross-Platform:** YARP is built on .NET, making it cross-platform. It can run on any platform that supports .NET Core, including Windows, Linux, and macOS.
- **Support for Latest Protocols:** YARP supports modern networking protocols including gRPC, HTTP/2, and WebSockets.
- **Great Performance:** It is optimized for high throughput and low latency, making it suitable for scenarios requiring high-performance proxying.
- **Health Checks:** Monitors the health of backend services and reroutes traffic as needed.
- **Modern HTTP Client:** Uses the latest HTTP client features in .NET for improved performance. Latest protocols (gRPC, HTTP/2, HTTP/2 WebSockets)
- **GitHub:** <https://github.com/microsoft/reverse-proxy>
- **Docs:** <https://microsoft.github.io/reverse-proxy/articles/getting-started.html>



YARP Architecture with Pipeline

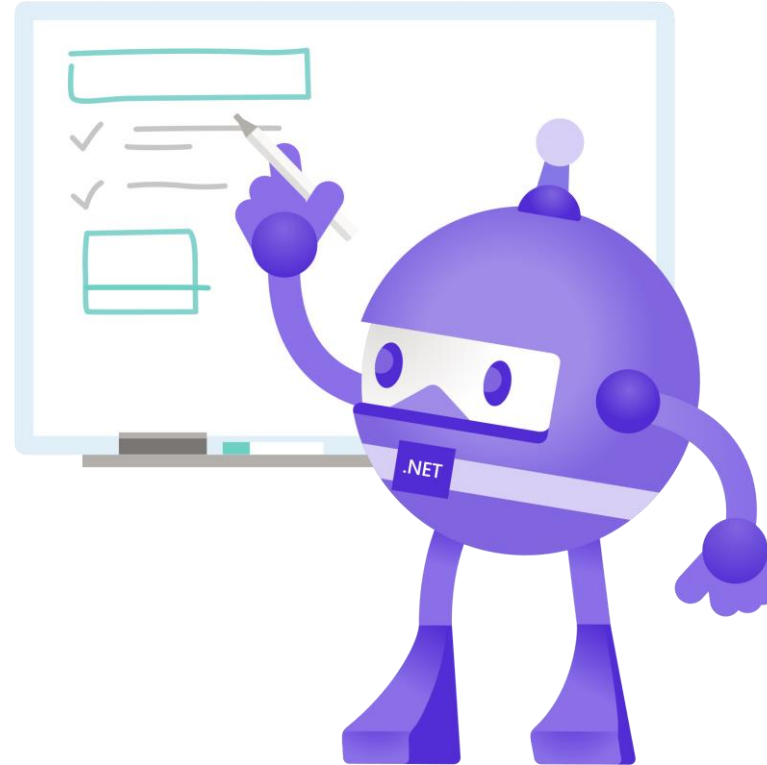


YARP - Extensibility points

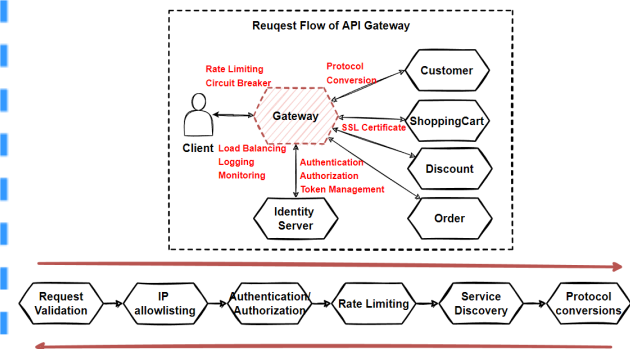
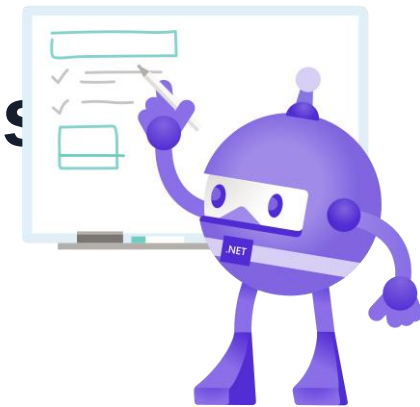
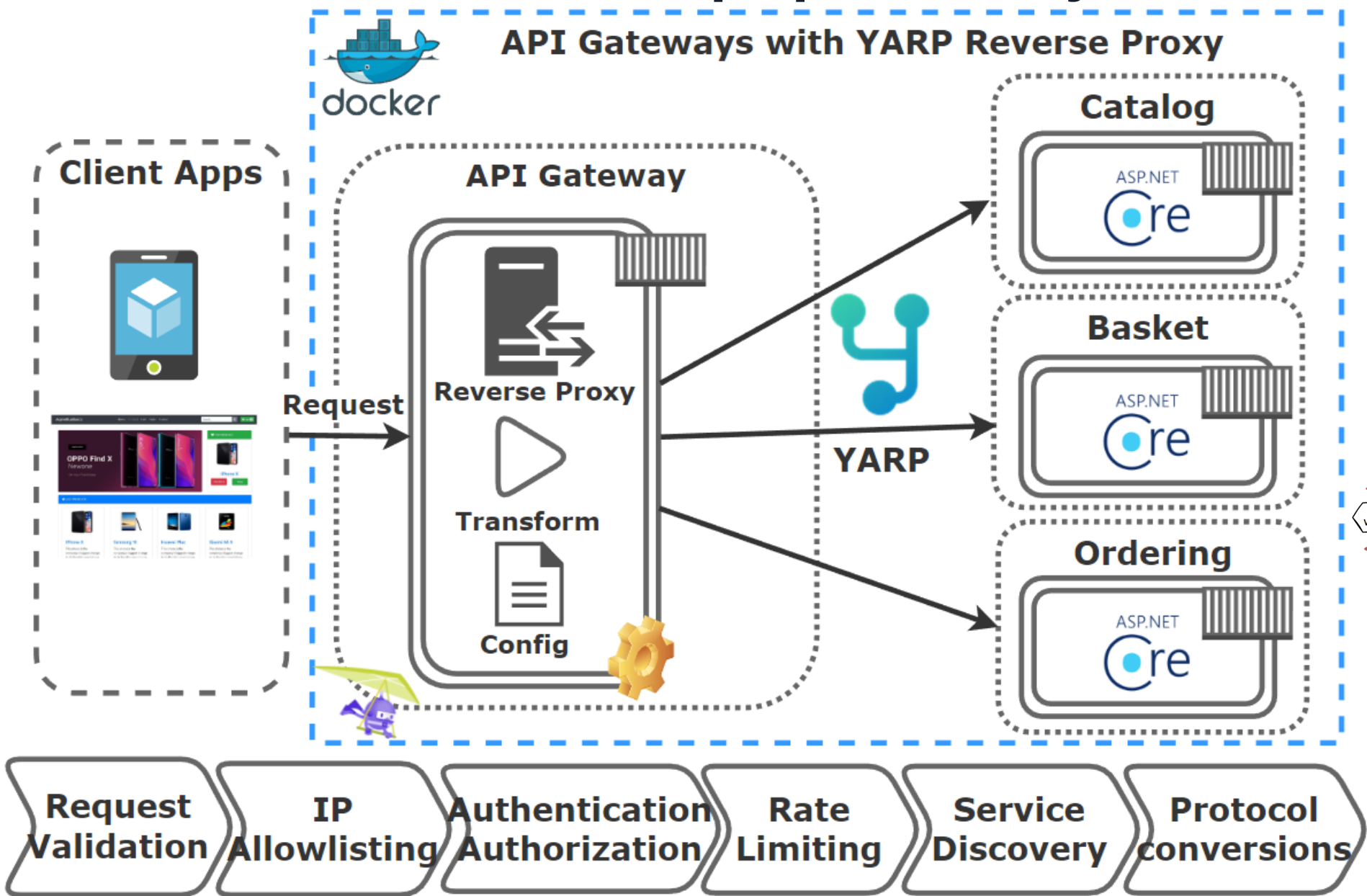


Domain Analysis of YarpApiGateway Microservices

1. Domain Models
2. Application Use Cases
3. Rest API Endpoints
4. Underlying Data Structures



1. Domain Models of YarpApiGateway Microservices



2. Application Use Cases of YarpApiGateway Microservices

CRUD Catalog Operations:

- Get all Products and Categories
- Get Product with product Id
- Get Products with category
- Create/Update/Delete Product

CRUD Ordering Operations:

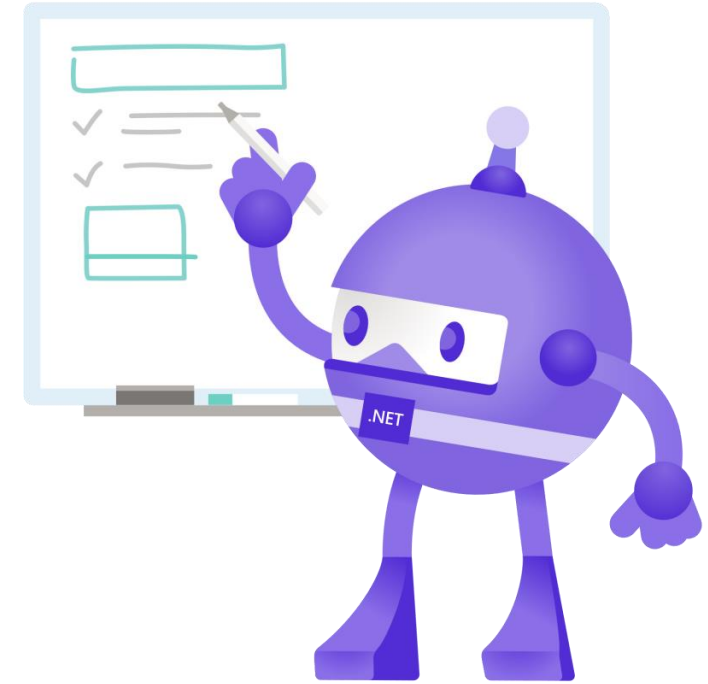
- Get Order with Items (can filter by Name and Customer)
- Create a new Order
- Update an existing Order
- Delete Order
- Add and Remove Item from Order

CRUD Basket Operations:

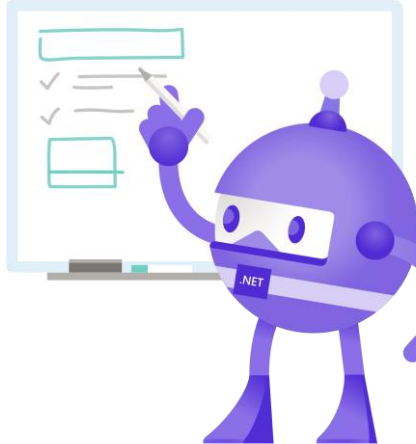
- Get ShoppingCart with Items
- Store (Upsert-Create and Update) ShoppingCart with Items
- Delete ShoppingCart with Items

Asynchronous Ordering Operations:

- **Basket Checkout:** Consume Basket Checkout event from RabbitMQ using MassTransit
- **Order Fulfilment:** Perform order fulfilment operations (billing, shipment, notification..)
- Raise OrderCreated Domain Event that leads to integration event



3. Rest API Endpoints of Internal Microservices



Method	Request URI	Use Cases
GET	/products	List all products
GET	/products/{id}	Fetch a specific product
GET	/products/category	Get p
POST	/products	Creat
PUT	/products/{id}	Updat
DELETE	/products/{id}	Remo

Method	Request URI	Use Cases
GET	/basket/{userName}	Get basket w/ username
POST	/basket/{userName}	Store Basket (insert-update)
DELETE	/basket/{userName}	Delete basket w/ username
POST	/basket/checkout	Checkout basket

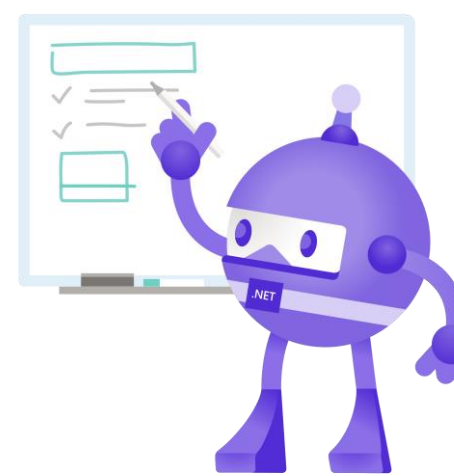
Method	Request URI	Use Cases
GET	/orders	Get Orders w/ Pagination
GET	/orders/{orderName}	Get Orders by OrderName
GET	/orders/customer/{customerId}	Get Orders by Customer
POST	/orders	Create a new Order
PUT	/orders	Update an existing Order
DELETE	/order/{id}	Delete Order with Id

3. Rest API Endpoints of YARP Microservices

- Put microservice name as a prefix of internal uri's
- {microservice_name}/{microservice_internal_url}

Method	YARP Request URI	Internal Ms Request URI
GET	/catalog/products	/products
GET	/catalog/products/{id}	/products/{id}
GET	/catalog/products/category	/products/category
POST	/catalog/products	/products
PUT	/catalog/products/{id}	/products/{id}
DELETE	/catalog/products/{id}	/products/{id}

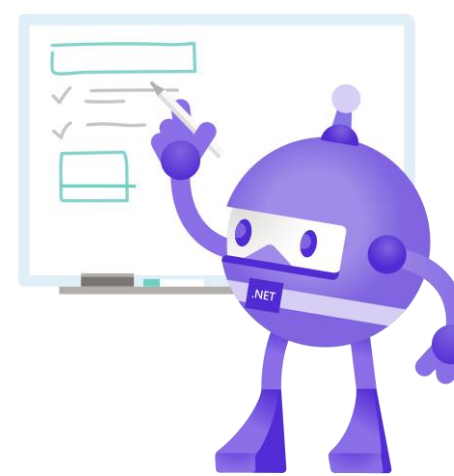
Method	YARP Request URI	Internal Ms Request URI
GET	/shoppingcart/basket/{userName}	/basket/{userName}
POST	/shoppingcart/basket/{userName}	/basket/{userName}
DELETE	/shoppingcart/basket/{userName}	/basket/{userName}
POST	/shoppingcart/basket/checkout	/basket/checkout



3. Rest API Endpoints of YARP Microservices -2

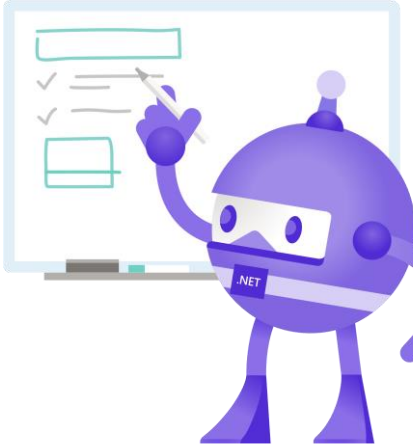
- Put **microservice name** as a **prefix** of internal uri's
- **{microservice_name}/{microservice_internal_url}**

Method	YARP Request URI	Internal Ms Request URI
GET	/ordering/orders	/orders
GET	/ordering/orders/{orderId}	/orders/{orderId}
GET	/ordering/orders/customer/{customerId}	/orders/customer/{customerId}
POST	/ordering/orders	/orders
PUT	/ordering/orders	/orders
DELETE	/ordering/order/{id}	/order/{id}



3. Rest API Endpoints of YARP Microservices

- Put microservice name as a prefix of internal uri's



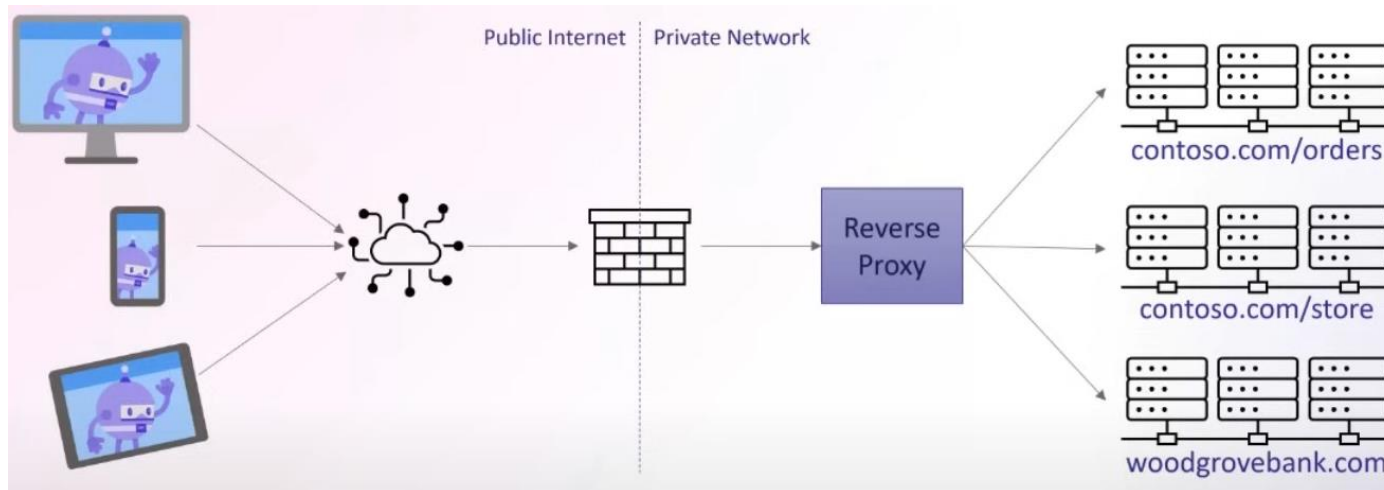
Method	Request URI	Internal Ms Request URI			
GET	/catalog/products	/products			
GET	/catalog/products/{id}	/products/{id}			
GET	/catalog/products/category	/products	Method	Request URI	Use Cases
POST	/catalog/products	/products	GET	/basket/{userName}	Get basket w/ username
PUT	/catalog/products/{id}	/products	POST	/basket/{userName}	Store Basket (insert-update)
DELETE	/catalog/products/{id}	/products	DELETE	/basket/{userName}	Delete basket w/ username
			POST	/basket/checkout	Checkout basket

Method	Request URI	Use Cases
GET	/orders	Get Orders w/ Pagination
GET	/orders/{orderName}	Get Orders by OrderName
GET	/orders/customer/{customerId}	Get Orders by Customer
POST	/orders	Create a new Order
PUT	/orders	Update an existing Order
DELETE	/order/{id}	Delete Order with Id

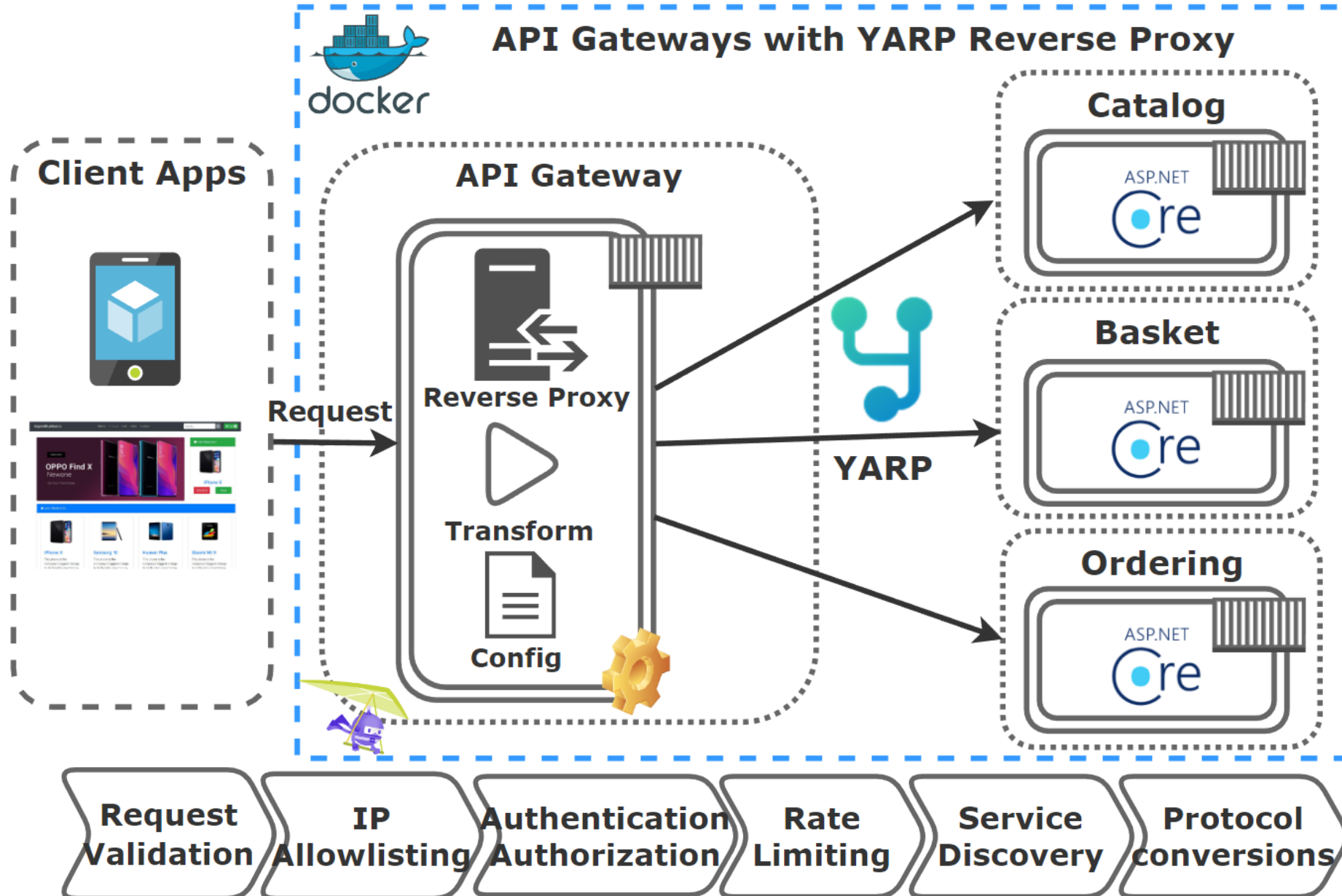
4. Underlying Data Structures of Yarp Microservices

Yarp API Gateway Microservices has NO Datastore

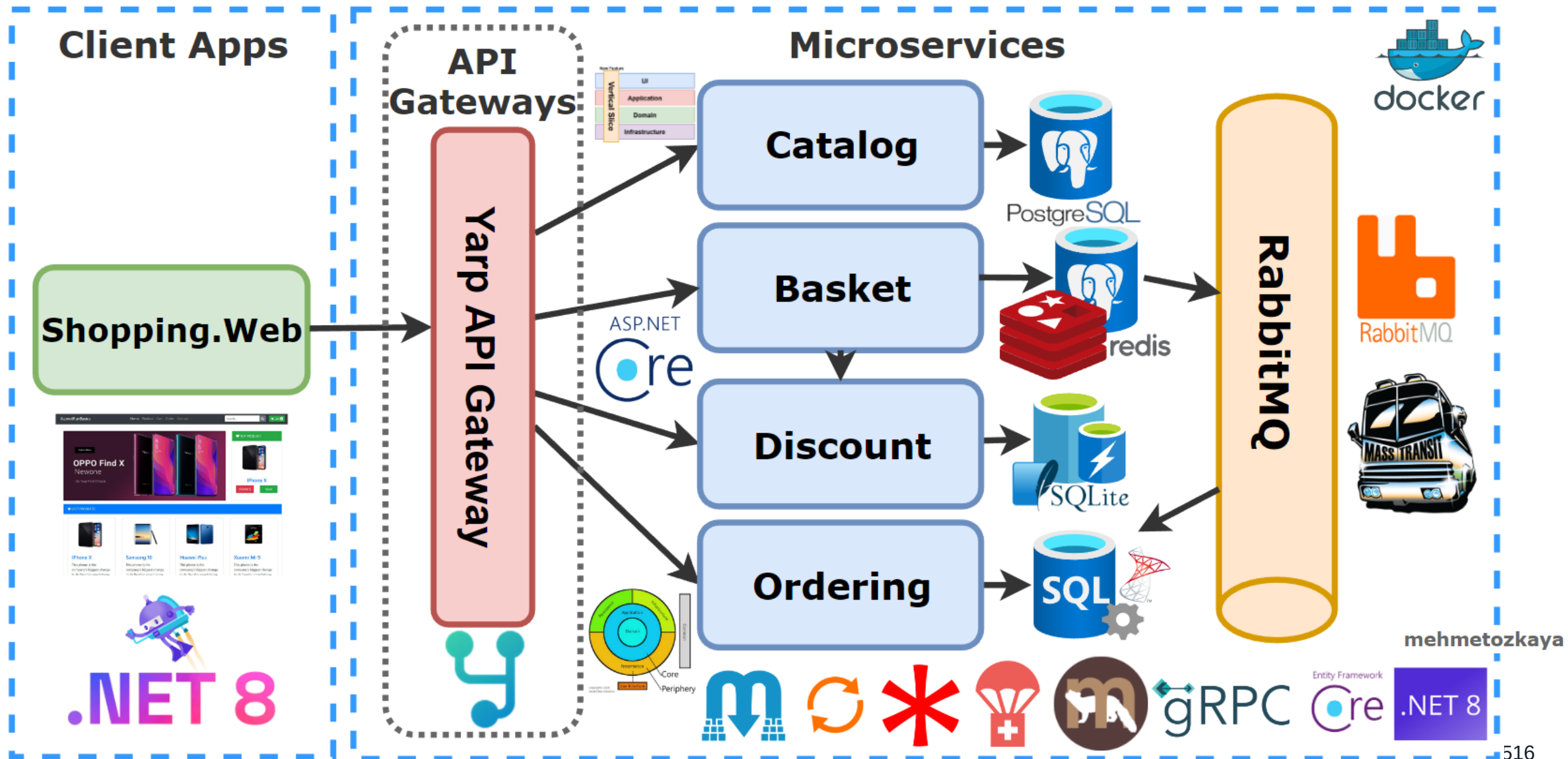
- Focus is more on routing and configuration rather than data structures.
- Doesn't interact with databases or manage state directly but ensures efficient and correct routing of requests to services that handle data.



API Gateways with YARP Reverse Proxy

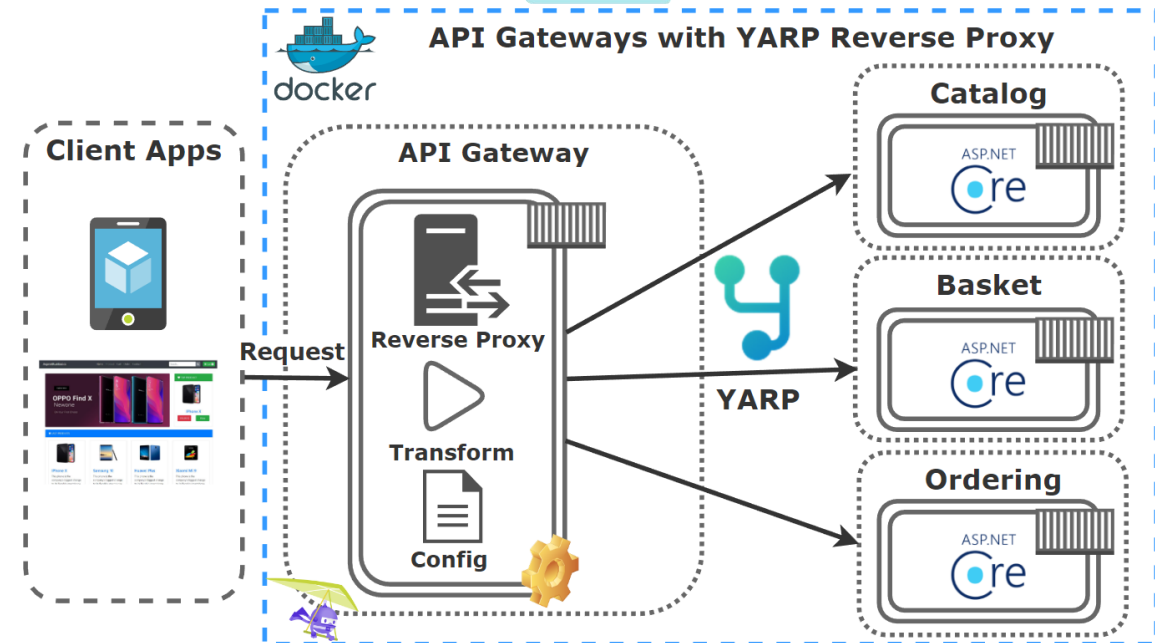
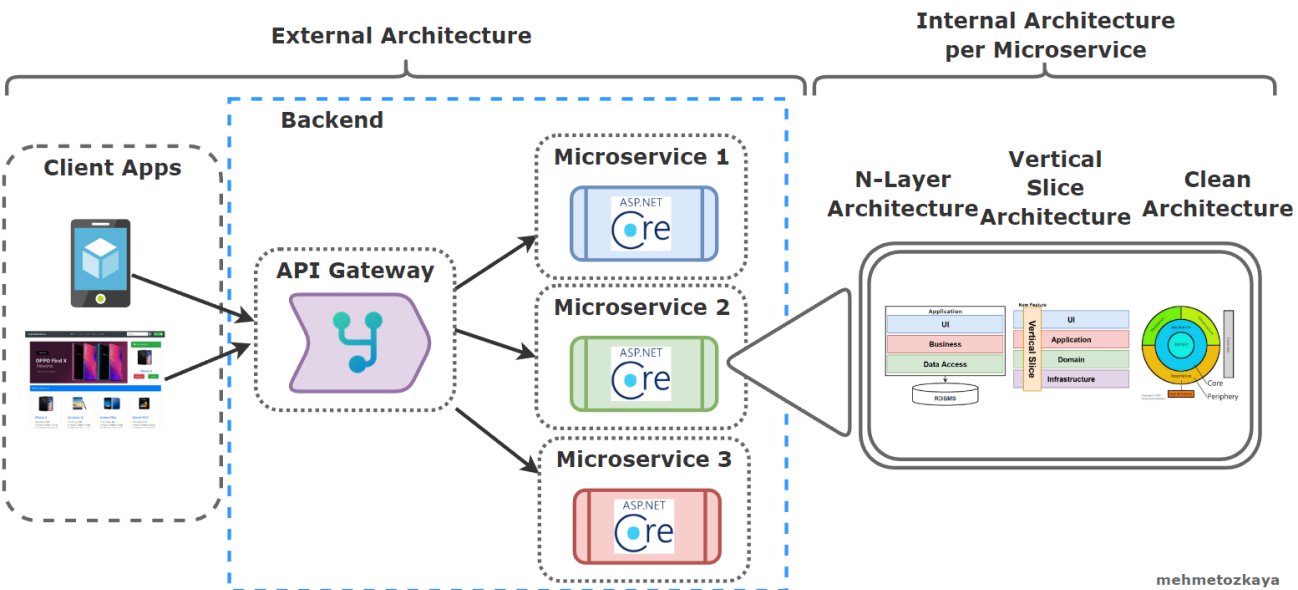


Big Picture – Course Project



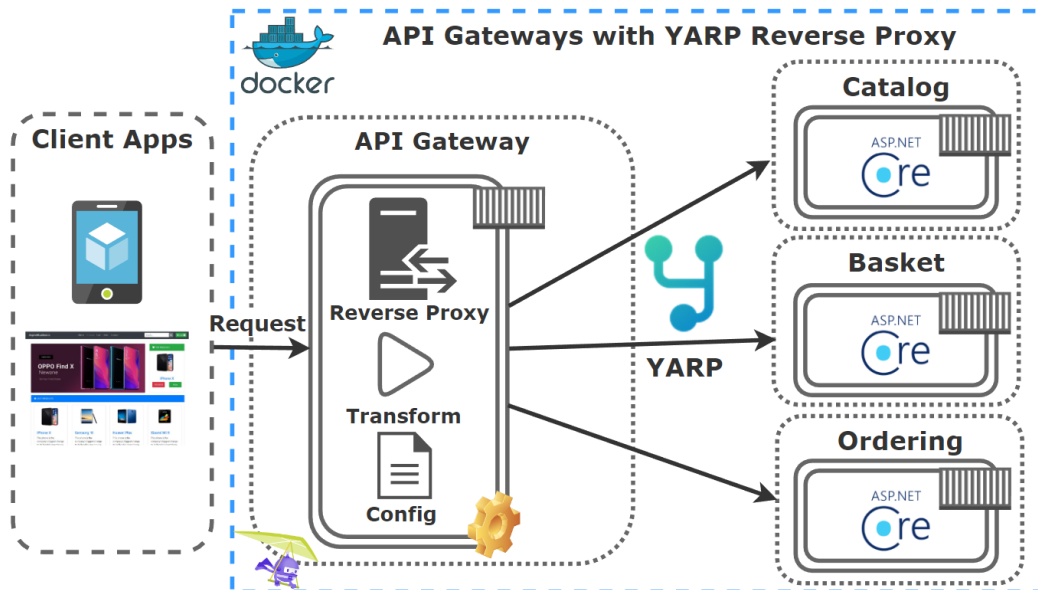
Technical Analysis of YarpApiGateway Microservices

1. Application Architecture Style
2. Patterns & Principles
3. Libraries Nuget Packages
4. Project Folder Structure



1. Application Architecture Style of YarpApiGateway

- No architecture, simple appsettings configuration for reverse proxy
- A reverse proxy is a server that sits in front of web servers and forwards client requests to these web servers.
- Configuration-based approach to route requests to different microservices.



2. Patterns & Principles of YarpApiGateway



Gateway Routing Pattern

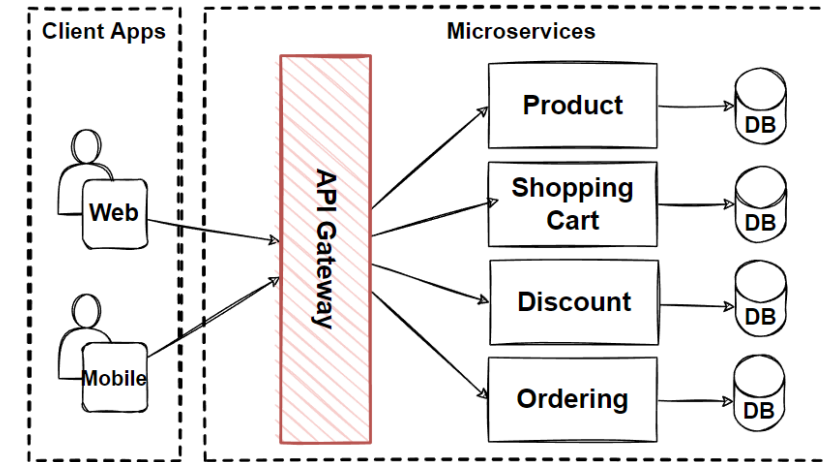
- Manages routing of requests to various services, handling path-based routing.

API Gateway Pattern

- Single entry point for multiple microservices, simplifying the client-side interaction.

Reverse Proxy Pattern

- Yarp acts as an intermediary for requests from clients seeking resources from the microservices it routes to.



3. Libraries Nuget Packages of YarpApiGateway



Package: Yarp.ReverseProxy

- Enables the implementation of reverse proxy capabilities within the application.
- YARP is a lightweight, highly customizable reverse proxy solution developed by Microsoft, tailored for .NET applications.
- Designed to integrate seamlessly with .NET Core, making it easy to add to existing .NET projects.
- Simplify the routing of requests to different backend services, offering capabilities for request transformation, load balancing, and more.



4. Project Folder Structure of YarpApiGateway

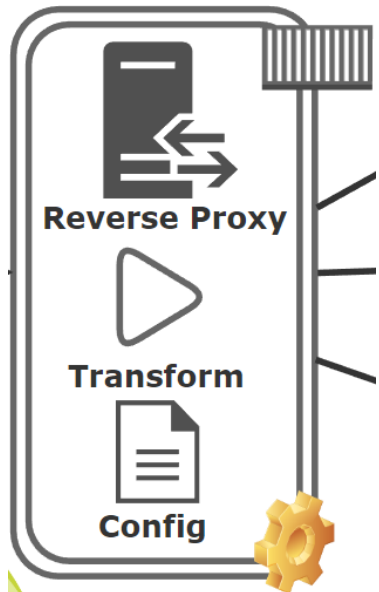


Program.cs:

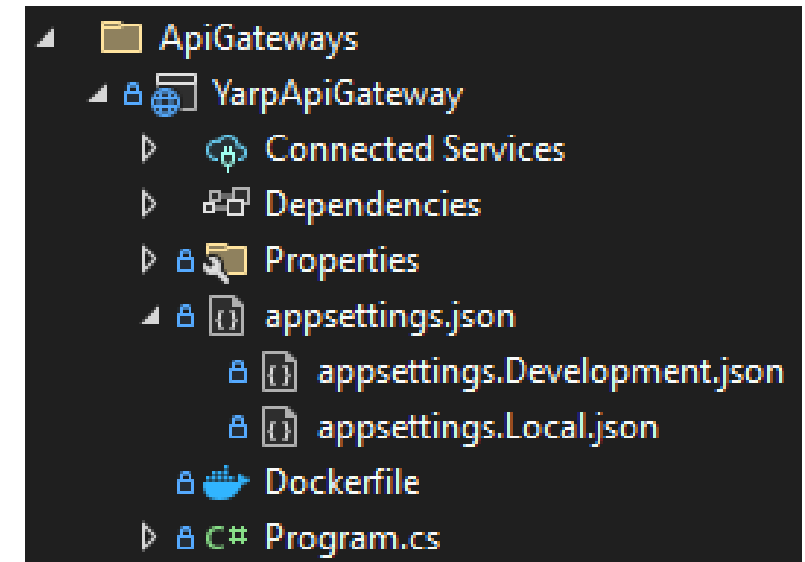
- Entry point of the application where Yarp and other services are configured.

appsettings.json:

- Contains all the routing configurations for the microservices.

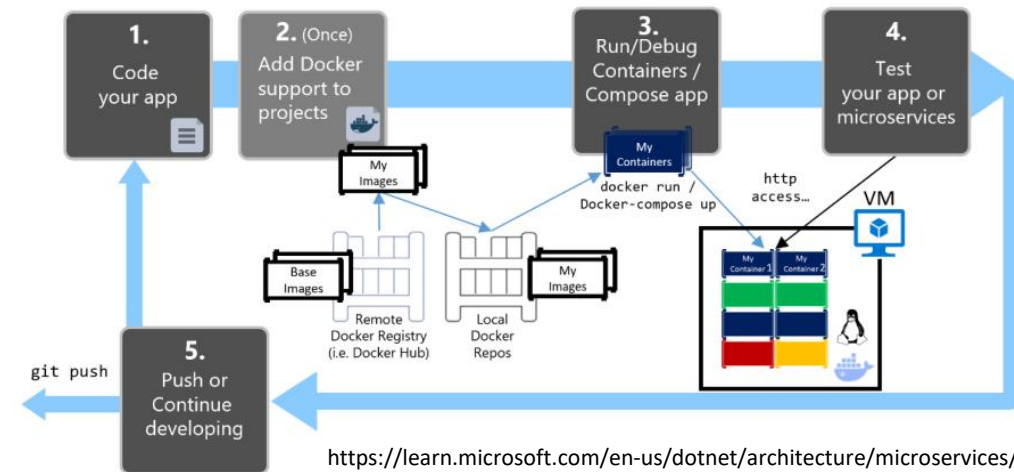
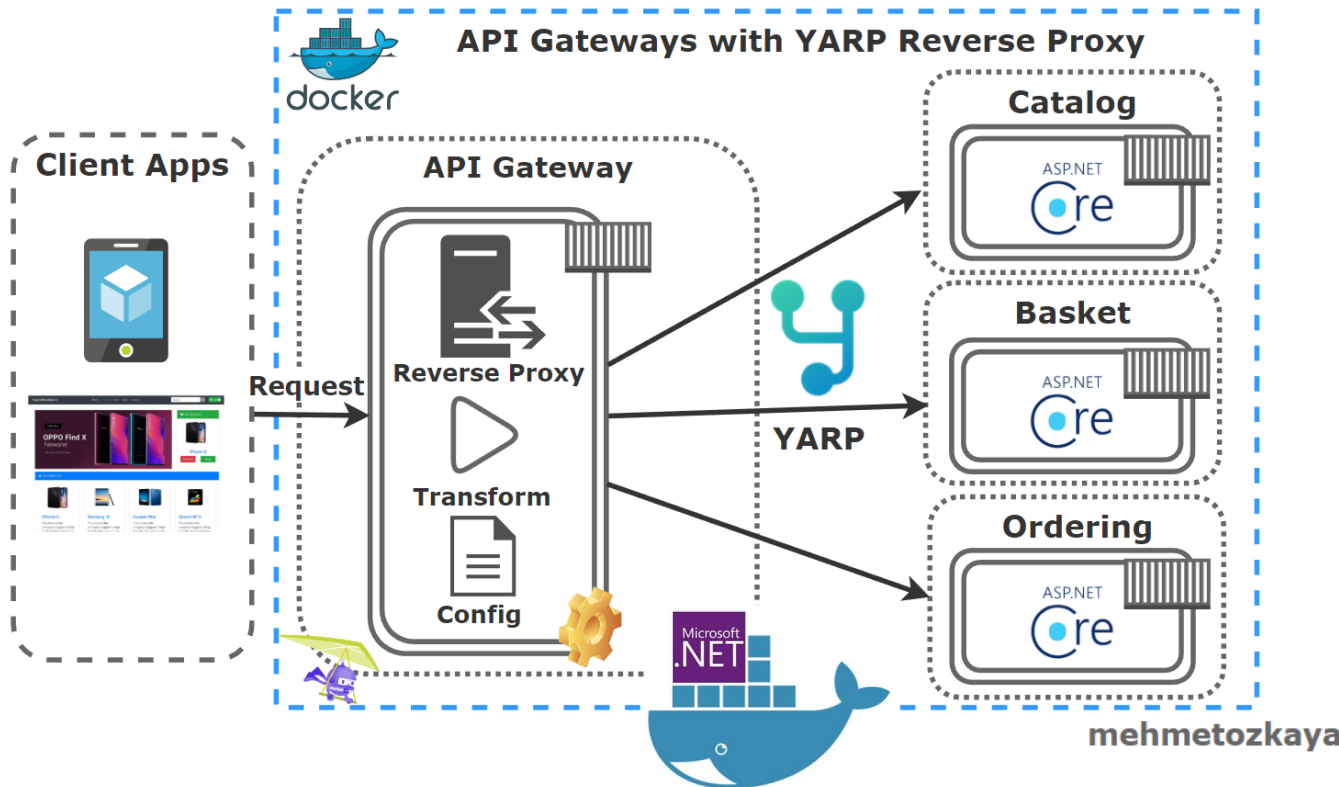


```
"ReverseProxy": {
  "Routes": {
    "catalog-route": {
      "ClusterId": "catalog-cluster",
      "Match": {
        "Path": "/catalog-service/{**catch-all}"
      },
      "Transforms": [ { "PathPattern": "{**catch-all}" } ]
    },
    "basket-route": {
      "ClusterId": "basket-cluster",
      "Match": {
        "Path": "/basket-service/{**catch-all}"
      },
      "Transforms": [ { "PathPattern": "{**catch-all}" } ]
    }
  }
}
```



Deployment of YarpApiGateway

- **Containerize** our service with **Docker**, ensuring deployment and integration with **Yarp API Gateway**.
- Implement **Dockerfile** and **docker-compose** file for running **YarpApiGateway** in Docker environment.



Port Numbers for YarpAPIGateway Microservice



Microservices	Local Env	Docker Env	Docker Inside
Catalog	5000 – 5050	6000 – 6060	8080 – 8081
Basket	5001 – 5051	6001 – 6061	8080 – 8081
Discount	5002 – 5052	6002 – 6062	8080 – 8081
Ordering	5003 – 5053	6003 – 6063	8080 – 8081
Yarp.ApiGw	5004 – 5054	6004 – 6064	8080 – 8081

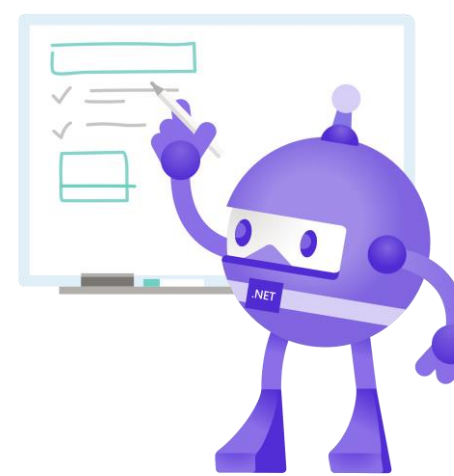
- ASP.NET Core **Ports** are **listed** as **http / https** for running application.

3. Rest API Endpoints of YARP Microservices

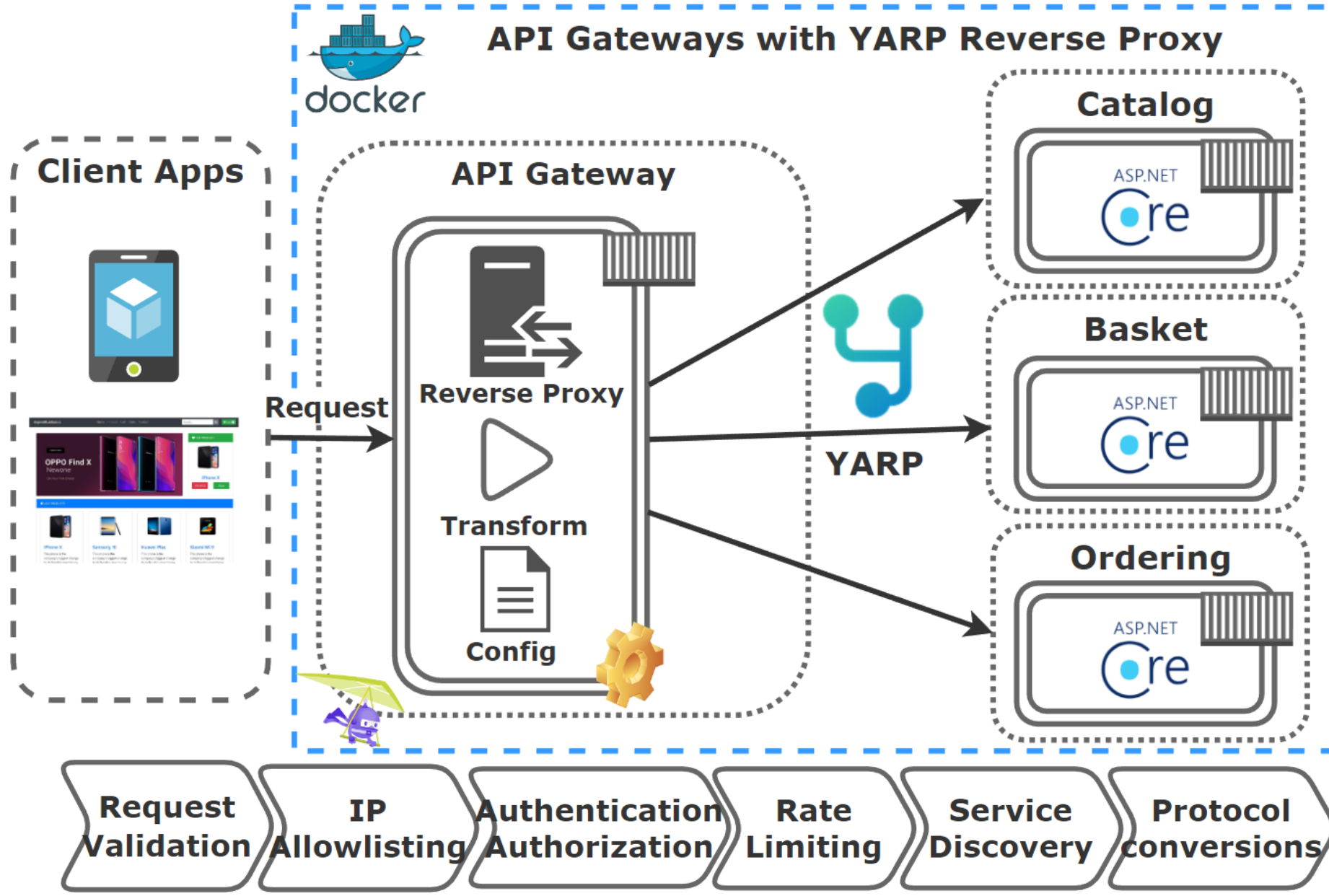
- Put microservice name as a prefix of internal uri's
- {microservice_name}/{microservice_internal_url}

Method	YARP Request URI	Internal Ms Request URI
GET	/catalog/products	/products
GET	/catalog/products/{id}	/products/{id}
GET	/catalog/products/category	/products/category
POST	/catalog/products	/products
PUT	/catalog/products/{id}	/products/{id}
DELETE	/catalog/products/{id}	/products/{id}

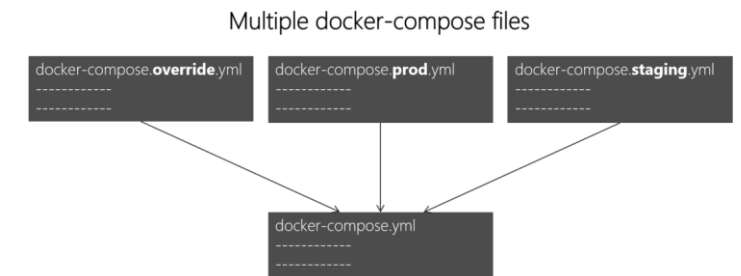
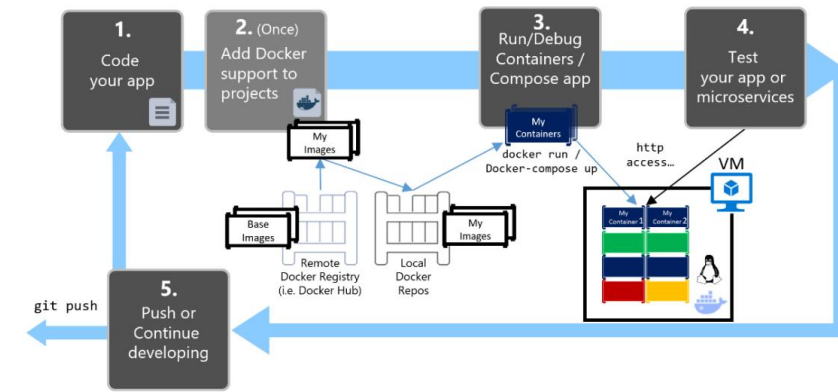
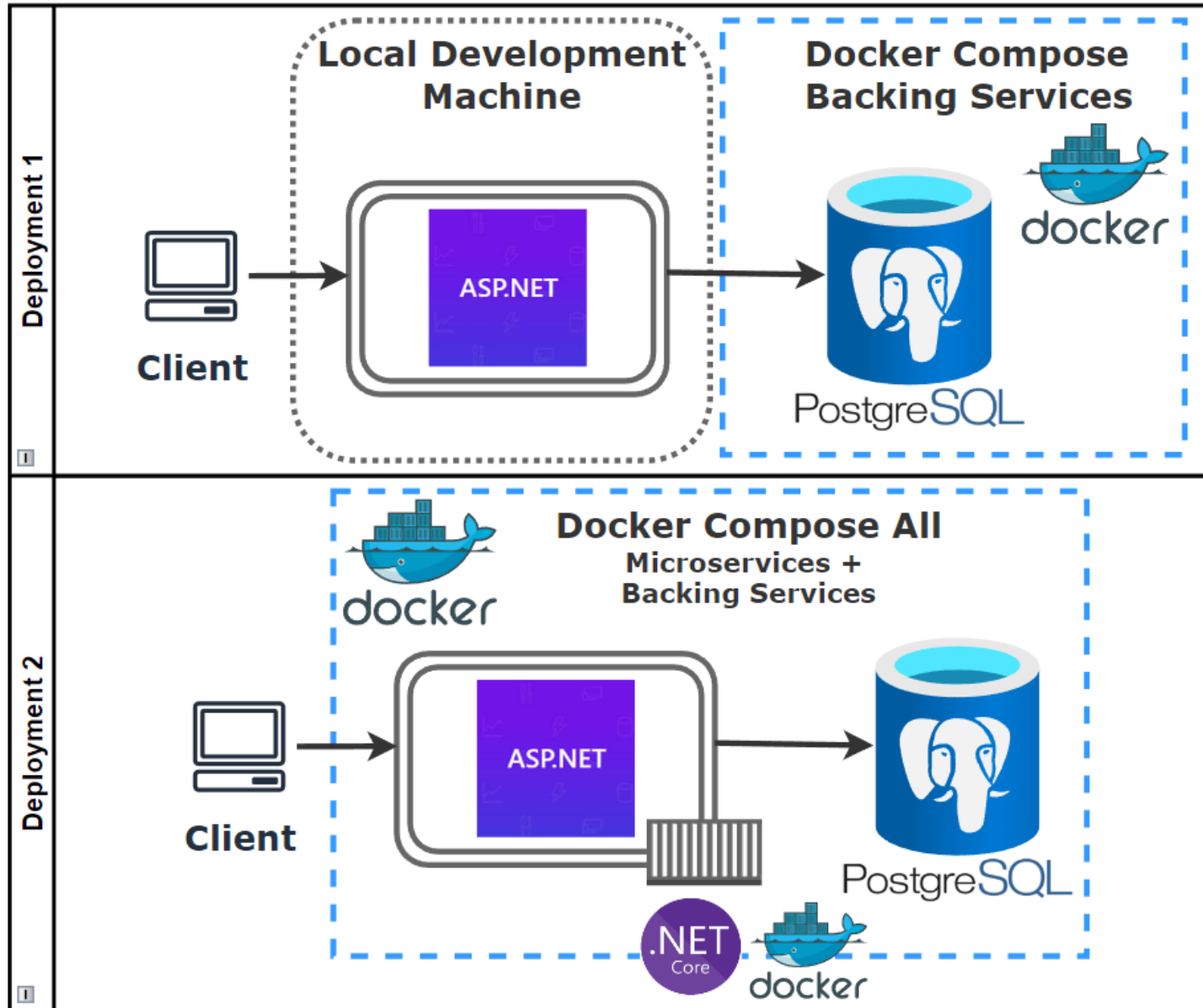
Method	YARP Request URI	Internal Ms Request URI
GET	/shoppingcart/basket/{userName}	/basket/{userName}
POST	/shoppingcart/basket/{userName}	/basket/{userName}
DELETE	/shoppingcart/basket/{userName}	/basket/{userName}
POST	/shoppingcart/basket/checkout	/basket/checkout



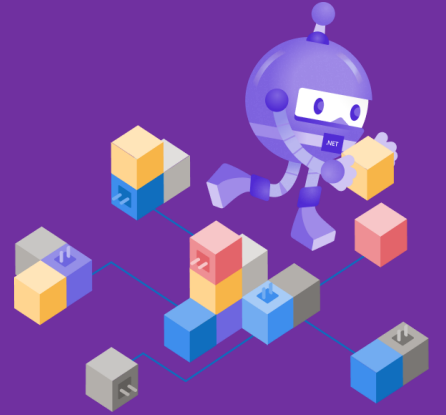
API Gateways with YARP Reverse Proxy



EShop Microservices Deployment Strategy



Shopping Web Client Application with Refit HttpClientFactory Library



Asp.Net Web Application Razor Pages with Bootstrap 4 Theme

ASP.NET Core Razor Tools — View Components, Partial Views, Tag Helpers, Model Bindings and Validations, Razor Sections

Consume YarpApiGateway Endpoints using Refit Library with Generated HttpClientFactory

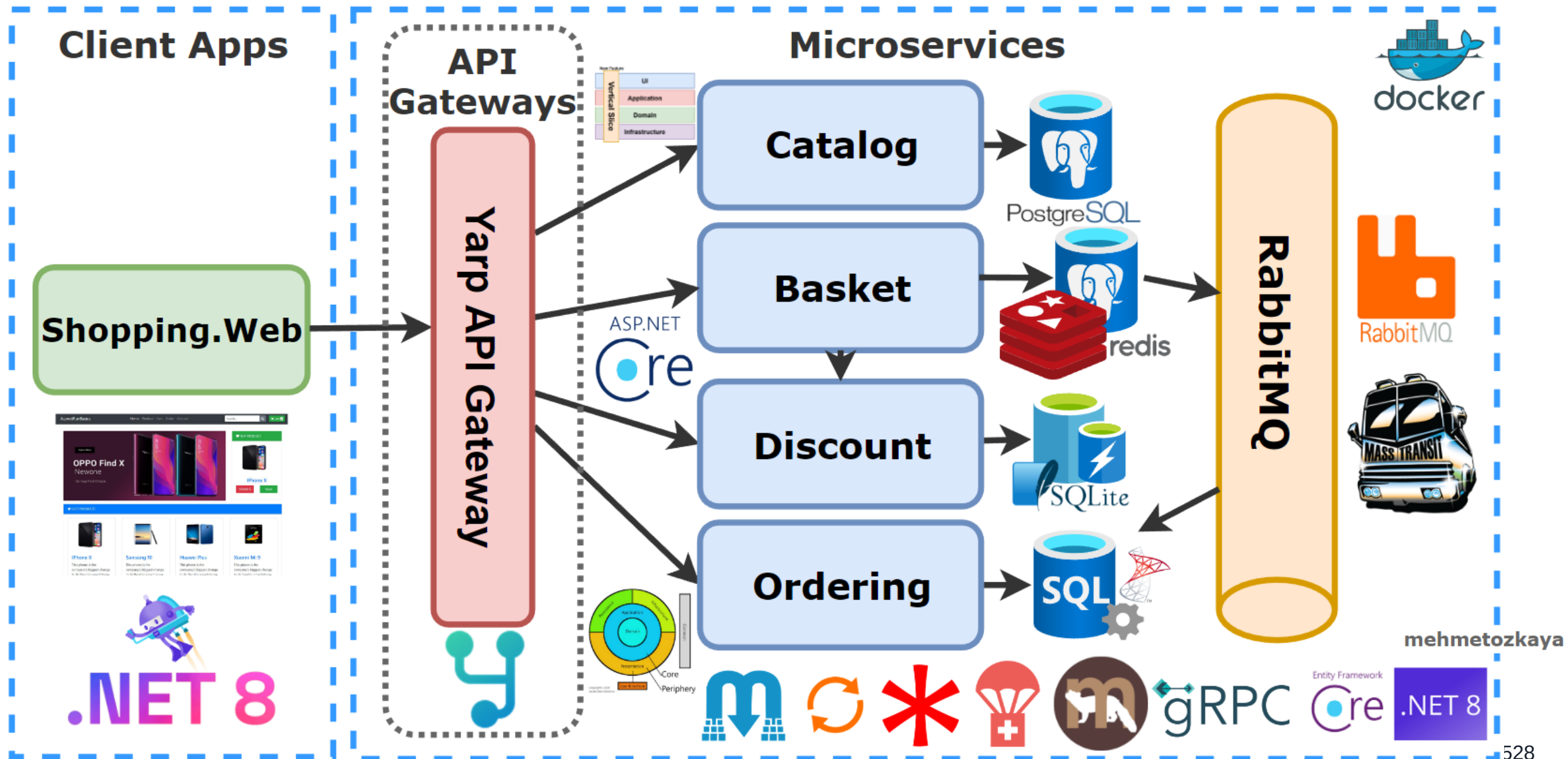
Refit HttpClientFactory Library: The automatic type-safe REST library

Develop Index/Cart/Product/Detail/Checkout/Order HTML and C#

Mehmet Ozkaya



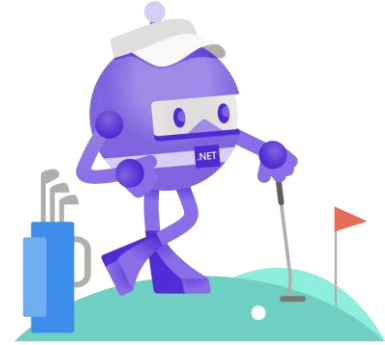
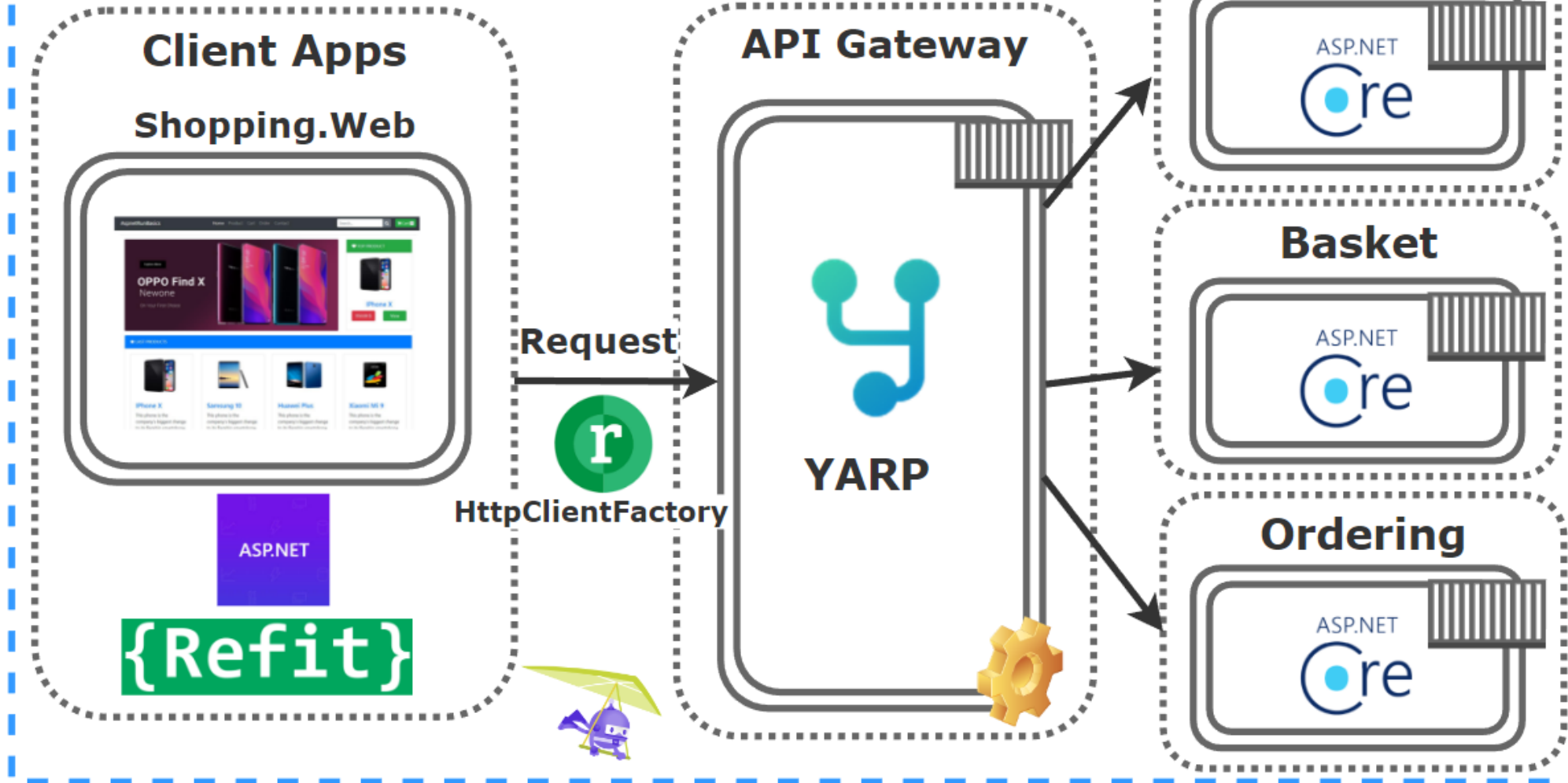
Big Picture – Course Project



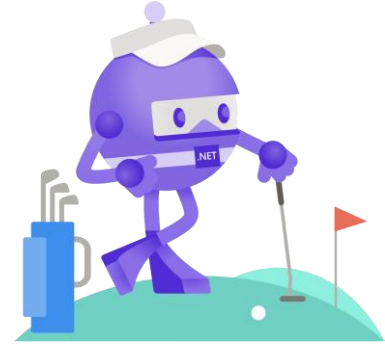
Shopping Web Client Application with Refit



Shopping.Web Client Application w/ Refit HttpClientFactory



Steps of Developing Shopping.Web



1. WebUI ShoppingApp Microservice

- Create an ASP.NET Web Application using Razor Pages.
- The application will be styled using Bootstrap 4
- ASP.NET Core Razor tools: View Components, Partial Views, Tag Helpers, Model Bindings

2. Data Models

- Define models for Catalog, Basket, and Ordering Model
- Models will be used to interact with the YarpApiGateway Endpoints.

3. Refit HttpClientFactory Library

- Use Refit, an automatic type-safe REST library
- Develop ICatalogService, IBasketService, and IOrderingService interfaces with Refit

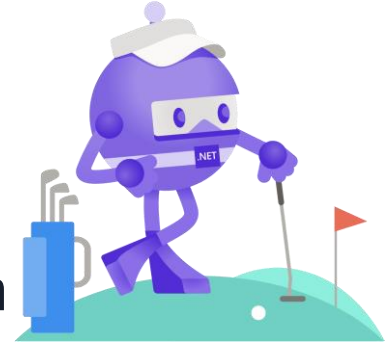
4. Razor Pages Development

- Index, Cart, Product, Detail, Checkout, Order HTML and C# Pages
- Consume the YarpApiGateway endpoints using services like ICatalogService

5. Containerization and Orchestration

- Containerize and orchestrate Shopping.Web within a Docker-Compose environment

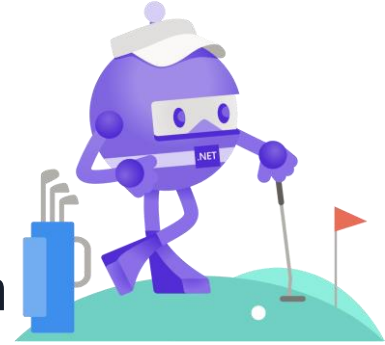
Shopping Web Client Application with Refit



- Asp.Net Web Application Razor Pages with Bootstrap 4 Theme Implementation using AspNetRunBasics repository
- ASP.NET Core Razor Tools — View Components, Partial Views, Tag Helpers, Model Bindings and Validations, Razor Sections
- Consume YarpApiGateway Endpoints using Refit Library with Generated HttpClientFactory
- Refit HttpClientFactory Library: The automatic type-safe REST library, Create Generated clients with Refit
- Develop Index/Cart/Product/Detail/Checkout/Order HTML and C# Pages
Consume YarpApiGateway Endpoints with IService with Refit HttpClientFactory Library
- Containerize and Orchestrate Shopping.Web Application in Docker-Compose

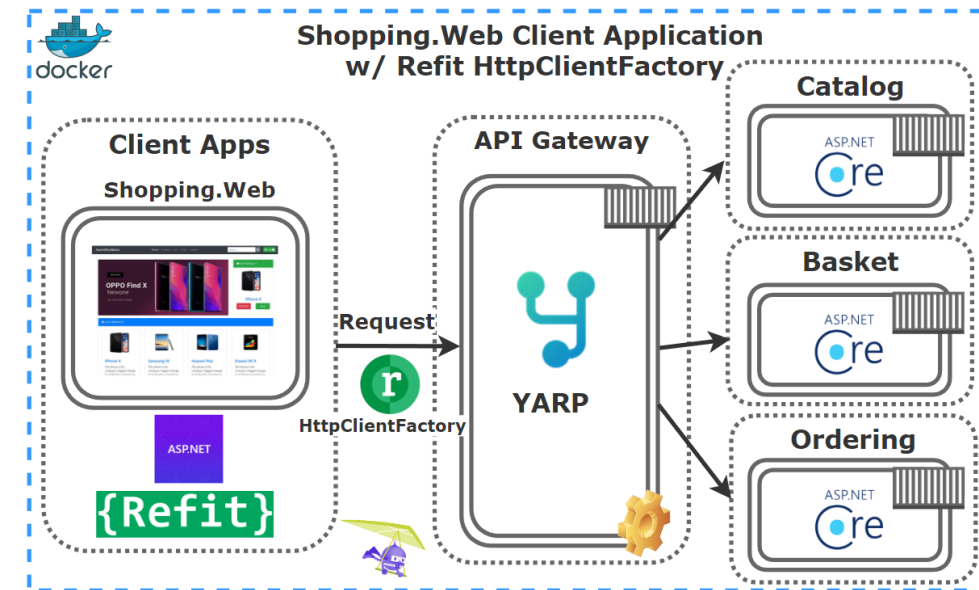
Background of Project

- Asp.Net Web Application Razor Pages with Bootstrap 4 Theme Implementation using AspNetRunBasics repository
- ASP.NET Core Razor Tools — View Components, Partial Views, Tag Helpers, Model Bindings and Validations, Razor Sections
- Consume YarpApiGateway Endpoints using Refit Library with Generated HttpClientFactory



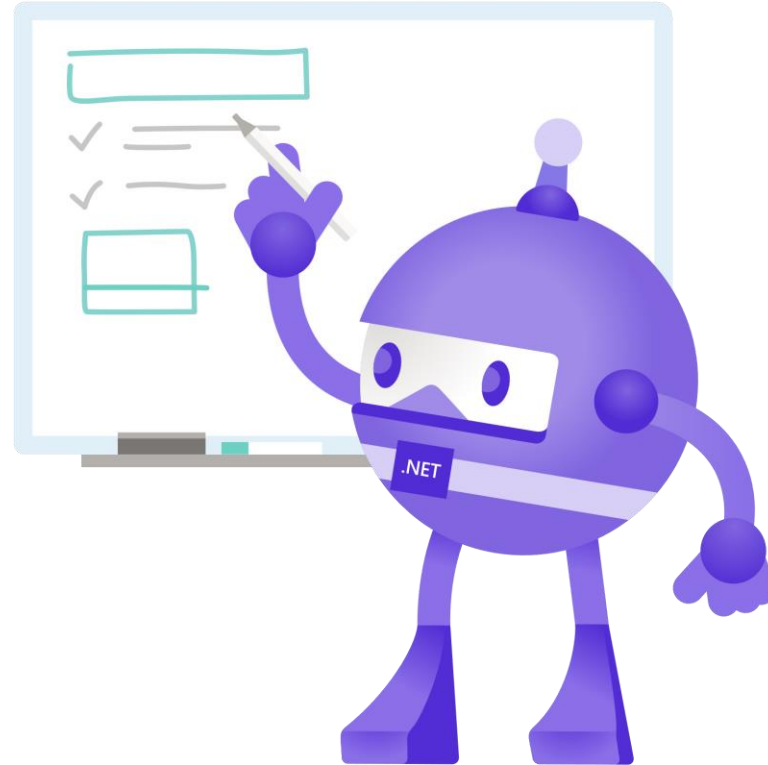
Base Repository of Shopping Web Application

- <https://github.com/aspnetrun/run-aspnetcore-basics>



Domain Analysis of Shopping.Web Microservices

1. Domain Models
2. Application Use Cases
3. Rest API Endpoints/Presentation Layer
4. Underlying Data Structures



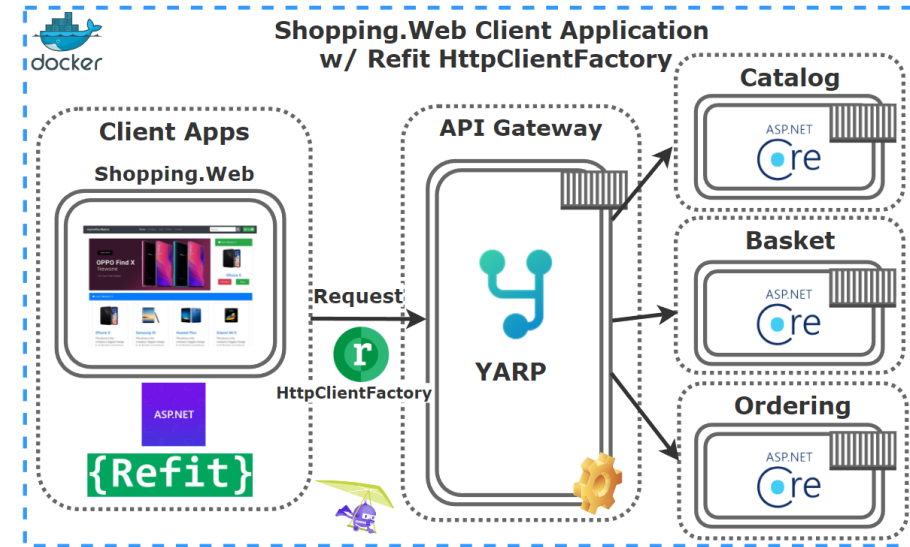
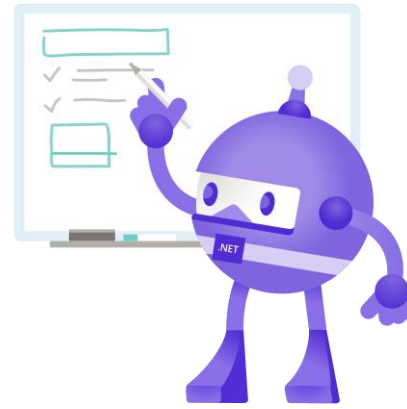
1. Domain Models of Shopping.Web Microservices

We have only **3 main domain model** per microservices:

- Catalog
- Basket
- Ordering

Create sub domain models per microservices:

- Catalog
 - Product - GetProductsResponse, GetProductByIdResponse..
- Basket
 - ShoppingCart, ShoppingCartItem, BasketCheckout - GetBasketResponse, StoreBasketResponse..
- Ordering
 - OrderDto, OrderItemDto, AddressDto, PaymentDto --GetOrdersResponse..



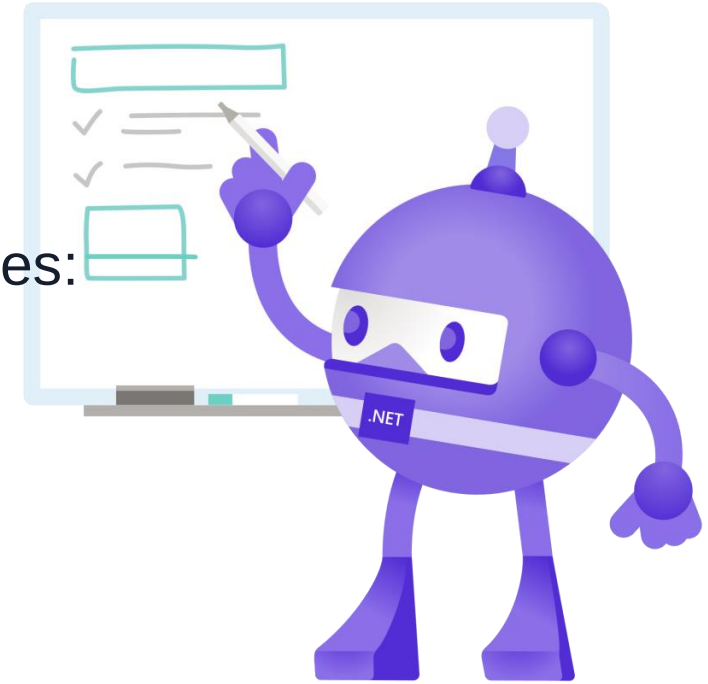
2. Application Use Cases of Shopping.Web Microservices

Our **main use cases**:

- Listing Products and Categories
- Add Product to Shopping Cart
- Checkout Basket and Create Order

We will use **YarpApiGateway endpoints** to perform these use cases:

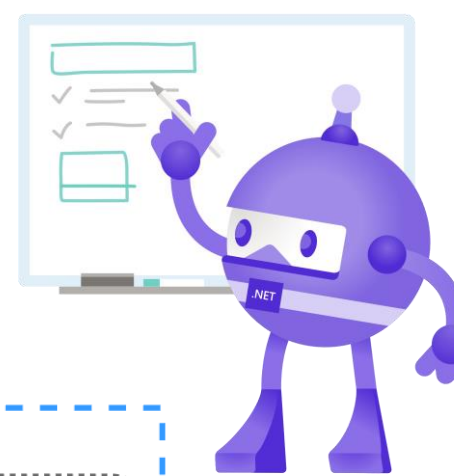
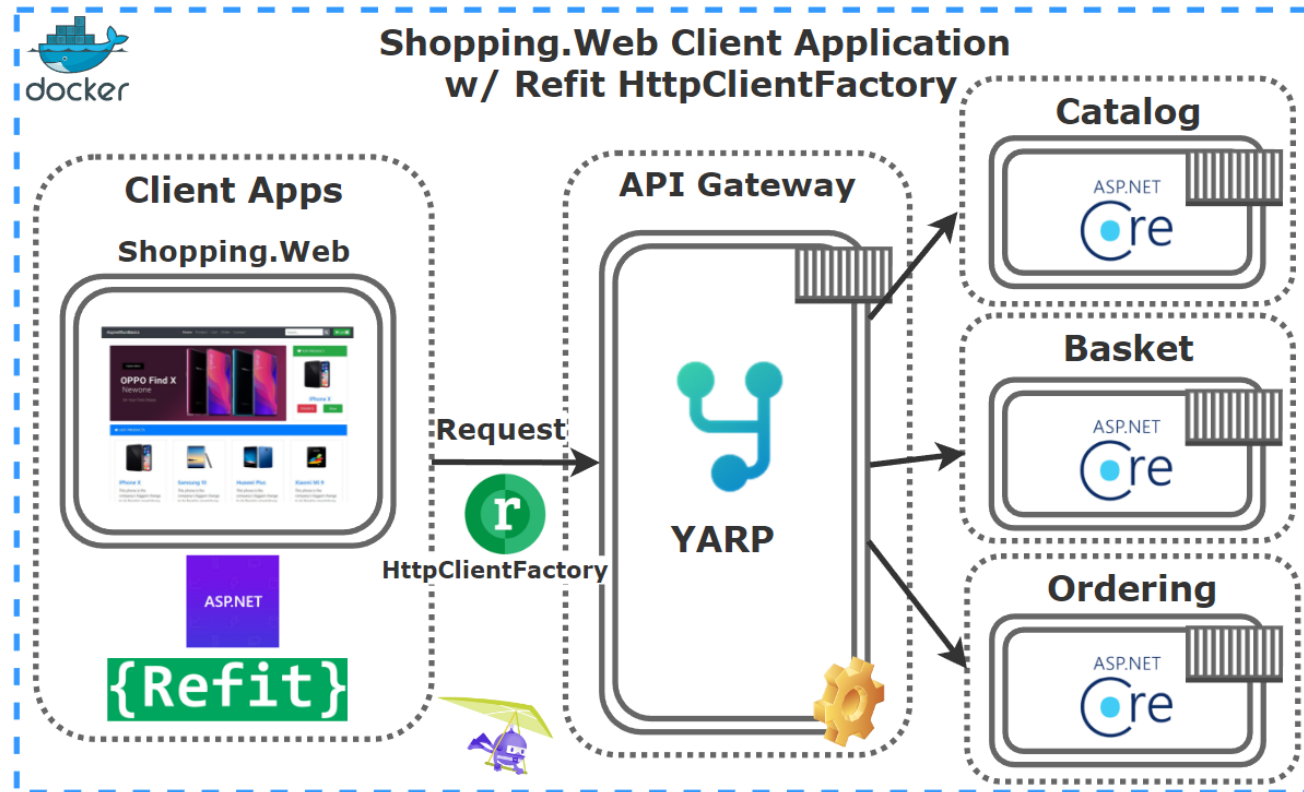
- GET catalog-service/products
- POST basket-service/basket
- POST basket-service/basket/checkout
- GET ordering-service/orders



3. Presentation - Shopping.Web Client Application

Asp.net Web Application Razor Pages:

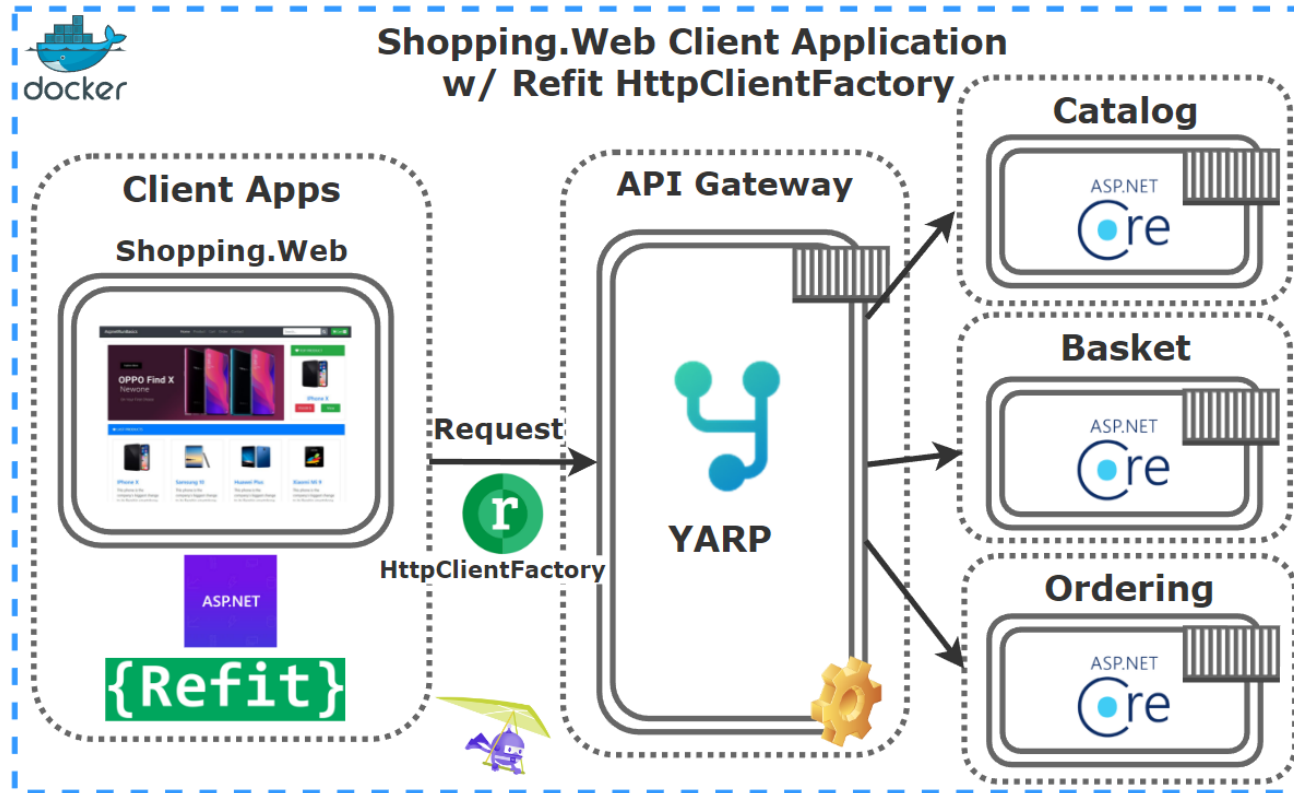
- Index.cshtml
- ProductList.cshtml
- ProductDetail.cshtml
- Cart.cshtml
- Checkout.cshtml
- Confirmation.cshtml
- OrderList.cshtml
- Contact.cshtml
- Error.cshtml
- Privacy.cshtml



4. Underlying Data Structures of Shopping.Web

Shopping.Web Microservices has NO Datastore

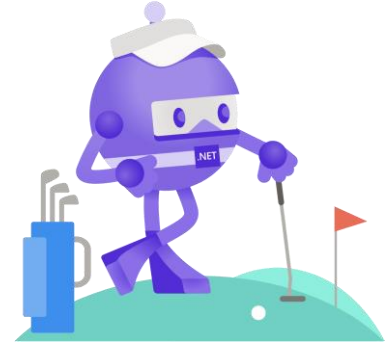
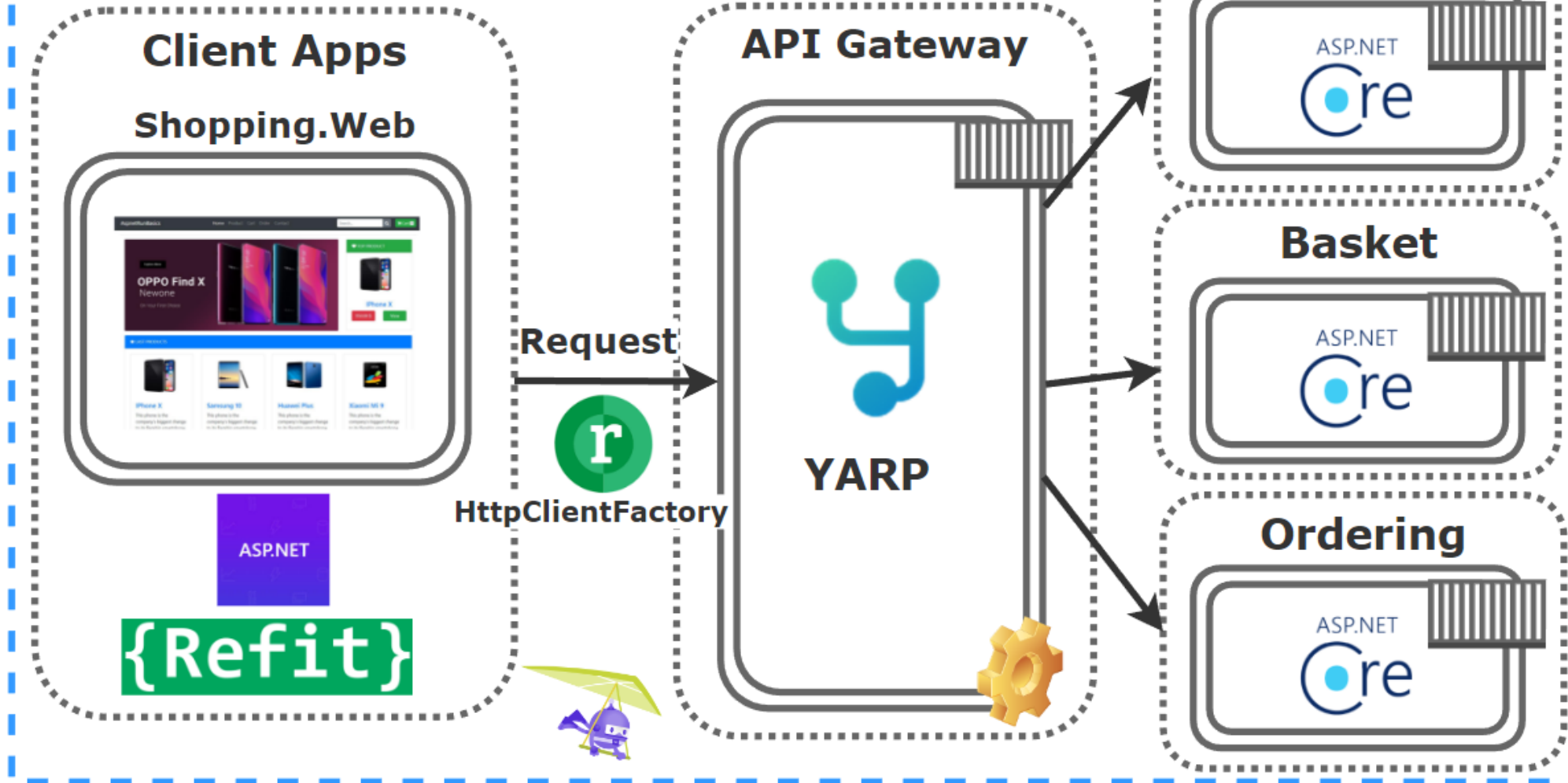
- Consume YarpApiGateway Endpoints using Refit Library with Generated HttpClientFactory



Shopping Web Client Application with Refit



Shopping.Web Client Application w/ Refit HttpClientFactory



Client Apps

Shopping.Web

API Gateways

Yarp API Gateway

Microservices

Catalog

Basket

Discount

Ordering

PostgreSQL

Redis

SQLite

SQL

RabbitMQ

Vertical Slice

UI

Application

Domain

Infrastructure

Core-Periphery

Core

Periphery

Supporting Technologies

.NET 8

ASP.NET Core

gRPC

Entity Framework

Docker

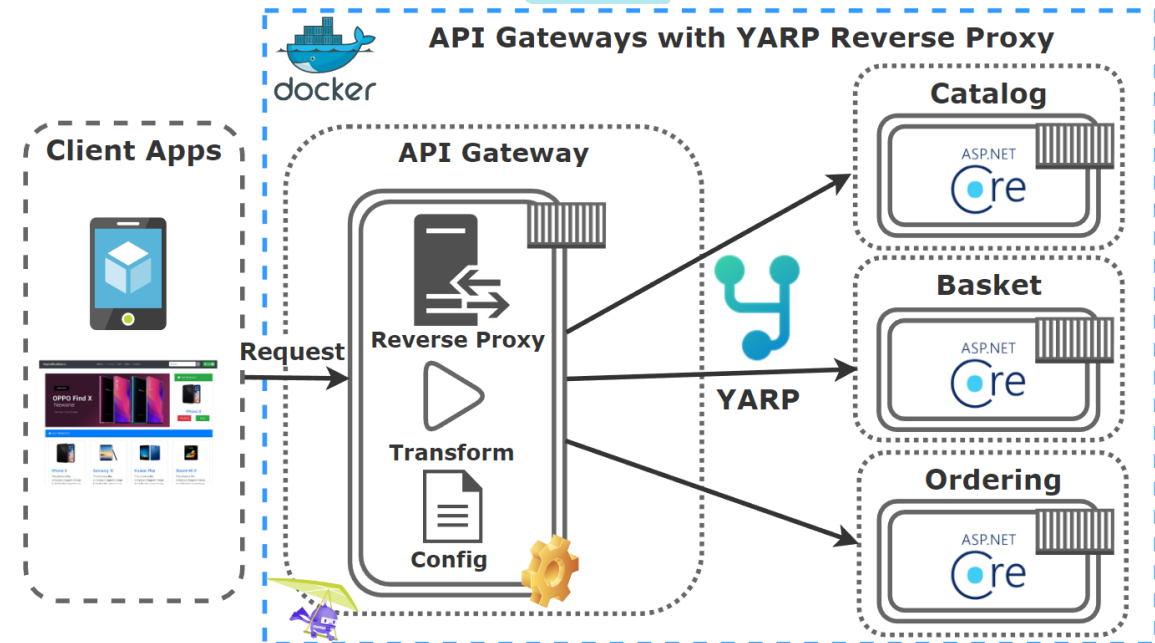
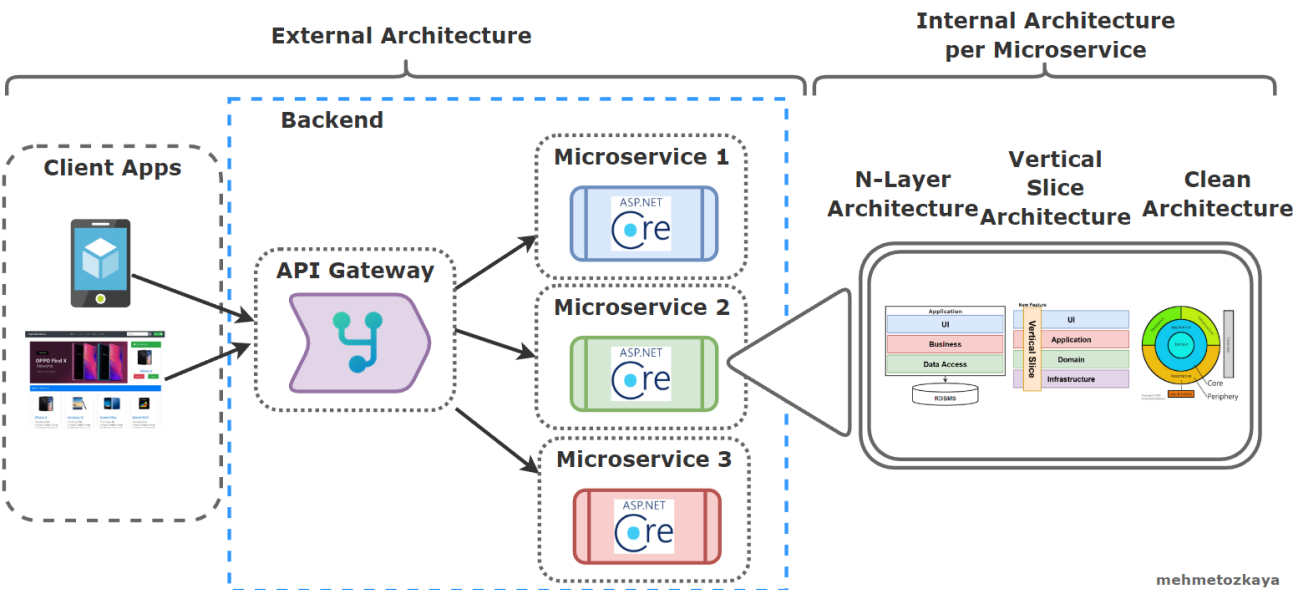
RabbitMQ

Mass Transit

mehmeto

Technical Analysis of Shopping.Web Microservices

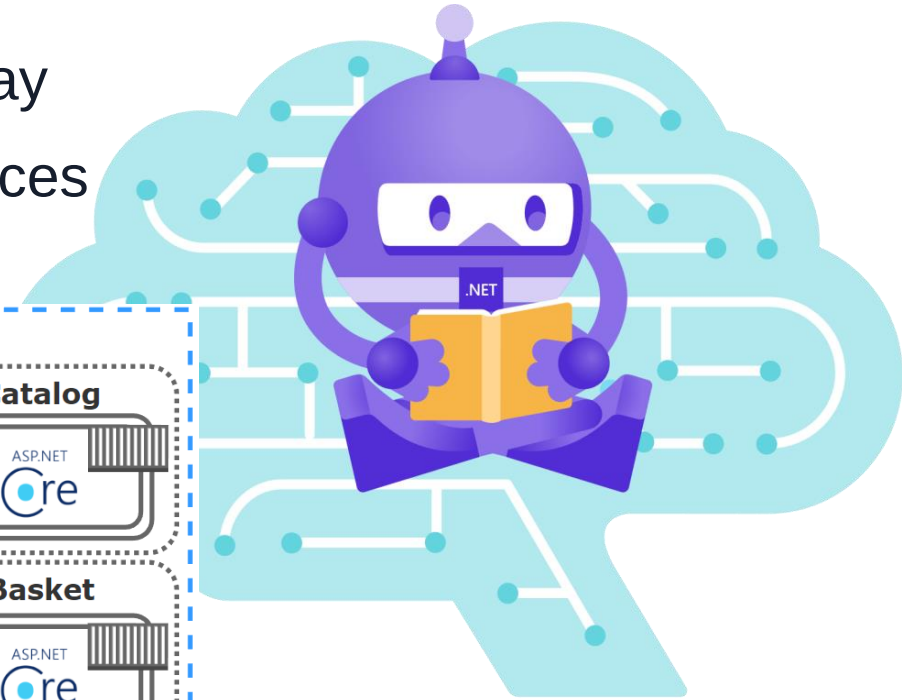
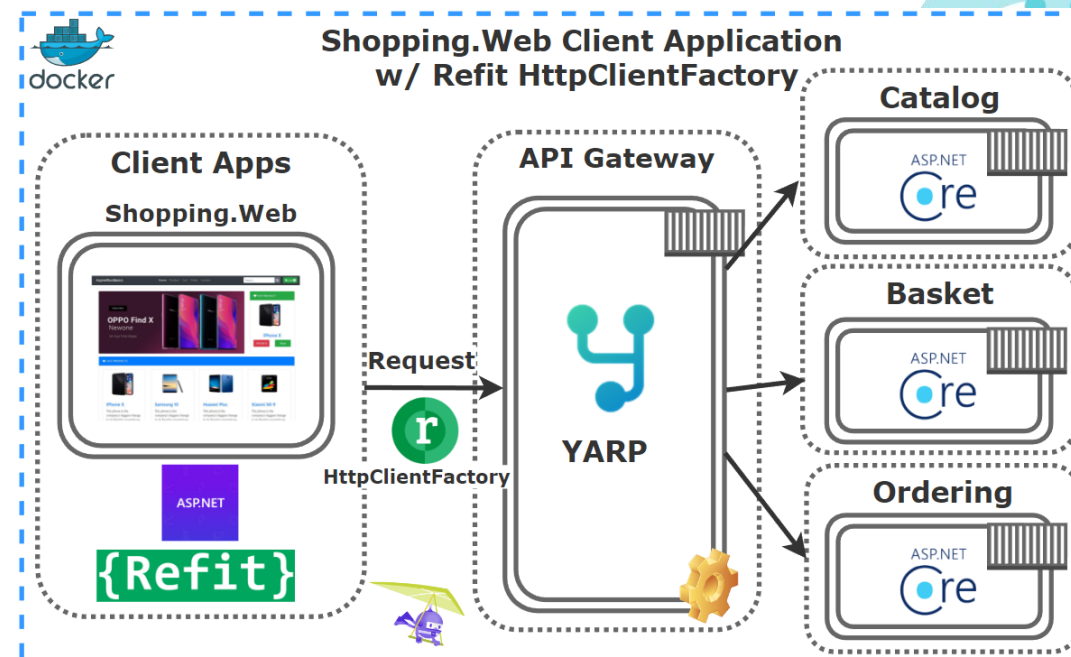
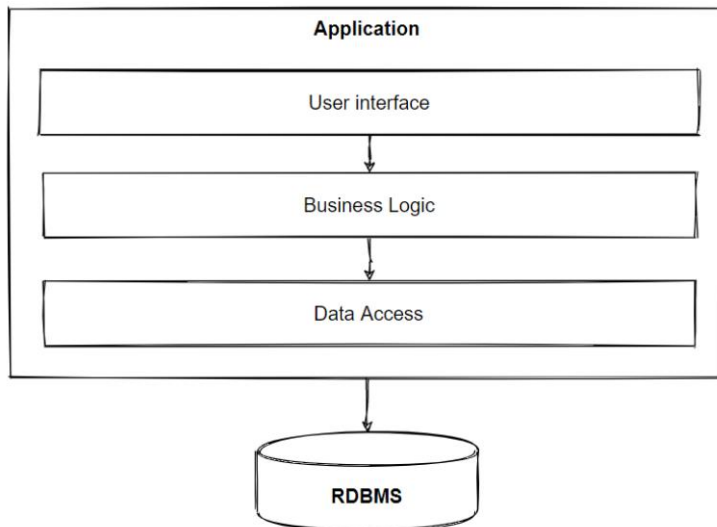
1. Application Architecture Style
2. Patterns & Principles
3. Libraries Nuget Packages
4. Project Folder Structure



1. Application Architecture Style of Shopping.Web

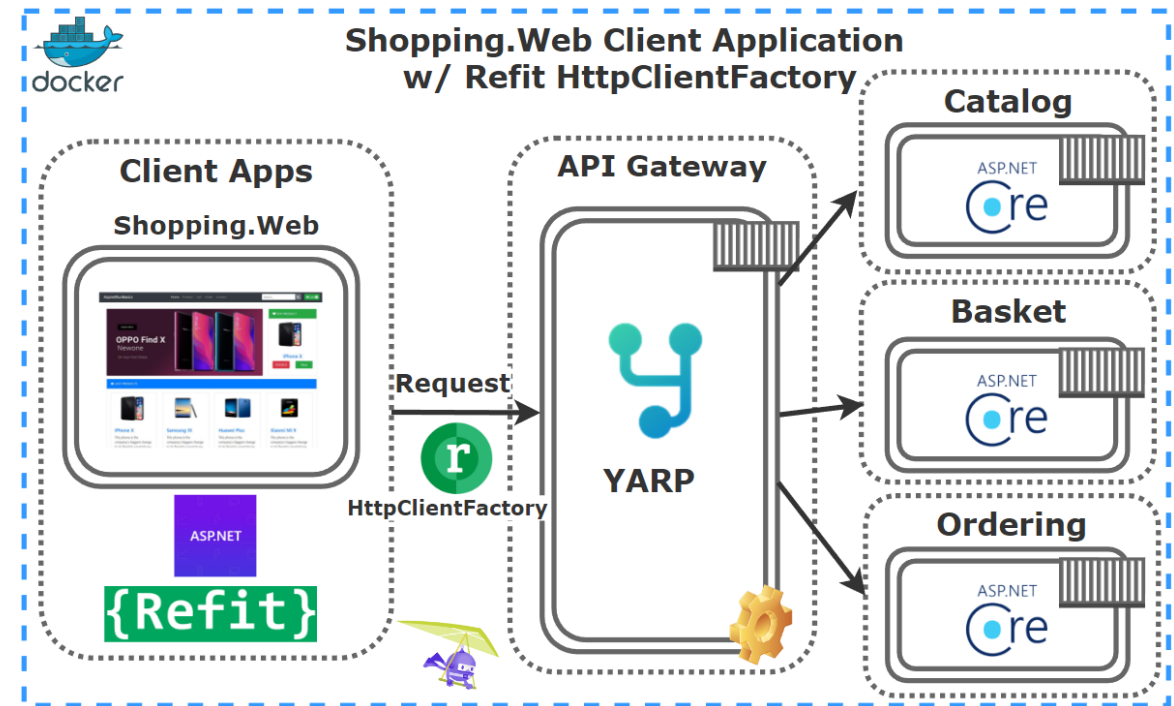
Traditional N-Layer Architecture

- **Presentation Layer:** cshtml razor web pages
- **Business Layer:** Service classes that calls YarpApiGateway
- **Data Layer:** YarpApiGateway calls downstream microservices



2. Patterns & Principles of Shopping.Web

- Type-safe REST Http Client Factory implementation
- Service Proxy Pattern
- Asp.net Razor Pages



3. Libraries Nuget Packages of Shopping.Web

Package: Refit.HttpClientFactory



{Refit}

4. Project Folder Structure of Shopping.Web

Presentation:

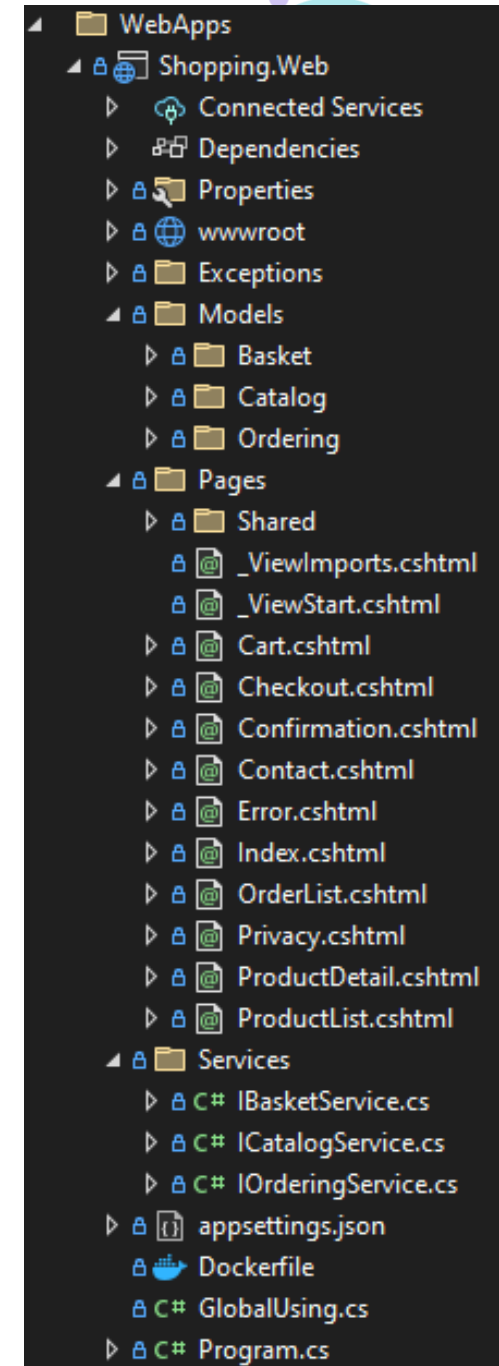
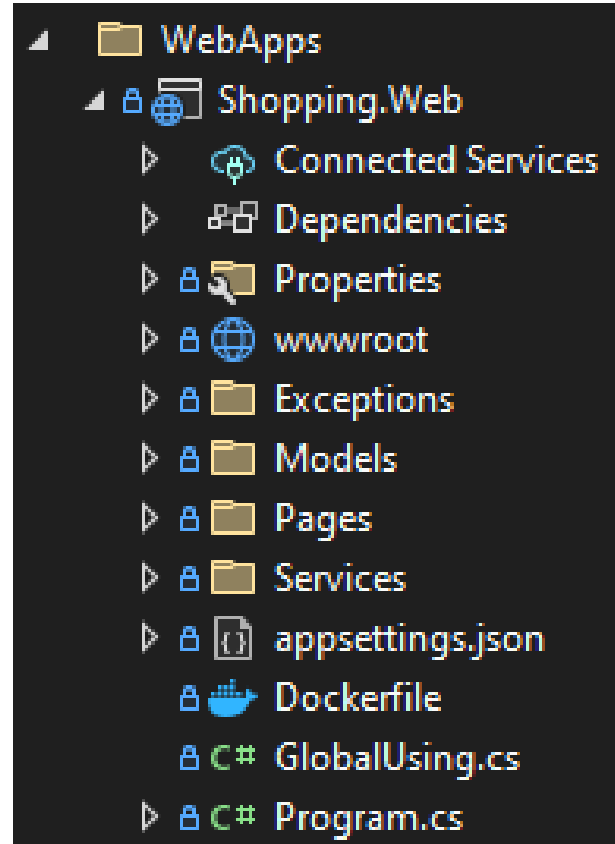
- Pages
- wwwroot

Models:

- Catalog
- Basket
- Ordering

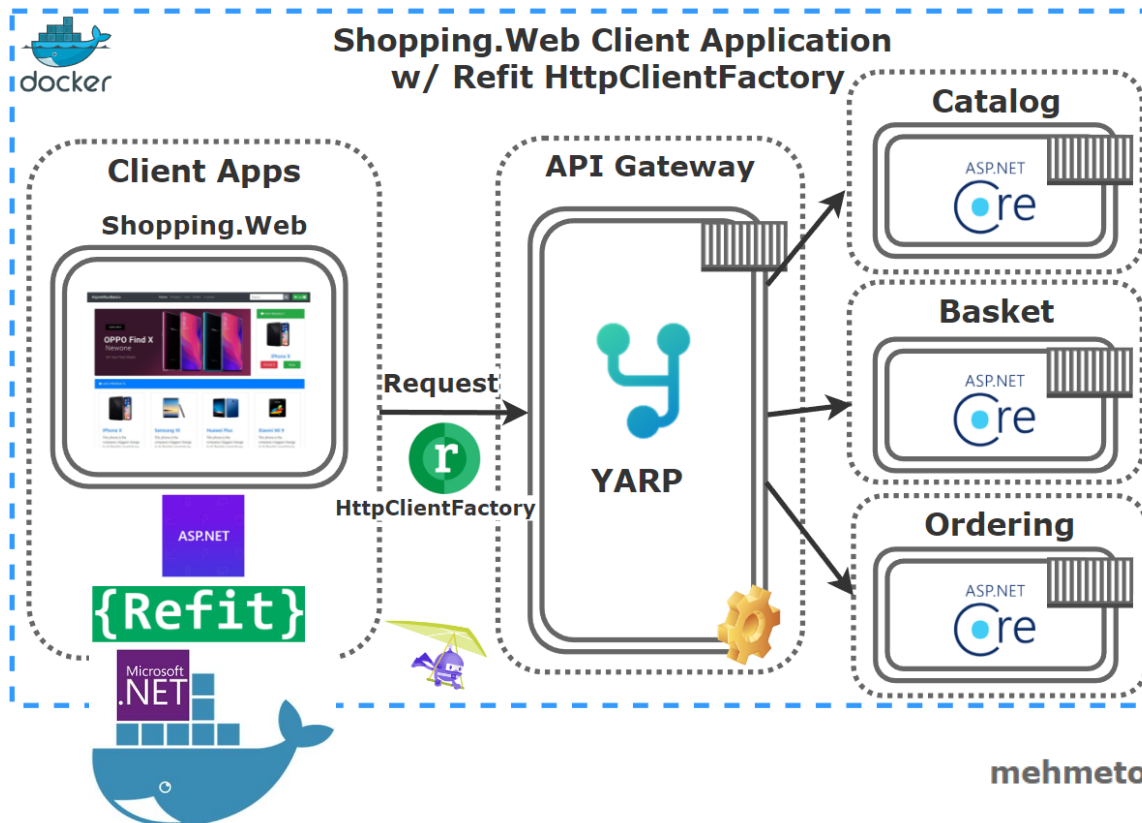
Services:

- ICatalogService
- IBasketService
- IOrderingService

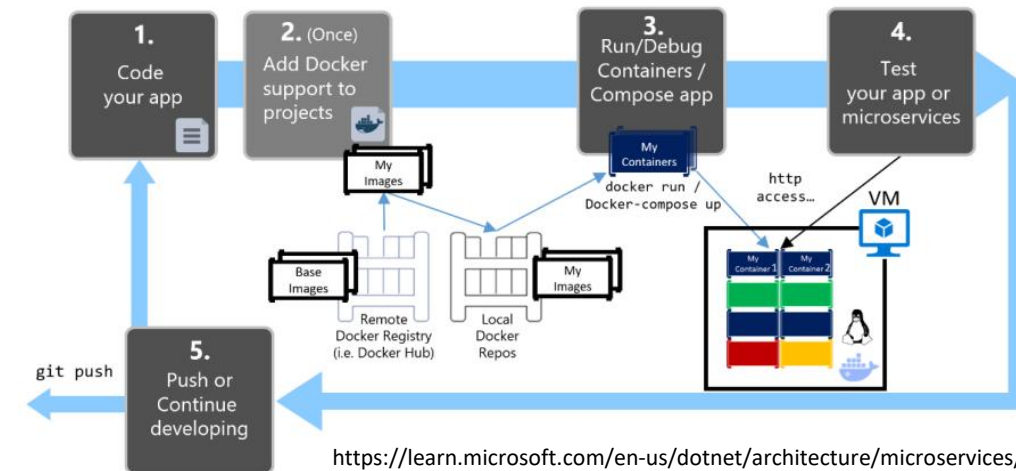


Deployment of Shopping.Web

- **Containerize** our service with **Docker**, ensuring deployment and integration with **Shopping.Web**.
- Implement **Dockerfile** and **docker-compose** file for running **Shopping.Web** in Docker environment.



mehmetozkaya

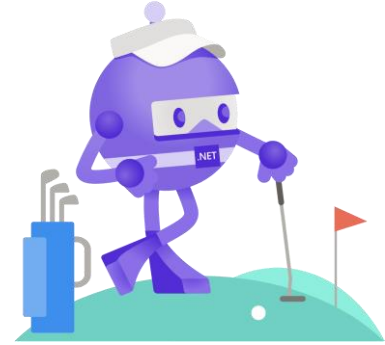
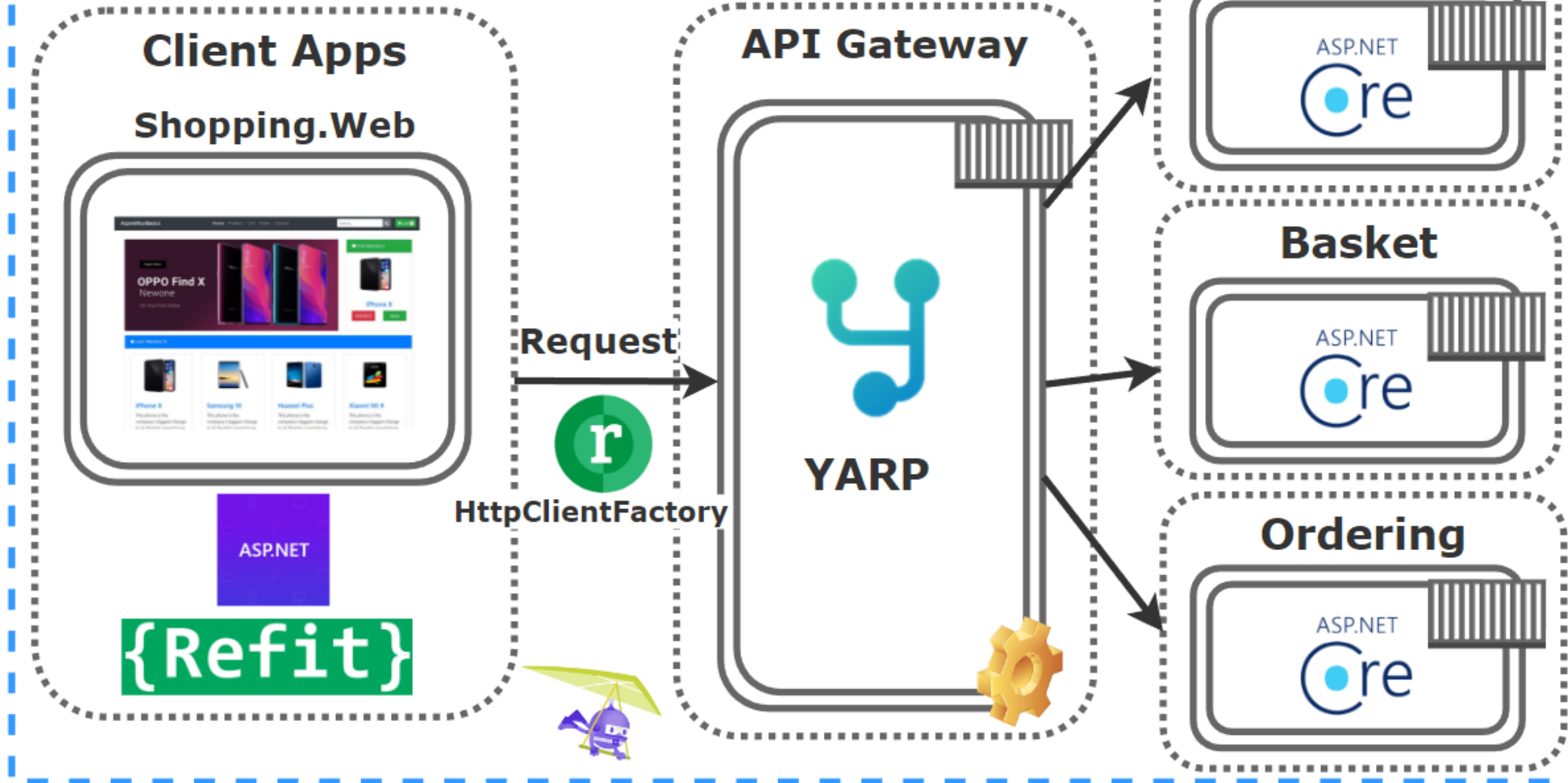


<https://learn.microsoft.com/en-us/dotnet/architecture/microservices/>

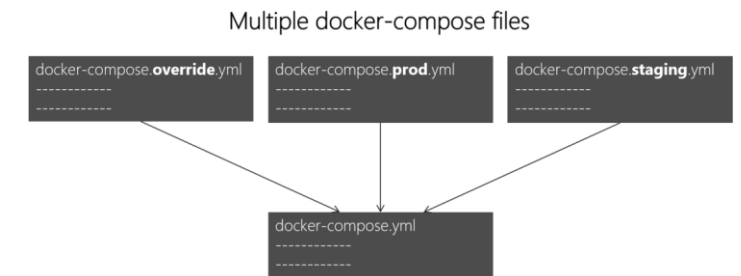
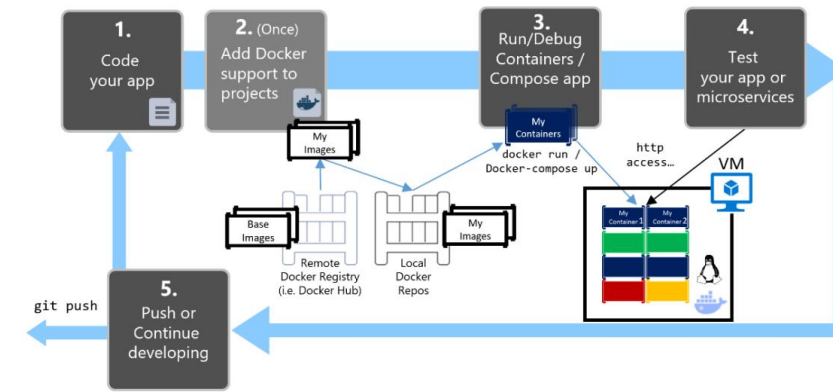
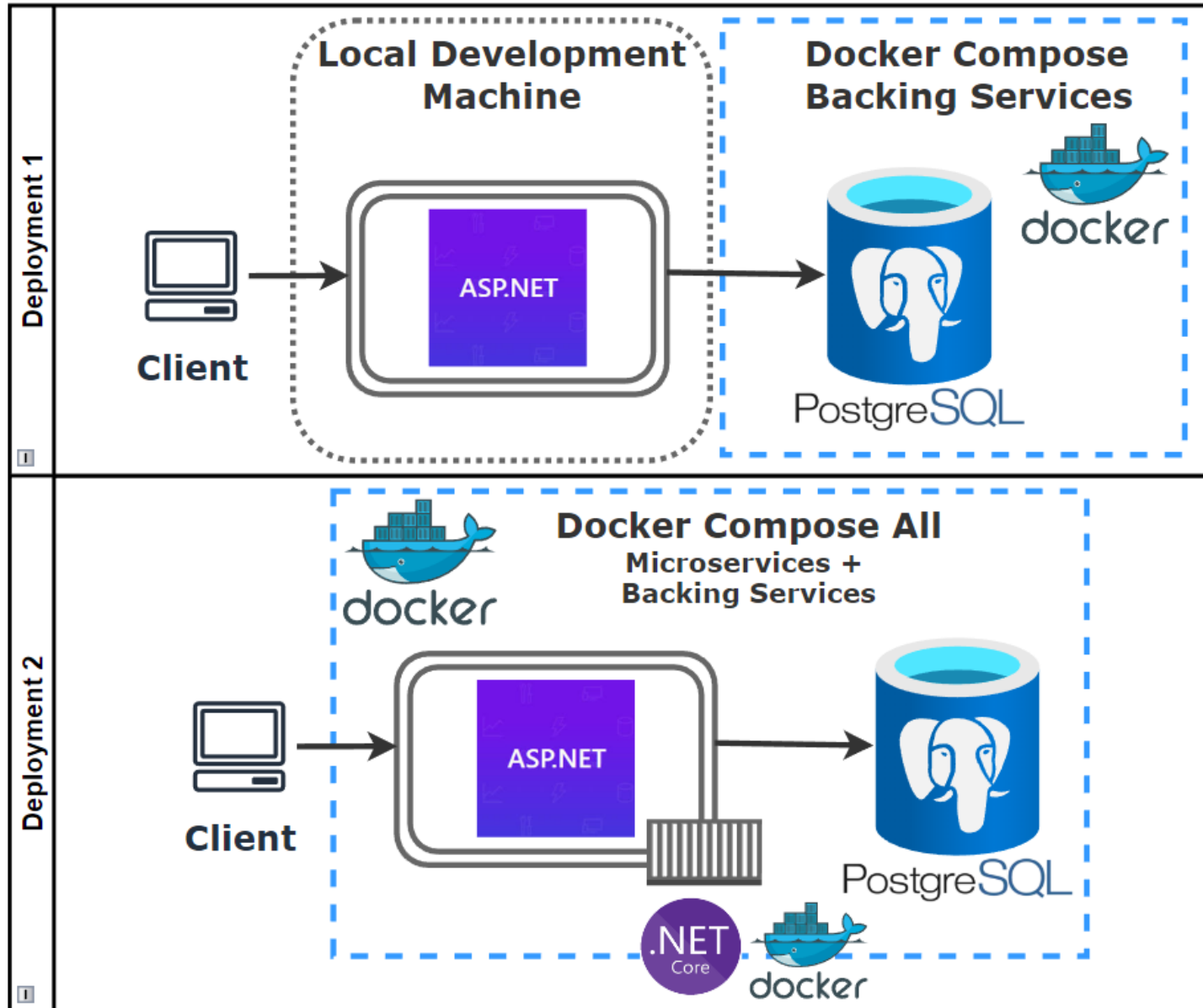
Shopping Web Client Application with Refit



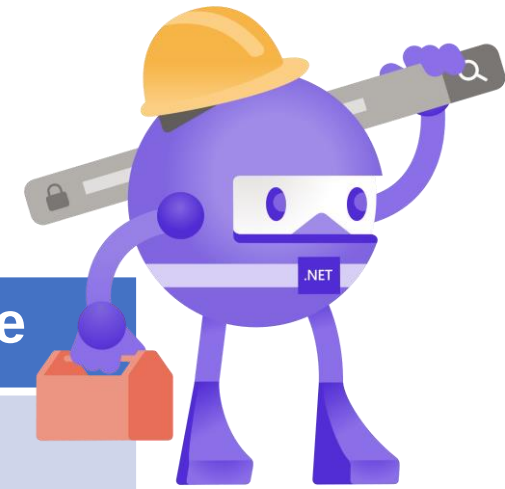
Shopping.Web Client Application w/ Refit HttpClientFactory



EShop Microservices Deployment Strategy



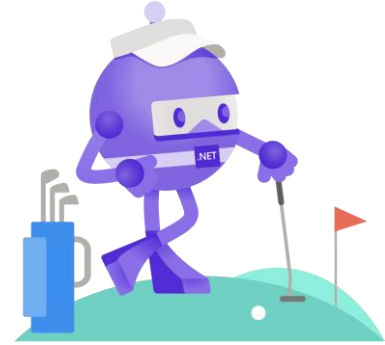
Port Numbers for Shopping.Web Microservice



Microservices	Local Env	Docker Env	Docker Inside
Catalog	5000 – 5050	6000 – 6060	8080 – 8081
Basket	5001 – 5051	6001 – 6061	8080 – 8081
Discount	5002 – 5052	6002 – 6062	8080 – 8081
Ordering	5003 – 5053	6003 – 6063	8080 – 8081
Yarp.ApiGw	5004 – 5054	6004 – 6064	8080 – 8081
Shopping.Web	5005 – 5055	6005 – 6065	8080 – 8081

- ASP.NET Core **Ports** are **listed** as **http / https** for running application.

Steps of Developing Shopping.Web



1. WebUI ShoppingApp Microservice

- Create an ASP.NET Web Application using Razor Pages.
- The application will be styled using Bootstrap 4
- ASP.NET Core Razor tools: View Components, Partial Views, Tag Helpers, Model Bindings

2. Data Models

- Define models for Catalog, Basket, and Ordering Model
- Models will be used to interact with the YarpApiGateway Endpoints.

3. Refit HttpClientFactory Library

- Use Refit, an automatic type-safe REST library
- Develop ICatalogService, IBasketService, and IOrderingService interfaces with Refit

4. Razor Pages Development

- Index, Cart, Product, Detail, Checkout, Order HTML and C# Pages
- Consume the YarpApiGateway endpoints using services like ICatalogService

5. Containerization and Orchestration

- Containerize and orchestrate Shopping.Web within a Docker-Compose environment

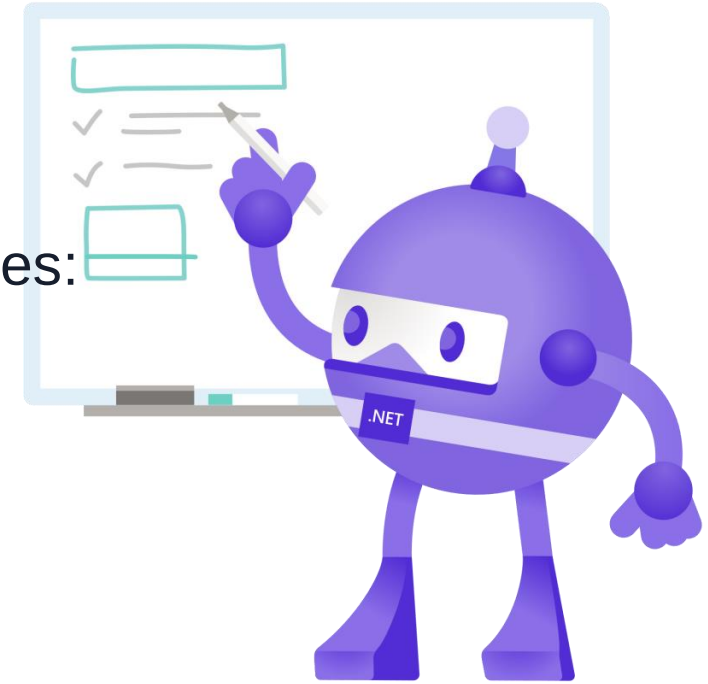
2. Application Use Cases of Shopping.Web Microservices

Our **main use cases**:

- Listing Products and Categories
- Add Product to Shopping Cart
- Checkout Basket and Create Order

We will use **YarpApiGateway endpoints** to perform these use cases:

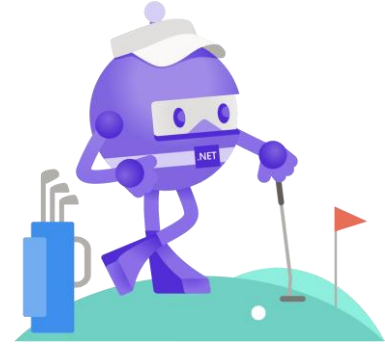
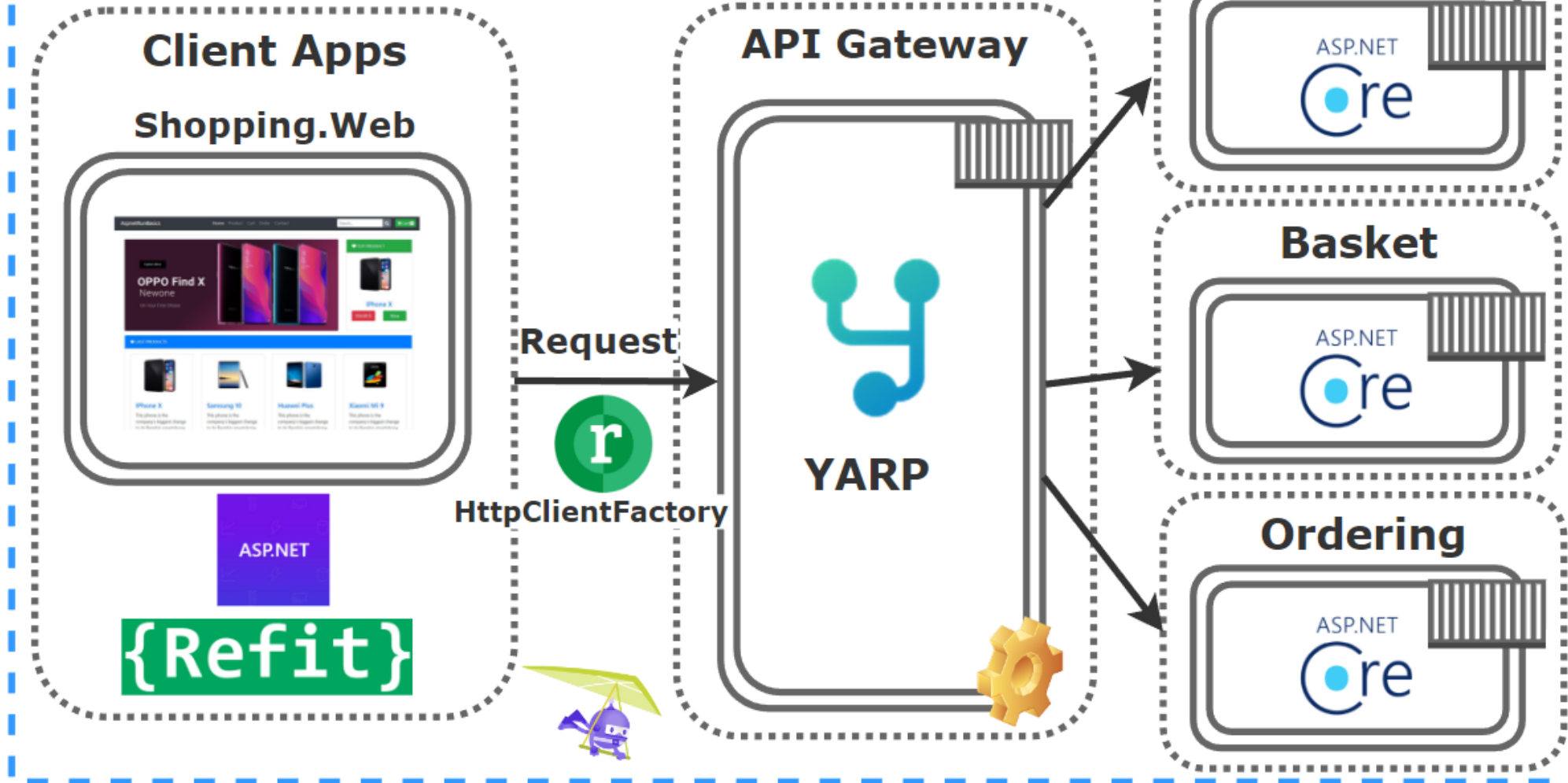
- GET catalog-service/products
- POST basket-service/basket
- POST basket-service/basket/checkout
- GET ordering-service/orders



Shopping Web Client Application with Refit

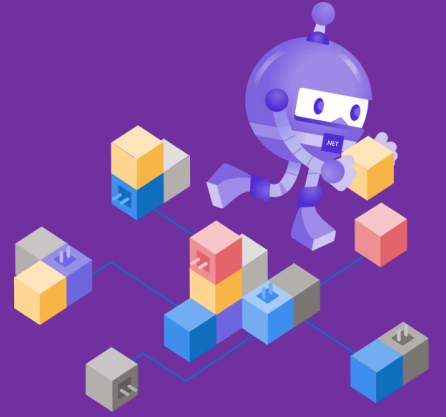


Shopping.Web Client Application w/ Refit HttpClientFactory



[illegible]

Thanks



Thank you so much for being with me on this journey.

Reviews and feedback is really encourage to me for pushing forward to create new courses like this.

Mehmet Ozkaya

